# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 09

群聊: 威海2022需求分析与系统设计

该二维码7天内(9月11日前)有效，重新进入将更新

# 课程开设的背景、目的和意义

引子

- 学会了C/C++语言，掌握了基本编程技巧
- 学会了Java语言，掌握了J2EE技术架构，熟练使用Eclipse IDE
- 学会了C#，熟练使用.NET平台
- 还学了Python、JS、JSon、XML、SSH
- 也熟悉了MySQL、MongoDB、SQL语言
- …… 

我们可以很自豪地说：

我们会编程序了，我们会编写很复杂的程序了……

# 课程开设的背景、目的和意义

事实

公司程序员经常自嘲：

码农，程序猿、悲催的程序员 …

这些软件是如何开发出来的？

QQ、微信、Facebook、王者荣耀 …

淘宝/京东商务平台、银行业务系统、企业ERP系统 …

软件开发仅仅是编程吗？

# 课程开设的背景、目的和意义

事实

## 软件开发仅仅是编程吗？

- 要编写什么样的程序/软件？（需求获取）
- 需求一般是业务过程描述和相关要求，我们能不能做？限定时间内能不能完成？（需求分析、可行性分析）
- 我们能做，但怎么做？（系统分析、设计与编码实现）
- 开发出来的软件能满足要求吗？（软件测试）
- 需求变更怎么办？版本如何管理？（软件配置管理）
- 软件版本发布、用户培训、升级与演化？（软件维护）
- 如何管理团队、控制进度、预算？（软件项目管理）

# 课程开设的背景、目的和意义

## SDLC -- 软件开发生命周期

### 分析阶段

可行性分析、需求分析、系统分析

### 设计阶段

架构设计、功能设计、数据设计、UI设计

### 实现阶段

编码实现、系统测试

### 维护阶段

安装实施、用户培训、售后服务、软件演化

### RUP软件过程

（Rational Unified Process）

初始阶段

细化阶段

构造阶段

交付阶段

核心工作流：

- 商业建模
- 需求
- 分析与设计
- 实现
- 测试
- 部署

### 敏捷方法

（Agile：XP/Scrum）

**Planning**

**Design**

**Coding**

**Test**

核心工作：

- **TDD**
- **Sprint**
- **Pair Programming**
- **Continuous Integration**
- **Daily Scrum Meeting**
- **… …**

# 课程开设的背景、目的和意义

## SDLC -- 软件开发生命周期

- 随着开发工具和环境的发展，在整个软件开发生命周期的第1个软件版本开发过程中，“编码”阶段的时间占比逐渐减少
  - 早年：70-80%
  - 目前：20-30%
- 而其他环节中，分析与设计阶段：
  - 成为软件项目开发成功的关键阶段
  - 工作量和时间占比最高的阶段，尤其是分析阶段
- 学会和掌握系统分析和设计方法/技术至关重要
  - 蓝领 → 白领蜕变进阶的关键技能

# 课程开设的背景、目的和意义

## 软件开发与管理的角色

- IT行业软件工程师
  - 软件程序员
  - 软件设计师
  - 测试工程师
  - 软件架构师
  - 系统分析师
  - CTO（首席技术官）
  - 产品经理
  - 实施顾问
  - 项目经理
- 企业或事业单位IT人员
  - 系统管理员（Administrator）
  - 数据库管理员（DBA）
  - 首席信息官（CIO）

# 课程开设的背景、目的和意义

## 软件工程专业学生的理想成功之路

程序员

编码、测试

系统分析员

需求分析、系统分析、架构设计、详细设计

技术管理

项目经理、产品经理、高级经理

三总人才

CEO、CTO、CIO / 创业者、投资人、管理者

# 课程开设的背景、目的和意义

## 课程教学目标

- 课程目标1：了解软件开发生命周期和基本过程模型，充分认识系统分析和设计在SDLC当中的地位和重要性
- 课程目标2：学会需求获取方法，掌握需求分析和建模的方法和模型，具备构建系统需求模型的能力
- 课程目标3：了解软件系统分析与设计的内容和传统方法，重点学会并掌握OO分析与设计的方法，具备系统功能设计、数据设计和UI设计能力
- 课程目标4：认识软件开发文档的重要性，学会撰写分析和设计规格说明文档，具备与需求方及团队成员沟通的能力

# 课程基本信息

- 课程编号：22WHSE31003
- 课程名称：需求分析与系统设计
- 英文名称：Requirements Analysis and System Design
- 课程类别：专业基础课
- 总 学 时：32学时，讲授：24学时，实验/实践： 8学时
- 总 学 分：2.0学分
- 先修课程：软件工程专业导论、高级语言程序设计、面向对象建模技术
- 适用专业：软件工程专业
- 开课学期：2秋
- 开课单位：计算学部/软件学院

# 课程基本内容

- 课程介绍
  1. 课程开设的目的和意义
  2. 课程内容介绍
  3. 参考资料和学习方法介绍
- 概述
  1. 软件开发生命周期SDLC
  2. 软件系统的分类
  3. 角色分工及任务
  4. 信息系统的构成及特点
  5. 信息系统开发方法简介
  6. CASE工具介绍（重点UML回顾）
- 软件需求与需求获取
  1. 软件需求的定义
  2. 需求的分类
  3. 需求获取方法
  4. 需求分析建模-事件和事物
- 需求/系统分析：结构化分析
  1. 需求的结构化分析方法
  2. 数据流图（DFD）
  3. 数据字典（DD）
  4. 数据分析（ERD、IDEF1X）
- 需求分析：用户故事与用例建模
  1. 敏捷开发中的用户故事分析
  2. 面向对象方法中的用例建模
  3. 用例建模的基本过程
  4. 业务活动分析（活动图&泳道图）
  5. 用例建模的提交物
- 系统分析：面向对象分析
  1. 面向对象的分析方法概述
  2. 建立静态结构模型
  3. 建立动态行为模型
  4. 案例分析

# 课程基本内容

- 系统设计：结构化设计
  1. 自顶向下的结构化模块层次及其接口设计
  2. 模块内部逻辑设计
  3. 具有系统自动化边界的DFD
  4. 系统模块结构图
  5. 结构化分析与设计案例

- 系统设计：面向对象设计
  1. 面向对象设计概述
  2. OO分析到系统设计的转换
  3. 包的设计
  4. 对象设计
  5. 基于UML的面向对象分析与设计案例

- 系统设计：数据库设计
  1. 数据库系统及关系数据库简介
  2. 数据库逻辑模型设计
  3. ERD模型及质量评价
  4. 物理数据库设计及建立
  5. 物理数据库提高效率的技巧
  6. 数据库的不同体系结构模型
  7. OO分析类图映射到ERD
  8. 关系型数据库规范化实例

- 系统设计：用户界面设计
  1. UI理论、UI设计原则
  2. WIMP界面设计
  3. 缺省设计、输入验证设计、系统响应设计
  4. Web界面设计
  5. 用户管理&系统权限管理设计

# 课程考核方式

| 考核环节 | 分值 | 考核/评价 |
| --- | --- | --- |
| （1）平时成绩 | 10% | 平时课堂表现、课堂练习/作业等，按完成情况评定成绩 |
| （2）实验 | 15% | 需求分析实验；系统设计实验；2人分组进行。 |
| （3）实践项目 | 25% | 根据给定项目需求，完成需求分析、系统分析、系统设计，提交系统分析与设计规格说明、原型系统；3-5人分组进行。 |
| （4）期末考试 | 50% | 采用闭卷笔试考试形式，试卷内容覆盖各章知识点，按照试卷参考答案评分。 |

课程最终成绩 = 平时考核（1）×10% + 实验（2）× 15% +
实践项目（3）×25% + 期末考试成绩（4）× 50%

# 课程学习方法建议

- 以授课PPT为主线，学习关键知识点
- 根据PPT知识点查阅相关图书或资料
- 注重案例学习，动手实践（一定重视课程大作业）
- 重点将UML建模方法应用到课程实践中
- 学有余力，推荐将大作业拓展到实现阶段
- “威海2022系统分析与设计”学习交流微信群

软件工程 = 最佳实践

# 课程学习方法建议

群聊：威海2022需求分析与系统设计

该二维码7天内(9月11日前)有效，重新进入将更新

- 实名制：修改群昵称为“学号+姓名”
- 学委负责：每个小班学委同学负责收作业等
- 学习小组：3-5人，按组完成大作业

# 教学参考书推荐

参考书 No.01：

《系统分析与设计》

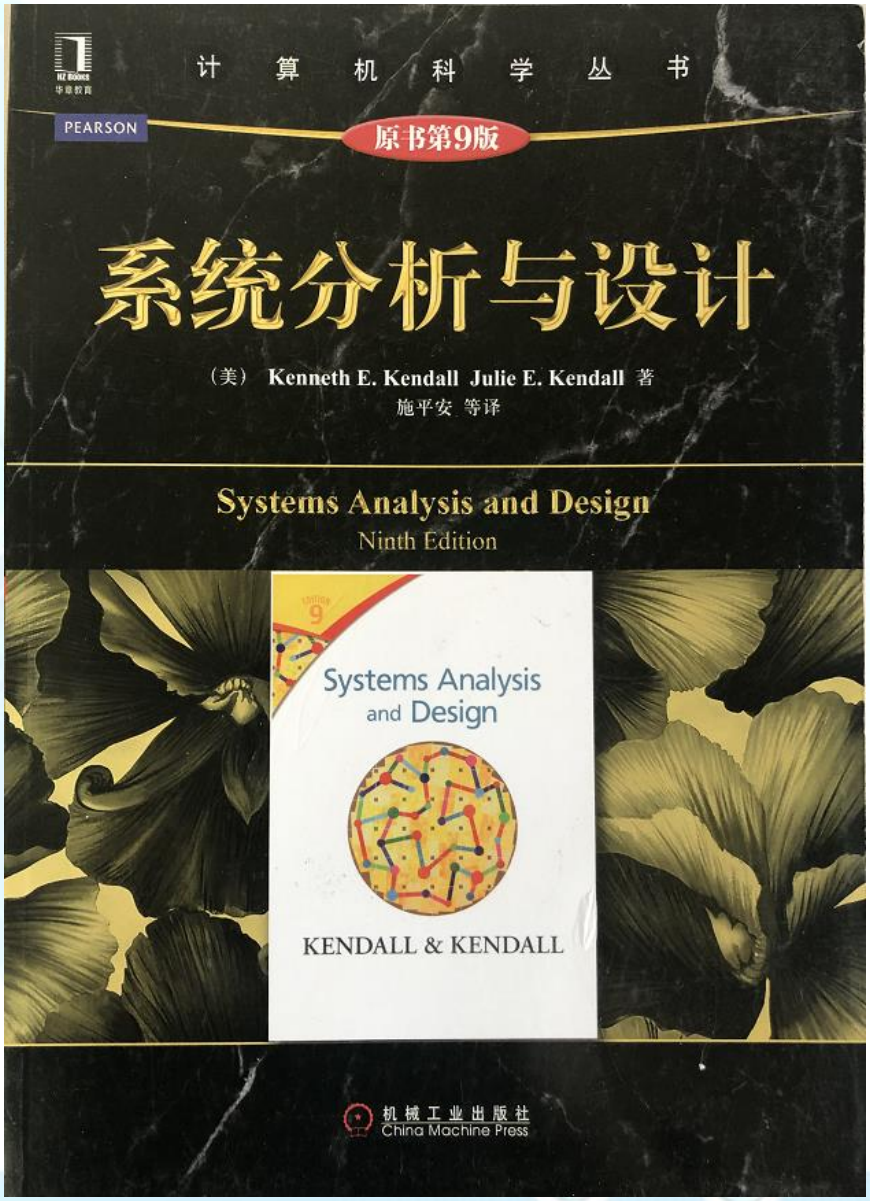
[美]Kenneth E. Kendall

等著

施平安等译

机械工业出版社

2016.10



# 教学参考书推荐

参考书 No.02：

《软件工程：实践者的研究方法》（第8版）

[美]Rogers S. Pressman著

郑人杰等译

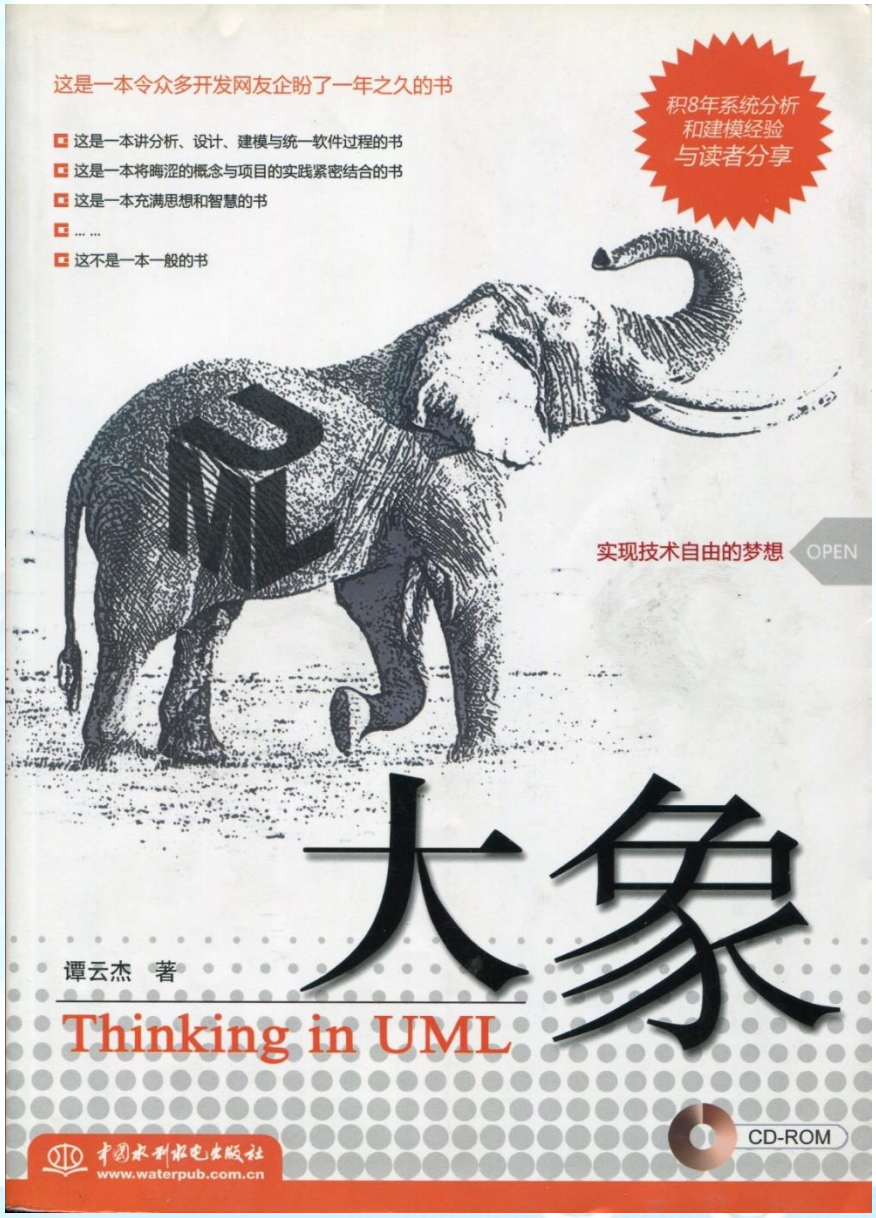
机械工业出版社

2021.08

# 教学参考书推荐

参考书 No.03：

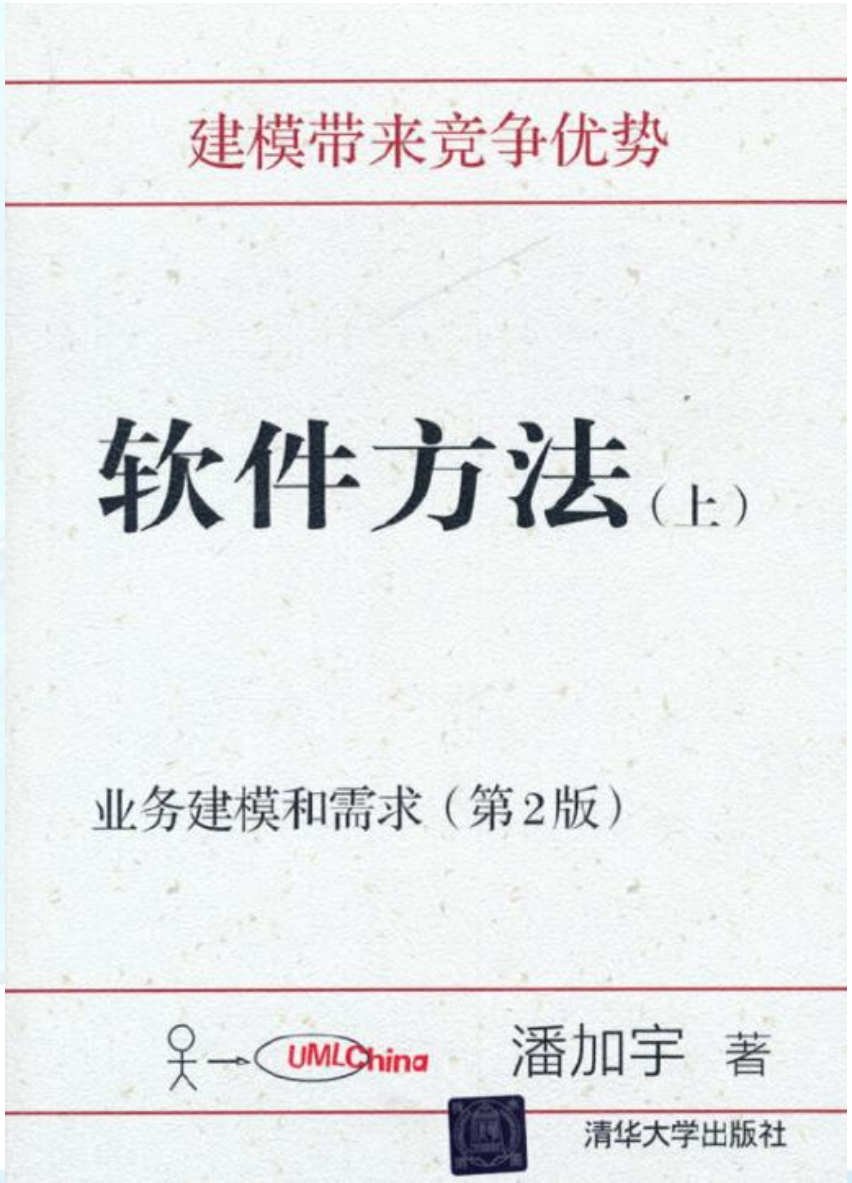
《大象-Thinking in UML》
（第2版）
谭云杰著
中国水利水电出版社
2012.06



# 教学参考书推荐

**参考书 No.04：**

**《软件方法（上）业务建模和需求》(第2版）**
**潘加宇著**
**清华大学出版社**
**2018.03**



# 教学参考书推荐

参考书 No.05：

《人月神话》

[美]Frederick P. Brooks著

汪颖等译

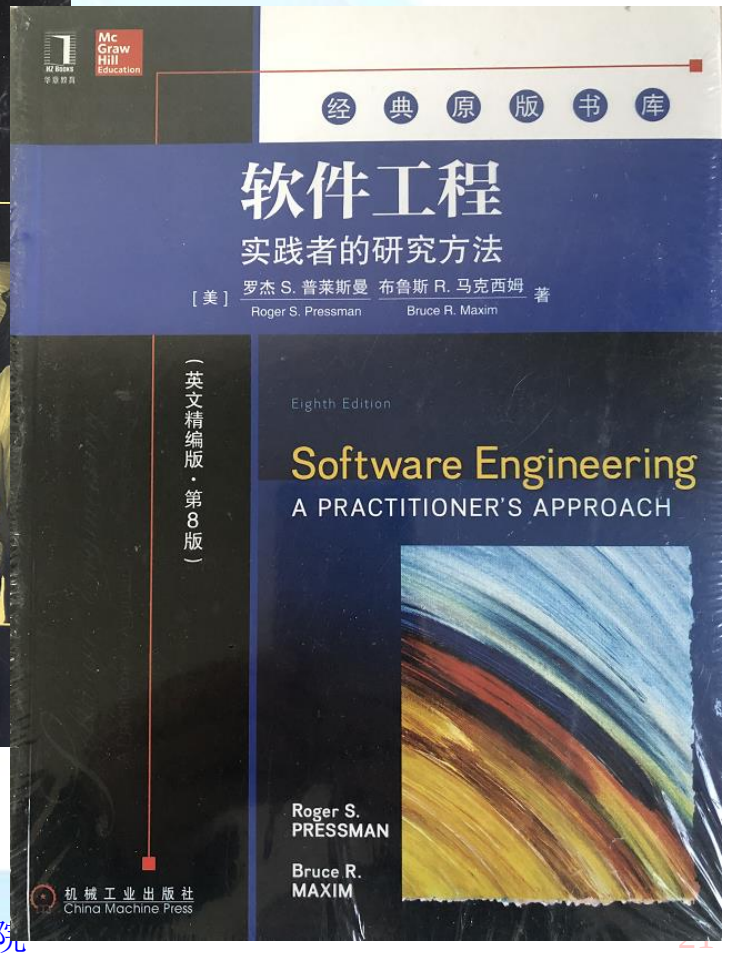
清华大学出版社

2015.04

# 教学参考书推荐

计 算 机 科 学 丛 书
PEARSON
原书第9版
系统分析与设计
(美) Kenneth E. Kendall Julie E. Kendall 著
施平安 等译
Systems Analysis and Design
Ninth Edition
Systems Analysis and Design
KENDALL & KENDALL
机械工业出版社
China Machine Press

计 算 机 科 学 丛 书
原书第8版
本科教学版
软件工程
实践者的研究方法
[美] 罗杰 S. 普莱斯曼 (Roger S. Pressman)
布鲁斯 R. 马克西姆 (Bruce R. Maxim) 著 郑人杰 马素霞 等译
Software Engineering
A Practitioner's Approach Eighth Edition · Chinese Abridgement
Eighth Edition
Software Engineering
A PRACTITIONER'S APPROACH
Roger S. PRESSMAN
Bruce R. MAXIM
机械工业出版社
China Machine Press


经 典 原 版 书 库
软件工程
实践者的研究方法
[美] 罗杰 S. 普莱斯曼 Roger S. Pressman 布鲁斯 R. 马克西姆 Bruce R. Maxim 著
(英文精编版·第8版)
Eighth Edition
Software Engineering
A PRACTITIONER'S APPROACH
Roger S. PRESSMAN
Bruce R. MAXIM
机械工业出版社
China Machine Press

# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 09

# 概　述

## 本章主要内容

- 基本概念
- 信息系统的分类
- 系统开发角色分工及任务
- 系统开发方法介绍
- UML统一建模语言回顾

# 概　述

## 基本概念

**系统** — 从抽象意义上讲，系统（System）是一组为实现共同目标而相互关联、相互作用的部件

**3层含义：**

（1）系统是由若干个部件组成的

可以根据系统大小可以划分为子系统 → 基本组成元素

（2）系统的部件间是相互关联、相互作用的

（3）系统有一个实现目标

该目标是系统功能的综合表现

**举例：**

地球生态系统；宇宙系统；政府机构系统；汽车系统；计算机系统（硬件系统、软件系统） …

# 概　述

## 基本概念

信息系统（Information System）— 是一种专门类型的系统，主要用于对信息进行采集、输入、处理、输出、存储、传播以及对信息进行管理。

3层含义：

（1）信息系统功能：信息输入、信息输出、信息处理、信息存储、信息传播、信息管理

（2）信息系统是一种人造系统

制作过程 - 信息系统开发
开发人员 - 系统分析员、程序员
核心工作 - 系统分析、系统设计、系统实现

（3）信息系统是一种自动系统

计算机等硬件设备
计算机网络技术
计算机软件技术

# 概　述

## 信息系统的分类

**应用于科学计算的信息系统** — 偏重于纯粹的数值计算

例如：数学计算软件Matlab、结构力学计算软件IDEAS、电磁场计算软件Ansys等

**应用于企业管理的信息系统** — 偏重于非数值信息处理

例如：人事管理系统，高考管理系统，ERP系统，OA系统

? 思考：

搜索引擎？

数据压缩工具？

多媒体处理系统？

本课程的重点：

“企业信息系统”

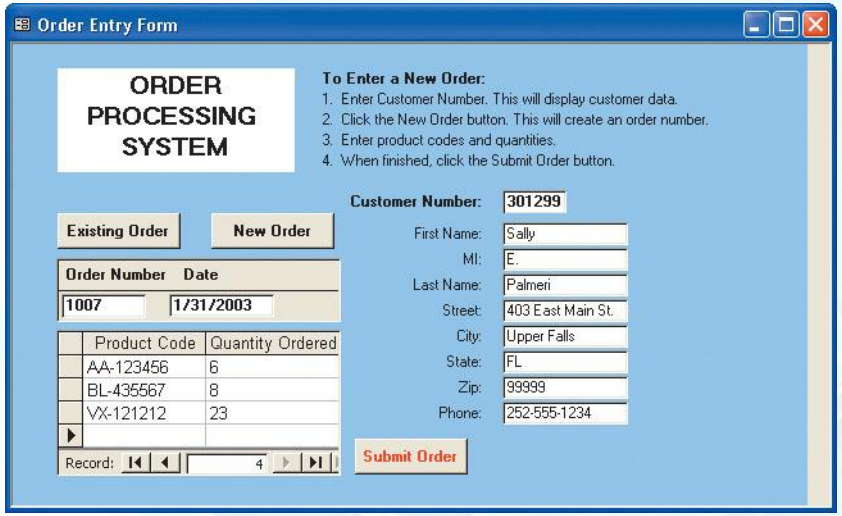
# 概　述

## 信息系统组成

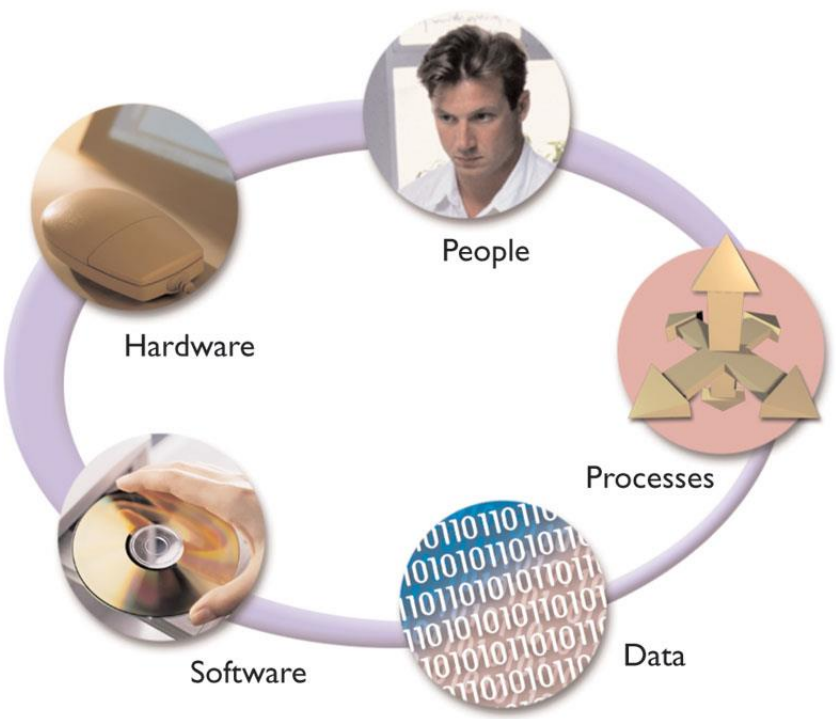
- **系统：** 一个能够产生特定结果的相关构件的集合
- **信息系统：** 以信息作为输入/输出对象，以计算机软、硬件为核心的系统
- **现代信息系统：** 以Internet为基础的信息系统
- 信息系统有5部分组成：
  **硬件**、**软件**、**数据**、**过程**和**人**

# 概　述

## 信息系统组成




订单处理系统



# 概　述

## 信息系统组成

Customer support system
Hardware
Inputs
People
Software
Outputs
Data
Procedures

# 概　述

## 信息系统组成

- 硬件
  - 由信息系统的物理层的各项组成

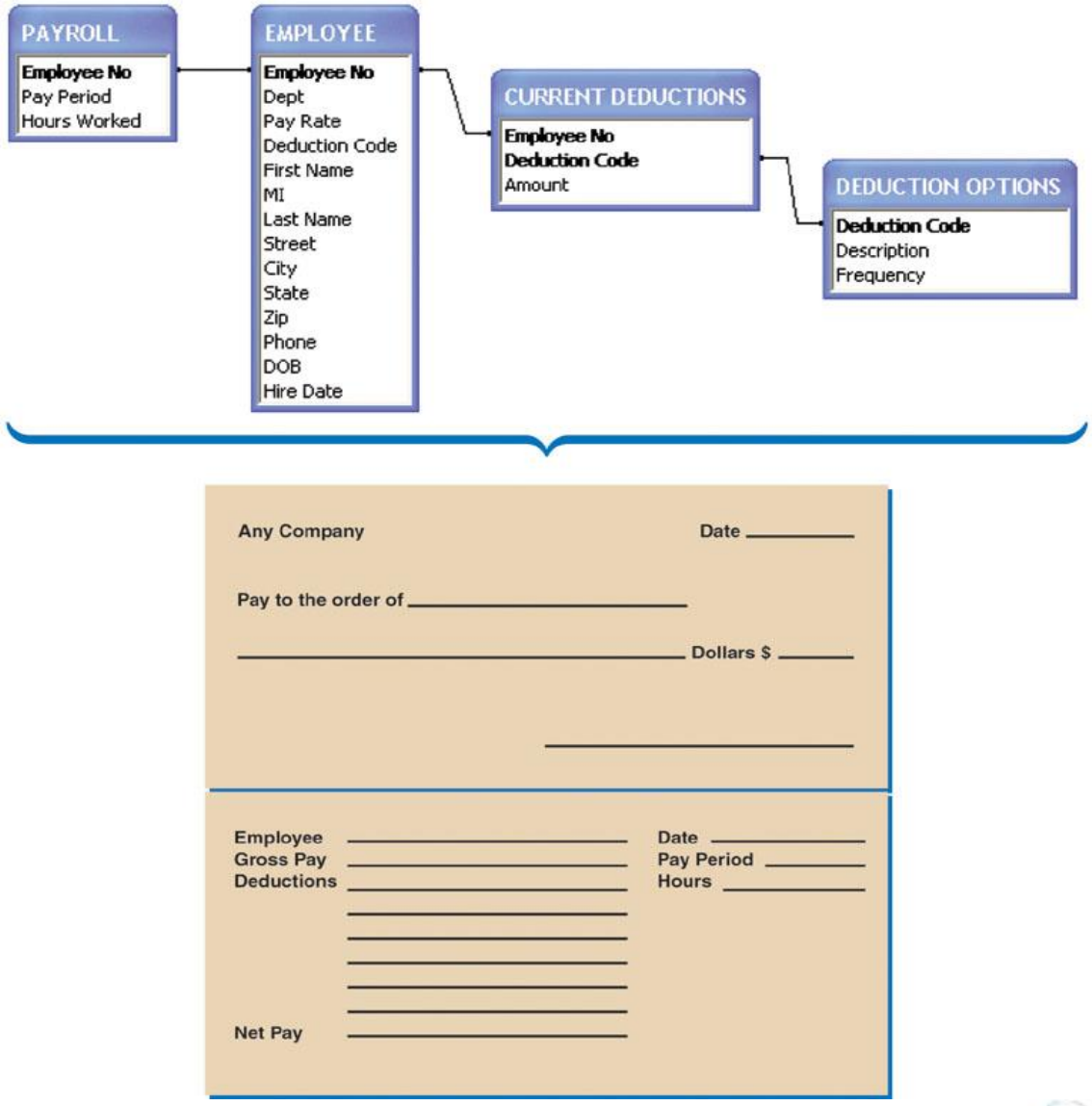
    如：服务器、工作站、网络、通讯设备、计算机、打印机、扫描仪等
- 软件
  - System software
  - Network operating system
  - Application software
  - Enterprise applications
  - Horizontal system （横向系统-通用系统）
  - Vertical system（纵向系统-特定系统）

# 概　述

## 信息系统

- 数据

数据




通过链接几张表可以生成一张具有特定信息的单子

# 概　述

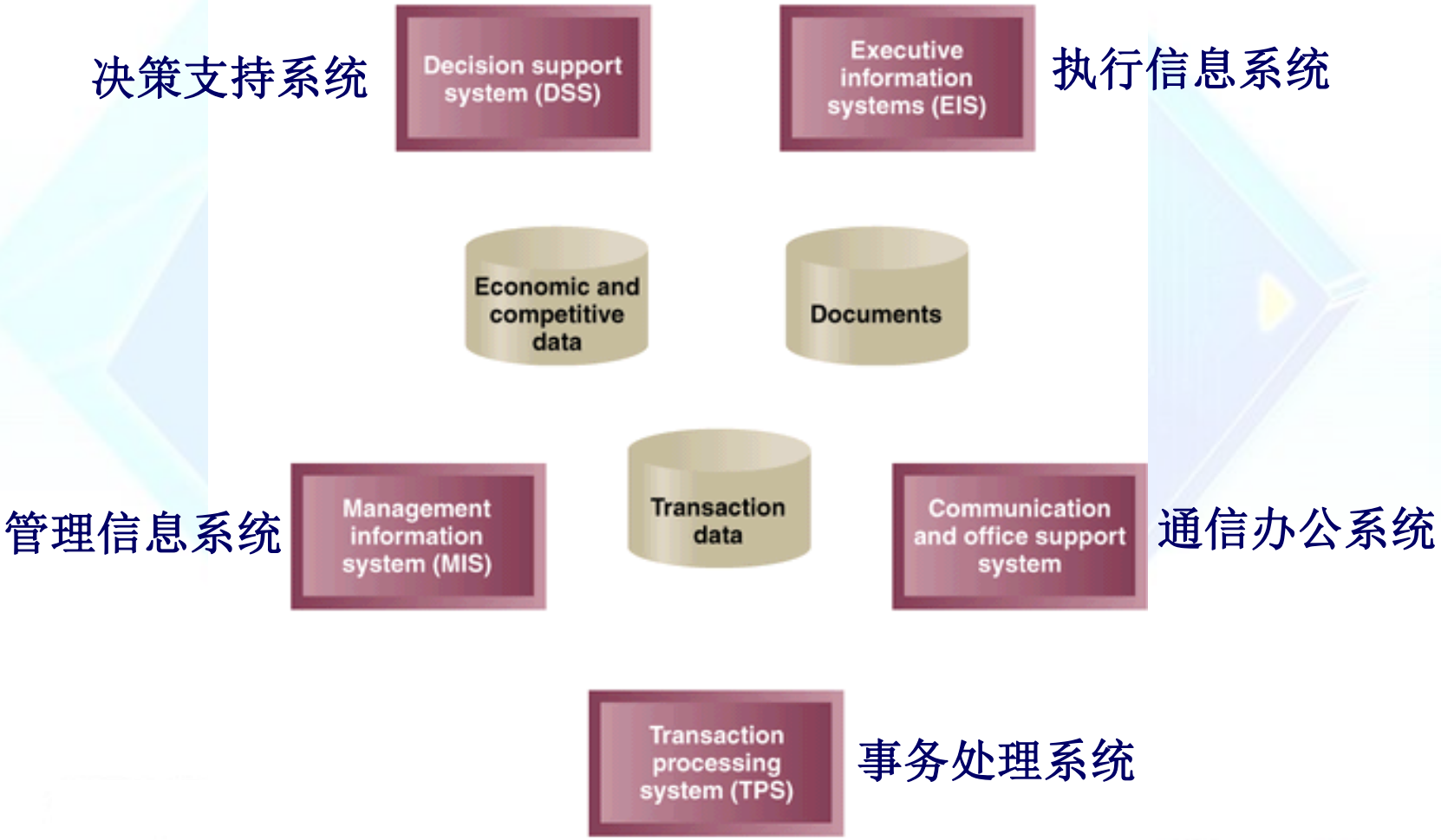
## 信息系统组成

- 过程
  - 定义用户、经理、职员所执行的要达到指定结果的任务和企业功能
- 人
  - 同信息系统打交道的所有人员

例如：系统的设计者、开发者
系统的维护人员、操作员
企业的员工用户、经理用户
利用系统获得服务的客户等

# 概　述

## 企业信息系统主要类型



# 概　述

## 实习或工作中常见信息系统类型

- **产品类系统**
  - 金山WPS、杀毒软件系统、用友ERP系统、游戏软件、超图GIS等
- **项目类系统**
  - ERP系统（实施）
  - 管理信息系统
  - 办公自动化系统
  - 电子商务平台
  - 客户关系管理系统
  - 嵌入式系统
  - 基于GIS平台的系统等

# 系统开发角色分工及任务

一个软件产品或者软件工程，参与角色通常包括：

- 高级经理 （战略部署、风险控制、资源提供）
- 产品经理或项目经理
  （制定计划、协调资源、关注和控制进度、控制客户期望值）
- 开发经理 （确定架构、开发工具，组织编码实现）
- 系统分析员（需求分析师、构架设计师、功能设计师）
- 测试经理 （制订测试方案，组织测试）
- 程序员 （编码实现、单元测试）
- 测试人员 （按测试方案测试）
- 业务顾问 （甲方派出，负责业务咨询和确认）
- 实施人员 （系统安装、割接、维护）

# 概　述

## 系统开发角色分工及任务

### 系统分析员任务：

- 了解客户的需求
- 建立分析模型
- 了解系统环境与平台
- 建立设计模型
- 编写分析与设计文档

系统分析说明书

系统设计说明书

# 系统开发方法介绍

- 宏观上（即从过程管理角度）：
  - 开发模型：瀑布、演化、螺旋、增量等
  - 过程管理：RUP（统一软件开发过程）
    RAD（快速应用程序开发，如Phased、Prototype）
    MSF（微软解决方案框架）
    JAD（联合应用软件开发）
    Agile系列方法（XP、SCRUM等）
- 微观上（即从分析、设计、实现角度）：
  - 结构化开发方法
  - 面向对象开发方法
  - 基于UML的面向对象开发方法

# 系统开发方法介绍

- 结构化开发方法
  - 起源时间：20世纪60年代
  - 思想方法：自顶向下、问题分解、分而治之
  - 表达模型：分析模型（数据流图、系统流程图、数据字典）
    设计模型（系统结构图、程序流程图、ERD）
  - 优　　点：思维自然，符合人们思考问题的方式
    总体可控性强
    适合偏重数学计算方面的项目
  - 缺　　点：不容易描述客观世界的需求
    耦合性相对高

# 系统开发方法介绍

- 面向对象开发方法
  - 起源时间：20世纪80年代（70年代开始OOP）
  - 思想方法：从客观世界的具体事物出发构建系统；
    自底向上，先考虑“对象”，再考虑“关系”
  - 表达模型：类层次结构图（表达对象的属性、行为、继承、消息连接等）
  - 优　　点：符合自然世界的状态，描述自然，思维简单
    适合复杂的事务处理、大量信息处理类的项目
    耦合性容易降低，容易复用
  - 缺　　点：OO模型表达系统需求分析和设计不够充分

# 系统开发方法介绍

- 基于UML的面向对象开发方法
  - 起源时间：20世纪90年代末
  - 思想方法：开发一整套系统分析与设计模型及描述方法，从形式上和概念上统一描述OO系统
  - 表达模型：用例图、活动图、状态图、序列图（协作图）、类图、组件图、包图、部署图等
  - 优　　点：统一、标准；容易交流；建模工具较多；适合大型信息系统开发
  - 缺　　点：模型不容易掌握

# 概　述

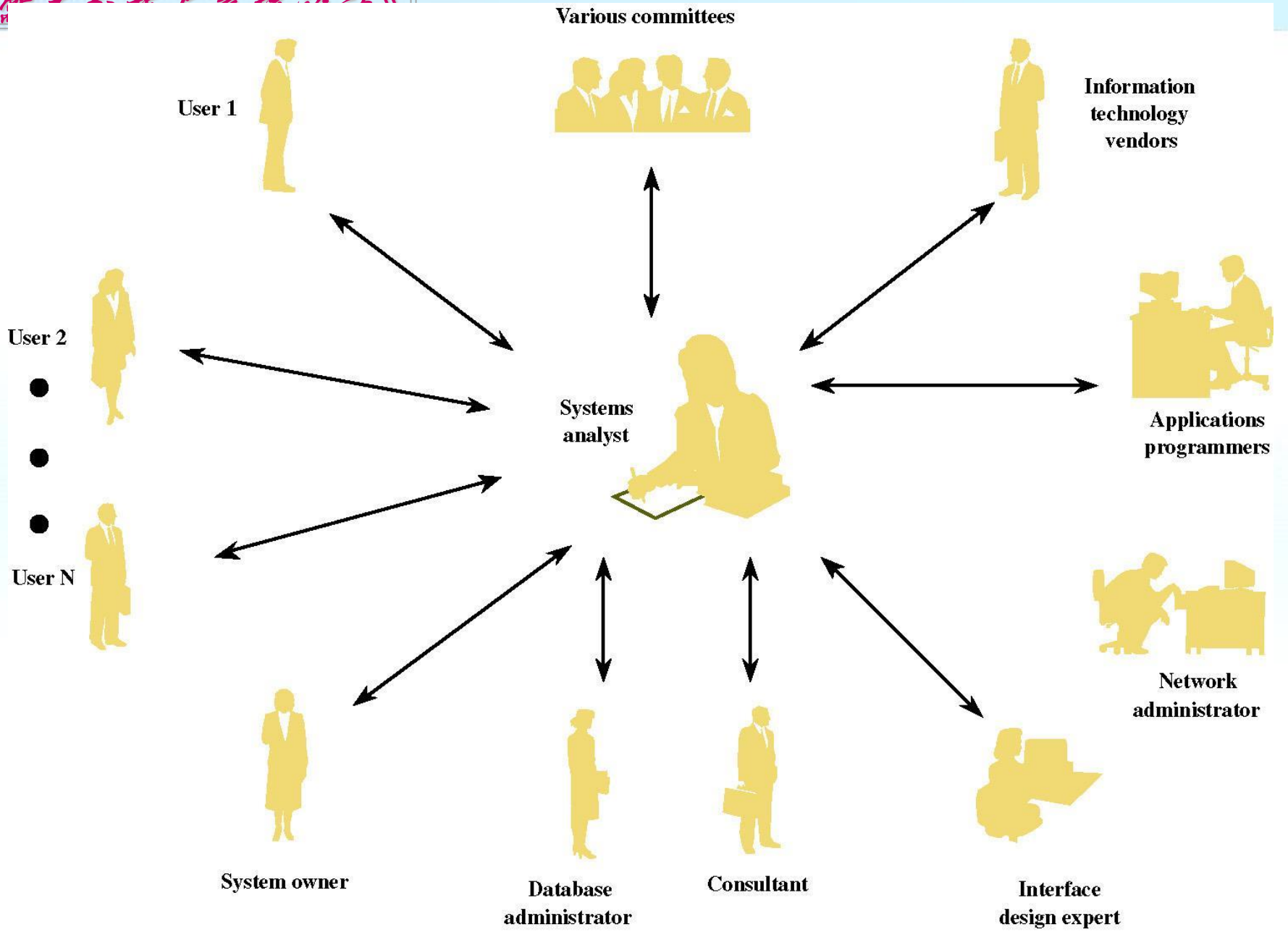
## 系统开发工具

- 计算机辅助系统工程工具 – **CASE工具**
- 图形化建模工具：**Rational Rose / Microsoft Visio / StarUML**
- 原型制作工具：**Balsamiq Mockups / ExtJS / jQuery / EasyUI / Vue.JS / 墨刀**
- 集成开发环境：**Visual Studio.NET / Eclipse**
- 数据库设计工具：**PowerDesigner / ERWin**
- 代码生成工具：**Rational Rose（含）**
- 项目管理工具：**Microsoft Project / 禅道**
- 软件配置工具：**Git/GitHub/GitLab/GitEE（码云）**
- **……**

# 概　述

## 需求分析与系统设计的内容--系统分析员的工作职责

- 参与进行可行性研究
- 参与制定软件项目开发计划
- 系统分析：
  - 需求获取和描述
  - 需求分析与建模
  - 方案评估及选择
- 系统设计：
  - 系统架构设计
  - 系统概要设计
  - 系统详细设计
  - 数据库设计
  - 用户界面设计




在需求分析与系统设计中作为核心人物的系统分析员及其打交道的各类人员

# 概　述

## 需求分析与系统设计设计的本质

- 医生眼里的人：

  目无全人，看到的都是器官和组织

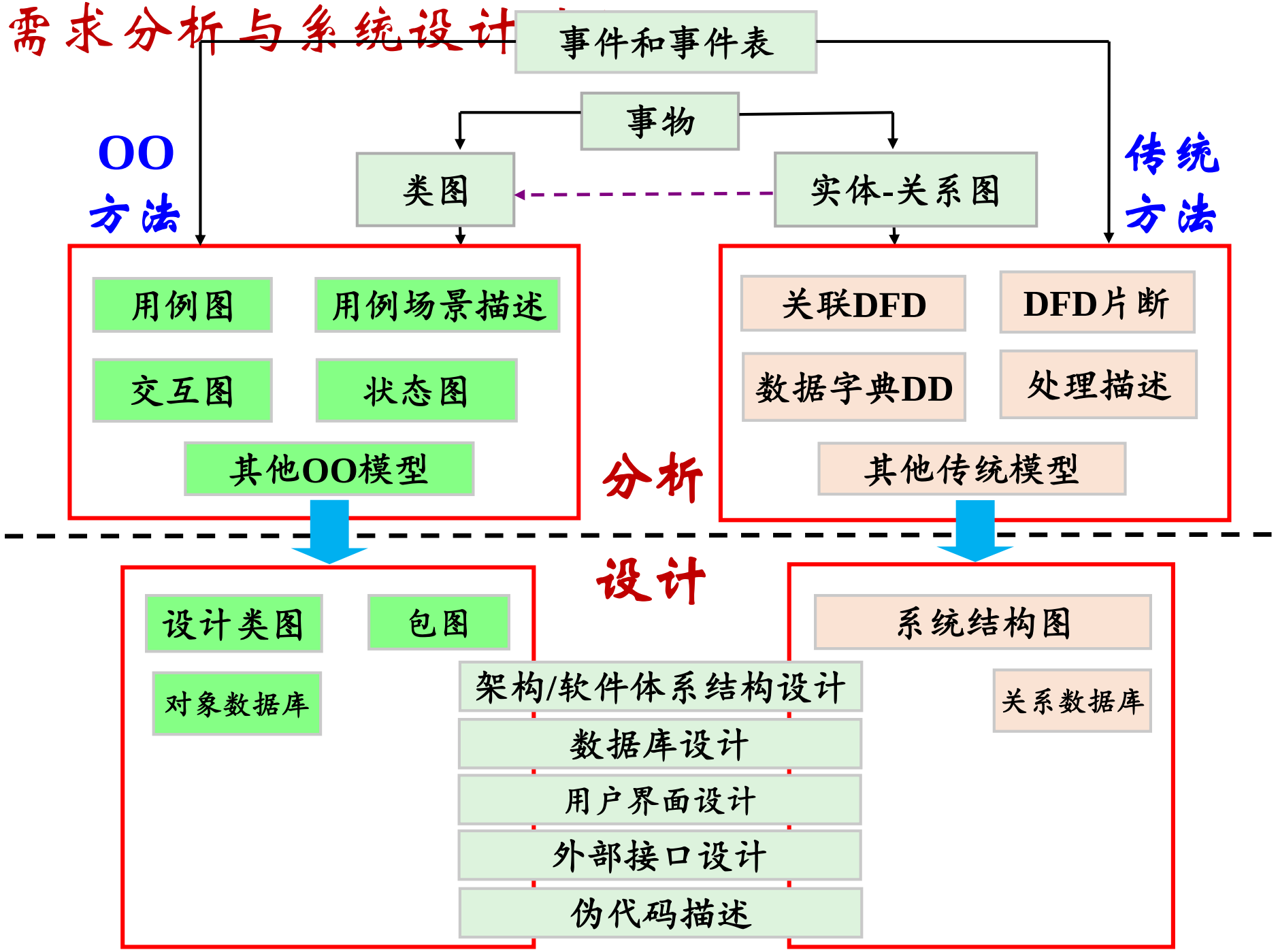
- 系统分析员眼里：

  静态的视图，运动的逻辑

- 系统建模本质：

  各种“视图”　－　盲人摸象

# 需求分析与系统设计过程


需求分析与系统设计
事件和事件表
事物
类图
实体-关系图
OO
方法
传统
方法
用例图
用例场景描述
交互图
状态图
其他OO模型
关联DFD
DFD片断
数据字典DD
处理描述
其他传统模型
分析
设计
设计类图
包图
对象数据库
架构/软件体系结构设计
数据库设计
用户界面设计
外部接口设计
伪代码描述
系统结构图
关系数据库

# 概　述

## 用于分析和设计的模型

### 分析阶段创建的模型

- 事件列表
- 事物列表

结构化方法

- 系统流程图
- 决策树/决策表
- 实体关系图
- 数据流图
- 数据字典
- 伪代码

基于UML的OO方法

- 用例图
- 用例场景描述
- 分析类图
- 活动图
- 状态图
- 时序图/协作图

### 设计阶段创建的模型

- 体系结构图
- 界面布局图

- 系统结构图
- 程序流程图

==========

- 设计类图
- 时序图
- 包图
- 组件图
- 网络图
- 部署图

# 概　述

## UML统一建模语言回顾

# 概述—UML统一建模语言回顾

## UML 定义

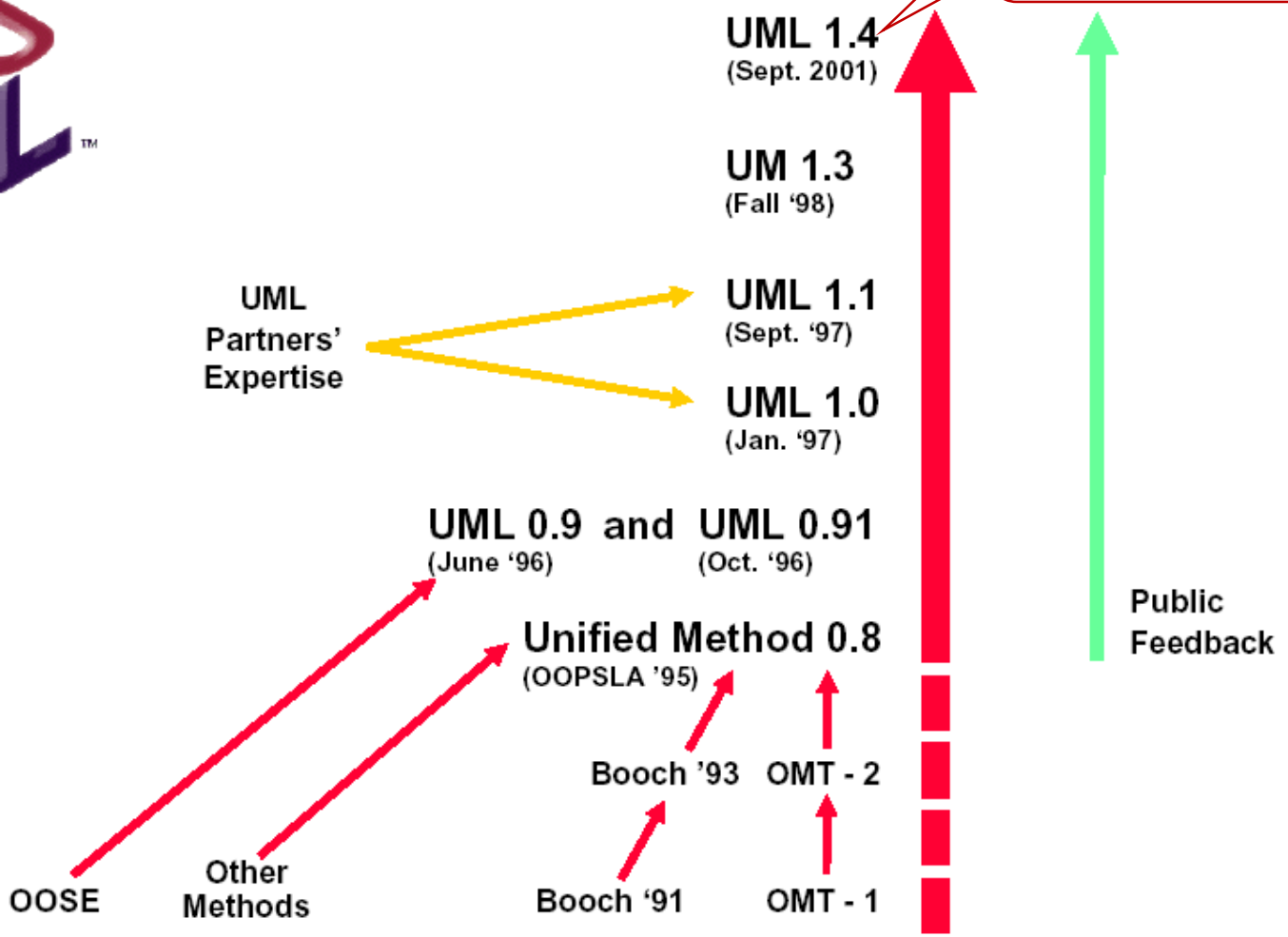
- **UML — Unified Modeling Language**
- **UML 是一种对软件系统的制作过程/产出物进行下述工作的描述语言：可视化（visualizing）、详述 （specifying）、构造 （constructing）、文档化（documenting）**

- UML 是可视化语言
  - UML 是图形化语言，便于交流
- UML 是一种可以详细描述的语言
  - 所建的模型是精确的，无歧义和完整的
- UML 是用于构造系统或理解系统的语言
  - UML 既支持正向工程，又支持反向工程
- UML 是文档化语言
  - 将所构造的系统记录下来
  - 便于后续人员跟进

# 概述—UML统一建模语言回顾

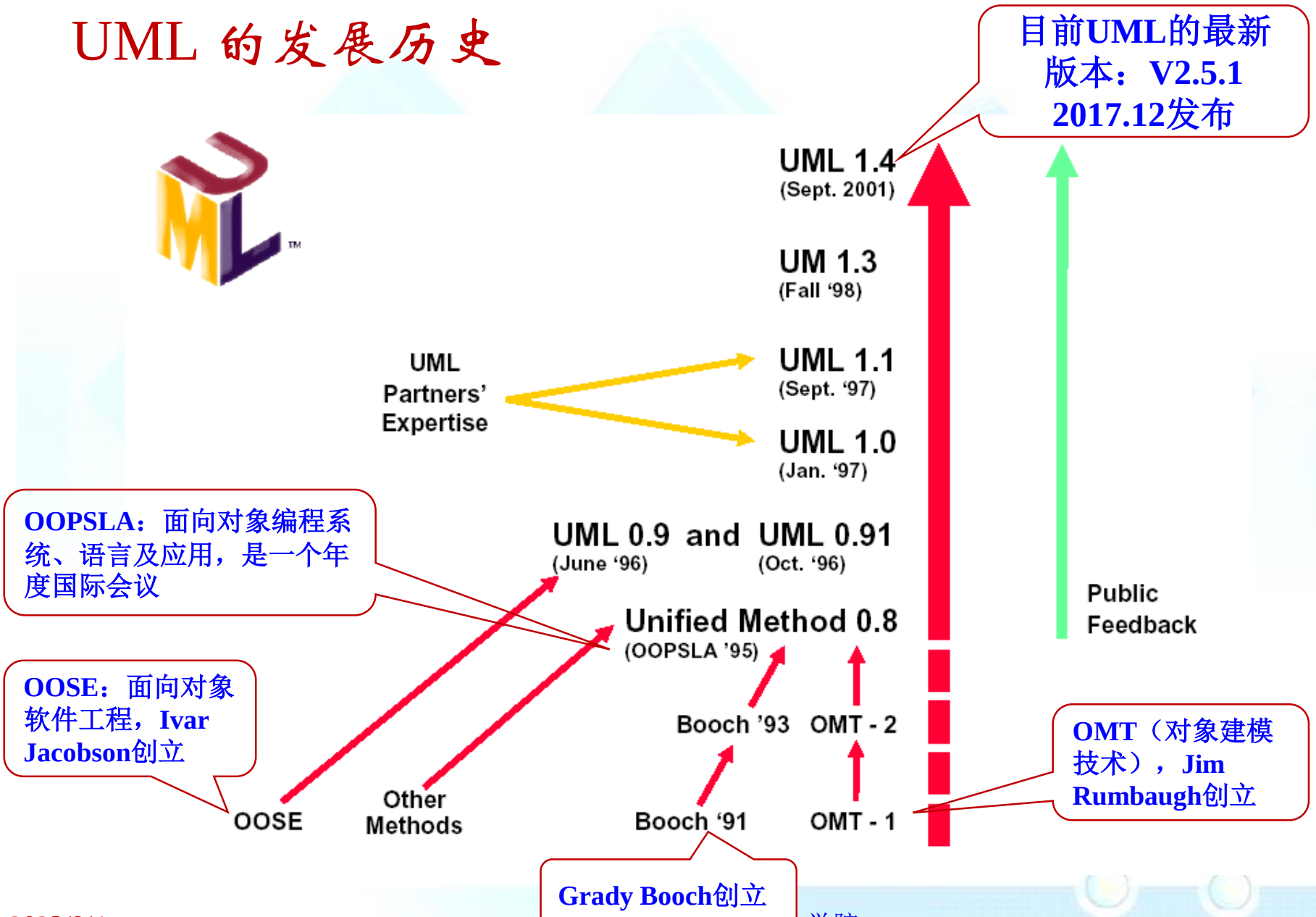
## UML 的发展历史

目前UML的最新版本：V2.5.1
2017.12发布



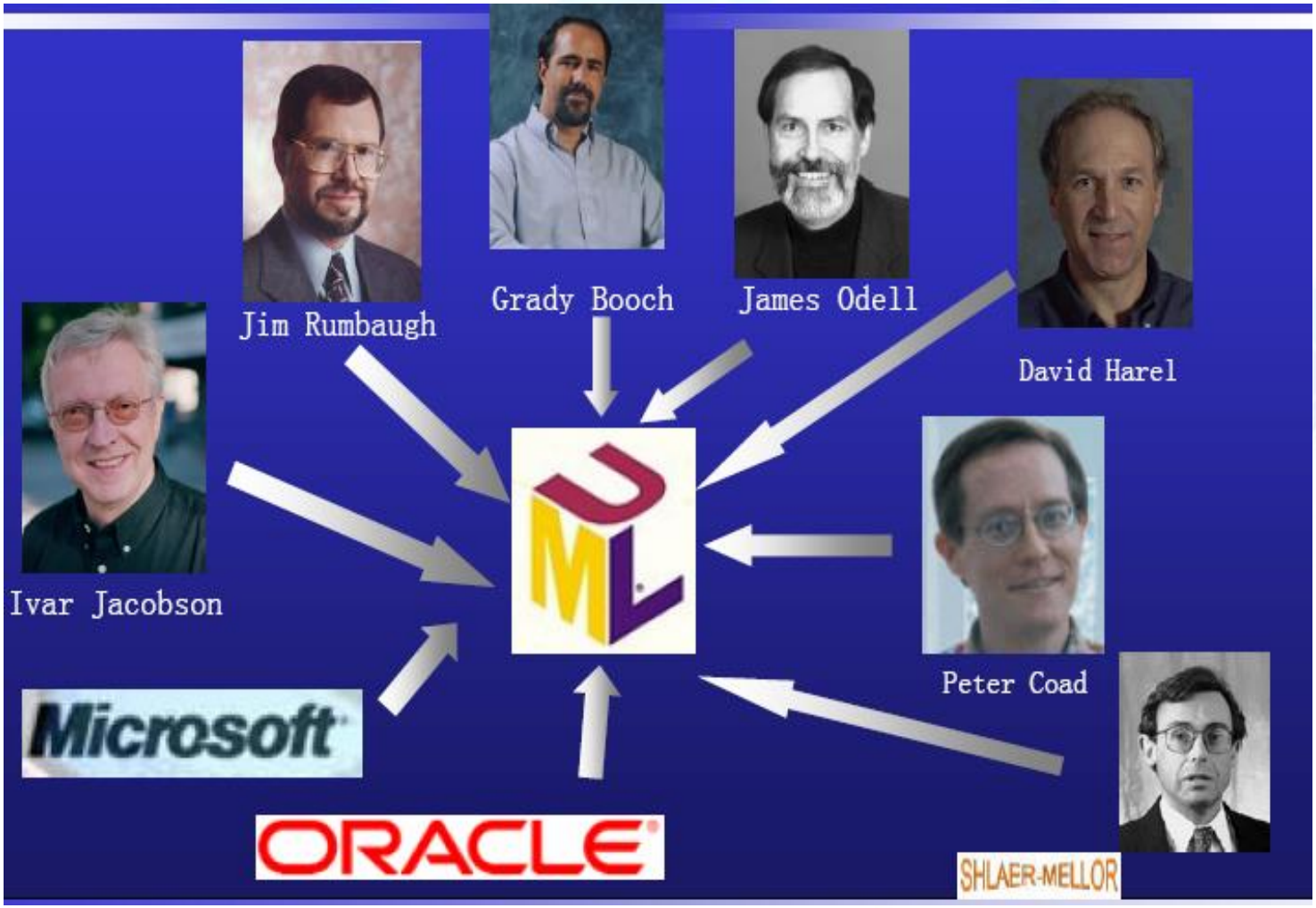
# 概述—UML统一建模语言回顾

## UML 的发展历史



# 概述—UML统一建模语言回顾

## 为 UML 创建做出贡献的人们



# 概述—UML统一建模语言回顾

## UML 工具

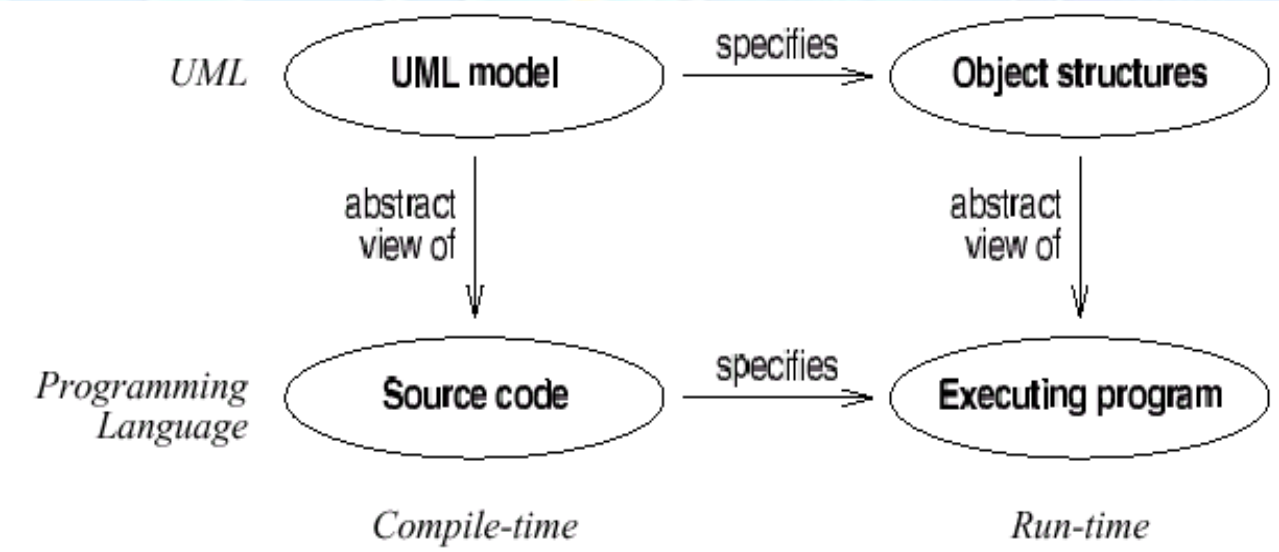
# 概述—UML统一建模语言回顾

## UML 的特点

- 统一标准：已成为面向对象的标准化的统一的建模语言
  - 面向对象
  - 可视化、表示能力强大
  - 独立于过程
  - 概念明确，建模表示法简洁，图形结构清晰，容易掌握使用

# UML 和代码的关系

- 用 Java，C++ 等 programming language 是用编码实现一个系统
- 用 UML 是对一个系统建立模型
- 一些工具（如Rational Rose）可以根据 UML 所建立的系统模型来产生 Java，C++ 或其它程序语言代码框架



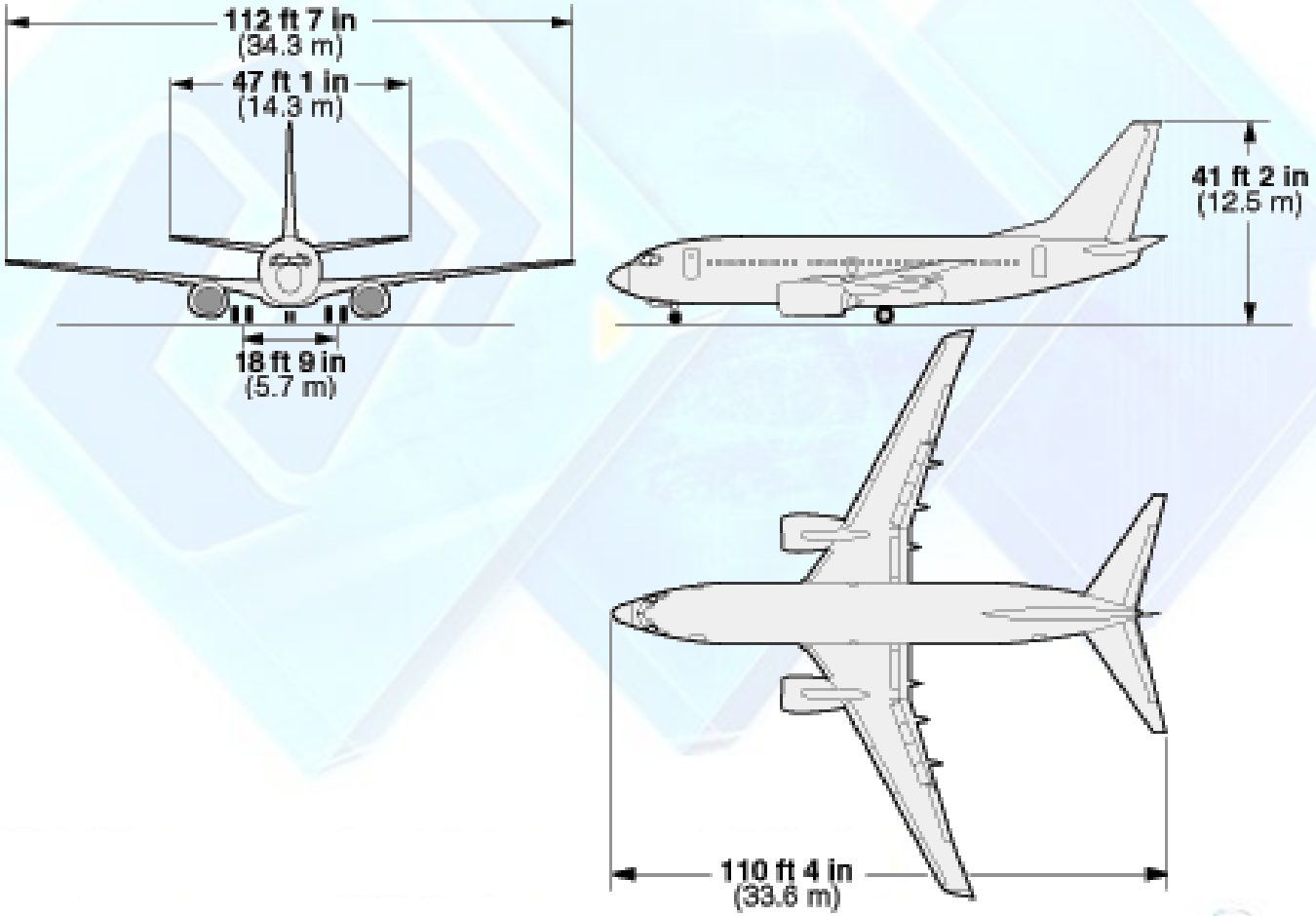
# 概述—UML统一建模语言回顾

## UML 的构成

- 视图（Views）
- 图（Diagrams）
- 模型元素
- *通用机制

# 视图（Views）

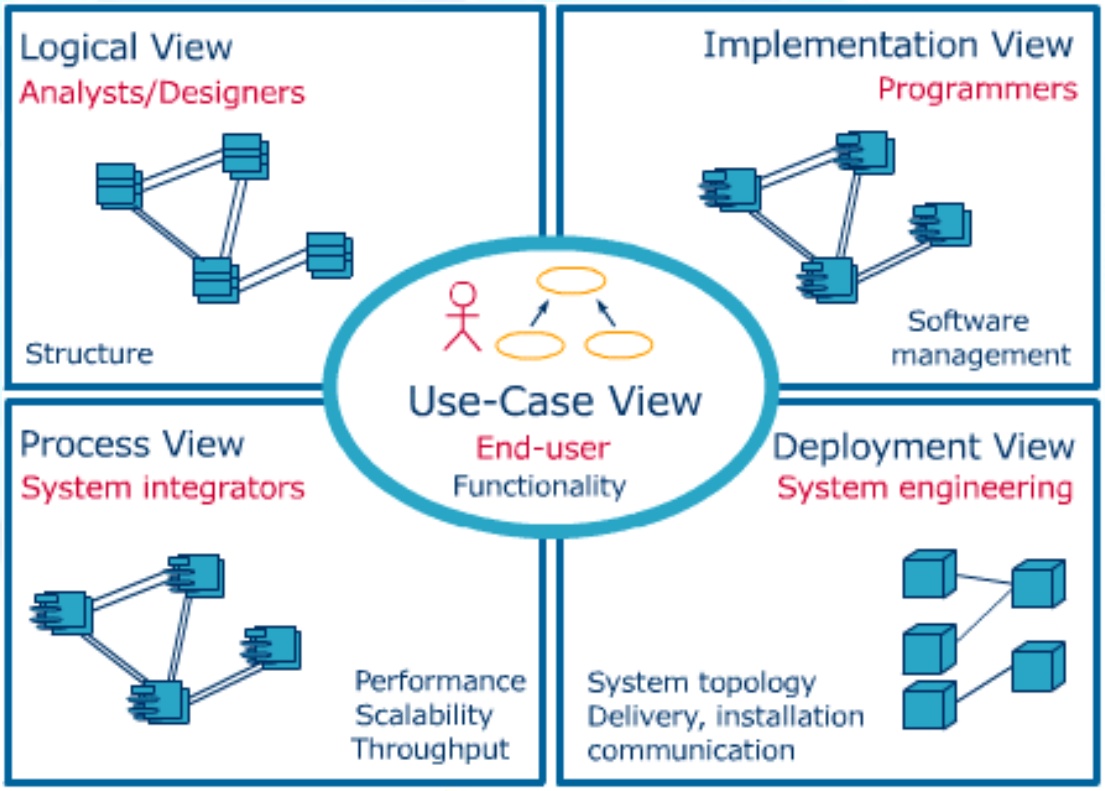
- 飞机的三视图



# 概述—UML统一建模语言回顾

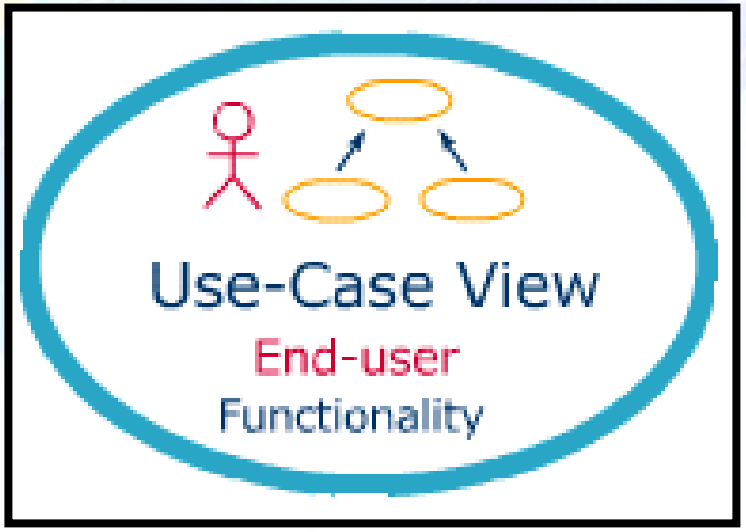
## 视图（Views）

- 视图是表达系统某一方面特征的 UML 建模元素的子集，它是由一个或者多个图组成的对系统某个角度的抽象
- 视图包括：
  - **Use Case View**
  - **Logical View**
  - **Process View**
  - **Implementation View**
  - **Deployment View**



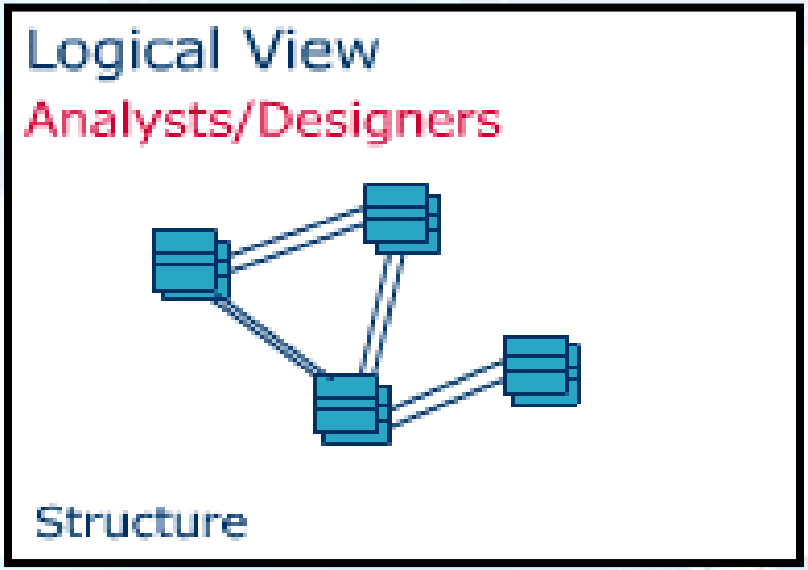
# Use-Case View（用例视图）

- 用途：描述系统应该具备的功能，即被称为参与者（执行者）的外部用户所能观察到的功能
- 用例视图是几个视图的核心，它的内容直接驱动其他视图的开发
- 包含UML图：用例图
- 使用者
  - 分析人员



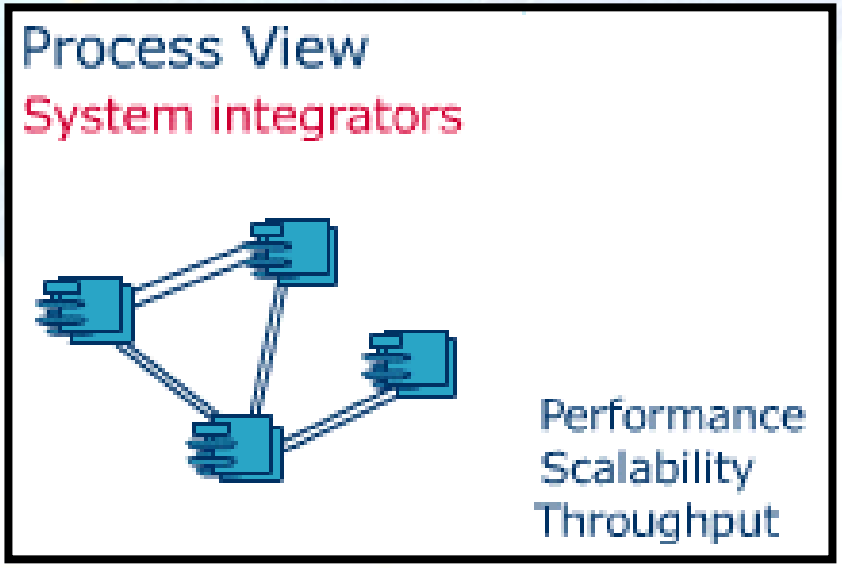
# Logical View（逻辑视图）

- 用途：描述用例视图中提出的系统功能的实现
- 逻辑视图既描述系统的静态结构，也描述系统内部的动态协作关系
- 包含UML图：类图、对象图、状态图、时序图、协作图、活动图
- 静态结构在类图和对象图中描述
- 动态模型在状态图、时序图、协作图、活动图中描述
- 使用者
  - 分析人员
  - 设计人员
  - 开发人员



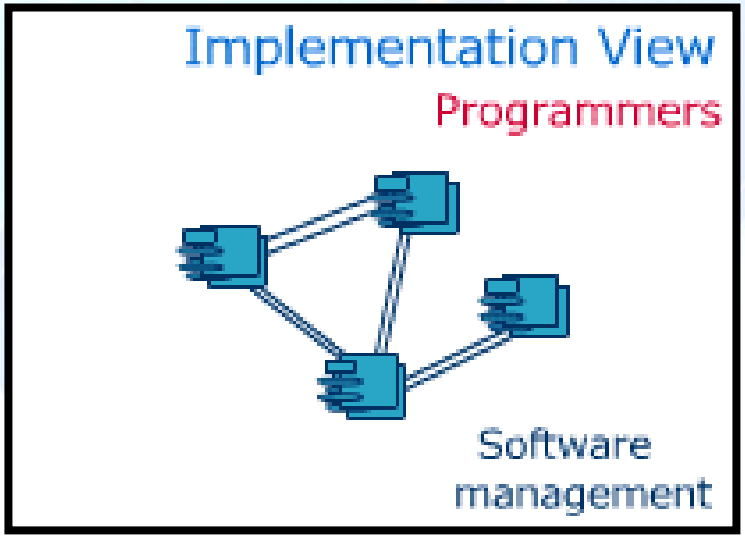
# Process View（进程视图）

- 用途：考虑资源的有效利用、代码的并行执行以及系统环境中异步事件的处理
- 解决在并发系统中存在的通信和同步问题
- 包含UML图：状态图、协作图、组件图、活动图
- 使用者
  - 开发人员
  - 系统集成人员



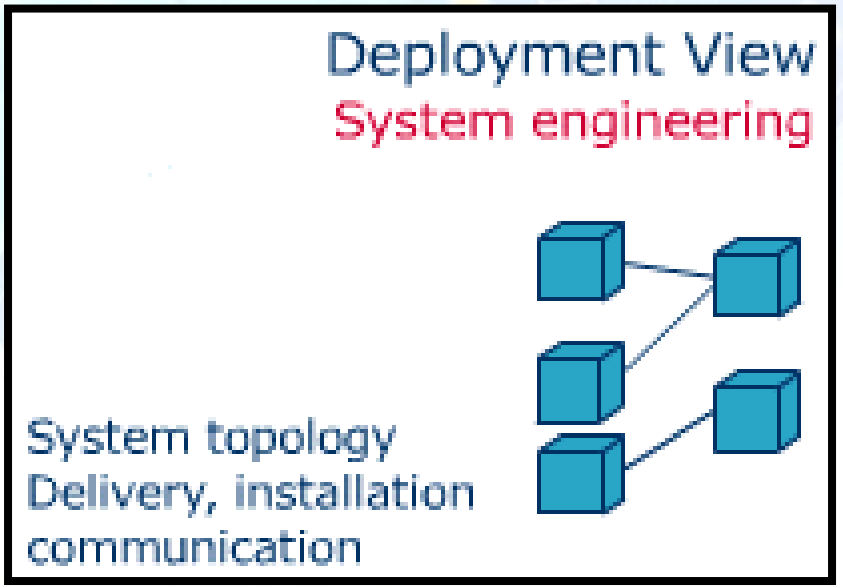
# Implementation View（实现视图）

- 用途：描述系统的实现模块以及它们之间的依赖关系
- 包含UML图：组件图
- 使用者
  - 开发人员



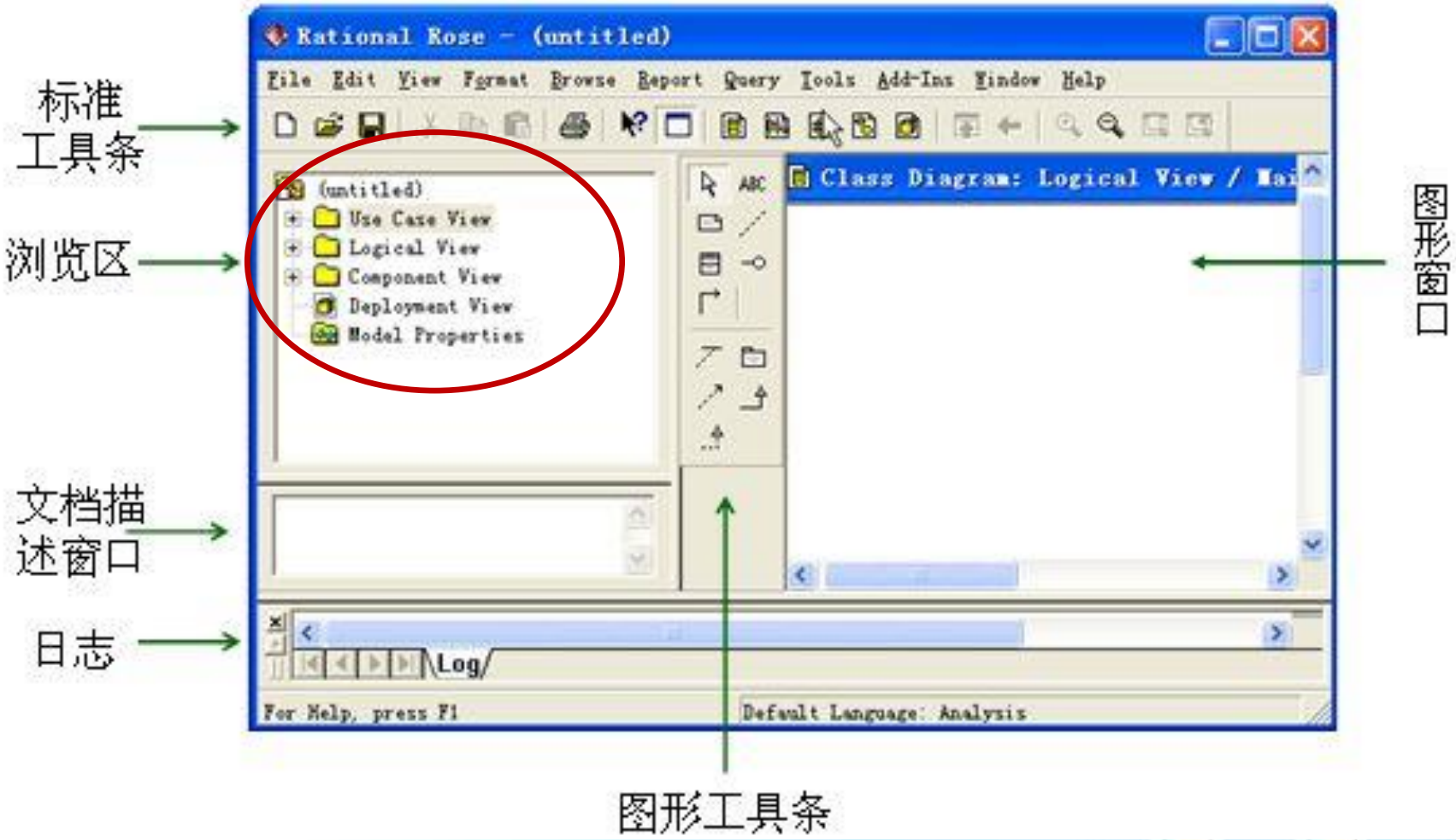
# Deployment View（部署视图）

- 用途：显示系统的物理部署，并描述位于节点实例上的运行组件实例的部署情况
- 组成：部署图
- 使用者
  - 开发人员
  - 系统集成人员
  - 测试人员



# 典型UML建模工具：Rational Rose

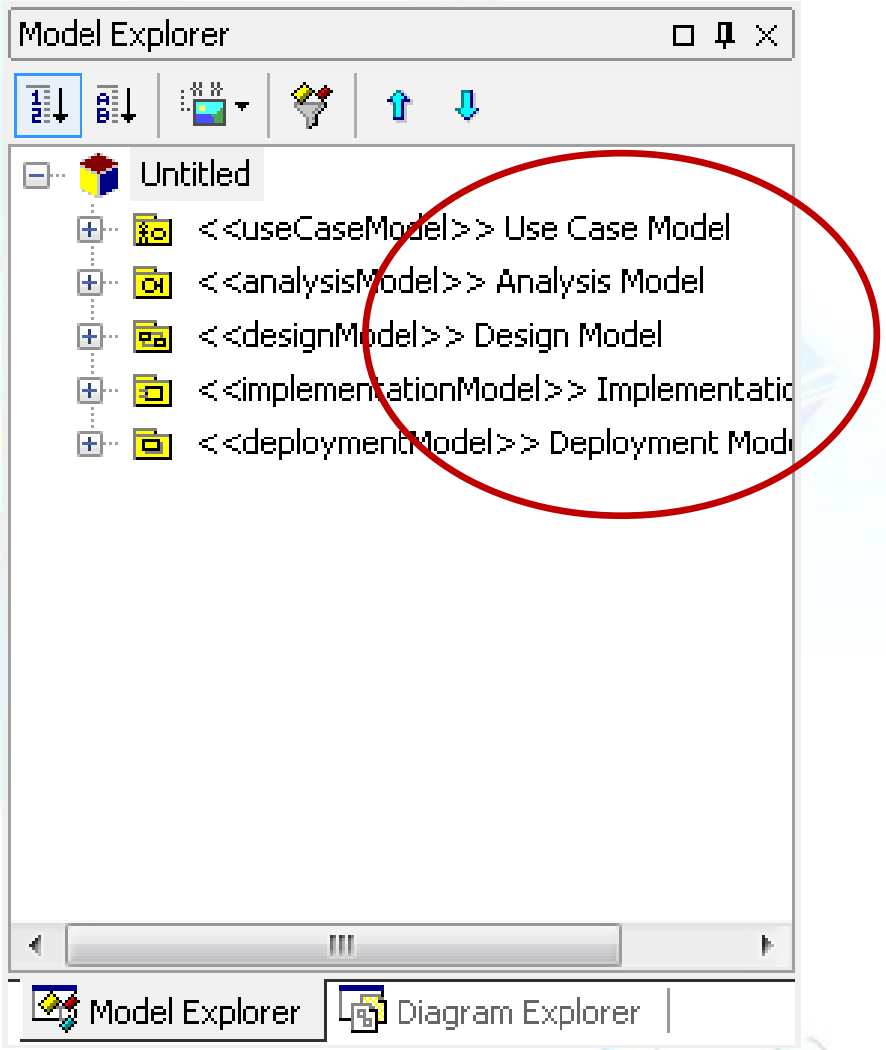
- **4个 View**



# 概述—UML统一建模语言回顾

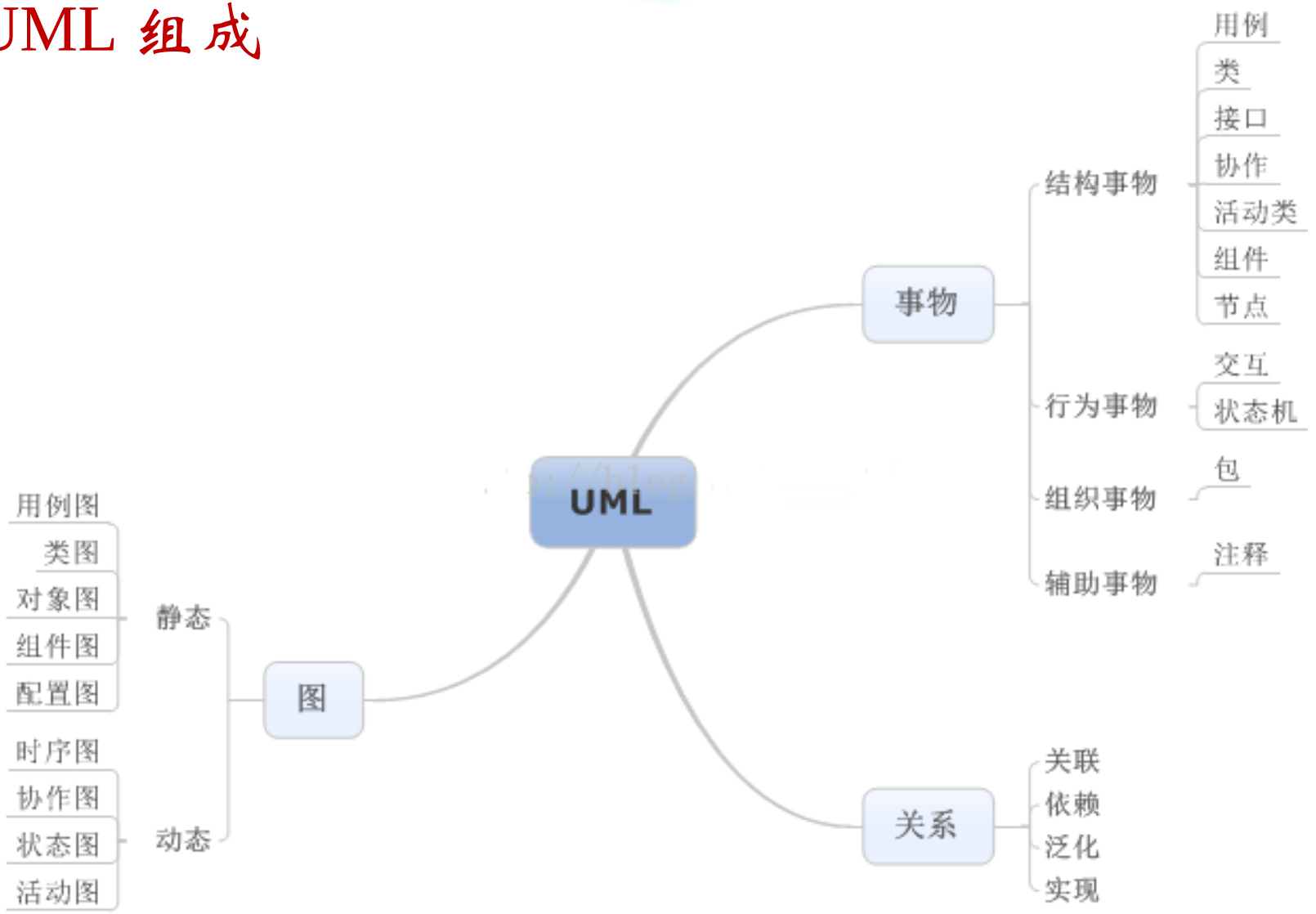
## 典型UML建模工具： StarUML

- 5个 Model



# 概述—UML统一建模语言回顾

## UML 组成



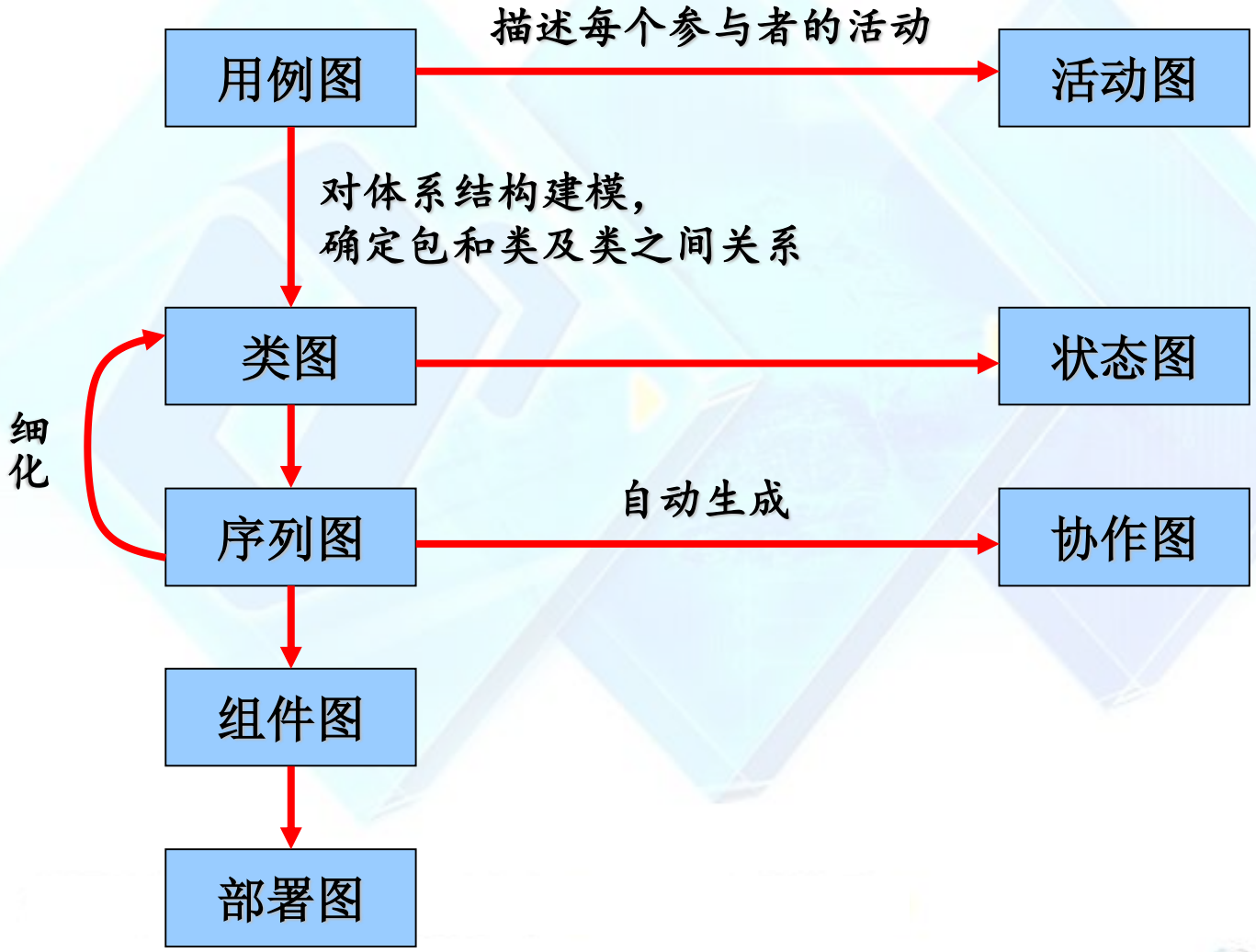
# 概述—UML统一建模语言回顾

## UML中的模型图（9种）

- 类图 (class diagram )
- 对象图 (object diagram )
- 用例图 (use case diagram )
- 时序图 (sequence diagram )
- 协作图 (collaboration diagram )
- 状态图 (statechart diagram )
- 活动图 (activity diagram )
- 组件图 (component diagram )
- 部署图 (deployment diagram )

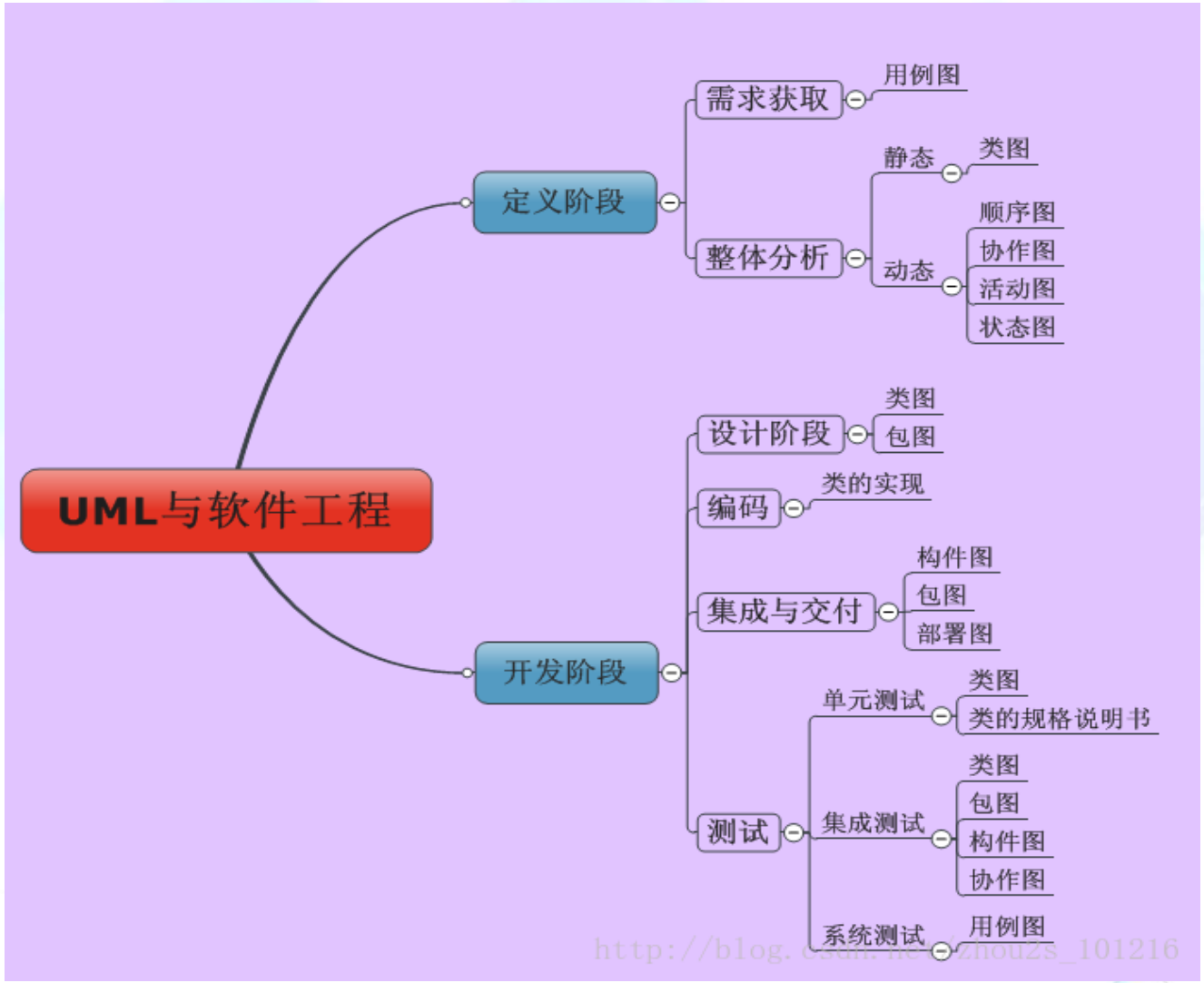
* 包图（结构化建模用）

# UML图之间的关系


用例图
描述每个参与者的活动
活动图
对体系结构建模，
确定包和类及类之间关系
类图
状态图
细化
序列图
自动生成
协作图
组件图
部署图

# 概述—UML统一建模语言回顾

## UML图与软件工程的关系



# 概述—UML统一建模语言回顾

## UML图与使用者之间的关系

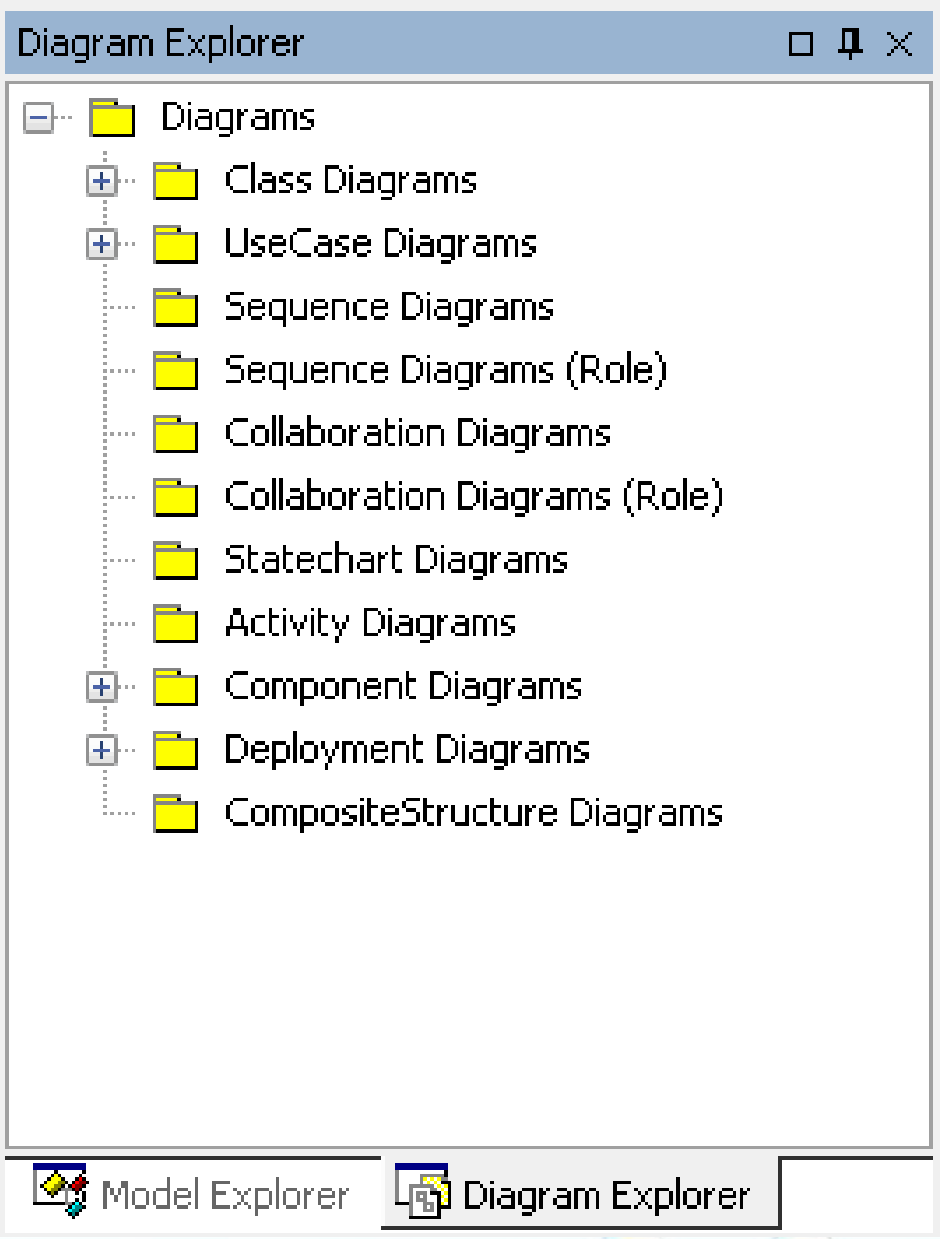
| UML图的使用人员 | | | | | |
|---|---|---|---|---|---|
| 人员 / 图 | 系统用户 | 分析人员 | 设计人员 | 开发人员 | 测试人员 |
| 用例图 | | | | | |
| 类图 | | | | | |
| 对象图 | | | | | |
| 序列图 | | | | | |
| 协作图 | | | | | |
| 状态图 | | | | | |
| 活动图 | | | | | |
| 构件图 | | | | | |
| 部署图 | | | | | |

http://blog.csdn.net/zhou2s_101216

# 概述—UML统一建模语言回顾

## StarUML支持UML图

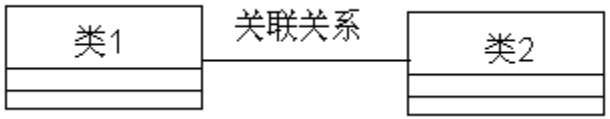
- **Diagram**

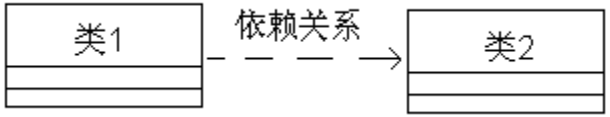


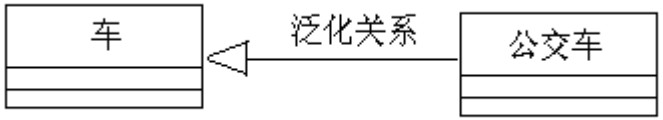
# 概述—UML统一建模语言回顾

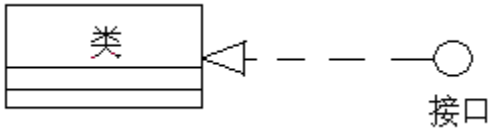
## UML中的关系

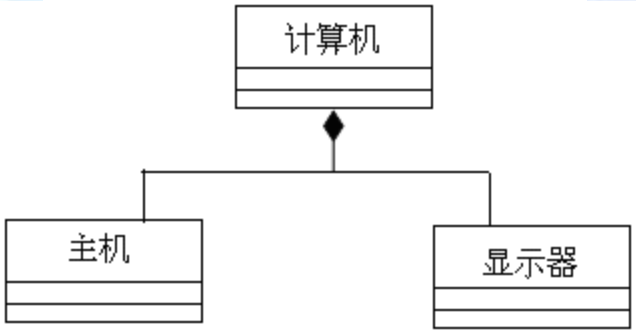
- 关联
- 依赖
- 泛化
- 实现
- 聚合/组合



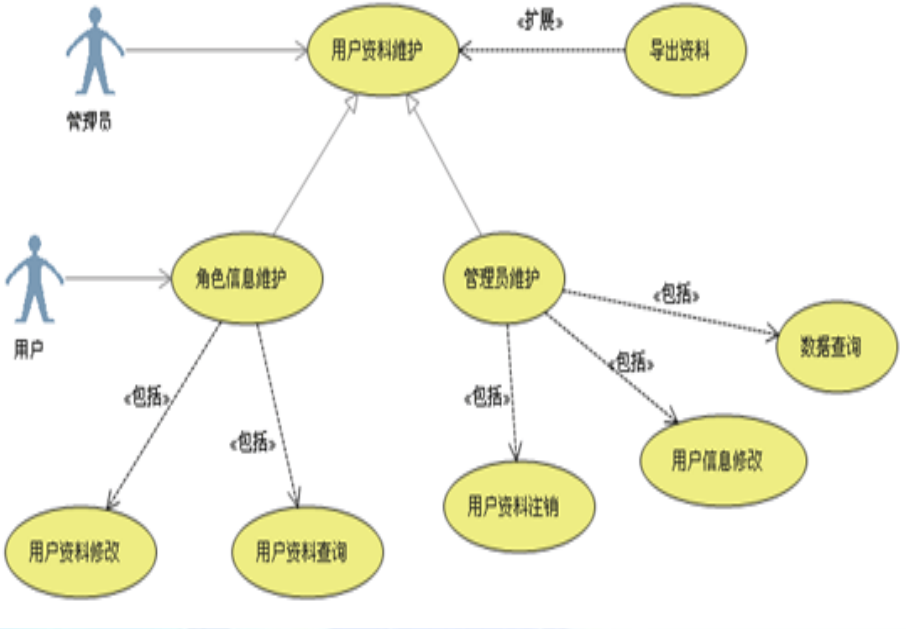



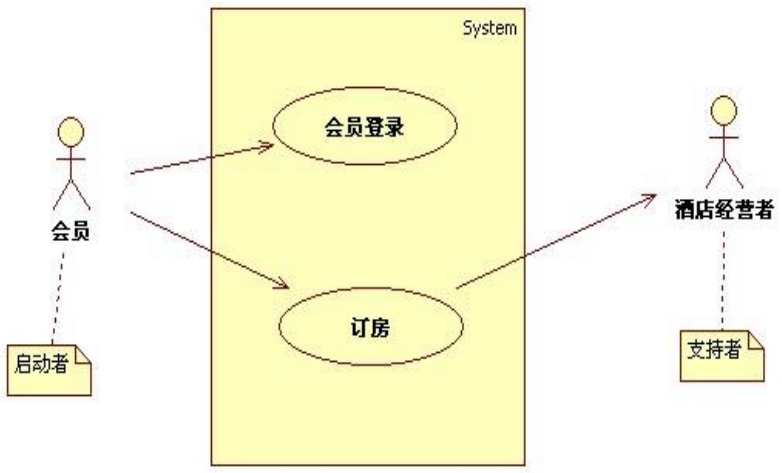



# 概述—UML统一建模语言回顾

## 用例图（UseCaseDiagram）

- 从用户的角度描述了系统的功能，并指出各个功能的执行者，强调用户的使用者，系统为执行者完成哪些功能

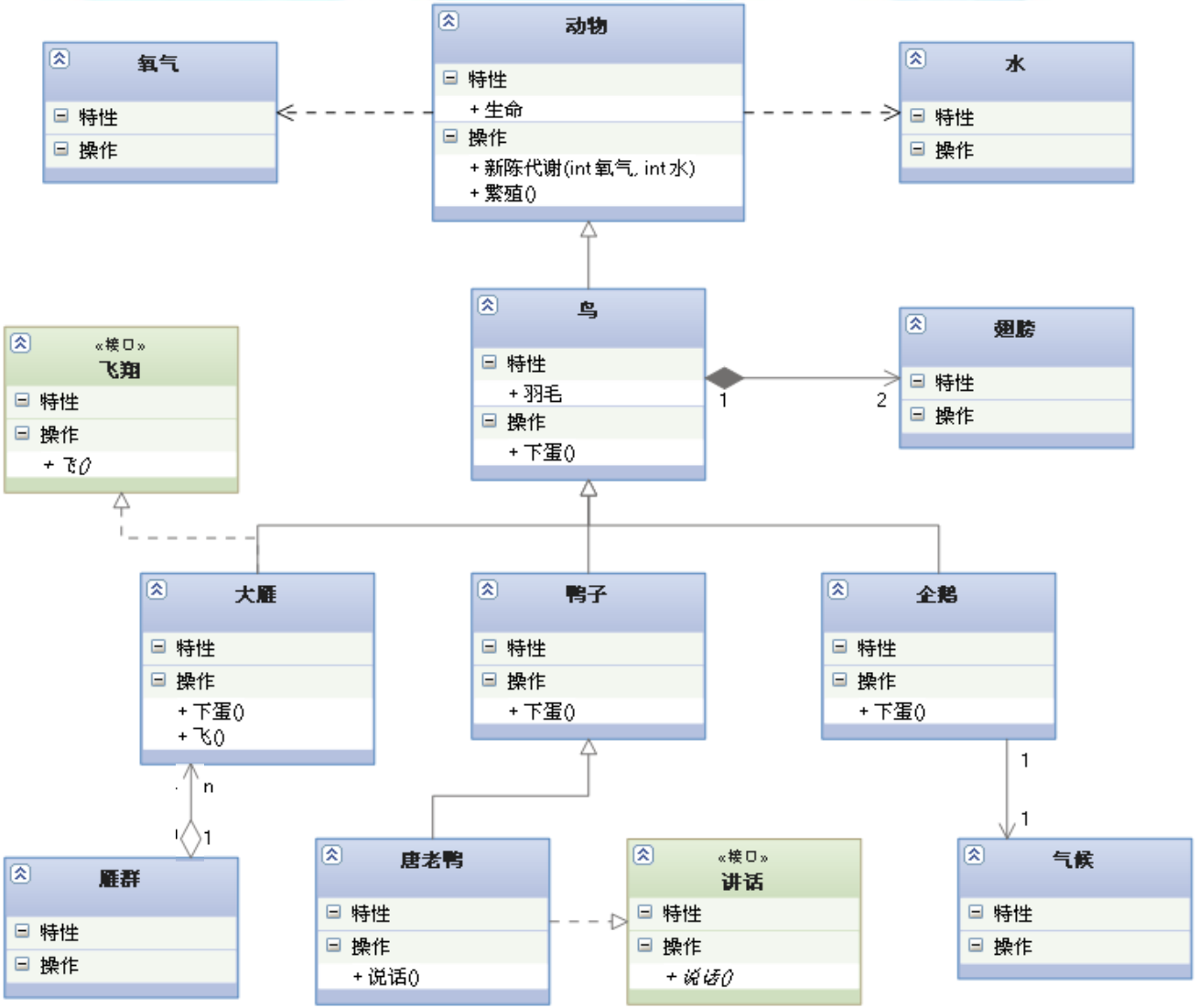


# 概述—UML统一建模语言回顾

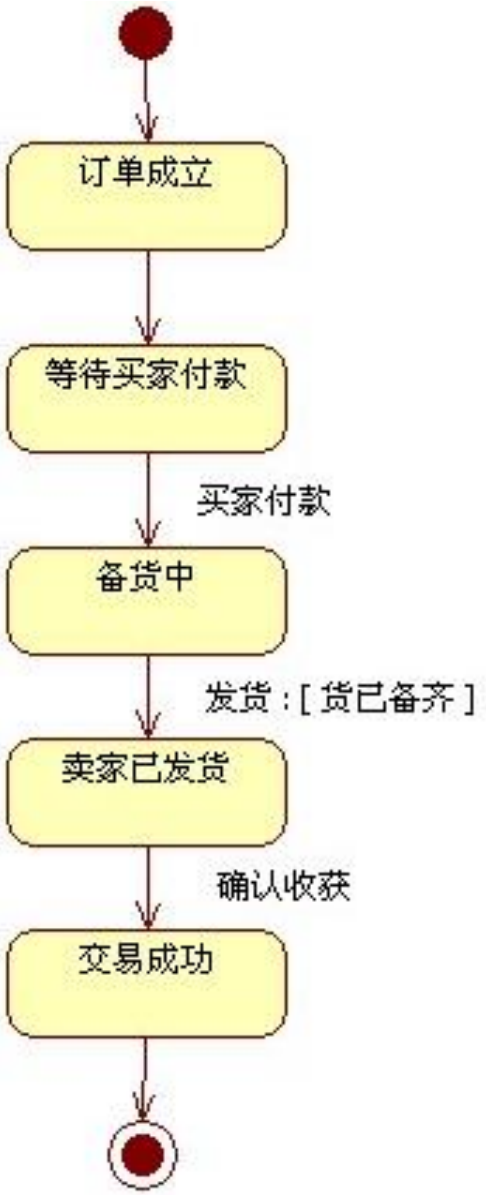
## 类图（ClassDiagram）

- 描述类的内部结构和类与类之间的关系，是一种静态结构图



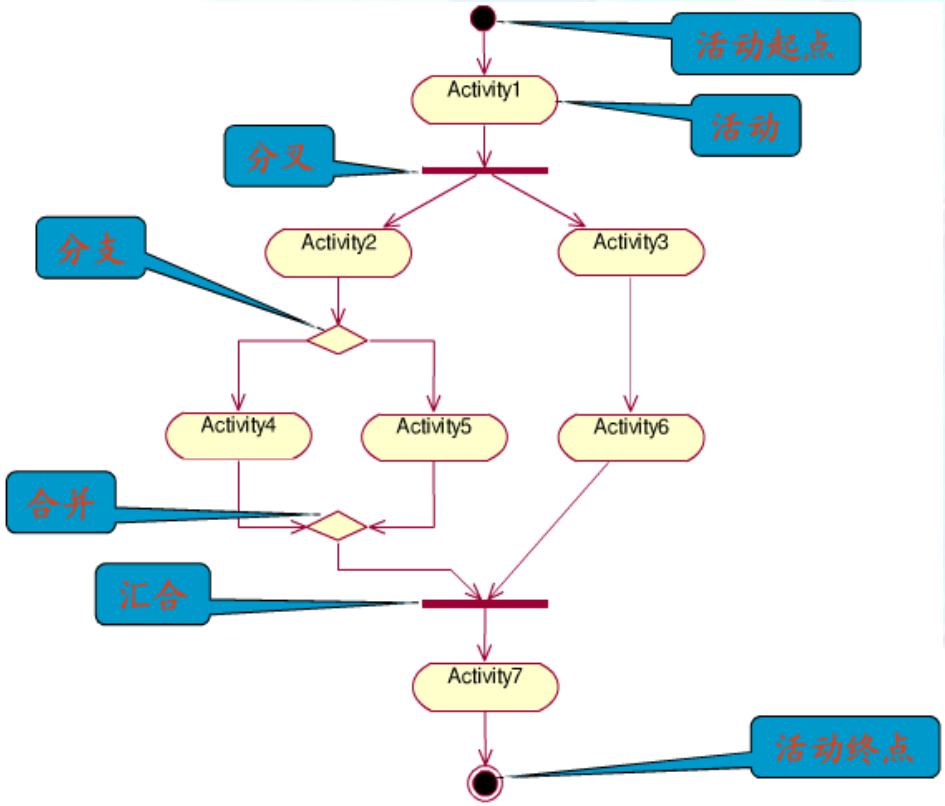
# 状态图（StatechartDiagram）

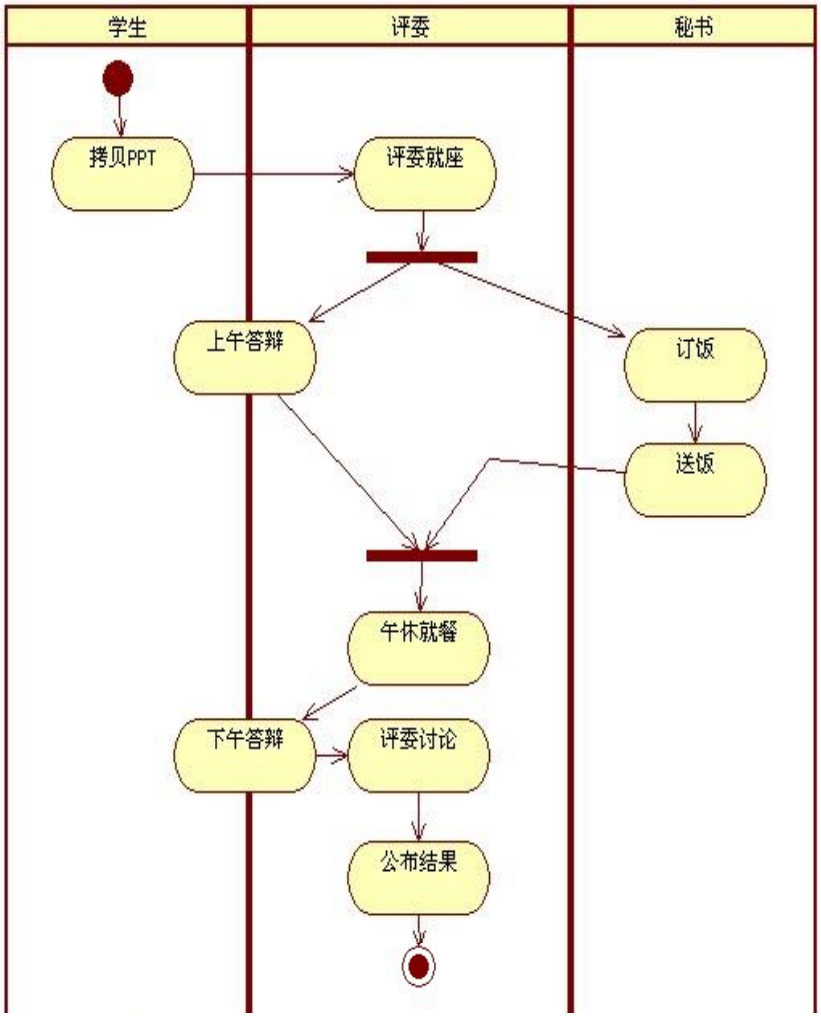
- 一种由状态、变迁、事件和活动组成的状态机，用来描述类的对象所有可能的状态以及时间发生时状态的转移条件



# 活动图（ActivityDiagram）

- 是状态图的一种特殊情况，这些状态大都处于活动状态。本质是一种流程图，它描述了活动到活动的控制流

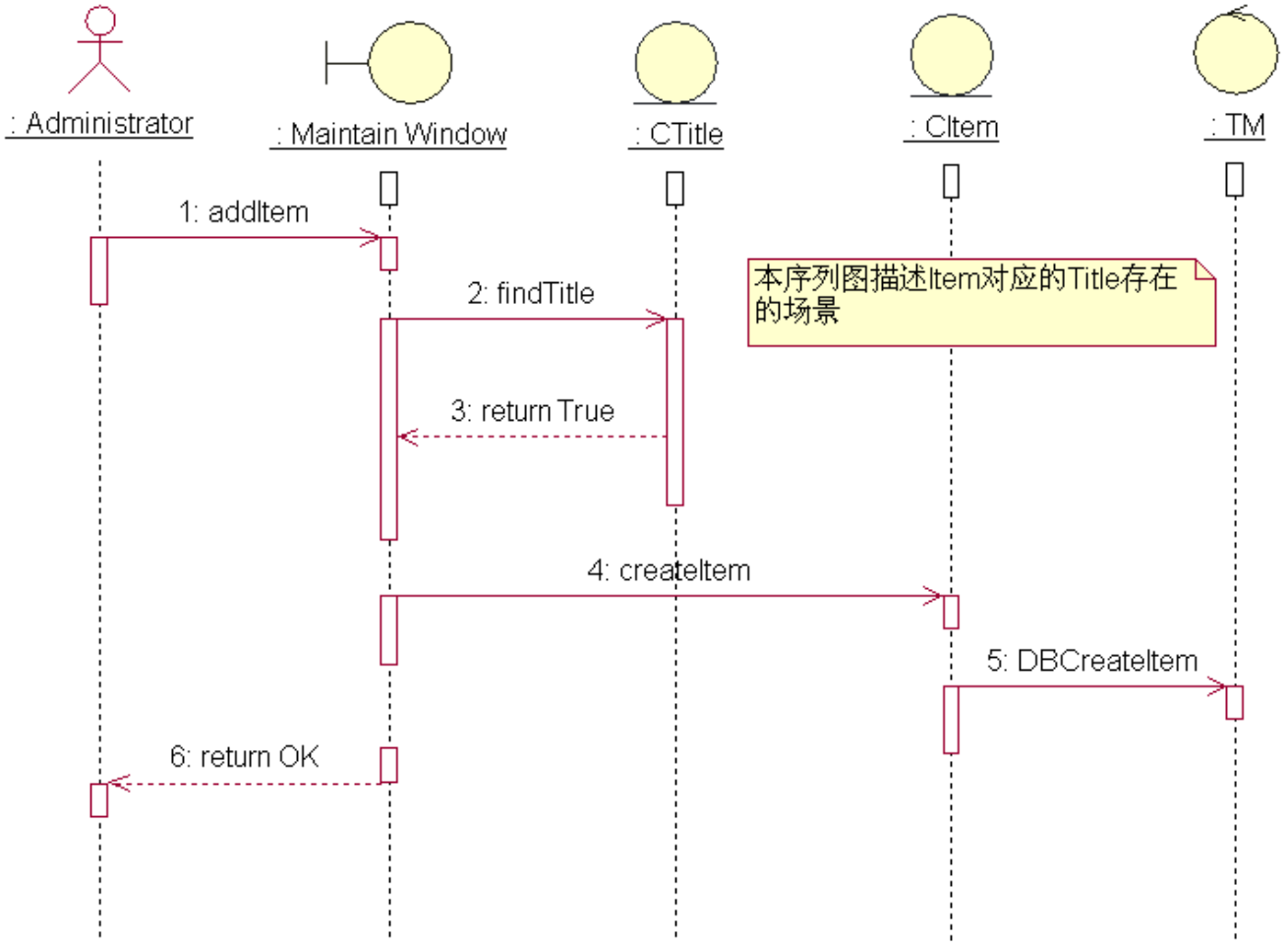


# 概述—UML统一建模语言回顾

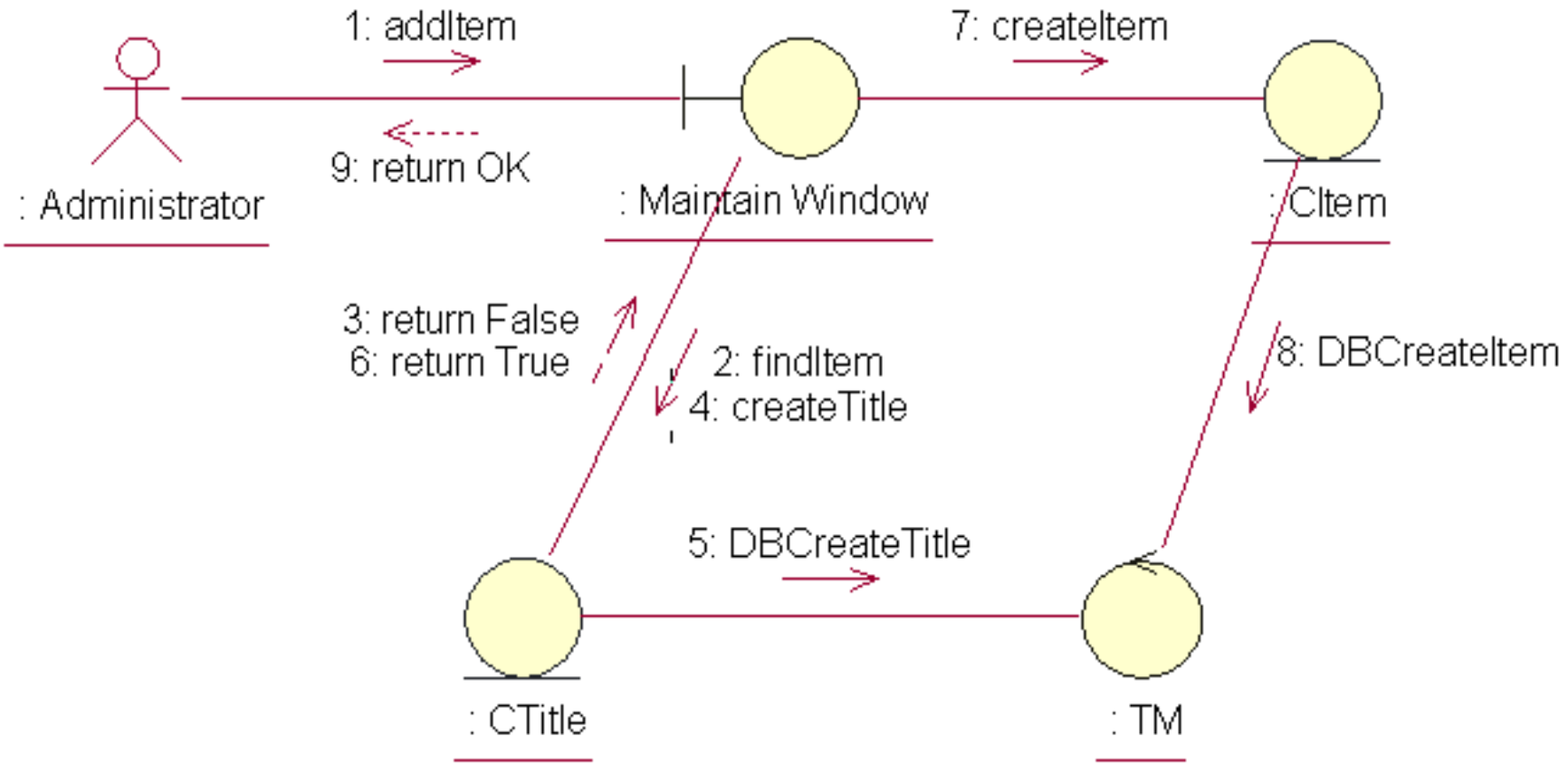
## 时序图（SequenceDiagram）

- 交互图的一种，描述了对象之间消息发送的先后顺序，强调时间顺序。
- 序列图的主要用途是把用例表达的需求，转化为进一步、更加正式层次的精细表达



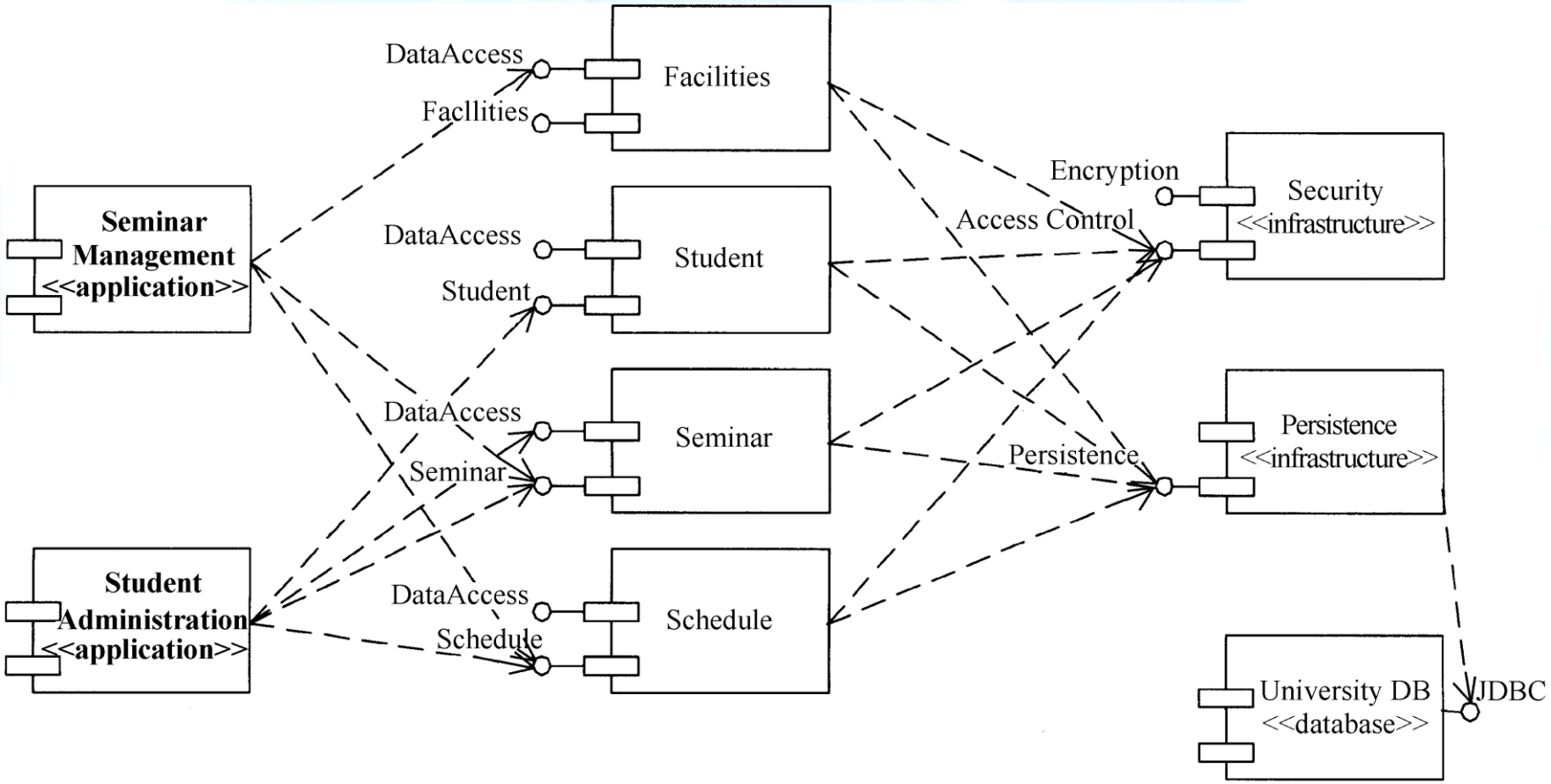
# 协作图（CollaborationDiagram）

- 交互图的一种，描述了收发消息的对象的组织关系，强调对象之间的合作关系



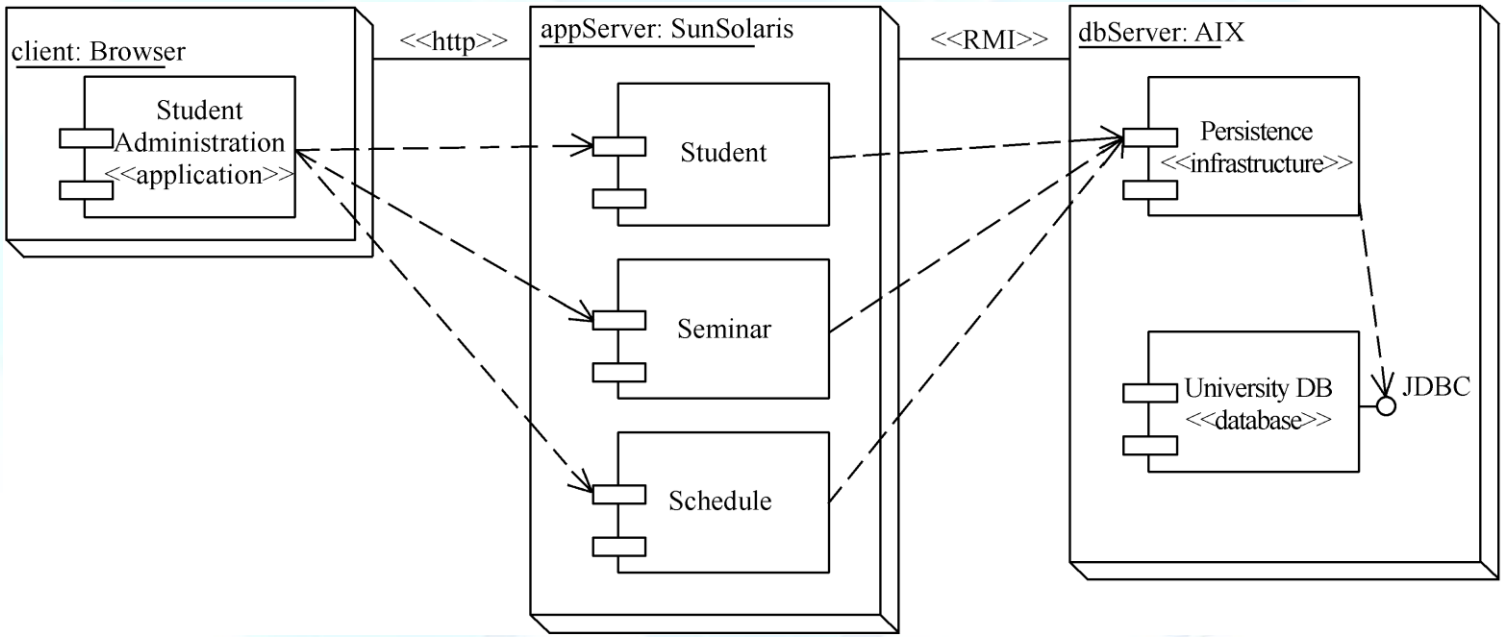
# 组件图（ComponentDiagram）

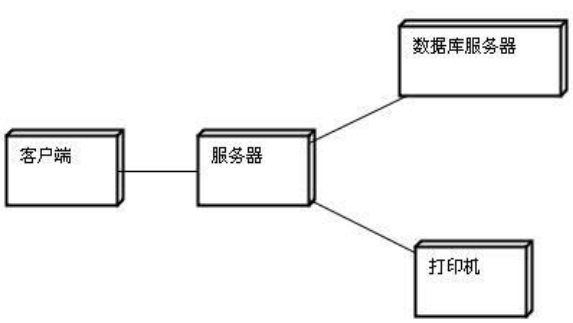
- 用来表示系统中组件与组件之间，类或接口与组件之间的关系图

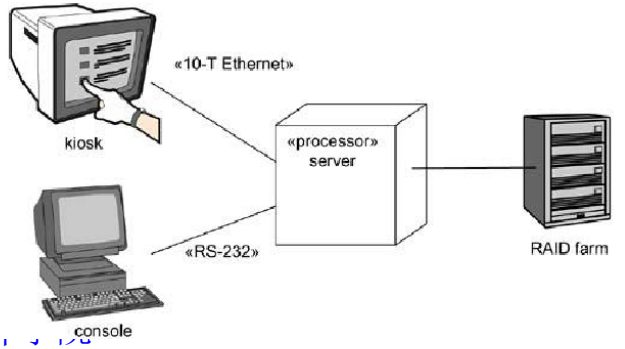


# 部署图（DeploymentDiagram）

- 描述了系统运行时进行处理的结点以及在结点上活动的构件的配置。强调了物理设备以及之间的连接关系



# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 09

# 软件需求与需求获取

## 主要内容

1. 软件需求的定义
2. 需求的分类
3. 需求获取方法
   - 3.1 收集现有书面资料
   - 3.2 面对面访谈
   - 3.3 需求研讨会
   - 3.4 现场观察/体验
   - 3.5 研究现有系统或解决方案
   - 3.6 头脑风暴

# 软件需求与需求获取

## 什么是“需求”？

我要一所大房子

有很大的落地窗户

阳光洒在地板上

也温暖了我的被子

我们晚上不睡觉

白天在床上思考

小狗在屋里奔跑

度过完美的一天

我要一所大房子

有很多很多的房间

一个房间有最快的网路

一个房间有很多的吉他

一个房间有我漂亮的衣服

一个房间住着朋友和他的爱人

一个房间一个房间

我也不知道该放些什么

# 软件需求与需求获取

## 什么是“软件需求”

- 软件需求(Software Requirements)：
  - 为用户解决问题以达到特定目标所需的能力（Capability）
  - 系统或系统构件要满足的合同、标准、规范或其他正式文档所需具备的能力（Capability）——IEEE, 1997
- 软件需求：以一种清晰、简洁、一致且无二义性的方式，描述用户对目标软件系统在功能、行为、性能、设计约束等方面的期望，是在开发过程中对未来系统的约束
- 需求通常用于表达“做什么”，而不描述“如何做”

# 关于“需求”的例子

- **Course Registration System(学生选课系统)**
  - 某大学希望采用计算机管理学生的选课
  - 学生可以在一个学期开始之前选择该学期开设的某些课程
  - 老师可以使用选课系统获得选课学生的名单，并登记学生的课程学习成绩
  - 学生不希望自己的学习成绩被他人查阅
  - ……(你可以补充吗？)
- 以下描述是否属于需求？为什么？
  - 系统通过JDBC与Oracle数据库CourseDB建立连接，并使用T-SQL语句从CourseOffering数据表中获得课程的开设信息

# 软件需求与需求获取

## [案例分析1] “只有结婚后才可以修改姓名吗？”

- Phil开发了一套人力资源软件，有一天他接到了人力资源部Maria打来的电话…

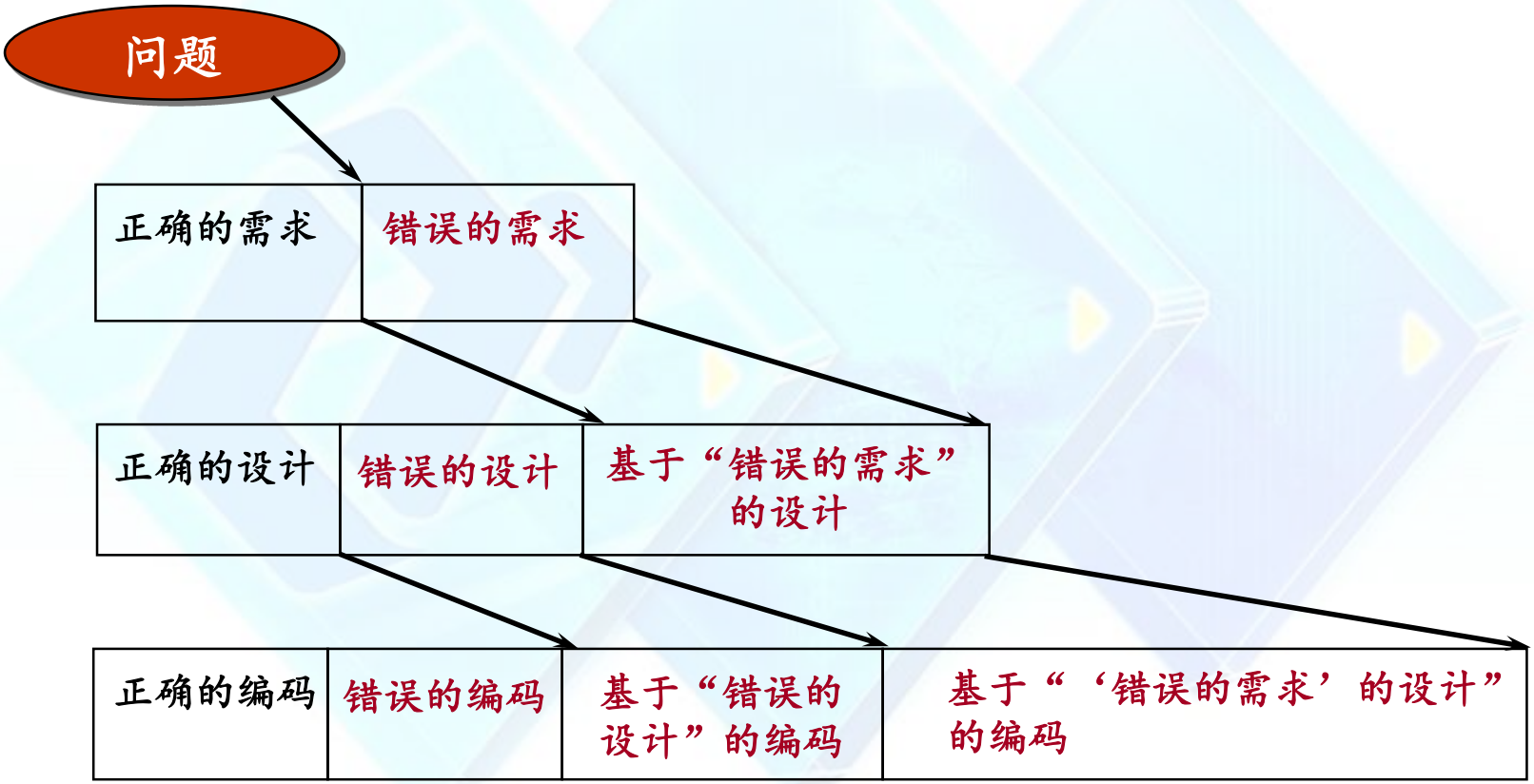
| | |
|---|---|
| Maria | 一个同事想把自己名字改为Sparkle Starlight，但系统不允许，能帮忙吗？ |
| Phil | 她嫁给了一个姓Starlight的人吗？ |
| Maria | 不，她并没有结婚，她只是想改名字而已。 |
| Phil | 系统只支持在改变婚姻状况时才可以改名字。 |
| Maria | 可是每个人只要愿意就可以随时改变自己的名字啊。 |
| Phil | 这并不是我的错！在开发系统之前，你从来没有向我提起过有这种需求！ |
| Maria | 不管如何，请尽快把这个功能修改完毕，否则Sparkle将无法支付她的银行帐单。 |
| Phil | 如果你一开始就告诉我你想随时改变某人的名字，那这些就都不会发生！ |

……

**用户需求的准确性和详尽程度极其重要！**

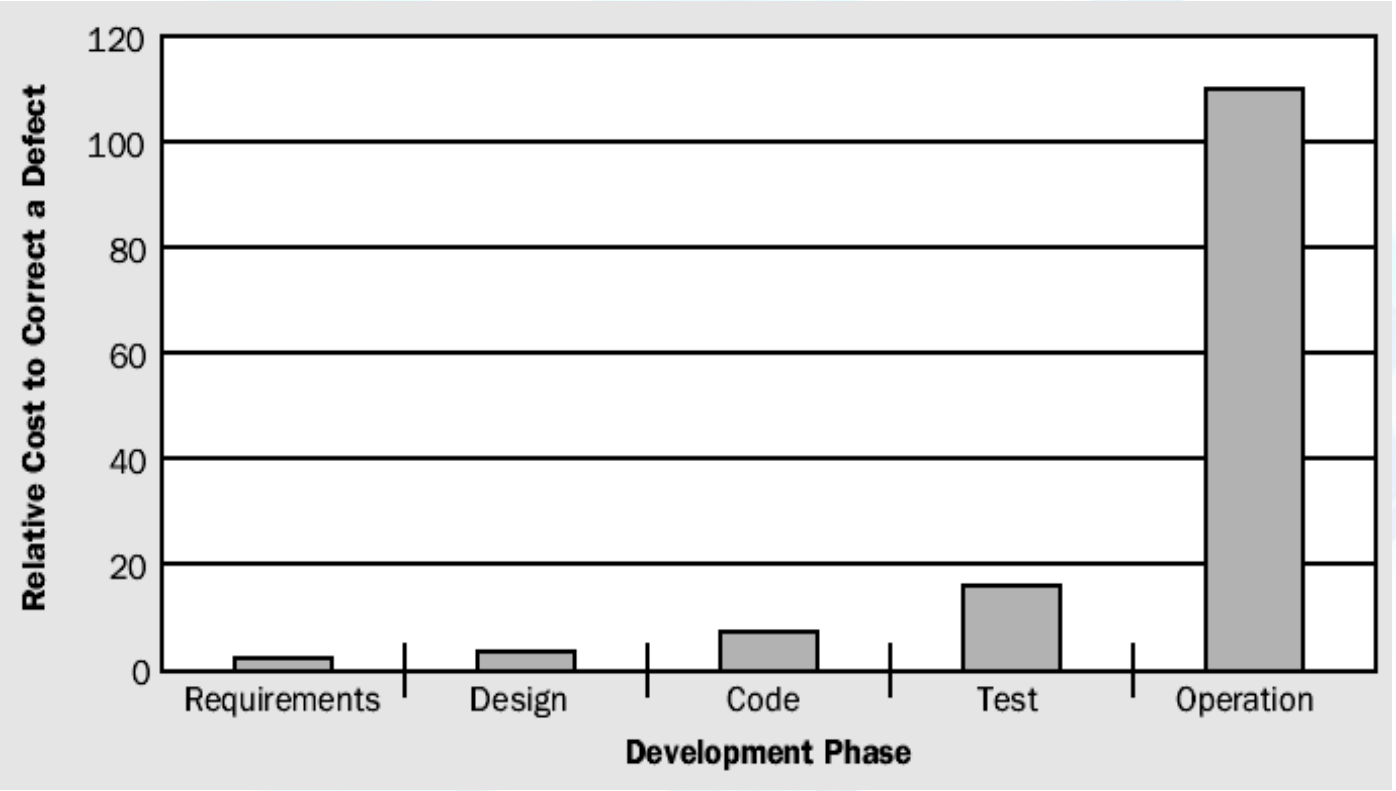
# 软件需求与需求获取

## “错误的需求”的扩散效应



# 软件需求与需求获取

## “错误的需求”的修复代价




“构建一个软件系统最困难的部分是确定构建什么… 在出错之后会严重影响随后实现的系统，并且在以后的修补是如此的困难…”

# 软件需求与需求获取

## 需求分类

- 业务需求（愿景需求）
- 用户需求
- 功能需求
- 非功能需求
- 业务规则
- 数据定义
- 约束条件
- 外部接口需求

# 软件需求与需求获取

## 需求分类：业务需求（愿景需求）

- **业务需求(Business Requirements)**：客户对于系统的**高层次目标要求(high-level objectives)**，定义了项目的**远景和范畴(vision and scope)**
  - 业务：属于哪类业务范畴？应完成什么功能？为何目的？
  - 客户：软件为谁服务？目标客户是谁？
  - 特性：区别于其他竞争产品的特性是什么？
  - 价值：商业价值体现在哪些方面？
  - 优先级：功能特性的优先级次序是什么？
- [例]“图书资料管理系统”的业务需求
  - 使用计算机实现图书资料的日常管理，提高工作效率和服务质量
  - 可让用户在网络上查询与浏览电子资料，改变原有借阅模式
  - 市场股票价格上升 X%
  - 每年节约 $Y
  - 替代了维护费用高的老一代系统 Z

# 软件需求与需求获取

## 需求分类：用户需求

- **用户需求(User Requirements)**：从用户角度描述的**系统功能需求与非功能需求**，通常只涉及系统的**外部行为**而不涉及内部特性

- [例]用户可以通过Internet随时查询图书信息和个人借阅情况，并可以快速查找和浏览需要的电子资料：
  - [功能需求]用户通过Internet查询图书信息
  - [功能需求]用户通过Internet浏览个人借阅情况
  - [功能需求]用户通过Internet查找和浏览电子资料
  - [非功能需求]随时、快速

# 需求分类：功能需求

- **功能需求(Functional Requirements, FR)**：系统应该提供的**功能或服务**，通常涉及用户或外部系统与该系统之间的交互，不考虑系统内部的实现细节
- 客户所说的诸如“用户应该能<执行某些功能>”或者“系统应该<具备某些行为>”，这是最可能的功能需求
- 功能需求描述了系统所展示的可观察的行为，并且大多数是处于执行者－系统响应顺序的环境中
- [例]
  - 用户可从图书资料库中查询或者选择其中一个子集
  - 系统可提供适当的浏览器供用户阅读馆藏文献
  - 用户每次借阅图书应对应一个唯一的标识号，它被记录到用户的账户上

# 软件需求与需求获取

## 需求分类：非功能需求

- **非功能需求(Non-Functional Requirements, NFR)**：从各个角度对系统的约束和限制，反映了客户对软件系统**质量和性能(quality and performance)**的额外要求，包括安全性、可靠性、互操作性、健壮性、易使用性、可维护性、可移植性、可重用性、可扩充性等

- [例]
  - 系统在20秒内响应所有的请求
  - 系统应该每周7天、每天24小时都可使用
  - 对一个没有经验的用户而言，经过2小时培训即可使用系统的所有功能
- 注意：非功能需求隐含了对可选设计方案的一些关键影响
  - 体系结构设计(e.g., 体系结构风格选择)
  - 算法设计(e.g., 排序策略的选择)

# 软件需求与需求获取

## 需求分类：业务规则

- **业务规则(Business Rule)**：对某些功能的可执行性或**内部执行逻辑**的一些限定条件
  - 通常表达为“如果…，那么…”的形式
  - 通常是一些容易发生变化的功能
- 例如：
  - 如果借书卡类型为“教师”，那么一次借阅的最大数量为8本
  - 如果订单金额大于10000元，那么该订单的折扣为10%
  - 如果采购单金额在10万到50万之间，那么需要总经理审批
  - 如果开具了药品A，并且病人年龄在65岁以上或12岁以下，其总剂量不能超过20片
  - 如果开具了药品B，那么不能同时开具药品C或D
  - 如果开具了检查项目M和N，那么就无需开具检查项目P
  - 同一处方中不能开具5种以上的药物或3项以上的检查

# 软件需求与需求获取

## 需求分类：数据定义

- **数据定义：**当客户描述一个数据项或一个复杂的业务数据结构的格式、允许值或缺省值时，他们正在进行数据定义。例如：
  - “邮政编码由6个数字组成，表示省、市、区...
  - 身份证18位数字和X构成，表示省、市、县（区）、出生日期、编号+验证码
- 收集用户当前正在使用的各类数据表格、单据等，基于它们来获得对数据的详细定义

# 软件需求与需求获取

## 需求分类：约束条件

- **约束条件(Constraints)**：系统设计和实现时必须满足的**限制条件**，对其进行权衡或调整是相当困难的，甚至是不可能的

- 下面是客户描述限制的一些习惯用语：
  - 必须使用<一个特定的数据库产品或语言>
  - 不能申请多于<一定数量的内存>
  - 操作必须与<其它系统>相同
  - 必须与<其它应用程序>一致

- 例如：
  - 系统必须用C++或其他面向对象语言编写
  - 系统用户接口需要采用图形化界面
  - 一个特定应用所消耗的可用计算能力平均不超过50%
  - 系统开发过程和交付文档需遵循GB/T 8567-2006标准
  - 通讯接口必须符合ISO七层架构

- 来源：法规政策、硬件/资源限制、开发语言等

# 需求分类：外部接口需求

- **外部接口需求(External Interface Requirement)**：描述系统与其所处的**外部环境之间如何进行交互**，包括：
  - 用户接口需求(UI)
  - 硬件接口需求
  - 软件接口需求
  - 通信接口需求
- 例如：
  - “从<某些设备>读取信号”
  - “给<其它系统>发送消息”
  - “以<某种格式>读取文件”
  - “能控制<某些硬件>”
  - “采用<某种类型的>用户界面”

# 需求分类：约束条件

- **约束条件：**系统设计和实现时必须满足的限制条件，它们是一种类型的非功能需求，必须把这些需求写入软件需求规格说明
- 下面是客户描述限制的一些习惯用语：
  - “必须使用<一个特定的数据库产品或语言>”
  - “不能申请多于<一定数量的内存>”
  - “操作必须与<其它系统>相同”
  - “必须与<其它应用程序>一致”
- 例如：
  - 系统必须用C++或其他面向对象语言编写
  - 系统用户接口需要采用图形化界面
  - 一个特定应用所消耗的可用计算能力平均不超过50%
  - 系统开发过程和交付文档需遵循GB/T 8567-2006标准
  - 通讯接口必须符合ISO七层架构

# 一个例子：拼写检查器

- 业务需求：
  - 用户能有效地纠正文档中的拼写错误
- 用户需求：
  - 找出文档中的拼写错误并通过一个提供的替换项来供选择替换拼错的词
- 功能性需求：
  - 找到拼写错误的单词并以高亮度提示
  - 显示提供替换词的对话框
  - 实现整个文档范围的替换
- 非功能性需求：
  - 正确的找到至少95%以上的错词并100%的加以正确替换
  - 拼写检查的速度应至少达到5000词/秒

# 关于需求的一些例子

- 系统必须有能力支持100个以上的并发用户，每个用户可以处理操作任务的任选组合，平均响应时间应该小于1秒，最大响应时间应小于5秒
- 必须在对话窗口的中间显示错误警告，使用红色的、 14点加粗Arial字体
- 系统必须有能力存储平均操作连续100天所产生的事务
- 系统应该在5分钟内计算出给定季度的总销售税
- 系统应该在1分钟内从1000000条记录中检索出一个销售订单
- 系统必须支持100个Windows工作站的并行访问
- 系统可从各型号的modem上读取信号作为系统输入

# 软件需求与需求获取

## 好的需求应具备的特征

- **完整性：** 每一项需求都必须将所要实现的功能描述清楚
- **正确性：** 每一项需求都必须准确地陈述其要开发的功能
- **可行性：** 每一项需求都必须是在已知系统和环境的权能和限制范围内可以实施的
- **必要性：** 每一项需求都应把客户真正所需要的和最终系统所需遵从的标准记录下来
- **划分优先级：** 给每项需求、特性或使用实例分配一个实施优先级以指明它在特定产品中所占的分量
- **无二义性：** 对所有需求说明的读者都只能有一个明确统一的解释
- **可验证性：** 检查一下每项需求是否能通过设计测试用例或其它的验证方法，如用演示、检测等来确定产品是否确实按需求实现

# 产生不合格需求的原因

- **无足够用户参与**
  - “我不明白为什么要花那么多功夫收集需求”
  - “与其与用户讨论浪费时间，不如写代码有意思”
  - “我已经明白用户需求了”
- **用户需求的不断增加**
  - 若不断增加新需求，项目就越来越庞大以致超过其计划及预算范围
  - 开发中不断延续的变更会使其整体结构日渐紊乱，补丁代码也使得整个程序难以理解和维护

# 产生不合格需求的原因

- **模棱两可的需求**
  - 诸多读者对需求说明产生了不同的理解
  - 单个读者能用不止一个方式来解释某个需求说明
  - 后果：返工，重做一些你认为已做好的事情
- **不必要的特性**
  - “画蛇添足”，开发人员力图增加一些“用户欣赏”但需求规格说明中并未涉及的新功能
  - 客户可能要求一些看上去很“酷”，但缺乏实用价值的功能，而实现这些功能只能徒耗时间和成本

# 产生不合格需求的原因

- **过于精简的规格说明**
  - 给开发人员带来挫折，使他们在不正确的假设前提和极其有限的指导下工作
  - 给客户带来烦恼，他们无法得到他们所设想的产品
- **忽略了用户分类**
  - 软件由不同的人使用其不同的特性
  - 使用频繁程度有所差异
  - 使用者受教育程度和经验水平也不尽相同
- **不准确的计划**
  - 对需求分析缺乏理解会导致过分乐观的估计
  - 原因：频繁的需求变更、遗漏的需求、与用户交流不够、质量低下的需求规格说明和不完善的需求分析

# 软件需求与需求获取

## [案例分析2]“他们忙，没有时间与你讨论需求…”

- Contoso公司的CEO Gerhard约见软件开发小组Cynthia，商讨为公司开发新系统的事情…

| | |
|---|---|
| Gerhard | 我们需要建立一套化学制品跟踪信息系统，可以记录并查询库房或某个实验室中已有的化学药品… 你们小组能在五个月内开发出该系统吗? |
| Cynthia | 我已经明白这个项目的重要性了，但在我制定计划前，我们必须收集一些系统的需求。 |
| Gerhard | 你什么意思？我不是刚告诉你我的需求了吗? |
| Cynthia | 你只说明了整个项目的概念与目标，这些高层次的业务需求并不能为我们提供足够的详细信息以确定究竟要开发什么样的软件，以及需要多长时间。我需要一些分析人员与一些知道系统使用要求的化学专家进行讨论，然后才能真正明白达到业务目标所需的各种功能和用户的要求。 |
| Gerhard | 那些化学专家都非常忙，没有时间与你们详细讨论各种细节，你不能让你的手下的人说明要做的系统吗? |
| Cynthia | 如果我们只是凭空猜想用户要求，结果不会令人满意。 |
| Gerhard | 行了，行了，我们没有那么多时间，我来告诉你需求，请马上开始开发系统，并随时将你们的进展情况告诉我。 |

……

# 好需求与坏需求

- 考虑以下需求是否满足“好需求”的标准，如不是，该如何修正？

1. 在用户每次存钱时系统将进行信用检查
2. 如果用户试图透支，系统将采取适当的行动
3. 系统将尽可能快的响应所有有效的请求
4. 系统允许立即使用所存资金
5. 只有在手工验证所存资金后，系统才能允许使用它

1. 在现实情况中，用户存钱时并不需要信用检查，因此这个需求描述是错误的
2. “适当的行动”对不同的人来说有不同的解释，显然是歧义的

   改正：如果用户试图透支，系统将显示错误信息并拒绝取款操作
3. “尽快”是不可验证的，应该给出具体数量值

   改正：系统将在20秒内响应所有有效的请求
4. 与5是矛盾的

# 软件需求与需求获取

## (1) 需求获取、(2)需求分析

- **需求获取(Requirement Elicitation)：**
  通过与用户的交流，对现有系统的观察及对任务进行分析，从而开发、捕获和修订用户的需求
  - 对用户进行分类
  - 聆听每一类用户的需求
  - 分析和整理所获取的需求
  - 形成文档化的描述
  - 签字确认

- **需求分析(Requirement Analysis)：**
  对收集到的需求进行提炼、分析和审查，为最终用户所看到的系统建立概念化的分析模型
  - 定义系统的边界
  - 建立软件原型
  - 分析需求可行性
  - 确定需求优先级
  - 建立需求分析模型
  - 创建数据字典

# (3) 需求规格说明、(4) 需求验证

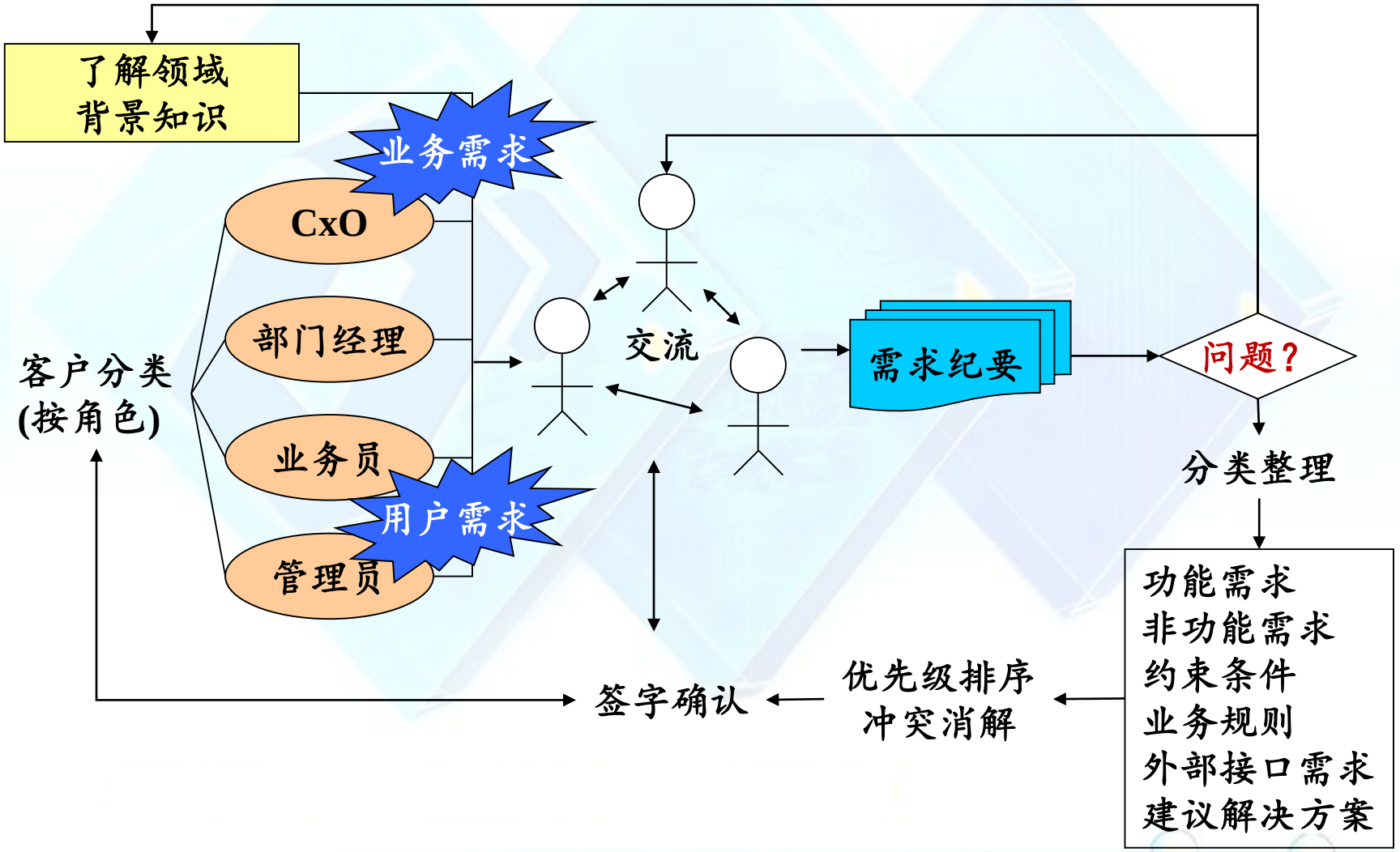
- **需求规格说明(Software Requirement Specification, SRS):**
  - 需求开发的结果
  - 精确的、形式化的阐述一个软件系统必须提供的功能、非功能、所要考虑的限制条件等
  - 作为用户和开发者之间的一个契约
  - 是用户、分析人员和设计人员之间进行理解和交流的依据

- **需求验证(Requirement Verification):**
  以需求规格说明为输入，通过评审、模拟或快速原型等途径，分析需求规格的正确性和可行性，发现存在的错误或缺陷并及时更改和补充

# (5) 需求管理

- **需求管理(Requirement Management)**
  - 定义需求基线(迅速制定需求文档的主体)
  - 评审提出的需求变更、评估每项变更的可能影响从而决定是否实施它
  - 以一种可控制的方式将需求变更融入到项目中
  - 使当前的项目计划与需求一致
  - 估计变更需求所产生影响并在此基础上协商新的承诺(约定)
  - 让每项需求都能与其对应的设计、源代码和测试用例联系起来以实现跟踪
  - 在整个项目过程中跟踪需求状态及其变更情况

# 需求获取的基本步骤


了解领域
背景知识
业务需求
CxO
部门经理
业务员
用户需求
管理员
客户分类
(按角色)
交流
需求纪要
问题？
分类整理
功能需求
非功能需求
约束条件
业务规则
外部接口需求
建议解决方案
优先级排序
冲突消解
签字确认

# 需求获取的基本步骤

- 第1步：了解相关背景和领域/行业的知识，确定未来系统相关用户类
- 第2步：与客户企业或组织的高层人员进行交流，了解实际用户任务和目标以及这些任务所支持的业务需求
- 第3步：与客户企业或组织的底层人员进行交流，获取每个用户类的详细的用户需求
- 第4步：整理需求纪要，发现新问题，并重复1-3步
- 第5步：需求分类，以区别功能需求、非功能需求、约束条件、业务规则、数据定义、外部接口需求、建议解决方法和附加信息
- 第6步：优先级排序，以及需求冲突解决
- 第7步：得到最终需求清单，并与客户做最终签字确认

# 软件需求与需求获取

“需求获取？不就是问问题吗？这有什么难的？”

- 看似简单，实际却很难 …

# 需求获取方法

- 收集现有书面资料
- 面对面访谈
- 需求研讨会
- 现场观察/体验
- 研究现有系统或解决方案
- 头脑风暴
- …… ……

# 软件需求与需求获取

## 收集现有书面资料

### 1）收集现有的报表、表格和过程描述

- “需求获取活动”的第一步
- 开始时，分析员可以请求用户提供正在使用的表格和报表的复印件，这是进一步收集需求的基础
- 通过“沟通”形式理解文档和报表
- 该过程有助于发现面谈中漏掉的业务规则

项目需求获取-现有资料研究

-- 辽河油田-环境保护信息系统

-- 大庆炼化公司-炼化建设项目后评价系统

-- 哈电集团发电设备故障诊断系统

# 收集现有书面资料

## 2）收集现有的工作流程图

- 工作流程图反映相关业务的“法定”流程
- 工作流程图一般是张贴公布的，容易获取
- 工作流程图没有严格的规范，但通俗易懂

例子1：软件学院工作流程规范及示例

例子2：物业管理服务费日常收取工作流程图

例子3：大庆炼化公司后评价工作流程图

# 收集现有书面资料

## 3）分发和收集需求调查表

- 使用场合：
  - （1）系统相关者较多
  - （2）地理上分布广
- 调查表问题类型：
  - （1）封闭问题
  - （2）定量问题
  - （3）开放问题
- 调查表在需求收集中的作用比较有限

收

# 调查表例子

第一部分：根据一个典型的4小时轮班工作情况，回答下列问题：

1.您接了多少个电话？

2.订购一件商品一般需要多少个电话？

封闭问题

…… 

第二部分：根据您同意或反对的强烈程度，选择1-7中适当的数字：

1.与顾客交谈时可用的大量产品描述对做好工作是有帮助的

选择： ◎1 ◎2 ◎3 ◎4 ◎5 ◎6 ◎7

2.计算机响应速度缓慢，从而导致响应顾客需求发生困难

选择： ◎1 ◎2 ◎3 ◎4 ◎5 ◎6 ◎7

定量问题

…… 

第三部分：请写下您的意见和建议：

请简要指出现有系统的问题，您希望在新系统中如何解决？

开放问题

# 软件需求与需求获取

## 面对面访谈

# 面对面访谈

- 需求获取中最直接的方法：用户面谈(interviewing)
- “看起来很美”，但“做起来并不容易”

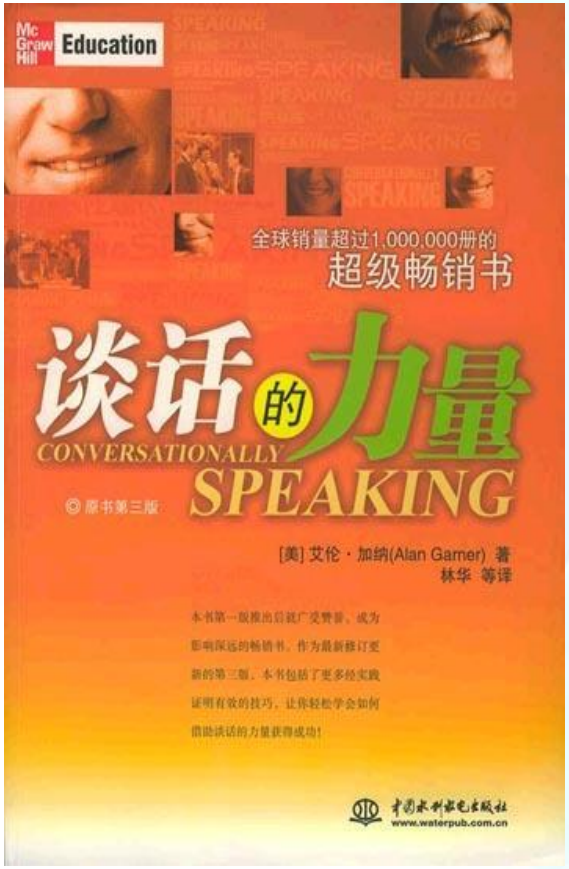
—需求分析者个人的偏见、事先的理解、以往的经验积累是导致面谈失败的最重要原因

—在面谈时，忘掉一切以往所作的事情，通过问题启发，倾听对方的陈述

—不要把自己放在“专家”的位置上

# 软件需求与需求获取

## 如何提问？




- 每个人都能提问题，但并不等于人人都会提问题 …
- 封闭式问题：对错判断或多项选择题，回答只需要一两个词
- 开放式问题：这种问题需要解释和说明，同时向对方表示你对他们说的话很感兴趣，还想了解更多的内容
- 通过提问题增强你对谈话进展和方向的控制
- 问题不能过于宽泛
- 最开始的问题不能太难
- 不能在提问之前就已经表示不赞同
- 访谈之前必须有意识的准备一些备用问题
- **Questioning skills**：一般以开放式问题开始谈话，发现问题；以选择式问题引导；以封闭式问题确认事实；以探究式问题(probing，除了....还有....吗？)发现真实想法

# 访谈问题的分类

- 上下文无关的问题(context-free questions)：充分理解用户的问题，不涉及具体的解决方案
  - 客户是谁？
  - 最终用户是谁？
  - 不同用户的需求是否不同？
  - 这种需求目前的解决方案是什么？
- 解决方案相关的问题(solution-context questions)：通过这类问题，探寻特定的解决方案并得到用户认可
  - 你希望如何解决这个问题？
  - 你觉得该问题这样解决如何？

## 面谈之前

- 确立面谈目的
- 确定要包括的相关用户
- 确定参加会议的项目小组成员
- 建立要讨论的问题和要点列表
- 复查有关文档和资料（做足功课！）
- 确立时间和地点
- 通知所有参加者有关会议的目的、时间和地点

# 软件需求与需求获取

## 面谈之中

- Step 1：事先准备一系列上下文无关的问题，并将其记录下来以便面谈时参考
- Step 2：面谈前，了解一下要面谈的客户公司的背景资料，不要选择自己能回答的问题而浪费时间
- Step 3：面谈过程中，参考事先准备的面谈模板，以保证提出的问题是正确的；将答案记录到纸面上，并指出和记录下未回答条目和未解决问题
- Step 4：面谈之后，分析总结面谈记录
- 辅助手段：录音/录像…（切记：征求对方是否允许）

## 面谈之后

- 复查笔记的准确性、完整性和可理解性
- 把所收集的信息转化为适当的模型和文档
- 确定需要进一步澄清的问题域
- 向参加会议的每一个人发出此次面谈的minutes(会议纪要)

# 面对面访谈的优缺点分析

- 优点：
  - 人们很愿意谈论自己的工作，并且总是很喜欢接受访谈
- 缺点：
  - 大多数人都采用专业术语和“行话”，而太多的专业术语让需求工程师难以理解，往往造成很多误解
  - 有些需求对用户来说太普通了，以至于他们不自觉地认为这些需求太基本，不值得去提；但它们对需求工程师来说却不是显而易见的；这往往会造成某些需求被忽略

# 软件需求与需求获取

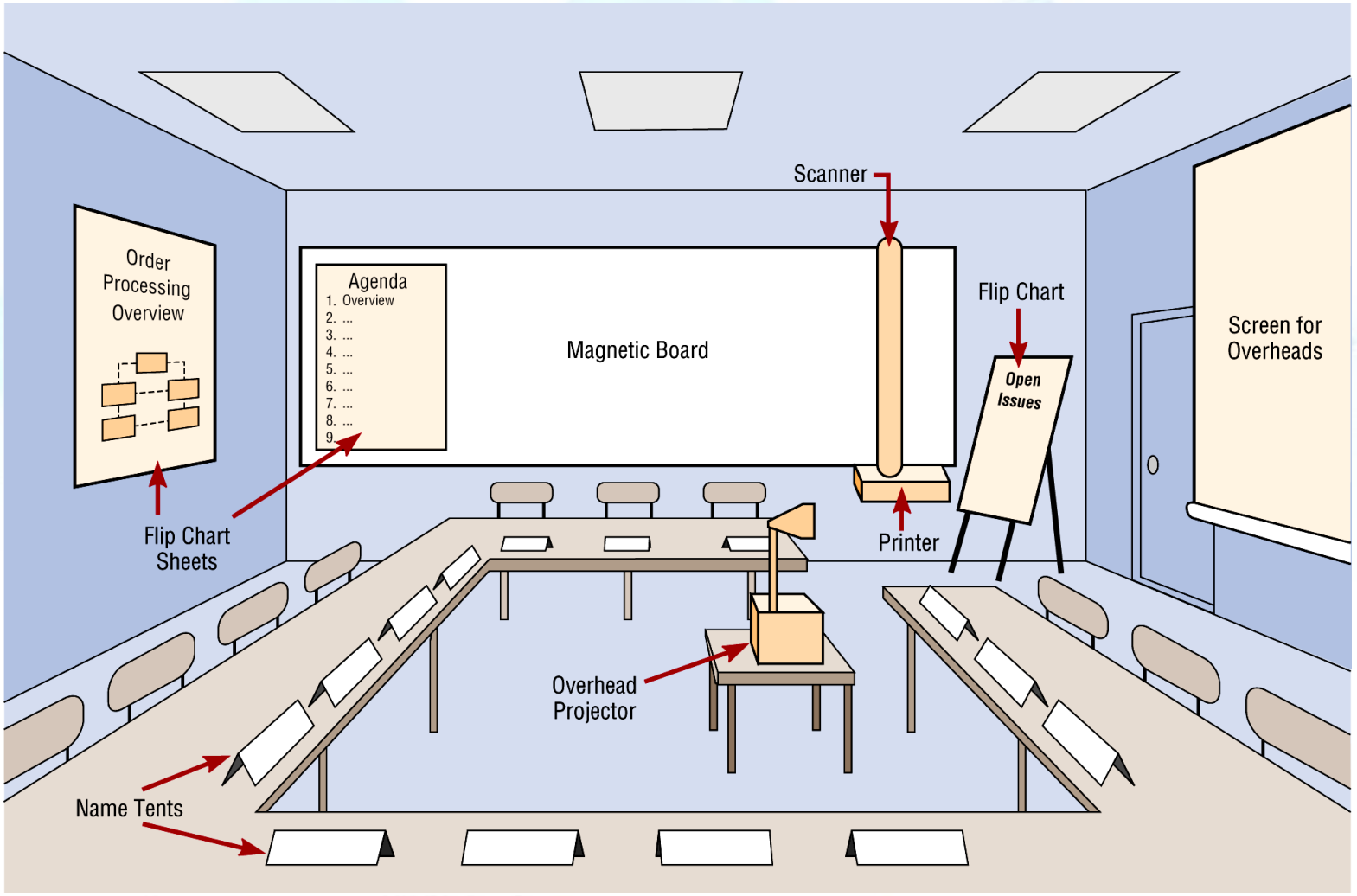
## 需求研讨会(Workshop)

《大庆炼化公司建设项目后评价管理信息系统》需求调研

# 软件需求与需求获取

## 需求研讨会(Workshop)

**JAD会议室**



# 需求研讨会(Workshop)

- 通过让所有相关人员一起参加某个单一会议来定义需求或设计系统，也称联合应用设计会议(Joint Application Design, JAD)
- 系统相关者在短暂而紧凑的时间段内集中在一起，一般为1至2天，与会者可以在应用需求上达成共识、对操作过程尽快取得统一意见
- 协助建立一支高效团队，围绕一个目的：项目的成功
- 所有人员都畅所欲言
- 促进用户与开发团队之间达成共识
- 能够揭露和解决那些妨碍项目成功的行政问题
- 最终很快产生初步的系统定义

# 软件需求与需求获取

## 需求研讨会(Workshop)

《大庆炼化公司建设项目后评价管理信息系统》JAD会议

# 需求研讨会(Workshop)

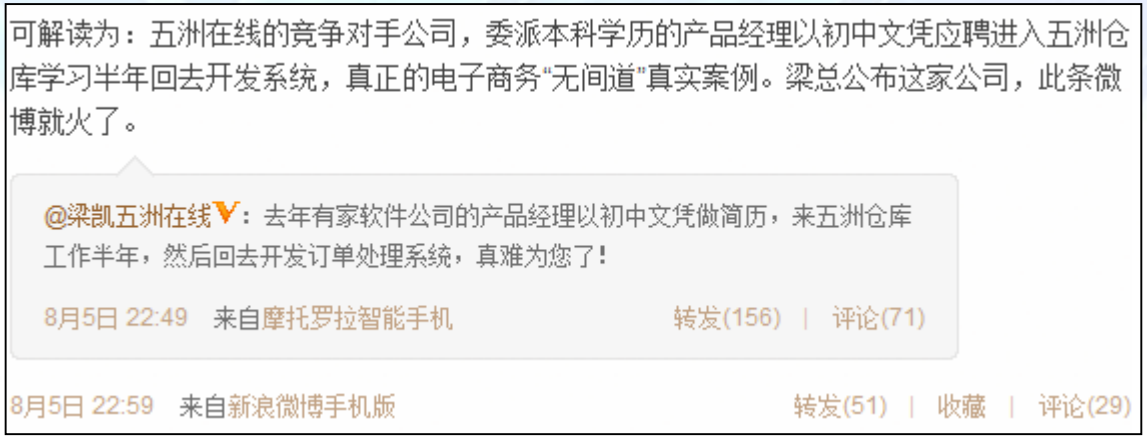
- 专题讨论会准备
  - 参加会议人员：主持人、用户、技术人员、项目组人员
  - 安排日程
  - 通常在具有相应支持设备的专用房间进行
- 举行会议
  - 可能出现人员之间的责备或冲突，主持人应掌握讨论气氛并控制会场
  - 最重要的部分是自由讨论阶段，这种技术非常符合专题讨论会的气氛，并且营造一种创造性的和积极的氛围，同时可以获得所有相关者的意见
  - 分配会议时间，记录所有言论

# 软件需求与需求获取

## 现场观察/体验

例子：观察湖南衡阳政务大厅

- 用户可能无法有效全面的表达自己的需求，通过面谈和会议也难以获得完整信息
- 在这种情况下，现场观察用户的工作流程有助于更深入全面了解需求。
- 两种方式：
  - 被动观察：用户实地工作，需求分析人员在旁边看
  - 主动体验：需求分析人员直接参与用户的实际工作



# 研究供应商的解决方案或现有系统

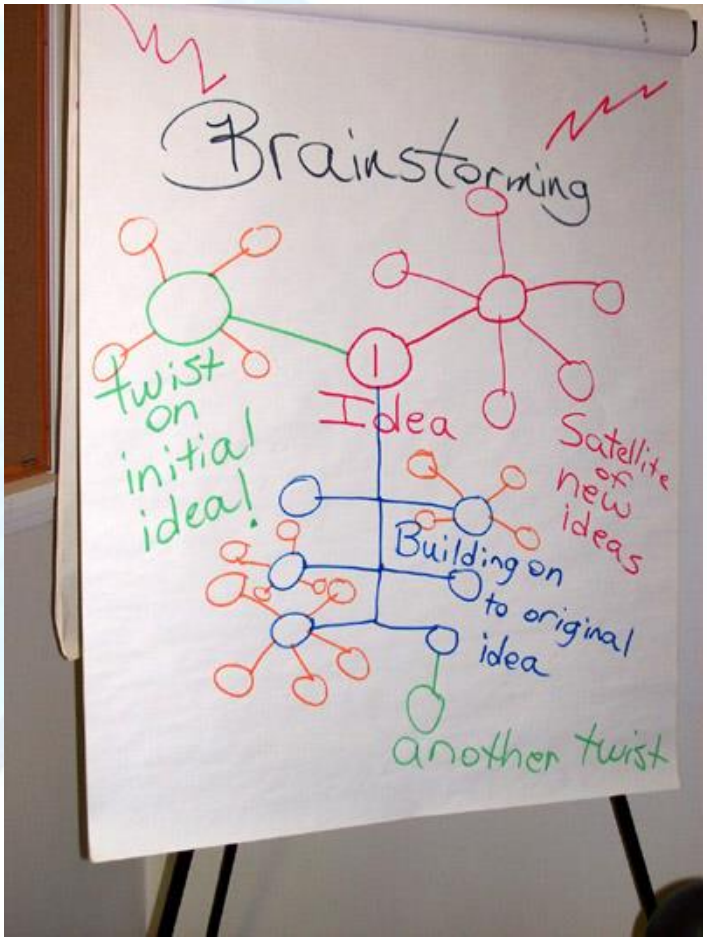
- 针对用户的业务需求，如果目前有类似的现有软件系统，则可以研究借鉴
- 针对用户的业务需求，如果目前有供应商有部分解决方案，可以参考

例子1：辽河油田环保管理系统调研-研究现有系统

例子2：大庆炼化公司后评价工作经济评价现有系统

# 软件需求与需求获取

## 头脑风暴(Brainstorming)



# 头脑风暴(Brainstorming)

- 一般以8-12人最佳：
  - 人数太少不利于交流信息和激发思维；人数太多则不容易掌握，并且每个人发言的机会相对减少
- 明确分工：1名主持人、2名记录员
- 成功要点：
  - 自由畅谈
  - 延迟批判、禁止批评
  - 禁止批评、自我批评、自谦
  - 追求数量
- 会后：修剪、分组、排序
- 适用场合：产品型系统，需要具有创新性特征，尚未投放市场，无明确的客户

# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 09

# 系统需求建模：事件和事物

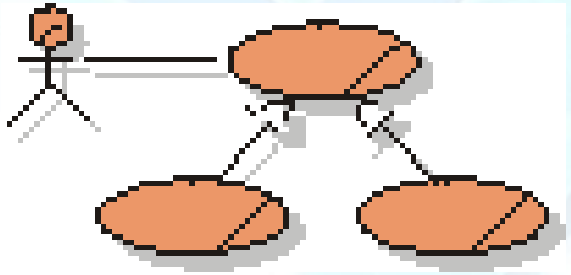
## 本章主要内容

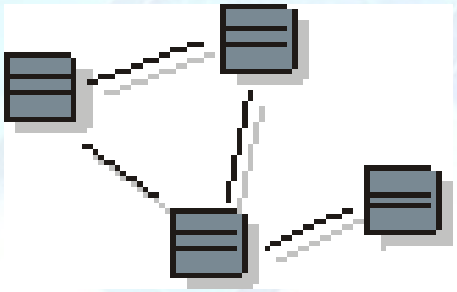
- 模型和建模
- 事件和系统需求
- 事物和系统需求
- 实体-关系图（ERD）
- 类图

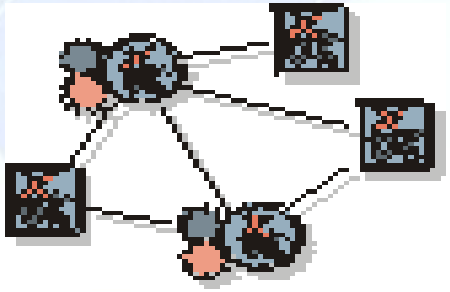
# 系统需求建模：事件和事物

## 1 模型和建模

- 分析员需要一组模型来充分描述信息系统的需求
- 不同的模型从不同的角度描述需求或解决方案

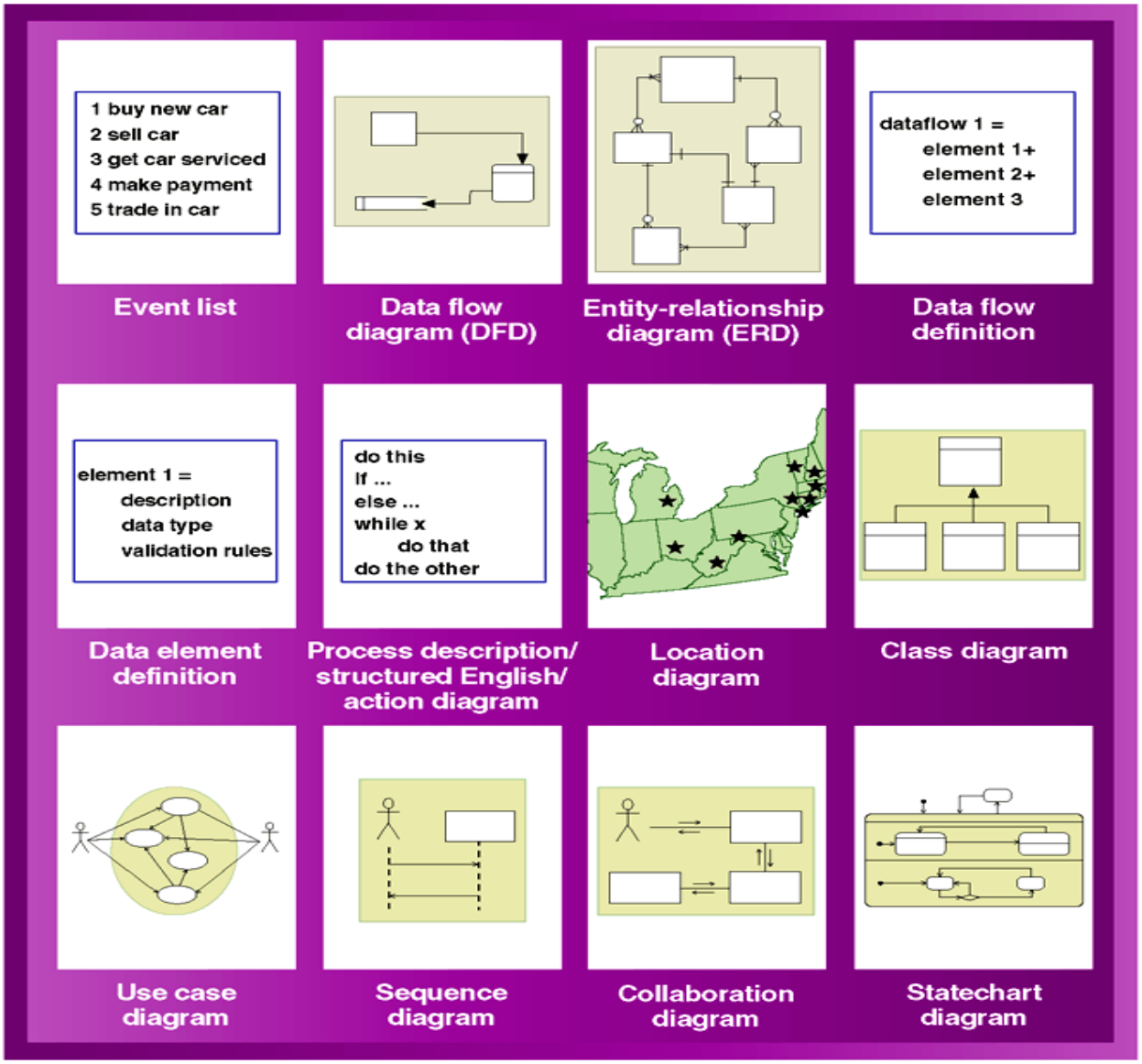






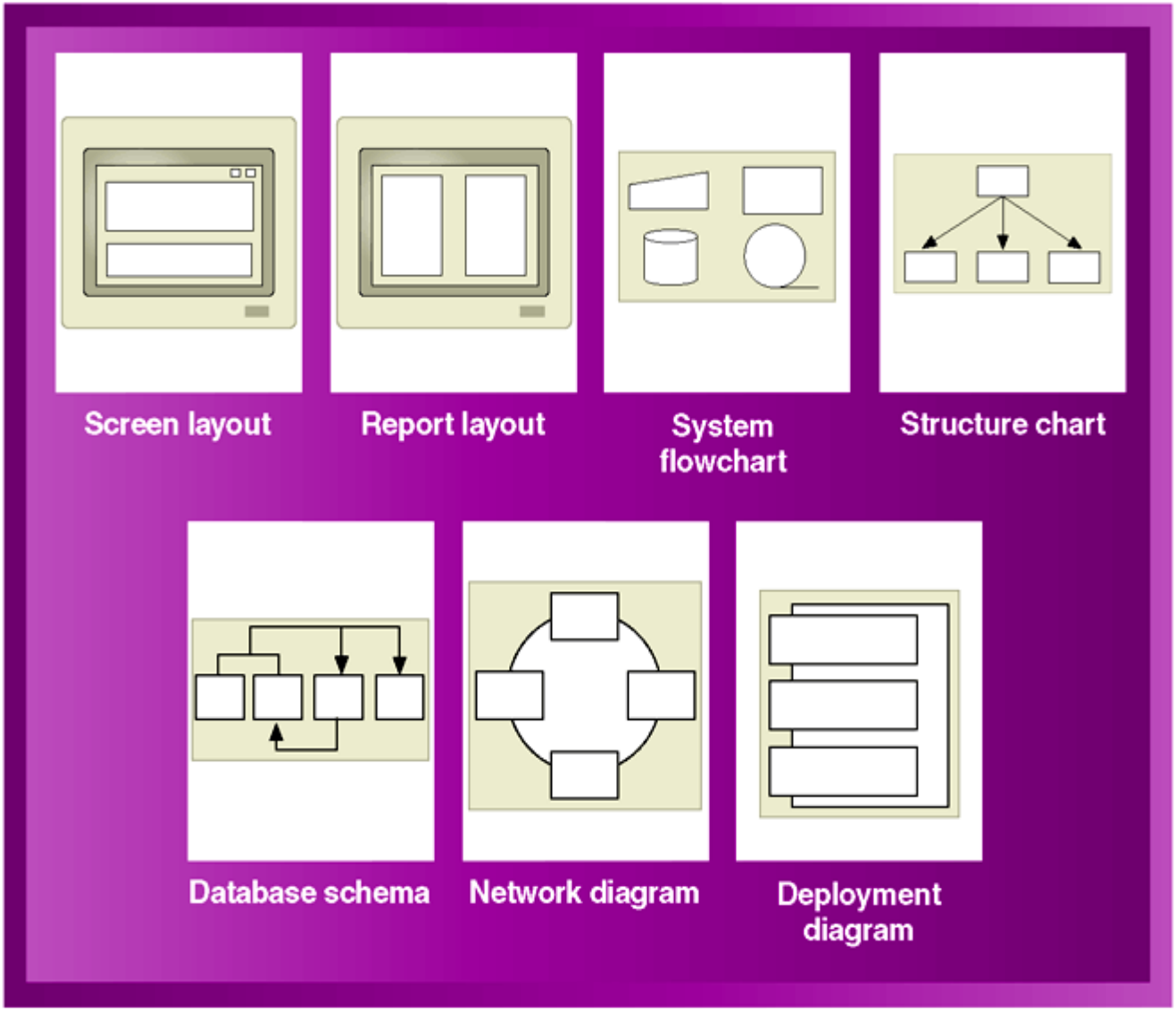
# 系统需求建模：事件和事物

## 1 模型和建模



# 系统需求建模：事件和事物

## 1 模型和建模



# 系统需求建模：事件和事物

## 1.1 模型的作用

- 建立模型的过程能帮助分析员澄清和改良设计
- 系统往往很复杂，模型能很好表示各部分之间的关系
- 模型便于项目组成员之间交流
- 模型有助于和系统用户之间进行交流并促进理解
- 建立的模型可以作为以后的维护和升级时的文档

# 系统需求建模：事件和事物

## 1.2 模型的类型

三种常见模型：数学模型、描述模型、图形模型

- 数学模型：描述系统技术方面的一系列数学公式
- 描述模型：描述系统某些方面的叙述性的备忘录、报表、列表、文字等
- 图形模型：描述系统的图表或系统某些方面的示意性表示

# 系统需求建模：事件和事物

## 1.3 用于分析和设计的模型

### 分析阶段创建的模型

- 事件列表
- 事物列表

结构化方法

- 系统流程图
- 决策树/决策表
- 实体关系图
- 数据流图
- 数据字典
- 伪代码

基于UML的OO方法

- 用例图
- 用例场景描述
- 分析类图
- 活动图
- 状态图
- 时序图/协作图

### 设计阶段创建的模型

- 体系结构图
- 界面布局图

- 系统结构图
- 程序流程图

==========

- 设计类图
- 时序图
- 包图
- 组件图
- 网络图
- 部署图

# 系统需求建模：事件和事物

## 2 事件和系统需求

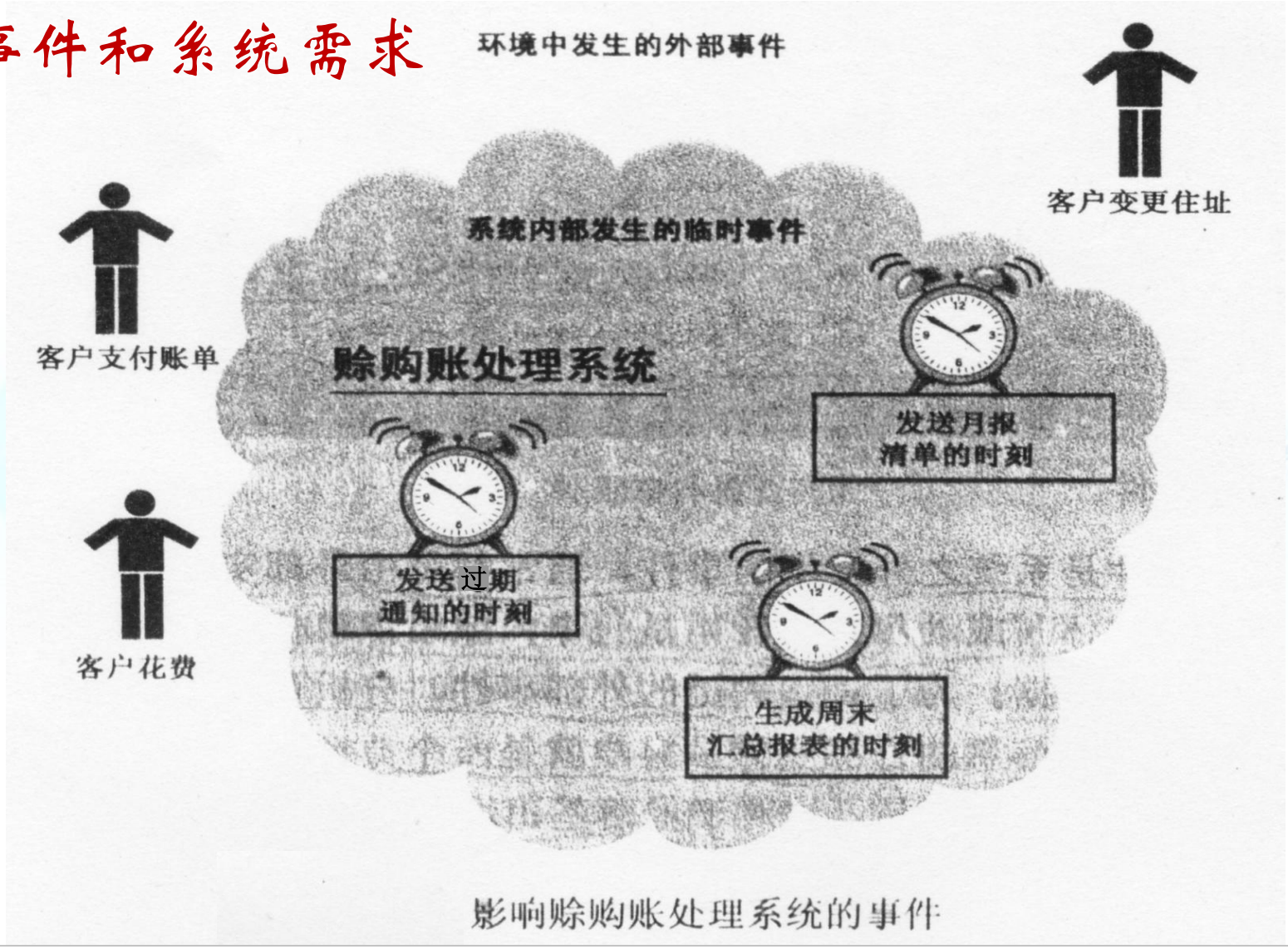
**事件：** 发生在某一特定的时间和地点、可描述并且系统应该记录下来的事情

**事件和系统需求的关系：** 事件发生时需要系统做出响应，能列出所有这样的事件就可以搞清楚用户对系统的需求

- 信息系统的所有处理过程都是由事件驱动/触发的
- 当定义系统需求时将所有事件罗列出来并加以分析是非常有意义的

# 系统需求建模：事件和事物

## 2 事件和系统需求




影响赊购账处理系统的事件

赊购账务处理系统：外部三个事件，内部三个事件

# 系统需求建模：事件和事物

## 2.1 事件的类型

事件分三大类：外部事件、临时事件、状态事件

- 外部事件：系统之外发生的事件，通常是由外部实体或系统参与者触发的
- 临时事件：由于到达某一时刻所发生的事件
- 状态事件：当系统内部发生了需要处理的情况时所引发的事件

# 系统需求建模：事件和事物

## 2.1 事件的类型

**外部事件实例：**

- 外部实体的需要触发一个事务处理，如客户下订单
- 外部实体想获得某些信息，如客户查询帐户信息
- 数据发生改变需要更新，如客户地址变更
- 管理部门想获取某些信息，如管理者需要销售情况分析报告

**临时事件实例：**

- 所需的内部输出结果，如定期报表
- 所需的外部输出结果，如定期结算单、帐单等
- 突发的情况，如系统掉电、内存溢出、读写错误等

**状态事件实例：**

- 所有外部事件和临时事件都要考虑是否有状态事件发生，如销售引起库存变化、签收引起订单状态变化等

# 系统需求建模：事件和事物

## 2.2 事件定义

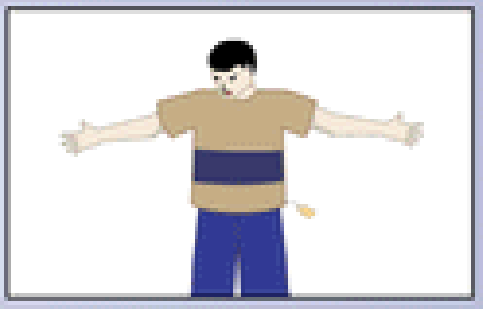
区分事件和随事件发生的一部分交互行为

客户想买一件新衬衫

客户开车来到购物中心

客户在Sears试穿衬衫

客户走进沃尔玛



客户在沃尔玛试穿衬衫

客户购买一件衬衫
（该事件直接影响系统）

导致影响系统的一个事件的一系列行为，不能都算作“事件”！

# 系统需求建模：事件和事物

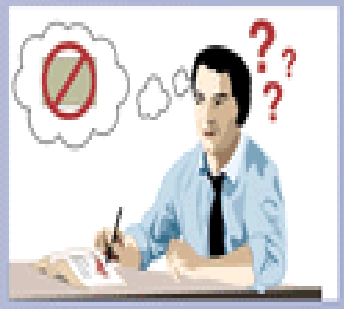
## 2.2 事件定义

跟踪事务处理生命周期可以得到一系列事件

用户想要一本商品目录

客户想确定商品信息的有效性

客户下订单



客户修改或取消订单

客户查询订单状态

客户修改帐户信息

客户退货

导致许多事件的某一特定客户的“事务”处理序列，都算作“事件”！

# 系统需求建模：事件和事物

## 2.2 事件定义

特别细节的事件在系统分析阶段不予考虑

用户登录系统

用户需要更改密码

系统要改变系统性能设置

系统崩溃，需要恢复数据库

该备份数据库了

用户该修改密码了

是“事件”，但在设计阶段之前可以忽略的事件

# 系统需求建模：事件和事物

## 2.3 系统需求建模：事件列表

事件列表属于“描述性模型”

引起系统执行某种操作所发生的事情

对外部事件而言，外部实体或参与者是数据进入系统的来源

系统产生了什么样的输出结果（如果有的话）？

| 事件 | 触发器 | 来源 | 活动/用例 | 响应 | 目的地 |
|---|---|---|---|---|---|
| 客户要确定可用商品信息 | 商品查询 | 客户 | 查询可用商品 | 可用商品信息 | 客户 |

系统如何知道事件发生了？对外部事件而言就是进入系统的数据；对临时事件来说就是定义好的、触发系统运行处理过程的时间点

当事件发生时，系统做了什么操作？

哪个外部实体获得了输出结果？

# 系统需求建模：事件和事物

## 2.3 系统需求建模：事件列表

落基山运动用品商店（RMO）客户支持系统的完整事件表

| 客户支持系统事件表 | | | | | |
|---|---|---|---|---|---|
| 事件 | 触发器 | 来源 | 活动/用例 | 响应 | 目的地 |
| 1. 客户想确定商品信息的有效性 | 商品查询 | 客户 | 查询可用商品 | 有效商品信息细节 | 客户 |
| 2. 客户发送订单 | 新订单 | 客户 | 生成新订单 | 实时连接 | 信用卡部门 |
| | | | | 订单确认 | 客户 |
| | | | | 订单细节 | 发货部门 |
| | | | | 交易处理 | 银行 |
| 3. 客户修改或取消订单 | 订单修改请求 | 客户 | 修改订单 | 修改确认 | 客户 |
| | | | | 订单修改细节 | 发货部门 |
| | | | | 事务处理 | 银行 |
| 4. 生成订单汇总报表的时刻 | 周末、月末、季度末、年末 | | 生成订单汇总报表 | 订单汇总报表 | 管理部门 |
| 5. 生成交易汇总报表 | 每天下班时 | | 生成交易汇总报表 | 交易汇总报表 | 会计 |
| 6. 客户或管理人员想查询订单状态 | 订单状态查询 | 客户或管理人员 | 查询订单状态 | 订单状态细节 | 客户或管理人员 |
| 7. 发货部门按订单发货 | 订单完成通知 | 发货部门 | 记录订单完成情况 | | |

# 系统需求建模：事件和事物

## 2.3 系统需求建模：事件列表

落基山运动用品商店（RMO）客户支持系统的完整事件表

| 客户支持系统事件表 | | | | | |
|---|---|---|---|---|---|
| 事件 | 触发器 | 来源 | 活动/用例 | 响应 | 目的地 |
| 8. 发货部确认延期订货单 | 延期订货单通知 | 发货部门 | 延期订货单通知 | 延期订货单通知 | 客户 |
| 9. 客户退货 | 订单退回通知 | 客户 | 生成订单 | 退货确认 | 客户 |
| | | | 退回记录 | 事务处理 | 银行 |
| 10. 生成完成情况汇总报表的时刻 | 周末、月末、季度末、年末 | | 生成完成情况汇总报表 | 完成情况汇总报表 | 管理部门 |
| 11. 潜在客户索取商品目录 | 索取商品目录的请求 | 潜在客户 | 提供商品目录信息 | 商品目录 | 潜在客户 |
| 12. 生成潜在的客户信息报表的时刻 | 月末 | | 生成潜在的客户活动报表 | 潜在的客户活动报表 | 市场部门 |
| 13. 客户修改账户信息 | 客户账户信息修改通知 | 客户 | 更新客户账户信息 | | |
| 14. 市场部门想给客户发送促销品 | 促销品细节 | 市场部门 | 分发促销品包 | 促销品包 | 客户和潜在客户 |
| 15. 管理人员调整对客户的收费 | 客户收费的调整 | 管理人员 | 生成客户收费调整记录 | 收费调整通知 | 客户 |
| | | | | 事务处理 | 银行 |

# 系统需求建模：事件和事物

## 3 事物和系统需求

事物：在传统的开发方法中，事物就是构成系统存储信息的相关数据

在面向对象的开发方法中，事物就是在系统中相互交互的对象

事件和事物的区别与联系：

事件 – 发生在瞬间，有一定的随机性

事物 – 客观存在，不以主观意志为转移

联系 – 事件的发生可以改变事物的状态或产生新的事物

# 系统需求建模：事件和事物

## 3 事物和系统需求

### 3.1 事物的类型

| 实物 | 角色 | 组织部门 | 设备 | 偶发事件、活动、交互 | 地点/位置 |
|---|---|---|---|---|---|
| 飞机<br>书<br>汽车<br>文档<br>工作表 | 雇员<br>顾客<br>医生<br>病人<br>最终用户<br>系统<br>管理员 | 管区<br>部门<br>工段<br>任务组<br>工作组 | 传感器<br>定时器<br>控制器<br>打印机<br>磁盘驱动器<br>键盘<br>显示器<br>窗口<br>鼠标<br>操作按钮 | 航班<br>服务电话<br>登录<br>退出<br>合同<br>购买<br>订单<br>支付 | 仓库<br>部门办公室<br>工厂<br>零售店<br>桌面 |

# 3 事物和系统需求

## 3.2 建立事物列表的过程

第一步：收集可能得到的名词

方法1：与客户谈话中获得

方法2：使用事件表及关于每一事件的信息获得

方法3：通过其他需求获取过程得到

第二步：将名词列表精简并确认

（1）确定名词是否应包含在列表中

（2）确定每个名词是否应该从列表中排除

（3）对每个名词提出问题确定是否需进一步研究

# 系统需求建模：事件和事物

## 3 事物和系统需求

RMO 系统中基于“名词”的事物的部分列表

| 确定的名词 | 将该名词作为事物来存储的一些注释 |
| --- | --- |
| 财务人员 | 我们知道都有哪些人，不需存储 |
| 延期订单 | 一个具体的订单类型，还是订单状态值？需要考虑 |
| 延期订单确认 | 从其他信息产生的输出 |
| 银行 | 仅有一个，不必存储 |
| 目录 | 需要记住，不同季节和年份的目录。应该保留 |
| 目录活动报表 | 可以从其他信息产生的输出。不必存储 |
| 目录细节 | 与目录类似吗？或者与这个目录中产品项类似？需要研究 |
| 改变请求 | 将导致对订单改变进行记录的输入 |
| 收费调整 | 导致一次交易的输入 |
| 颜色 | 关于产品项的一条信息 |
| 确认 | 从其他信息产生的一个输出。不需存储 |
| 信用卡信息 | 订单的一部分？还是客户信息的一部分？需要考虑 |
| 客户 | 是一个关键事物，带有很多细节。应该保留 |
| 客户账号 | 如果包含 RMO 支付计划，则是必要的。需要研究 |
| 完成报表 | 从运输信息产生的输出。不需存储 |
| 库存数量 | 关于产品项的一条信息。需要研究 |
| 产品项 | RMO 包含在目录中用于销售的。应该保留 |

# 系统需求建模：事件和事物

## 3 事物和系统需求

RMO 系统中基于“名词”的事物的部分列表

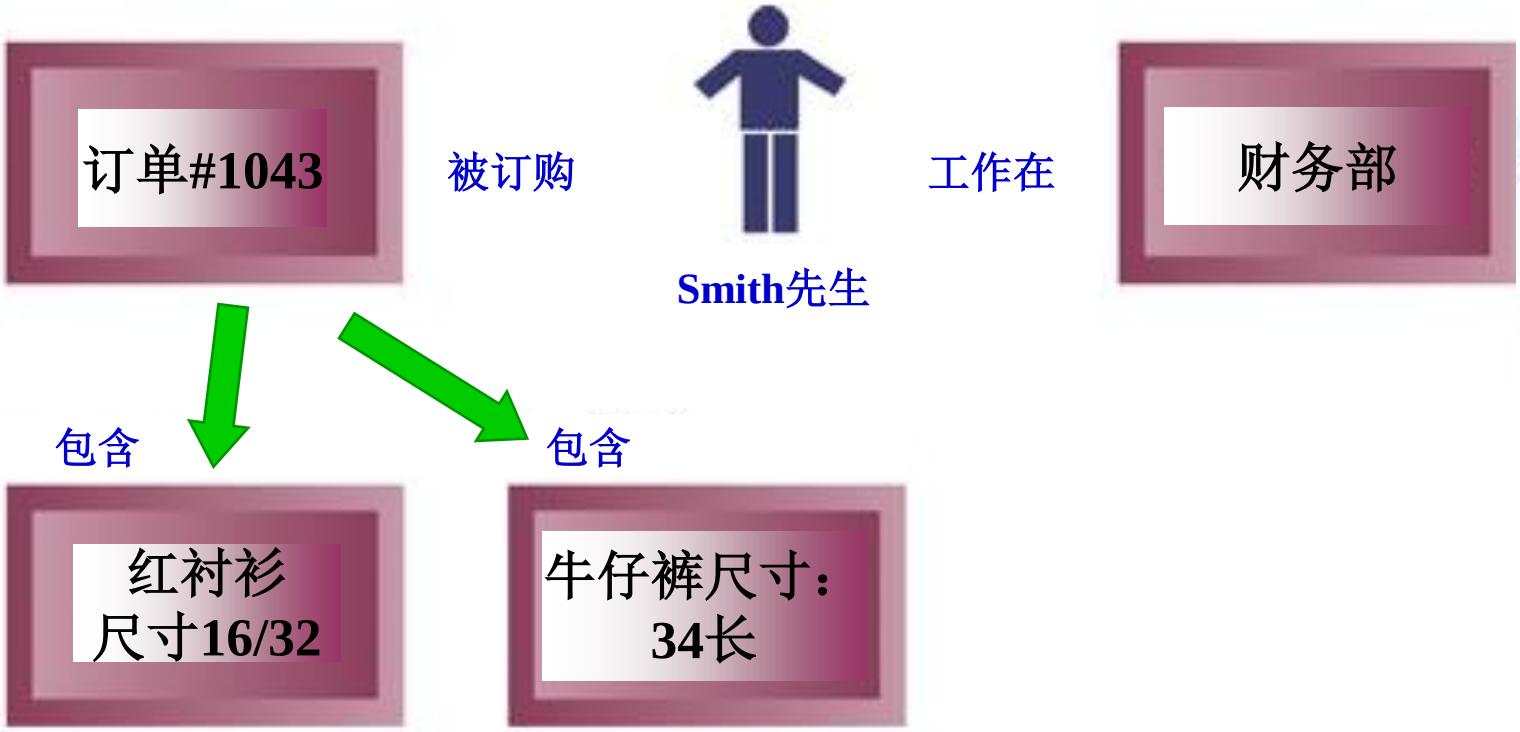
| 确定的名词 | 将该名词作为事物来存储的一些注释 |
| --- | --- |
| 管理人员 | 我们知道都有哪些人，不需存储 |
| 市场人员 | 我们知道都有哪些人，不需存储 |
| 销售人员 | 我们知道都有哪些人，不需存储 |
| 订单 | 关键的系统响应。应该保留 |
| 支付方式 | 订单的一部分。需要研究 |
| 价格 | 产品项的一部分。需要研究 |
| 促销资料 | 一个输出？还是存储在范围之外的文档？需要研究 |
| 潜在客户 | 可能与客户类似。需要研究 |
| 退货 | 订单的反面。应该保留 |
| 退货确认 | 退货信息产生的输出。不必存储 |
| RMO | 只有一个。不必存储 |
| 季节 | 目录的一部分？还是还有更多的？需要研究 |
| 运输 | 用于跟踪的关键事物。应该保留 |
| 运输人员 | 他们常有变动且我们需要跟踪订单。应该保留 |
| 运输部门 | 我们的部门，不需存储 |
| 运输部门地址 | 客户的一部分？还是订单的？还是运输的？需要研究 |

# 系统需求建模：事件和事物

## 3 事物和系统需求

### 3.3 事物间的关系

关系：某些事物间自然发生的联系，比如客户发送订单、雇员在某部门工作等




事物之间自然存在的关系

# 系统需求建模：事件和事物

## 3 事物和系统需求

### 3.3 事物间的关系

根据每件事物的关联数目来理解每种关系的本质非常重要

基数（重数）：发生在事物间关联的数目

例如：一个客户发送量多个订单

一个雇员在一个部门工作等

基数可以：0，1，0..n，1..n，m..n，0..*，*..*

二元关系：两种不同类型事物之间的关系，如Teacher与Student

一元(回归)关系：同一类型的两个事物间的关系，例如一个人和另一个人的婚姻关系

*三元关系：三种不同类型事物之间的关系，如Course、Teacher和Student

*n元关系：n（n为任意数）种不同类型事物之间的关系

# 3 事物和系统需求

## 3.4 事物的属性

属性：有关事物的一条特定信息

标示符（关键字）：能唯一标志事物的一个属性

复合属性：包括了许多相关属性的属性，如客户全名：名+姓

| 所有客户具有如下属性 | 每个客户的每个属性都有一个值 | | |
|---|---|---|---|
| 客户编号 | 101 | 102 | 103 |
| 名 | John | Mary | Bill |
| 姓 | Smith | Jones | Casper |
| 住宅电话 | 555-9182 | 423-1298 | 874-1297 |
| 公司电话 | 555-3425 | 423-3419 | 874-8546 |

# 3 事物和系统需求

## 3.5 数据实体和对象

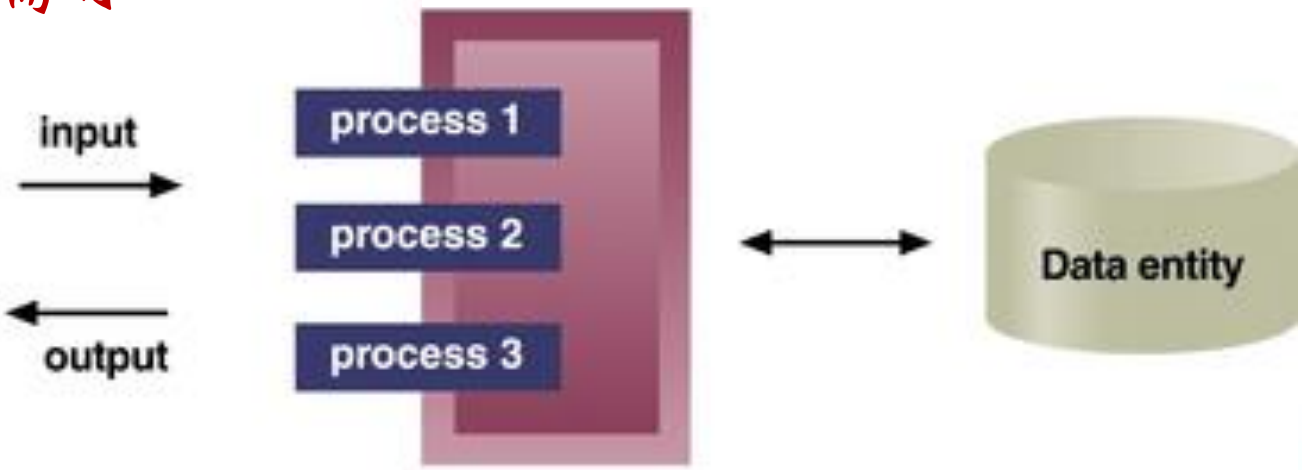
数据实体：在传统的系统开发方法中，事物被称为数据实体

对象：在面向对象的系统开发方法中，将事物称为对象

# 3 事物和系统需求

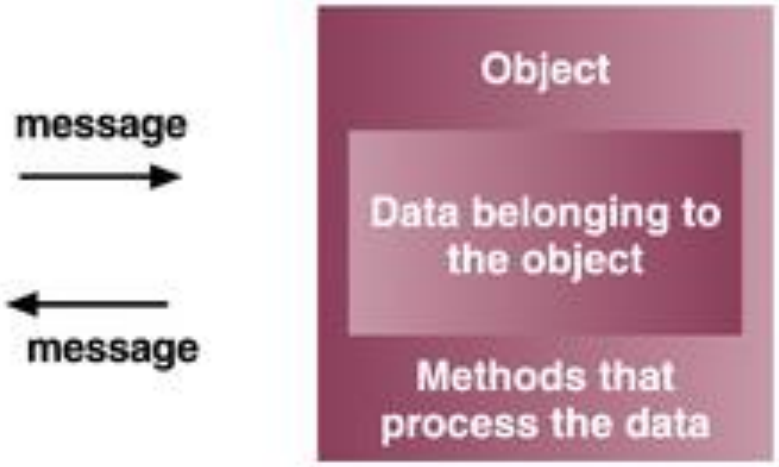
## 3.5 数据实体

数据实体与过程是分离的




对象将数据和处理数据的方法封装成一个单元

要让某个对象执行某种操作，可以让另外一个对象给该对象发送一个消息



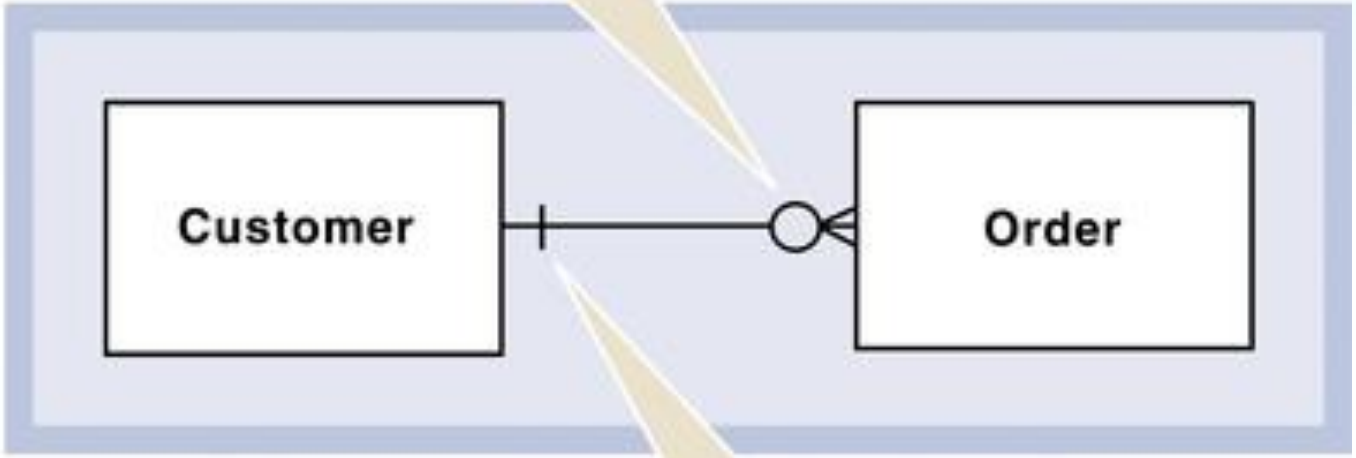
# 系统需求建模：事件和事物

## 4 实体-关系图

- 传统的系统开发方法都将重点集中在数据存储需求上
- 数据存储需求包括数据实体、数据实体的属性以及它们之间的关系

### 4.1 ERD概念的实例

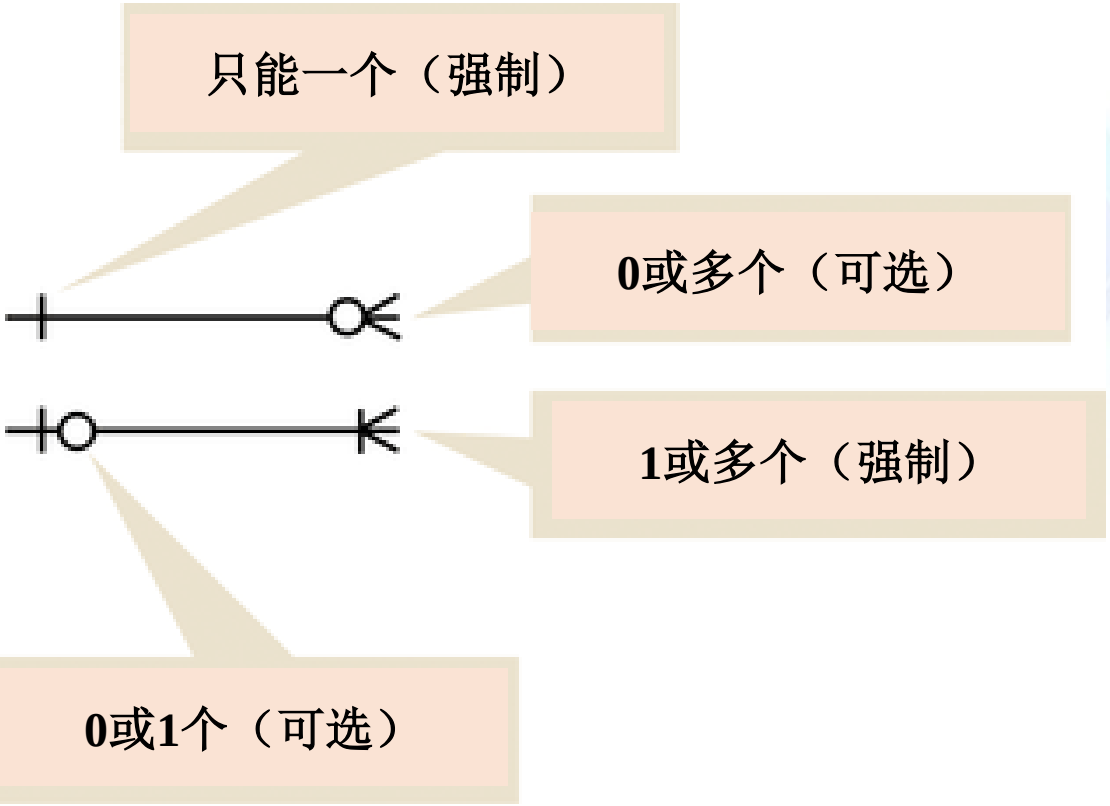
一个客户可以发送
0个或多个订单




一个订单必须对应一个客户

# 4 实体-关系图

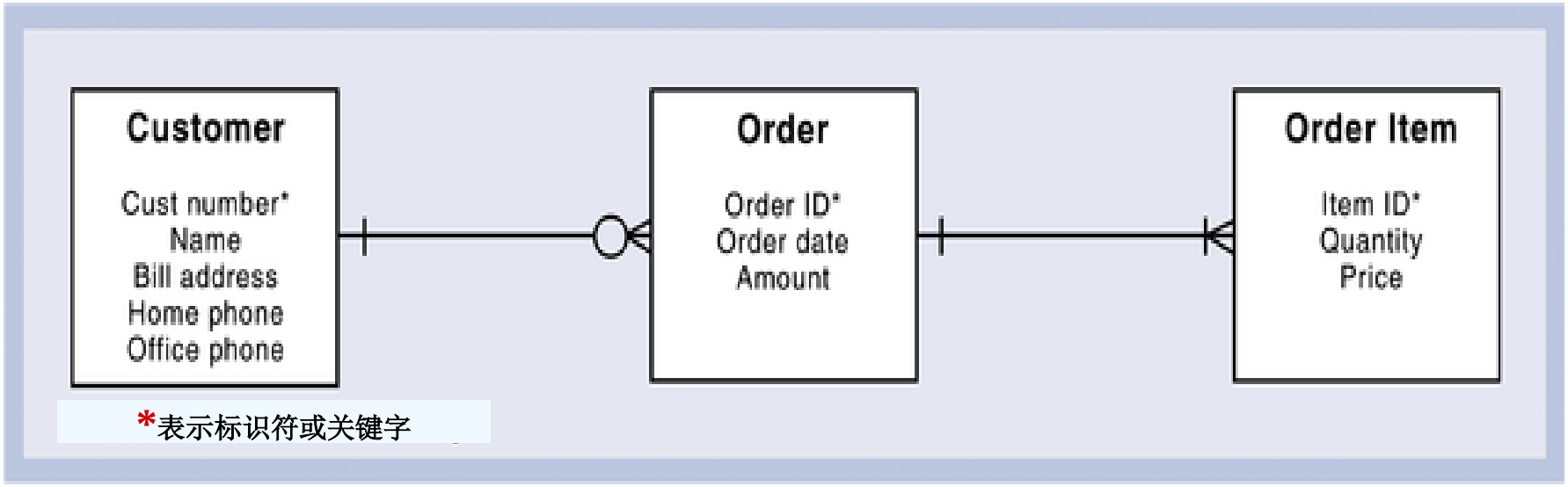
## 4.1 ERD概念的实例




实体之间关系的基数符号

# 4 实体-关系图

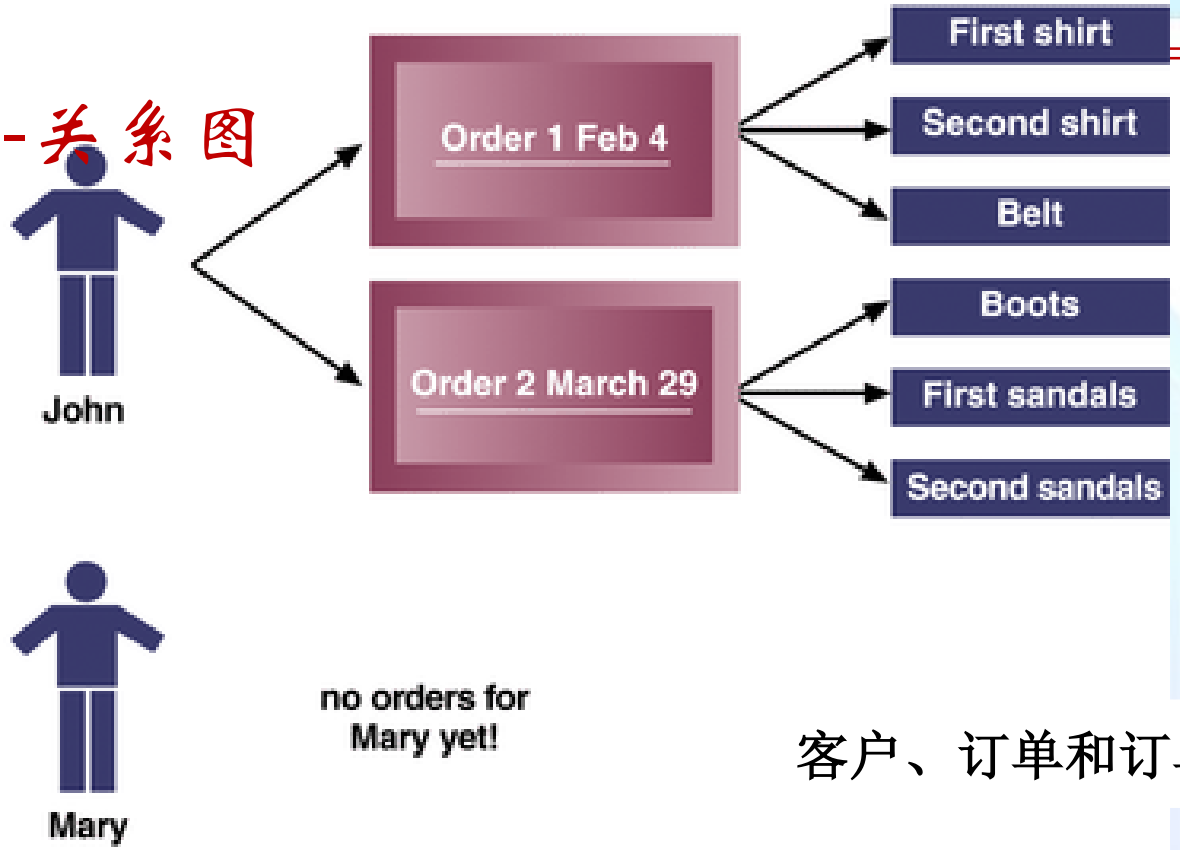
## 4.1 ERD概念的实例




带有属性的扩展ERD图

# 4 实体-关系图

## 4.1 E




客户、订单和订单中的商品

该图显示了实际的数据在事务处理中呈现的形式

# 4 实体-关系图

## 4.1 ERD概念的实例

- 分析员在建模的过程中，常常对ERD进行细化的工作就是处理多对多的关系
- 由于关系数据库中不能直接实现多对多的关系，因此必须建立一个单独的表，来消除多对多的关系
- 关联实体 – 解决上述问题的人为增加的数据实体，它一定包含两端数据实体的关键字

# 系统需求建模：事件和事物

## 5 类图

- 面向对象的方法也强调对系统中所包含事物的理解
- 面向对象的方法给事物建立的模型即是“类图”
- “类”和“实体”是明显区别的

# 5 类图

## 5.1 有关对象类的更复杂的问题

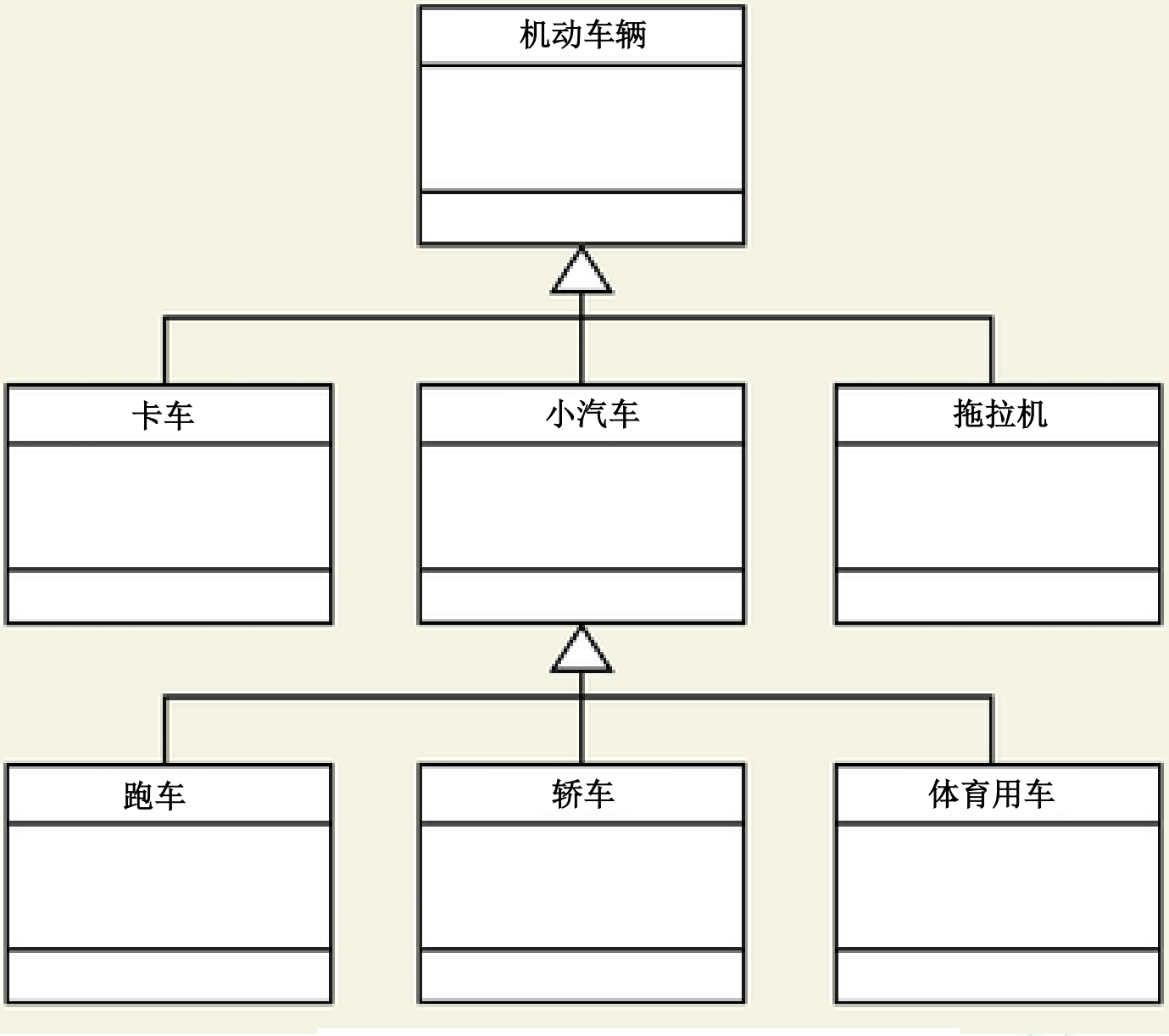
- **泛化/具体层次图** – 把类按照从最概括的父类到最具体的子类的顺序进行排列的层次图，有时也被称作继承层次图
- **继承** – 允许子类共享其父类的所有的特征

# 系统需求建模：事件和事物

5 类图

5.1 有

◆ 泛



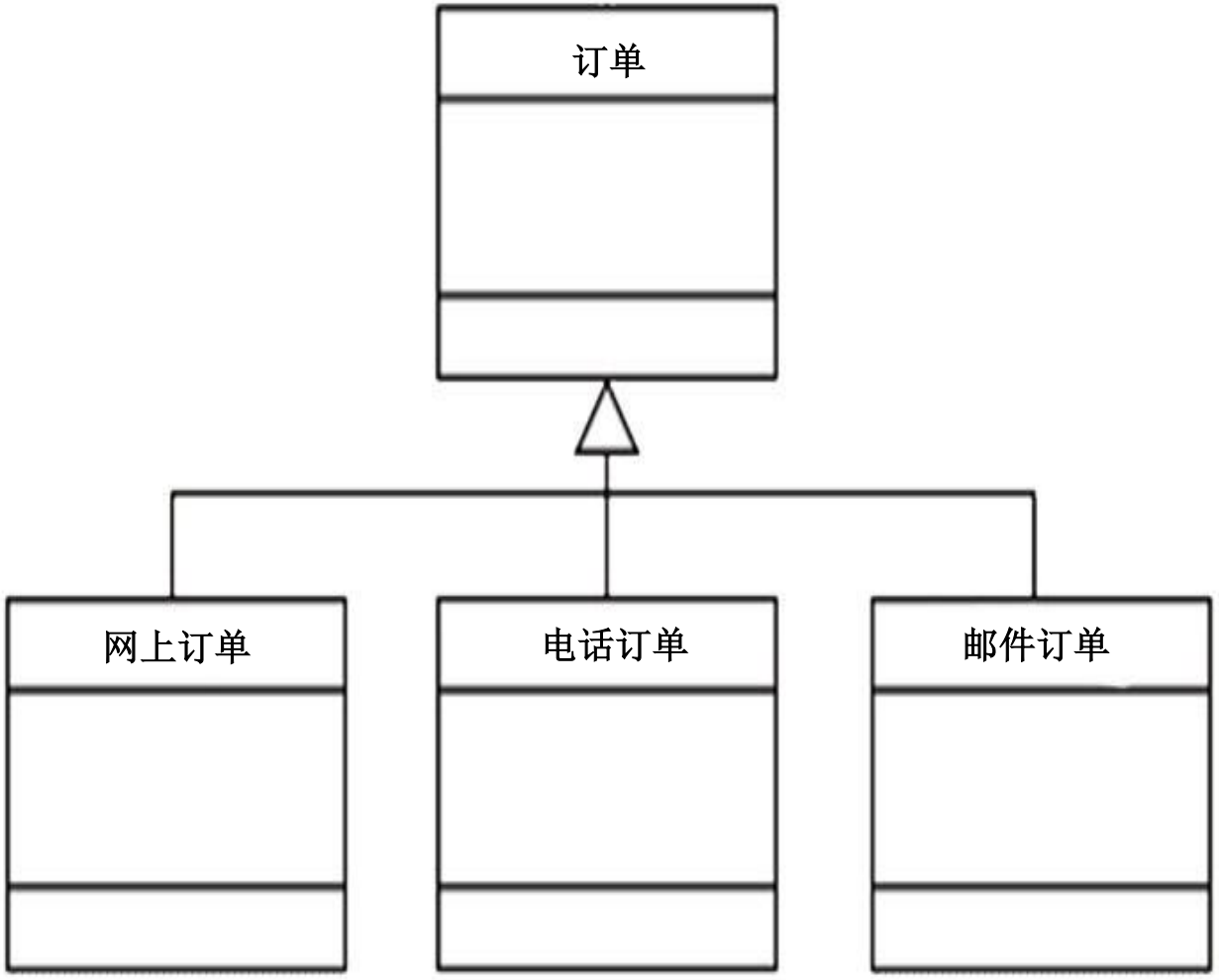

机动车辆的泛化/具体层次图（继承关系）

# 系统需求建模：事件和事物

## 5 类图

### 5.1 有关对象

- 泛化/具体




订单的泛化/具体层次图（继承关系）

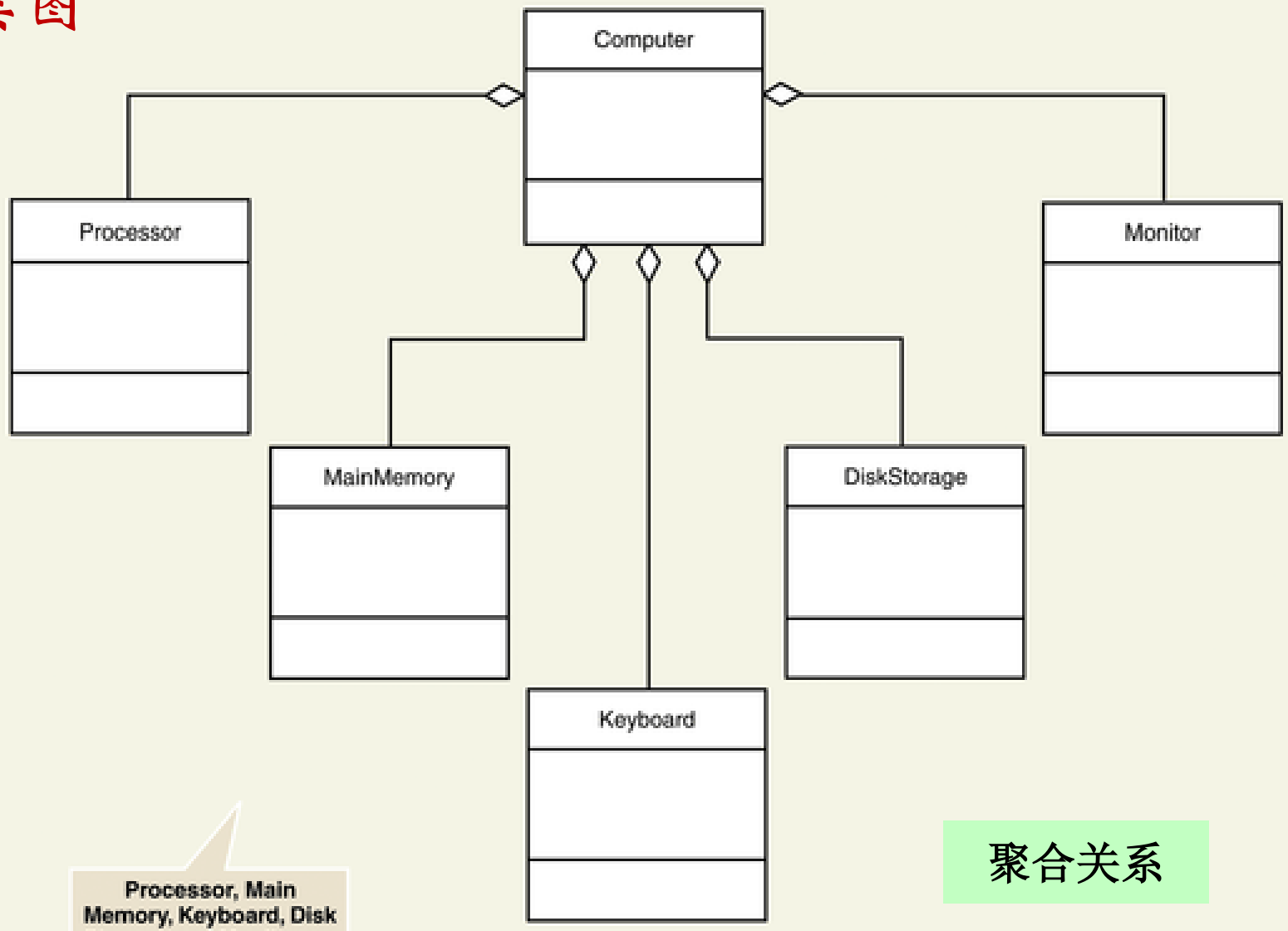
# 5 类图

## 5.1 有关对象类的更复杂的问题

- **整体/局部层次图：** 按照类之间的关联组件将类进行结构化分解的层次图
- **聚合：** 对象及其各个部分之间的一种整体-局部关系
- **组成：** 对象及其与它不可分割的各个部分之间的一种整体-局部关系

# 系统需求建模：事件和事物

## 5 类图

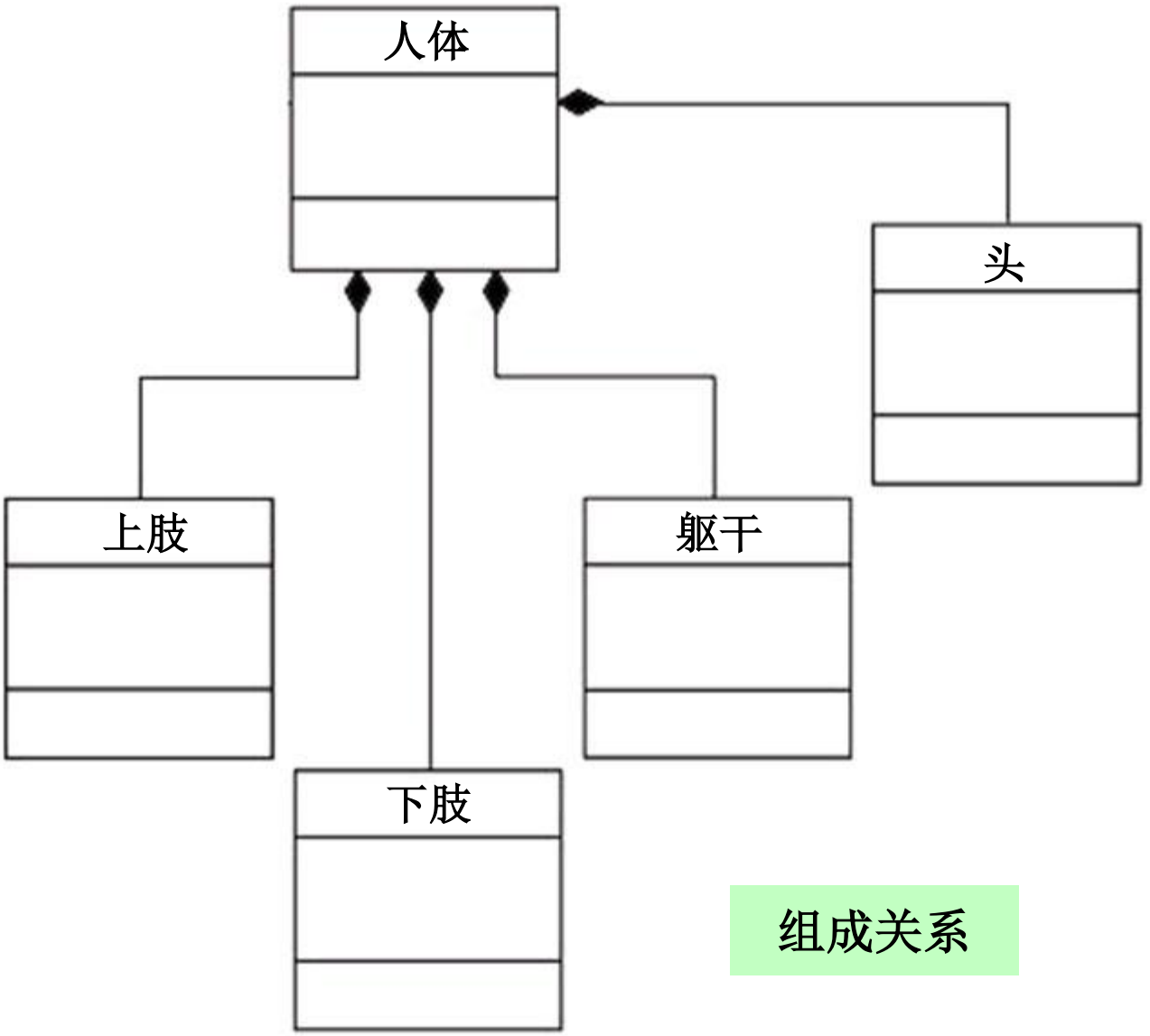


# 系统需求建模：事件和事物

## 5 类图

### 5.1 有关对象

- 整体/局部
- 聚合：对
- 组成：对
  整



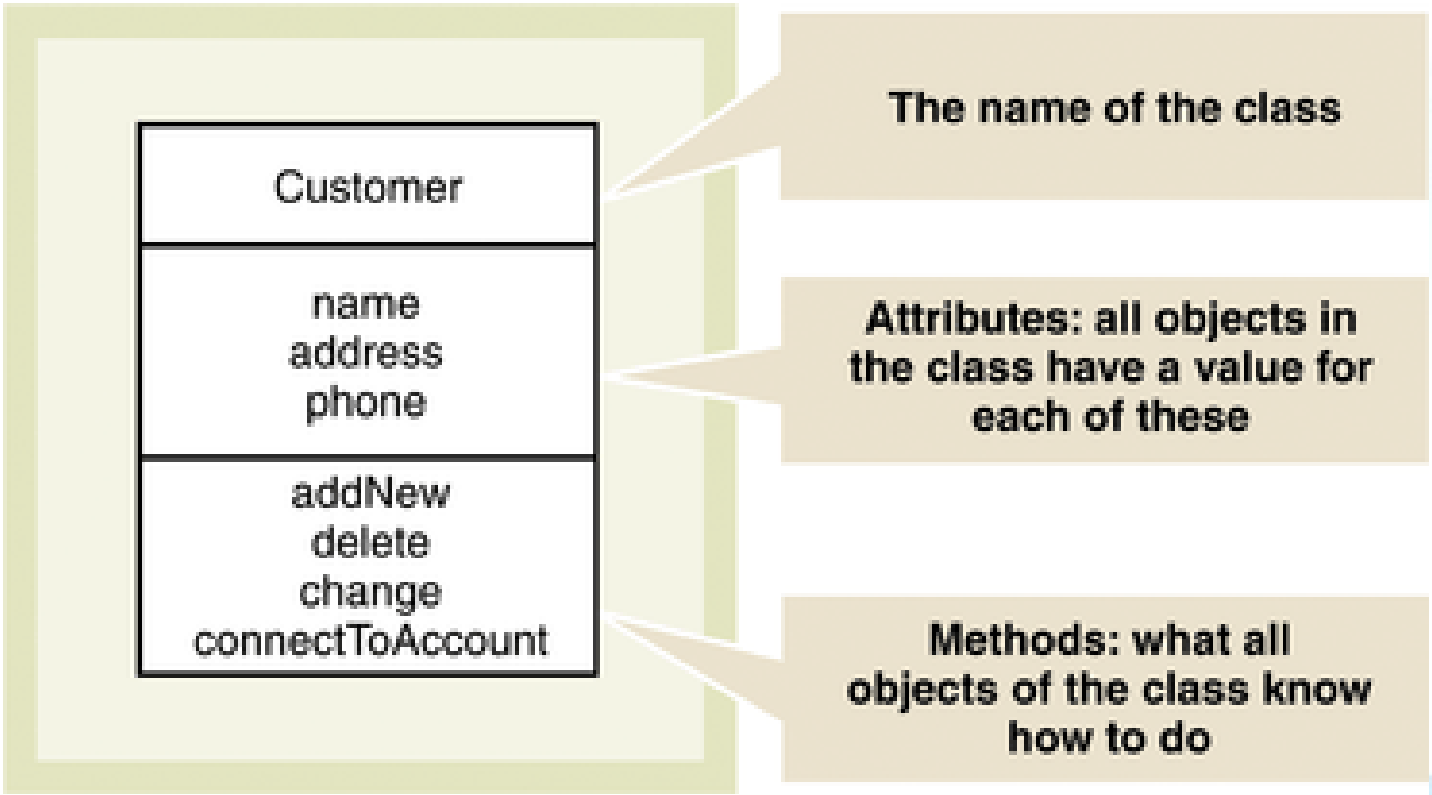

组成关系

# 系统需求建模：事件和事物

## 5 类图

### 5.1 有关对象类的更复杂的问题

- UML类图符号



# 5 类图

## 5.2 类图符号的实例

- **抽象类：** 一种不能被实例化（即不能创建对象）的类，仅为了使其子类能够继承他的属性与方法
- **具体类：** 能够被实例化的类
- **关联类：** 一种表示两种不同类之间多对多的关系的类

# 系统需求建模：事件和事物

## 事件、事物抽取实例

### WaitersOnCall送餐管理系统需求分析过程的对话资料

- Sue Brickford和Tom Brickford在上大学期间都曾在餐馆打工，他们一直梦想着开一家自己的餐馆，但资金有限未能如愿。
- Brickford兄弟发现：许多餐馆都提供外卖，而且一些餐馆（例如比萨饼店）还提供送货上门的服务。
- Sue和Tom认为电话订餐是最佳的选择。于是他们开了一个名为WaitersOnCall的送餐店，和全城各种知名的餐馆联系，接受客户订单并负责将全部饭菜送货上门。当饭店准备好了预定的饭菜后，按批发价交给他们，而饭菜送到后，客户按零售价支付，并付给他们服务费和小费。
- WaitersOnCall刚起步时规模很小，仅包括两家餐馆和一个配送司机。随着生意越做越大，Brickford兄弟意识到：必须有一套专门的计算机系统来支持业务运转。
- 于是他们聘请了信息系统顾问Sam来进行技术咨询。下面是顾问Sam同兄弟俩的关于建立送餐管理信息系统的对话。

# 系统需求建模：事件和事物

## 事件、事物抽取实例

### WaitersOnCall送餐管理系统需求分析过程的对话资料

- **Sam**：“在你们打理生意时，哪些事情促使你们决定采用计算机管理？告诉我通常这些业务是如何进行的。”
- **Sue**：“是这样，当客户打电话订餐时，我需要把它记下来，然后通知给相应的餐馆。我需要决定派哪一个司机去送货，因此要司机打电话告诉我他们什么时间有空。有时，客户会又打电话更改订单内容，因此我必须找到原始订单，然后通知餐馆更改。”
- **Sam**：“好的，那你们又怎么管理现金呢？”
- **Tom**：“司机取饭菜时会从餐馆直接拿到账单的副本，账单和我们的计算应该是一致的，然后司机送货时收取相应的现金并加上服务费。在下班时，司机报账，我们把司机收到的现金汇总起来，和我们的记录进行比较，所有的司机都交完账后，我们需要开张银行存款单，存入当天的现金总收入。每周末，我们按提前约定的批发价来计算欠餐馆多少钱，把结算单和支票寄给他们。”

# 系统需求建模：事件和事物

## 事件、事物抽取实例

### WaitersOnCall送餐管理系统需求分析过程的对话资料

- **Sam**：“那你们还想从这个系统中获取别的信息吗？”
- **Sue**：“如果每周末能统计出每个餐馆有多少订单、城里每个区有多少订单等信息就更好了。这能帮助我们制定广告策略及与餐馆的合同，而且我们还需要每月财务状况统计结果。”
- 在**Sue**及**Tom**说话时，**Sam**记下了几个要点，画了几张草图。之后，他花了一些时间仔细考虑，总结出了**WaitersOnCall**的需求状况。

# 系统需求建模：事件和事物

## 事件、事物抽取实例

### Waiters on Call – 餐馆送货系统– 事件与事物的抽取

1.在发生如下事件时，系统可以进行处理：

- 客户打电话下订餐订单
- 客户打电话修改订单
- 送货司机汇报工作情况
- 送货司机上交一天的收入

2.在特定时间内系统产生所需信息（临时事件）：

- 生成日结算存款单
- 生成周末餐馆支付帐单
- 生成周销售报表
- 生成月财务报表

3.系统还需要处理以下事件：

- 新餐馆加盟
- 餐馆更改了菜单
- 雇佣了新的送货司机（送餐员）
- 送货司机（送餐员）离职

4.需要存储的信息（事物）：

- 餐馆
- 菜单
- 客户
- 订单
- 订单支付帐单
- 司机（送餐员）

# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 09

# 用例分析的重要概念

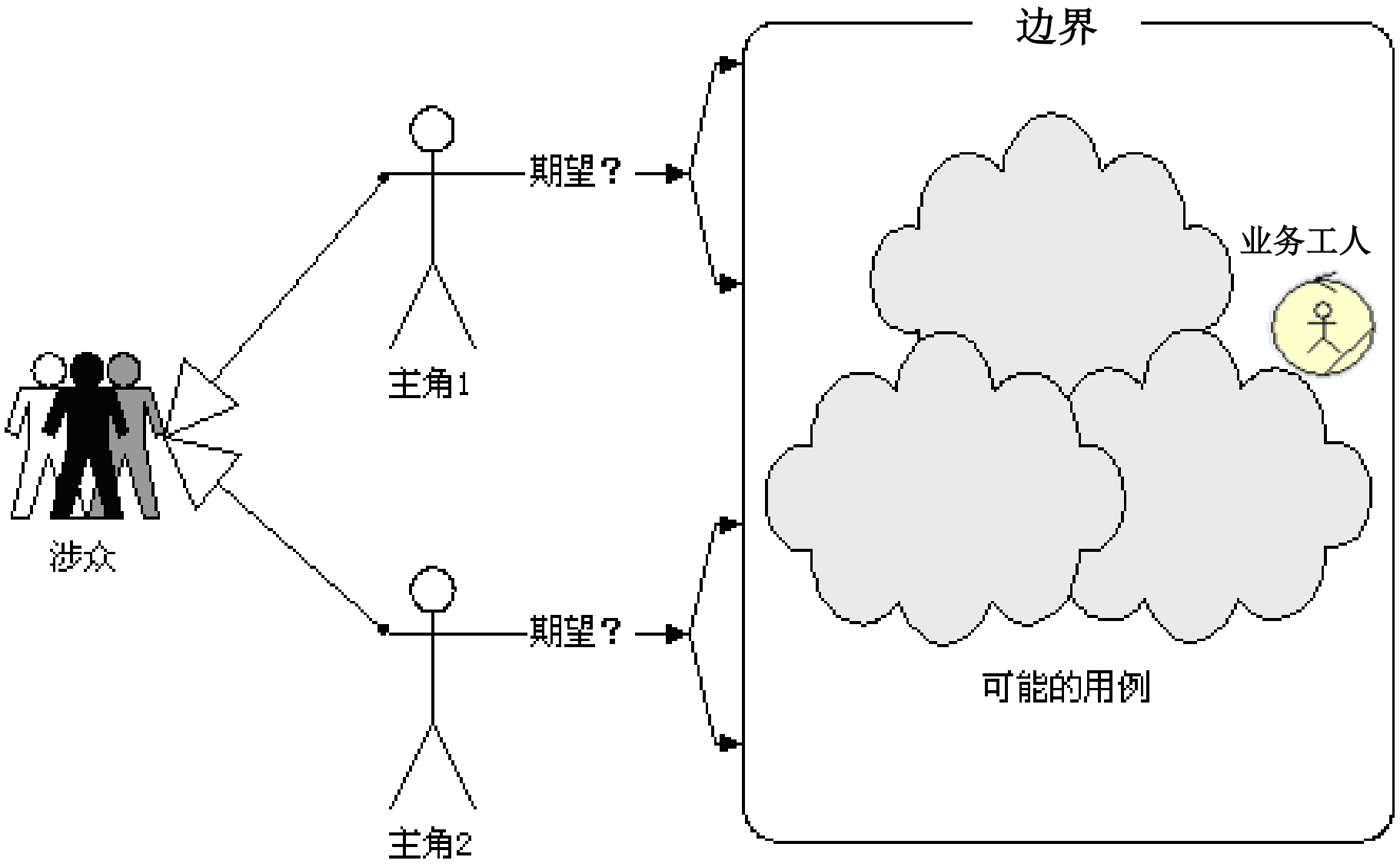
**用例分析：**

是一种把现实世界的需求捕获下来的方法；

如果将所有对系统有愿望的人要做的**所有事情**都找全了，则该系统的需求就被确定下来了

**用例图：** 用例分析的UML模型图

# 用例分析的重要概念



# 用例分析的重要概念

**涉　众：**与未来系统有利益相关的一切人或事（外部系统），涉众的利益诉求会影响系统的建设结果

**参与者：**涉众中的一部分，直接对未来系统发出动作，或直接从系统中接收反馈

**主　角：**处于系统边界之外的参与者，直接驱动用例

**业务工人：**一种参与者，处于系统边界之内，直接参与业务过程的完成

# 用例分析的重要概念

**用例（Use Case）：**

官方定义：一个用例定义了一组用例实例，其中每个实例都是系统所执行的一系列操作，这些操作生成特定主角可以观测的值

通俗解释：用例就是一件事情，要完成这件事情，需要做一系列的活动，并产生期望的结果

用例实例（用例场景）：做一件事情可以有很多种不同的方法和步骤，每种方法和步骤称之为用例场景，一个场景就是一个用例实例

# 用例分析的重要概念

业务用例 ： 用于业务建模的用例，定义业务范围

概念用例 ： 获取业务用例中的核心业务逻辑步骤

系统用例 ： 用于功能需求分析的用例，定义系统范围

用例粒度 ： 用例内涵的大小

边　　界 ： 用例分析工作的范围，可以是组织/系统/领域

粒度 vs 边界：

边界定义的大小与用例粒度大小成正比关系，在边界不变的情况下，边界内的用例粒度是不变的

# 用例分析的重要概念

用例分析的目标：

最终找到系统用例，完成功能性需求分析

业务用例 → 概念用例 → 系统用例

用例的命名：动宾词组构成

用例识别（目标 vs 步骤）：

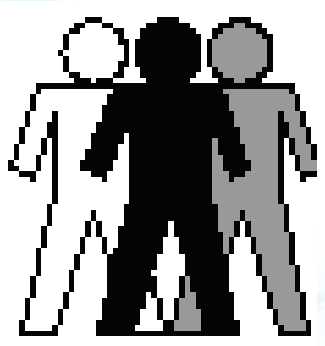
（1）确定参与者（主角），即确定了边界；

（2）找出相关用例，方法是以该用例是否完成了参与者的某个完整目标为依据

# 用例分析的重要概念

## 用例分析案例：评教系统

**1.涉众分析：**



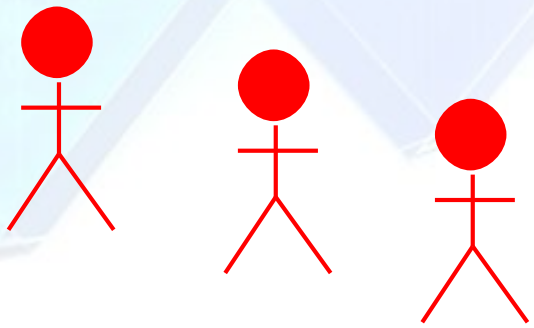
（1）学校领导、学院领导、教务处领导

（2）教师、学生、督导专家（教学督导委员）

（3）教学秘书、教学督导委员会秘书

（4）系统管理员

**参与者（主角）：**



（1）学生

（2）督导专家

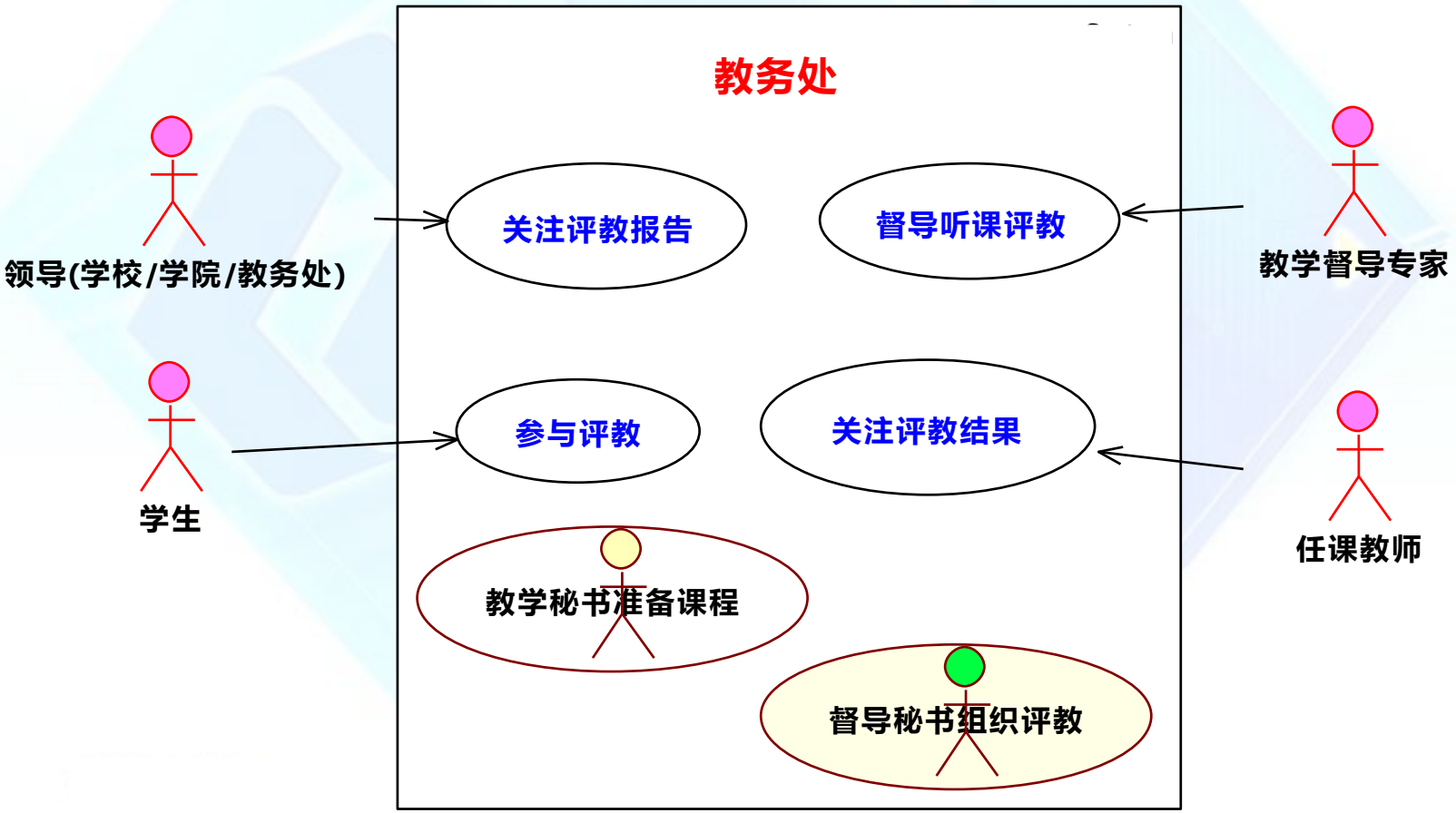
（3）领导

（4）教师

# 用例分析的重要概念

## 用例分析案例：评教系统

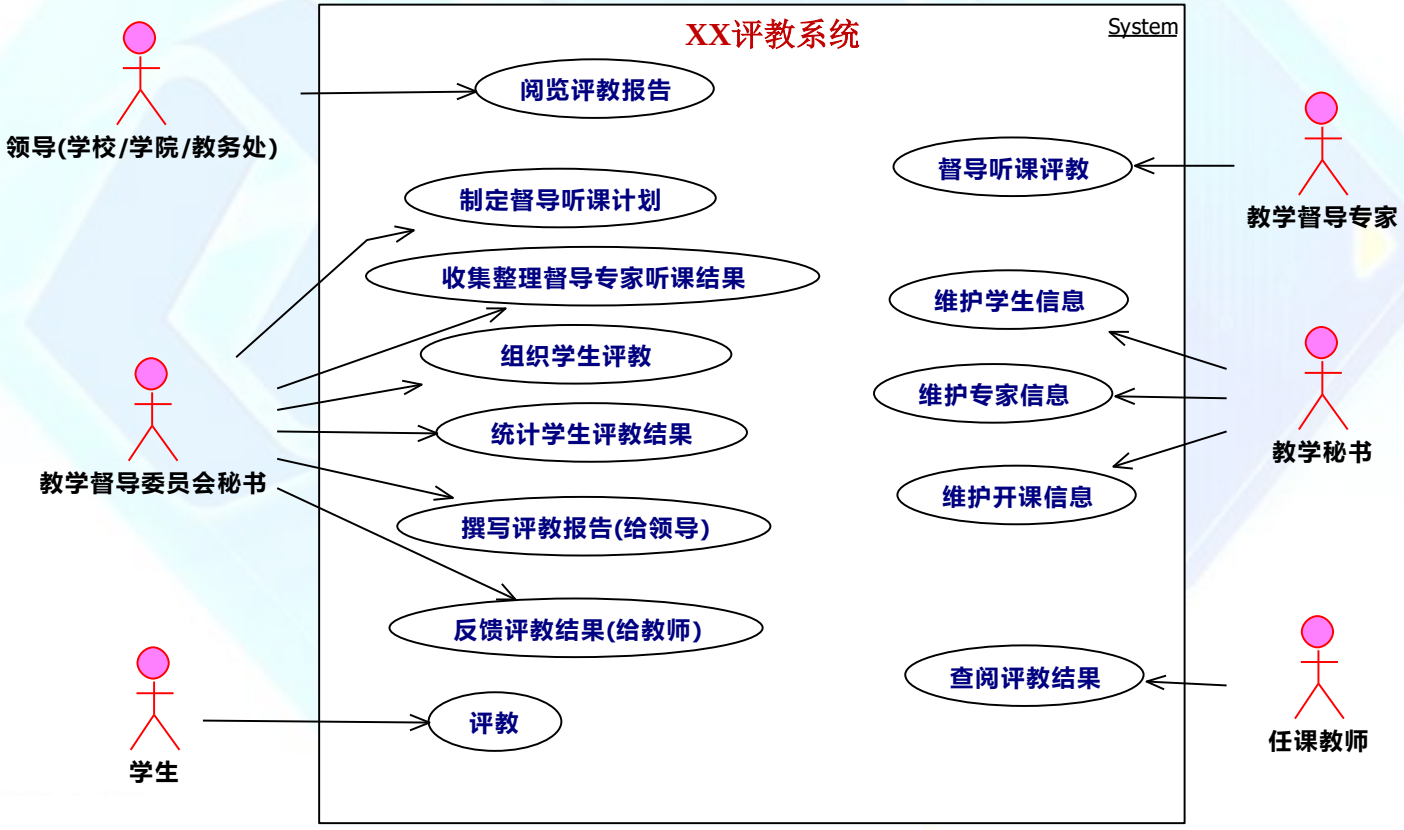
### 2. 业务用例分析 -- 业务用例图



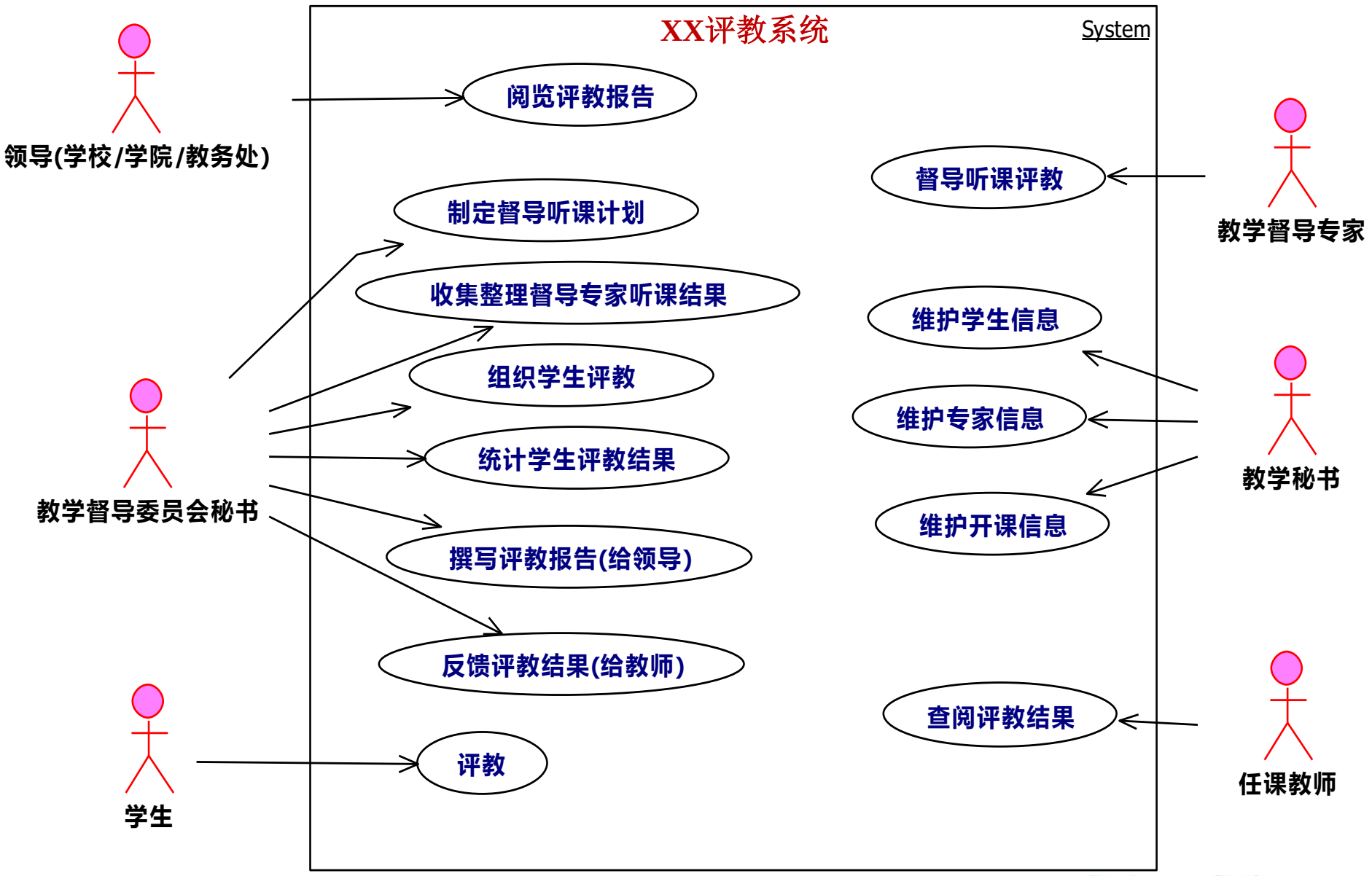
# 用例分析的重要概念

## 用例分析案例：评教系统

### 3. 概念用例分析 -- 概念用例图



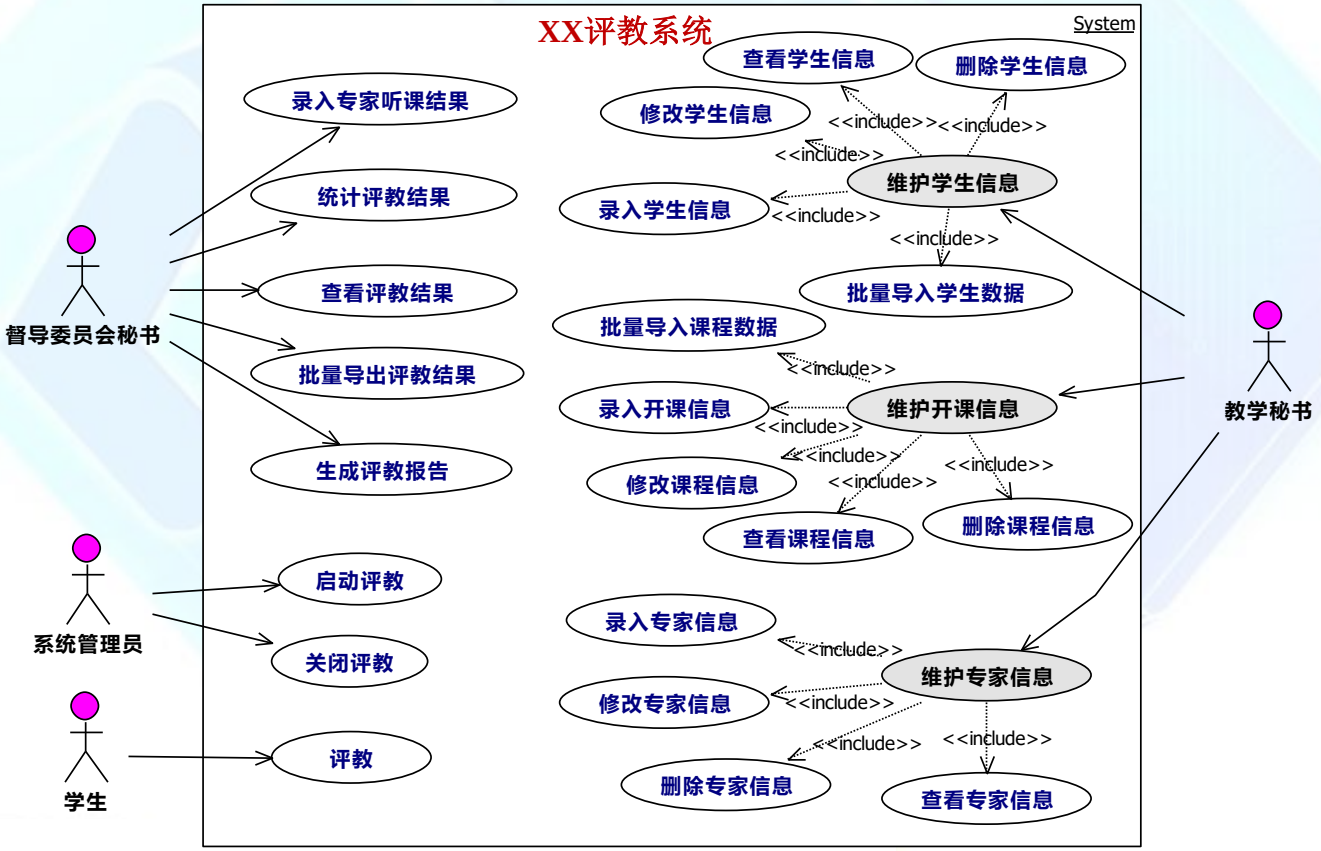
# 用例分析的重要概念


XX评教系统
System
领导(学校/学院/教务处)
阅览评教报告
制定督导听课计划
收集整理督导专家听课结果
组织学生评教
统计学生评教结果
撰写评教报告(给领导)
反馈评教结果(给教师)
评教
教学督导委员会秘书
学生
督导听课评教
教学督导专家
维护学生信息
维护专家信息
维护开课信息
教学秘书
查阅评教结果
任课教师

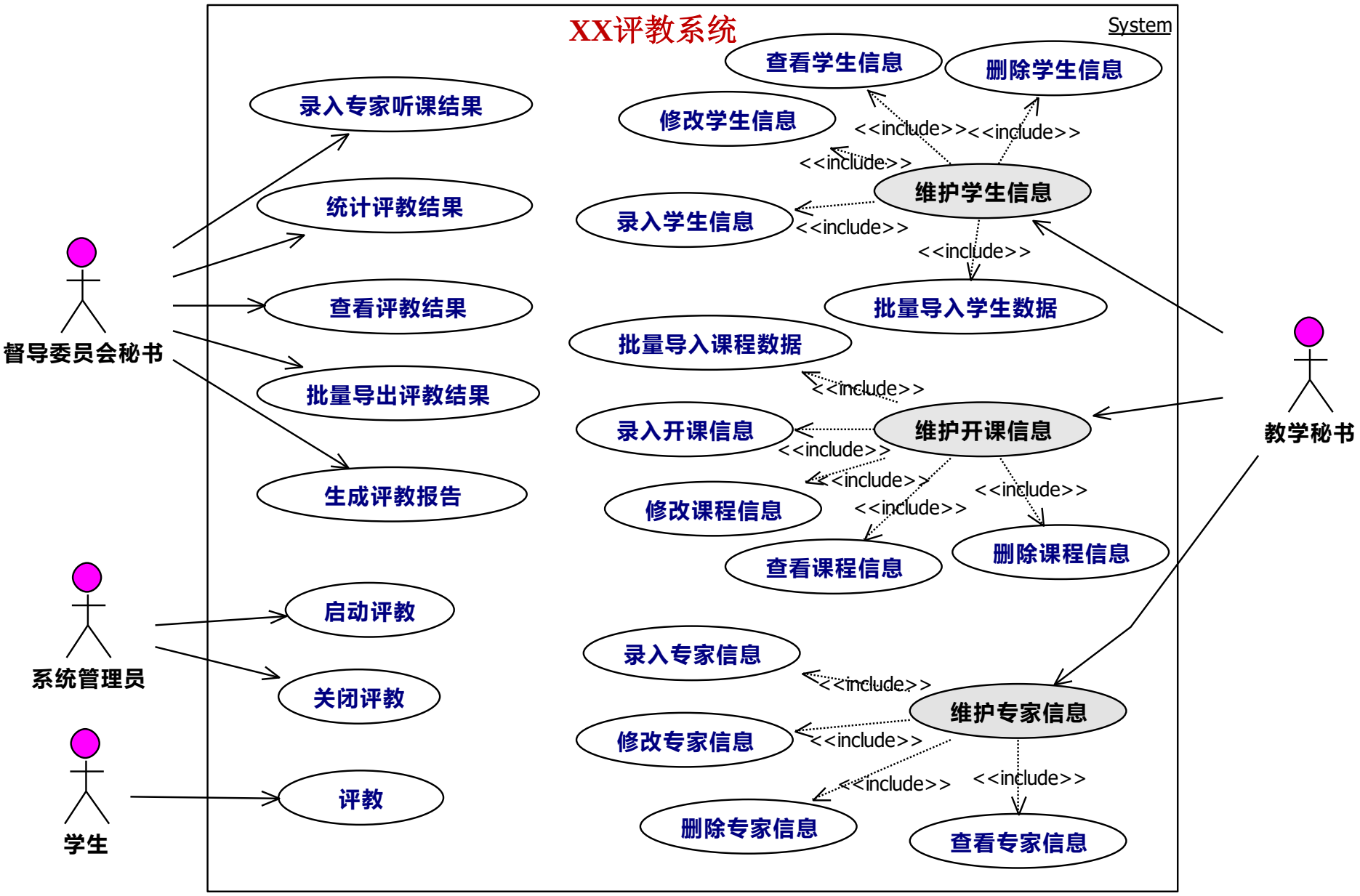
# 用例分析的重要概念

## 用例分析案例：评教系统

### 4. 系统用例分析 -- 系统用例图

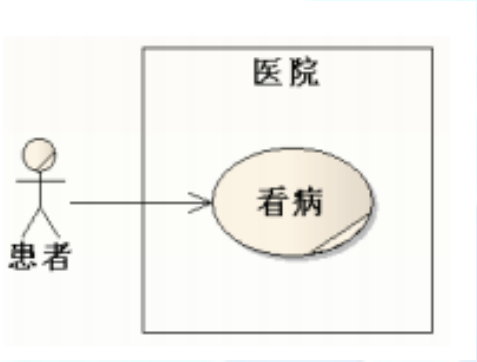


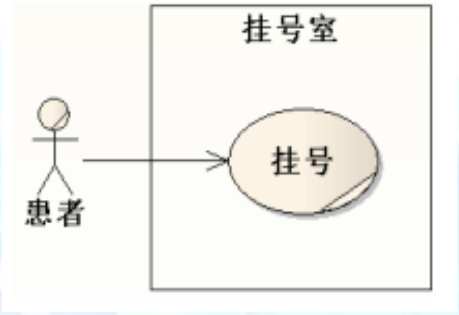
# 用例分析的重要概念

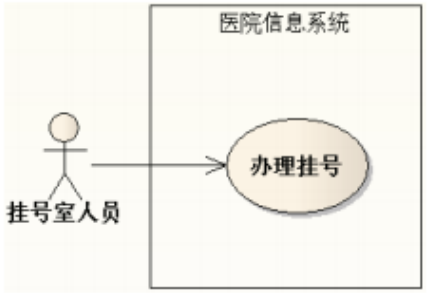

XX评教系统
System
查看学生信息
删除学生信息
修改学生信息
<<include>>
<<include>>
<<include>>
维护学生信息
录入学生信息
<<include>>
<<include>>
批量导入学生数据
批量导入课程数据
<<include>>
录入开课信息
维护开课信息
<<include>>
<<include>>
<<include>>
<<include>>
修改课程信息
查看课程信息
删除课程信息
录入专家信息
<<include>>
维护专家信息
修改专家信息
<<include>>
<<include>>
<<include>>
删除专家信息
查看专家信息
录入专家听课结果
统计评教结果
查看评教结果
批量导出评教结果
生成评教报告
启动评教
关闭评教
评教
督导委员会秘书
系统管理员
学生
教学秘书

# 用例分析的重要概念

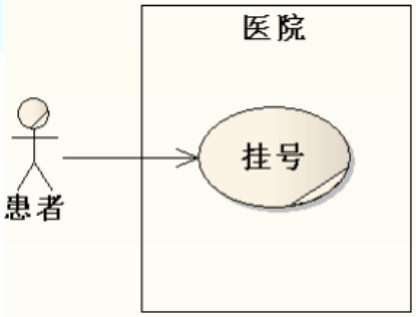
## 边界 VS 用例及粒度

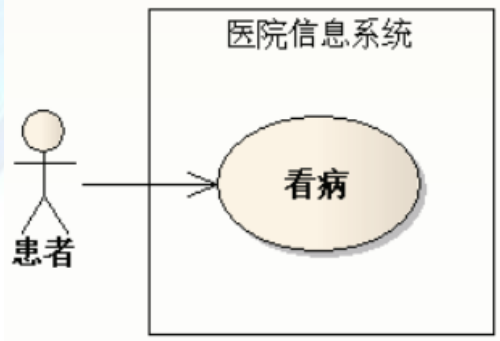



正确的边界与正确的用例 √

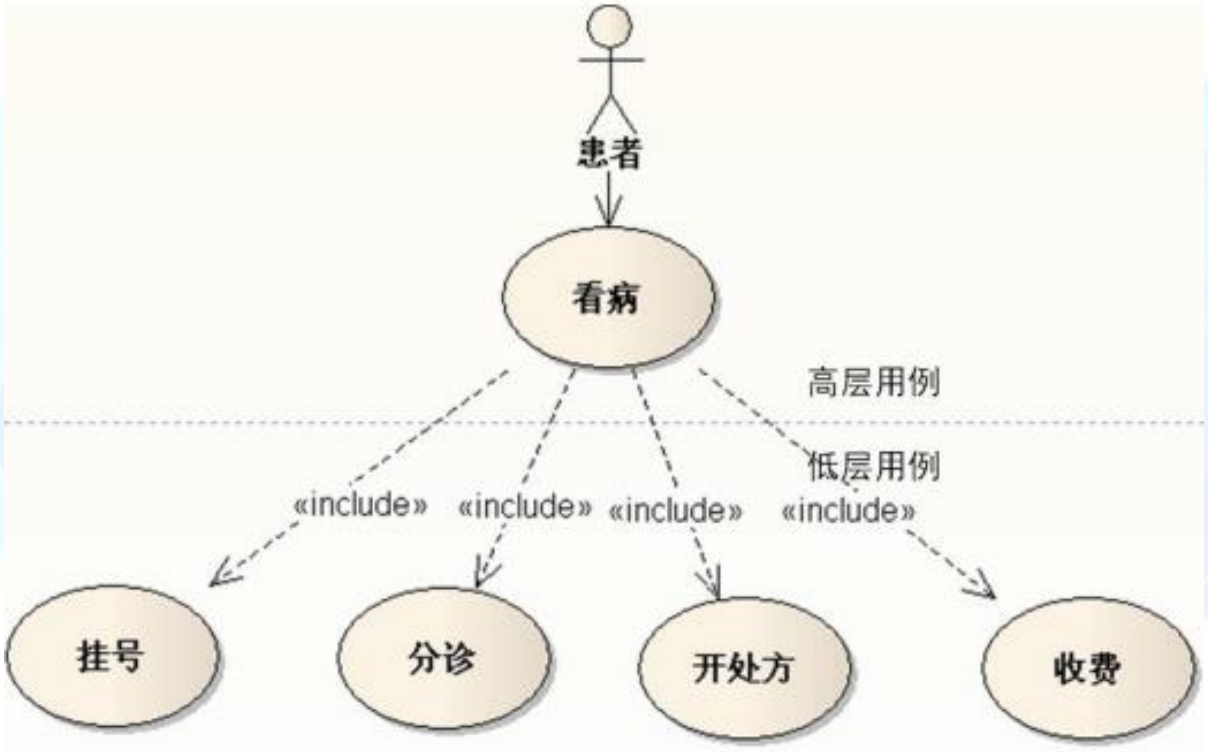



正确的边界、错误的用例 ×

# 用例分析的重要概念

## 边界 VS 用例及粒度

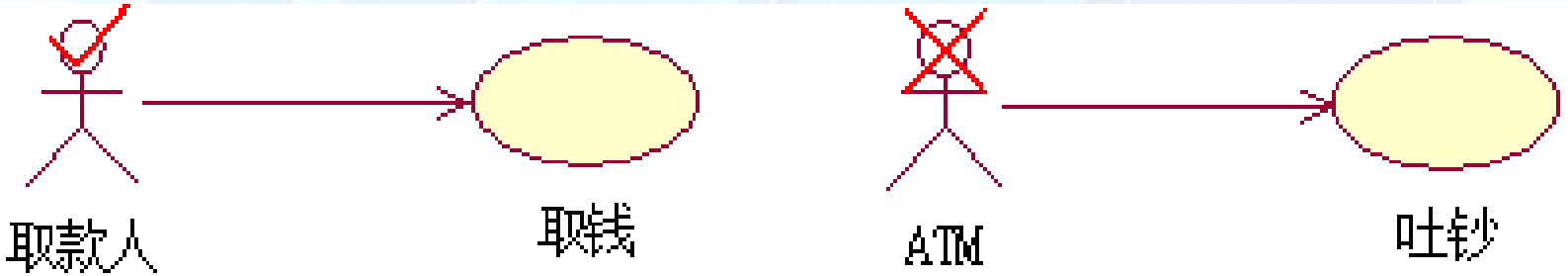



边界混乱：上面的边界是“医院”，下面的边界是“医院管理系统”

# 用例分析的重要概念

## 错乱的用例及粒度

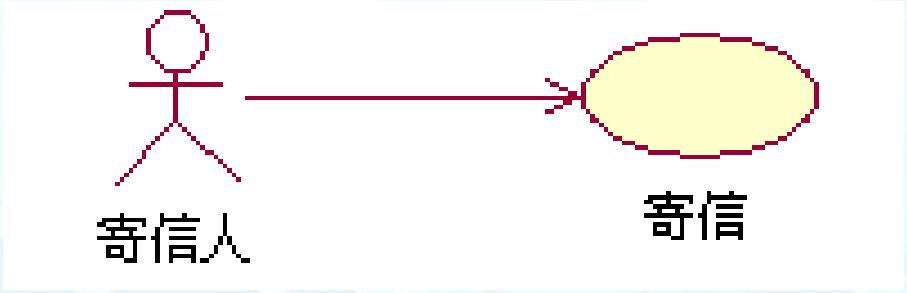
填写取款单不是取款人的目的，因此不是用例



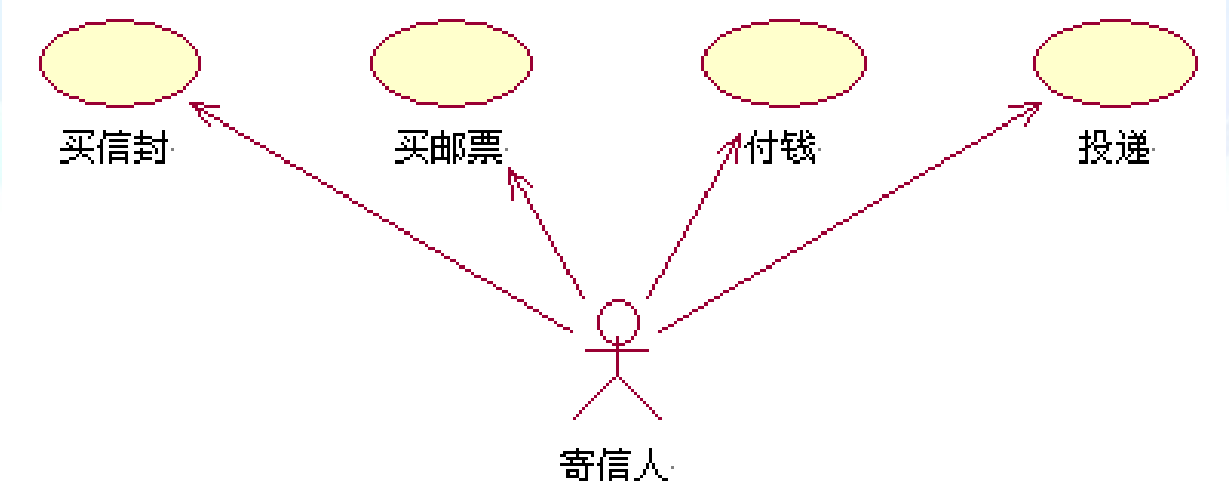

ATM是没有吐钞的愿望的，因此不能驱动用例

# 用例分析的重要概念

## 错乱的用例及粒度



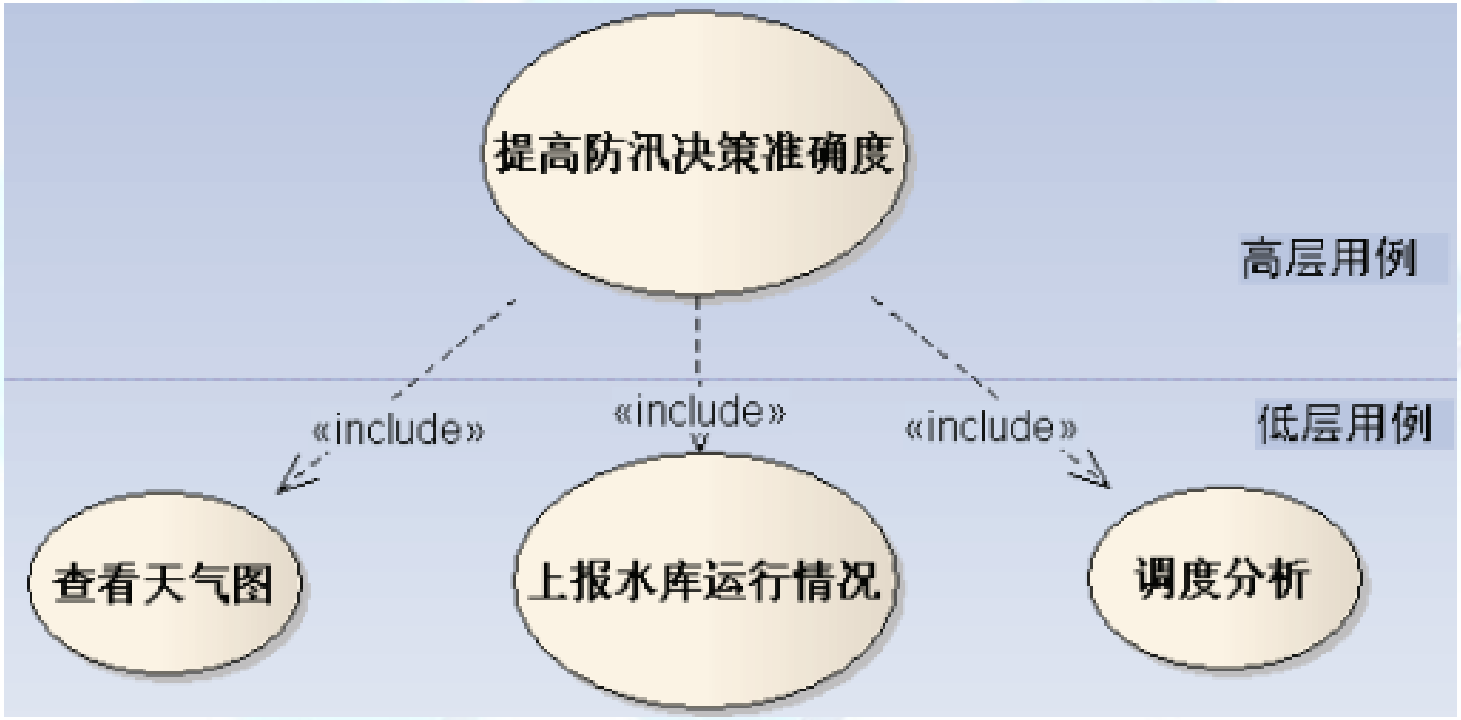

以完整目标作为用例！！！




不能以步骤作为用例！！！

# 用例分析的重要概念

## 错乱的用例及粒度




不要把领导的愿望作为用例

# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 09

# 用户故事与用例建模

## 本章主要内容

1. 敏捷开发中的“用户故事”（User Story）
2. 面向对象方法中的“用例”（Use Case）
3. 用例建模的基本过程
4. 用例模型的提交物
5. 活动图 & 泳道图

# 用户故事与用例建模

## 说明

- 在敏捷开发中，需求表述为一组“用户故事（User Story）”
- 在传统的OO分析方法中，需求被表述为一组“用例（Use Case）”
- 二者都是对需求的描述形式
- 区别在于大小不同、具体形式不同
- 对初学者来说，可以将二者看作等价模型，在实践中汇集二者的优点

# 用户故事与用例建模

## 本章主要内容

1. 敏捷开发中的“用户故事”（User Story）
2. 面向对象方法中的“用例”（Use Case）
3. 用例建模的基本过程
4. 用例模型的提交物
5. 活动图 & 泳道图

# 用户故事与用例建模

## 何谓“用户故事”？

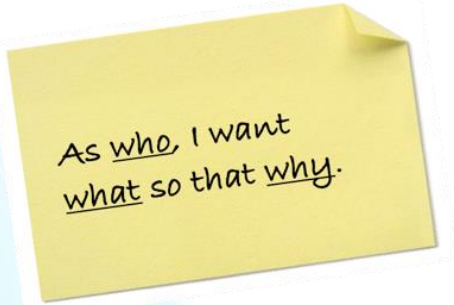
- 用户故事：
  - 对软件用户（或所有者）有价值的功能性的简明的书面描述
- 从用户的角度来描述用户渴望得到的软件功能：
  - 角色(谁要使用这个功能)、目标/活动(需要完成什么样的功能)、商业价值(为什么需要这个功能，这个功能带来什么样的价值)
- 三个组成部分：
  - 卡片（Card）：用户故事一般在小卡片上写着故事的简短描述，工作量估算等
  - 交谈（Conversation）：用户故事背后的细节来源于和客户或者产品负责人的交流沟通
  - 确认（Confirmation）：通过验收测试确认用户故事被正确完成

# 用户故事与用例建模

## 用户故事的描述

As a [user role] I want to [goal] so I can [reason]

作为一个<角色>, 我想要<活动>, 以便于<商业价值>




Who (user role) -- 角色：谁要使用这个功能?

What (goal) -- 功能：需要完成什么样的功能?

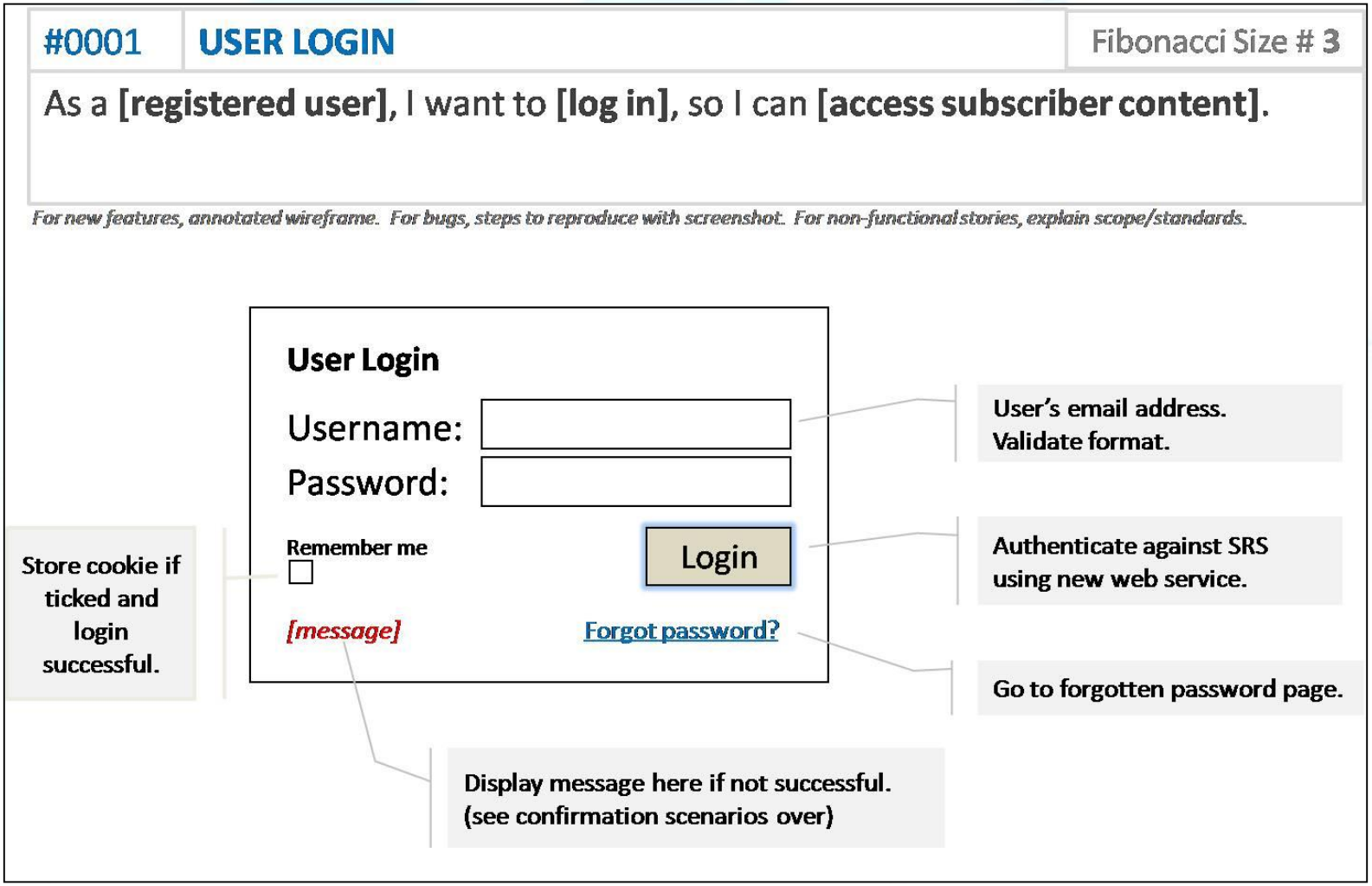
Why (reason) --价值：为什么需要这个功能，功能带来什么样的价值?

例如：

① As a registered user I want to log in so I can access subscriber-only content.

② 作为一个“网站管理员”，我想要“统计每天有多少人访问了我的网站”，以便于“赞助商了解我的网站会给他们带来什么收益”。

# 用户故事卡的正面：Conversation



# 用户故事卡的反面：Confirmation

**Confirmation**

1. Success – valid user logged in and referred to home page.
   a. 'Remember me' ticked – store cookie / automatic login next time.
   b. 'Remember me' not ticked – force login next time.

2. Failure – display message:
   a) "Email address in wrong format"
   b) "Unrecognised user name, please try again"
   c) "Incorrect password, please try again"
   d) "Service unavailable, please try again"
   e) Account has expired – refer to account renewal sales page.

# 用户故事与用例建模

## 用户故事板

# 用户故事与用例建模

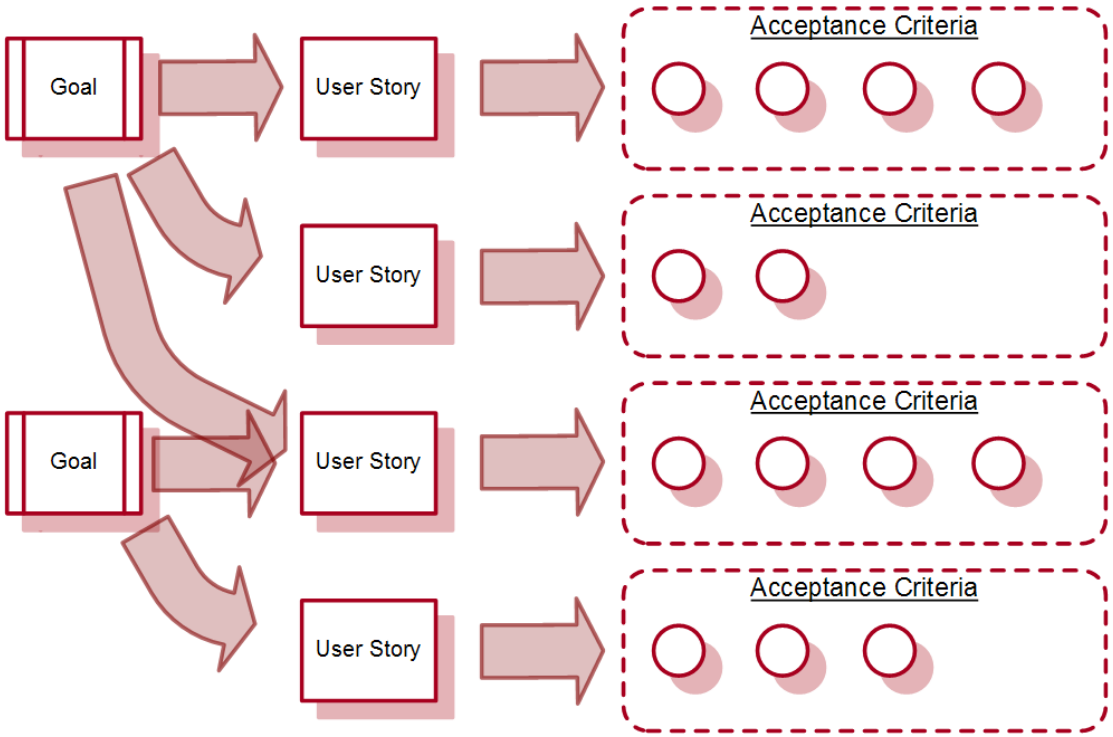
## 好的用户故事应具备的特征：INVEST

- **I**ndependent – 尽可能独立
- **N**egotiable – 可讨论的：它不是一个合同，没有详细的规约，后续开发阶段可以不断协商和改进
- **V**aluable – 对用户/客户有价值的，以用户可理解的语言书写，是系统的“特性”而非“开发任务”
- **E**stimatable – 其工作量可以估计
- **S**mall – 小，而不是大
- **T**estable – 可测试的、可验证的

# 用户故事与用例建模

## 用户故事支持验收测试*

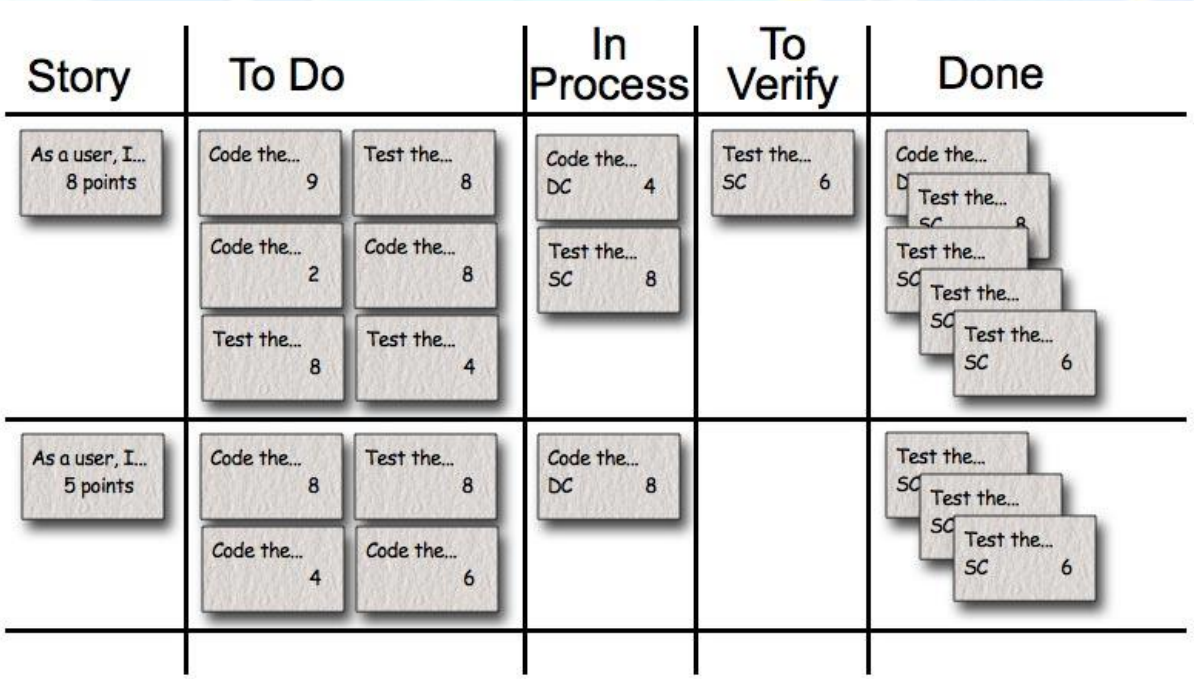
- 在每个用户故事背后，列出未来用户测试时可能使用的“测试用例”，作为该故事是否已被完整实现的基本标准
- 对应于敏捷开发的一个基本思想：在写代码之前先写测试(Test-Driven Development, TDD)



# 用户故事与用例建模

## 用户故事支持敏捷迭代计划*

- 针对识别出的每一个故事，使用Story Point估算其工作量
  - 故事点：一个达到共识的基本时间单位，例如1天
  - 使用预定的值：1/2、1、2、3、5、8、13、20、40、80
- 团队成员分别估计(而不是由项目经理一人决定)，差异较大时面对面讨论，发现分歧，形成共识
- 形成估算清单



# 用户故事与用例建模

## 用户故事支持敏捷迭代计划*

- 对用户故事排列优先级
- 安排责任人
- 汇聚为迭代计划并发布
- 开发过程中监控进度

13

## 课外阅读

- **Mick Cohn. User Stories Applied: for Agile Software Development (《用户故事与敏捷方法》， 清华大学出版社，2010年4月，ISBN 9787302223405)**

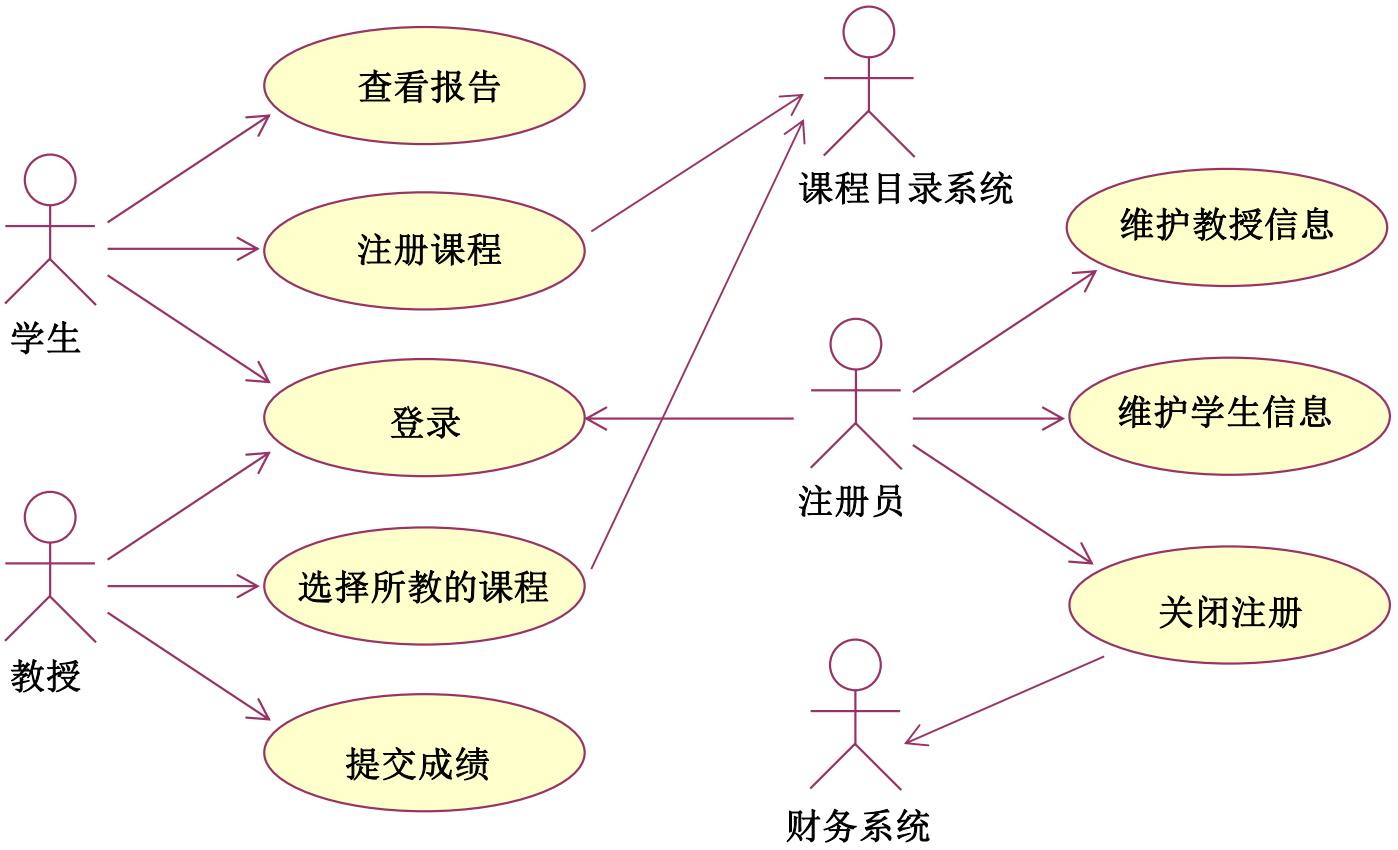
# 用户故事与用例建模

## 本章主要内容

1. 敏捷开发中的“用户故事”（User Story）
2. 面向对象方法中的“用例”（Use Case）
3. 用例建模的基本过程
4. 用例模型的提交物
5. 活动图 & 泳道图

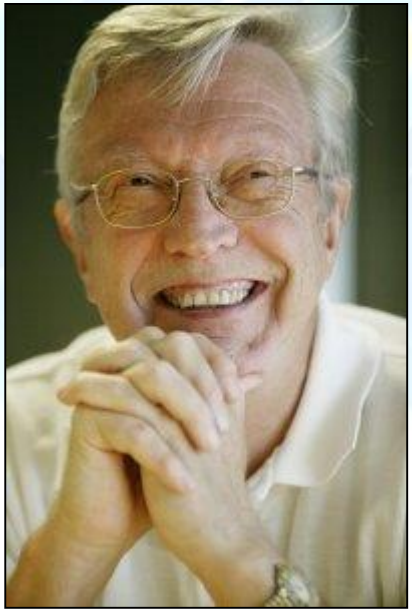
# 用户故事与用例建模

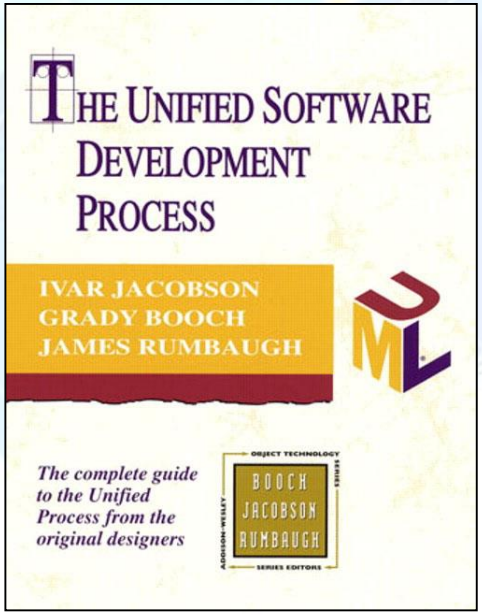
## 需求分析技术：用例



# 什么是用例(Use Case)?

- 用例(Use Case)：表示系统所提供的服务或可执行的某种行为
  - 定义了系统是如何被参与者所使用的，描述了参与者为了使用系统所提供的某一完整功能而与系统之间发生的一段“对话”
  - 用例的概念在1986年由Ivar Jacobson正式提出之后被广泛接受，迅速发展，已成为OO、UML、RUP的标准规范和方法



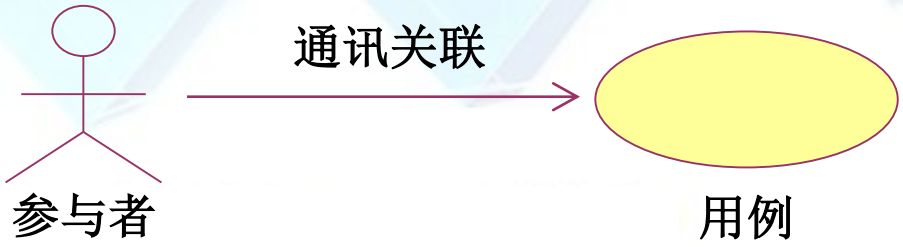


# 什么是用例(Use Case)?

- 用例：站在用户角度定义软件系统的外部特性
- 四大特征：
  - 行为序列(sequences of actions)：一个用例由一组可产生某些特定结果的行为构成，这些行为是不可再分解的(接收用户输入、执行、产生结果)
  - 系统执行(system performs)：系统为外部角色提供服务
  - 可观测到的、有价值的结果(observable result of value)：用例必须对用户产生价值
  - 特定的角色(particular actor)：某人、某台设备、某外部系统、等等，能够触发某些行为

# 用例方法的基本思想

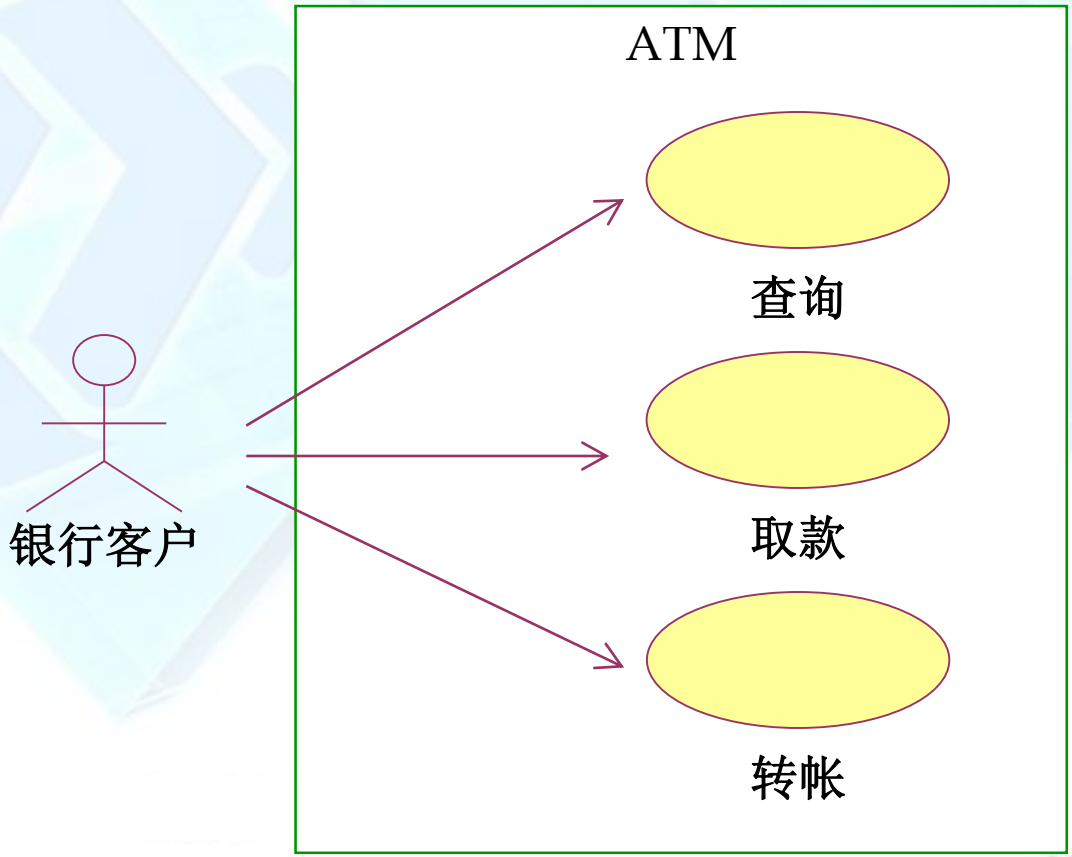
- 用例方法的基本思想：从用户的角度来看，他们并不想了解系统的内部结构和设计，他们所关心的是系统所能提供的服务，也就是被开发出来的系统将是如何被使用的
- 用例模型主要由以下模型元素构成：
  - 参与者(Actor)：存在于被定义系统外部并与该系统发生交互的人或其他系统，代表系统的使用者或使用环境
  - 用例(Use Case)：表示系统所提供的服务或可执行的某种行为
  - 通讯关联(Communication Association)：用于表示参与者和用例之间的对应关系，它表示参与者使用了系统中的哪些服务(用例)、系统所提供的服务(用例)是被哪些参与者所使用的



# 用户故事与用例建模

## 示例：ATM系统的用例

- 参与者：银行客户
- 用例：银行客户使用自动提款机来进行银行帐户的查询、提款和转帐交易

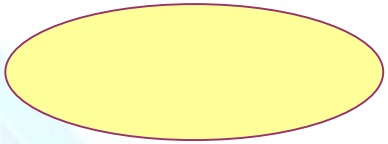


# 关于“通讯关联”的几点说明

- 通讯关联表示的是参与者和用例之间的关系：
  - 箭头表示在这一关系中哪一方是对话的主动发起者，箭头所指方是对话的被动接受者
  - 如果不想强调对话中的主动与被动关系，可以使用不带箭头的关联实线
  - 通讯关联不表示在参与者和用例之间的信息流，并且信息流向是双向的，它与通讯关联箭头所指的方向没有关系

# 用例的内部剖析

- 用例 ≠ 椭圆 + 名字！！



**Use case**

**Name of the Use Case (用例的名字)**

Description (描述)
Actor(s) (参与者)
Flow of events(事件流)
  Basic flow(常规流)
    Event 1 (事件)
    Event 2
  ……
  Alternate flow(备选流)
Pre-conditions (前置条件)
Post-conditions (后置条件)
……

## 用例方法的优点

- 系统被看作是一个黑箱，并不关心系统内部是如何完成它所提供的功能的
- 首先描述了被定义系统有哪些外部使用者(抽象为Actor)、这些使用者与被定义系统发生交互
- 针对每一参与者，又描述了系统为这些参与者提供了什么样的服务(抽象成为Use Case)、或者说系统是如何被这些参与者使用的
- 用例模型容易构建、也容易阅读
- 完全站在用户的角度上，从系统外部来描述功能
- 帮助系统的最终用户参与到需求分析过程中来，其需求更容易表达出来

# 用户故事与用例建模

## 本章主要内容

1. 敏捷开发中的“用户故事”（User Story）
2. 面向对象方法中的“用例”（Use Case）
3. 用例建模的基本过程
4. 用例模型的提交物
5. 活动图 & 泳道图

# 用例建模的基本过程

- Step 1：识别并描述参与者(actor)；
- Step 2：识别用例(use case)，并给出简要描述；
- Step 3：识别参与者与用例之间的通讯关联(Association)；
- Step 4：给出每一个用例的详细描述
- Step 5：细化用例模型

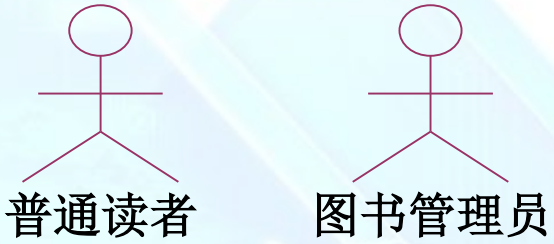
# Step 1：识别并描述参与者

- 通过以下问题来识别Actor：
  - 谁使用这个系统的功能？
  - 谁从该系统获得信息？
  - 谁向该系统提供信息？
  - 该系统需要访问(读写)那些外部硬件设备？
  - 谁来负责维护和管理这个系统以保证其正常运行？
  - 该系统需要与其他系统进行交互吗？

# 用户故事与用例建模

## 识别并描述参与者

- 例1：对一个图书馆管理系统来说，有哪些参与者？
  - 普通读者
  - 图书管理员




- 例2：对ATM系统来说，有哪些参与者？
  - 银行客户
  - ATM维护人员

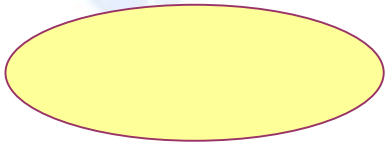
# 用户故事与用例建模

## 特殊的参与者：系统时钟

- 有时候需要在系统内部定时的执行一些操作，如检测系统资源使用情况、定期生成统计报表等等
- 但这些操作并不是由外部的人或系统触发的
- 对于这种情况，可以抽象出一个系统时钟或定时器参与者，利用该参与者来触发这一类定时操作
- 从逻辑上，这一参与者应该被理解成是系统外部的，由它来触发系统所提供的用例对话

# Step 2：识别用例(use case)

- 找到参与者之后，据此来确定系统的用例，主要是看各参与者需要系统提供什么样的服务，或者说参与者是如何使用系统的
- 寻找用例可以从以下问题入手(针对每一个参与者)：
  - 参与者使用该系统执行什么任务？
  - 参与者是否会在系统中创建、修改、删除、访问、存储数据？如果是的话，参与者又是如何来完成这些操作的？
  - 参与者是否会将外部的某些事件通知给该系统？
  - 系统是否会将内部的某些事件通知该参与者？



Use case

## 识别用例

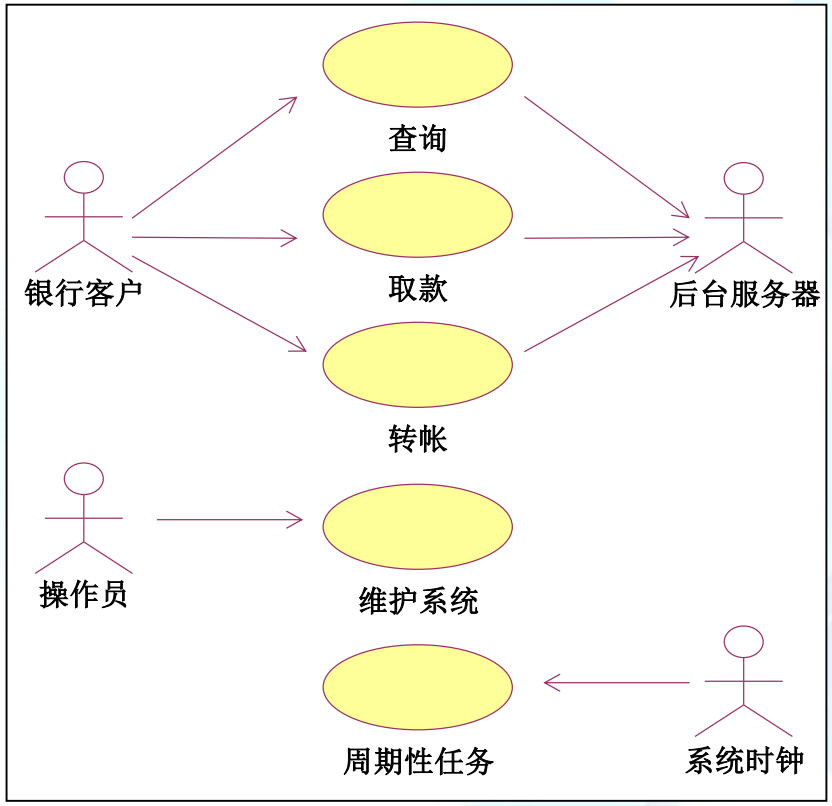
- 例1：对图书馆管理系统来说，有哪些用例？
  - 图书管理员
    - 登录
    - 管理读者信息
    - 管理图书信息
    - 登记借书
    - 登记还书
  - 普通读者：
    - 登录
    - 预订图书
    - 取消预订
    - 查询浏览图书信息
- 例2：对ATM系统来说，有哪些参与者？
  - 银行客户
    - 查询
    - 取款
    - 转帐
  - ATM维护人员
    - 维护系统

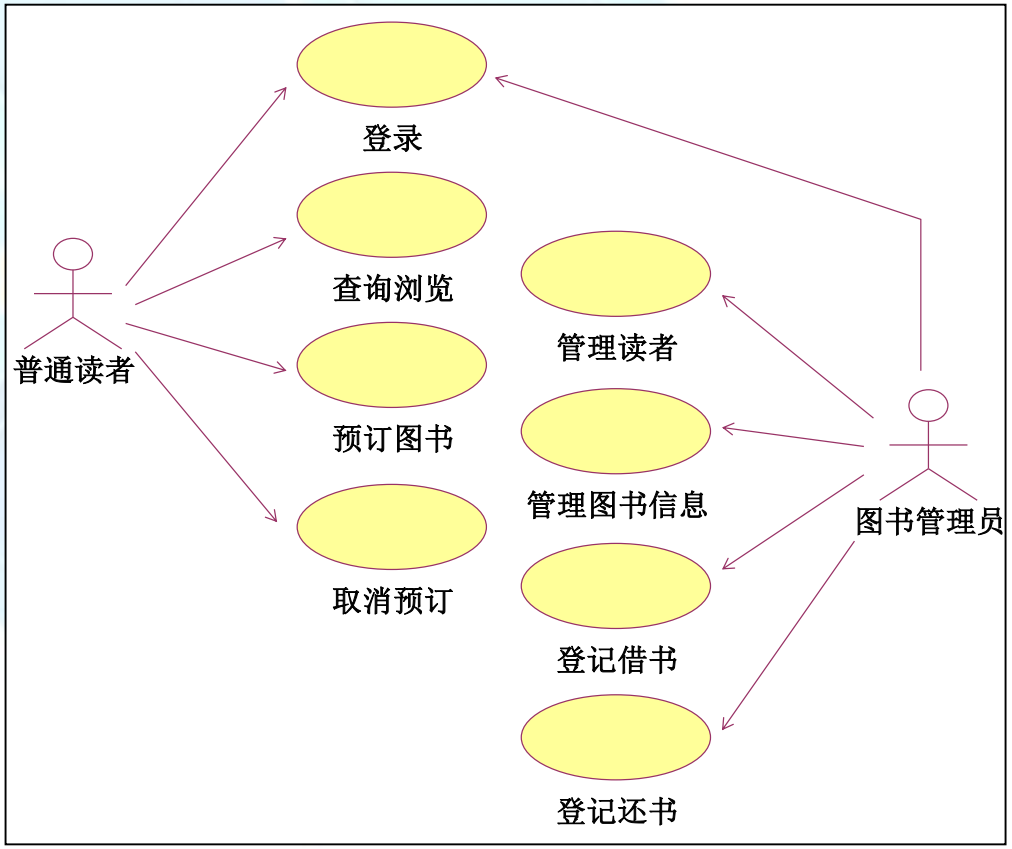
# 识别用例的几点注意事项

- 用例必须是由某一个actor触发而产生的活动，即每个用例至少应该涉及一个actor
- 如果存在与actor不进行交互的用例，需要将其并入其他用例，或者是检查该用例相对应的参与者是否被遗漏
- 反之，每个参与者也必须至少涉及到一个用例，如果发现有不与任何用例相关联的参与者存在：
  - 仔细考虑该参与者是如何与系统发生对话的
  - 由参与者确定一个新的用例
  - 该参与者是一个多余的模型元素，应该将其删除

# 用户故事与用例建模

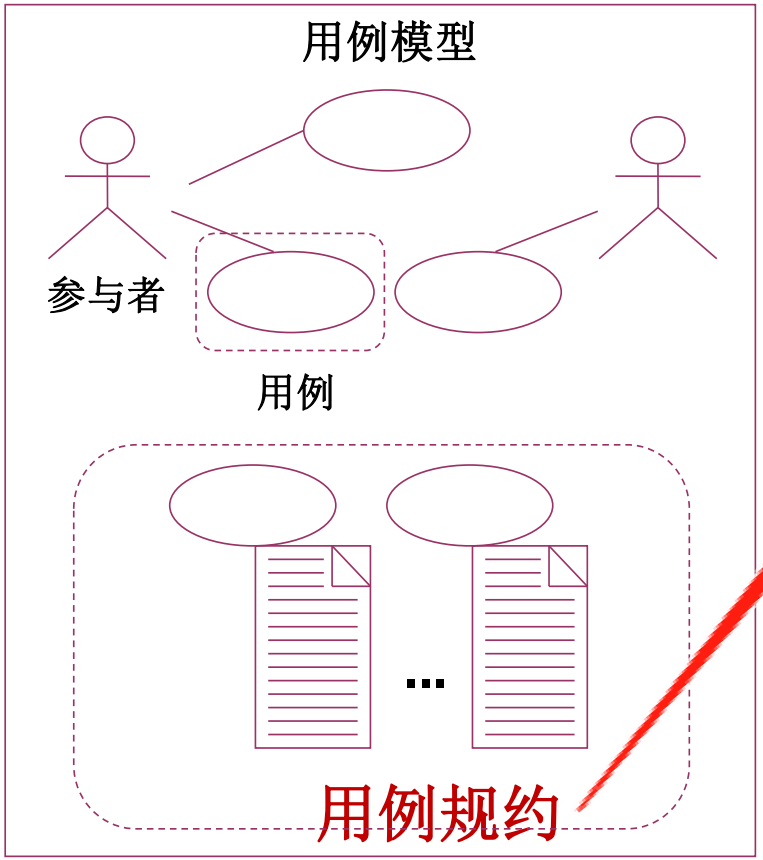
## Step 3：识别参与者与用例之间的通讯关联



# Step 4：给出用例的详细描述

- 单纯的用例图并不能描述完整的信息，需要用文字描述不能反映在图形上的信息

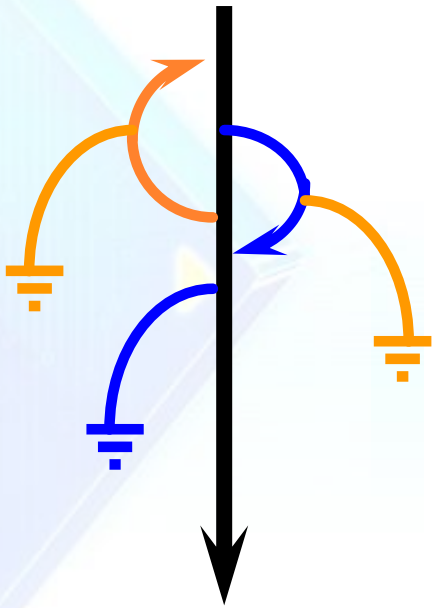



**Name of the Use Case (用例的名字)**

Description (描述)
Actor(s) (参与者)
Flow of events(事件流)
  Basic flow(常规流)
    Event 1 (事件)
    Event 2
  ……
  Alternate flow(备选流)
Pre-conditions (前置条件)
Post-conditions (后置条件)
……

# 事件流

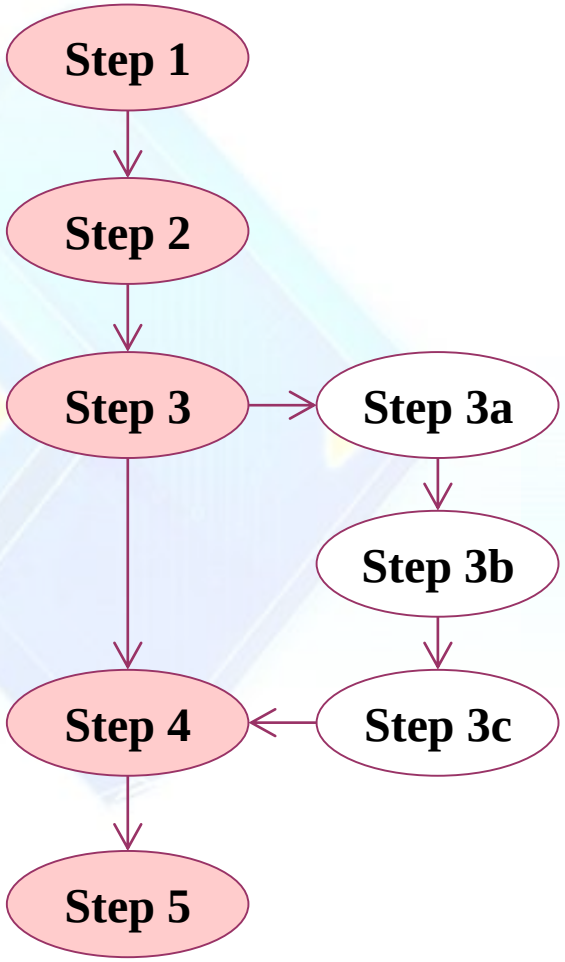
- 用例的事件流：
  - 说明用例如何启动，即哪些参与者在何种情况下启动用例？
  - 说明参与者与用例之间的信息处理过程
  - 说明用例在不同条件下可以选择执行的多种方案
  - 说明用例在什么情况下才能被视作完成
- 分为常规流和备选流两类：
  - 常规流：描述该用例最正常的一种场景，系统执行一系列活动步骤来响应参与者提出的服务请求
  - 备选流：负责描述用例执行过程中异常的或偶尔发生的一些情况



# 用户故事与用例建模

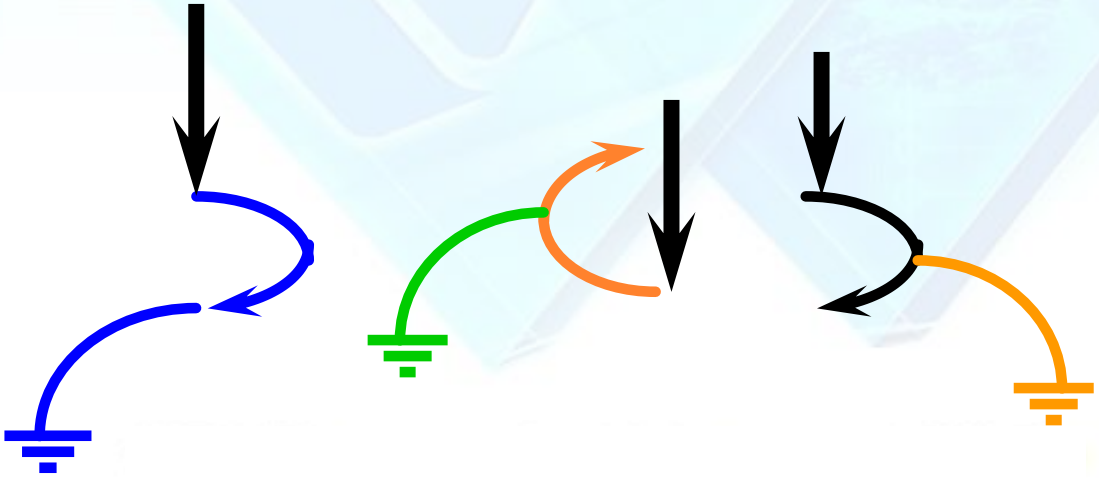
## 常规事件流

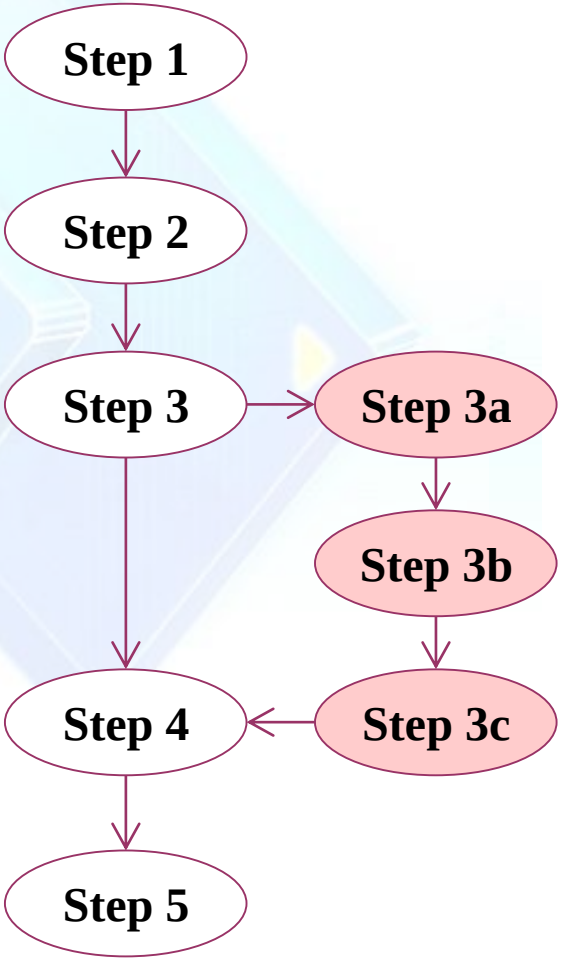
- 每一个步骤都需要用数字编号以清楚地标明步骤的先后顺序
- 用一句简短的标题来概括每一步骤的主要内容
- 对每一步骤，从正反两个方面来描述
  - 参与者向系统提交了什么信息
  - 对此系统有什么样的响应



# 备选事件流

- 备选流的描述格式可以与基本流的格式一致，也需要编号并以标题概述其内容
  - 起点：该备选流从事件流的哪一步开始
  - 条件：在什么条件下会触发该备选流
  - 动作：系统在该备选流下会采取哪些动作
  - 恢复：该备选流结束之后，该用例应如何继续执行





# 用户故事与用例建模

## [案例]用例描述

### 用例：登记借书

1. 目标：

本用例允许图书管理员登记普通读者的借书记录

2 事件流：

2.1 常规流程

当读者希望借书、图书管理员准备登记有关的借书记录时，本用例开始执行

(1) 系统要求管理员输入读者的注册号和所借图书号；
(2) 图书管理员输入信息后，系统产生一个唯一的借书记录号；
(3) 系统显示新生成的借书记录；
(4) 图书管理员确认后，系统增加一个新的借书记录

2.2 备选流程

(1) 读者没有注册：

在主流程中，如果系统没有读者的注册信息，系统将显示错误信息，用例结束

(2) 所借图书不存在

在主流程中，如果所借图书已被借出或系统中无该图书，系统将显示错误信息，用例结束

3 前提条件：用例开始前，图书管理员必须在系统登录成功

4 后置条件：如果用例执行成功，该读者的借书记录被更新，否则，系统状态不变

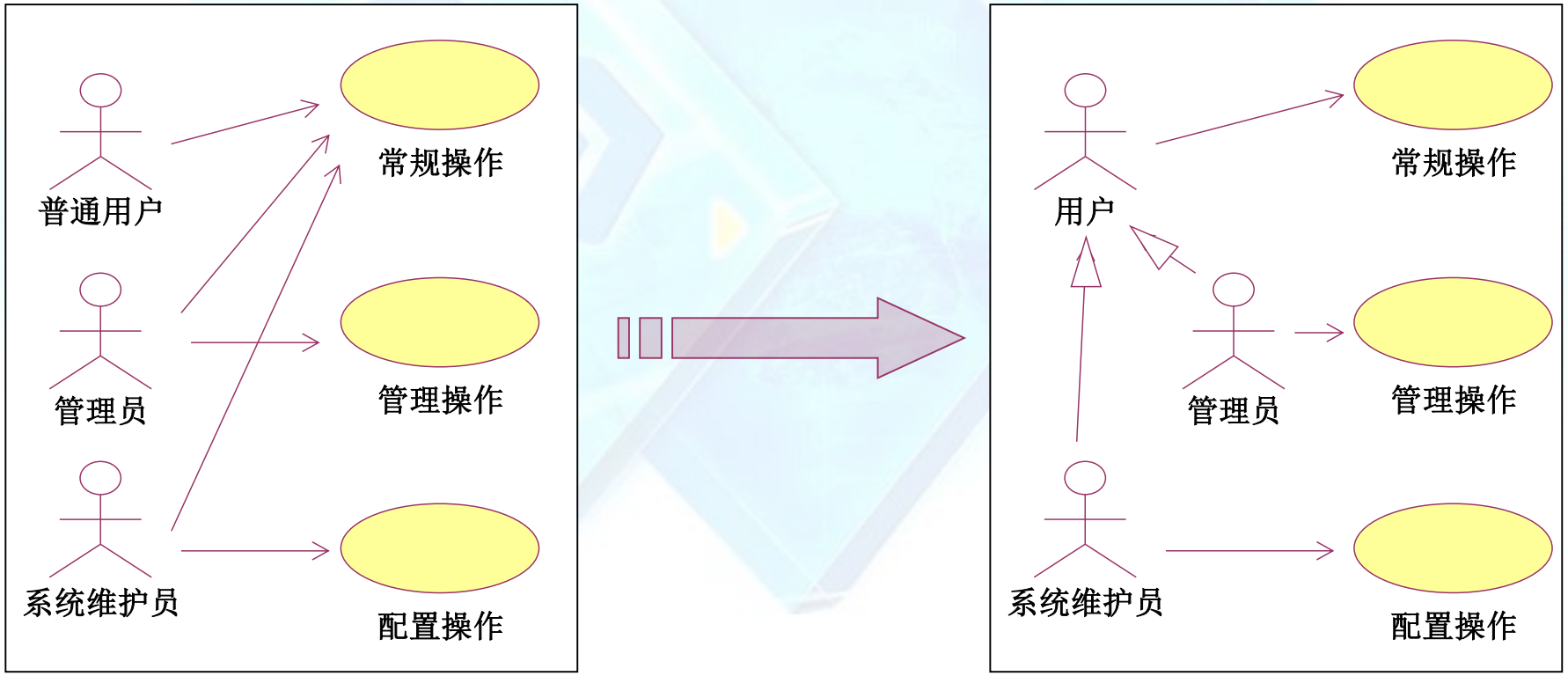
# Step 5：细化用例模型

- 在一般的用例图中，只需表述参与者和用例之间的通讯关联
- 除此之外，还可以描述：
  - 参与者与参与者之间的泛化(generalization)
  - 用例和用例之间的包含(include)
  - 用例和用例之间的扩展(extend)
  - 用例和用例之间的泛化(generalization)关系
- 利用这些关系来调整已有的用例模型，把一些公共的信息抽取出来复用，使得用例模型更易于维护

# 用户故事与用例建模

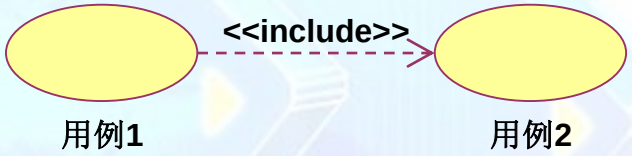
## 参与者之间的关系

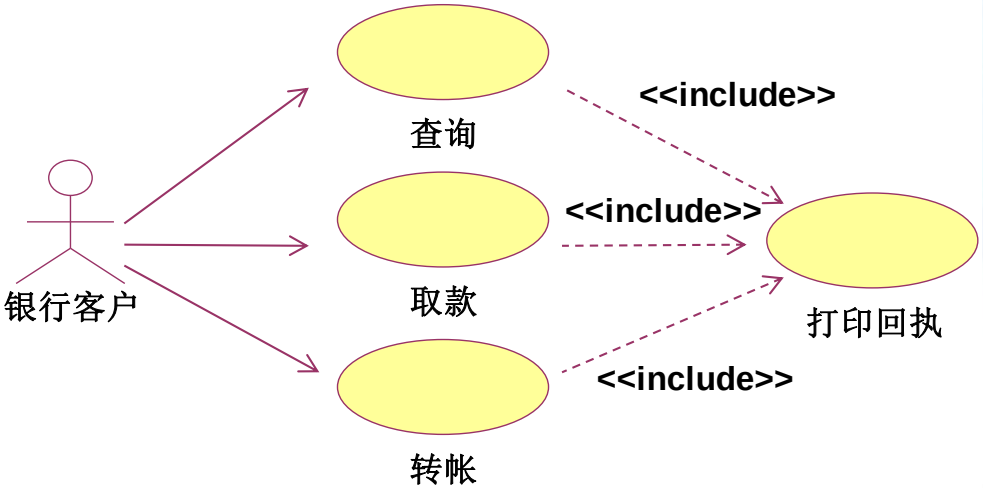
- 参与者之间也可以有泛化(Generalization)关系



# 用例之间的关系：包含(include)

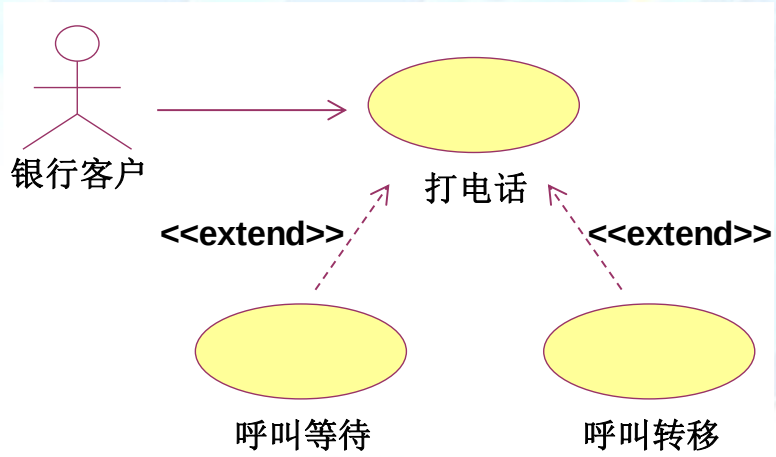
- “包含关系”是通过在关联关系上加入<<include>>标记来表示
- 语义：用例1会用到用例2，用例2的事件流将被插入到用例1的事件流中



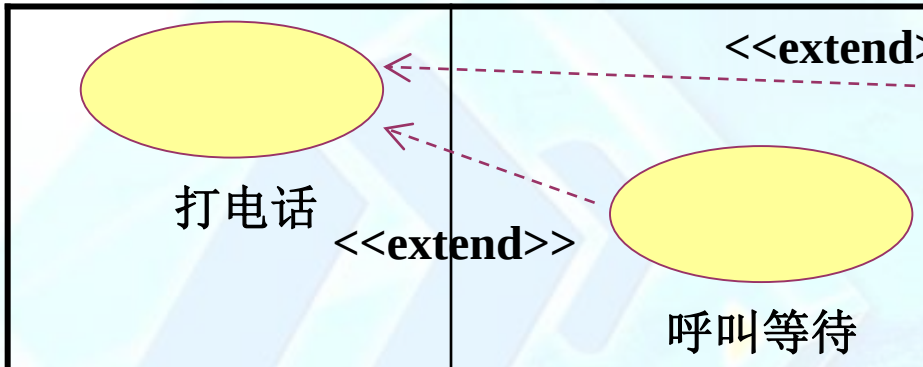
# 用例之间的关系：扩展(extend)

- “扩展关系”是通过在关联关系上加入<<extend>>标记来表示
- 语义：用例2在某些特定情况下会用到用例1，此时，用例1的事件流将被插入到用例2的事件流中



# 用户故事与用例建模

## 用例之间的关系：扩展(extend)

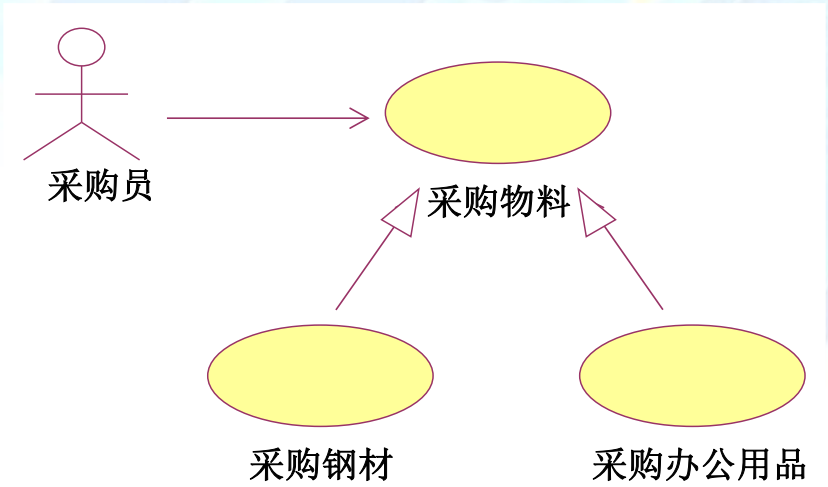



| | | |
|---|---|---|
| 常规流：<br>1 拨号<br>2 建立通话链路<br>3 通话<br>4 挂机 | 常规流：<br>1 如果应答方正忙，用铃声提示应答方并保持拨号呼叫 | 常规流：<br>1 如果应答方无应答，进行呼叫转移 |

实际上相当于第一个用例的“备选流”

# 用例之间的关系：泛化(generalization)

- 当多个用例共同拥有一种类似的结构和行为的时候，可将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例
- 子用例继承了父用例所有的结构、行为和关系



# 用户故事与用例建模

## “用例的关系”

- 参与者与参与者之间的泛化(generalization)
- 用例和用例之间的包含(include)
- 用例和用例之间的扩展(extend)
- 用例和用例之间的泛化(generalization)关系

- 为什么要引入上述关系？有什么优越性？

# 用户故事与用例建模

## 需注意问题1：用例的粒度

- 用例识别的标准：actor与系统之间的一次独立交互
  - 如果多次交互总是同时发生且不会单独发生，可合并为一个用例
- 用例也可称为user story，将来使用时可看作一个独立存在的功能体
- 例如“查询/浏览宝贝或者店铺”，这个用例过大，拆分为：
  - 按类别浏览宝贝
  - 按关键字查询宝贝
  - 浏览宝贝的详细信息（商品特性、评价信息、已售出信息、etc）
  - 查询店铺
  - 浏览店铺的详细信息
- 例如“卖家对发布宝贝进行管理”：什么是“管理”？
  - 上架宝贝、下架宝贝、暂停销售
  - 修改宝贝描述、修改当前库存、设定价格等

# 需注意问题2：用例是actor与系统的交互

- 用例是actor与系统的交互：
  - actor对系统发出的请求
  - 系统对actor请求的响应
- actor与actor在现实当中的交互不应包含在use case中
  - 例如“讨价还价”：
    并非系统提供的交互，而是二者在外部系统（旺旺）内完成的
  - 例如“处理纠纷”和“处理投诉”：
    需要细化为淘宝客服与买家、卖家在淘宝系统中的操作/交互；若完全是人工操作，无需在系统中出现，只需要管理处理的结果即可

# 需注意问题3：actor与外部系统的区分

- 买家
- 卖家
- 淘宝平台管理员（小二）
- 物流系统
- 支付宝系统
- 前三者是user，后二者是external system

- “淘宝平台”就是你所面对的系统，既不是user也不是外部系统
- 系统自己做的事情，不是单独的用例：系统的行为受到actor的触发(可能是系统时钟)

# 用户故事与用例建模

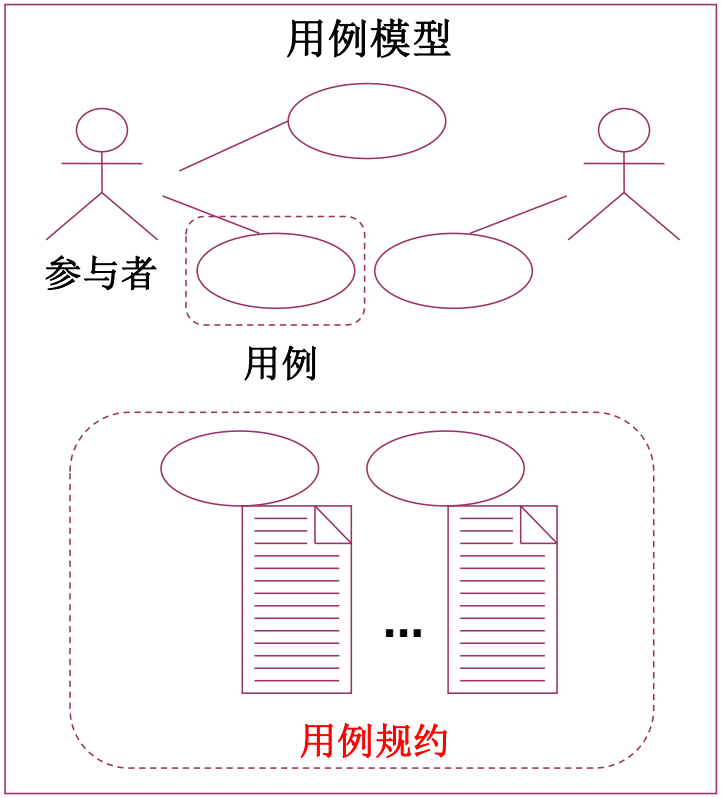
## 本章主要内容

1. 敏捷开发中的“用户故事”（User Story）
2. 面向对象方法中的“用例”（Use Case）
3. 用例建模的基本过程
4. 用例模型的提交物
5. 活动图 & 泳道图

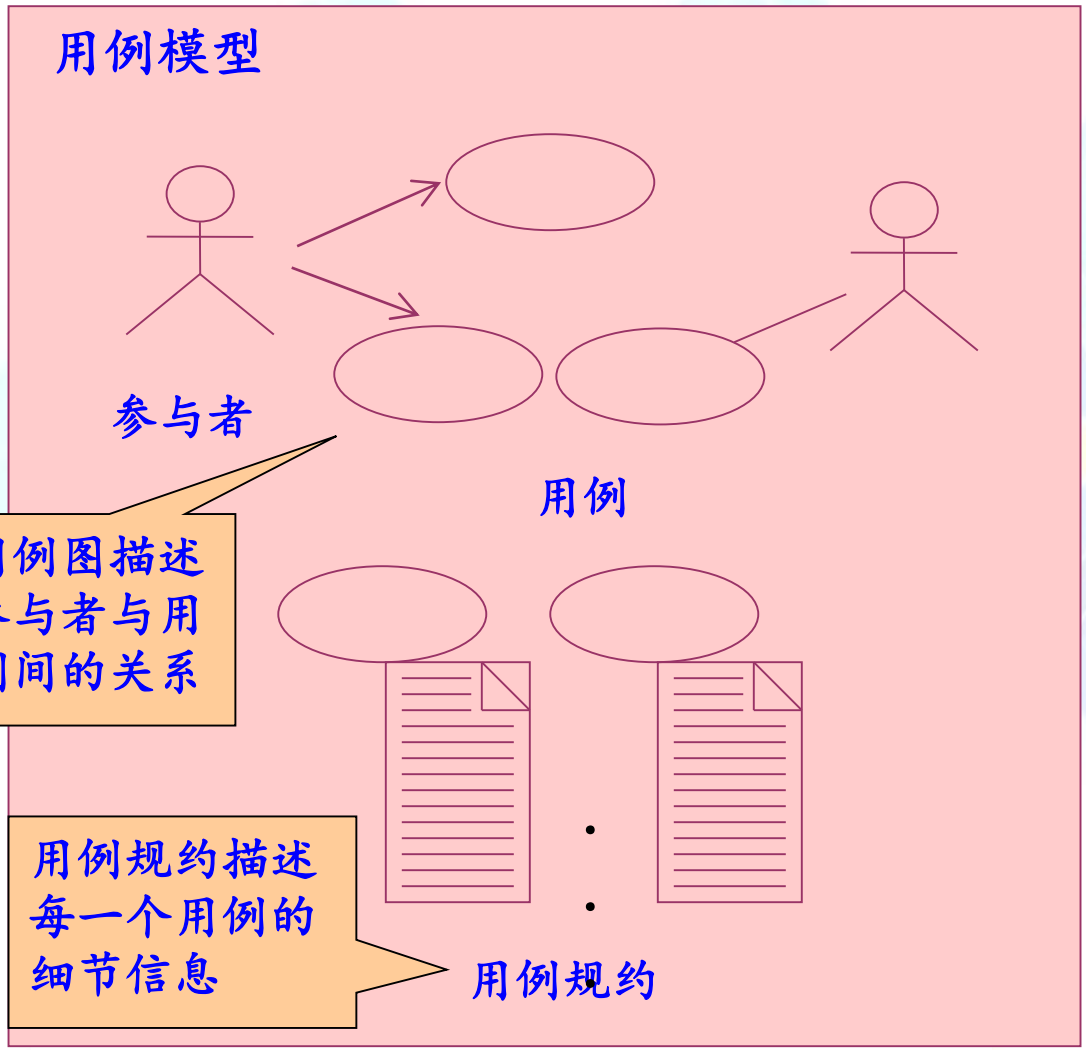
# 用户故事与用例建模

## 用例模型的提交物

- 1 用例模型
- 2 每个用例的详细描述
- 3 术语表：所用到的术语说明
- 4 补充规约：非功能性需求的说明



# 用户故事与用例建模




用例图描述参与者与用例间的关系

用例规约描述每一个用例的细节信息

术语表

记录一些系统需求相关的术语

补充规约

记录一些全局性的功能需求、非功能性需求和设计约束等

# 用户故事与用例建模

## 本章主要内容

1. 敏捷开发中的“用户故事”（User Story）
2. 面向对象方法中的“用例”（Use Case）
3. 用例建模的基本过程
4. 用例模型的提交物
5. 活动图 & 泳道图

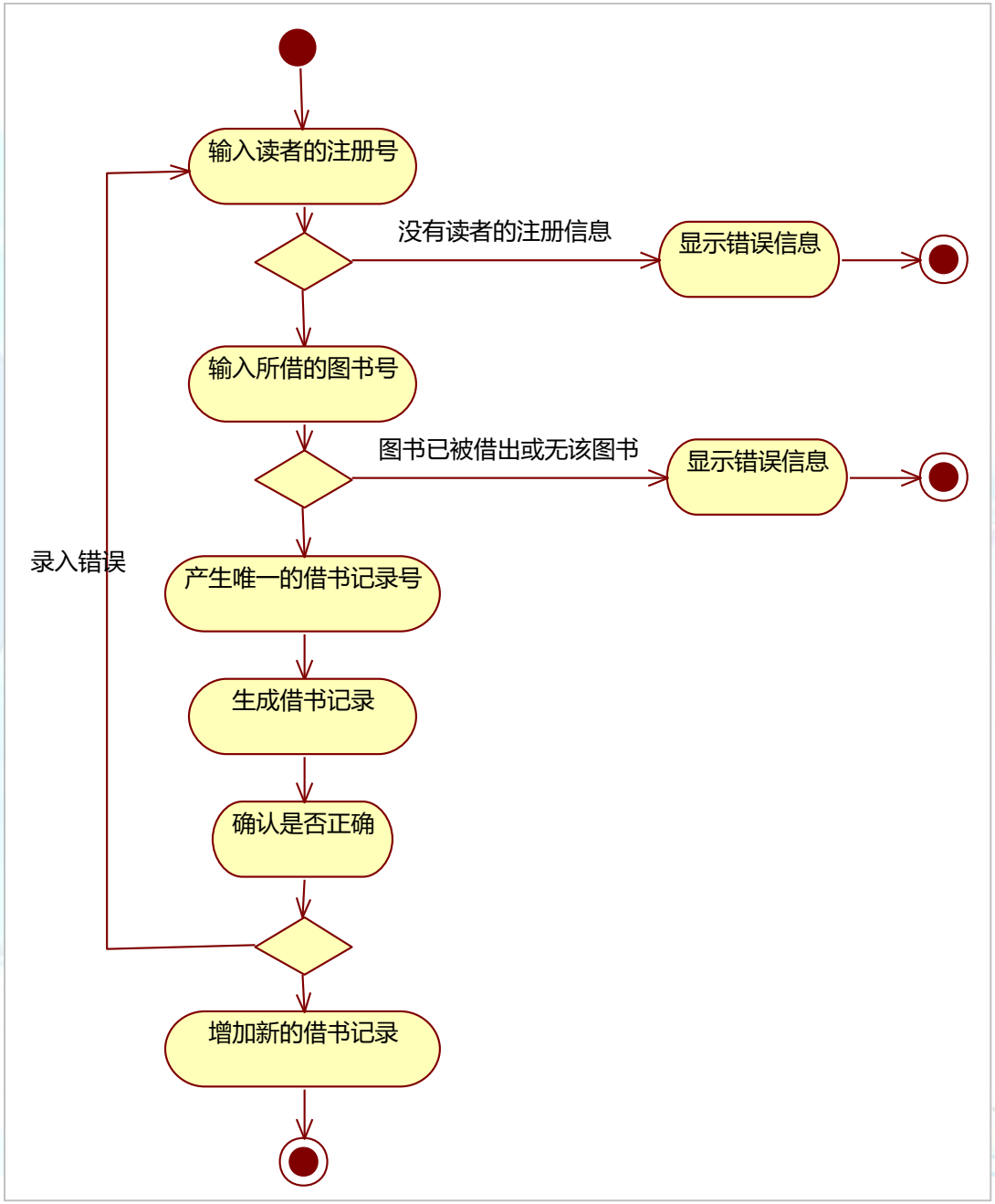
# 用户故事与用例建模

## 活动图&泳道图：对用例描述的图形化补充

- UML活动图(Activity Diagram)提供一种可视化的流程图方式，对use case的事件流进行直观展示，以便于读者更好的理解
- 同时，UML活动图也可以用来描述多个用例之间所形成的大粒度流程
- 两种形式：
  - 传统的活动图：只涉及一个参与者
  - 泳道图(swim-lane diagram)：侧重于描述多个参与者的活动之间的交互关系
- 活动图的基本要素：
  - 起始点、结束点
  - 活动；决策点
  - 活动之间的时序连接；活动之间的并发点
  - 泳道

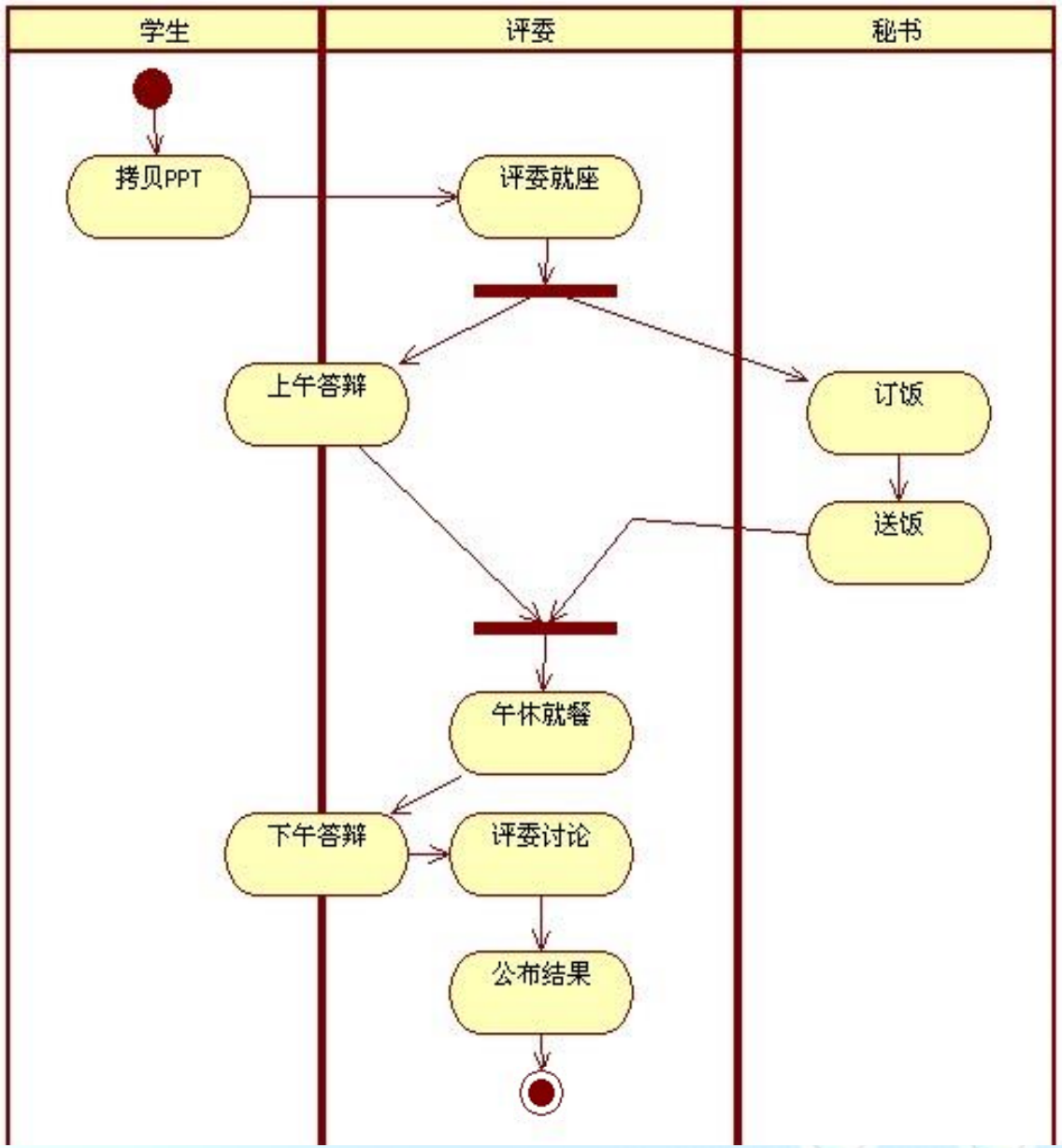
# 用户故事与用例建模

## UML活动图



# 用户故事与用例建模

## UML泳道图



# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 09

# 结构化系统分析

## 本章主要内容

1. 结构化方法vs面向对象方法
2. 结构化的系统分析方法
3. 数据流图（DFD）
4. 数据字典（DD）
5. 数据分析（ERD、IDEF1X）

# 结构化系统分析

## 本章主要内容

1. 结构化方法vs面向对象方法
2. 结构化的系统分析方法
3. 数据流图（DFD）
4. 数据字典（DD）
5. 数据分析（ERD、IDEF1X）

# 结构化系统分析

## 软件工程方法：结构化VS面向对象

### 结构化方法

- 复杂世界 → 复杂处理过程（事情的发生发展）
- 设计一系列功能（或算法）以解决若干问题
- 寻找适当的方法存储数据

### 面向对象方法

- 任何系统都是由能够完成一组相关任务的对象构成
- 如果对象依赖于一个不属于它负责的任务，那么就需要访问负责此任务的另一个对象（调用其他对象的方法）
- 一个对象不能直接操作另一个对象内部的数据，它也不能使其它对象直接访问自己的数据
- 所有的交流都必须通过方法调用

# 结构化系统分析

## 软件工程方法：结构化VS面向对象

### 举例：五子棋游戏

- 面向过程（事件）的设计思路是首先分析问题的步骤：
  1. 开始游戏，初始化画面（棋盘）
  2. 黑子走，绘制画面（棋盘）
  3. 判断输赢，如分出输赢，跳至步骤6
  4. 白子走，绘制画面（棋盘）
  5. 判断输赢，如未分出输赢，返回步骤2
  6. 输出最后结果
- 面向对象的设计思路是分析与问题有关的对象：
  1. 玩家：黑白双方，这两方的行为是一模一样的
  2. 棋盘：负责绘制画面
  3. 规则：负责判定诸如犯规、输赢等

# 结构化系统分析

## 软件工程方法：结构化VS面向对象

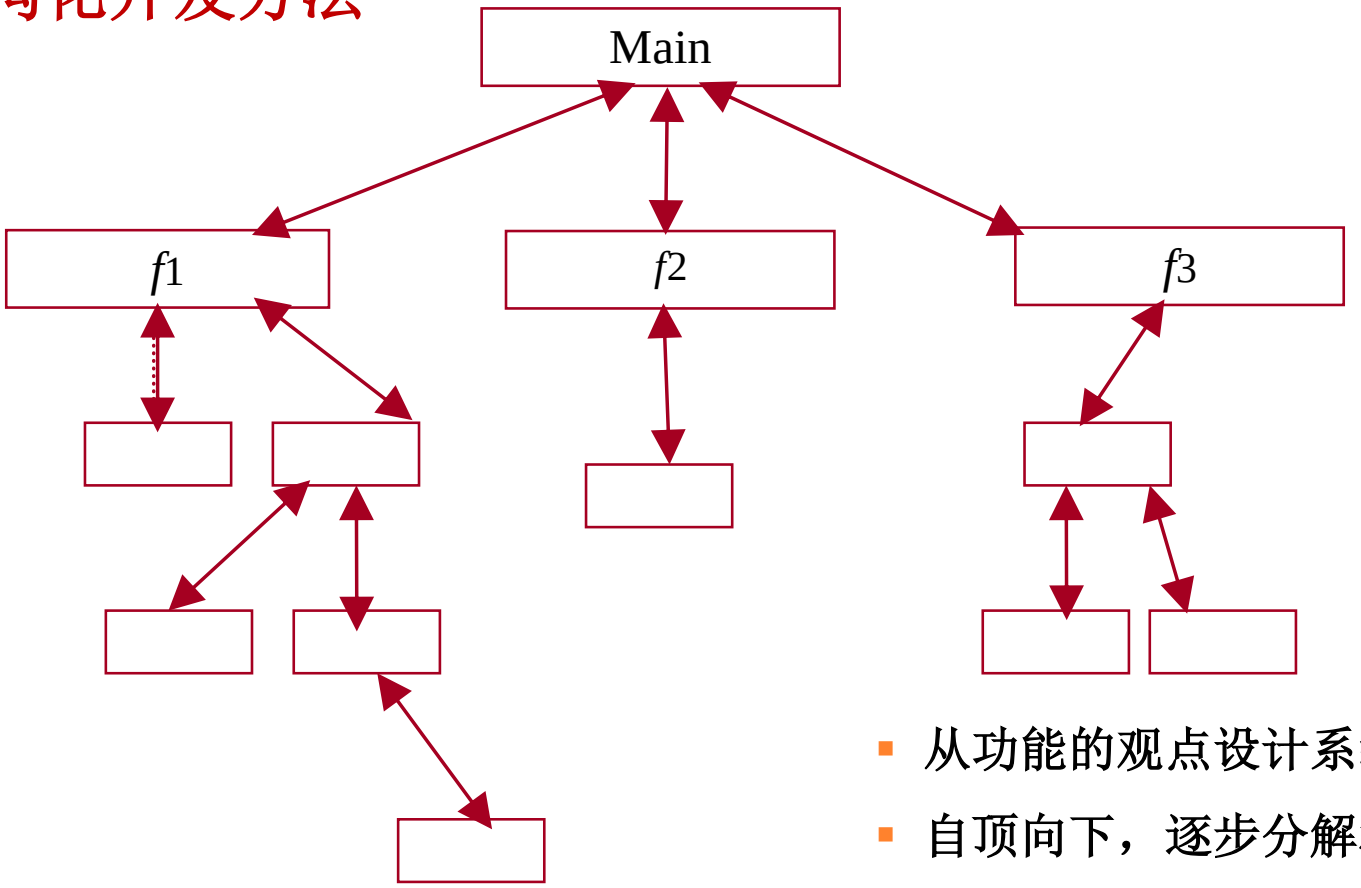
### 结构化开发方法

- 起源时间：20世纪60年代产生软件危机，70年代出现“结构化开发方法”
- 思想方法：自顶向下、问题分解、分而治之、由分到合；
  软件/程序 = 数据结构 + 算法，软件结构化、模块化、层次化
- 表达模型：分析模型（系统流程图、数据流图、 ERD、数据字典）
  设计模型（系统结构图、程序流程图、ERD）
- 分析与设计线索：
  面向过程（处理） – 过程驱动
  面向数据 – 数据驱动
- 优　　点：思维自然，符合人们思考问题的方式；总体可控性强；适合偏重数学计算方面的项目
- 缺　　点：不容易描述客观世界的需求；耦合性相对高

# 结构化系统分析

## 软件工程方法：结构化VS面向对象

### 结构化开发方法




- 从功能的观点设计系统
- 自顶向下，逐步分解和细化
- 将大系统分解为若干模块，主程序调用这些模块实现完整的系统功能

# 软件工程方法：结构化VS面向对象

## 面向对象开发方法

- 起源时间：20世纪80年代（70年代开始OOP）
- 思想方法：从客观世界的具体事物出发构建系统；
  自底向上，先考虑“对象”，再考虑“关系”
- 表达模型：类层次结构图（对象的属性、行为、继承、消息连接等）
- 优　　点：符合自然世界的状态，描述自然，思维简单
  适合复杂的事务处理、大量信息处理类的项目
  耦合性容易降低，容易复用
- 缺　　点：OO模型表达系统需求分析和设计不够充分

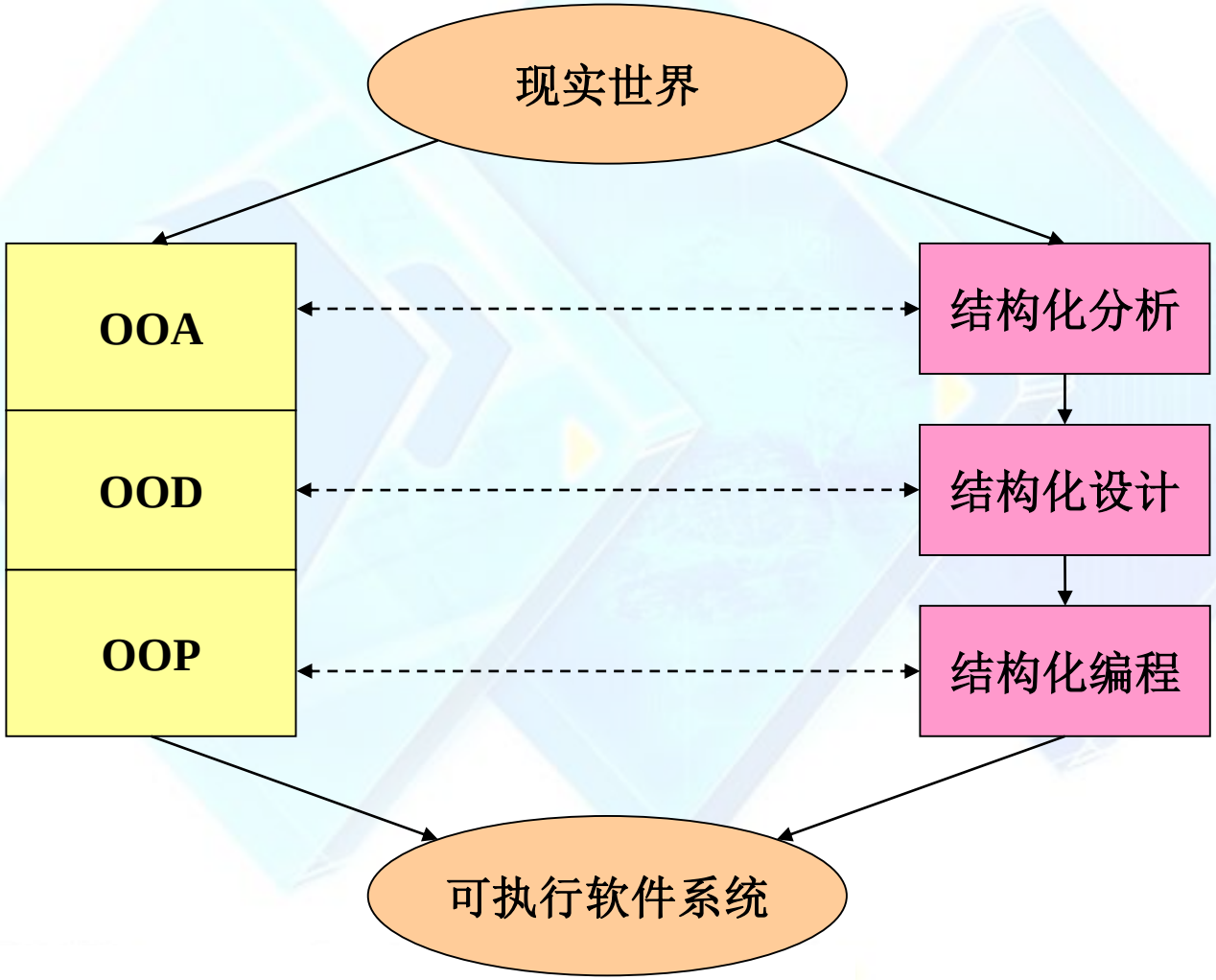
# 软件工程方法：结构化VS面向对象

## 基于UML的面向对象开发方法

- 起源时间：20世纪90年代末
- 思想方法：开发一整套系统分析与设计模型及描述方法，从形式上和概念上统一描述软件系统
- 表达模型：用例图、活动图、状态图、序列图（协作图）、类图、组件图、包图、部署图等
- 优　　点：统一标准；容易交流；建模工具较多；适合大型信息系统发
- 缺　　点：模型不容易掌握

# 结构化系统分析

## 软件工程方法：结构化VS面向对象



# 结构化系统分析

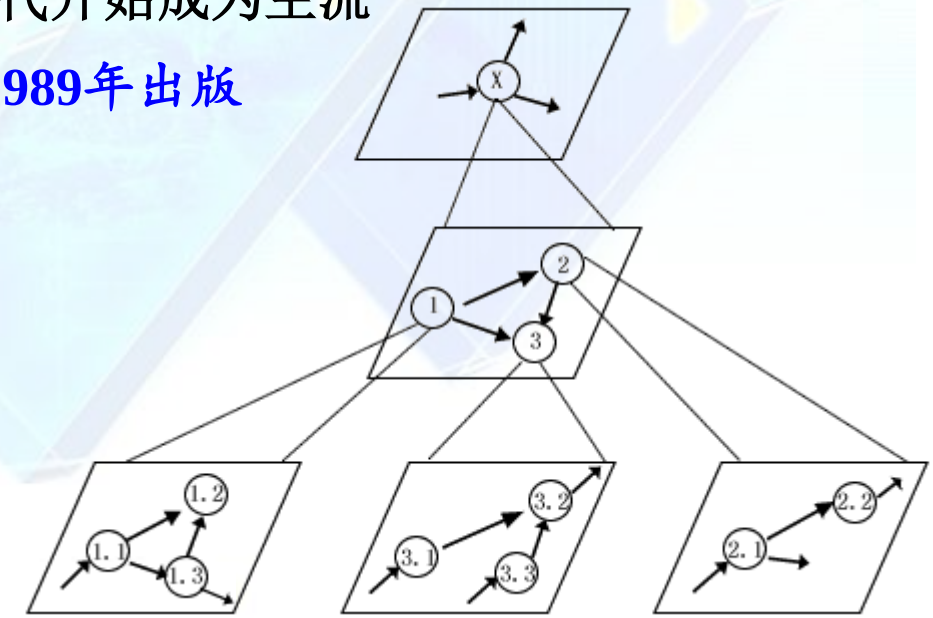
## 本章主要内容

1. 结构化方法vs面向对象方法
2. 结构化的系统分析方法
3. 数据流图（DFD）
4. 数据字典（DD）
5. 数据分析（ERD、IDEF1X）

# 结构化系统分析

## 结构化的系统分析方法

- 结构化分析方法(SA)：将待解决的问题看作一个系统，从而用系统科学的思想方法(抽象、分解、模块化)来分析和解决问题
  - 起源于结构化程序设计语言(事先设计好每一个具体的功能模块，然后将这些设计好的模块组装成一个软件系统)
  - 以动词性的“功能”为核心展开分解
- 最早产生于1970年代中期，1980年代开始成为主流
  - Yourdon（美国软件工程师）于1989年出版《Modern Structured Analysis》



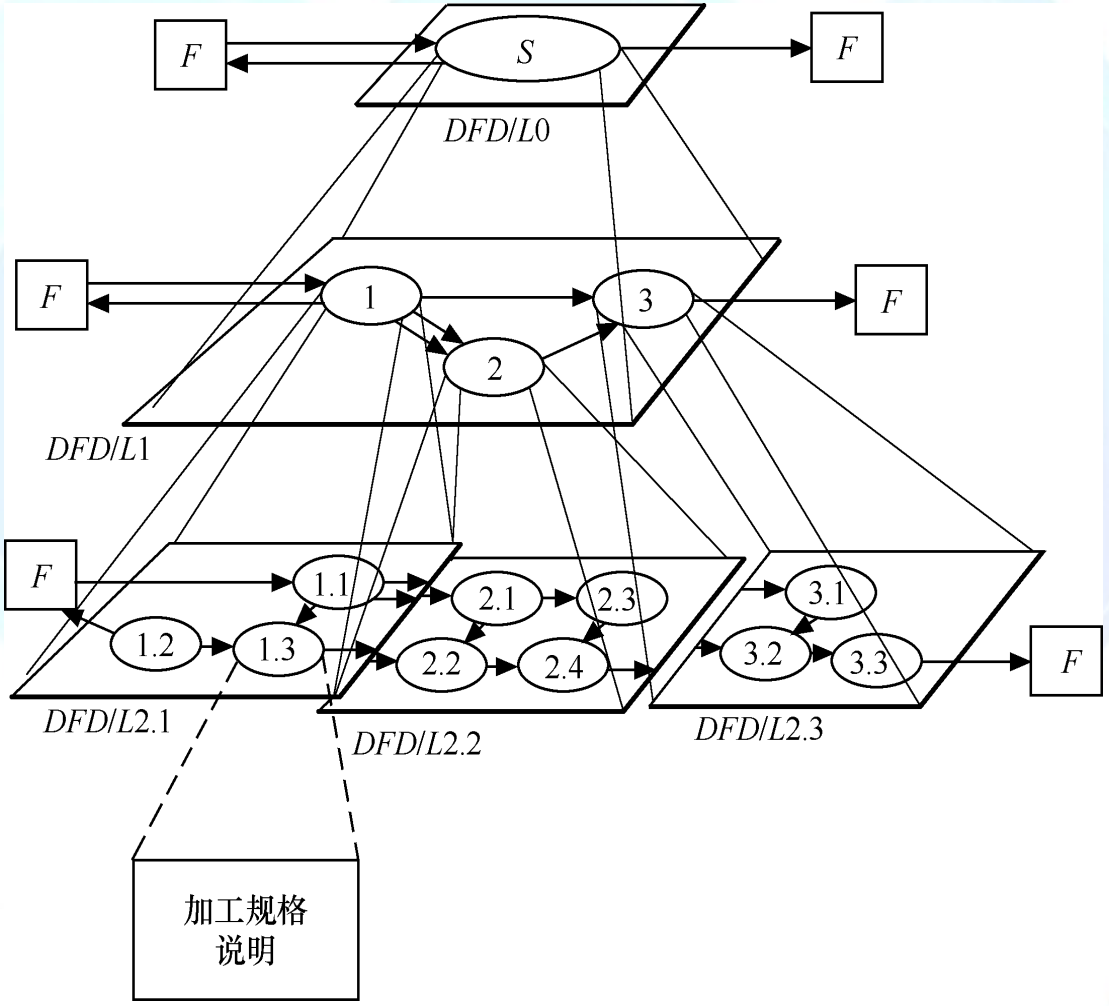

- 核心思想：
  - 自顶向下的分解(top-down)

# 结构化的系统分析方法

- 结构化分析：帮助开发人员定义系统需要做什么（处理需求），系统需要存储和使用哪些数据（数据需求），系统需要什么样的输入和输出以及如何把这些功能结合在一起来完成任务
  - 数据流图（DFD图）
  - 实体-关系图（ERD，IDEF1X图）

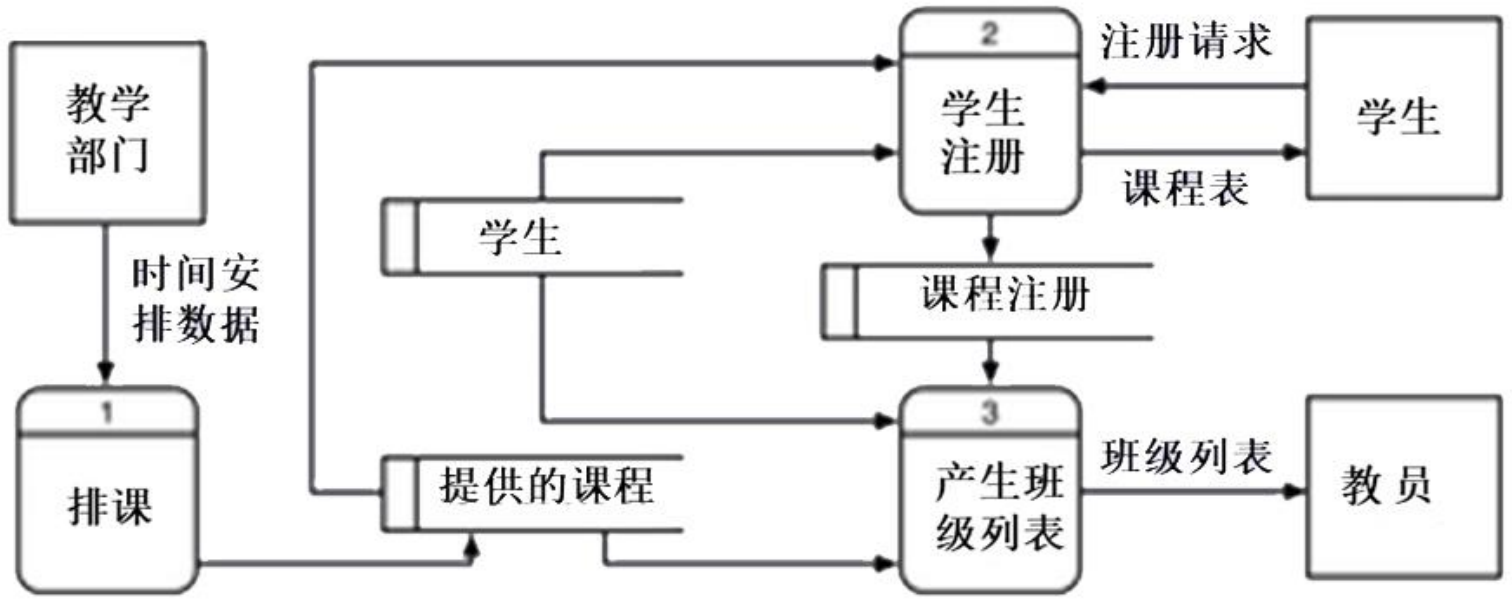
# 结构化的系统分析方法

## 基于数据流的需求分析建模 -- DFD



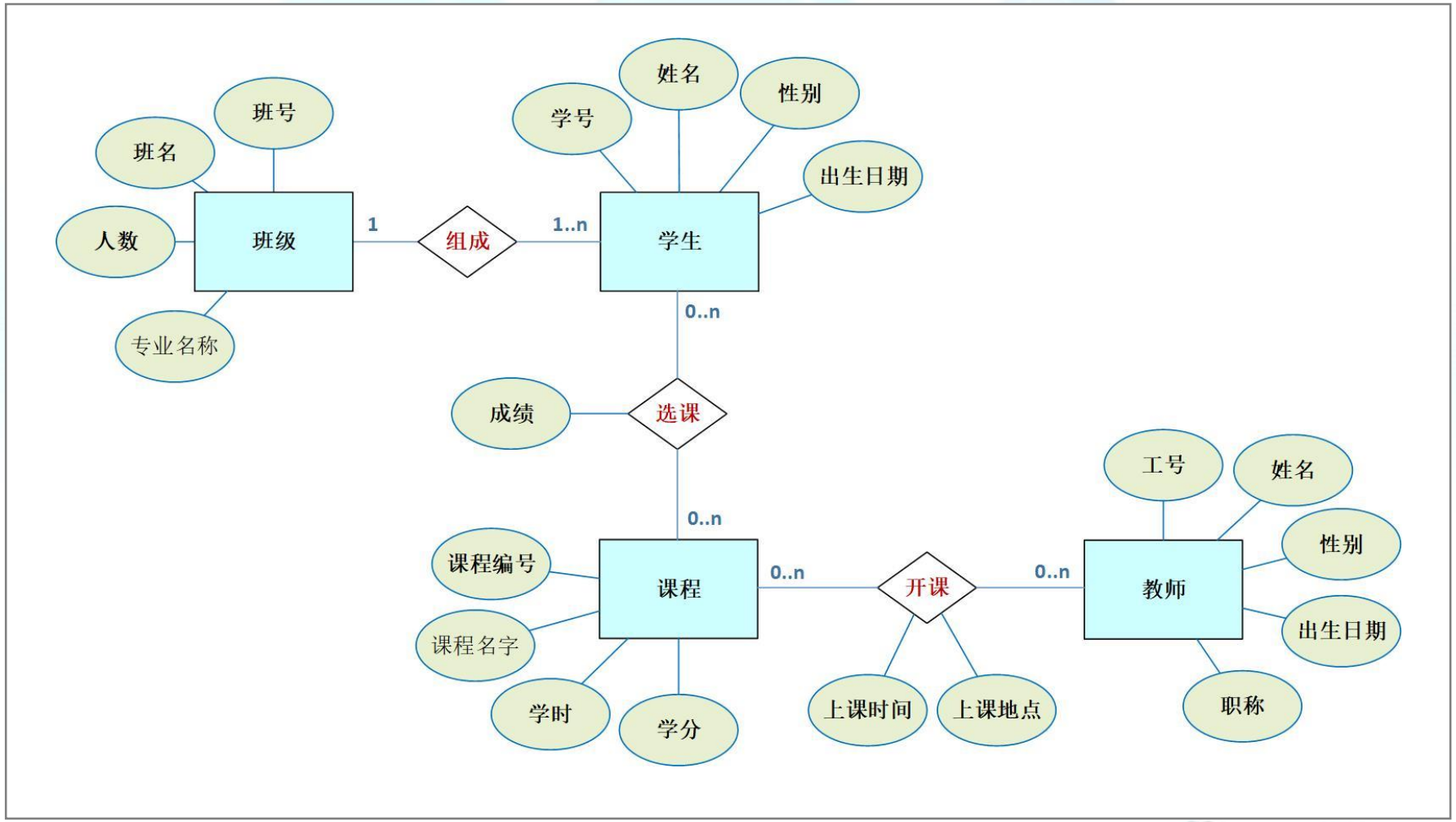
# 结构化的系统分析方法

## 基于数据流的需求分析建模 -- DFD



# 结构化的系统分析方法

## 数据分析建模 -- ERD

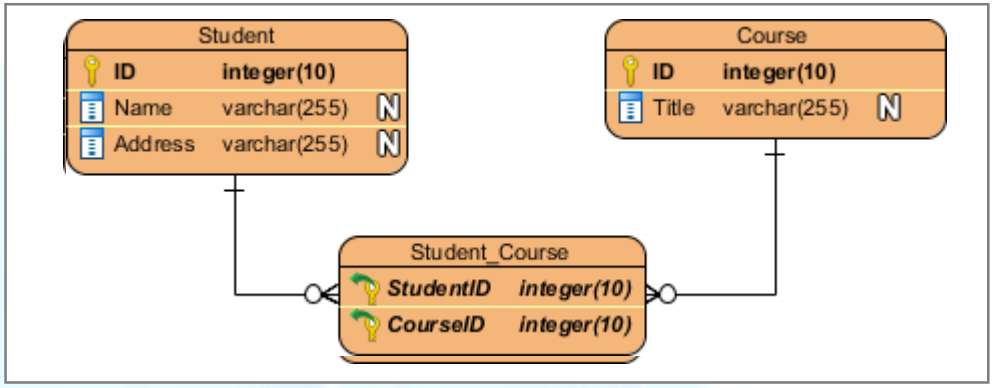


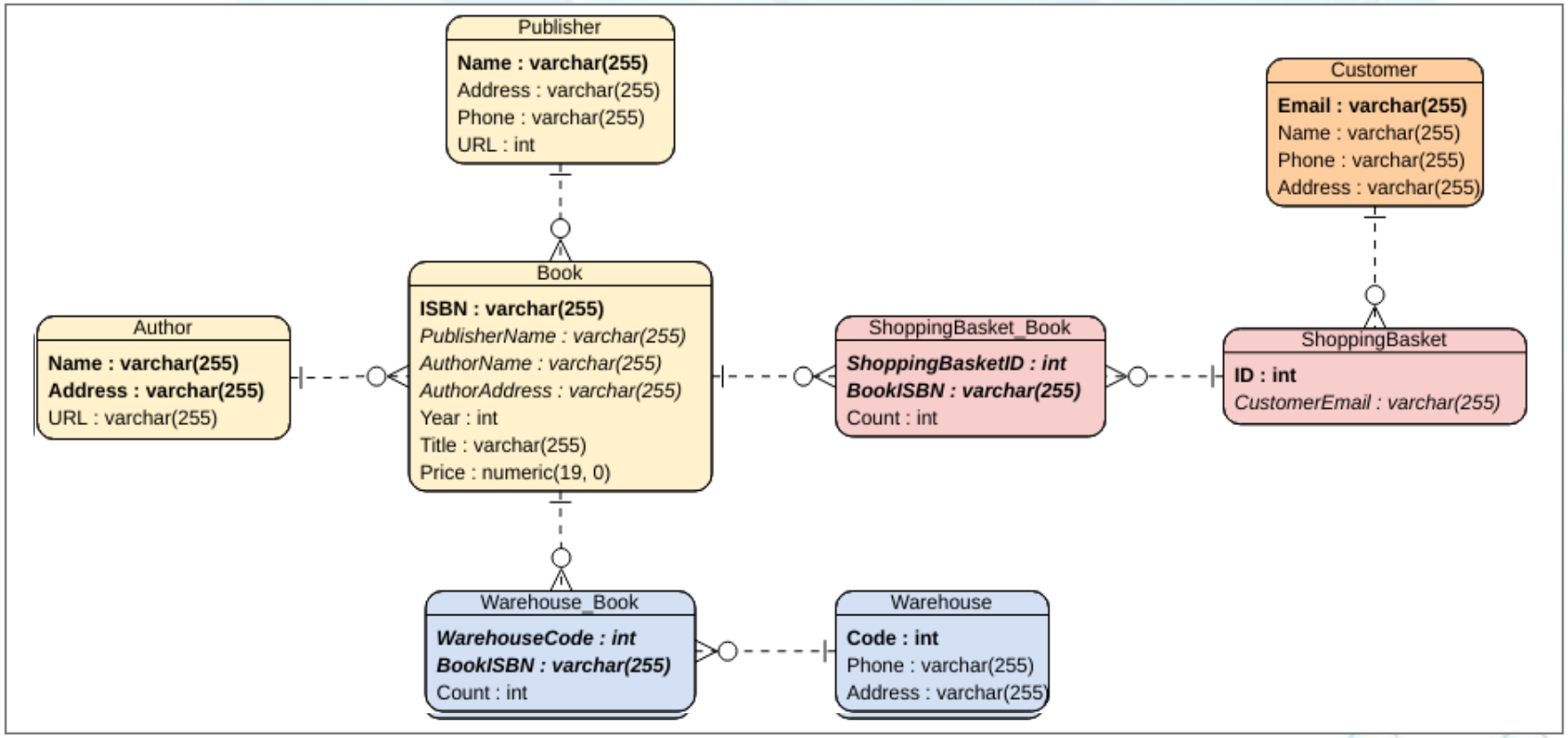

传统的Chen's ERD

# 结构化系统分析

## 结构化的系统分析方法

### 数据分析建模 -- ERD

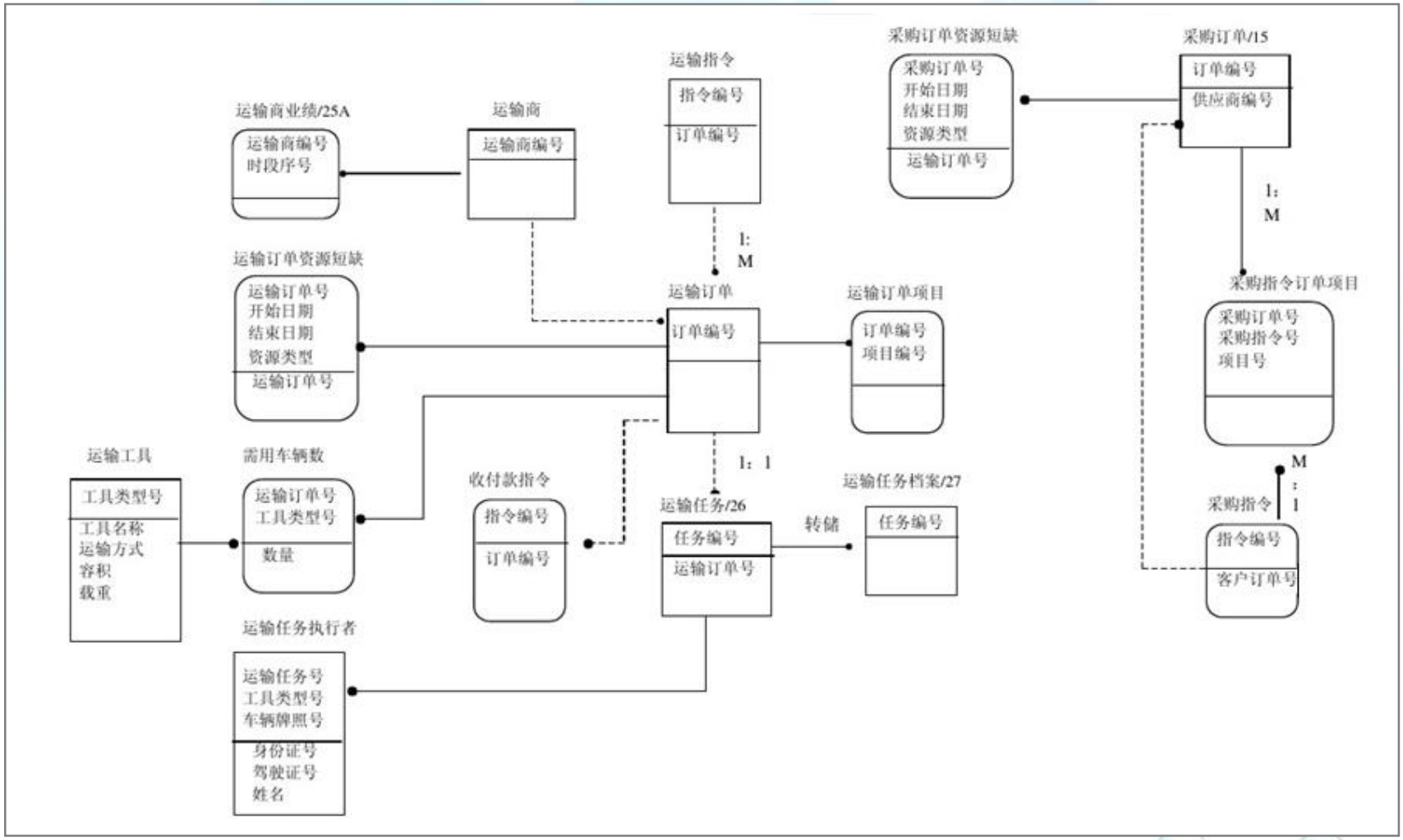



采用“乌鸦脚（Crow's Foot）”形式的 ERD

# 结构化的系统分析方法

## 数据分析建模 – IDEF1X图




IDEF1X形式的 ERD

# 结构化系统分析

## 本章主要内容

1. 结构化方法vs面向对象方法
2. 结构化的系统分析方法
3. 数据流图（DFD）
4. 数据字典（DD）
5. 数据分析（ERD、IDEF1X）

# DFD建模 - 基于数据流的系统分析

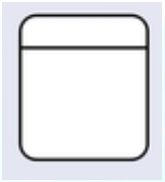
**数据流图（DFD）：** 用处理、外部实体、数据流以及数据存储来表示系统需求的图表

## DFD的特点

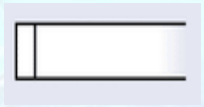
- 图形元素少且符号简单易懂
- 较充分表达系统的主要需求：输入、输出、处理和数据存储
- 最终用户、管理人员和系统开发人员只需稍加培训即可读懂DFD图，方便交流

# DFD建模 - 基于数据流的系统分析

## DFD数据流图的符号说明

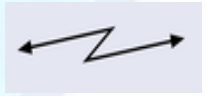


**处理** – 一步步地执行指令，将输入转换成输出（由人、机器或两者共同完成该处理任务）

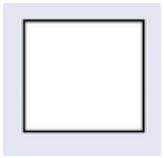


**数据存储** – 存放起来以备将来使用的数据。通常与ERD图中的数据实体相对应

**数据流** – 从一处流向另一处的数据，如处理的输入或输出



**实时连接** – 当处理执行时，外部实体与处理之间来回通信(如信用卡验证)

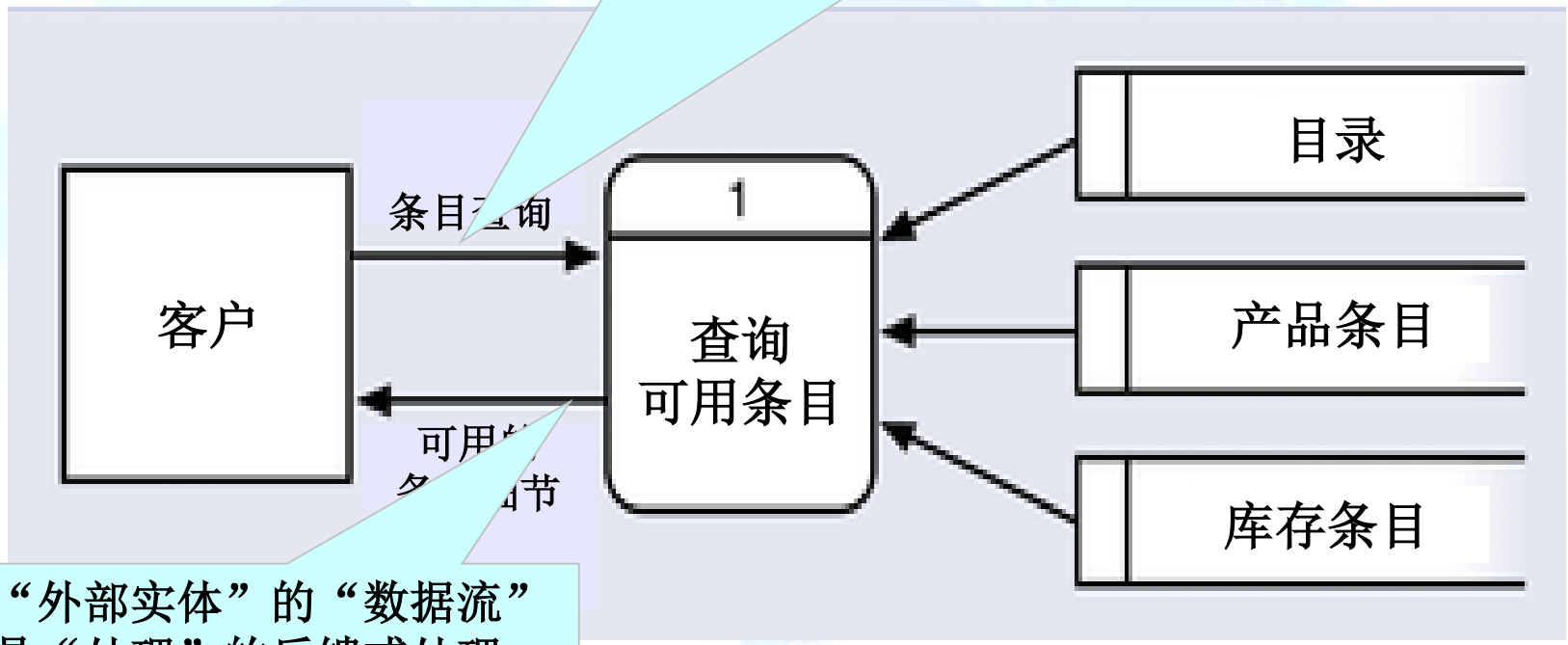


**外部实体** – 系统之外的数据源或目的地

# DFD建模 - 基于数据流的系统分析

数据流图例子：

从外部实体出发的“数据流”可以是：
1. 外部实体发出的“处理请求”，即一个事件
2. 外部实体给出的“输入数据”



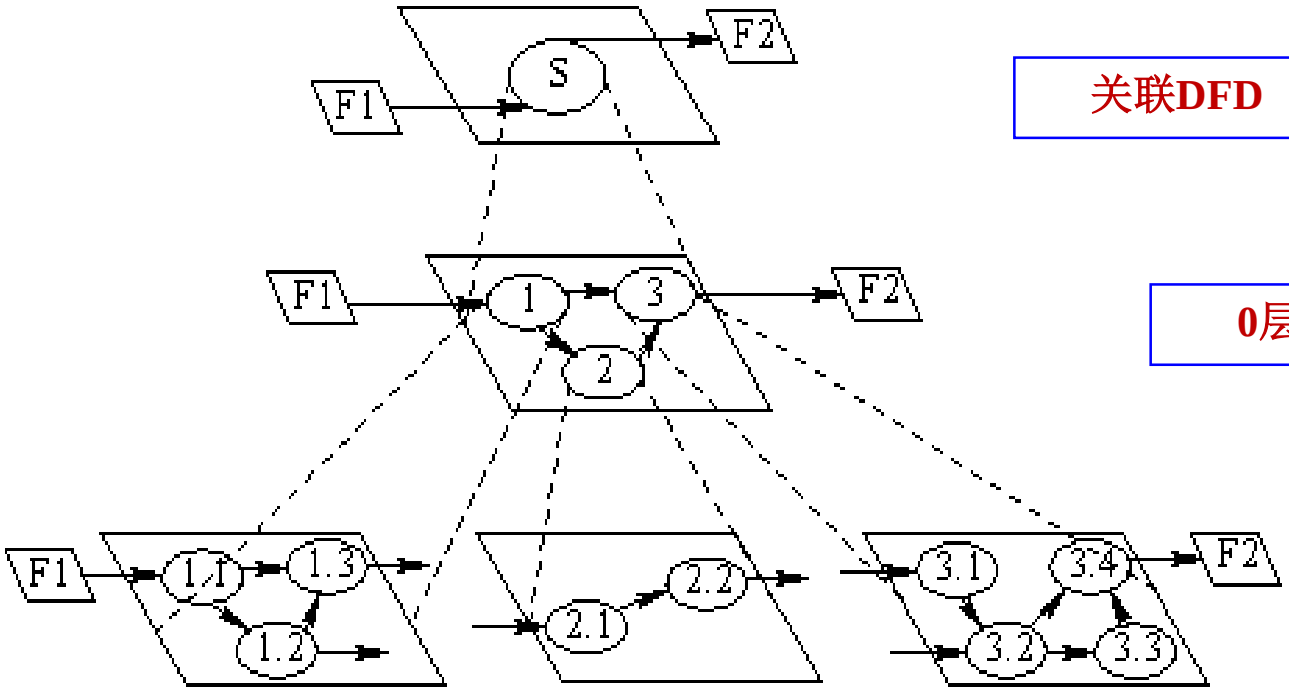

指向“外部实体”的“数据流”一般是“处理”的反馈或处理结果

显示处理“查询可用条目”的DFD

# DFD建模 - 基于数据流的系统分析

**DFD图可以描述高层次的具有高度概括的系统处理**
**也可以描述低层次的具有更详细分解的系统处理**

抽象层次：把系统分解成一个逐步细化的分层集合的建模技术

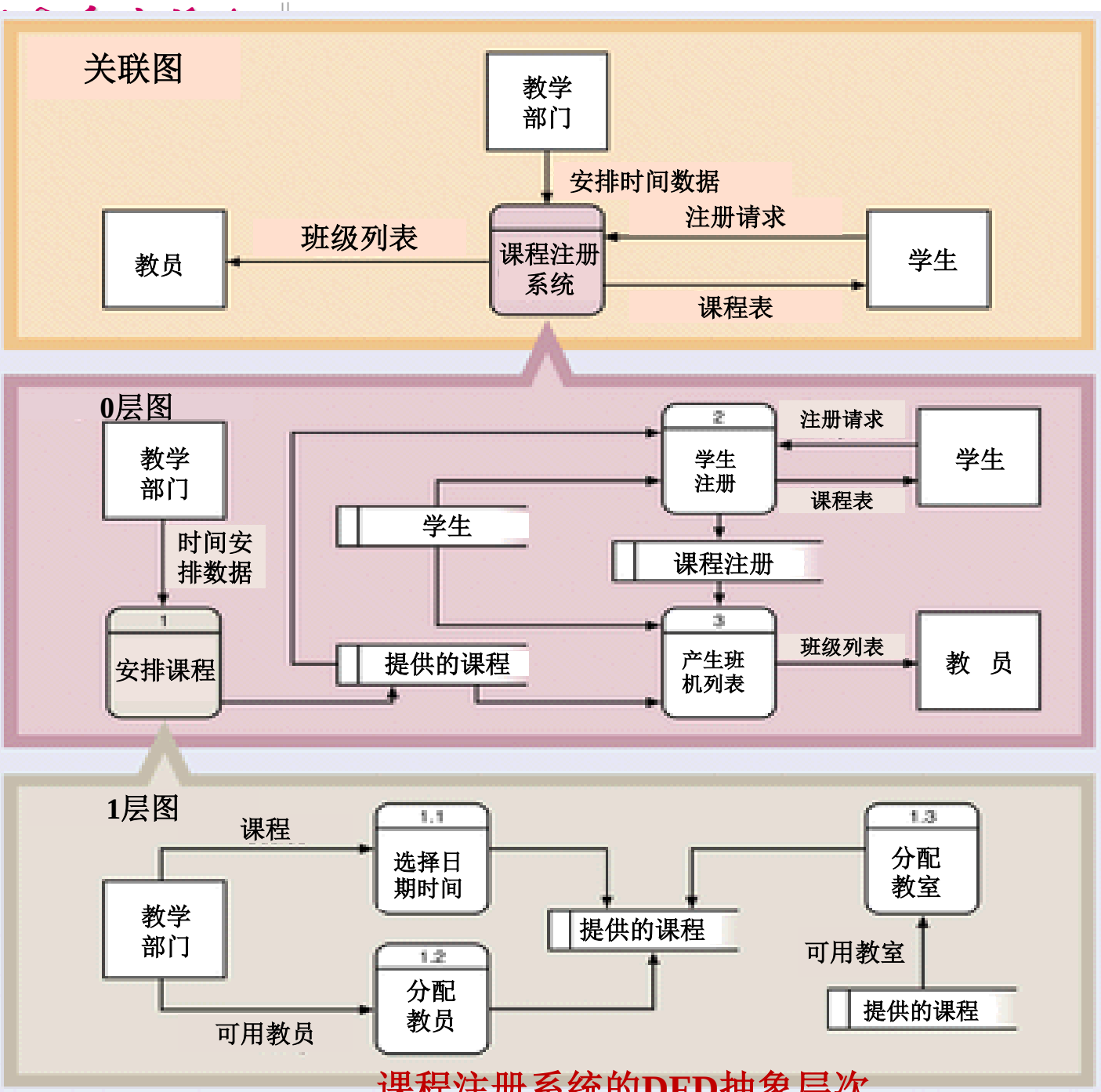



关联DFD

0层DFD

1层DFD

DFD




课程注册系统的DFD抽象层次

# DFD建模 - 基于数据流的系统分析

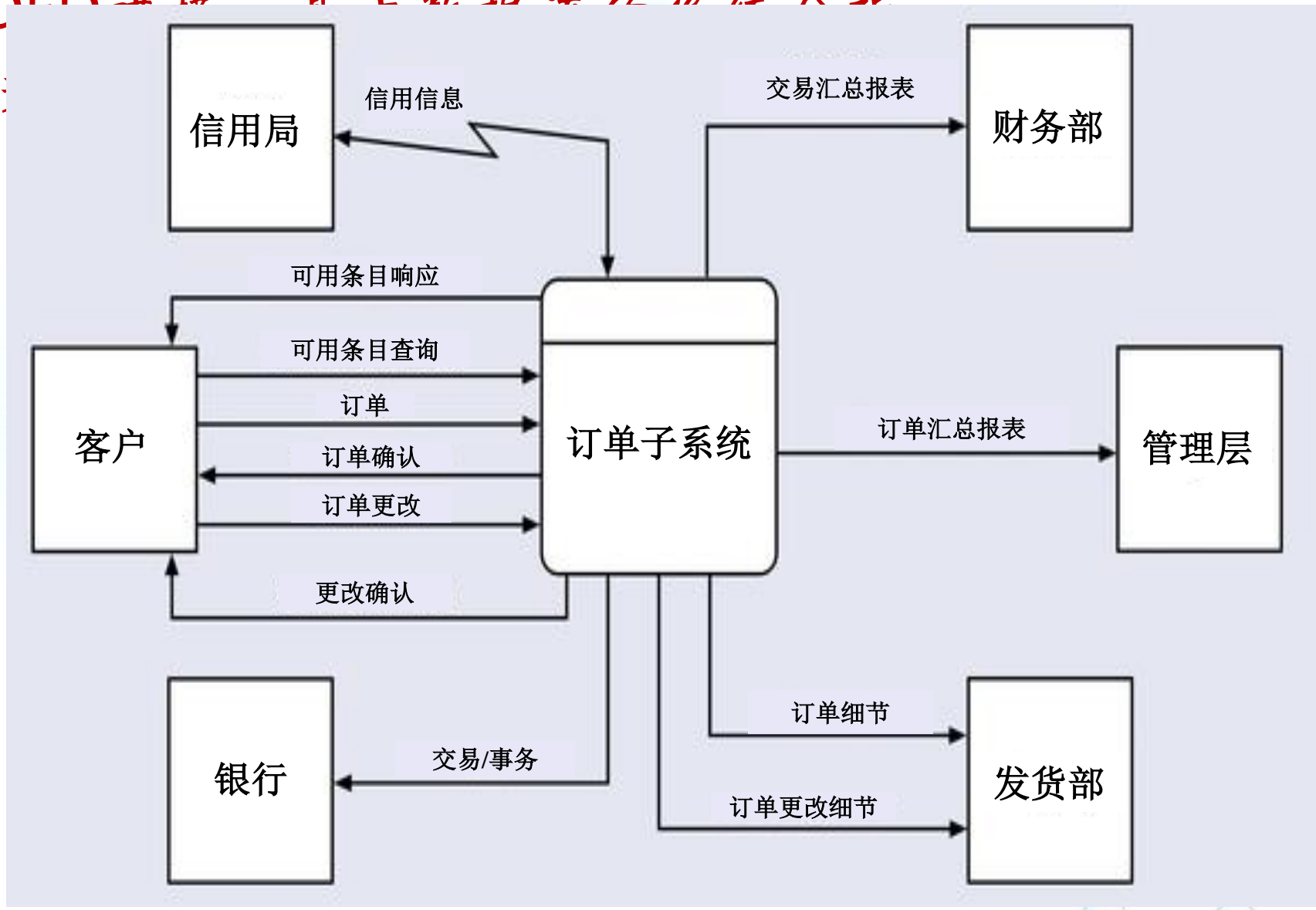
## 关联DFD图

关联图：在单个处理符号中概括系统内所有处理活动的DFD

关联图在表达系统边界时用处很大
系统的范围是通过单个的处理和外部实体所表示的事物来定义的
数据存储不画在关联图中是因为它本身被认为是系统内部的内容

当一个系统响应事件较多时，常常将系统分成多个子系统，并为每个子系统创建一张关联图

# 结构化系统分析




订单子系统的关联DFD图

# DFD建模 - 基于数据流的系统分析

## DFD片段

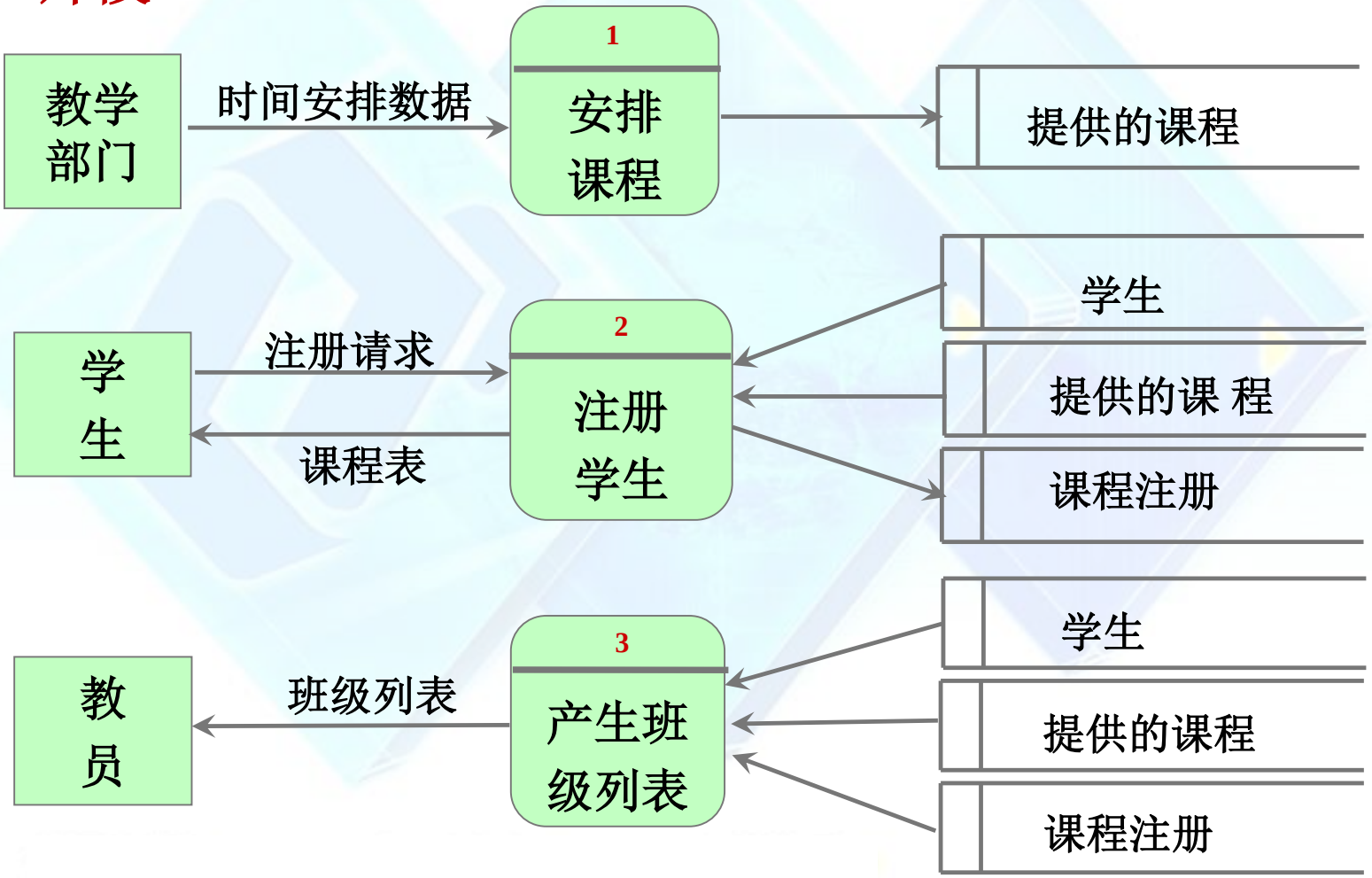
**DFD片段：用一个单一处理符号表示系统响应一个事件的DFD**

- 在DFD片段中，展示了处理、外部实体和内部数据存储之间的交互细节
- 每个DFD片段仅显示要响应该事件的相关的那些数据存储

- 一个DFD片段是为事件表中的每一个事件创建的

# DFD建模 - 基于数据流的系统分析

## DFD片段




课程注册系统的DFD片段

# DFD建模 - 基于数据流的系统分析

## DFD的0层图

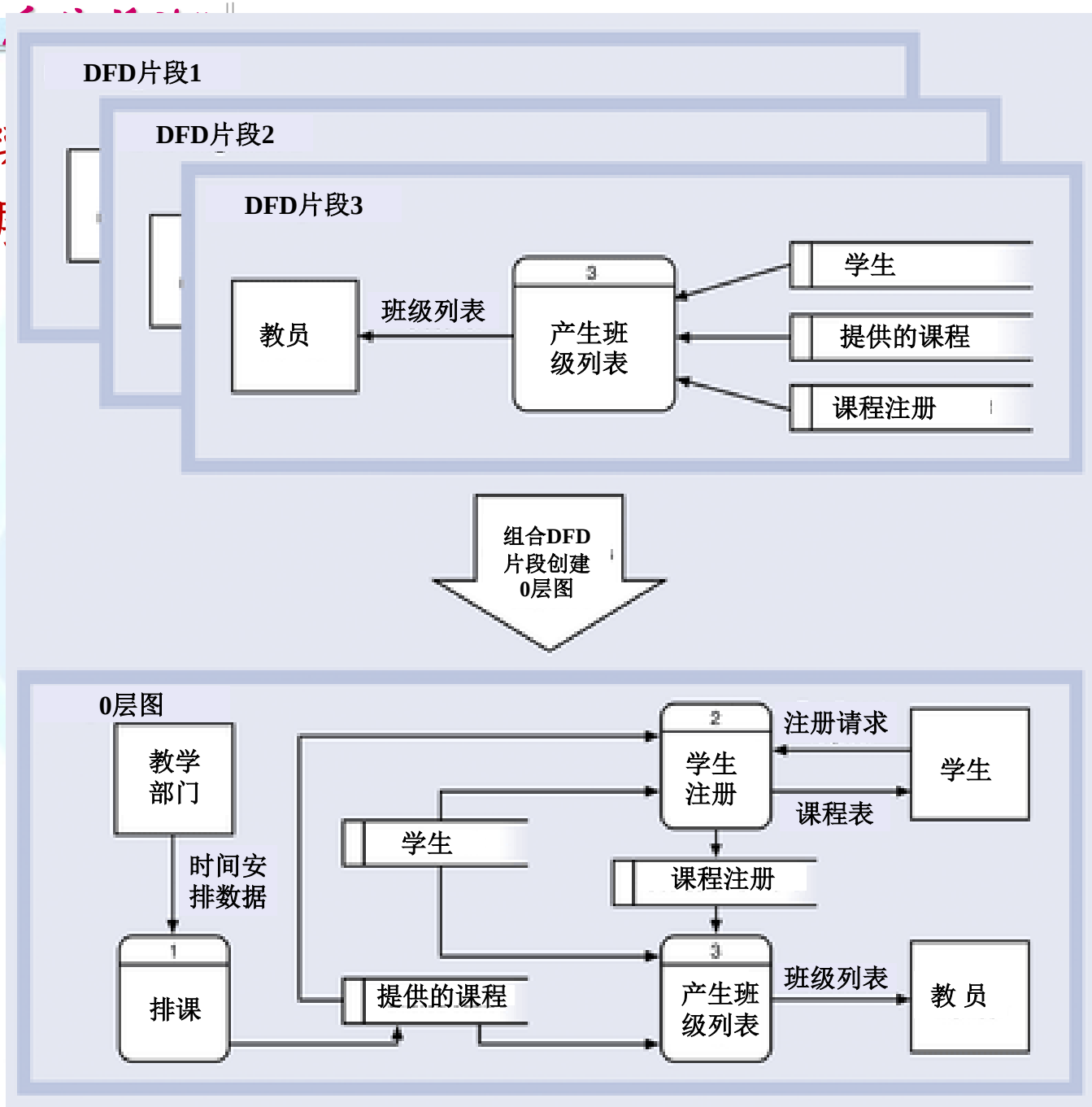
**DFD的0层图：**

将一个系统或子系统的所有DFD片段组合到一个单个的DFD图中，这样的DFD图称为事件分离的系统模型/0层图

**0层DFD图：** “处理”的编号为i

# DFD建模

## DFD的0层




课程注册系统中将DFD片段组合创建的0层图

# DFD建模 - 基于数据流的系统分析

## DFD的层次

分解“处理”-- 理解“处理”的细节：

一个DFD片段中的处理也可以包含多个更小的处理

有时分析员需要将该DFD片段进一步细化，生成更详细层次的DFD图

**1层DFD图：** 将0层DFD中的处理进一步细化等到的DFD图

“处理”的编号为“i.j”

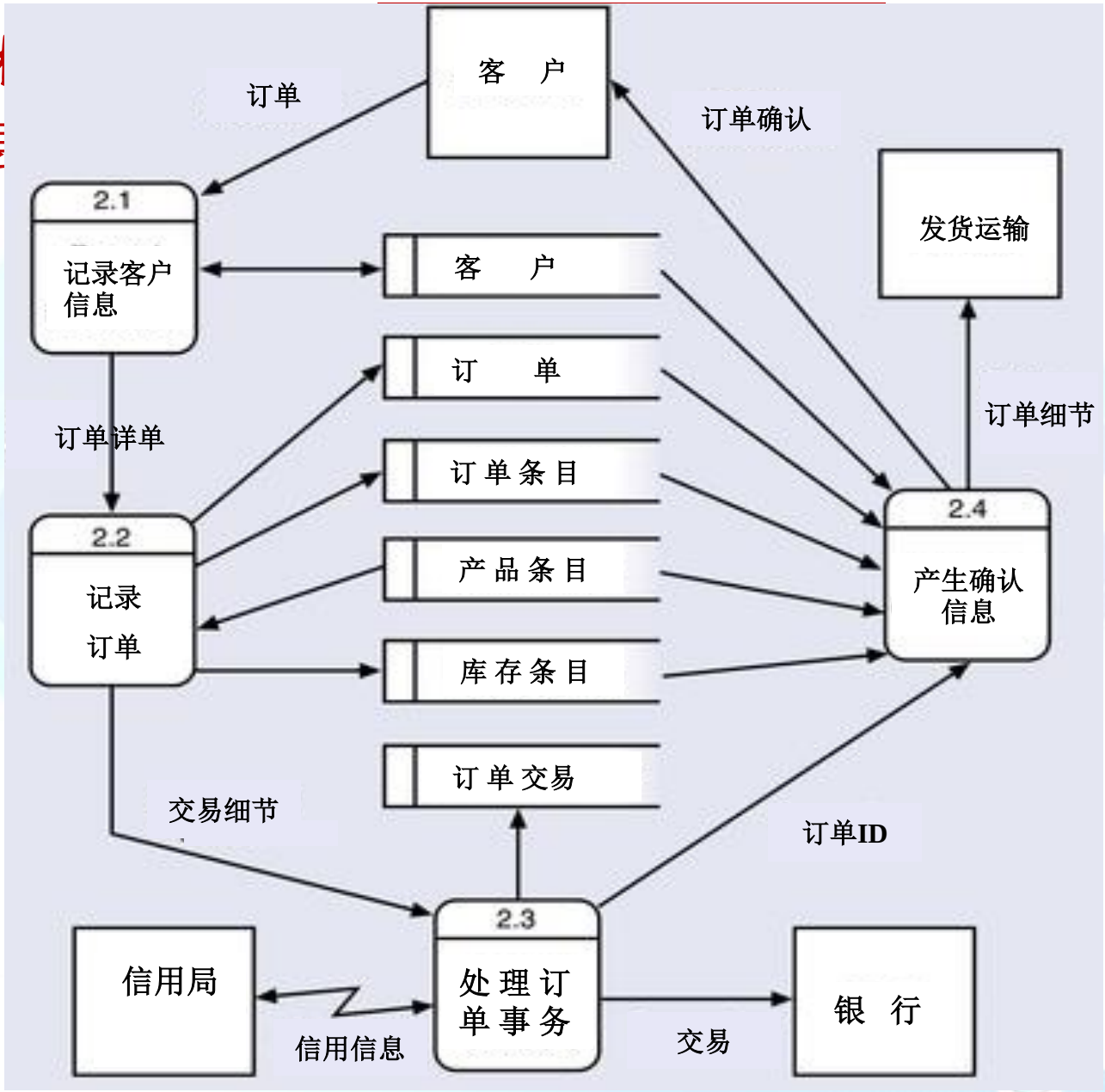
**2层DFD图：** 将1层DFD中的处理进一步细化等到的DFD图

“处理”的编号为“i.j.k”

# 结构化系统分析

## DFD建

## DFD的层




1层DFD实例：创建新订单的更详细的DFD图

# DFD建模 - 基于数据流的系统分析

## DFD的质量评估

**高质量的DFD：**

可读性强、内部一致、能够准确描述系统需求

**措施：**

- 最小化复杂度
- 保证数据流一致性

**最小化复杂度：**

就是使每幅DFD图尽量简单易懂，避免信息超量

**信息超量：**

当太多的信息同时显现时所发生的难以理解的情况

# DFD建模 - 基于数据流的系统分析

## DFD的质量评估

### 最小化复杂度

措施：

采用分层结构将DFD划分为小的且相对独立的子集
这样可以逐级阅读、考察DFD

构造DFD图的7±2规则：

- 单个DFD中不应有超过7±2个处理
- 单个DFD中不应超过7±2个数据流进出同一个处理/数据存储

接口最小化：

- DFD中各个元素之间的连接数越少越好

# DFD建模 - 基于数据流的系统分析

## DFD的质量评估

保证数据流一致性

数据流一致性表现在三个方面：

- 一个“处理”和该“处理”被详细分解后在数据流内容上应该一致
- 对一个“处理”，有数据流入则必须有相对应的数据流出
- 对一个“处理”，有数据流出则必须有相对应的数据流入

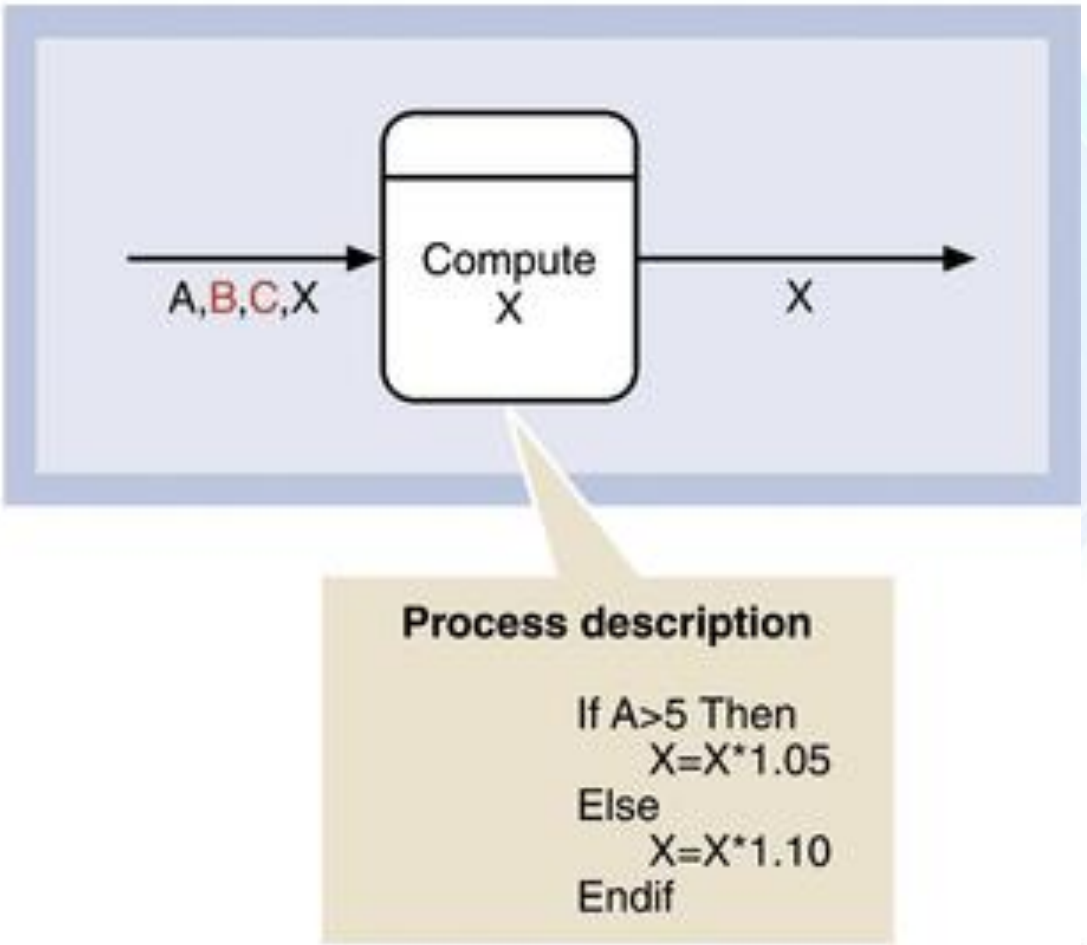
# DFD建模 - 基于数据流的系统分析

## DFD的质量评估

黑洞 --- 带有输入数据的但并不用其产生输出数据的处理或数据存储

# DFD建模 - 基于数据流的系统分析

## DFD的质量评估




带有不必要数据输入的处理—一个黑洞

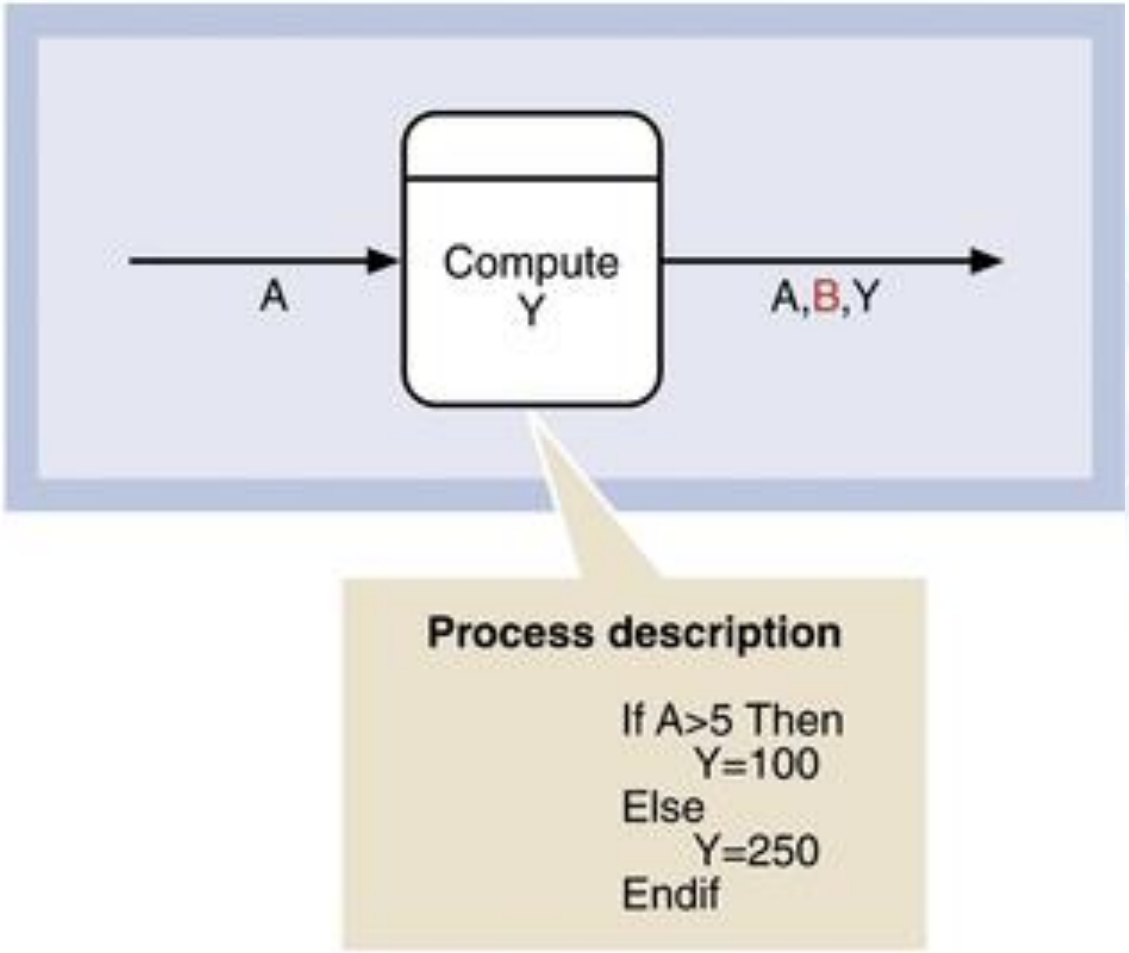
# DFD建模 - 基于数据流的系统分析

## DFD的质量评估

奇迹 --- 没有足够数据元素作为输入或产生来源的一个处理或数据存储

# DFD建模 - 基于数据流的系统分析

## DFD的质量评估




有不可能的数据输出的处理—一个奇迹

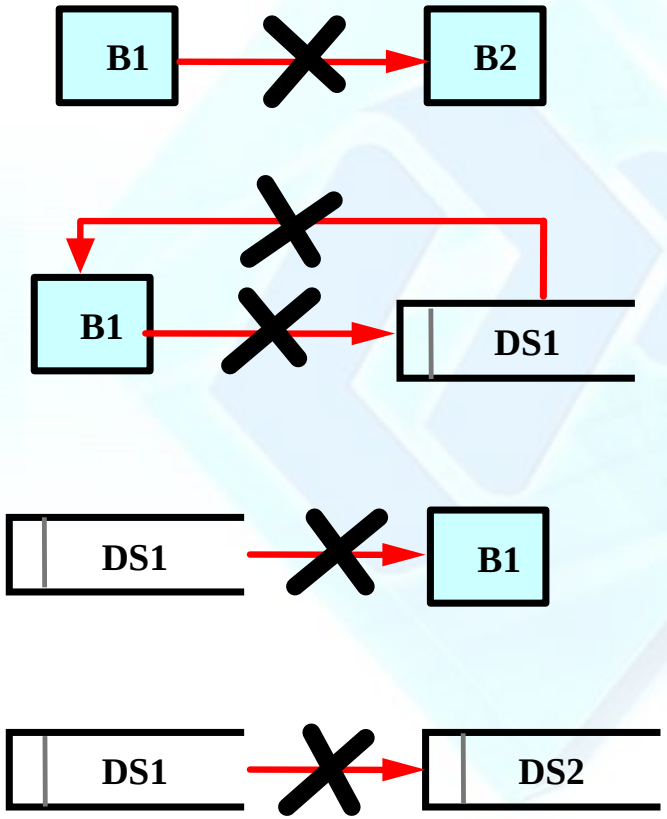
# DFD建模 - 基于数据流的系统分析

## DFD的质量评估

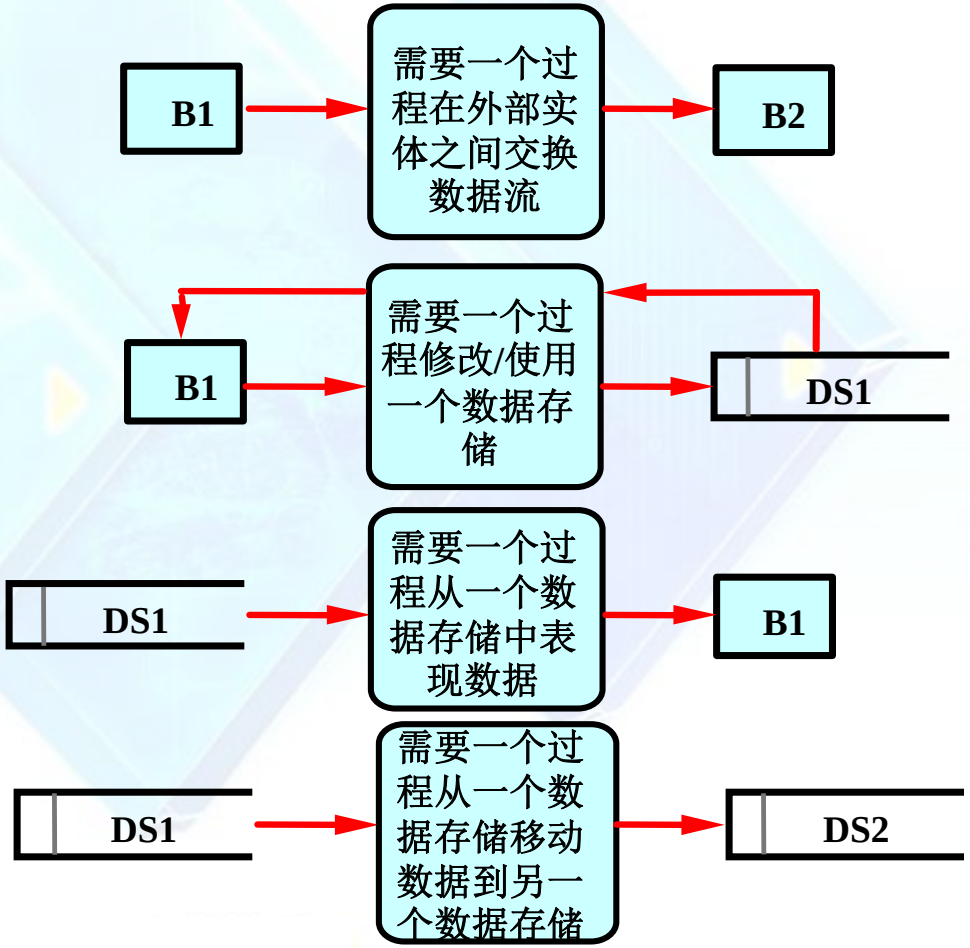
## DFD图典型错误

# DFD建模 - 基于数据流的系统分析

非法的数据流




改正错误后的数据流

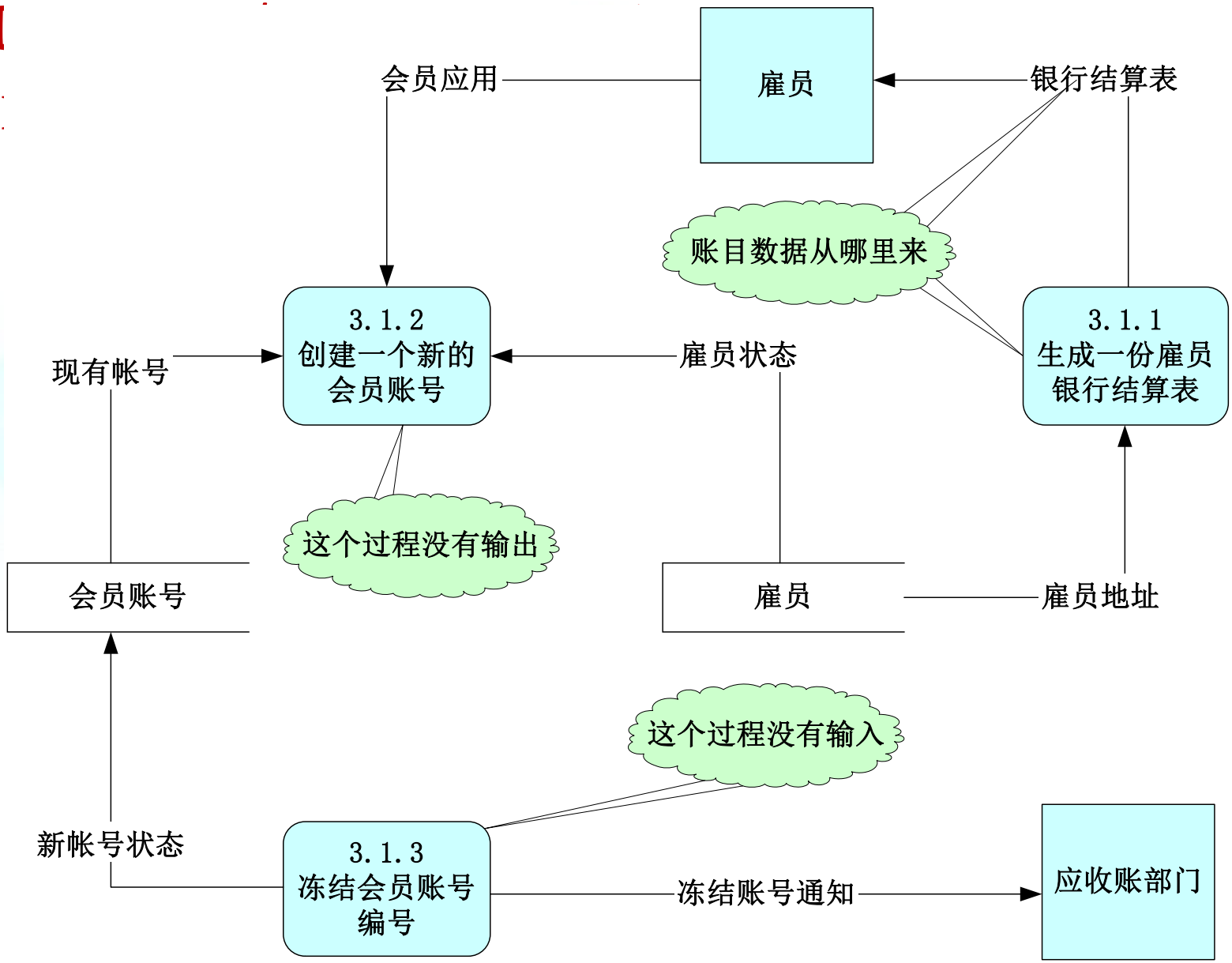



数据流图中典型错误总结

# 结构化系统分析

DFI

DF




数据流图中典型错误例子

# DFD建模 - 基于数据流的系统分析

## DFD模型总结

- 在关联图中不画数据存储
- 数据流不反映处理顺序，显示数据通过系统的流程，因此“处理”可以并行工作
- “处理/数据存储”既要有输入，又有输出
  - 若输入数据流不完全用来产生输出数据流，称之为黑洞
  - 若输出数据流不完全依赖于输入数据流，称之为奇迹

# DFD建模 - 基于数据流的系统分析

## DFD细节内容描述

在传统方法中，**DFD**图在一个图中描述了三种元素：

- 处理 -- 每个最底层“处理”需要详细描述其细节
- 数据流 -- “数据流”包含的数据元素需要定义
- 数据存储 -- “数据存储”需要在**ERD**中定义

使用“数据字典 – DD”来描述，见下一小节

# DFD建模 - 基于数据流的系统分析

## DFD细节内容描述 – 处理的细节描述

1. “处理”细分解，层层分解，直到可详细描述细节

2. 结构化语言/伪代码

3. 决策表/决策树

“处理”细分解的方法前面已经提到

# DFD建模 - 基于数据流的系统分析

## DFD细节内容描述描述 – 处理的细节描述

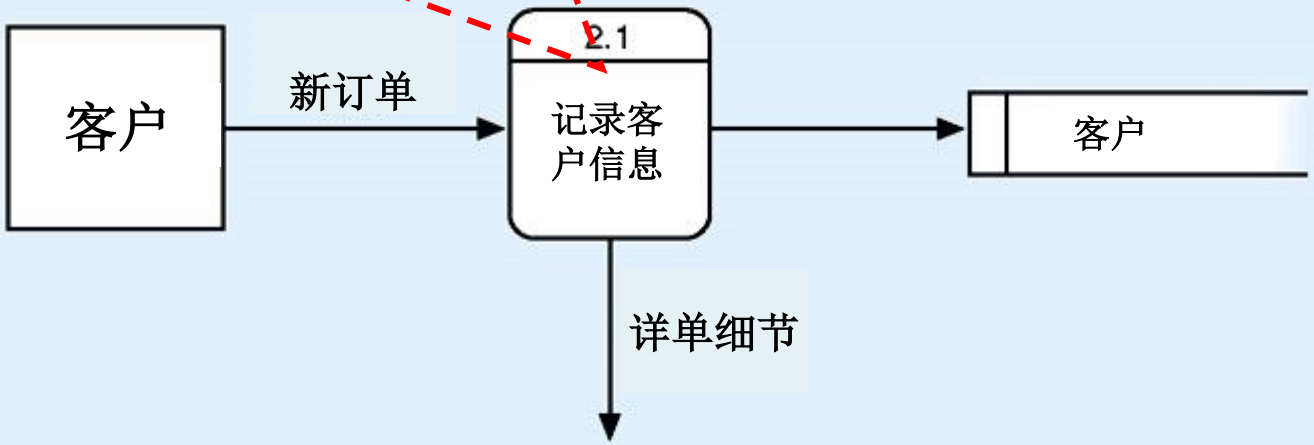
结构化语言/伪代码：

- 一种描述“处理”的规范方法，将结构化编程技术和叙述性英语结合起来
- 结构化英语适合用来描述处理步骤清楚、控制逻辑相对简单的情况

DFD
DFD

```
Process 2.1 - Record Customer Information

Ask if customer has an account (or has made a previous order)
If customer has an account then
   Ask for identification information
   Query database with identifying information
   Copy query response data to Order details
Else
   Create an empty Customer record in the database
   Ask customer for Customer attributes
   Update empty Customer record with Customer attributes
Endif
Ask customer for order information for first item
While more order items Do
   Update Order details with order information
Endwhile
```




处理2.1（记录客户信息）和它的结构化语言描述

# DFD建模 - 基于数据流的系统分析

## DFD细节内容描述描述 – 处理的细节描述

### 结构化语言的特点

- 介于自然语言和形式语言之间的语言
- 书写规则：
  - 使用短句
  - 多层缩进
  - 将结构化编程技术和叙述性语言结合
  - 无确定语法
  - 可分层、嵌套

# DFD建模 - 基于数据流的系统分析

## DFD细节内容描述描述 – 处理的细节描述

### 决策表和决策树

**决策表：** “处理”逻辑的一种表格形式的表示方法，其中包括决策变量、决策变量值、行为或公式

**决策树：** 用树形结构组织起来的线条对“处理”逻辑进行图形化的描述

# DFD建模 - 基于数据流的系统分析

## DFD细节内容描述描述 – 处理的细节描述

### 决策表

| | | 规　则 | | | |
|---|---|---|---|---|---|
| 条件 | 考试总分 | ≥ 435 | ≥ 435 | < 435 | < 435 |
| | 单科成绩 | ≥ 90 | ≥ 85 | ≥ 60 | < 60 |
| 动作 | 发表扬证 | √ | √ | × | × |
| | 发奖金 | √ | × | × | × |
| | 警告 | × | × | × | √ |

学生成绩处理“决策表”

# DFD建模 - 基于数据流的系统分析

## DFD细节内容描述描述 – 处理的细节描述

决策树

# 本章主要内容

1. 结构化方法vs面向对象方法
2. 结构化的系统分析方法
3. 数据流图（DFD）
4. 数据字典（DD）
5. 数据分析（ERD、IDEF1X）

# 数据字典（DD）－DFD细节内容描述

数据字典是数据分析的描述模型，包括：

数据项定义，数据结构定义，数据流描述，数据存储描述

**数据项定义：** 定义特定数据项的组成和意义

**数据结构定义：** 定义有数据项组成的表达数据的基本数据

**数据流描述：** 描述数据流的数据构成，并指明其来源或去向

**数据存储描述：** 描述保存在存储介质上的数据文件或数据库表的格式和内容

广义的数据字典，也可以包含对“处理”的描述

# 数据字典（DD）－DFD细节内容描述

## 数据项定义

数据项：数据的基本单位

- 数据项名
- 数据项说明
- 数据类型
- 长度
- 取值范围
- 语义定义
- 与其他数据项的关联

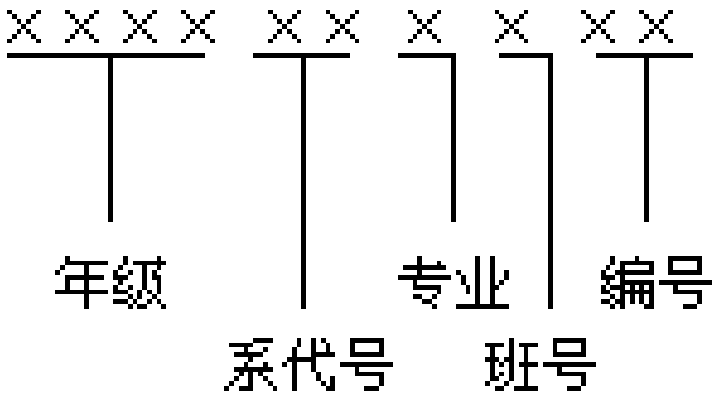
# 数据字典（DD）－DFD细节内容描述

## 数据项定义

数据元素条目

名称：学号
说明：本校学生编号
数据值类型：离散
类型：数字
长度：10
有关数据结构：学生成绩
学生登记卡
选课卡

总编号：1—001
编号：001
有关编码说明：



# 数据字典（DD）－DFD细节内容描述

## 数据结构定义

数据结构：由数据项组成，它给出了数据基本结构单位

- 数据结构名
- 数据结构说明
- 数据结构组成：｛数据项/数据结构｝
- 数据结构约束

# 数据字典（DD）－DFD细节内容描述

## 数据结构定义

数据结构条目

名称：学生登记卡　　总编号：2－001

说明：新生入学时填写基本信息的卡片　　编号：001

结构：学号
姓名
性别
出生日期
入学日期
民族
家庭住址
联系电话

有关数据流，数据存储：
学籍表

# 数据字典（DD）－DFD细节内容描述

## 数据流定义

数据流：数据结构在系统中的流通路径

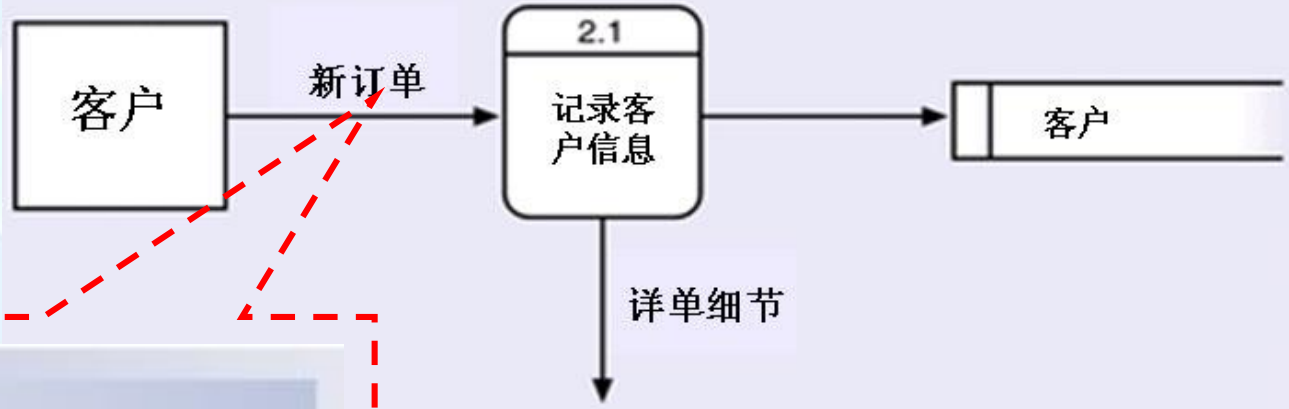
- 数据流名
- 数据流说明
- 数据流来源
- 数据流去向
- 数据流组成：｛数据结构｝
- 平均流量
- 高峰流量

# 结构化系统分析

## 数据字典（DD）－DFD细节内容描述

### 数据流定义

数据流是数据元素的集合，数据流定义就是列出其包含的所有数据项




```
Customer-Name
Customer-Address
Credit-Card-Information
Item-Number
Quantity
```

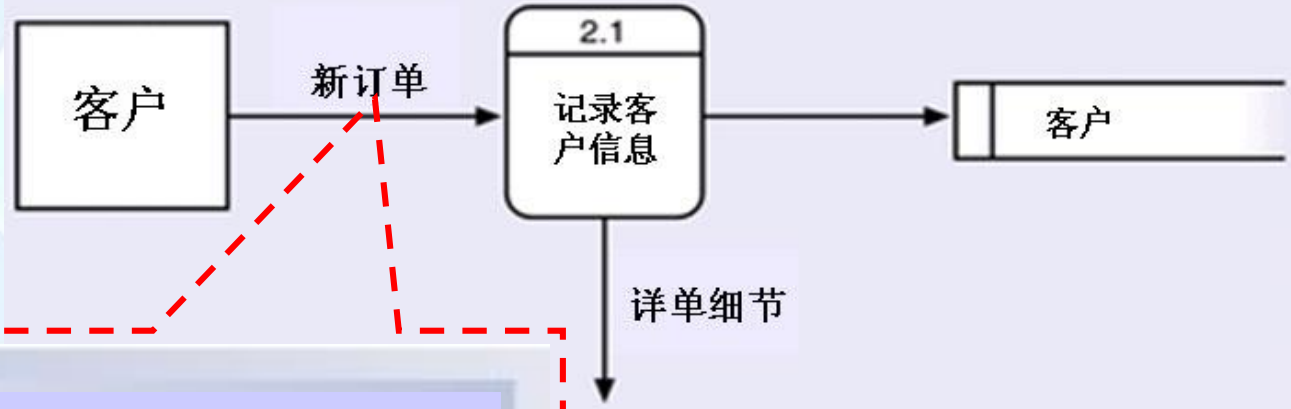
数据元素还需进一步的定义：说明类型、长度等，一般在设计阶段详细定义

只列出数据元素的数据流定义

# 数据字典（DD）－DFD细节内容描述

## 数据流定义

数据流是数据元素的集合，数据流定义就是列出其包含的所有数据项




New-Order=Customer-Name
+ Customer-Address
+ Credit-Card-Information
+ {Item-Number+Quantity}

代数公式方式定义数据流

N
1

# 数据字典（DD）－DFD细节内容描述

## 数据流定义

数据流条目

名称：新生信息　　总编号：3－001

说明：新生入学时带来的基本信息　　编号：001

流通量：1000份/学期

数据流来源：招生办

数据流去向：P1.1

包含的数据结构：学号
姓名
性别
出生日期
入学日期
民族

# 数据字典（DD）－DFD细节内容描述

## 数据存储定义

数据存储：数据结构保存或停留之处，数据文件或数据库表

- 数据存储名
- 数据存储说明
- 输入的数据流
- 输出的数据流
- 数据存储组成：｛数据结构｝
- 数据量
- 存取频度
- 存取方式

# 数据字典（DD）－DFD细节内容描述

## 数据存储定义

数据存储条目

名称：学籍表　　总编号：4－001

说明：存储学生所有信息的记录　　编号：001

结构：基本信息　　有关数据流：

奖惩记录　　P1.1　D1

学生动态　　P1.2　D1

考试成绩　　P2.1　D1

D1　P3.1

D1　P3.2

# 数据字典（DD）－DFD细节内容描述

## 数据处理（广义DD）

数据处理：给出处理的流程和说明信息

- 数据处理名
- 数据处理说明
- 输入数据：　{数据结构}
- 输出数据：　{数据结构}
- 处理过程简要描述

# 数据字典（DD）－DFD细节内容描述

## 数据处理（广义DD）

处理过程条目

总编号：5－001

编号：P1.1

名称：信息输入

说明：输入新生的基本信息

输入：招生办　　P1.1

输出：P1.1　　D1

处理：将由招生办来的学生登记卡的内容作为基本信息输入到学籍记录中。

# 结构化系统分析

## 本章主要内容

1. 结构化方法vs面向对象方法
2. 结构化的系统分析方法
3. 数据流图（DFD）
4. 数据字典（DD）
5. 数据分析（ERD、IDEF1X）

# 数据分析（ERD、IDEF1X）

## 实体关系图 – ERD

- 对DFD中的数据存储的分析非常重要，单独进ERD建模
- 数据存储需求包括数据实体、数据实体的属性以及它们之间的关系
- 分析员在数据建模的过程中，常常对ERD进行细化的工作，即处理多对多的关系以及细化属性说明
- 由于关系数据库中不能直接实现多对多的关系，因此必须建立一个单独的表，来消除多对多的关系
- 关联实体 – 解决上述问题的人为增加的数据实体，它一定包含两端数据实体的关键字

# 数据分析（ERD、IDEF1X）

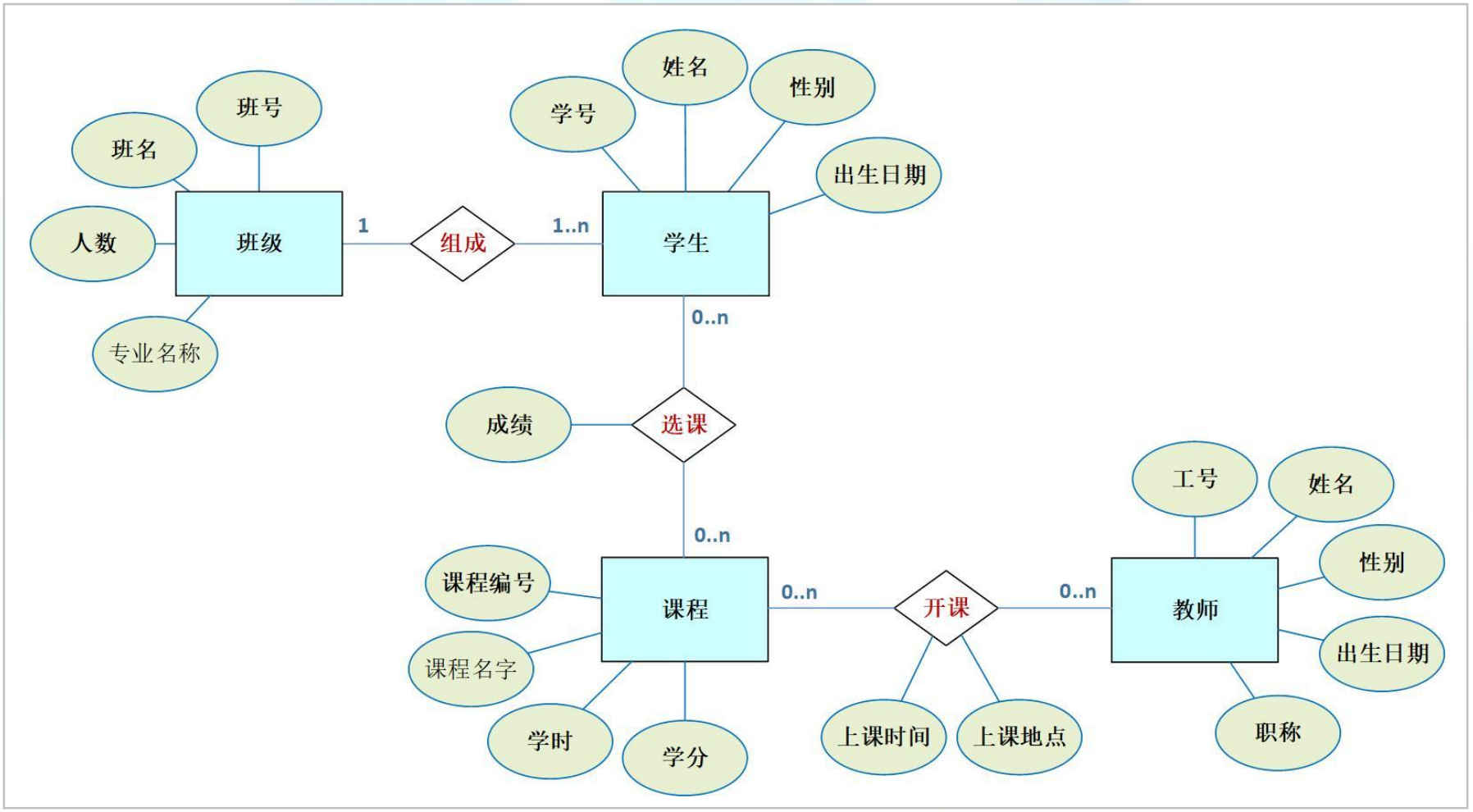
## 实体关系图 – ERD

- 概念ERD
- 逻辑ERD
- 物理ERD

| ERD形式 | 概念ERD | 逻辑ERD | 物理ERD |
|---|---|---|---|
| 实体（名称） | √ | √ | √ |
| 关系 | √ | √ | √ |
| 关联实体 | | √ | √ |
| 属性（名称） | | √ | √ |
| 属性的类型、长度、说明 | | 随意 | √ |
| 主键 | | | √ |
| 外键 | | | √ |

# 数据分析（ERD、IDEF1X）

## 实体关系图 – ERD

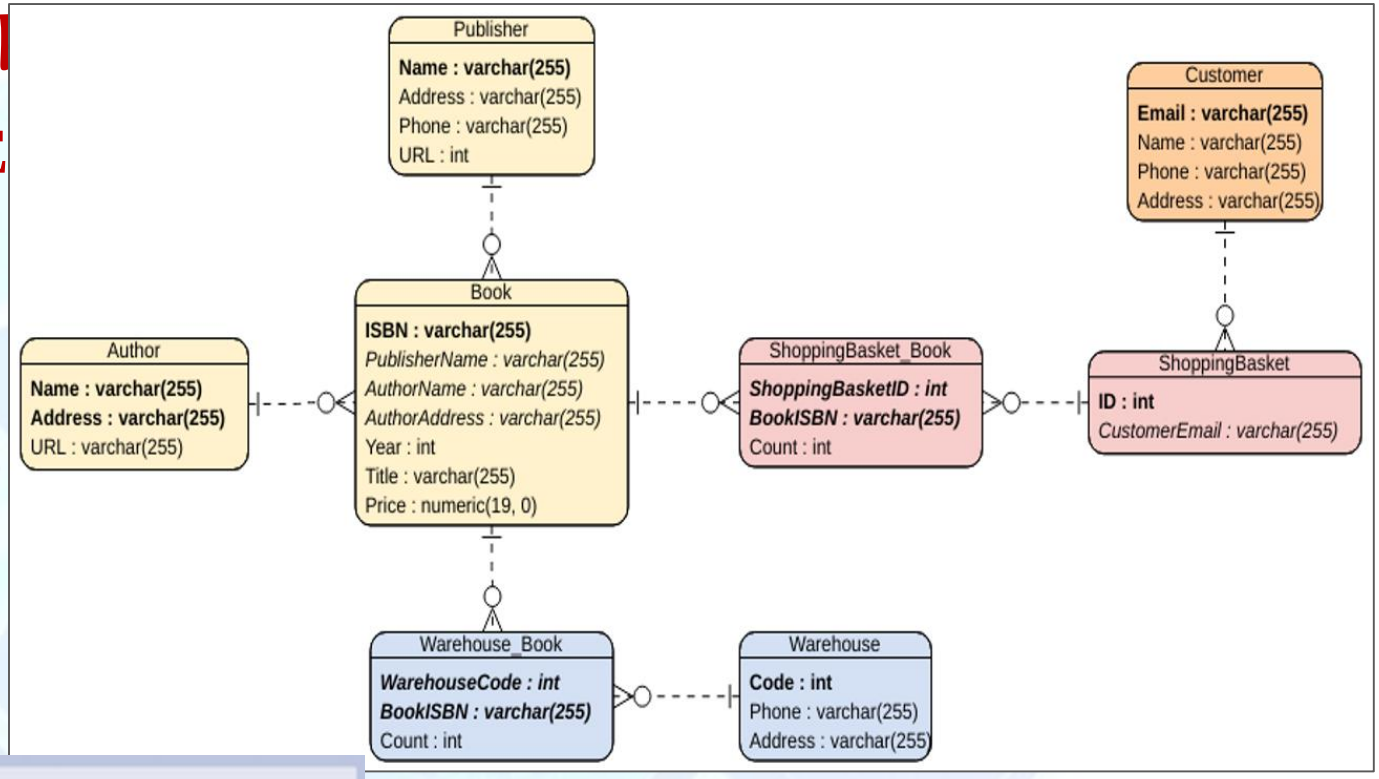


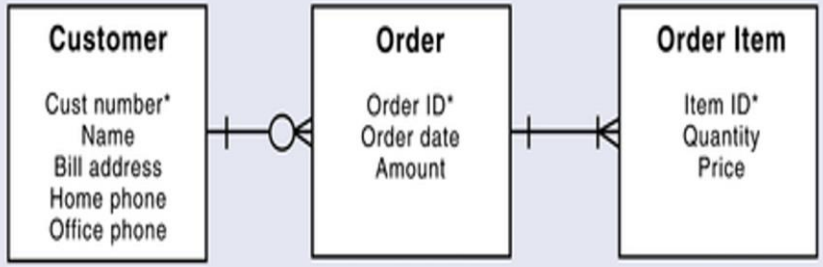

传统的标准画法的ERD（Chen's ERD）

# 结构化系统分析

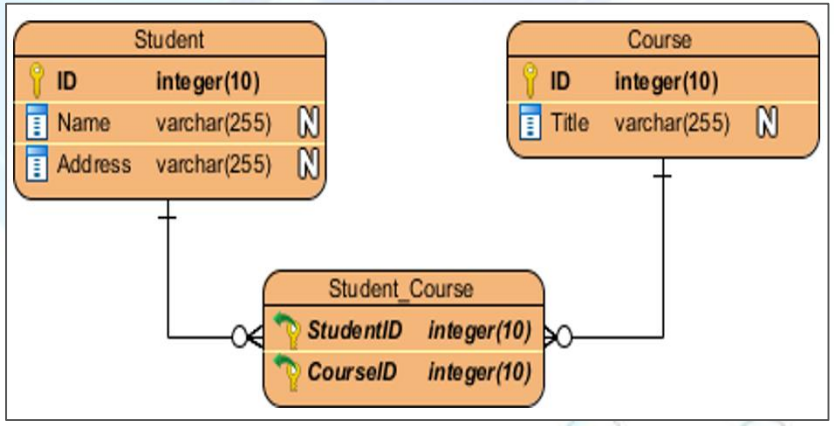
## 数据分析（E

## 实体关系图 – E



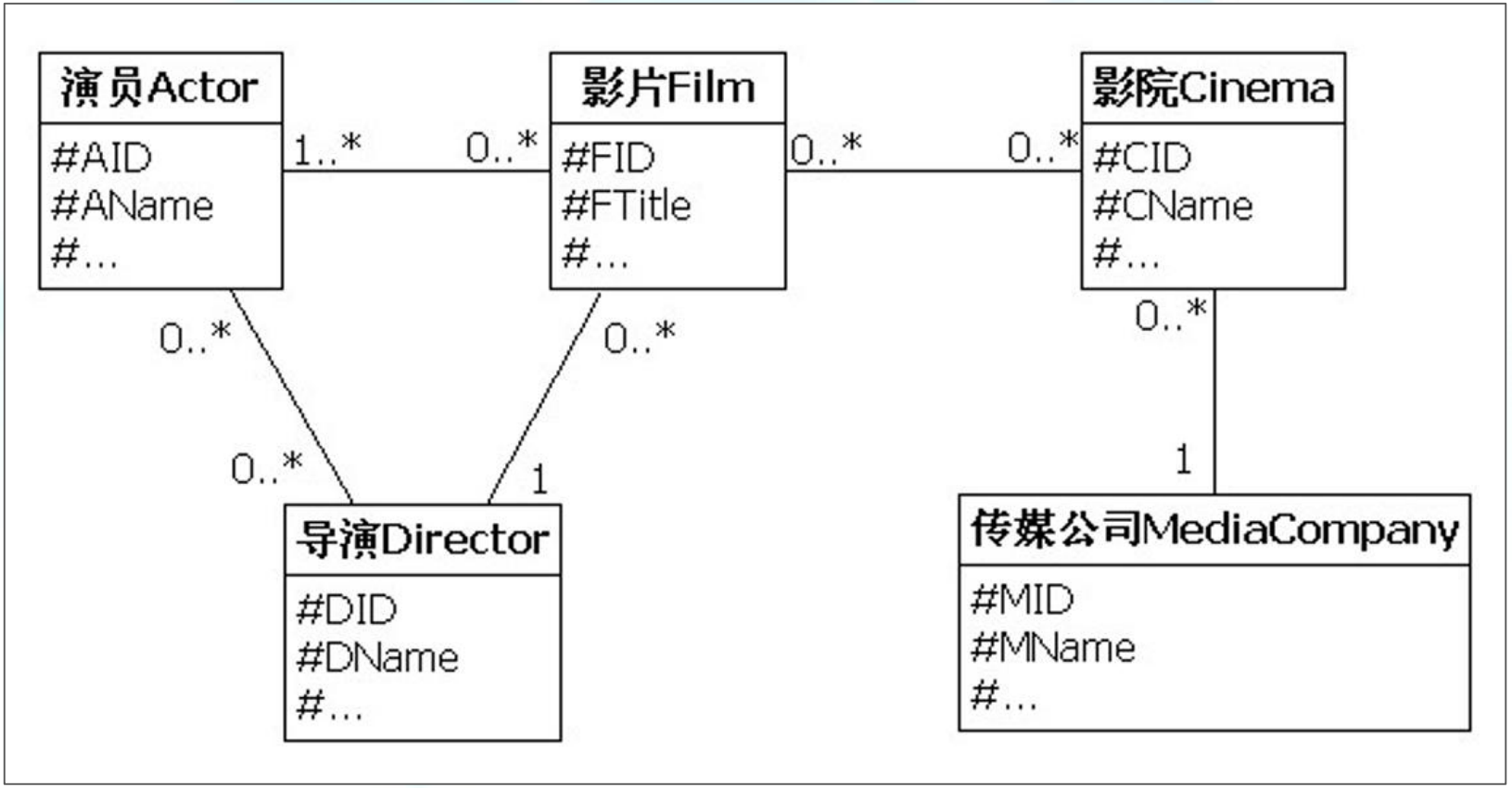

*表示标识符或关键字




用乌鸦脚（Crow's Foot）标注基数关系的ERD

# 数据分析（ERD、IDEF1X）

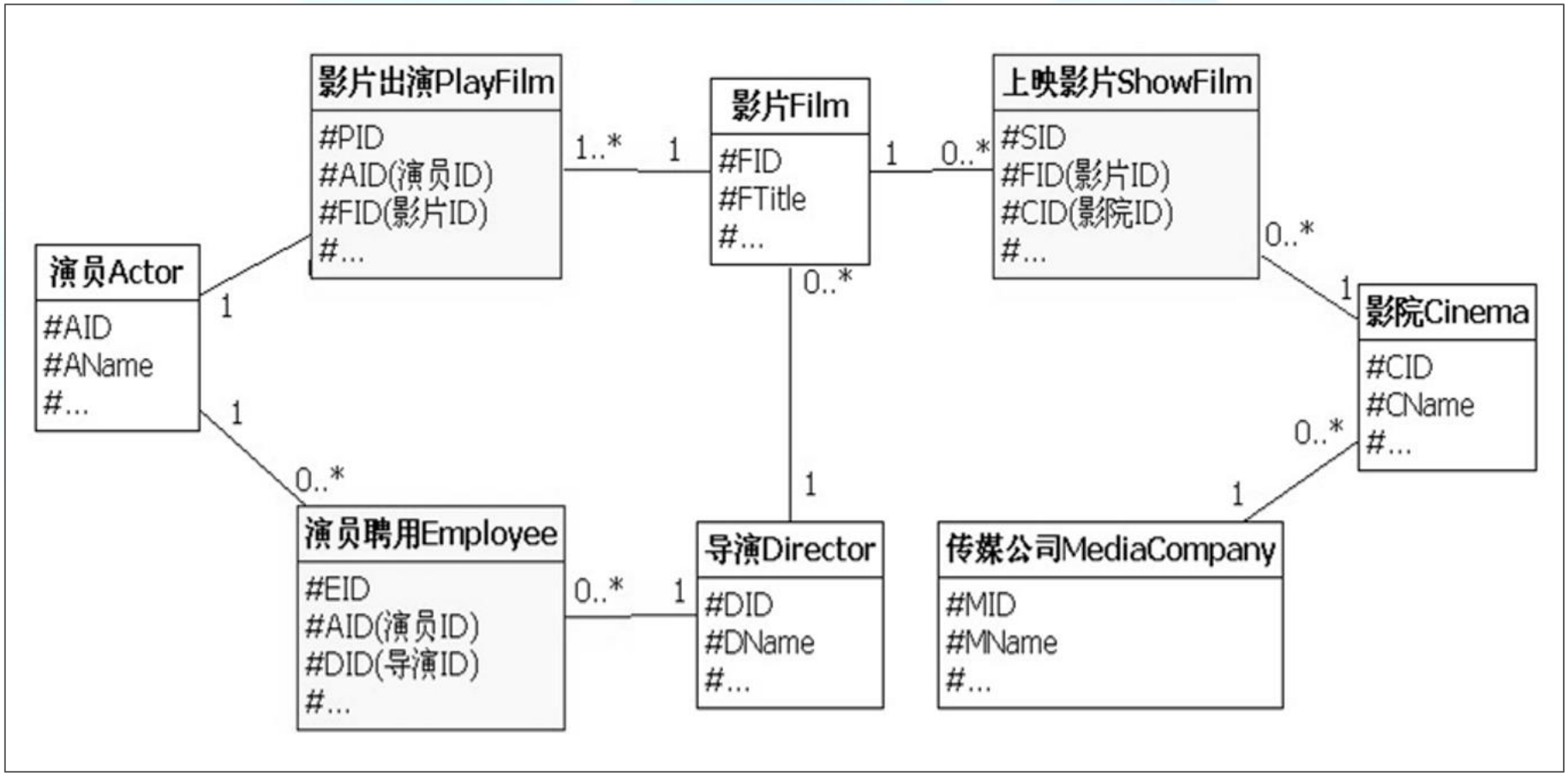
## 实体关系图 – 类IDEF1X图




含有*..*关系的ERD（类IDEF1X形式）

# 数据分析（ERD、IDEF1X）

## 实体关系图 – 类IDEF1X图




消除*..*关系增加了关联实体的ERD（类IDEF1X形式）

# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 09

# 面向对象系统分析

## 本章主要内容

1. 面向对象的分析方法概述
2. 建立静态结构模型
3. 建立动态行为模型
4. 案例分析

# 面向对象系统分析

## 本章主要内容

1. 面向对象的分析方法概述
2. 建立静态结构模型
3. 建立动态行为模型
4. 案例分析

# 面向对象的分析

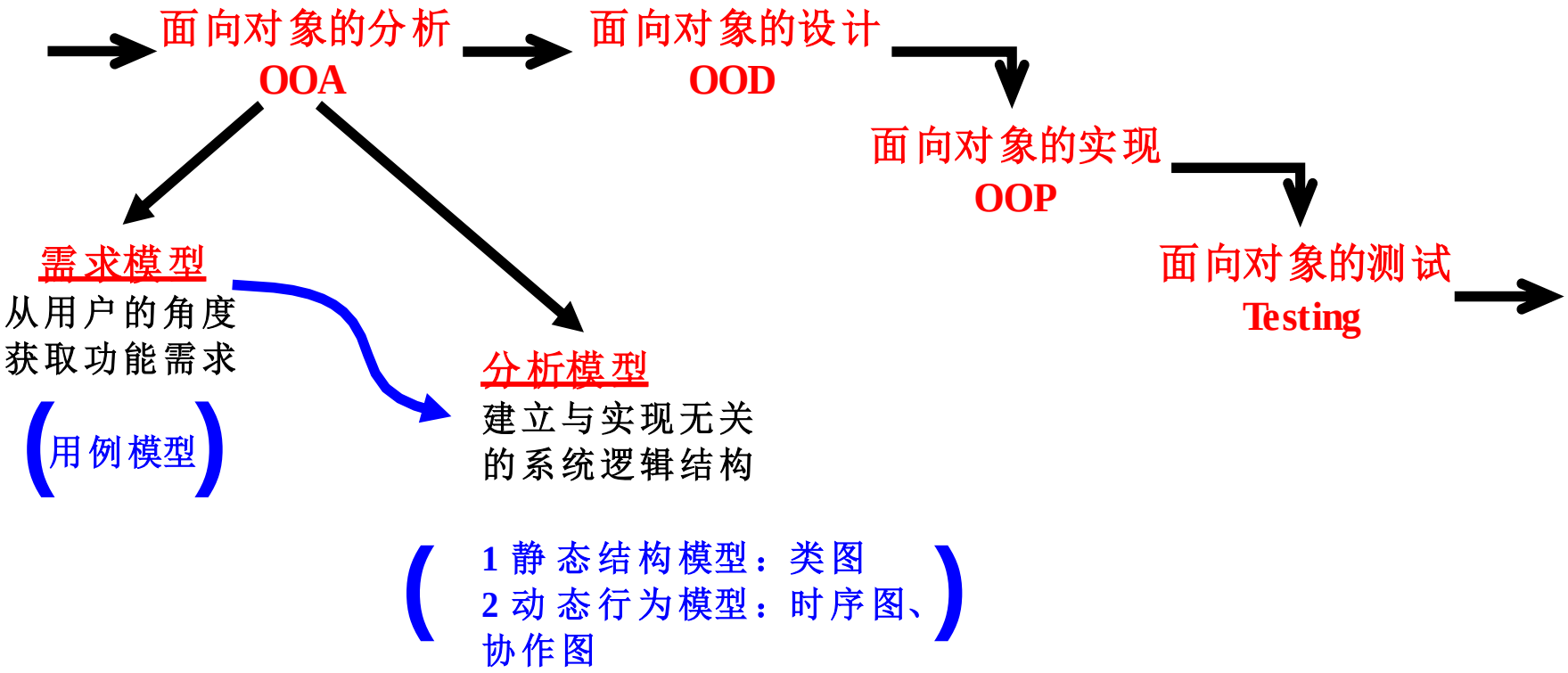
- 分析业务领域，找出问题解决方案，发现对象，分析对象的内部构成和外部关系，建立软件系统的对象模型
- 着重分析业务领域和系统责任，建立独立于实现的OOA模型，暂时忽略与系统实现有关的问题
- 主要使用5种图描述完整的系统需求：
  - 用例图
  - 类图
  - 时序图
  - 协作图
  - 状态图

# 面向对象的分析建模

- 面向对象的分析模型由三类模型构成：
  - 功能模型：从用户的角度获取功能需求，由用例模型表示
  - 静态结构模型(分析对象模型)：描述系统的概念实体，由类图表示
  - 动态行为模型：描述对象之间的交互行为，由时序图和协作图表示

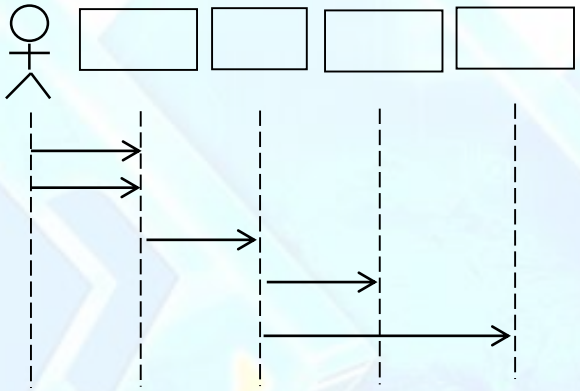
建立与实现技术无关的系统逻辑结构

# 面向对象的分析建模

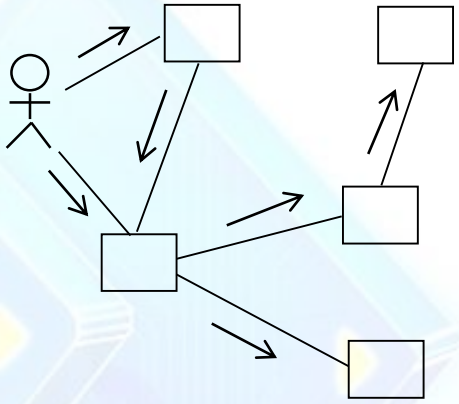

面向对象的分析
OOA
面向对象的设计
OOD
面向对象的实现
OOP
面向对象的测试
Testing
需求模型
从用户的角度
获取功能需求
(用例模型)
分析模型
建立与实现无关
的系统逻辑结构
(1 静态结构模型：类图
2 动态行为模型：时序图、
协作图)

# 面向对象的分析

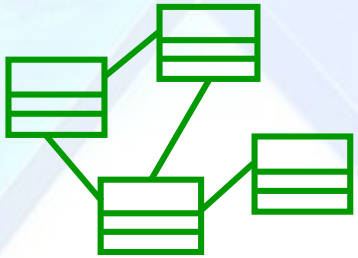
Use Case



Sequence Diagrams



Collaboration Diagrams



Class Diagrams

# 面向对象分析的过程

- 第一阶段：业务领域分析
  - 分析应用领域的业务范围、业务规则和业务处理过程，确定系统的责任、范围和边界，确定系统的需求
  - 在分析中需要着重对系统与外部的用户和其他系统的交互进行分析，确定交互的内容、步骤和顺序

业务用例模型 ⇨ 系统用例模型

# 面向对象分析的过程

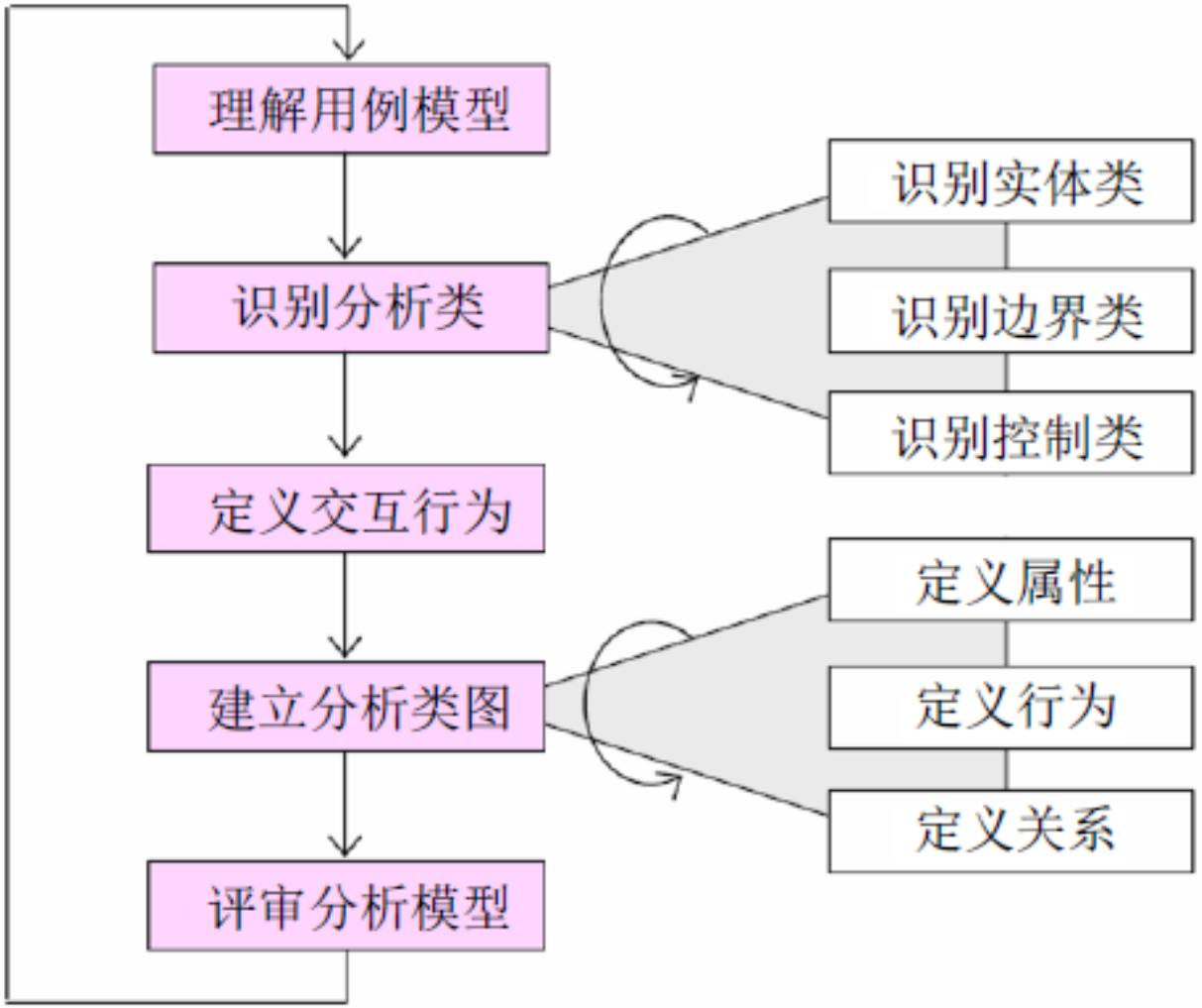
- 第二阶段：发现和定义对象和类
  - 识别对象和类，确定它们的内部特征：属性与服务操作
  - 这是一个从现实世界到概念模型的抽象过程，而抽象是面向对象分析的基本原则

- 第三阶段：识别对象的外部联系
  - 在发现和定义对象和类的过程中，需要同时识别对象与对象、类与类之间的各种外部联系，如一般与特殊、整体与部分、实例连接(关联)、消息连接等联系
  - 对象和类是现实世界中的事物的抽象，它们之间的联系也要从分析现实世界事物的各种真实的联系中获得

# 面向对象分析的过程

- 第四阶段：建立系统的静态结构模型
  - 分析系统的行为，建立系统的静态结构模型，并将其用图形和文字说明表示出来，如绘制类图、对象图、系统与子系统结构图等，编制相应的说明文档
- 第五阶段：建立系统的动态行为模型
  - 分析系统的行为，建立系统的动态行为模型，并将其用图形和文字说明表示出来，如绘制活动图、交互图、状态图等，编制相应的说明文档

# 面向对象分析的过程


理解用例模型
识别分析类
定义交互行为
建立分析类图
评审分析模型
识别实体类
识别边界类
识别控制类
定义属性
定义行为
定义关系

# 本章主要内容

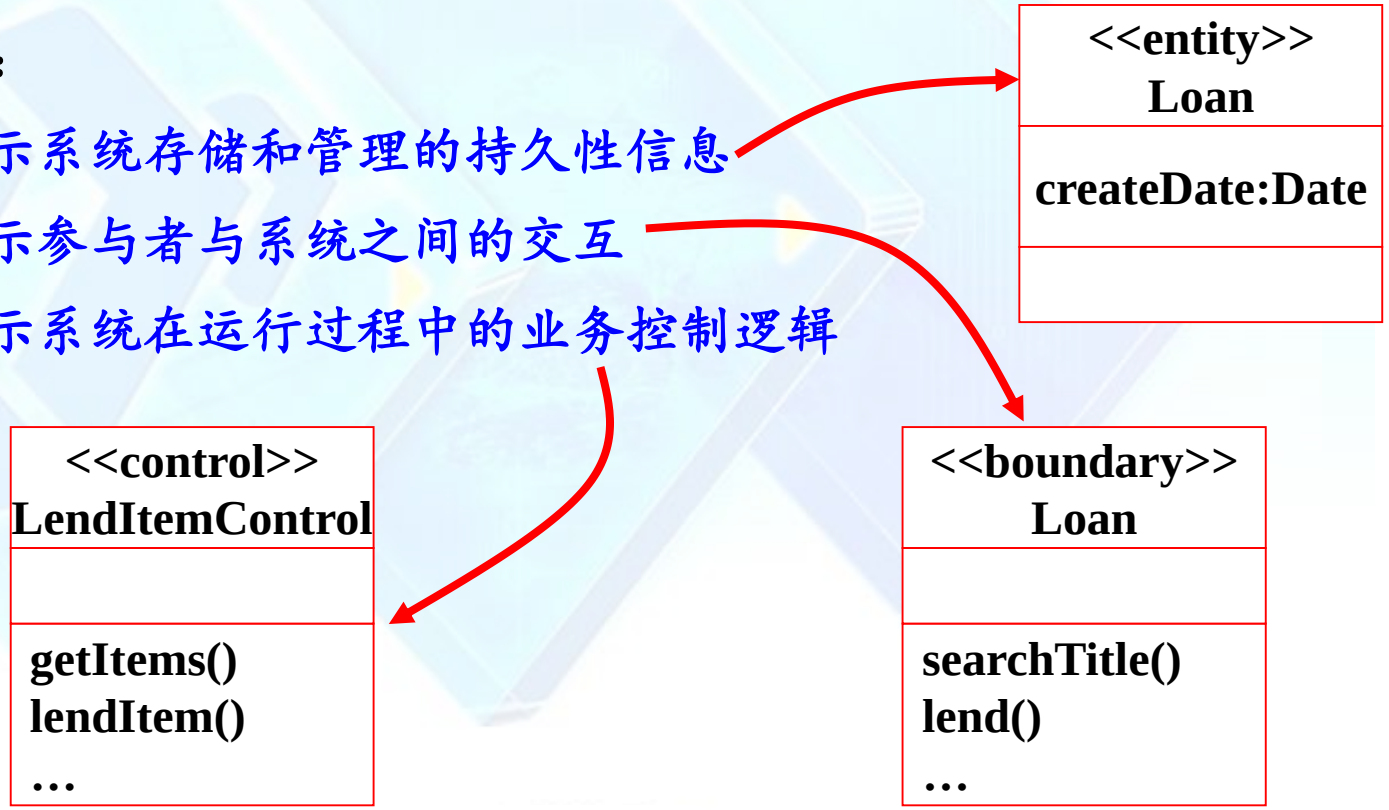
1. 面向对象的分析方法概述
2. 建立静态结构模型
3. 建立动态行为模型
4. 案例分析

# 建立静态结构模型

- 基本的分析过程：
  - Step 1：从用例模型入手，识别分析类；
  - Step 2：描述各个类的属性；
  - Step 3：定义各个类的操作；
  - Step 4：建立类之间的关系；
  - Step 5：绘制类图(class diagram)

# 什么是“分析类”？

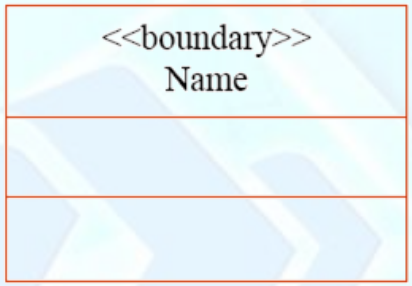
- 分析类是概念层次上的内容，用于描述系统中较高层次的对象
- 分析类直接与应用逻辑相关，而不关注于技术实现的问题
- 分析类的类型：
  - 实体类：表示系统存储和管理的持久性信息
  - 边界类：表示参与者与系统之间的交互
  - 控制类：表示系统在运行过程中的业务控制逻辑

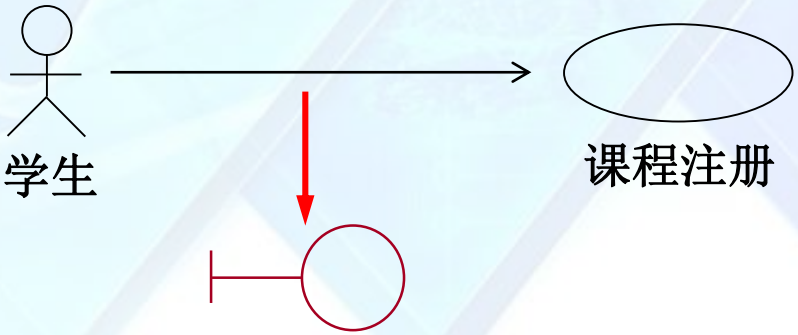


# 边界类(Boundary Class)

- 边界类：
  - 描述外部的参与者与系统之间的交互
  - 目的：将用例的内部逻辑与外部环境进行隔离，使得外界的变化不会影响到内部的逻辑部分
  - 类型：用户界面、系统接口、设备接口
- 对用户界面来说：
  - 描述用户与系统的交互信息(输入哪些信息/指令，输出哪些信息/指令)，而不是用户界面的显示形式(如按钮、菜单等)
- 对系统接口/设备接口来说：
  - 描述通信协议，但不必说明协议如何实现的

# 边界类(Boundary Class)



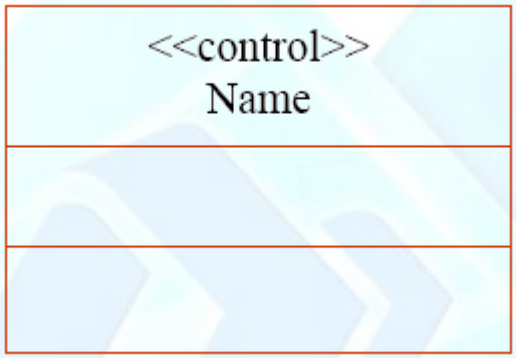


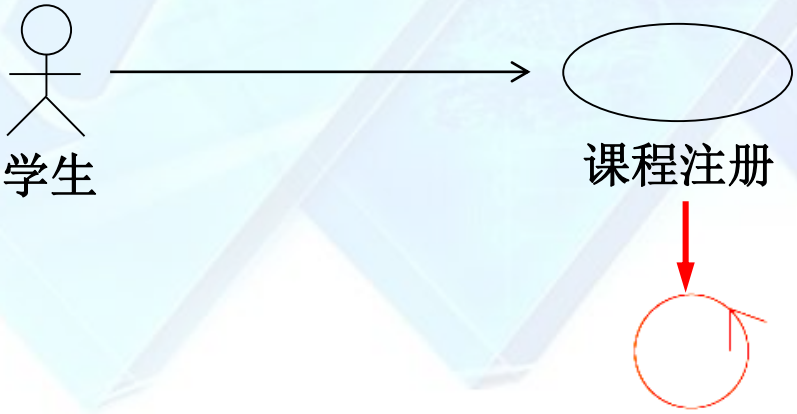

“学生注册课程”的界面

# 控制类(Control Class)

- 控制类
  - 描述一个用例所具有的事件流的控制行为， 本身并不处理具体的任务，而是调度其他类来完成具体的任务
  - 实现了对用例行为的封装，将用例的执行逻辑与边界和实体进行隔离，使得边界类和实体类具有较好的通用性

# 控制类(Control Class)



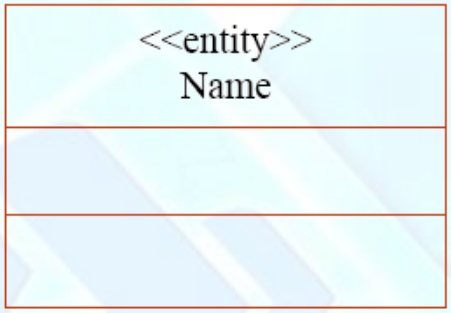



“课程注册”控制类

# 实体类(Entity Class)

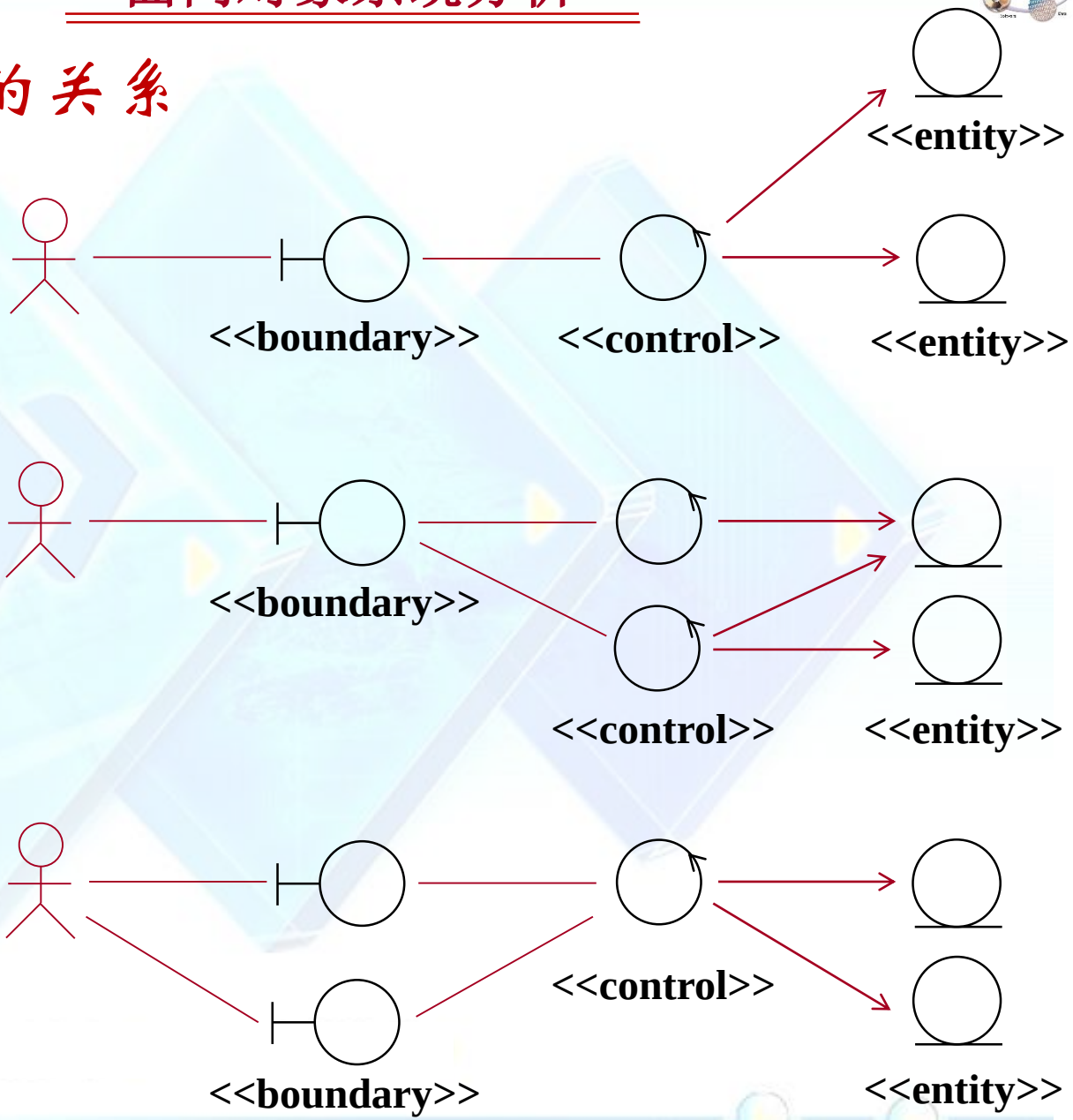
- 实体类
  - 描述必须存贮的信息及其相关行为
  - 对系统的核心信息建模，通常这些信息需要长久的保存
  - 通常对应现实世界中的“事物”

# 实体类(Entity Class)



# 三种分析类之间的关系

- 三种分析类之间是“关联”关系(association)
- 三种分析类之间是多对多(m:n)关系：
  - 一个边界类可以与多个控制类相关联
  - 一个控制类可以与多个边界类相关联、与多个实体类关联
  - 一个实体类可以与多个控制类相关联



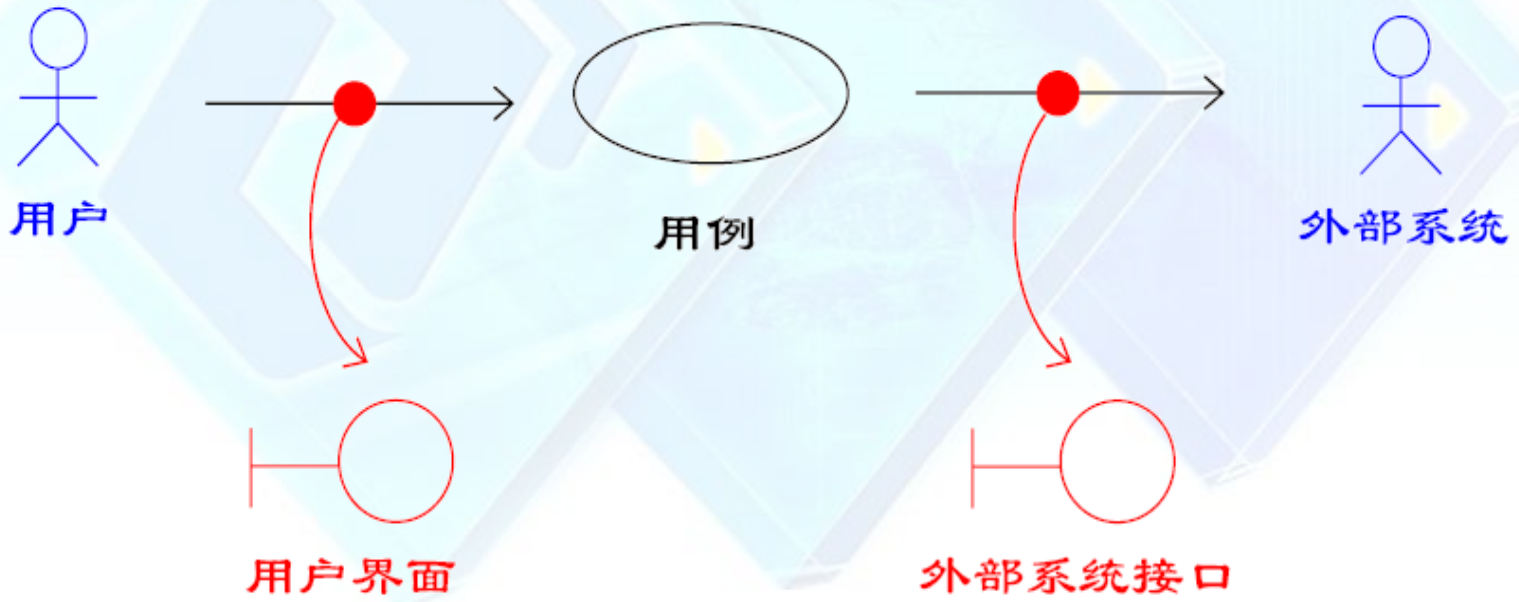
# Step 1：识别分析类

## 通用方法：

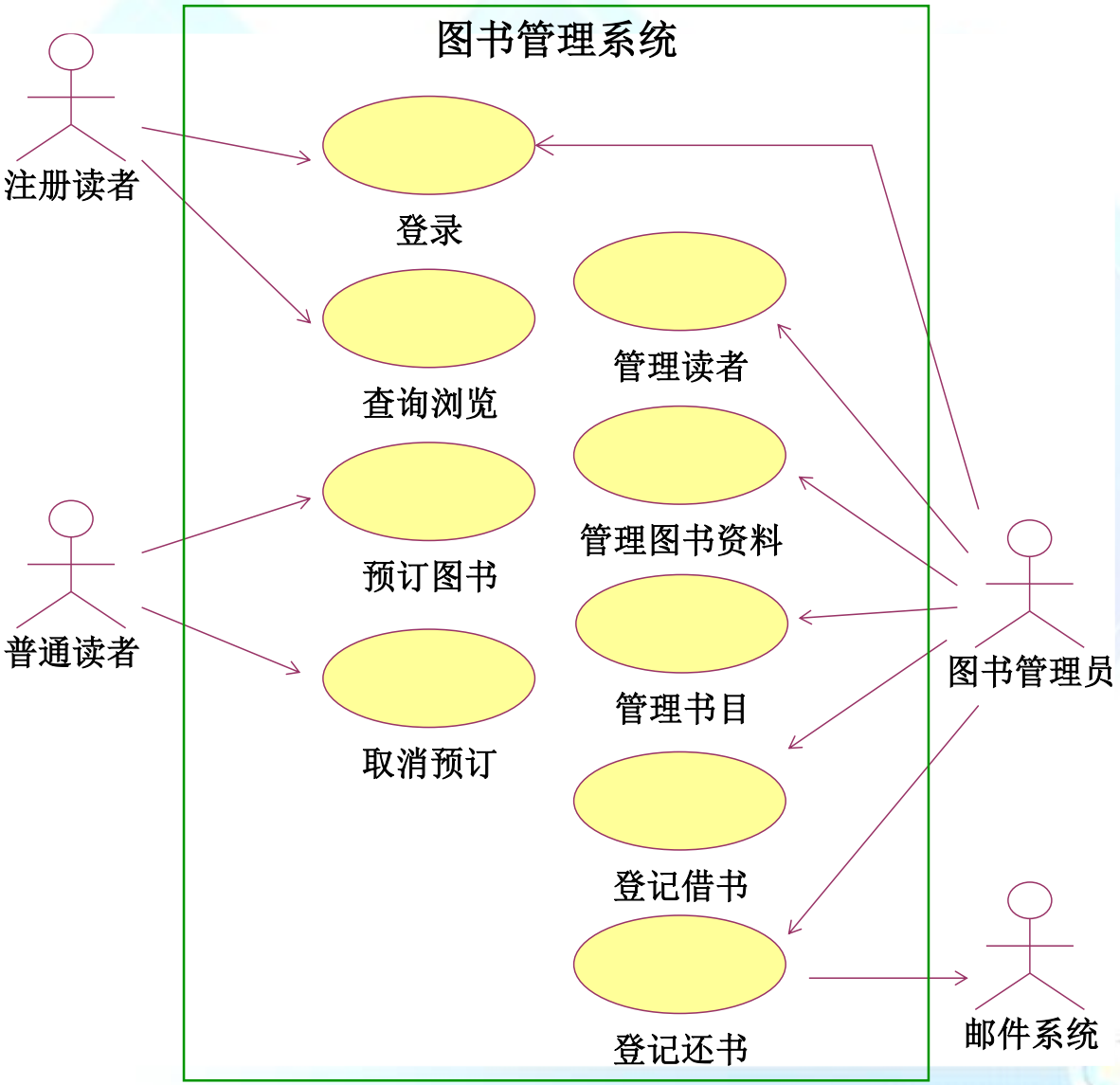
- 方法1：重用和修改现有的模型
  - 参考已有系统或书籍资料等
- 方法2：使用分类列表（不同领域不同的分类列表）
  - 外部实体、事物、事件、角色、组织、场地、结构等
- 方法3：对用例文本进行“语法分析”
  - 分析用例文本，对用户需求陈述进行“语法分析”，找出所有的名词或名词短语，对其标注下划线
  - 合并同义词
  - 将最终得到的每一个名词映射为一个类
  - 动词映射为类的操作，形容词/名词映射为类的属性

# Step 1：识别分析类之边界类

- 识别边界类
  - 通常，一个参与者与一个用例之间的交互或通信关联对应一个边界类



# [案例]图书管理系统的用例图



# [案例]图书管理系统的边界类

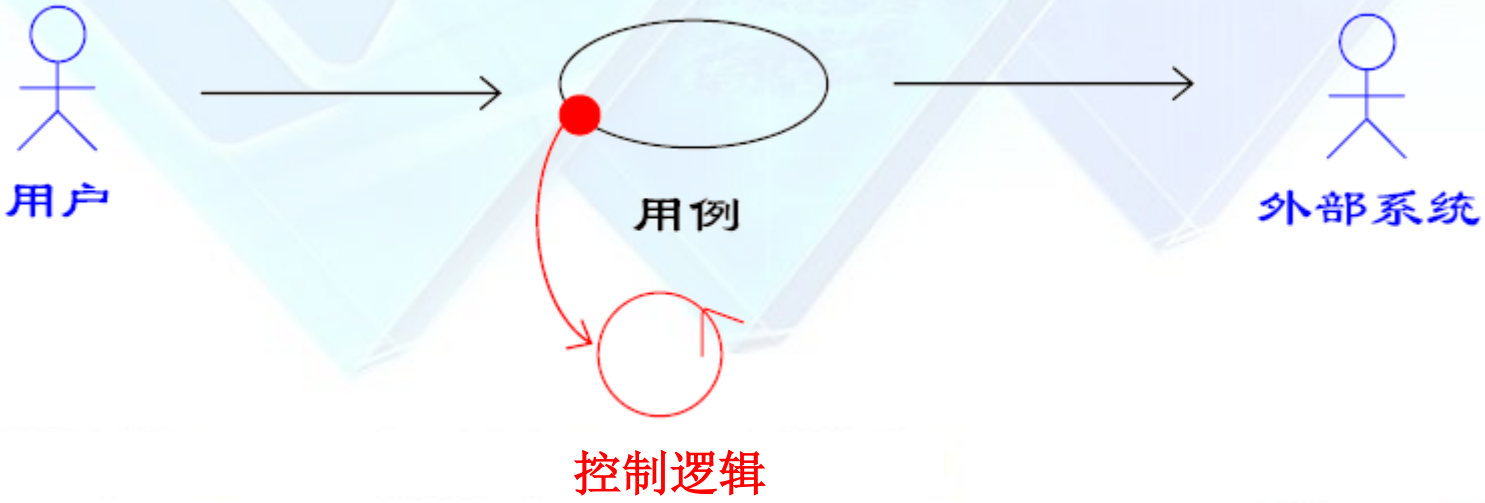
| 边界类 | 说明 |
| --- | --- |
| LoginForm | 注册用户进行登录的操作界面 |
| BrowseForm | 注册用户进行查询浏览的操作界面 |
| MakeReservationForm | 普通读者预定图书的操作界面 |
| RemoveReservationForm | 普通读者取消预定的操作界面 |
| ManageBrowsersForm | 图书管理员管理读者的操作界面 |
| ManageTitlesForm | 图书管理员管理图书资料的操作界面 |
| ManageItemsForm | 图书管理员管理书目的操作界面 |
| LendItemForm | 图书管理员登记借书的操作界面 |
| ReturnItemForm | 图书管理员登记还书的操作界面 |
| MailSystemAPI | 与邮件系统的接口 |

# 识别边界类应当注意的问题

- 边界类应关注于参与者与用例之间交互的信息或者响应的事件，不要描述窗口组件等界面的组成元素
- 在分析阶段，力求使用用户的术语描述界面
- 边界类实例的生命周期并不仅限于用例的事件流，如果两个用例同时与一个参与者交互，那么它们有可能会共用一个边界类，以便增加边界类的复用性

# Step 1：识别分析类之控制类

- 识别控制类
  - 控制类负责协调边界类和实体类，通常在现实世界中没有对应的事物
  - 负责接收边界类的信息，并将其分发给实体类
  - 控制类与用例存在着密切的关系，它在用例开始执行时创建，在用例结束时取消
  - 一般来说，一个用例对应一个控制类



# [案例]图书管理系统的控制类

| 控制类 | 说明 |
| --- | --- |
| BrowseControl | 负责执行注册用户的查询浏览 |
| MakeReservationControl | 负责执行普通读者的预定图书 |
| RemoveReservationControl | 负责执行普通读者的取消预定 |
| ManageBrowsersControl | 负责执行图书管理员对读者的管理 |
| ManageTitlesControl | 负责执行图书管理员对图书资料的管理 |
| ManageItemsControl | 负责执行图书管理员对书目的管理 |
| LendItemControl | 负责执行图书管理员登记借书 |
| ReturnItemControl | 负责执行图书管理员登记还书 |

# 识别控制类应当注意的问题

- 当用例比较复杂时，特别是产生分支事件流的情况下，一个用例可以有多个控制类
- 在有些情况下，用例事件流的逻辑结构十分简单，这时没有必要使用控制类，边界类可以实现用例的行为
  - 例如：图书管理系统中的“登录”用例
- 如果不同用例包含的任务之间存在着比较密切的联系，则这些用例可以使用同一个控制类，其目的是复用相似部分以便降低复杂性
  - 通常情况下，应该按照一个用例对应一个控制类的原则识别出多个控制类，再分析这些控制类找出它们之间的共同之处，以及是否需要在分解控制类

# Step 1：识别分析类之实体类

- 如何从用户的需求陈述中找到“实体类”？
  - 名词驱动的识别方法
- 实体类通常是用例动作的参与对象，对应着现实世界中的“事物”
- 对用户需求陈述进行“语法分析”，找出所有的名词或名词短语，对其标注下划线
- 判断一个“名词”是否为实体类，其标准是：
  - 系统是否需要管理该名词所拥有的信息？
  - 系统是否需要管理该名词所能发出或接受的动作？
- 合并同义词
- 将最终得到的每一个名词映射为一个实体类
- 动词映射为类的操作，形容词/名词映射为类的属性

# [案例]图书管理系统的实体类

| 实体类 | 说明 |
| --- | --- |
| BrowserInfo | 普通读者的基本信息 |
| Loan | 普通读者的借书记录 |
| Reservation | 普通读者的预定信息 |
| Title | 图书资料的基本信息 |
| Item | 书目 |
| 由于图书资料中包括书籍和杂志等类型，因此可以进一步划分子类 | |
| BookTitle | 书籍的基本信息 |
| MagazineTitle | 杂志的基本信息 |

# 识别实体类应当注意的问题

- 实体类的识别质量在很大程度上取决于分析人员书写文档的风格和质量
- 自然语言是不精确的，因此在分析自然语言描述时应该规范化描述文档中的一些措辞，尽量弥补这种不足
- 在自然语言描述中，名词可以对应类、属性或同义词等多种类型，开发人员需要花费大量的时间进行筛选

# 注意：actor == 实体类？

- “角色”(actor)是否一定是实体类？—— NO
- 除非系统需要在各用例中管理和维护该角色的信息(不是指ID和密码)，否则只需将其作为actor，无需作为实体类
- 例如：
  - 选课系统中：教学秘书仅属于actor，无需作为实体类；但学生和老师既是actor也是实体类，因为课程/课表/选课单等实体中均需要使用学生和老师的信息
  - 淘宝系统中：淘宝平台管理员/支付宝/物流系统均为actor
- 对用户(actor)的权限管理，是任何系统均具备的通用功能，无需在实体类中考虑；将来放在基础设施层实现即可

# Step 2：描述分析类的属性

- 每个类都应回答的问题：“什么数据项(组合项、基本项)能够在当前问题环境内完整地定义这个类？”
- 按照一般常识，找出对象的某些属性
  - 人员的姓名、性别、年龄、地址等
- 认真研究问题域，找出对象的某些属性
  - 商品的条形码、学生的学号等
- 根据系统责任的要求，找出对象的某些属性：比如权限
- 考虑对象需要系统保存的信息，找出对象的相应属性：比如工作进度
- 对象为了在服务中实现其功能，需要增设一些属性：比如临时数据
- 识别对象需要区别的状态：比如订单状态
- 确定属性表示整体与部分结构和实例连接：比如父节点的ID
- ……

# Step 2：描述分析类的属性之边界类、控制类

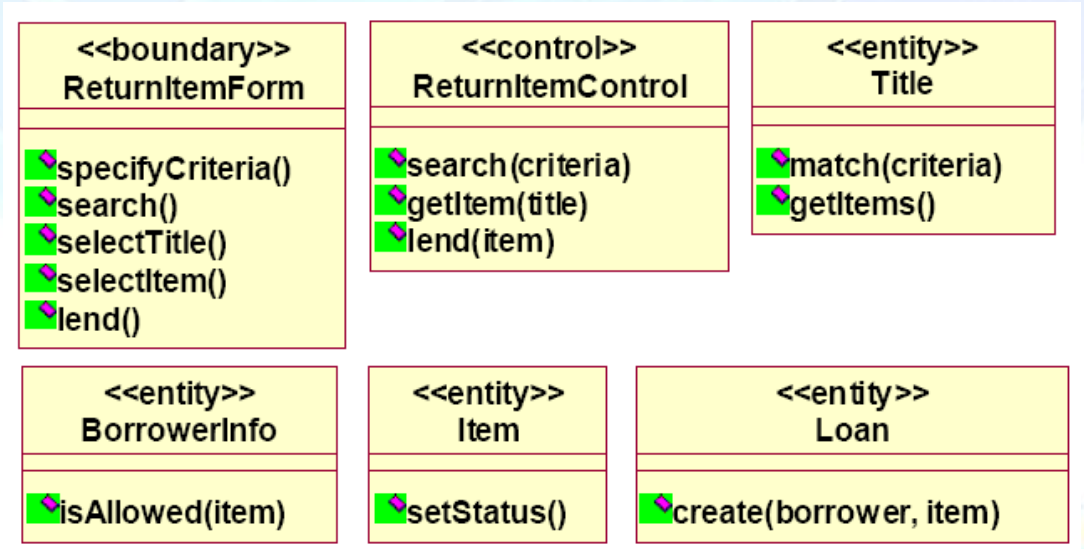
- 对UI类型的边界类：
  - 需要actor输入的各个数据
  - 系统反馈给actor的各个数据
  - 需要临时保存的、用于在边界类和控制类之间传递的临时数据
    例如：登录次数、短信验证码发送时间
- 对API类型的边界类：
  - 需要向外部系统(软件/硬件)传递的数据
  - 需要从外部系统(软件/硬件)接收的数据
- 控制类的属性：
  - 从UI接收的数据
  - 为进行事件流执行所需的临时数据
  - 需要调用的实体类
  - 经过计算之后、需要发送给UI的数据

# Step 2：描述分析类的属性之实体类

- 基本属性：
  - 按照一般常识，找出实体类的基本属性
  - 考虑对象需要系统保存的信息，找出对象的相应属性（例如访问权限、学生学费缴纳情况）
- 状态属性：
  - 识别对象需要区别的状态，考虑是否需要增加一个属性来区别这些状态
- 关联属性：
  - 确定属性表示整体与部分结构和实例连接
- 派生属性：
  - 通过计算其他属性的值所得到的新属性（例如实发工资、扣税）

# Step 3：定义分析类的操作

- 将用例行为分配到相应的分析类之后，系统的一些分析类具有相应的职责；可通过动词分析获得操作
- 操作的四种类型：
  - 以某种方式操作数据（例如：增加、删除、重新格式化、选择等）
  - 执行计算的操作
  - 请求某个对象的状态的操作
  - 监视某个对象发生某个控制事件的操作



# Step 3：定义分析类的操作

- 边界类的操作：
  - 输入数据验证（如长度、字符集、范围、格式）
  - 提供给用户的、可在UI上进行的各类操作（如搜索、结算、设置）
  - 对从控制类返回数据进行各类临时处理而进行的操作（如大写转换）
  - 提供给其他系统的API（如扣款、查询账户余额）
- 控制类的操作：
  - 对从边界类接收到的数据进行各类临时处理而进行的操作（如分词）
  - 向实体类所发出的调用操作（如增、删、改、查）
  - 对从实体类接收到的数据进行临时处理而进行的操作（如排序）
- 实体类的操作：
  - 对属性进行CRUD的操作（Create/Retrieve/Update/Delete）
  - 对状态进行更新的操作
  - 相关的业务操作（如匹配、统计汇总、计算）

# 从用例出发，识别类的操作

- 用例的事件流描述了一系列的“动作”：
  - actor做的；系统自身做的
- 如何把这些动作映射到分析类的操作中？简要归纳一下原则：
  - actor自己做的动作，都是边界类的操作，与控制类和实体类无关
  - 边界类的操作会调用控制类的操作，并进一步被分解为对各个不同实体类的操作的调用序列
  - actor发出的动作通常都比较大，往往涉及到对多个实体类的属性的增删改查(CRUD)，不能简单的将它直接放置在某个实体类中
- 实体类的操作分为两类：
  - 对该实体的生命周期的操作：如new( )、destroy( )
  - 对该实体内部的一个或多个属性的CRUD操作
    注意：是自己的属性，不是其他实体的！
- 因此，对用例中actor所发出的每个动作，需要仔细分析它对哪些实体类的哪些属性做CRUD

# 从用例出发，识别类的操作

- 例如：用例“从购物车生成订单”
  - 事件流：
    - 1） 用户点击“去结算”；系统根据当前购物车对象中包含的商品信息，生成一张订单
    - 2） 用户填写订单的收货地址、付款方式等信息，点击“去付款”，系统永久存储这张订单，引导用户到达支付宝页面
- 用例中涉及的实体类有：
  - 购物车、商品、收货地址、订单、订单项等

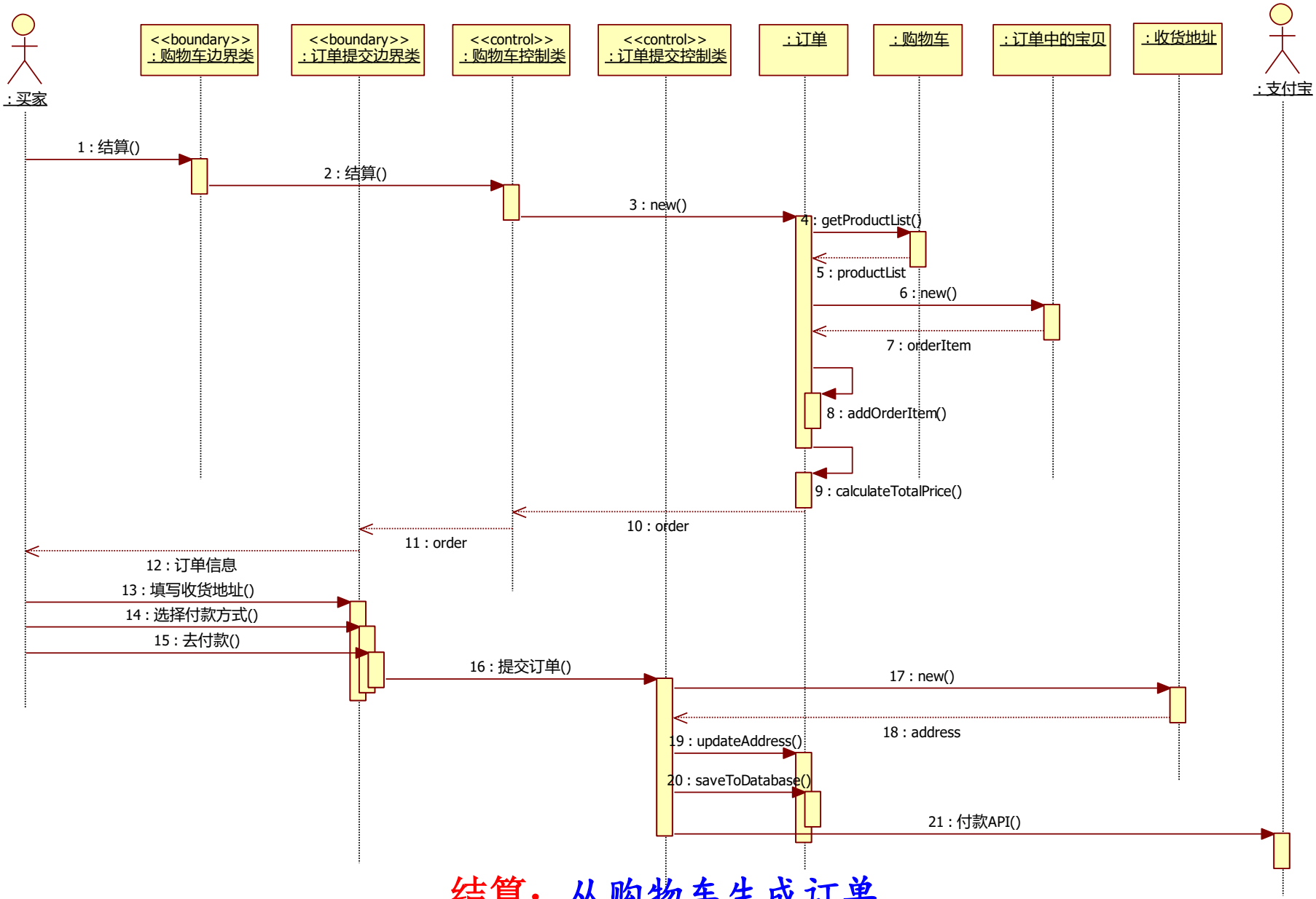
# 从用例出发，识别类的操作

- 为了完成上述事件流，“从购物车生成订单”控制类逐步调用这些实体类的多个操作 (此刻程序运行空间中必然已经有一个“购物车”类的实例对象cart)：
  - “订单”类的new操作：Order newOrder = new Order(cart)，然后再从cart对象中读取商品列表，形成一个新的订单对象newOrder
  - 为此，“购物车”对象需要提供一个操作供订单类的new操作所使用：getProductList( )，此操作返回购物车对象中包含的所有商品(来自购物车类的关联属性ProductList，这是一个数组，由0到多个“商品”类构成)
  - 获得商品信息之后，订单对象还需调用多次“订单项”类的new操作，构造多个订单项对象，并与自己聚合起来(需要一个操作addOrderItem( ))

## 从用例出发，识别类的操作

- 接下来，控制类还需要根据订单包含的商品，计算总价格并展示；总价格是“订单”类的一个派生属性，订单类需要一个操作**calculateTotalPrice( )**，该操作会读取订单对象中聚合的多个订单项对象并加以计算，将结果更新到订单类的“总价格”属性中
- 经过上述步骤之后，控制类才算真正构造完一个订单对象
- 接下来，当用户填写完收货地址后，需要首先调用“收货地址”类的**new**操作，根据用户填写的信息构造一个收货地址对象**address**
- 调用“订单”对象的操作：**updateAddress(address)**，将刚才构造的**address**传递进去，更新订单对象**order**的属性**address**(其类型就是“收货地址”类)
- 当用户点击付款时，意味着需要将该订单进行持久化存储，故需要调用“订单”类的另一个操作：**saveToDatabase( )**，该操作将对象中保存的所有属性信息更新到关系数据表中
- 同时，系统调用支付宝的**API**，触发支付宝的行为。该行为就不需要刻画了，因为它属于外部系统，通过**API**调用即可

# 面向对象系统分析




结算：从购物车生成订单

# 从用例出发，识别类的操作

- 汇总一下，从这个用例中得到的实体类属性和操作如下：

**订单类**

---

订单号: string
创建日期: Date
包含的商品: List
收货地址: Address
总价格: double

---

new(Cart)
addOrderItem (List)
calculateTotalPrice()
updateAddress(Address)
saveToDatabase()

**订单项类**

---

商品: Product
价格: double
数量: int

---

new(Product)

**商品类**

---

名字: string
价格: double

---

getPrice()

**收货地址类**

---

收货人: string
电话: string
地址: string

---

new(…)

**购物车类**

---

商品清单: List

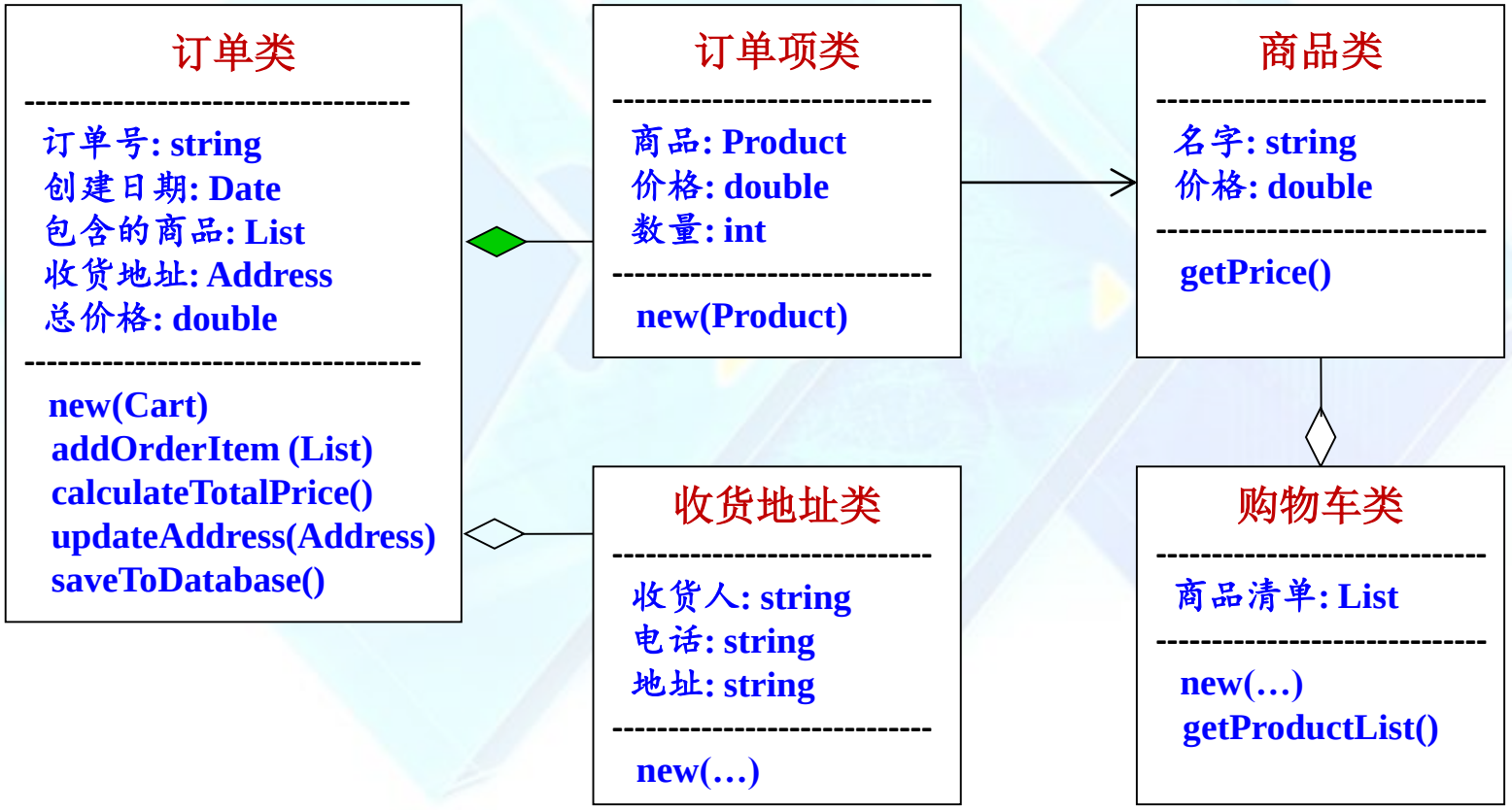
---

new(…)
getProductList()

**讨论：请给出该类图中各类之间的联系**

# 从用例出发，识别类的操作

- 汇总一下，从这个用例中得到的实体类属性和操作如下：




增加实体类间联系

## 从用例出发，识别类的操作

- 但是，这几个实体类的属性和操作并不完整
  - 继续考虑其他每一个用例，在其他用例中，类似的会发现其他所需要的属性和操作，继续加入到实体类中
  - 当考虑完所有的用例之后，实体类的属性集和操作集就完整了
- 判断类的“操作”设计是否合理的标准：
  - 对该实体类的任何一个属性的CRUD，均有特定的操作加以完成（而不是将属性的可见性设定为public，由使用者直接读取和更新操作的值）
  - 若一个操作CRUD的对象不是该实体类的属性，那么该操作不该出现在这里，把它移到相应的实体类中去
  - 若发现某个操作op会CRUD多个实体类的属性值，通常意味着这个操作需要分解为多个小粒度操作(op1,op2,…)，每个opi放在它专门的实体类中。此时，op本身就不会成为某个实体类的操作，而通常将它放在某个控制类中，由该控制类分别来调用op1,op2,…
    - “从购物车生成订单”这个操作，就分解为了“订单”、“购物车”、“订单项”、“收货地址”这四个实体类的多个小操作

# 从用例出发，识别类的操作

- 有一类用例中的操作，没办法分解为小粒度op，尤其查询类的操作
  - 例如：“用户通过关键字查询商品”，返回的是一组商品；该操作绝不会是“商品”类的操作，因为任何一个“商品”对象都不可能包含淘宝上的全体商品
- 所以，这个操作只能由控制类加以完成：
  - 根据用户提供的关键字，直接到数据库中检索出满足条件的商品清单
  - 基于数据库中的每条商品记录，使用“商品”类的new操作构建一个“商品”对象，形成一个List，返回给相应的边界类，边界类对其加以展示
- 另一个办法：在领域模型中增加一个实体类“商品清单”，其中包含了所有的“商品”对象。这样的话，“通过关键字查询”就成了该实体类的一个操作
  - 问题在于：需要在系统初始化时就将该实体类构造出来，否则无法被使用；这明显是不可能的，占用内存空间太大

# Step 4：建立类之间的关系

- 五种关系：
  - 泛化(generalization)
  - 关联(association)
  - 组合(composition)
  - 聚合(aggregation)
  - 依赖(dependency)
- 例如：
  - “读者”类与“预定”类之间有1:n的关联关系
  - “读者”类与“借书记录”类之间有1:n的关联关系
  - “图书资料”类与“借书记录”类之间有1:n的关联关系
  - “书目”类与“书籍”类、“杂志”类之间是泛化关系

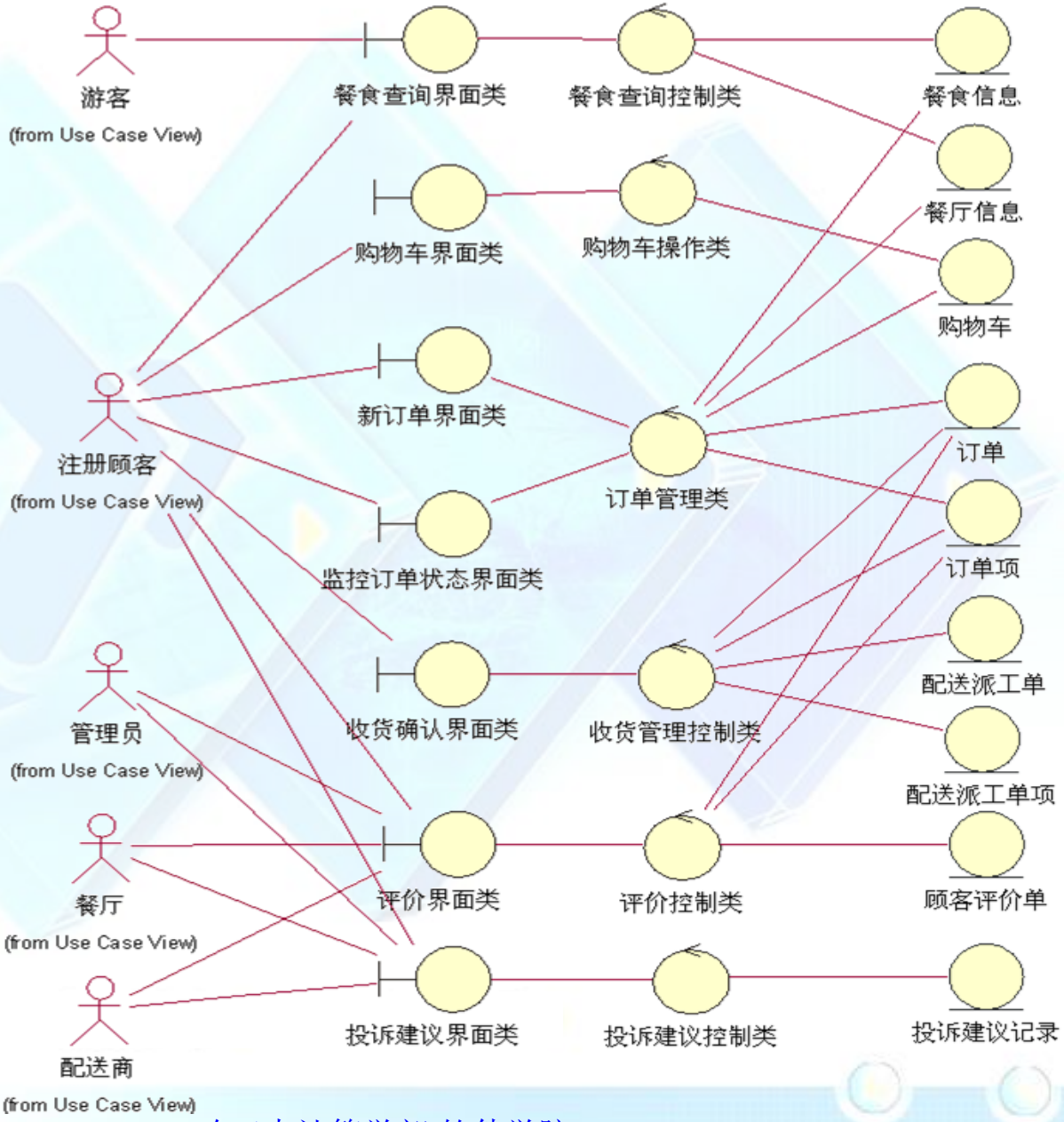
# Step 5：绘制类图

- 两种类图：
  - 分析类图：描述各边界类、实体类、控制类之间的关联关系，无需刻画属性与操作集
  - 领域类图：可以不包含边界类与控制类，侧重描述各实体类之间的五种关系，需要给出详细的属性与操作集合

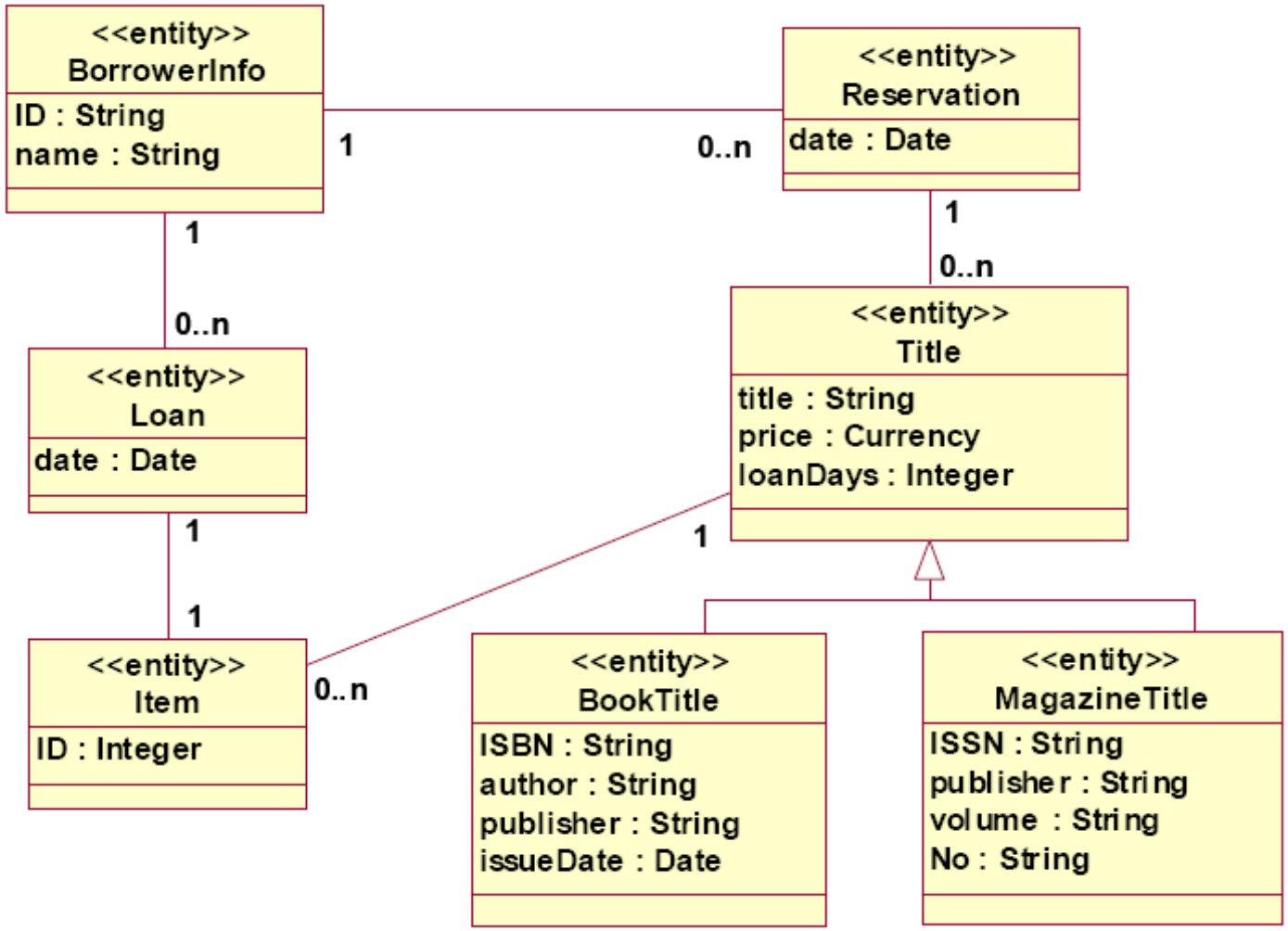
- 在不加说明的情况下，领域类图更有价值

# 面向对象系统分析

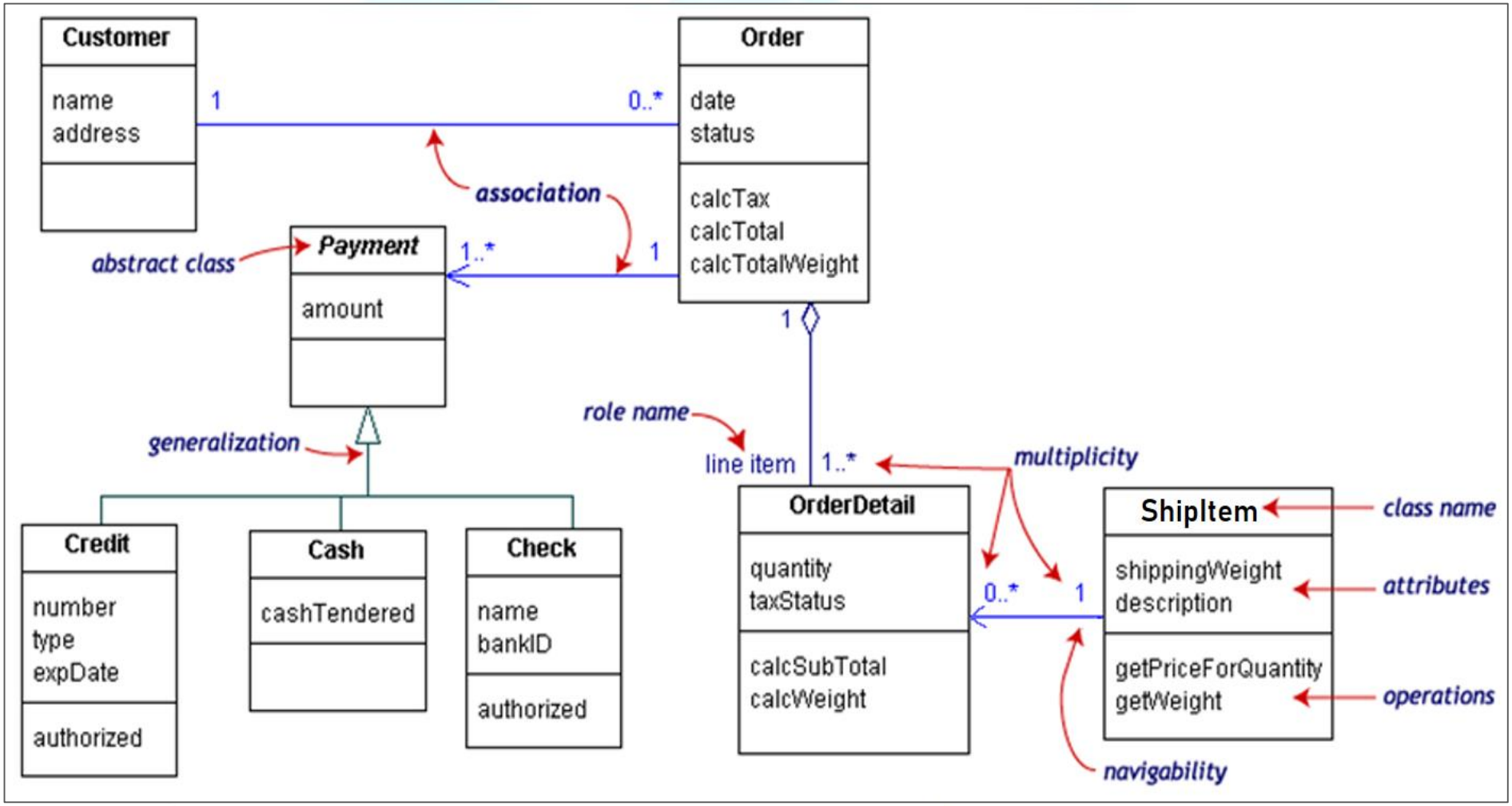
## 分析类图实例



# 领域类图实例(1)


<<entity>>
BorrowerInfo
ID : String
name : String
<<entity>>
Reservation
date : Date
1
0..n
1
0..n
<<entity>>
Loan
date : Date
<<entity>>
Title
title : String
price : Currency
loanDays : Integer
1
1
1
1
0..n
<<entity>>
Item
ID : Integer
<<entity>>
BookTitle
ISBN : String
author : String
publisher : String
issueDate : Date
<<entity>>
MagazineTitle
ISSN : String
publisher : String
volume : String
No : String

# 领域类图实例(2)

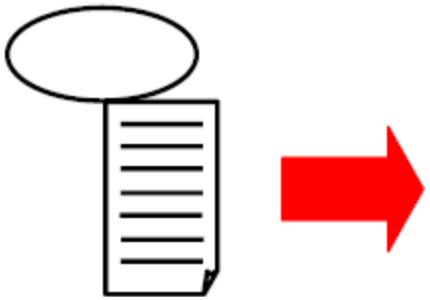

Customer
name
address
Order
date
status
calcTax
calcTotal
calcTotalWeight
1
0..*
association
1..*
1
abstract class
Payment
amount
1
role name
line item
1..*
multiplicity
generalization
Credit
number
type
expDate
authorized
Cash
cashTendered
Check
name
bankID
authorized
OrderDetail
quantity
taxStatus
calcSubTotal
calcWeight
0..*
1
ShipItem
shippingWeight
description
getPriceForQuantity
getWeight
class name
attributes
operations
navigability

# 本章主要内容

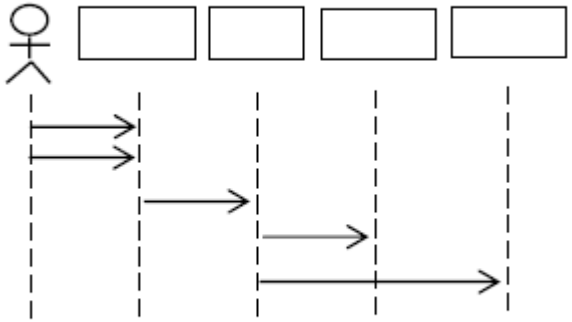
1. 面向对象的分析方法概述
2. 建立静态结构模型
3. 建立动态行为模型
4. 案例分析

# 建立动态行为模型

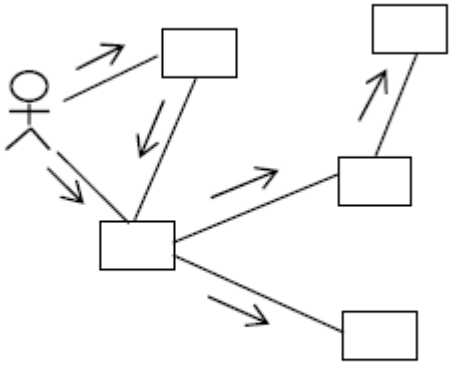
- 动态行为模型可用两个新视图描述：
  - 时序图/顺序图(Sequence Diagram)
  - 协作图(Collaboration Diagram)



用例



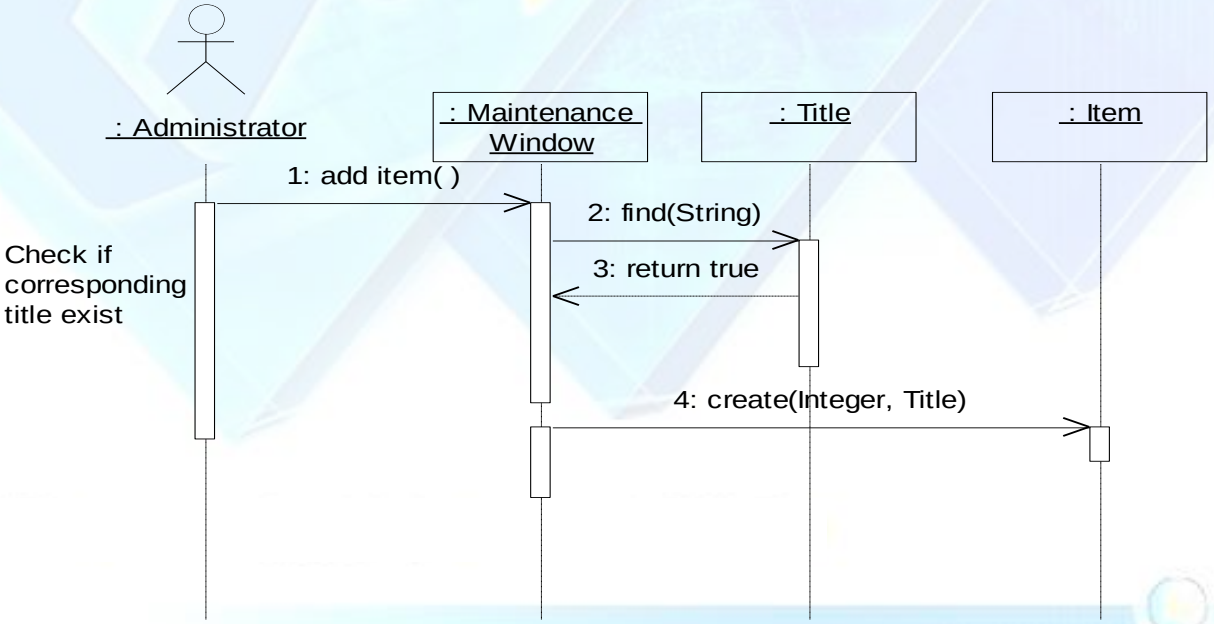
时序图



协作图

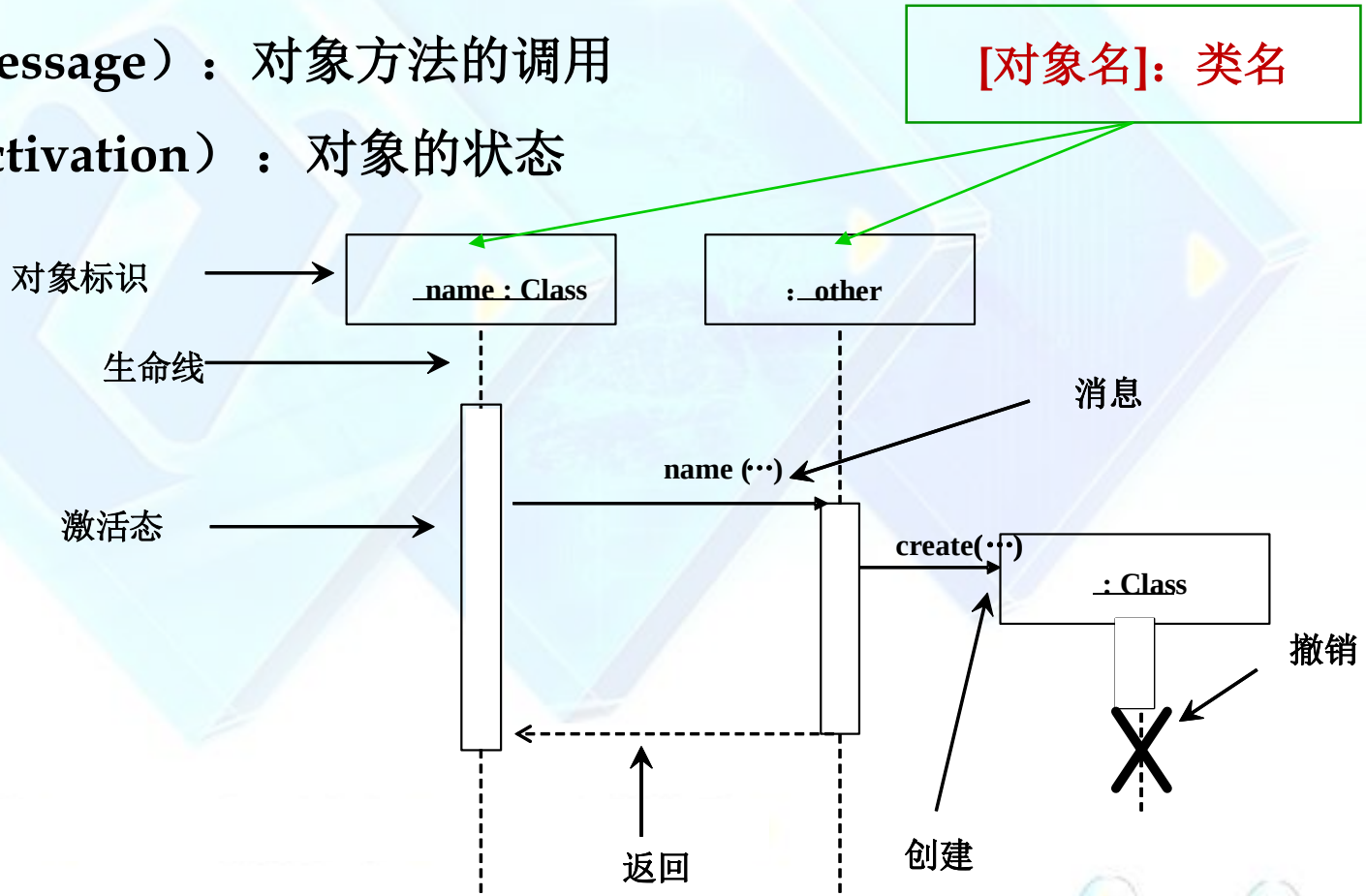
# 时序图

- 时序图：将用户与分析类结合在一起，实现将用例的行为分配到所识别的分析类中
- 时序图是强调消息时间顺序的交互图
- 时序图描述了对象之间传送消息的时间顺序，表示用例中的行为顺序
- 时序图将交互关系表示为一个二维图：纵轴是时间轴，时间沿竖线向下延伸；横轴代表了在协作中各独立的对象



# 时序图的组成

- 对象（Object）：类的实例
- 生命线（Lifeline）：生命线是一个时间线
- 消息（Message）：对象方法的调用
- 激活（Activation）：对象的状态



# 消息

- 消息定义的是对象之间某种形式的通信，它可以激发某个操作、唤起信号或导致目标对象的创建或撤销
- 消息是两个对象之间的单路通信，从发送方到接收方的控制信息流
- 消息可以用于在对象间传递参数
- 消息可以是信号，也可以是调用
- 在UML中，消息使用箭头来表示，箭头的类型表示了消息的类型

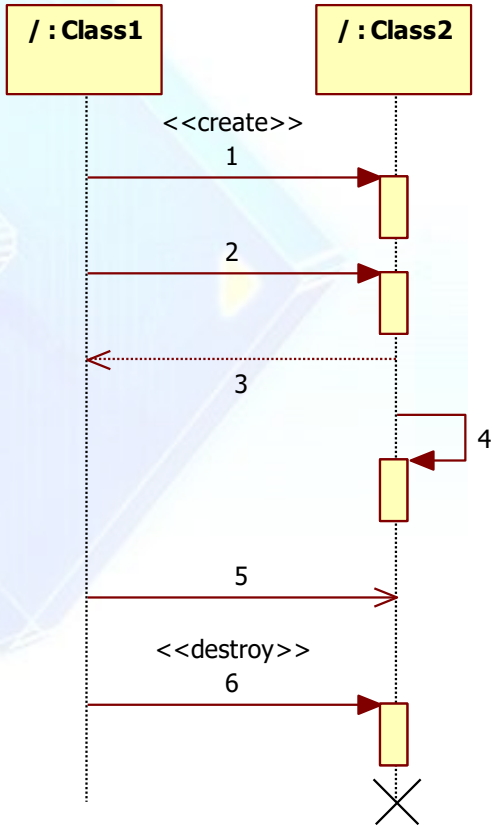
同步消息

异步消息

返回消息

# 时序模型

- 时序模型中的每一列：本质上是object，每个出现的object，其相应的class必须都在分析类模型/领域模型中已经定义过了
  - 这些object的排列次序无所谓，但按照惯例，一张时序图从左到右应该是：actor(用户)、边界类、控制类、实体类、actor(外部系统)
- 出现在时序图中的箭头，UML中分了六种类型：
  - Create：obj1调用Class2的new操作，创建出obj2实例
  - Call：obj1调用obj2的某个操作
  - Return：obj2被调用的操作执行结束后，将结果返回obj1
  - Self-call：obj2自己调用自己内部的某个操作；
  - Send：obj1向obj2发送消息来触发(而不直接调用其操作)
  - Destroy：obj1调用Class2的destroy操作，销毁obj2实例
  - 图形表示方式分别如右图中的1~6箭头所示



# 关于时序模型

- 任何类型的箭头(create、call、return、self-call、send、destroy)，均代表着该箭头需要触发obj2的某个操作
- 这就意味着，箭头上标识的操作，必须在obj2的操作集合中有所定义
- 所以，如果在分析类和领域建模阶段没有想全，建立时序模型时可以通过一致性检查发现缺失

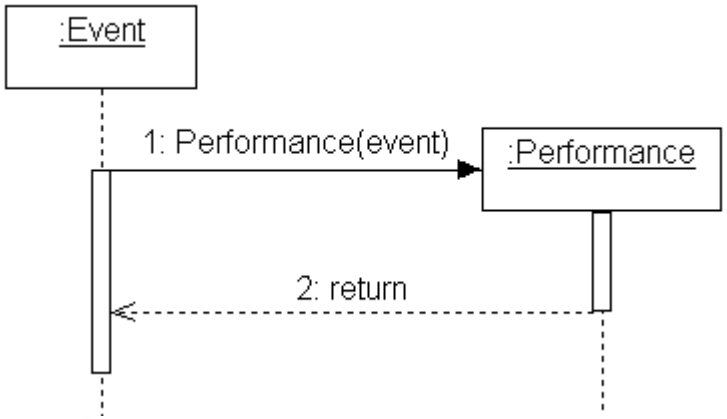
## 激活

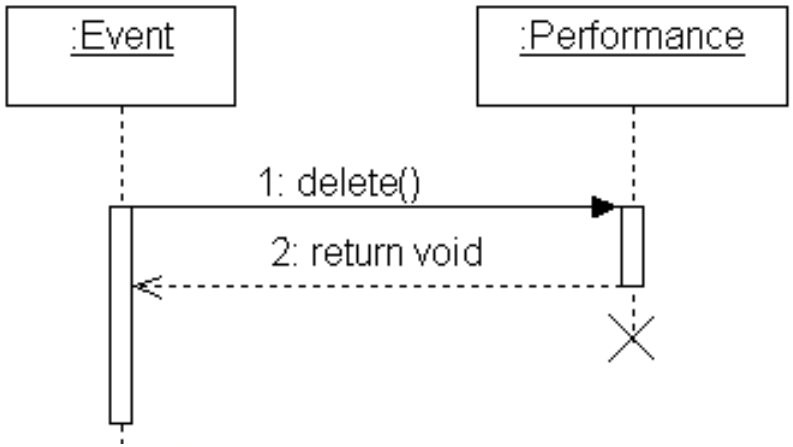
- 激活表示该对象被占用以完成某个任务，去激活指的则是对象处于空闲状态、在等待消息
- 在UML中，为了表示对象是激活的，可以将该对象的生命线拓宽成为矩形。其中的矩形称为激活条或控制期，对象就是在激活条的顶部被激活的，对象在完成自己的工作后被去激活

- 激活条

# 对象的创建和撤销

- 如果对象位于时序图的顶部，说明在交互开始之前该对象已经存在了；如果对象是在交互的过程中创建的，那么它应当位于图的中间部分
- 对象在创建消息发生之后才能存在，对象的生命线也是在创建消息之后才存在的
- 如果要撤销一个对象，只要在其生命线终止点放置一个“X”符号即可，该点通常是对删除或取消消息的回应

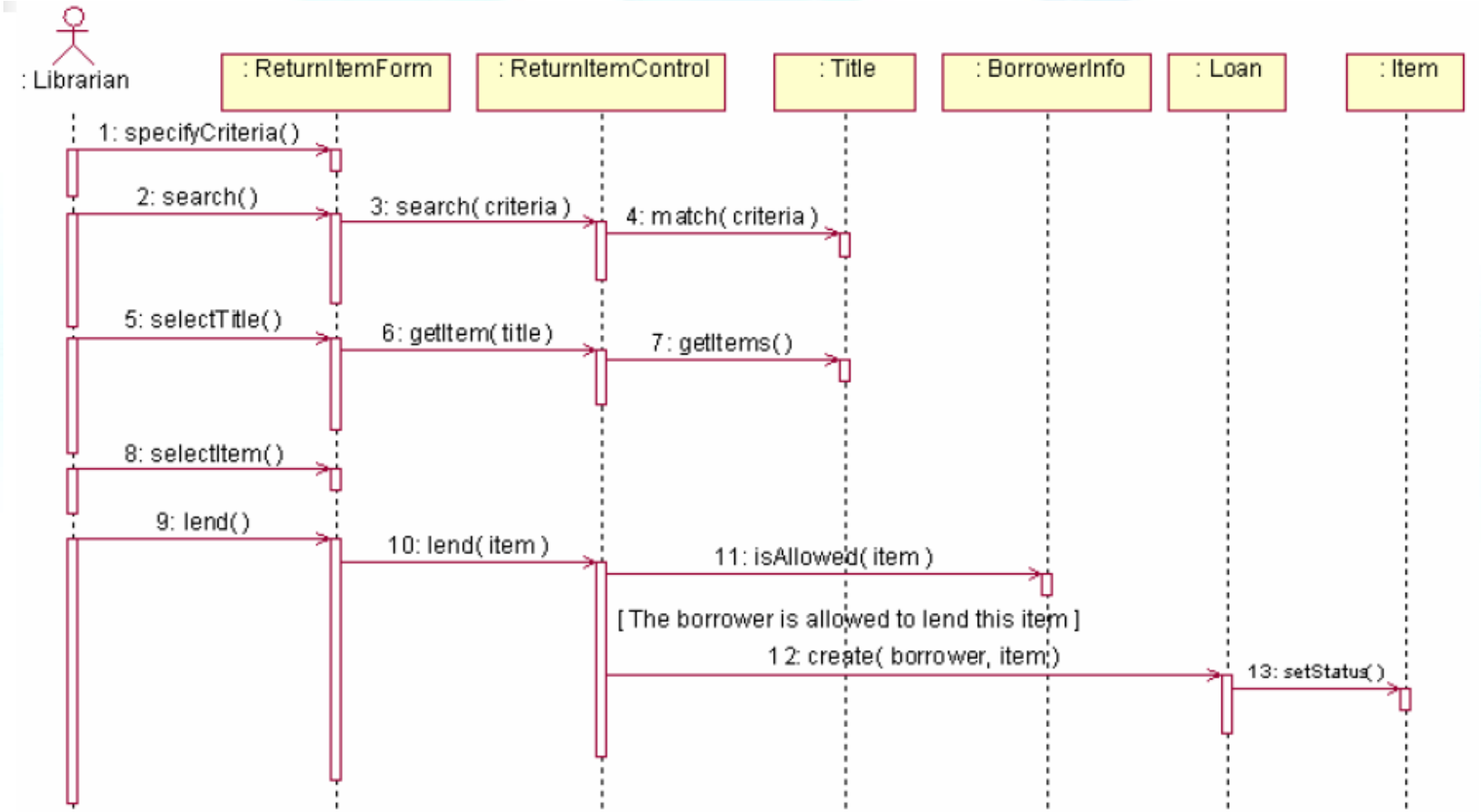


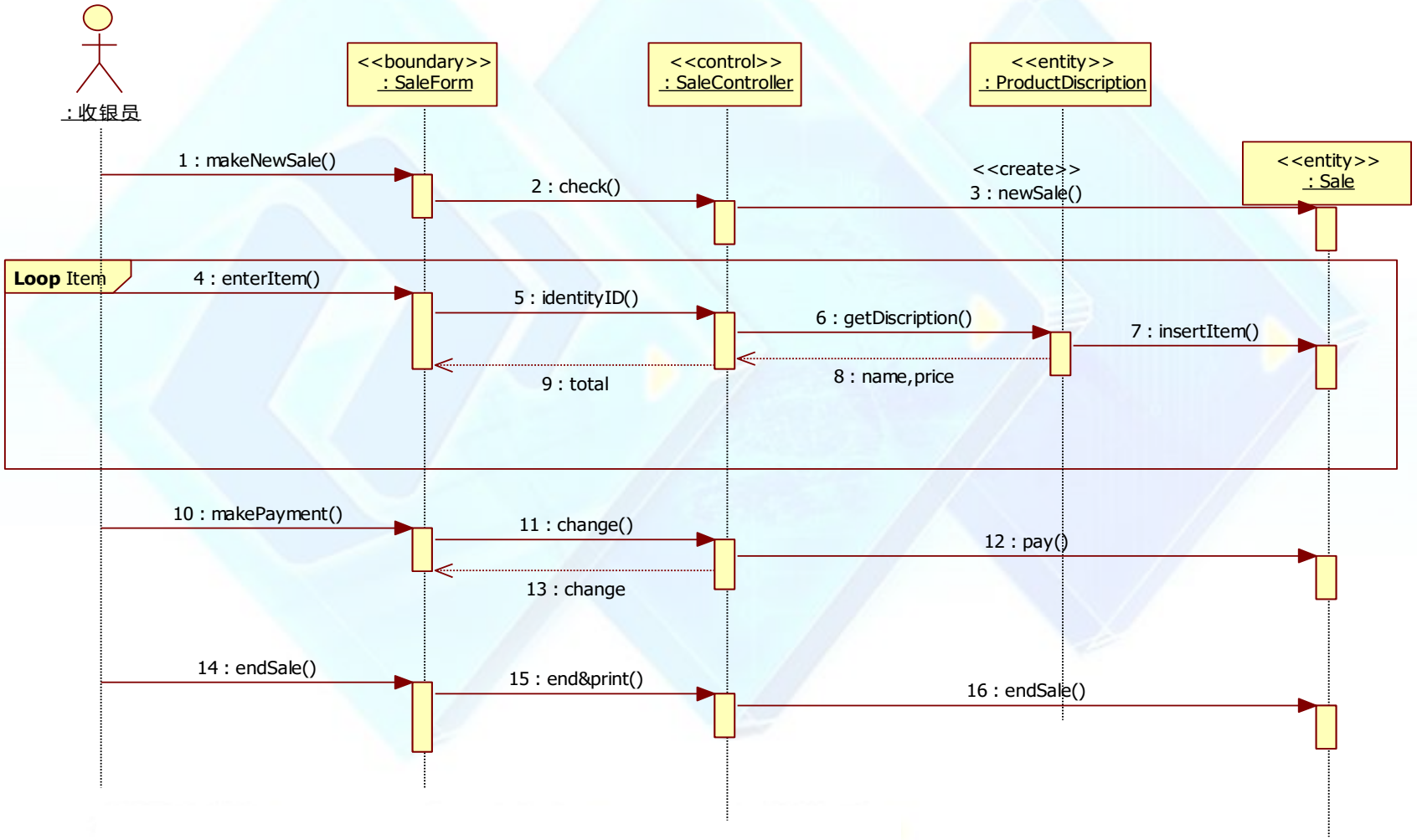
# 时序模型建模过程

- 时序图/顺序图(Sequence Diagram)：将用户与分析类结合在一起，实现将用例的行为分配到所识别的分析类中
- 绘制步骤：
  - 列出启动该用例的参与者
  - 列出启动用例时参与者使用的边界对象
  - 列出管理该用例的控制对象
  - 根据用例描述的流程，按时间顺序列出分析类之间进行消息访问的序列
- 说明：
  - 每个用例对应一张时序图
  - 时序图描述的消息序列需要与用例的事件流保持一致
  - 时序图中出现的每个箭头代表一个操作，需要包含在箭头所指向的分析类中

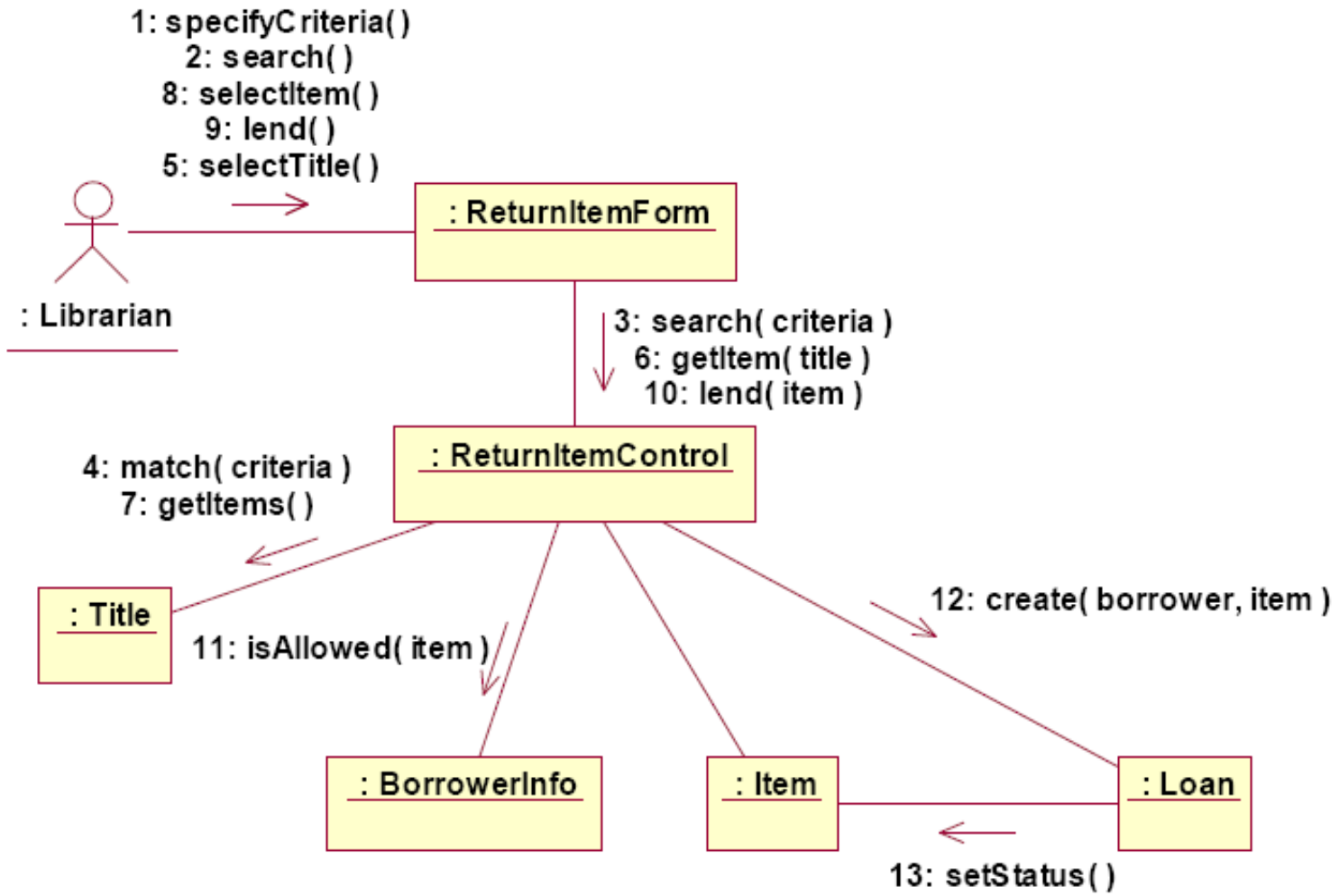
# [案例]图书管理系统的时序图



# [例]处理销售的对象时序图



# [案例]图书管理系统的协作图



# 关于时序模型与用例模型的关系

- 用例事件流中：
  - 用户请求XX
  - 系统响应XX
- 在时序图中：
  - 前者对应于actor向边界类发出的指令
  - 后者对应于边界类向控制类、实体类发出的指令与执行的反馈结果
- 用例事件流描述用自然语言，但时序图将其转化为类之间的函数调用与消息传递，是对事件流的“可实现化的描述”
- 对比：
  - 用例的事件流更粗放，是给用户看的
  - 时序模型的事件流更细致，是给程序员看的
  - 若二者完全一致了，说明你设计得不够详细！

# 本章主要内容

1. 面向对象的分析方法概述
2. 建立静态结构模型
3. 建立动态行为模型
4. 案例分析

## 案例：学生选课系统

- 教学秘书需要录入可选课程信息、任课教师信息、学分政策，并从学籍管理系统中导入学生信息
- 教师登录进入系统，查询本学期所开设课程清单，并选择自己所承担的课程
- 学生登录进入系统，查询本学期可选课程的清单，并创建自己的选课单，将某些课程加入到选课单中；学生可对选课单进行维护，包括加入其他课程、删除已选课程等
- 学生也可对选课单中包含的数据进行学分政策验证，判断所选课程是否满足学校要求
- 在规定时间之前，学生将选课单做正式提交
- 教学秘书检查每个学生的选课单，若不符合学分政策，退回重选；否则，根据所有学生提交的选课单，生成课表和每门课程的学生清单
- 教师可查看自己承担课程的课表与学生清单，学生可查询自己的课表

# OO分析的步骤

- **Step 1：角色识别**
- **Step 2：用例识别**
- **Step 3：建立用例模型**
- **Step 4：对用例图进行精化**
- **针对每个用例：**
  - **Step 5：撰写用例描述**
  - **Step 6：绘制用例的活动(泳道)图**
  - **Step 7：识别分析类(边界类、控制类、实体类)**
  - **Step 8：识别每个类的属性和方法**
  - **Step 9：建立分析类图**
  - **Step 10：建立领域类图**
  - **Step 11：建立时序图**

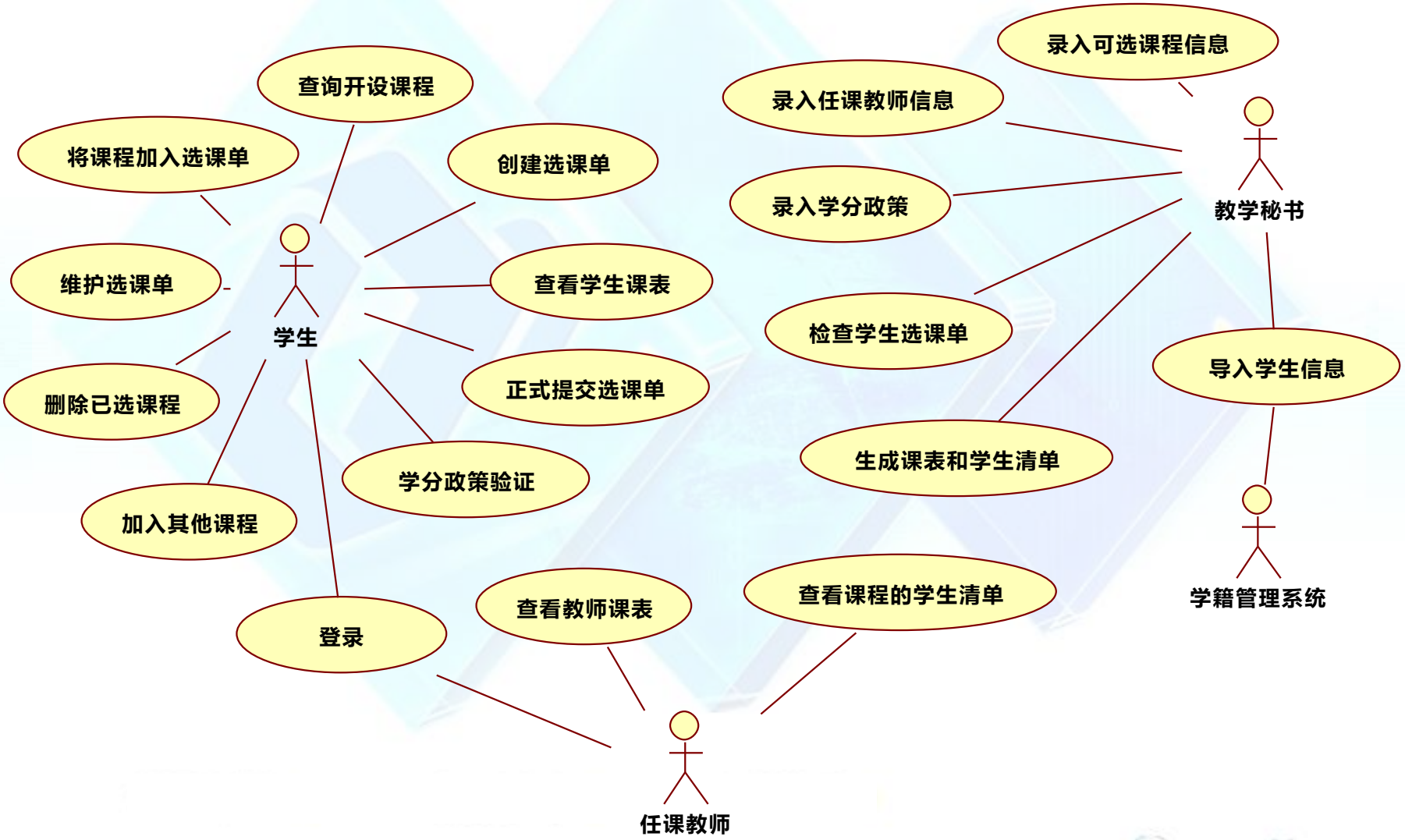
# Step1：角色识别

- 教学秘书需要录入可选课程信息、任课教师信息、学分政策，并从学籍管理系统中导入学生信息
- 教师登录进入系统，查询本学期所开设课程清单，并选择自己所承担的课程
- 学生登录进入系统，查询本学期可选课程的清单，并创建自己的选课单，将某些课程加入到选课单中；学生可对选课单进行维护，包括加入其他课程、删除已选课程等
- 学生也可对选课单中包含的数据进行学分政策验证，判断所选课程是否满足学校要求
- 在规定时间之前，学生将选课单做正式提交
- 教学秘书检查每个学生的选课单，若不符合学分政策，退回重选；否则，根据所有学生提交的选课单，生成课表和每门课程的学生清单
- 教师可查看自己承担课程的课表与学生清单，学生可查询自己的课表

# Step2：用例识别

- 教学秘书需要录入可选课程信息、任课教师信息、学分政策，并从学籍管理系统中导入学生信息
- 教师登录进入系统，查询本学期所开设课程清单，并选择自己所承担的课程
- 学生登录进入系统，查询本学期可选课程的清单，并创建自己的选课单，将某些课程加入到选课单中；学生可对选课单进行维护，包括加入其他课程、删除已选课程等
- 学生也可对选课单中包含的数据进行学分政策验证，判断所选课程是否满足学校要求
- 在规定时间之前，学生将选课单做正式提交
- 教学秘书检查每个学生的选课单，若不符合学分政策，退回重选；否则，根据所有学生提交的选课单，生成课表和每门课程的学生清单
- 教师可查看自己承担课程的课表与学生清单，学生可查询自己的课表

# Step3：建立用例模型



# Step4：对用例图进行精化

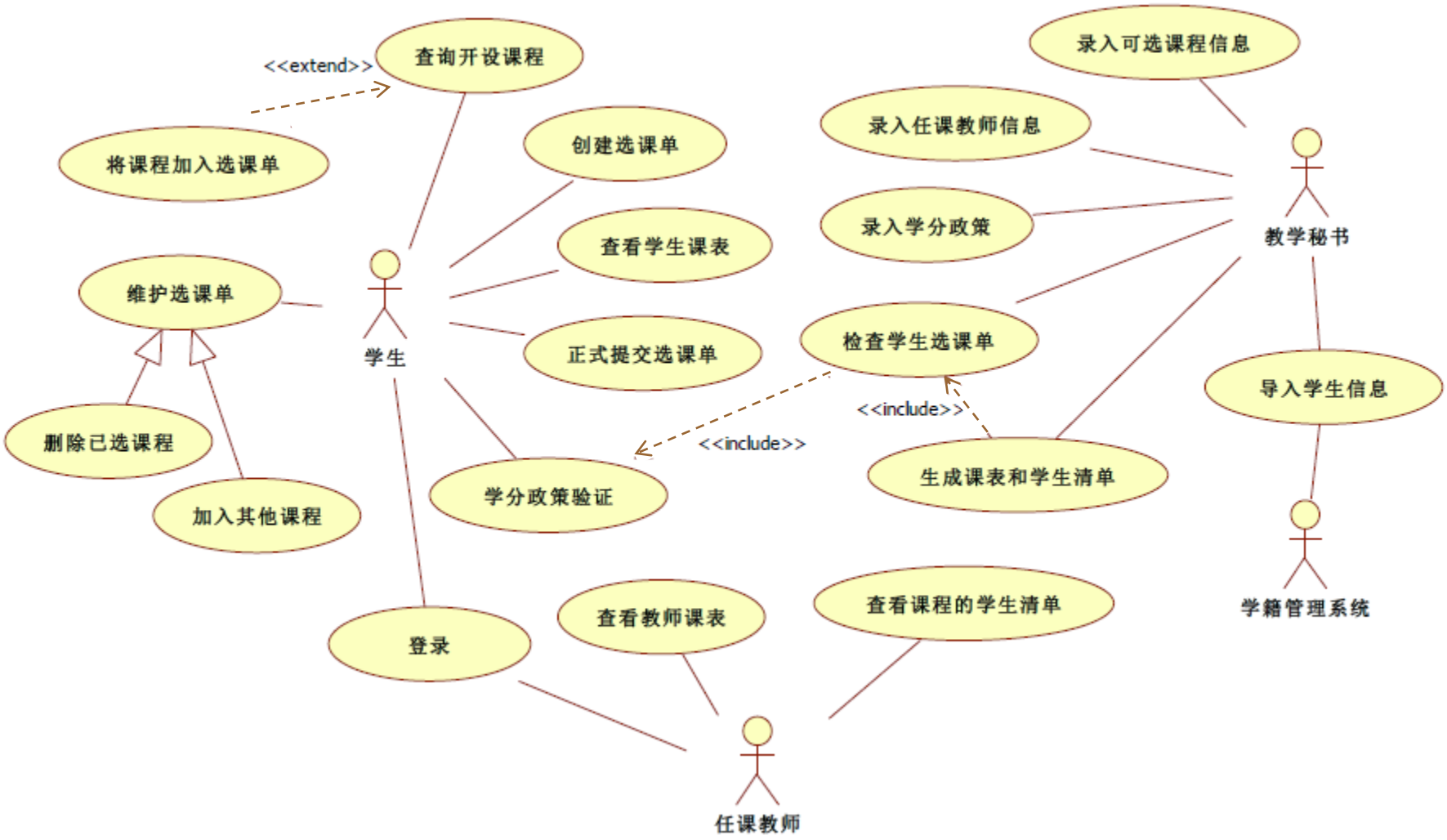
课堂讨论

- 问题1：
  - 用例“将课程加入选课单”是否是一个独立交互？
  - 它与“查询开设课程”之间的关系是什么？——include? extend?
- ——哪种更恰当？为什么？
- 问题2：“维护选课单”与“加入其他课程”、“删除已选课程”之间是什么关系？
- 问题3：“检查学生选课单”是否有必要独立存在？它与“生成课表和学生清单”是什么关系？
- 问题4：“检查学生选课单”与“学分政策验证”是什么关系？

# Step4：对用例图进行精化

- 问题1：
  - 用例“将课程加入选课单”是否是一个独立交互？—不是，需要先查询，再将课程加入选课单。
  - 它与“查询开设课程”之间的关系是什么？—include? extend?
- 如何修改？
  - 前者→<extend>→后者？
  - 后者→<include>→前者？ —哪种更恰当？为什么？
- 问题2：“维护选课单”与“加入其他课程”、“删除已选课程”之间是什么关系？ —generalization
- 问题3：“检查学生选课单”是否有必要独立存在？它与“生成课表和学生清单”是什么关系？—通常不会单独检查，而是在生成之前检查
- 问题4：“检查学生选课单”与“学分政策验证”是什么关系？—前者调用后者，include关系

# Step4：对用例图进行精化



# Step5：撰写用例描述

## 用例1：查询开设课程

**1. 目标：**

**本用例允许学生查询本学期所开设课程，进而选择课程加入到选课单中；**

**2 事件流：**

**2.1 常规流程**

(1) 学生输入课程查询条件(无条件意味着列出全部)，点击查询；

(2) 系统查询出所有可选课程，以列表形式展示；随后可执行下面的任一步骤：

(3) 学生可对课程列表按开课时间/任课教师排序，

(4)选择某一门课程查看详细信息；

(5) 学生选择某一门课程，可进入用例2“将课程加入选课单”。该步骤可重复多次；

(6) 学生选择退出。

**2.2 备选流程**

无

**3 前提条件：用例开始前，学生须在系统登录成功；**

**4 后置条件：如果用例执行成功，学生可以看到满足条件的全部可选课程。若学生选课，则所有被选课程将加入到选课单中，否则选课单保持不变。**

# Step5：撰写用例描述

## 用例2：将课程加入选课单

**1. 目标：**

本用例允许学生将一门课程加入到选课单中；

**2 事件流：**

**2.1 常规流程**

(1) 学生选择一门课程，点击“加入选课单”；

(2) 系统将该课程加入到选课单，并弹出窗口，展示目前已选课程；

**2.2 备选流程**

在(2)中，若系统发现该课程已经在选课单中，则提示“该课已选”，选课单状态不更新。

**3 前提条件：用例开始前，学生须在系统登录成功并已经查询到课程清单；**

**4 后置条件：如果用例执行成功，被选课程将加入到选课单中，否则选课单保持不变。**

# Step5：撰写用例描述

## 用例3：导入学生信息

**1. 目标：**

本用例允许教学秘书手工将已注册学生的信息从外部的学籍管理系统中导入到本系统；

**2 事件流：**

**2.1 常规流程**

(1) 教学秘书输入要导入的班级号码，点击“导入学生数据”；

(2) 系统将系统内该班级的学生信息清空，同时生成调用指令，调用学籍管理系统所提供的API接口；

(3) 学籍管理系统返回该班级中已注册的学生名单；

(4) 系统接收学生名单，将其保存起来；

(5) 系统将学生名单显示给教学秘书；

**2.2 备选流程**

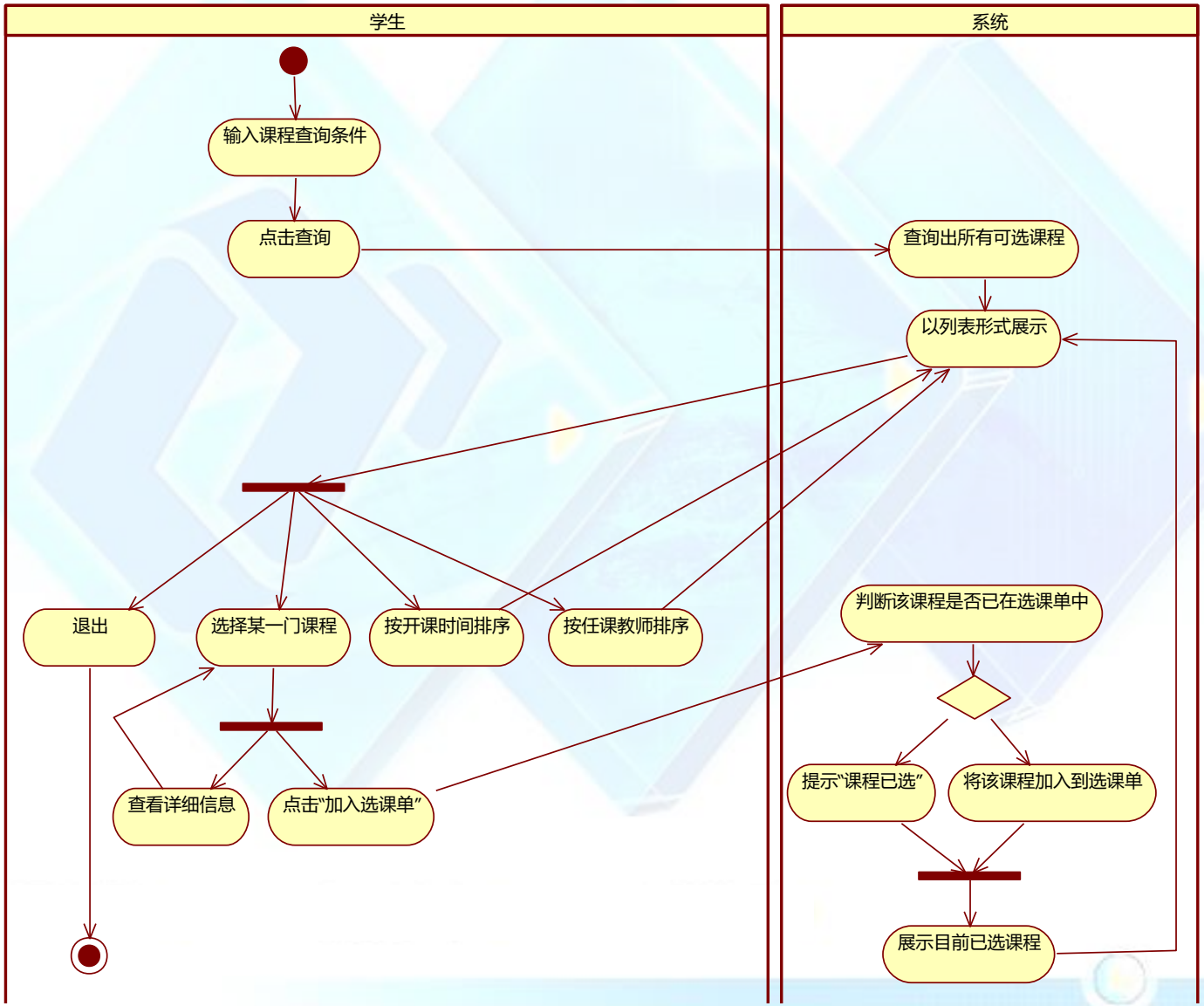
在步骤(2)中，若系统发现该班号不存在，则提示“输入班号错误”，返回重新输入。

在步骤(4)中，若系统发现学生名单为空，则提示“该班尚未有学生注册”，直接退出用例；

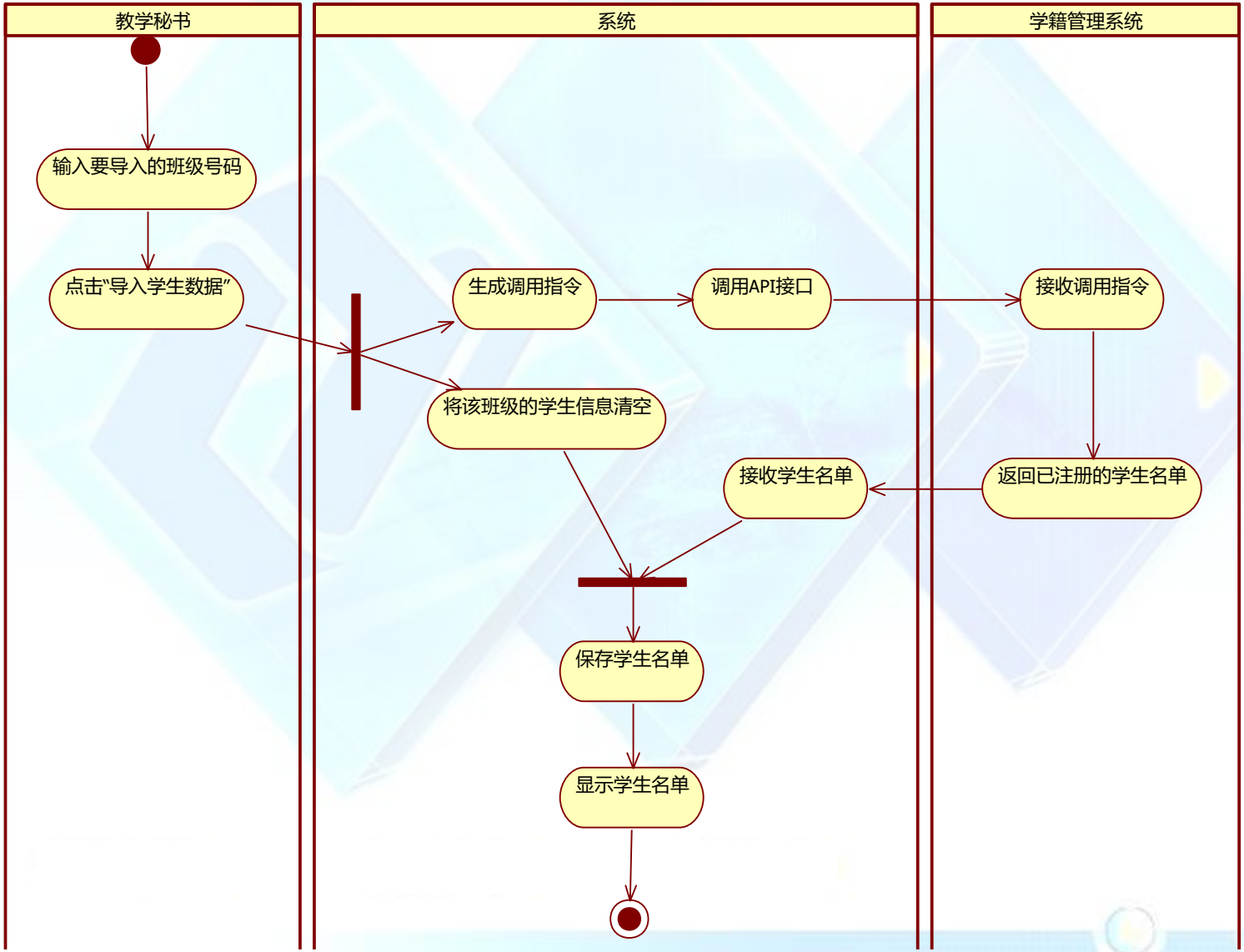
**3 前提条件：无；**

**4 后置条件：如果用例执行成功，系统将某班级的学生信息更新为学籍系统中的信息。**

# Step 6：绘制用例的活动(泳道)图

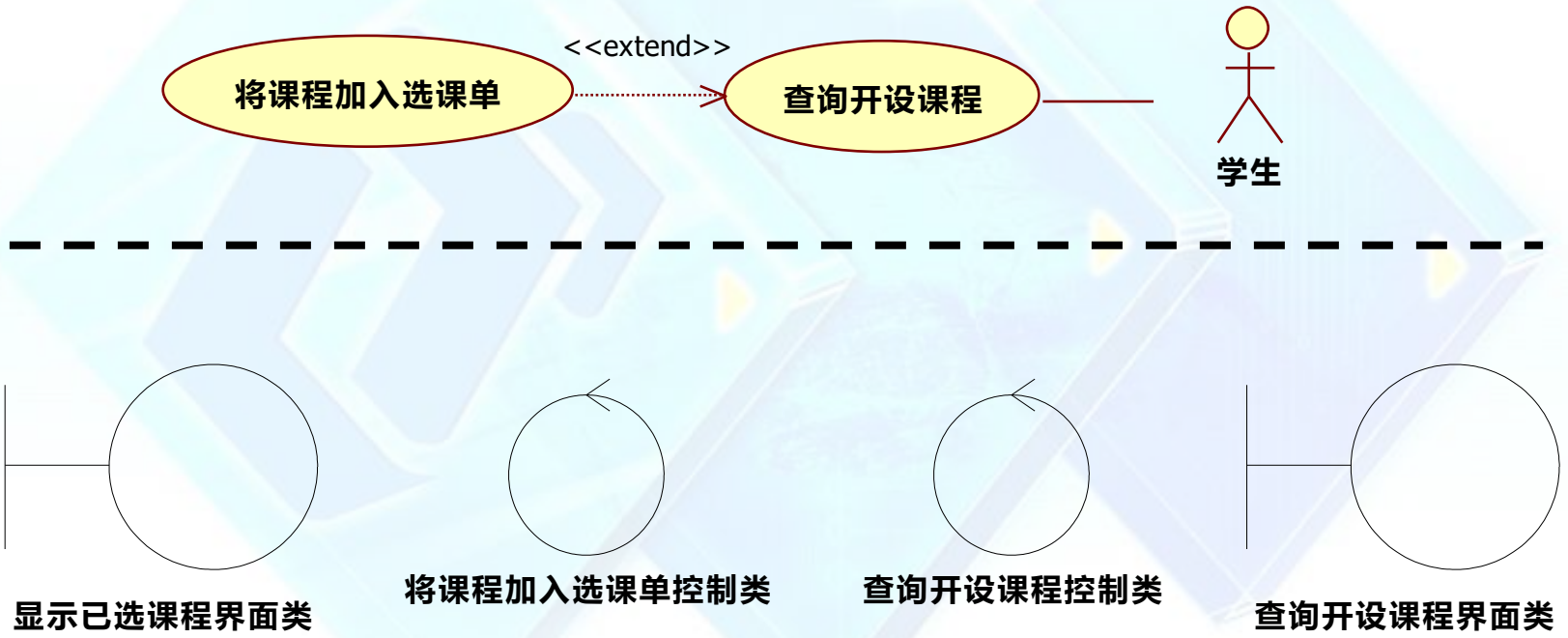


# Step 6：绘制用例的活动(泳道)图



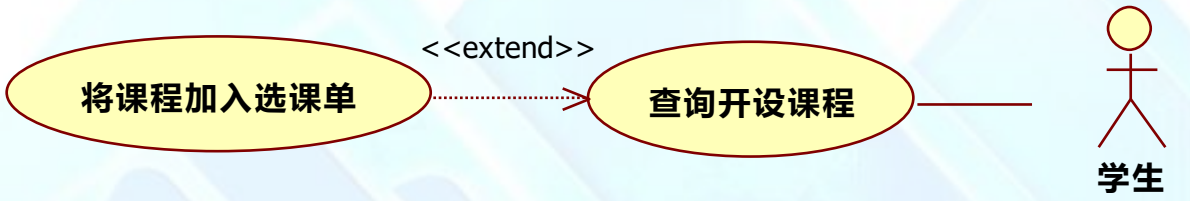
# Step 7：识别分析类(边界类、控制类、实体类)

- 针对“查询开设课程”、“将课程加入选课单”这两个用例的分析类：

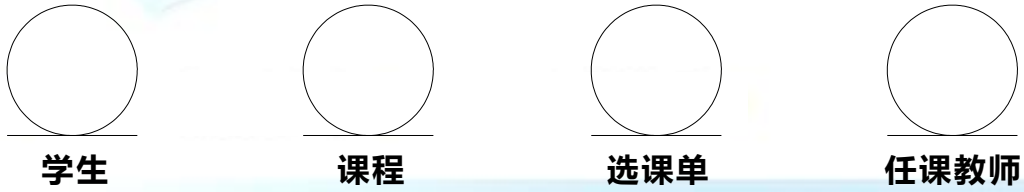


# Step 7：识别分析类(边界类、控制类、实体类)

- 针对“查询开设课程”、“将课程加入选课单”这两个用例的分析类：




(1) 学生输入课程查询条件(无条件意味着列出全部)，点击查询
(2) 系统查询出所有可选课程，以列表形式展示
(3) 学生可对课程列表按开课时间/任课教师排序，或选择某一门课程查看详细信息
(4) 学生选择某一门课程，可进入用例2“将课程加入选课单”；该步骤可重复多次
    (1) 学生选择一门课程，点击“加入选课单”
    (2) 系统将该课程加入到选课单，并弹出窗口，展示目前已选课程
(5) 学生选择退出



# Step 8：识别每个类的属性和方法

<<boundary>>
显示已选课程UI类

-已选课程清单

+刷新已选课程清单( )
+关闭( )

<<control>>
查询开设课程控制类

+查询开设课程( )
+查看课程详细信息( )
+将课程加入选课单( )

<<control>>
将课程加入选课单控制类

+判断课程是否已在清单中( )
+将课程加入选课单( )

<<boundary>>
查询开设课程UI类

-查询条件1（课程名）
-查询条件2（教师名）
-查询到的课程清单
-当前所选课程

+查询开设课程( )
+显示课程清单( )
+按开课时间排序( )
+按任课教师排序( )
+从列表中选择课程( )
+查看课程详细信息( )
+将课程加入选课单( )
+退出( )

<<entity>>
任课教师

-工号
-姓名

<<entity>>
学生

-学号
-姓名

<<entity>>
课程

-课程编号
-开课时间
-周数
-每周课时数
-学分
-任课教师工号

+查看课程详细信息( )

<<entity>>
选课单

-学号
-选课项集合
-提交状态

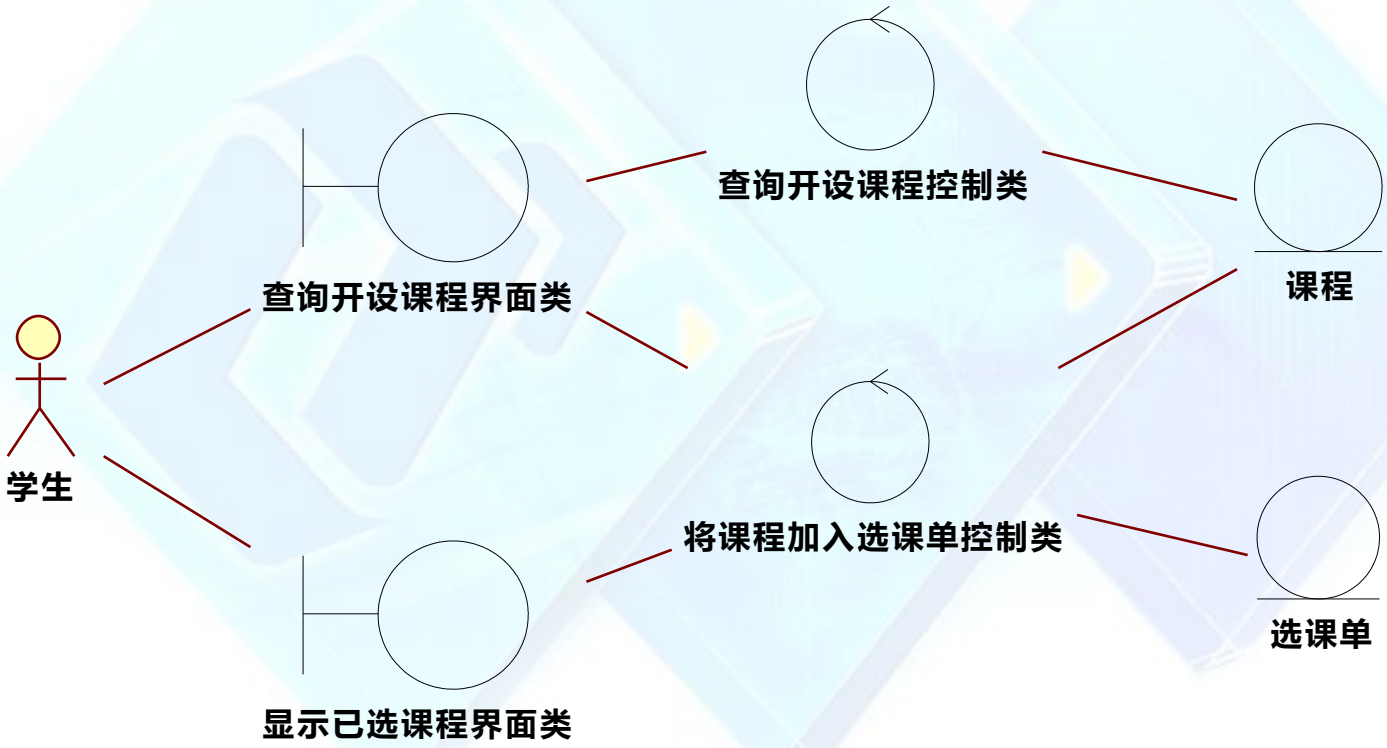
+New( )
+判断某门课程已选( )
+将课程加入选课单( )
+查询已选课程清单( )

<<entity>>
选课项

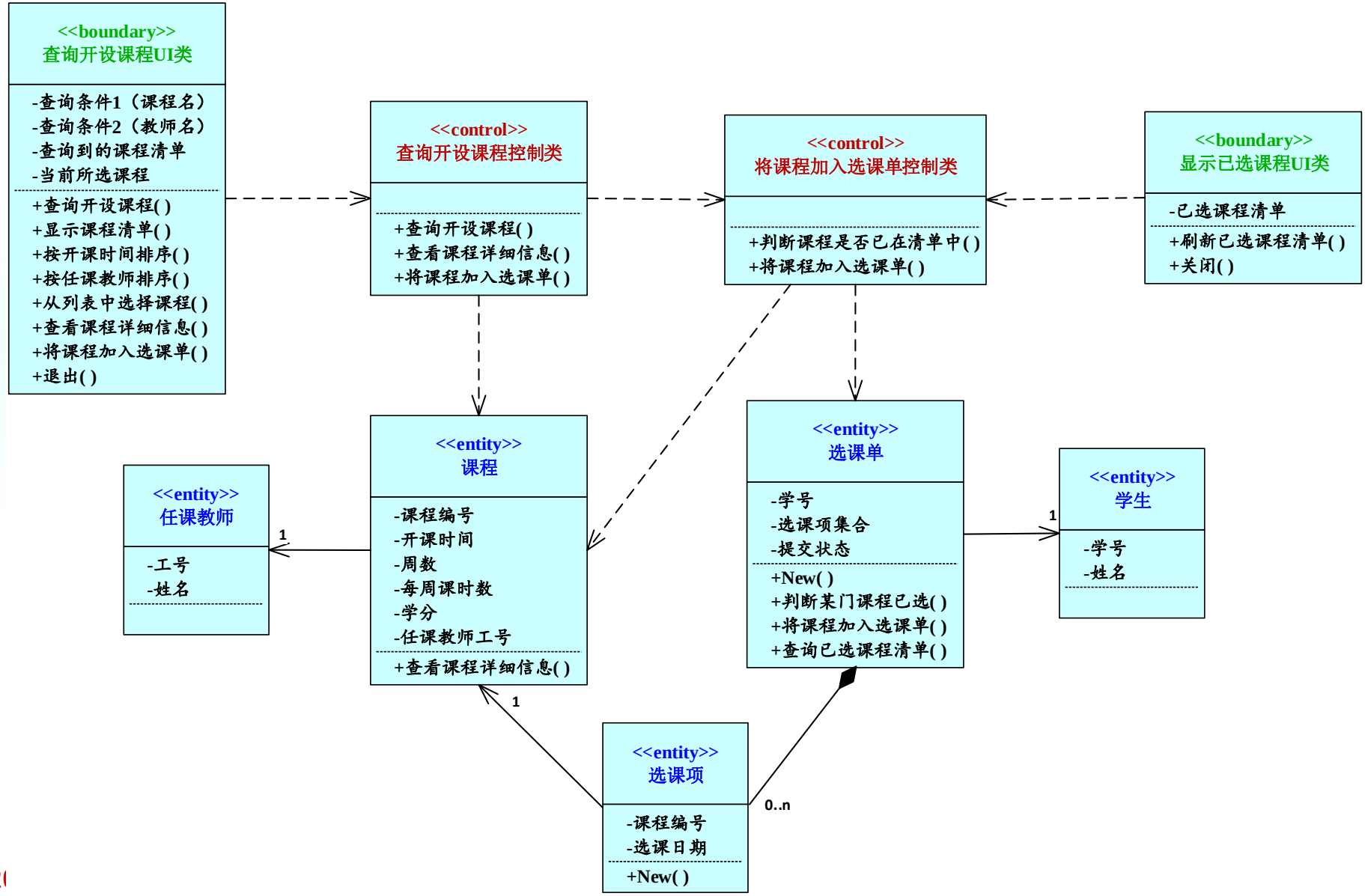
-课程编号
-选课日期

+New( )

# Step 9：绘制分析类图

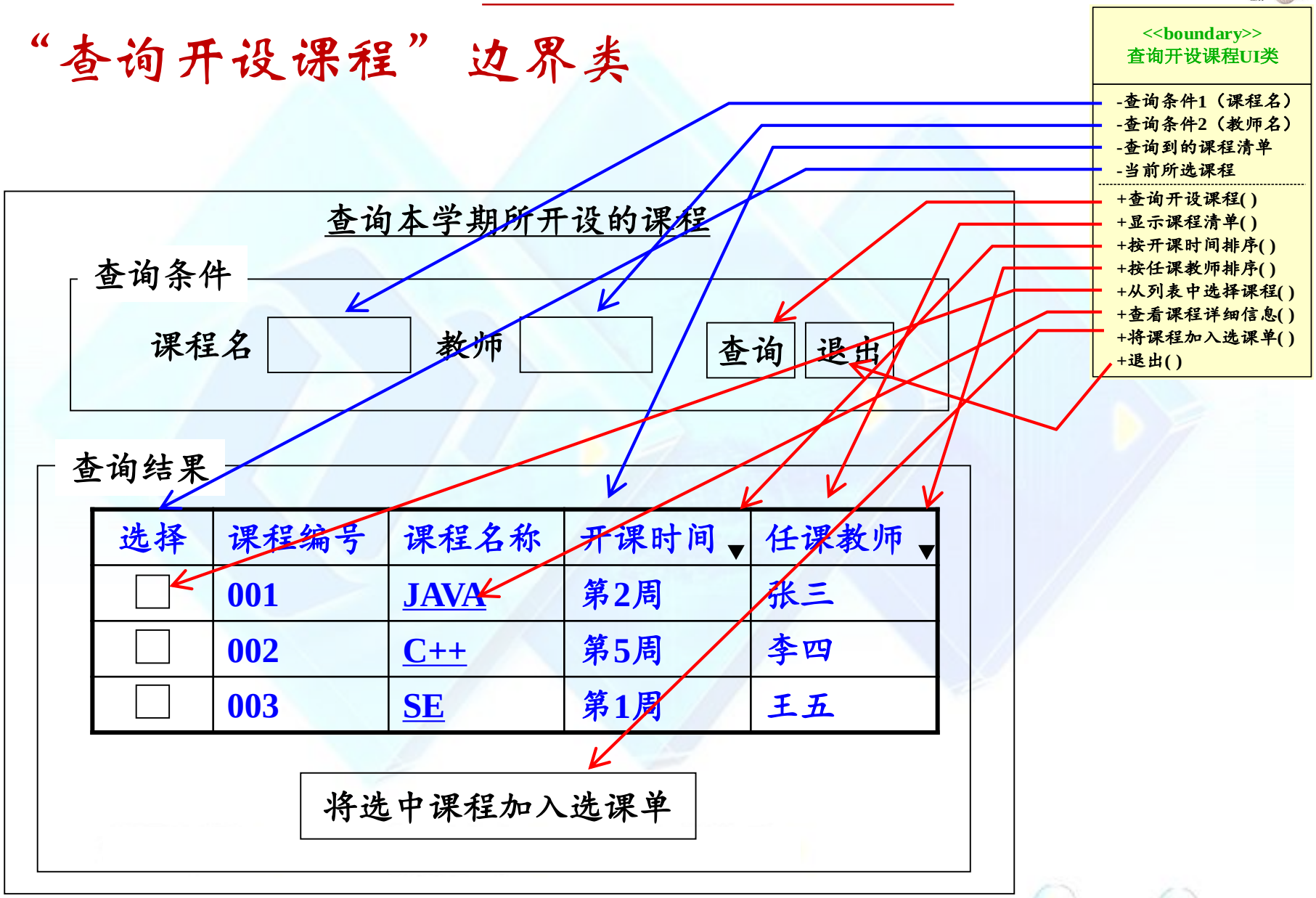


# Step 10：绘制领域类图

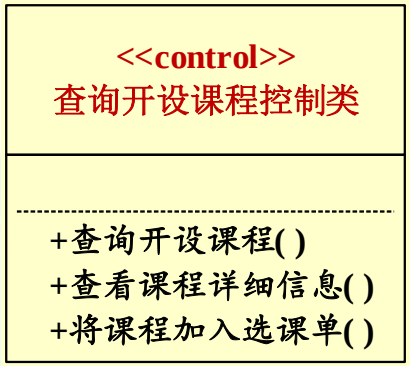


# 面向对象系统分析

## “查询开设课程”边界类



# “查询开设课程”控制类



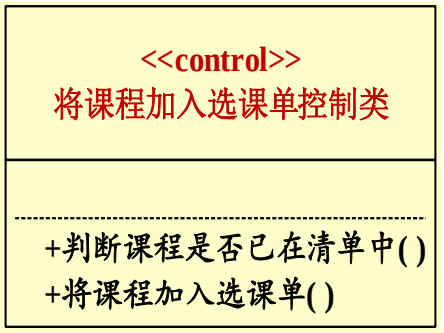

- 查询开设课程()

```
List QueryCourses(String TName, String CName) {
    连接数据库，从数据库中读取所有满足条件的课程;
    构造List，返回UI;
}
```

- 将课程加入选课单()

```
void AddCourseToForm(Course c, Student s) {
    构造“将课程加入选课单”控制类addCourseCtrl;
    addCourseCtrl. AddCourseToForm (s, c);
}
```

# “将课程加入选课单”控制类




- 将课程加入选课单()

**Void AddCourseToForm (Student s, Course c) {**

**从数据库中查询学生s的选课单信息；**

**根据这些信息，调用选课单类的new()操作，构造选课单对象form;**

**调用form的“将课程加入选课单”操作，输入参数为c;**

**}**

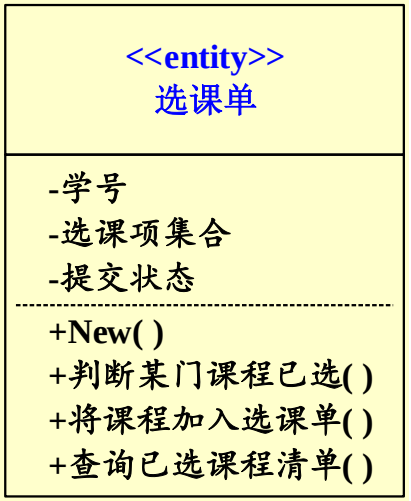
# “选课单”实体类

- 判断某门课程已选()

```
Boolean isCourseSelected (Course course) {
    循环从选课项集合中逐一取出每一项item {
     if (item.课程 == course)
         return true;
     }
     return false;
}
```

- 将课程加入选课单()

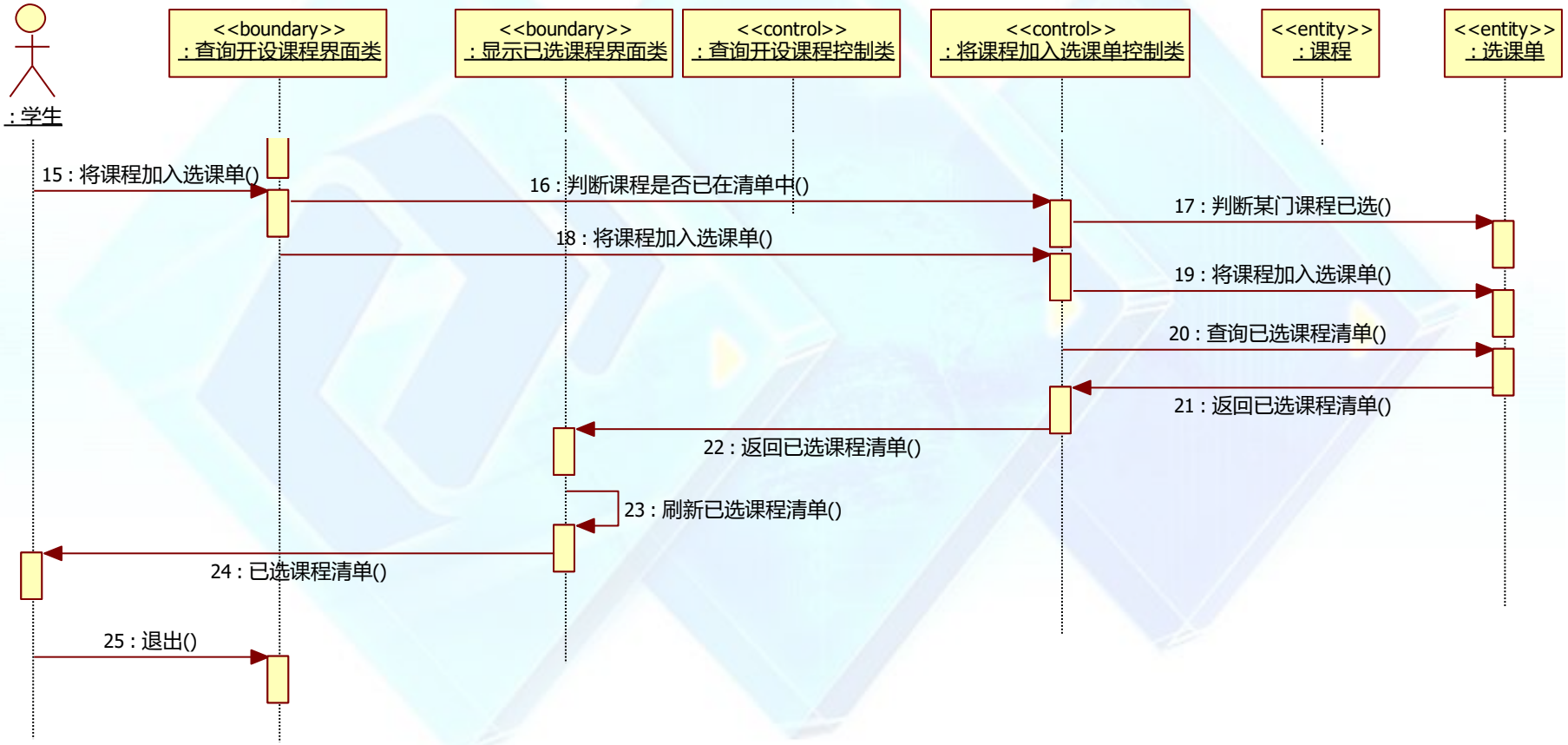
```
Void AddNewCourse(Course course) {
      Item item = new Item(course, 当前日期);
      将item加入到self.选课项集合中;
}
```



# Step 11：绘制时序图

查询开课课程用例行为分析

# Step 11：绘制时序图



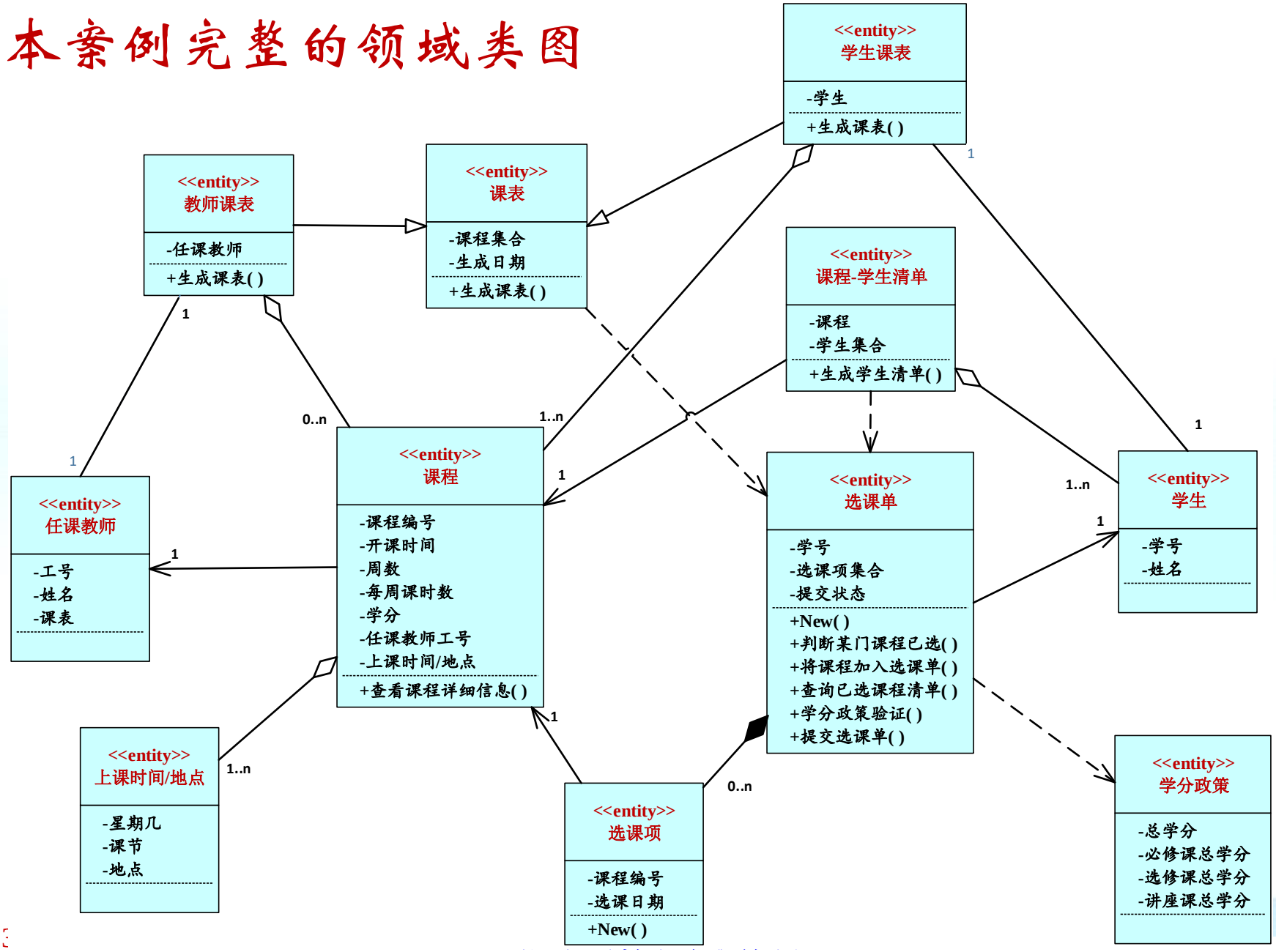

将课程加入选课单用例行为分析

## 完整的实体类识别

- 教学秘书需要录入可选课程信息、任课教师信息、学分政策，并从学籍管理系统中导入学生信息
- 教师登录进入系统，查询本学期所开设课程清单，并选择自己所承担的课程
- 学生登录进入系统，查询本学期可选课程的清单，并创建自己的选课单，将某些课程加入到选课单中；学生可对选课单进行维护，包括加入其他课程、删除已选课程等
- 学生也可对选课单中包含的数据进行学分政策验证，判断所选课程是否满足学校要求
- 在规定时间之前，学生将选课单做正式提交
- 教学秘书检查每个学生的选课单，若不符合学分政策，退回重选。否则，根据所有学生提交的选课单，生成课表和每门课程的学生清单
- 教师可查看自己承担课程的课表与学生清单，学生可查询自己的课表

# 面向对象系统分析

## 本案例完整的领域类图



## 还进一步需要做的事情…

- **Step 12：按照上述过程，完成该案例其他用例的OO分析**
- **Step 13：对分析类图和领域类图做进一步精化，设定每个属性的详细信息，尝试着书写每个操作的伪代码**

# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

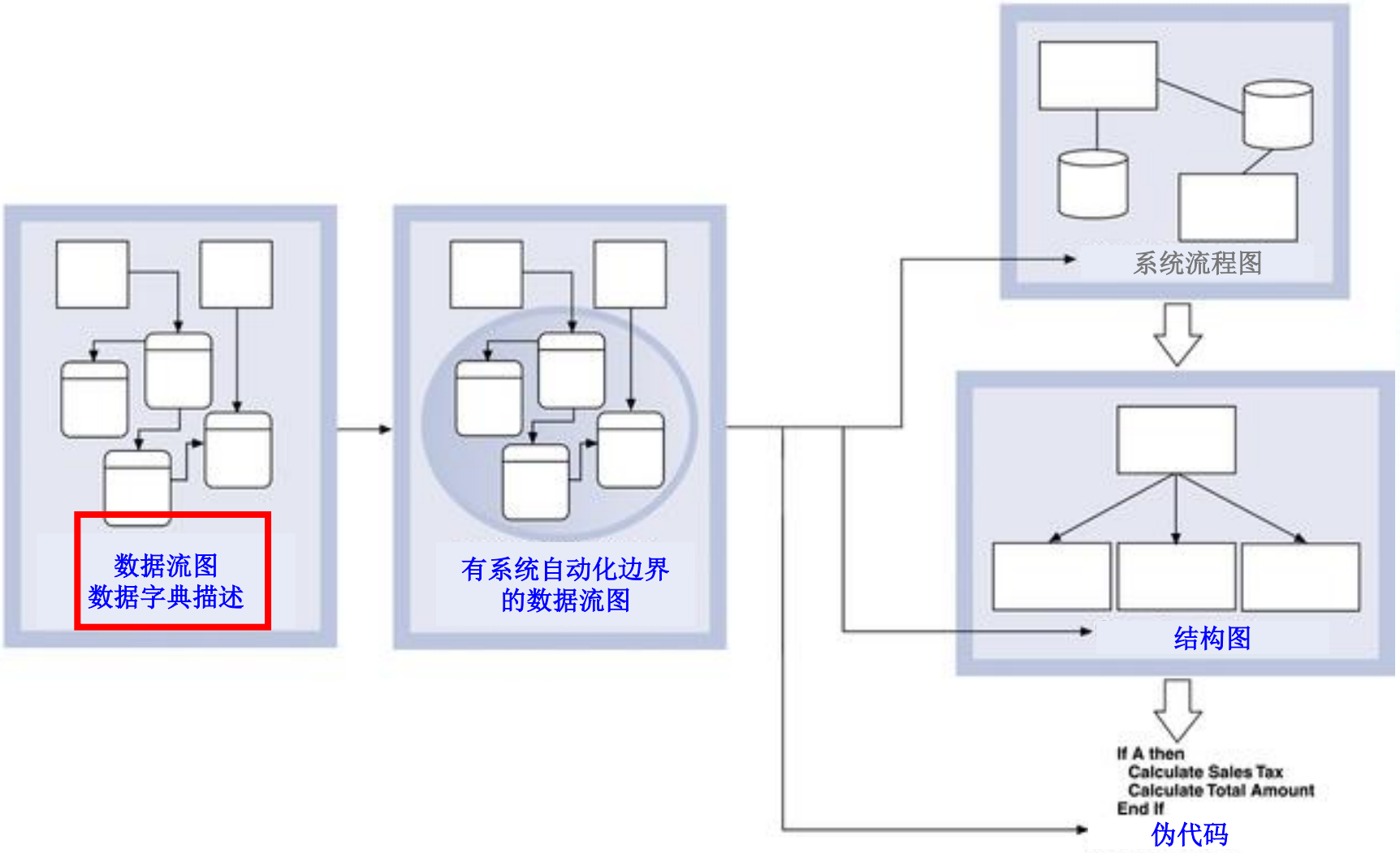
2023. 10

# 结构化设计的内容

- 结构化的系统设计主要考虑：
  - 模块的层次化
  - 模块之间的接口
  - 数据库设计（后续章节讲授）
  - 用户界面设计（后续章节讲授）
- 为每个模块设计内部逻辑
- 采用自顶向下的方法进行设计
  - 具有系统自动化边界的DFD
  - 结构图

# 结构化系统设计

## 结构化设计模型



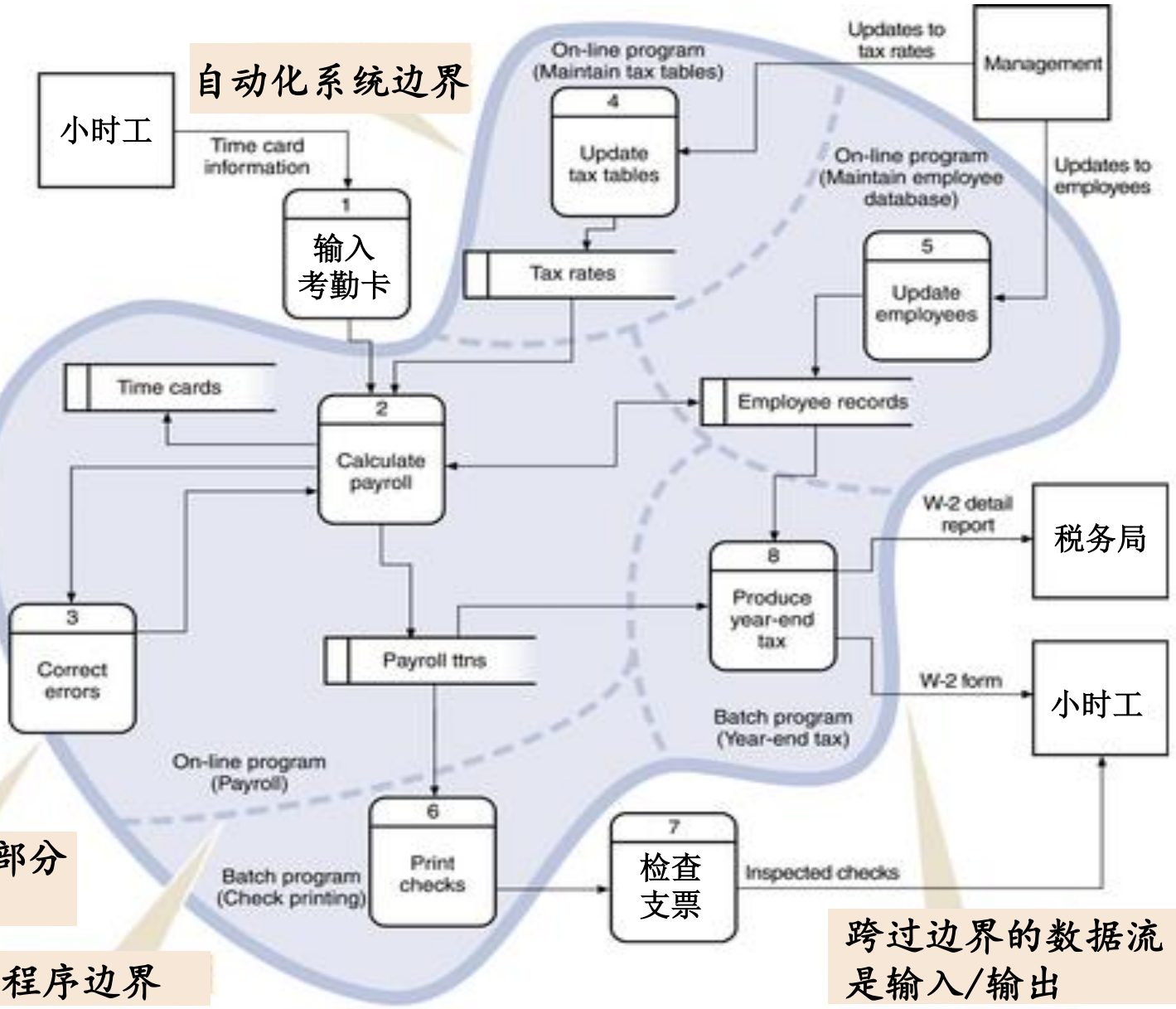
# 结构化系统设计

## 自动化系统边界划分（Automation System Boundary）

- 将数据流图划分为手工处理部分和系统能自动完成的部分
- 程序的处理过程可以在系统边界内部或外部
- 数据流可以在系统边界内部或外部
- 穿过系统界线的数据流代表了系统的输入和输出
- 在最终的系统中，数据流将成为用户界面中的表单、报表、供其他系统使用的数据文件等

# 结构化系统设计

自




部分输入、部分输出的过程

程序边界

跨过边界的数据流是输入/输出

带自动化系统边界的DFD

# 结构图（structure chart）

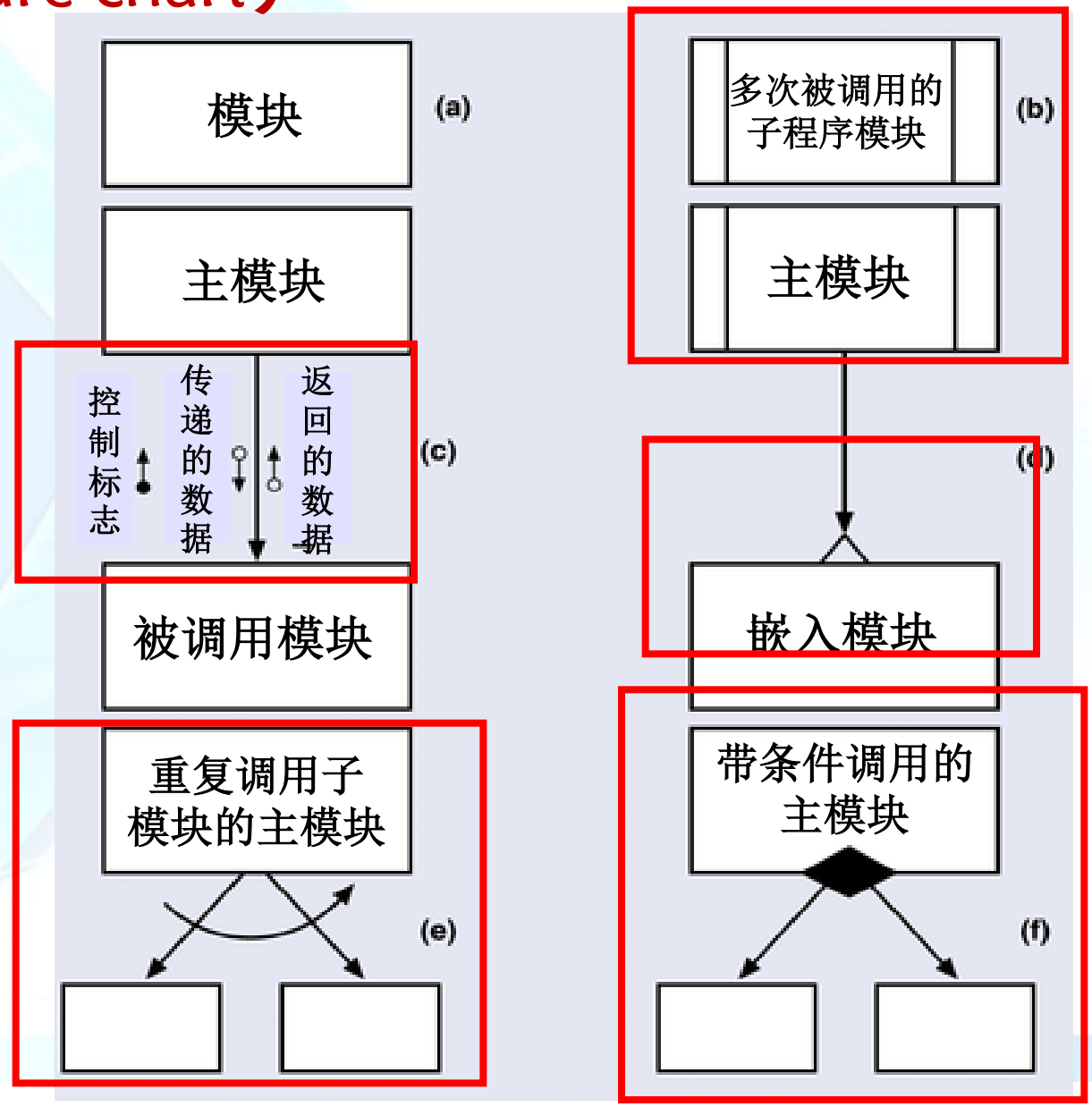
结构图定义：以模块为基础、以模块间的调用为关联所构成的图称模块结构图，简称结构图

结构图的作用：

- 通过层次的方法来描述系统每部分的功能和子功能
- 展示计算机程序模块间的联系

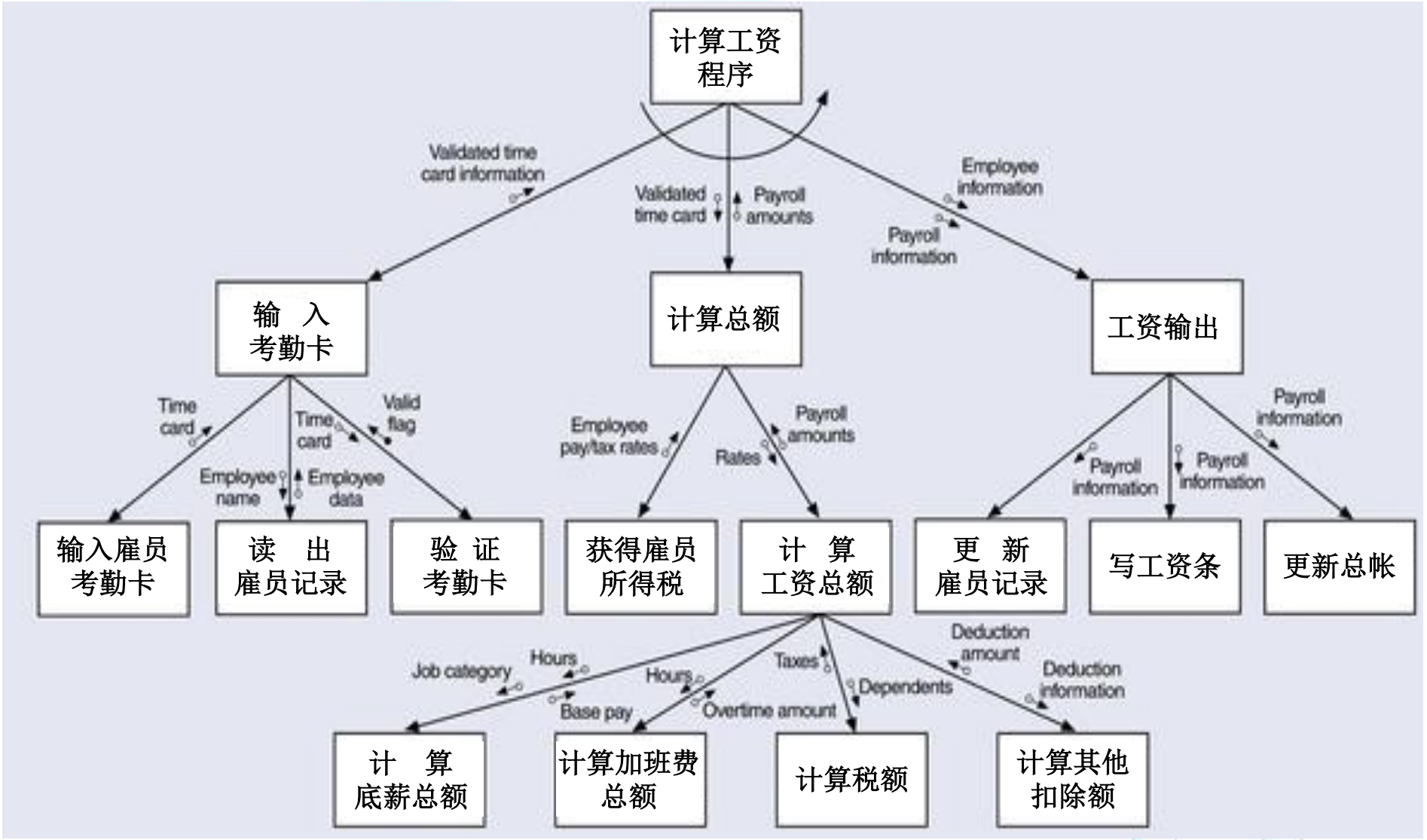
# 结构图（structure chart）

## 结构图符号



# 结构化系统设计

## 结构图（structure chart）




结构图示例：工资计算系统的结构图

# 结构图的创建方法

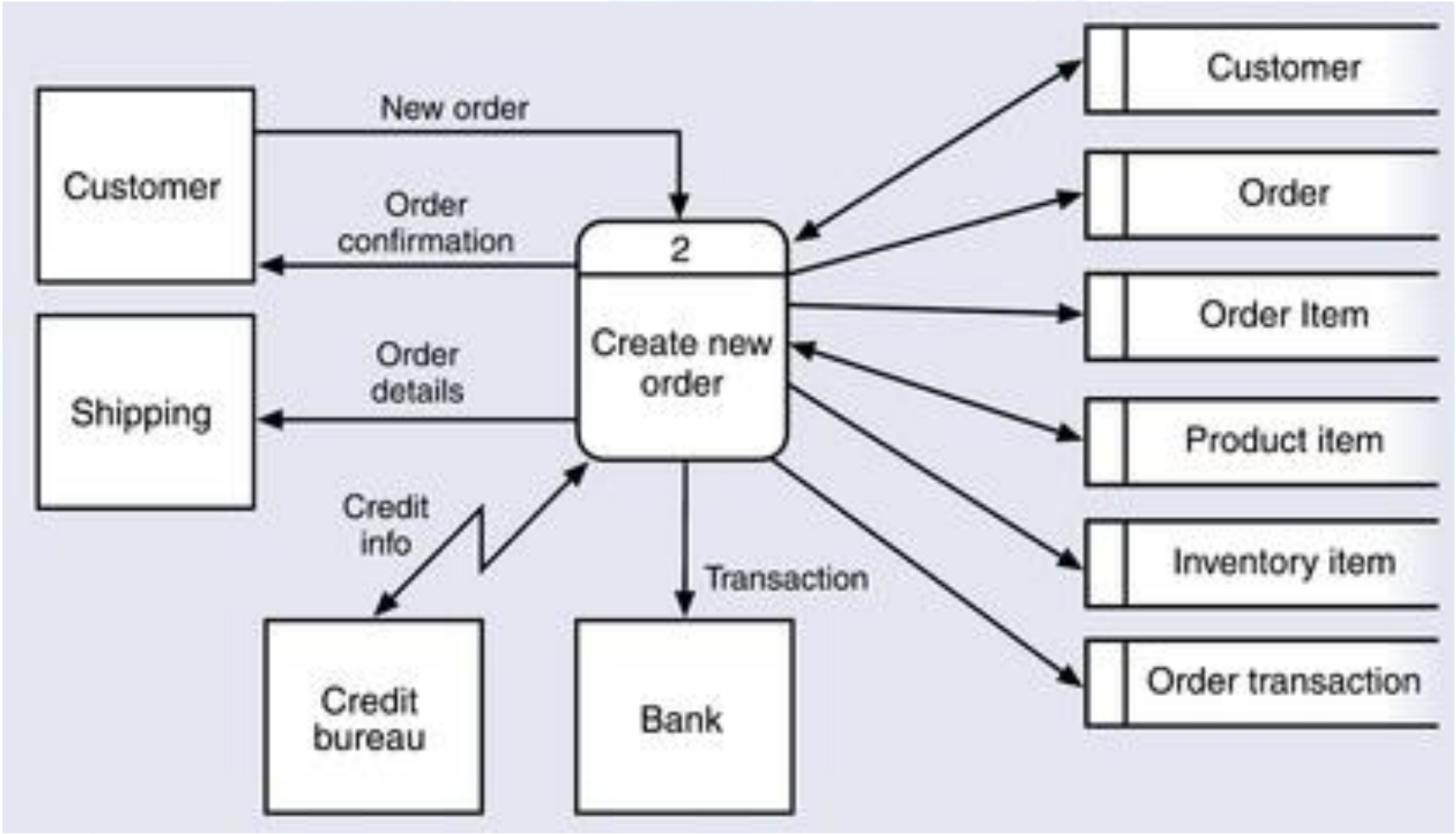
## 从一个DFD片断建立一个结构图的步骤

- 确定主要的信息流
- 找出能代表输入和输出间最基本变化的过程
- 重画数据流图并把输入放在左边，输出放在右边
- 初步建立一个结构图草图
- 加入其它模块实现下列功能
  - 数据输入、数据处理、数据输出
- 使用结构化语言或决策树添加模块间逻辑
- 进一步求精

# 结构图的创建方法

- 转换分析方法
  - “处理”对输入到输出的转换（I-P-O）
  - 结构图包括输入子树、计算子树和输出子树
  - 用数据流图片断作为输入
- 事务分析方法
  - 含有“处理”分支的情况
  - 某“处理”对输入数据流进行分析，根据分析结果选择不同的“处理”

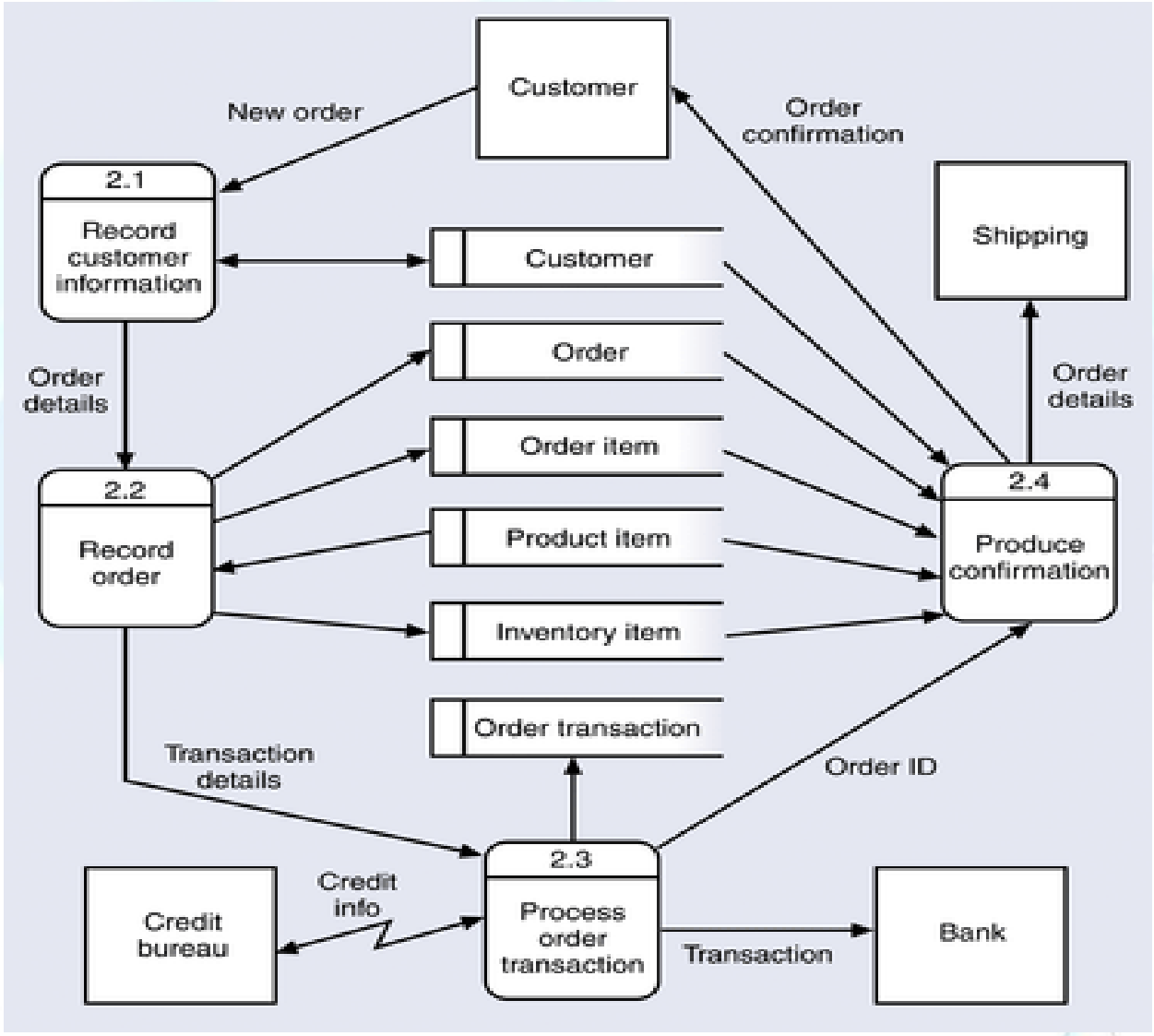
# 结构图的创建方法




“创建新订单”DFD片断

# 结构化系统设计

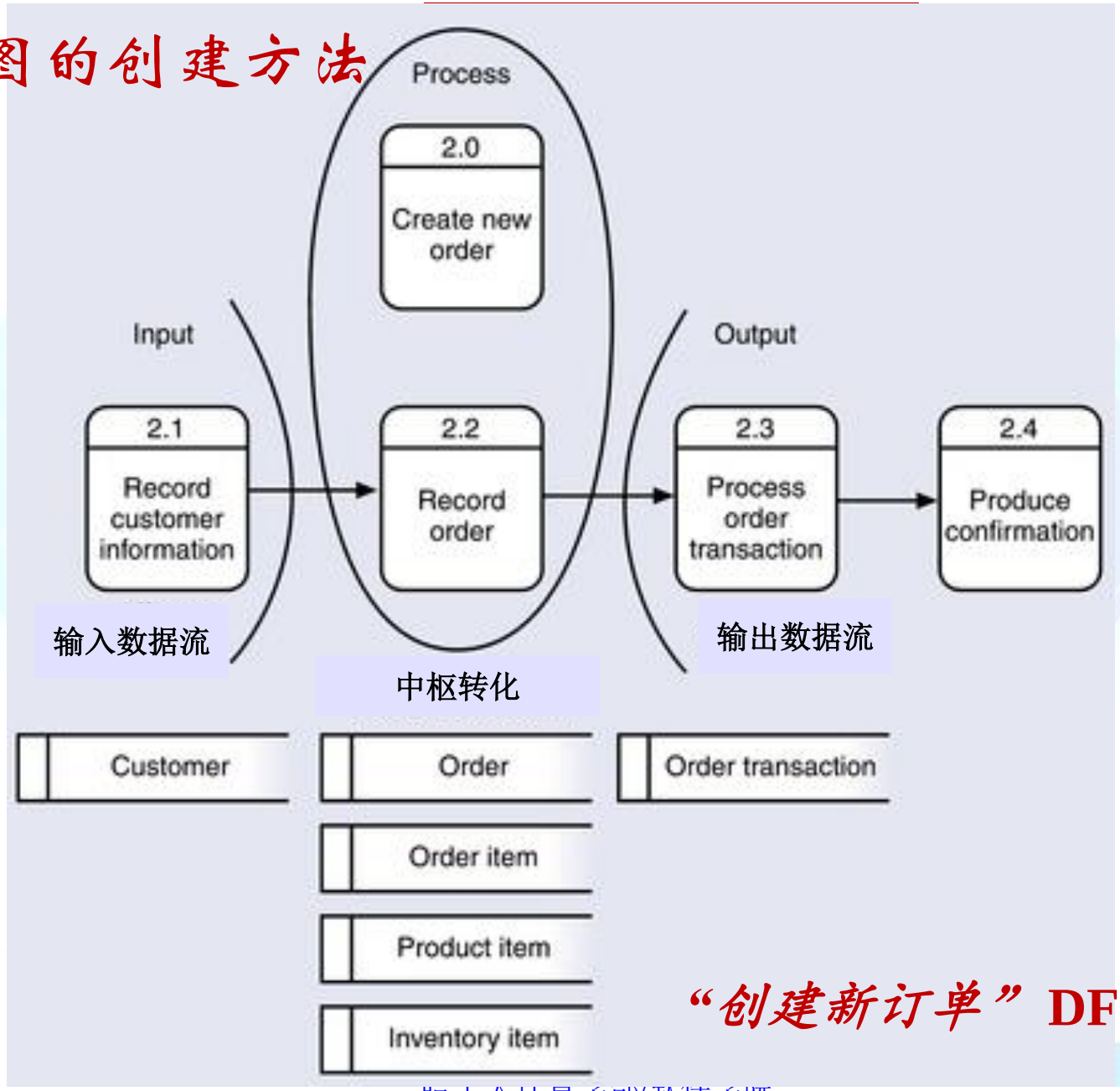
## 结构图的创建方法




“创建新订单” DFD的分解图

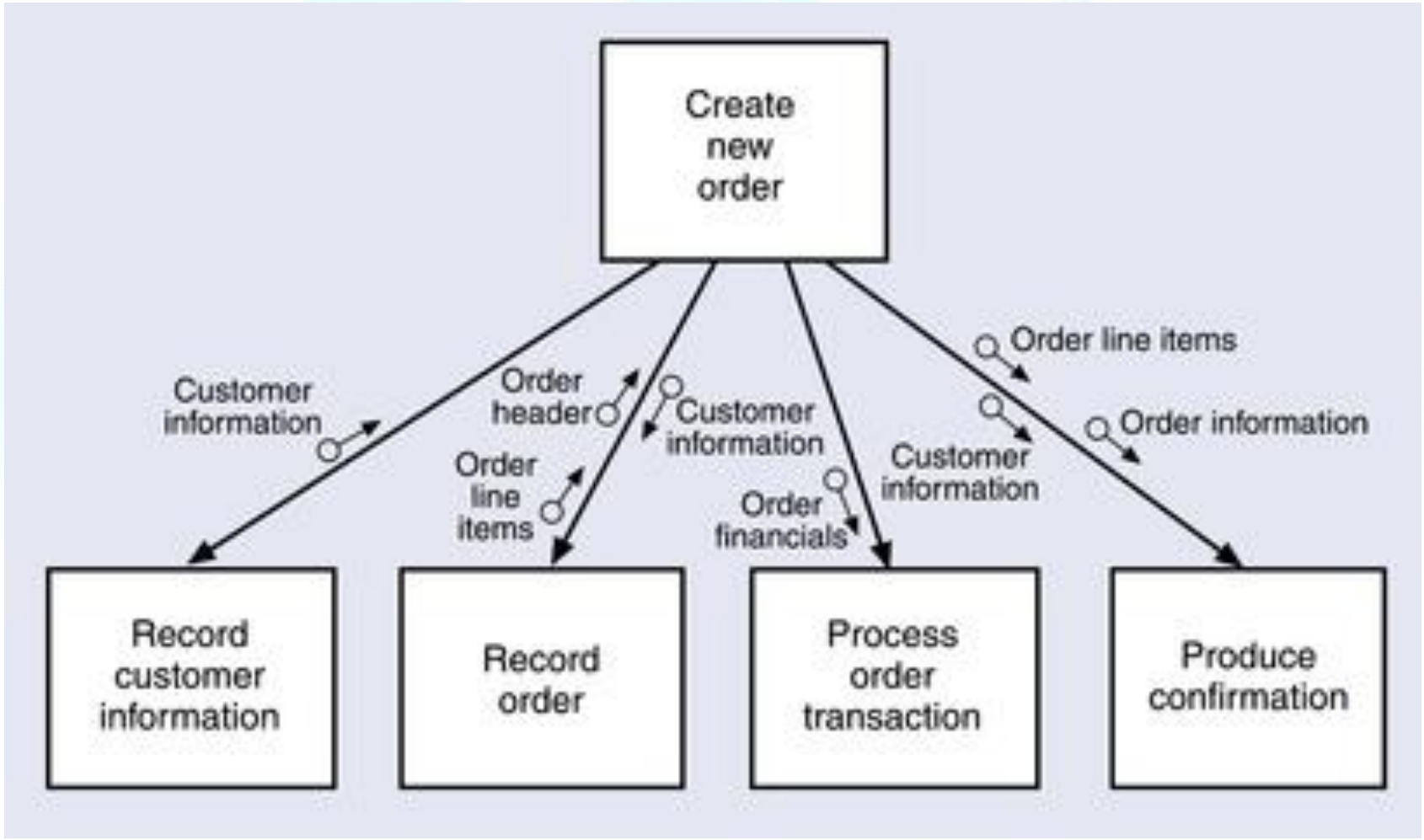
# 结构化系统设计

## 结构图的创建方法




“创建新订单”DFD的重组

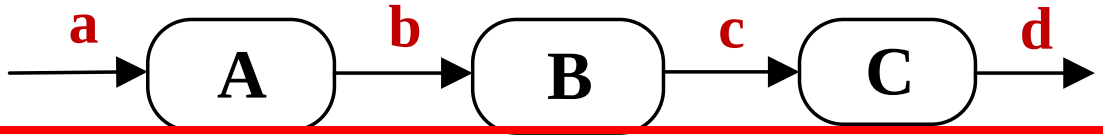
# 结构图的创建方法



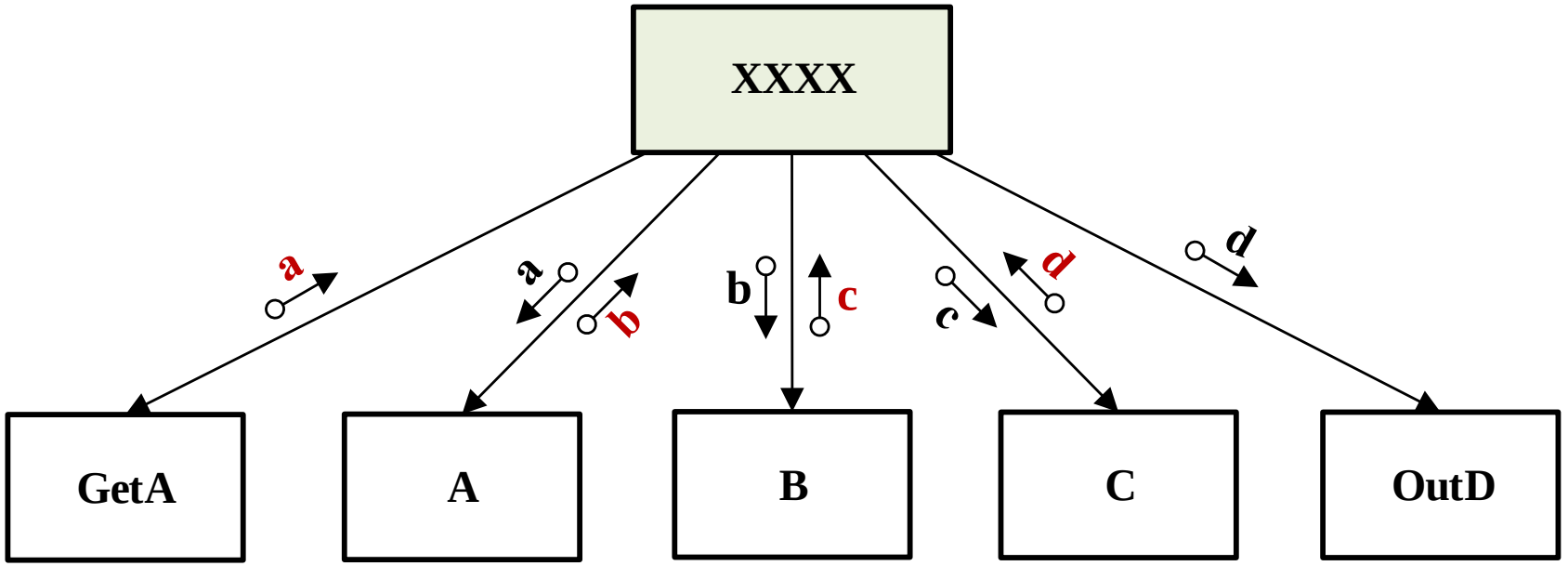

“创建新订单”模块的结构图

# DFD到系统结构图转换的基本模式

## 1.简单变换型DFD→系统结构图转换




第1种情况：不考虑扇入扇出度的问题，得到系统结构图

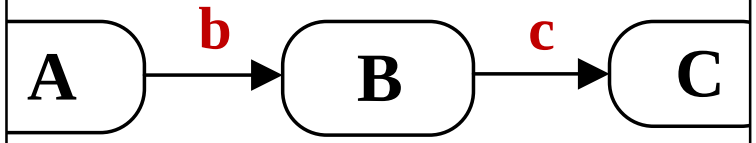


# 结构化系统设计

## DFD到系统结构图转换的基本模式

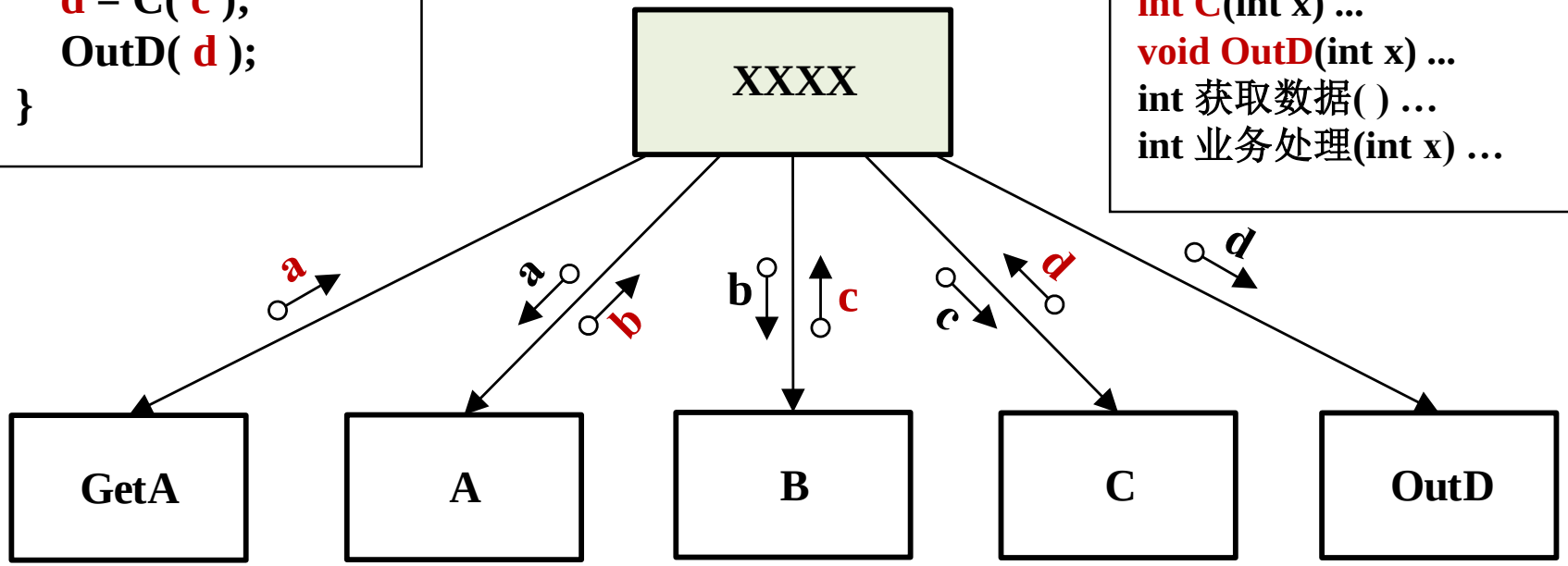
### 1.简单变换型DFD→系统结构图转换

```
/*系统控制主结构*/
main()
 { int a, b, c, d;
    a = GetA( );
    b = A( a );
    c = B( b );
    d = C( c );
    OutD( d );
 }
```



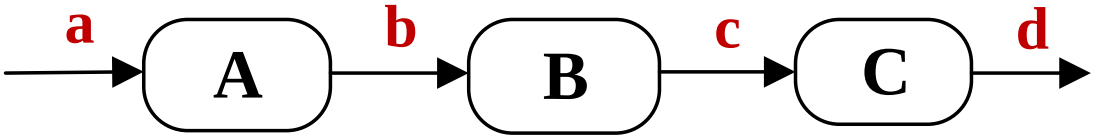

考虑扇入扇出度的问题，

```
/*其他函数定义*/
int GetA( );
 {   int x;
     x = 获取数据( );
     return x;
 }
int A(int x);
 {  inty;
     y = 业务处理(x);
    return y;
 }
int B(int x) ...
int C(int x) ...
void OutD(int x) ...
int 获取数据( ) …
int 业务处理(int x) …
```

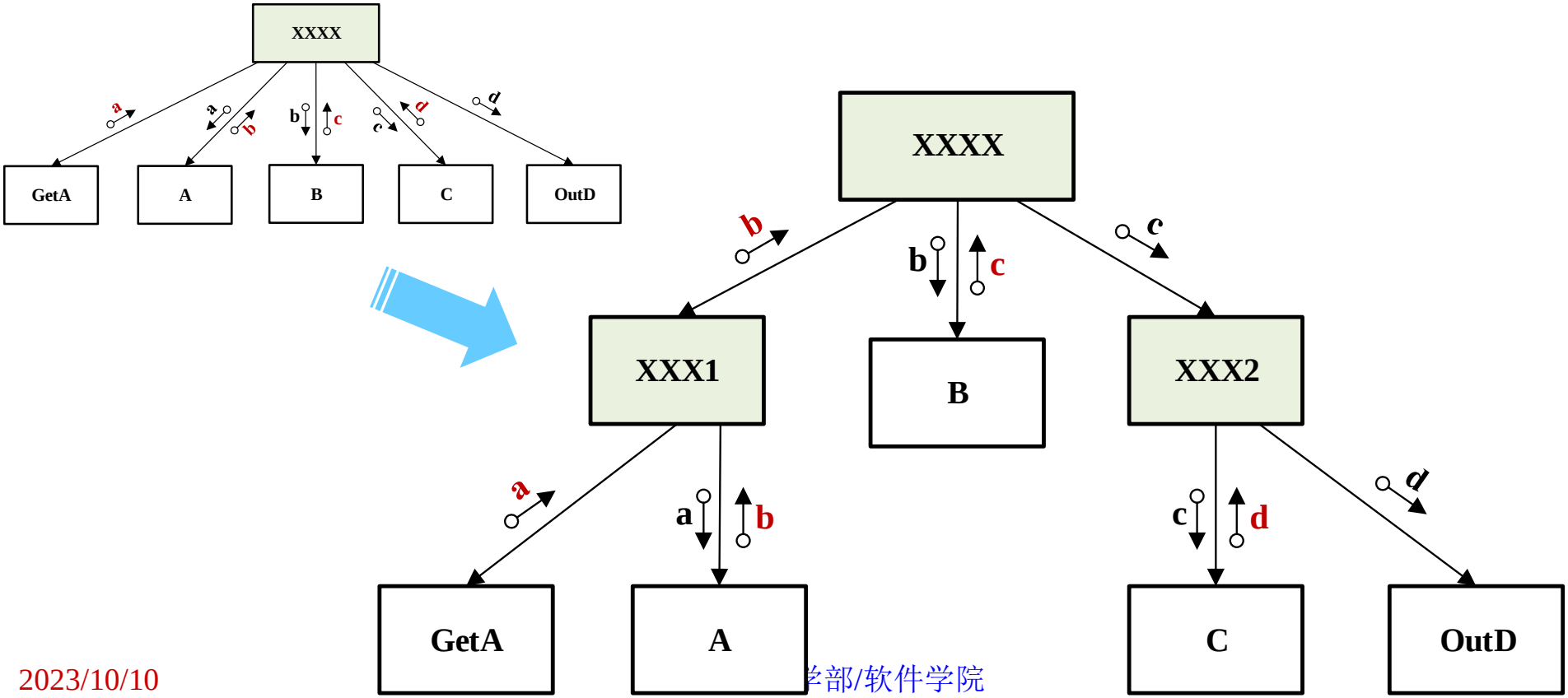


# DFD到系统结构图转换的基本模式

## 1.简单变换型DFD→系统结构图转换



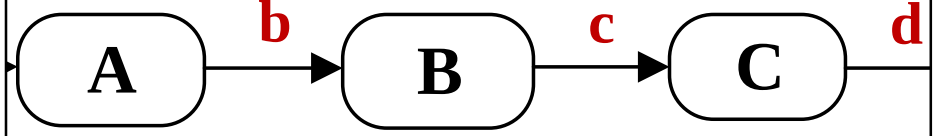

第2种情况：虑扇出度≤3时，得到系统结构图



# 结构化系统设计

## 结构图转换的基本模式

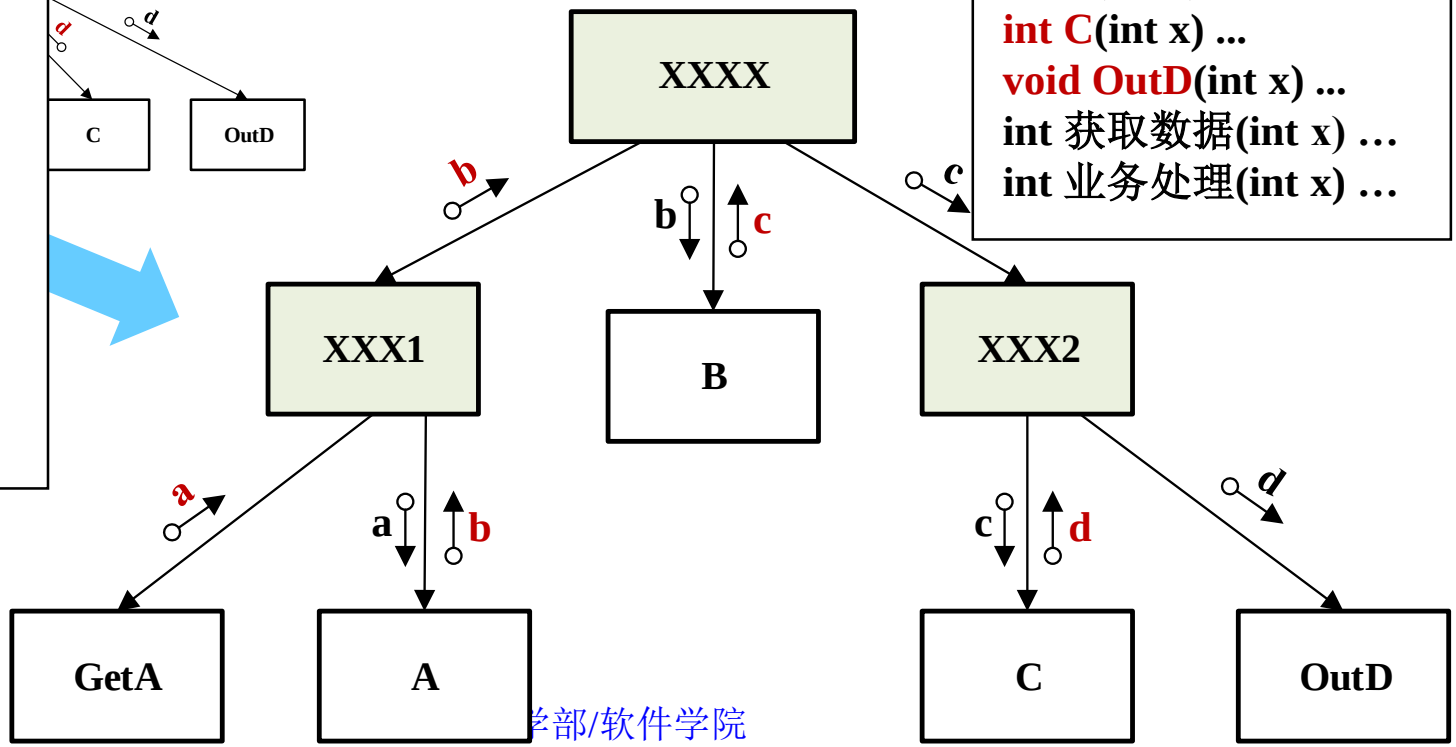
### DFD→系统结构图转换




考虑扇出度≤3时，得到系统结构

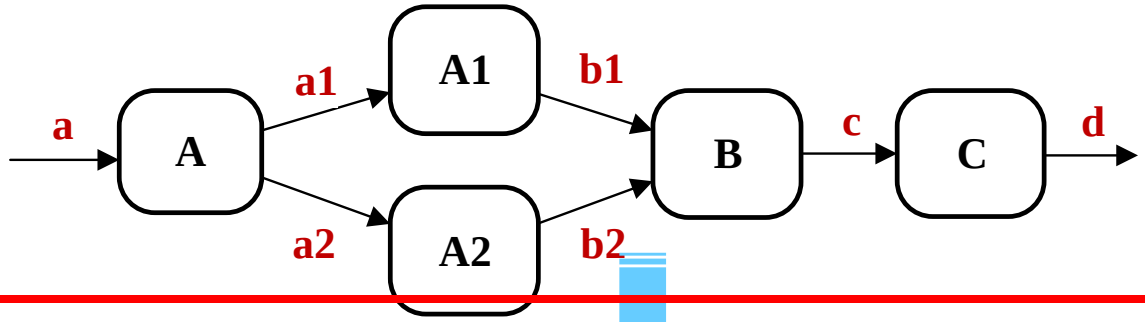
```
/*系统控制主结构*/
main()
 {  int b, c;
    b = XXX1( );
    c = B( b );
    XXX2( c );
 }
/*系统控制子结构*/
int XXX1( )
 {  int a, b;
    a = GetA( );
    b = A( a );
    return b;
 }
void XXX2( int x )
 {  int d;
    d = C( x );
    OutD( d );
 }
```

```
/*其他函数定义*/
int GetA( )
  {  int x;
     x = 获取数据( );
     return x;
  }
int A(int x)
  {  inty;
     y = 业务处理( x );
     return y;
  }
int B(int x) ...
int C(int x) ...
void OutD(int x) ...
int 获取数据(int x) …
int 业务处理(int x) …
```

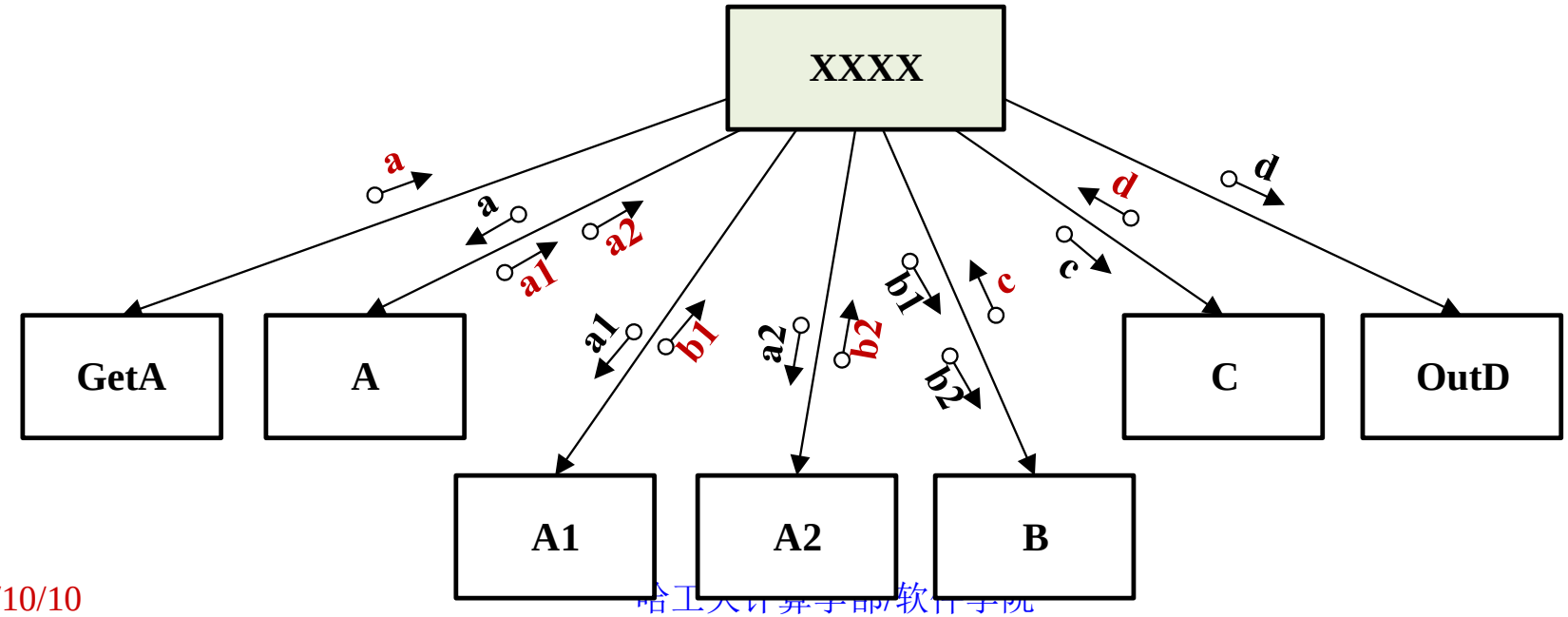


# DFD到系统结构图转换的基本模式

## 2.复杂变换型DFD→系统结构图转换



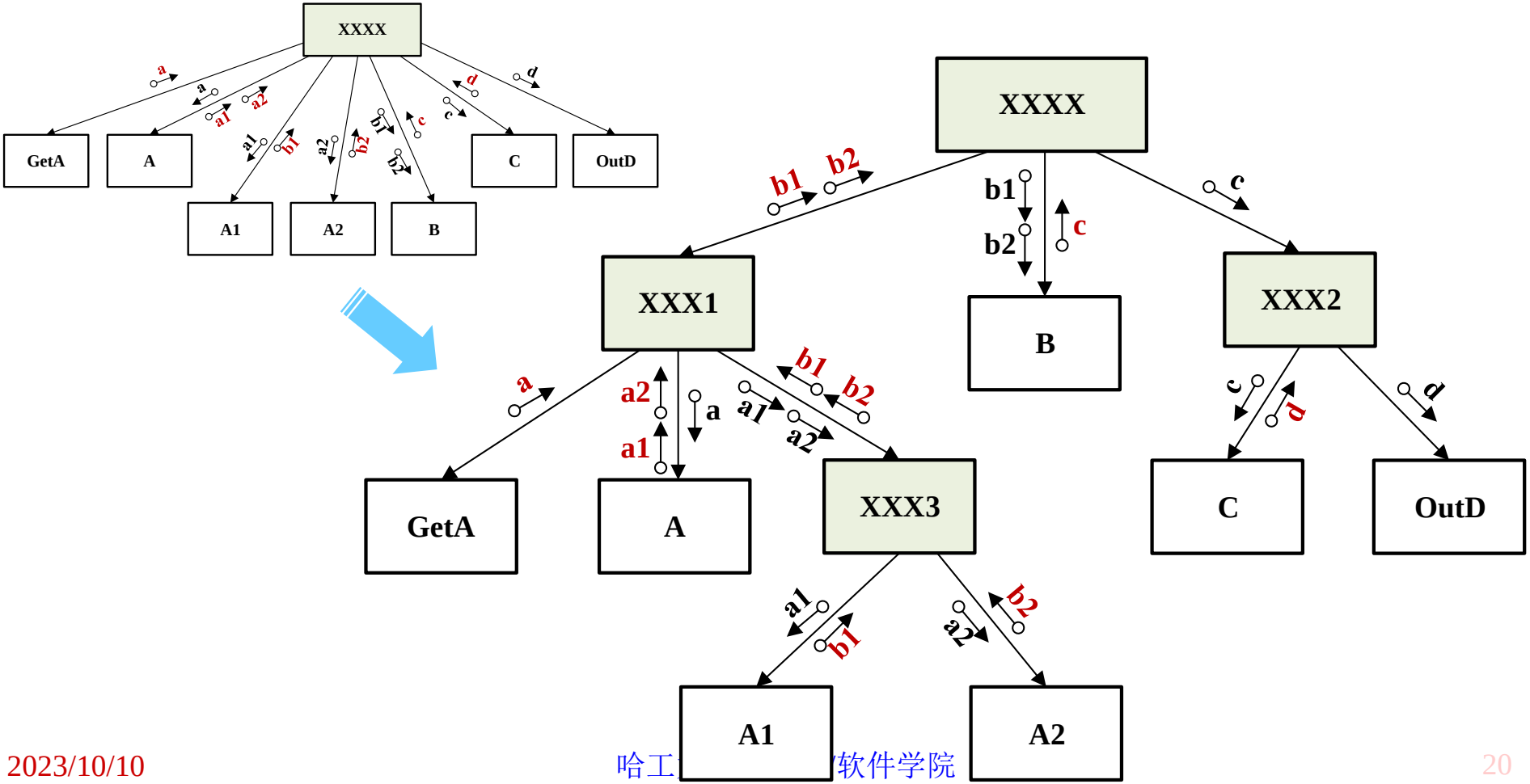

第1种情况：不考虑扇出度得到系统结构图



# DFD到系统结构图转换的基本模式

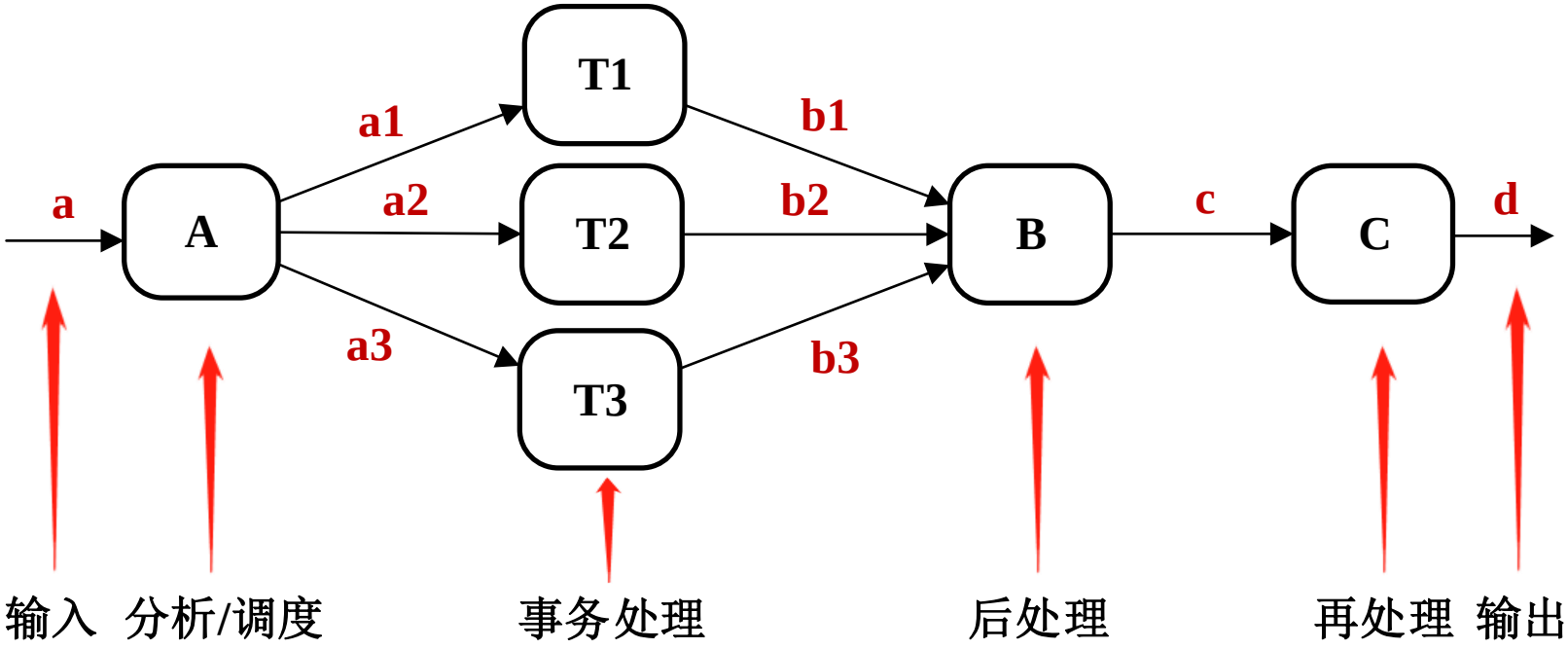
## 2.复杂变换型DFD→系统结构图转换

**第2种情况：考虑扇出度≤3时得到系统结构图**



# DFD到系统结构图转换的基本模式

## 3.事务型DFD→系统结构图转换

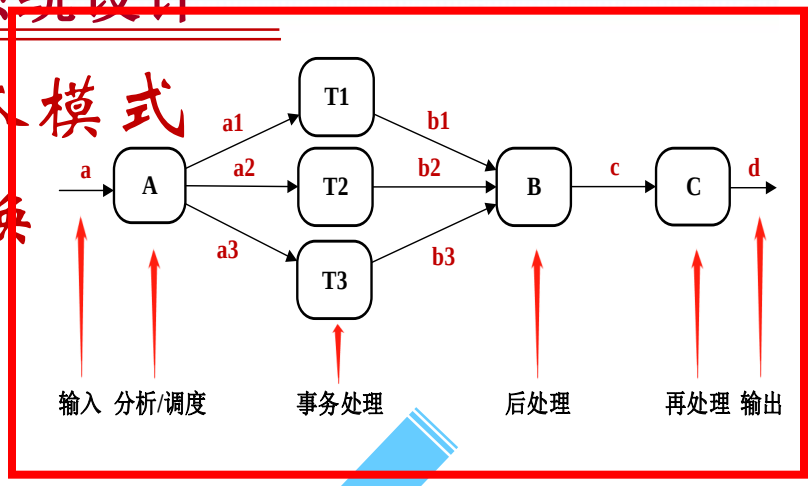


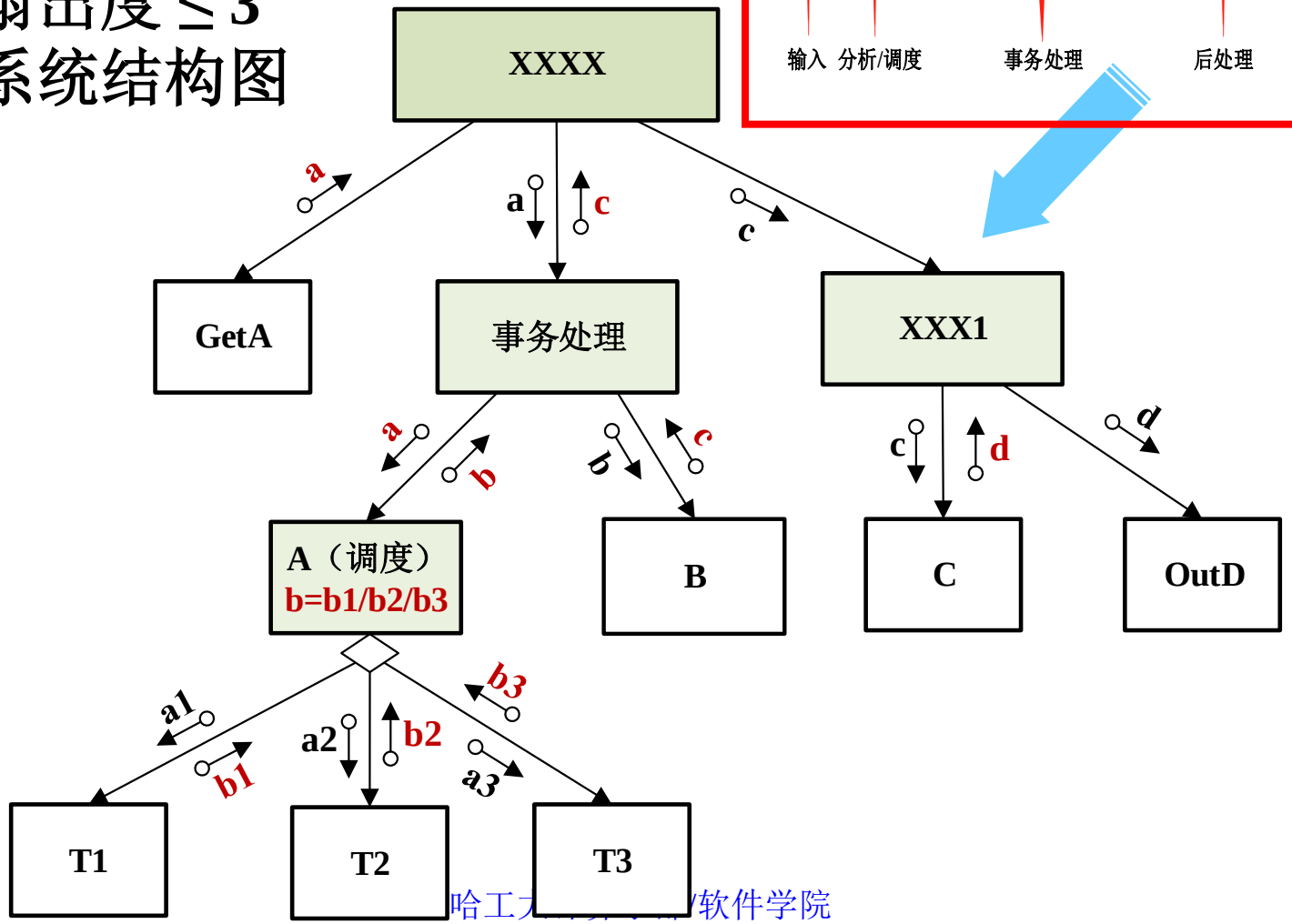
# 结构化系统设计

## DFD到系统结构图转换的基本模式

### 3.事务型DFD→系统结构图转换

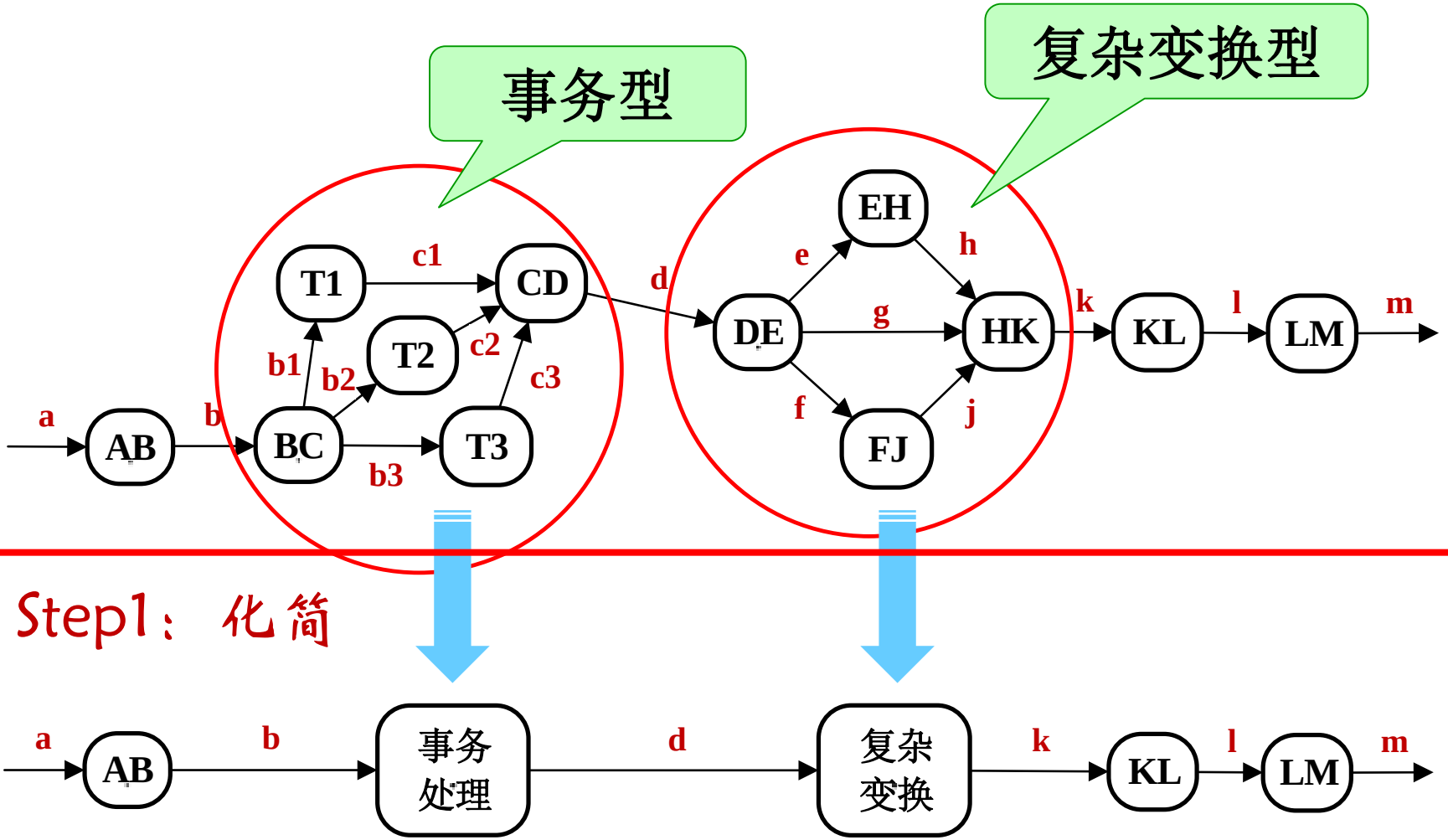
考虑扇出度≤3
得到系统结构图



# DFD到系统结构图转换的基本模式

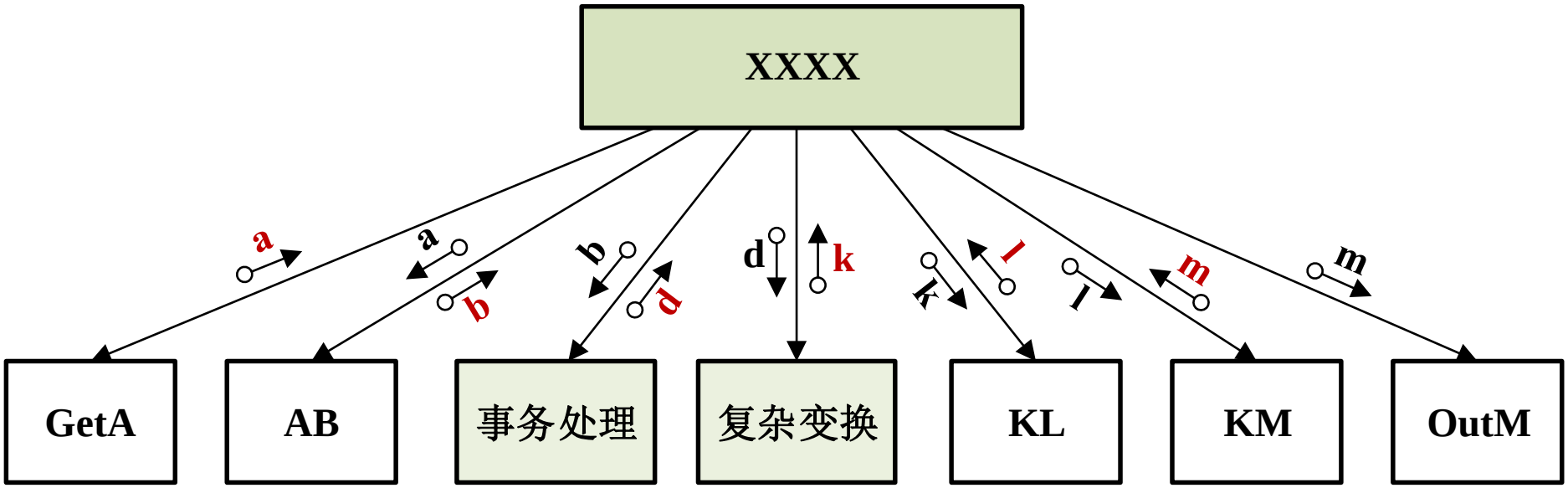
## 4.事务型与变换型混合复杂DFD→结构图转换



# DFD到系统结构图转换的基本模式

## 4.事务型与变换型混合复杂DFD→结构图转换

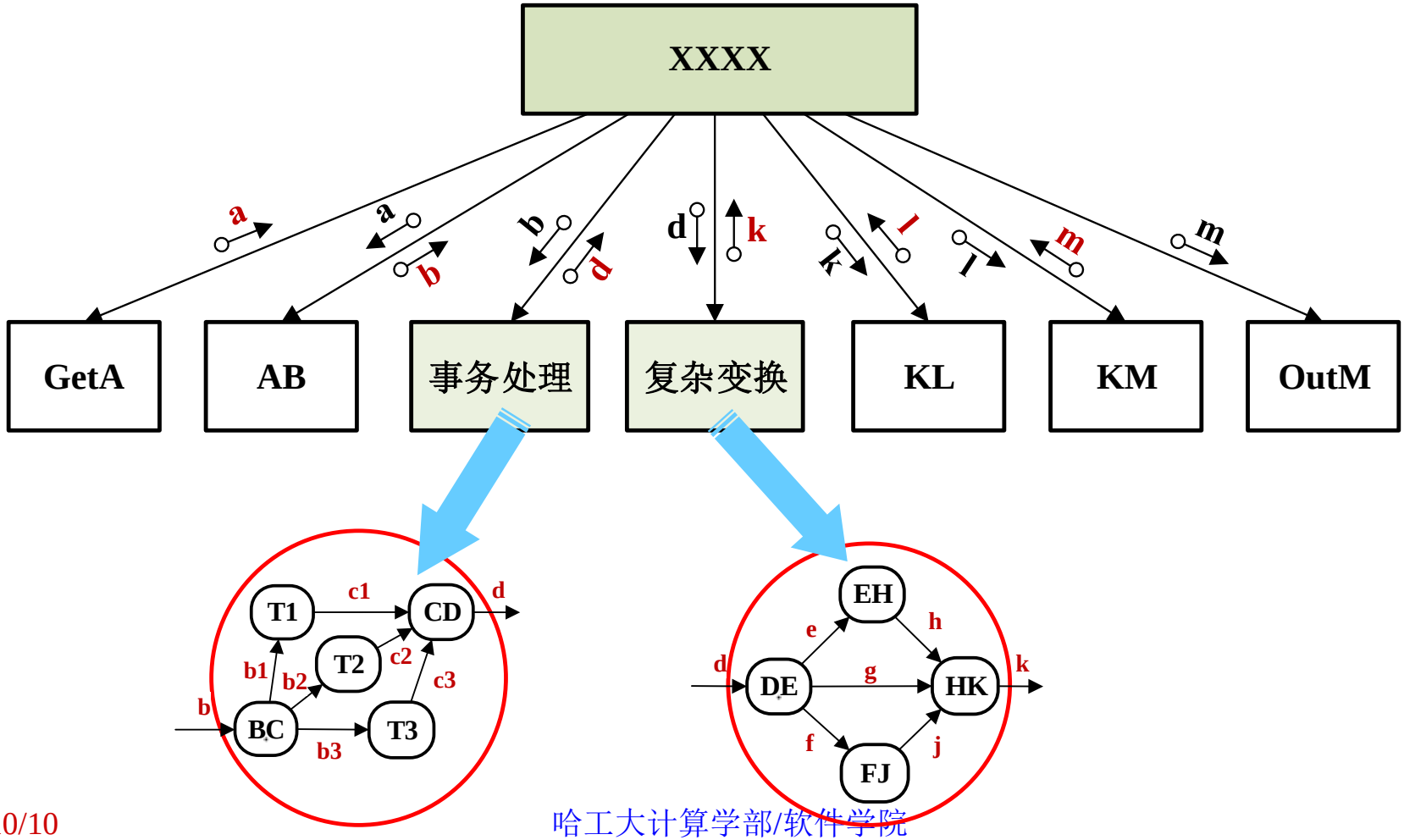
Step2：将化简的DFD→结构图转换



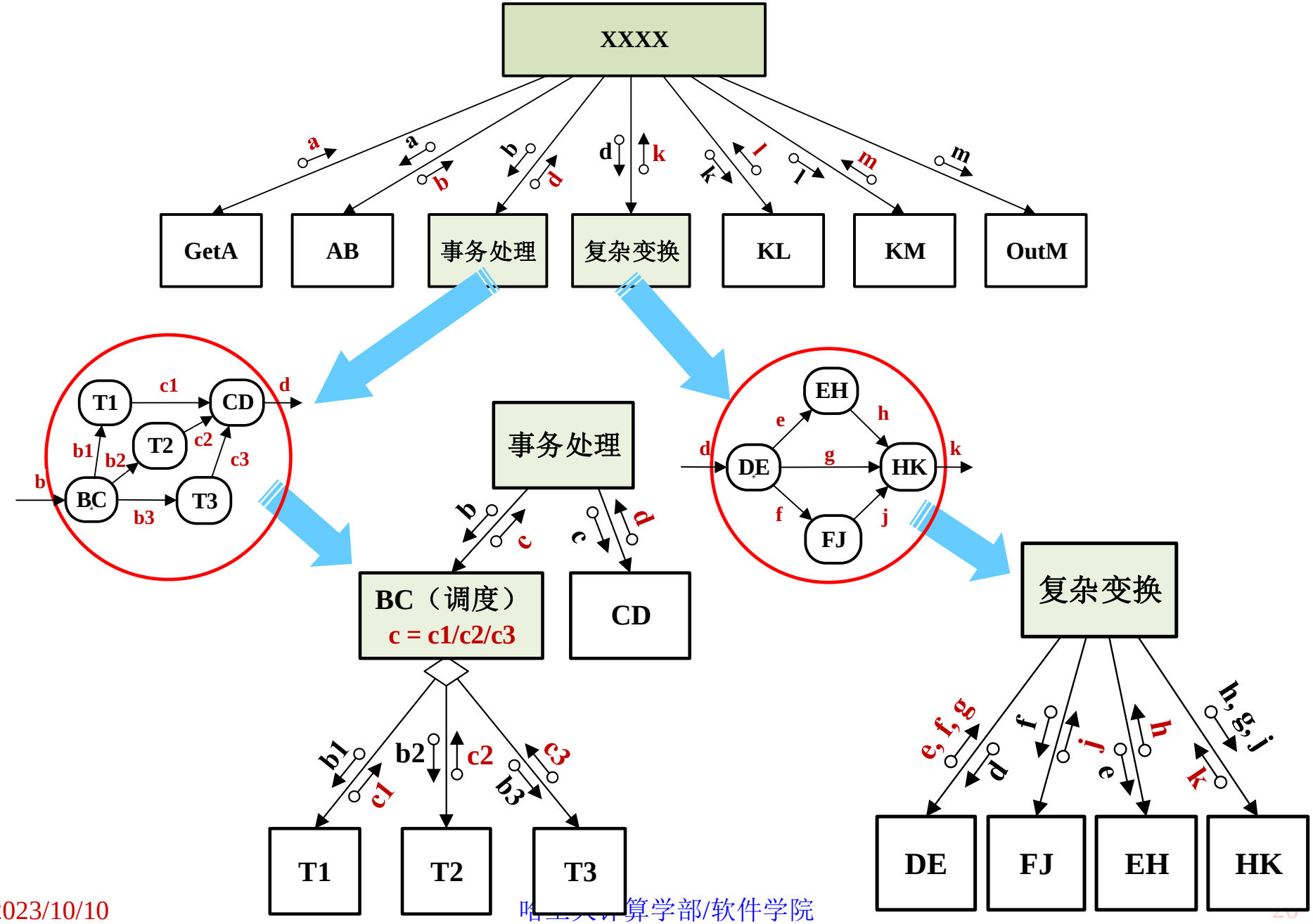
# DFD到系统结构图转换的基本模式

## 4.事务型与变换型混合复杂DFD→结构图转换

Step3：拓展事务处理和复杂变换模块的结构图



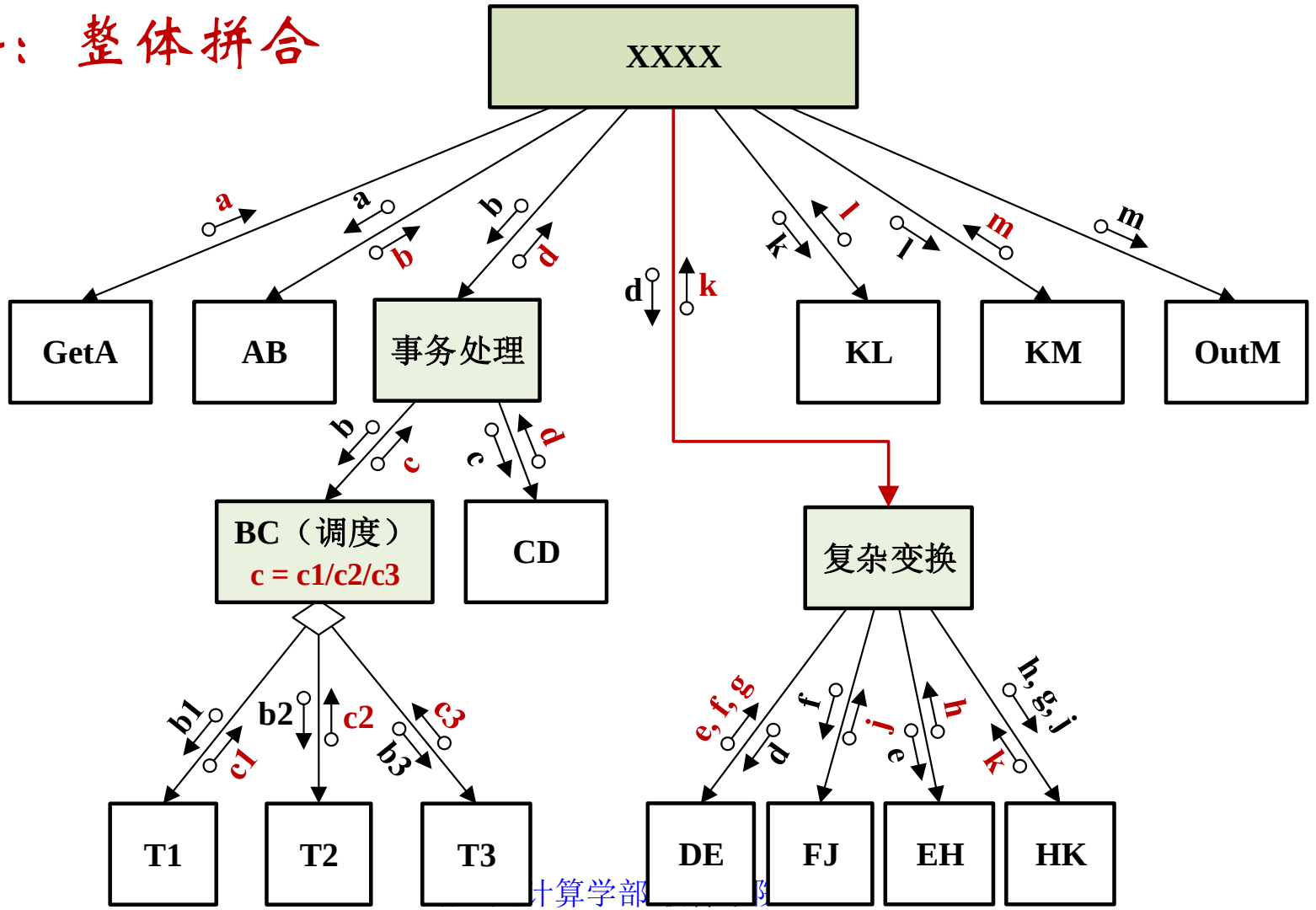
# Step3：拓展事务处理和复杂变换模块的结构图



# DFD到系统结构图转换的基本模式

## 4.事务型与变换型混合复杂DFD→结构图转换

Step4：整体拼合

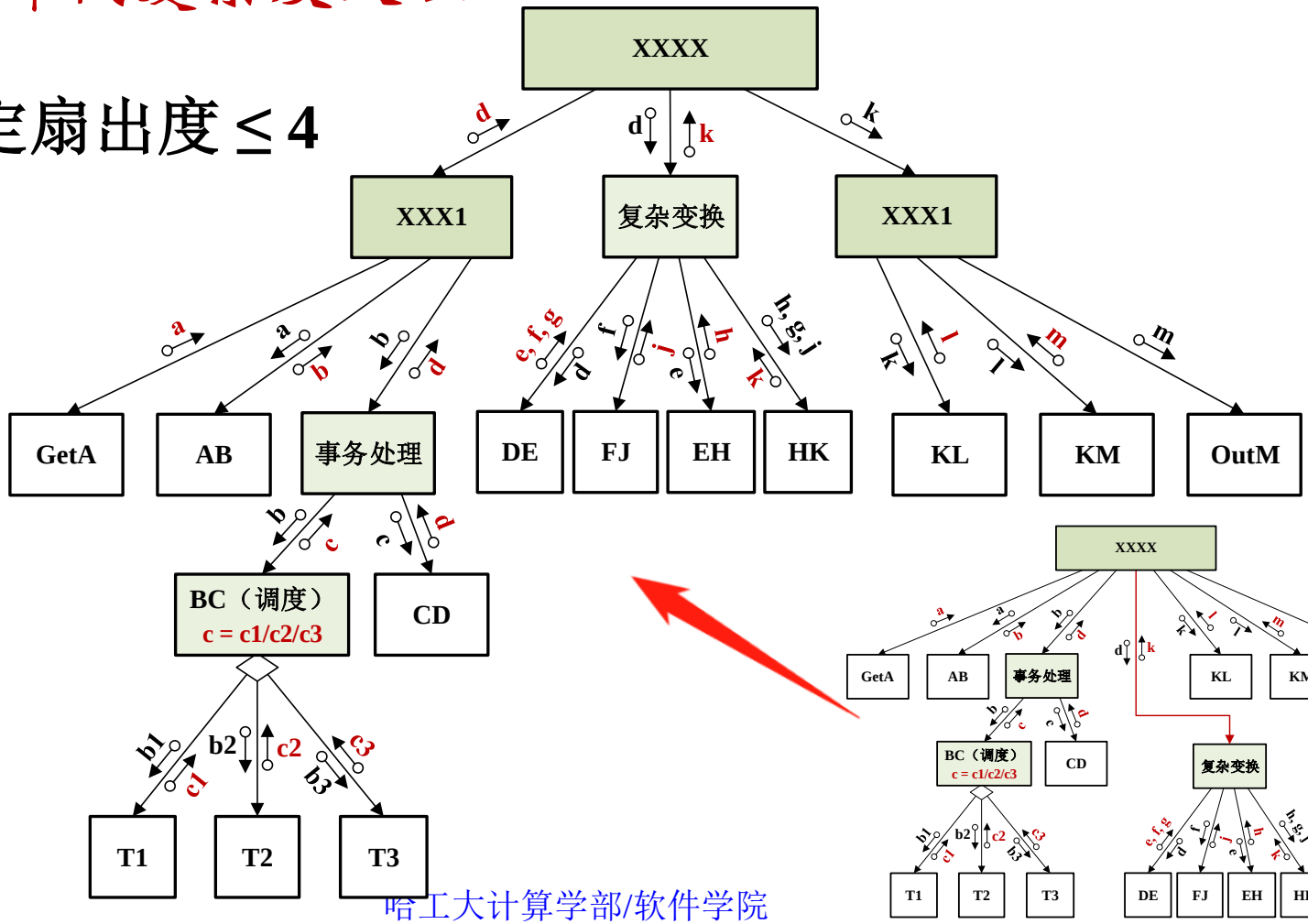


# DFD到系统结构图转换的基本模式

## 4.事务型与变换型混合复杂DFD→结构图转换

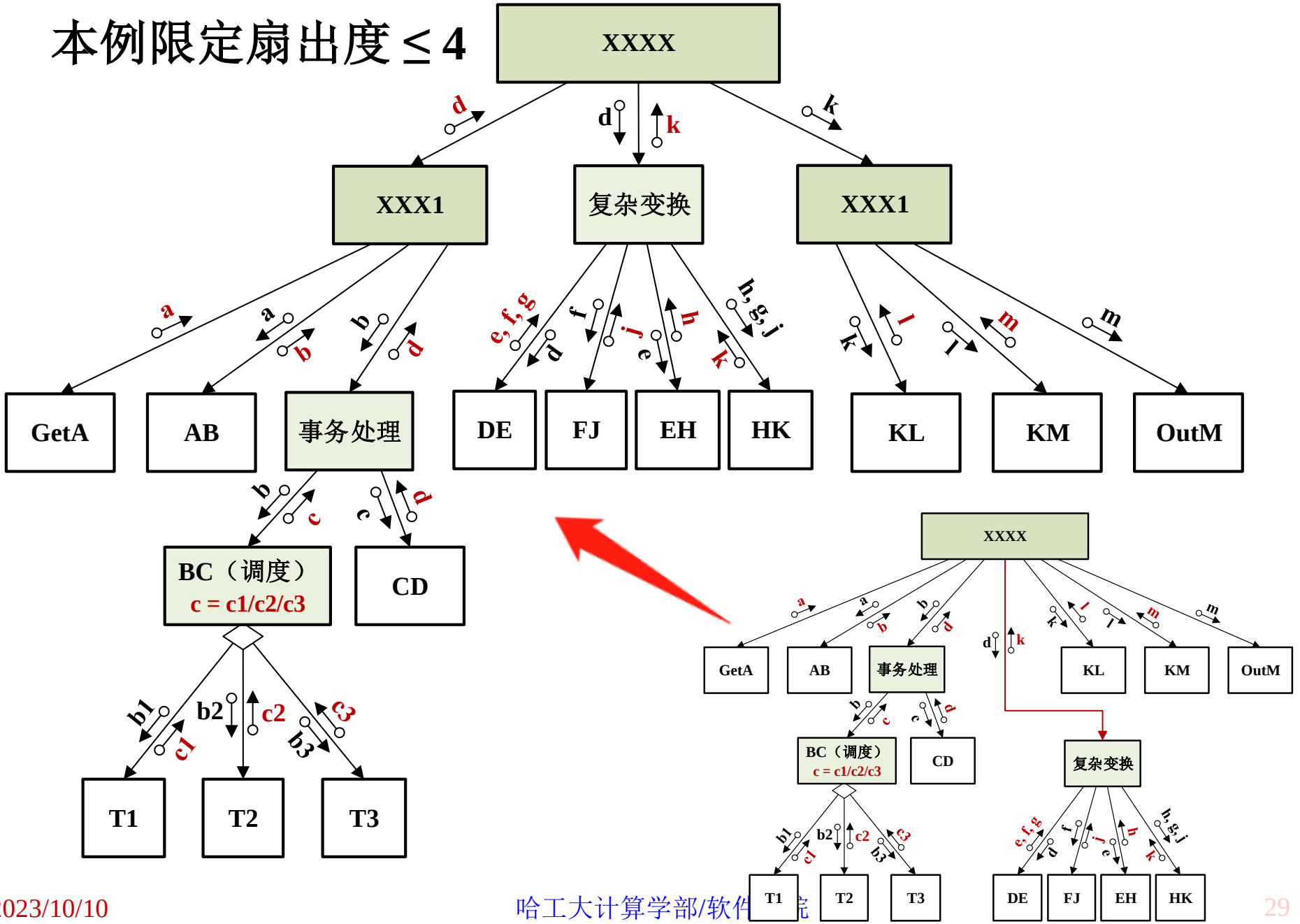
### Step5：降低复杂度处理

本例限定扇出度≤4



# Step5：降低复杂度处理

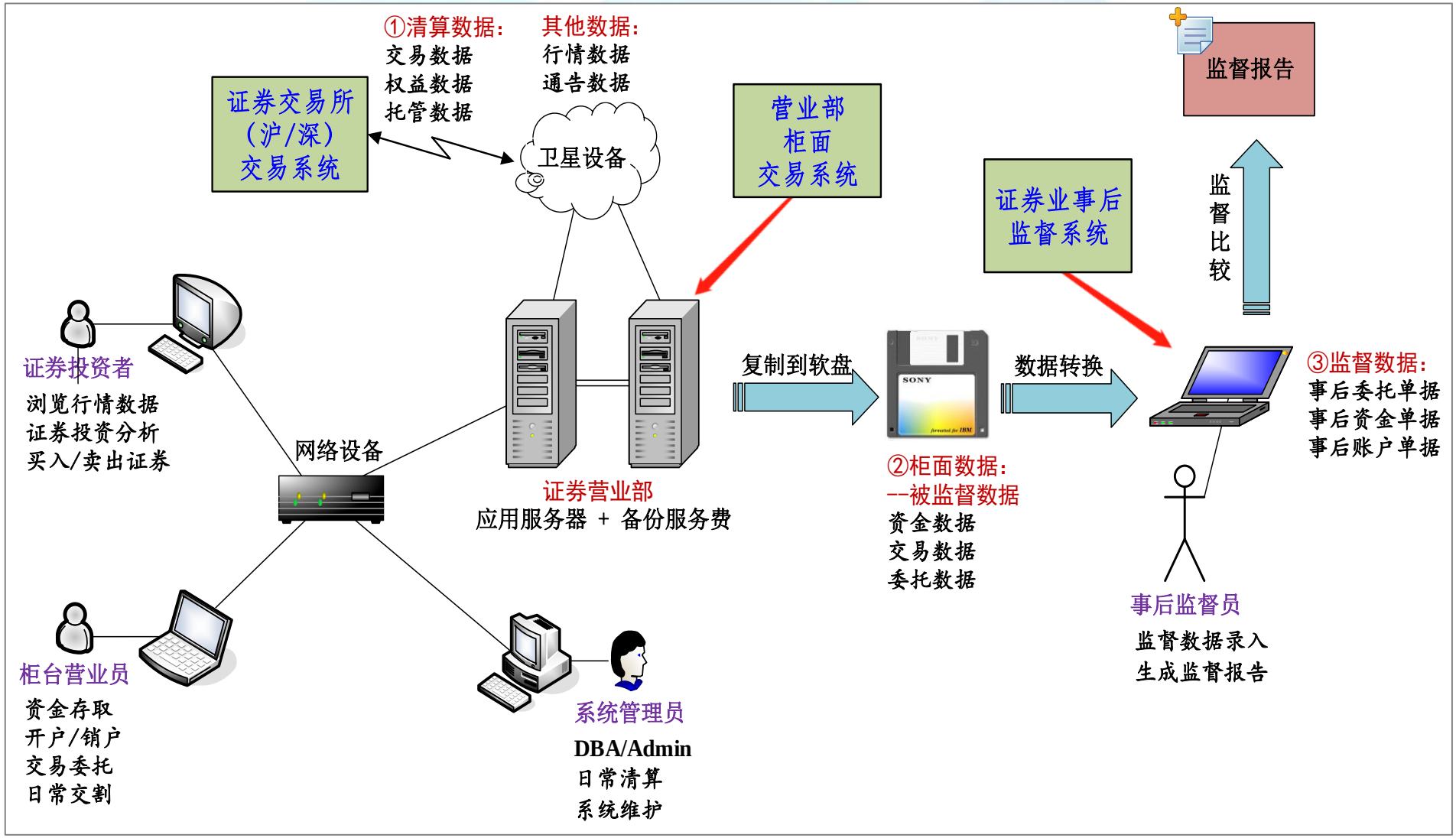
本例限定扇出度≤4



# 结构化系统设计

## 结构化分析与设计实例

## 《证券业事后监督系统》需求介绍



# 结构化分析与设计实例

**公司负责人愿景** 即通过事后监督系统能够：

① 及时发现日常工作差错

② 防止营业部内部违法犯罪

**事后监督系统目标（愿景分解）：**

① 监督营业部日常资金存取安全

② 监督证券交易环节准确无误、无违规操作

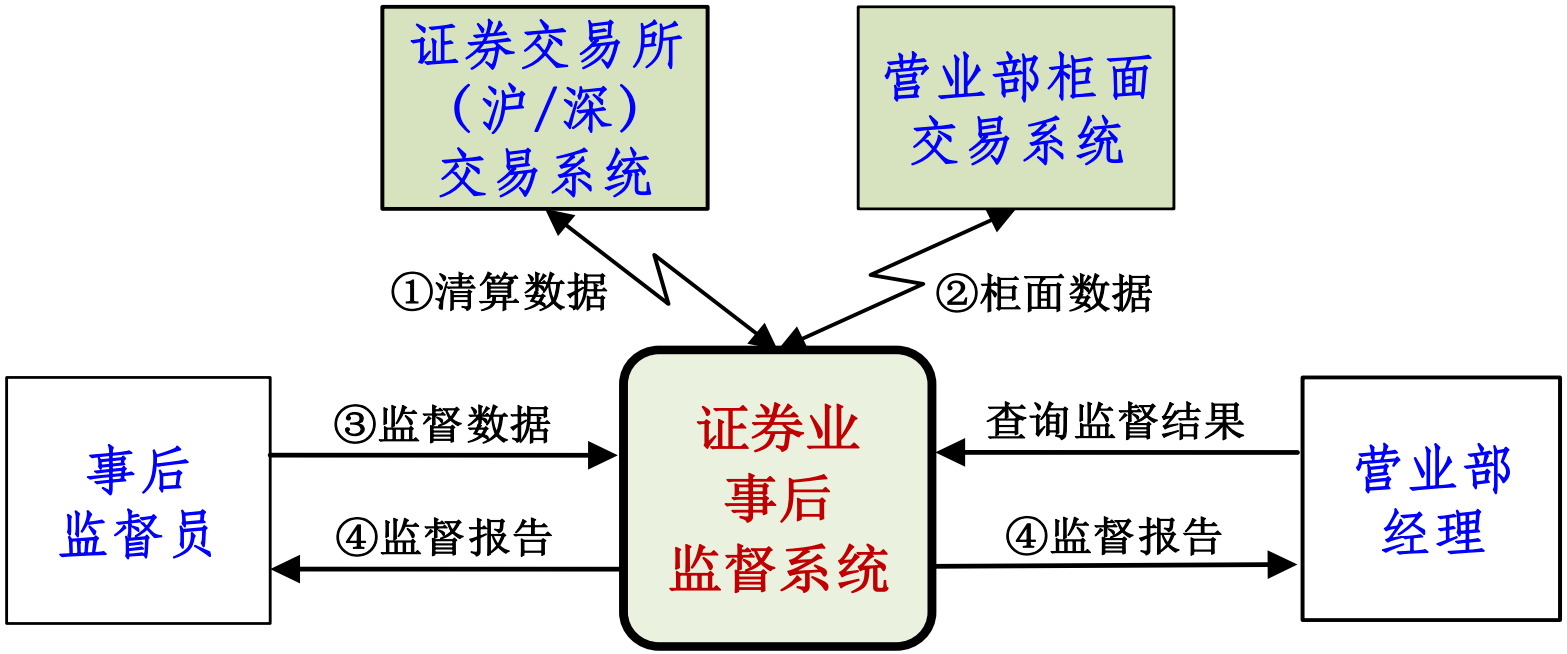
③ 监督营业部清算环节无差错、无违法犯罪

**差　　错：** 资金存取、交易委托、清算数据等

**违规违法：** 资金透支、资金转移、透支交易、中签转移、计息克扣、证券余额转移、权益转移等

# 结构化分析与设计实例

## 《证券业事后监督系统》顶层DFD




数据流说明：

①清算数据：
- 交易数据
- 权益数据
- 托管数据

②柜面数据：
- 资金数据
- 交易数据
- 委托数据

③监督数据：
- 事后委托单据
- 事后资金单据
- 事后账户单据

④监督报告：
- 比较类报告
- 自查类报告
- 清算类报告

# 结构化分析与设计实例

## 《证券业事后监督系统》分析与设计

| | |
|---|---|
| 01-SHJD数据逻辑流程 | 数据流图DFD建立 |
| 02-SHJD数据结构说明 | 数据字典DD建立 |
| 03-SHJD功能模块分布表 | 结构图建立 |
| 04-SHJD数据库关系表 | 实体关系图ERD建立 |
| 05-SHJD文件-目录分布图 | 部署设计 |

# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

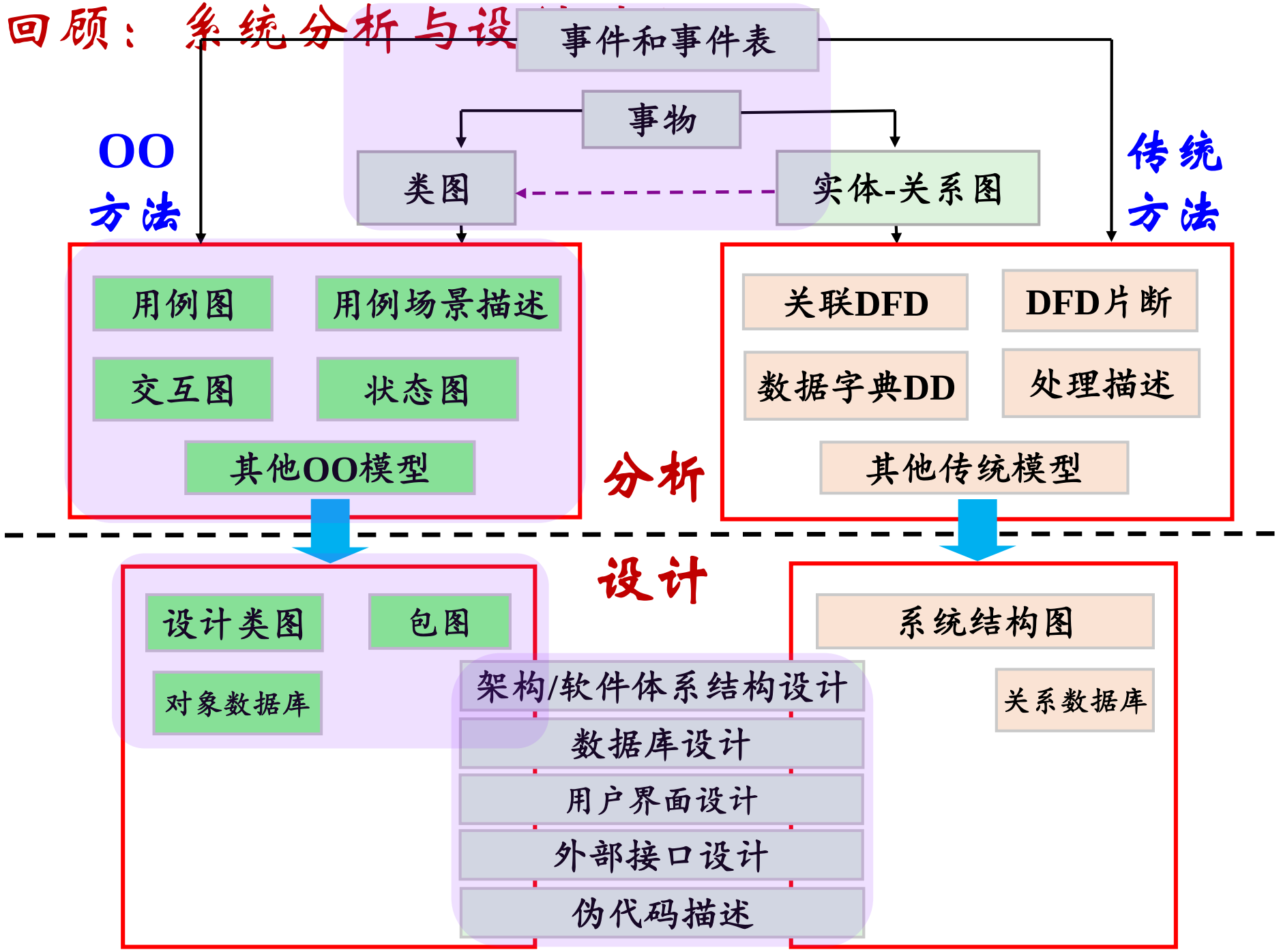
邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 10


回顾：系统分析与设
事件和事件表
事物
类图
实体-关系图
OO
方法
传统
方法
用例图
用例场景描述
交互图
状态图
其他OO模型
关联DFD
DFD片断
数据字典DD
处理描述
其他传统模型
分析
设计
设计类图
包图
对象数据库
架构/软件体系结构设计
数据库设计
用户界面设计
外部接口设计
伪代码描述
系统结构图
关系数据库

# OOAD案例 -- 评教系统

## 评教系统：需求描述

某大学教务处计划开发一套评教管理系统，基本要求及要达到的主要目标如下：

①在课程教学过程完成后，由学生对所选修的课程的教师授课情况进行打分评价；

②在课程授课过程中，由督导专家通过抽查听课对教师授课情况进行打分评价；

③最终对每门课程的学生评分和督导专家评分结果分别进行加权平均计算，统计得到每门课程授课情况的评价结果；

④评教结果将通过书面形式反馈给任课教师、学校、教务处及学院领导，最终达到改进和提高教学质量的目的。

# OOAD案例 -- 评教系统

## 评教系统：需求描述

通过对该系统的需求进行分析，我们得到如下信息：

①每学期开学初，教学秘书将录入该学期的所有开课的课程信息，包括*课程编号*、*课程名称*、*任课教师*、*开课班级*等；当然他/她也可以查看、修改、删除已有的课程信息。同时，教学秘书还要录入所有学生的信息，包括*学号*、*学生姓名*、*年级*等信息；当然他/她也可以查看、修改、删除已有的学生信息。而且允许教学秘书通过准备好的Excel文件批量导入课程信息和学生信息。

②教学秘书准备好上述数据后，由系统管理员启动评教系统，学生才可以登录并进行评教活动。当系统管理员关闭评教系统后，评教活动即结束了。

# OOAD案例 -- 评教系统

## 评教系统：需求描述

通过对该系统的需求进行分析，我们得到如下信息（续）：

③督导专家的听课打分结果是由教学督导委员会秘书录入的。最后，教学督导委员会秘书将统计学生和督导专家的评教结果，他/她可以浏览查看所有课程的评教结果，也可以批量导出课程的评教结果到Excel文件中，还可以生成并打印每门课程的评价报告（内容包括*学生各项评价平均分*、*督导专家各项评价平均分*、*学生评语汇编*、*督导专家评语汇编*等），并以书面形式反馈给任课教师。

④学生对某门课程的评教信息主要有5项内容，即*授课状态分*、*课堂互动分*、*课件评价分*、*总体印象分*、*简要评语*等。

# OOAD案例 -- 评教系统

## 评教系统：需求描述

通过对该系统的需求进行分析，我们得到如下信息（续）：

⑤督导专家对课程的听课评价内容主要有7项，即*听课时间、学生出勤人数、听课状态定性评价、符合教学大纲分数、授课水平和技巧分数、本次授课内容记录、简要评语*等。

⑥要求该评教系统允许学生、教学督导委员会秘书在任何地点均可以使用，只要能够访问Internet。但系统管理员、教学秘书只能在局域网内使用。

# OOAD案例 -- 评教系统

## 评教系统：需求分析

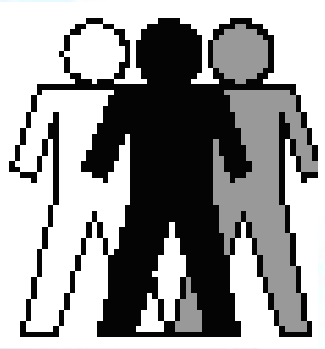
### 1.涉众分析：

（1）学校领导、学院领导、教务处领导

（2）教师、学生、督导专家（教学督导委员）

（3）教学秘书、教学督导委员会秘书

（4）系统管理员



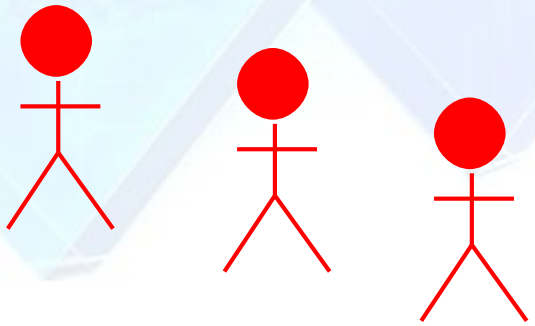
### 参与者（主角）：

（1）学生

（2）教学督导委员会秘书

（3）教学秘书

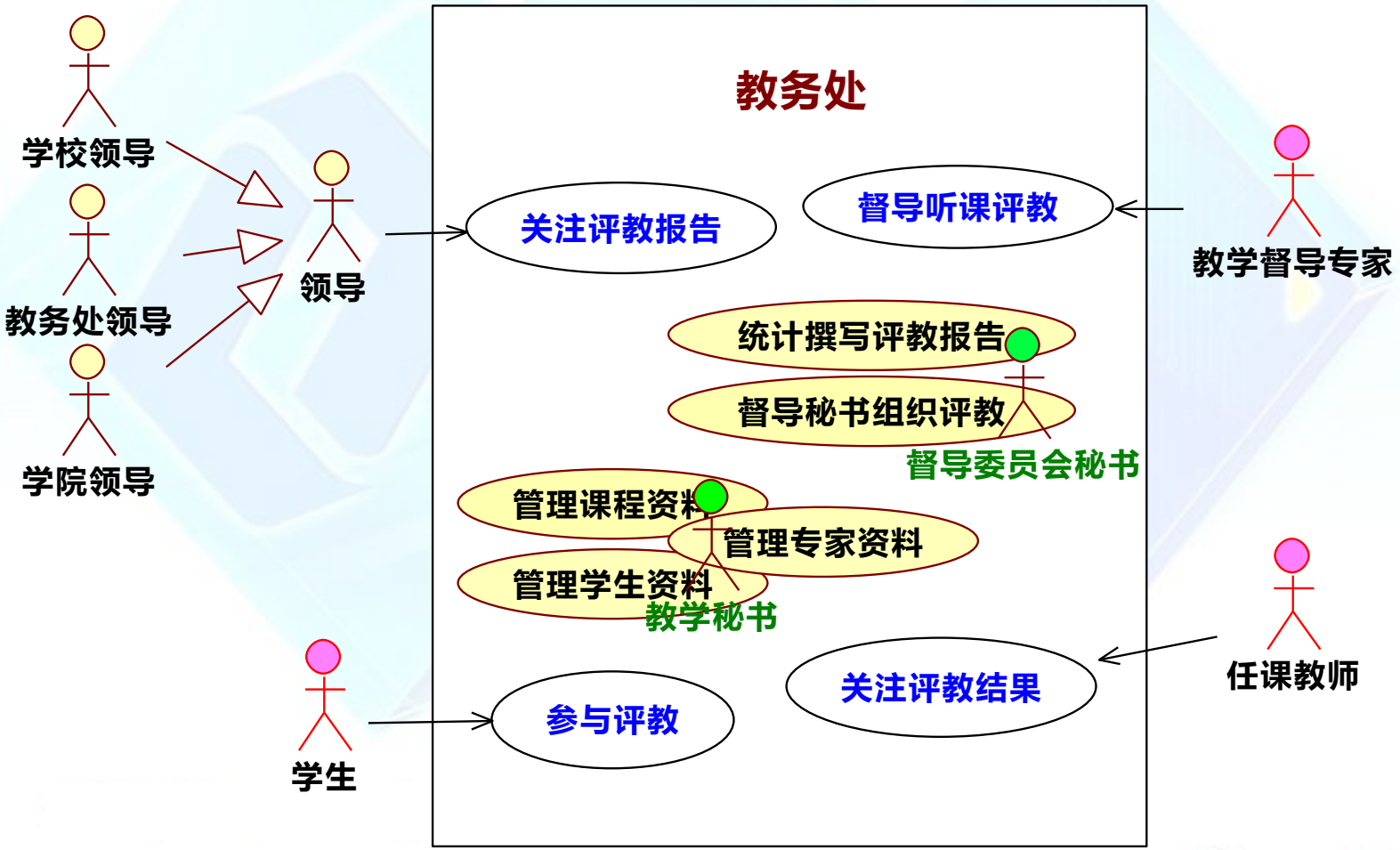
（4）系统管理员



# OOAD案例 -- 评教系统

## 评教系统：需求分析

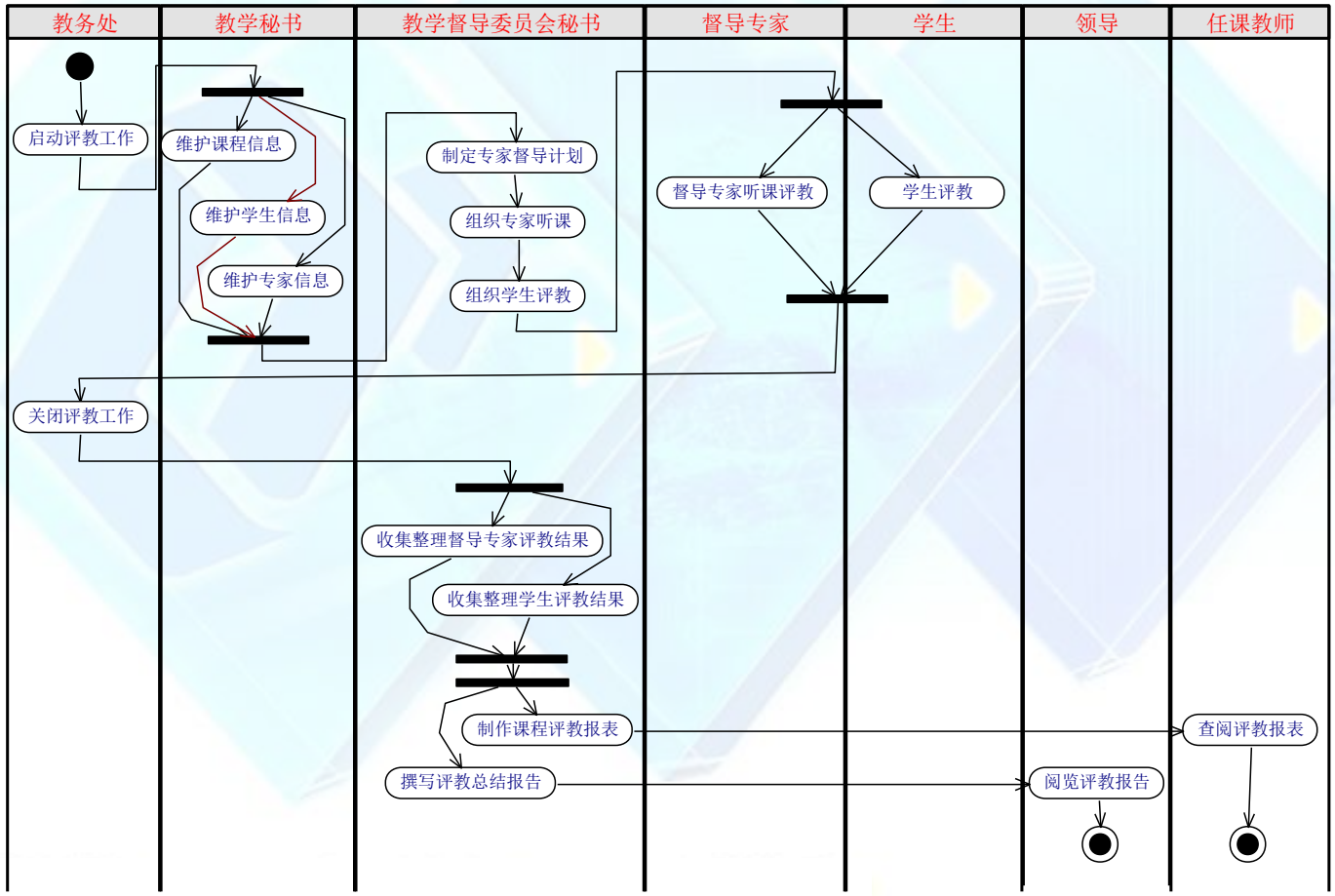
### 业务用例分析 -- 业务用例图

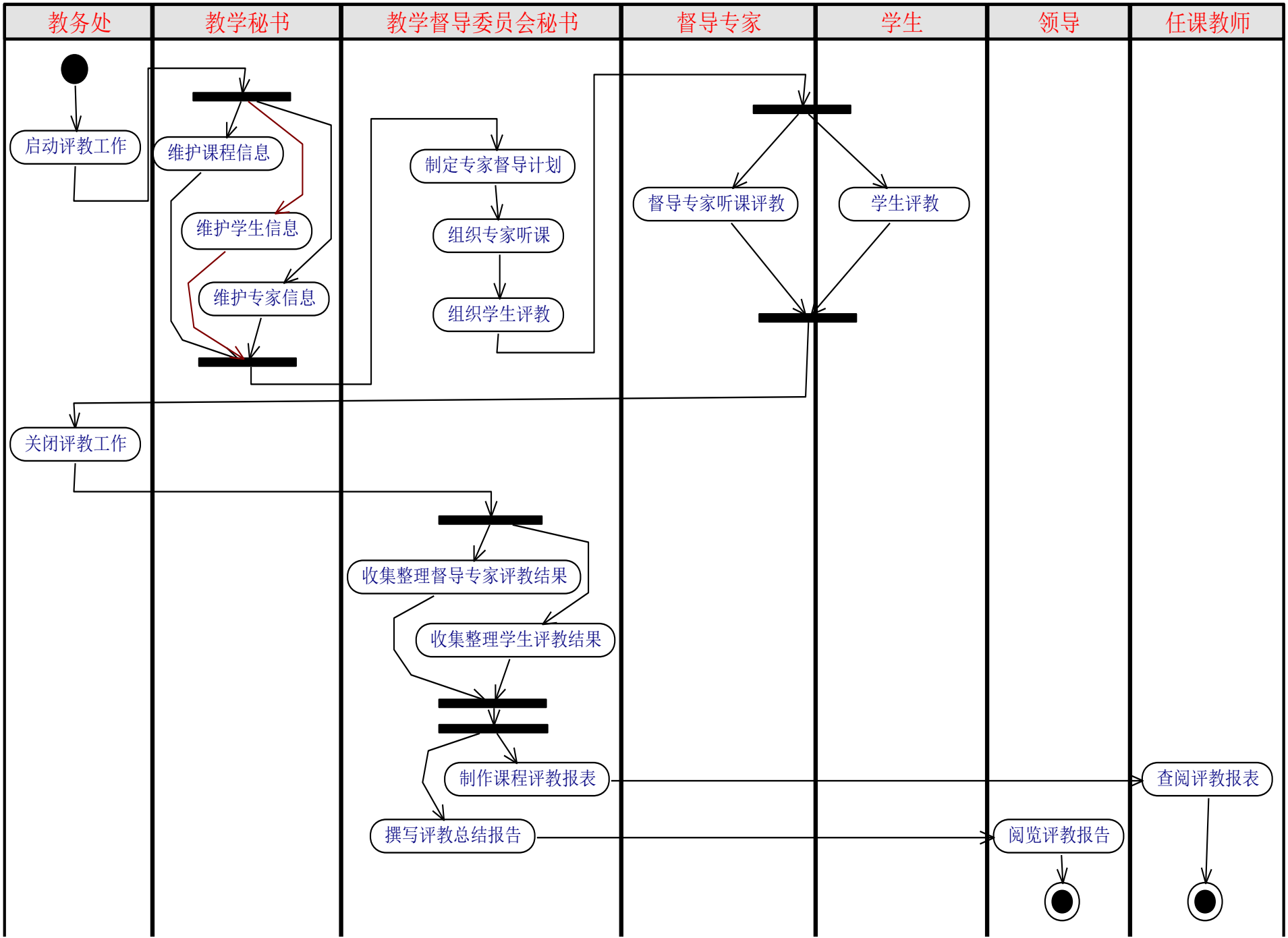


# OOAD案例 -- 评教系统

## 评教系统：需求分析

### 业务用例过程分析 -- 业务活动图

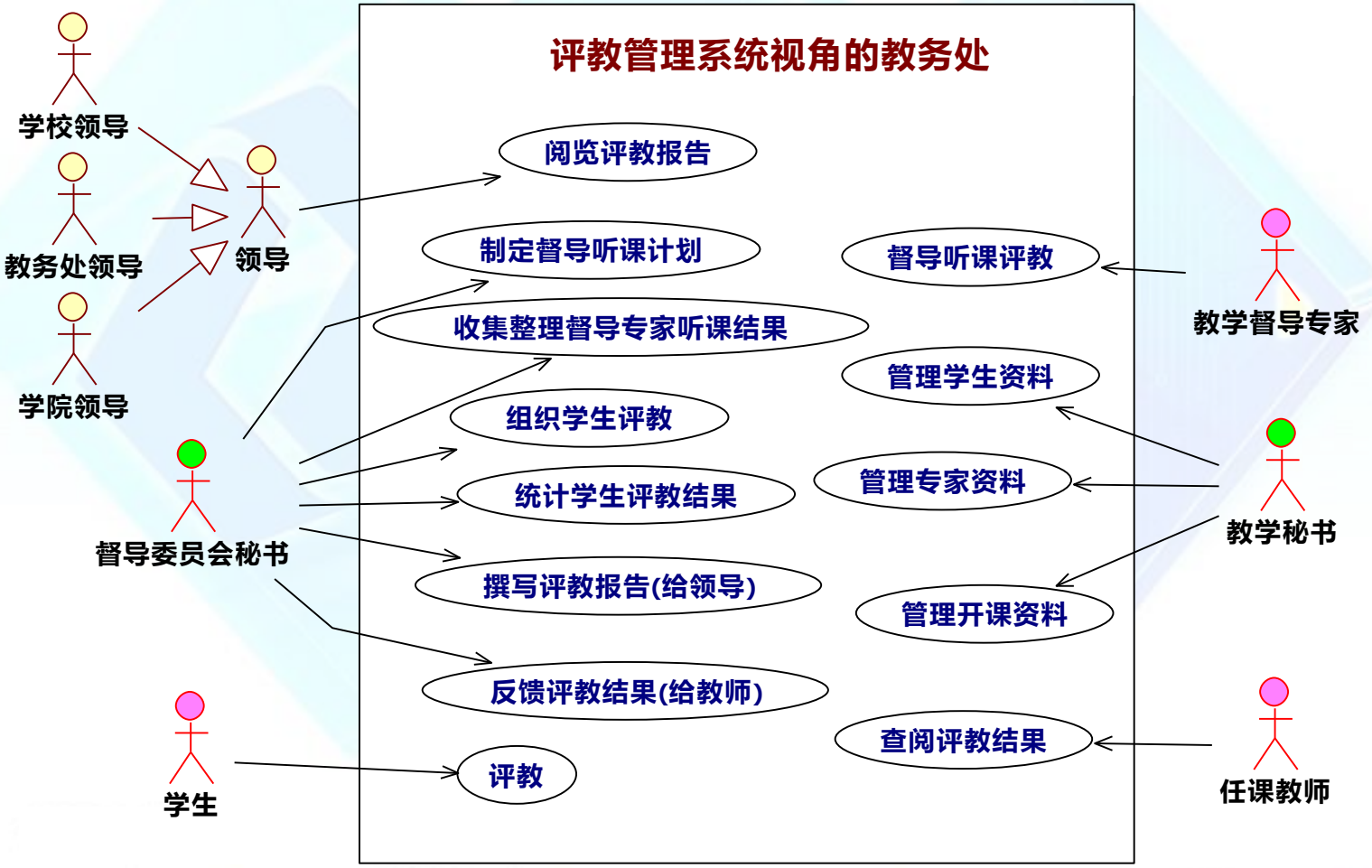



教务处
教学秘书
教学督导委员会秘书
督导专家
学生
领导
任课教师
启动评教工作
维护课程信息
维护学生信息
维护专家信息
制定专家督导计划
组织专家听课
组织学生评教
督导专家听课评教
学生评教
关闭评教工作
收集整理督导专家评教结果
收集整理学生评教结果
制作课程评教报表
撰写评教总结报告
阅览评教报告
查阅评教报表

# OOAD案例 -- 评教系统

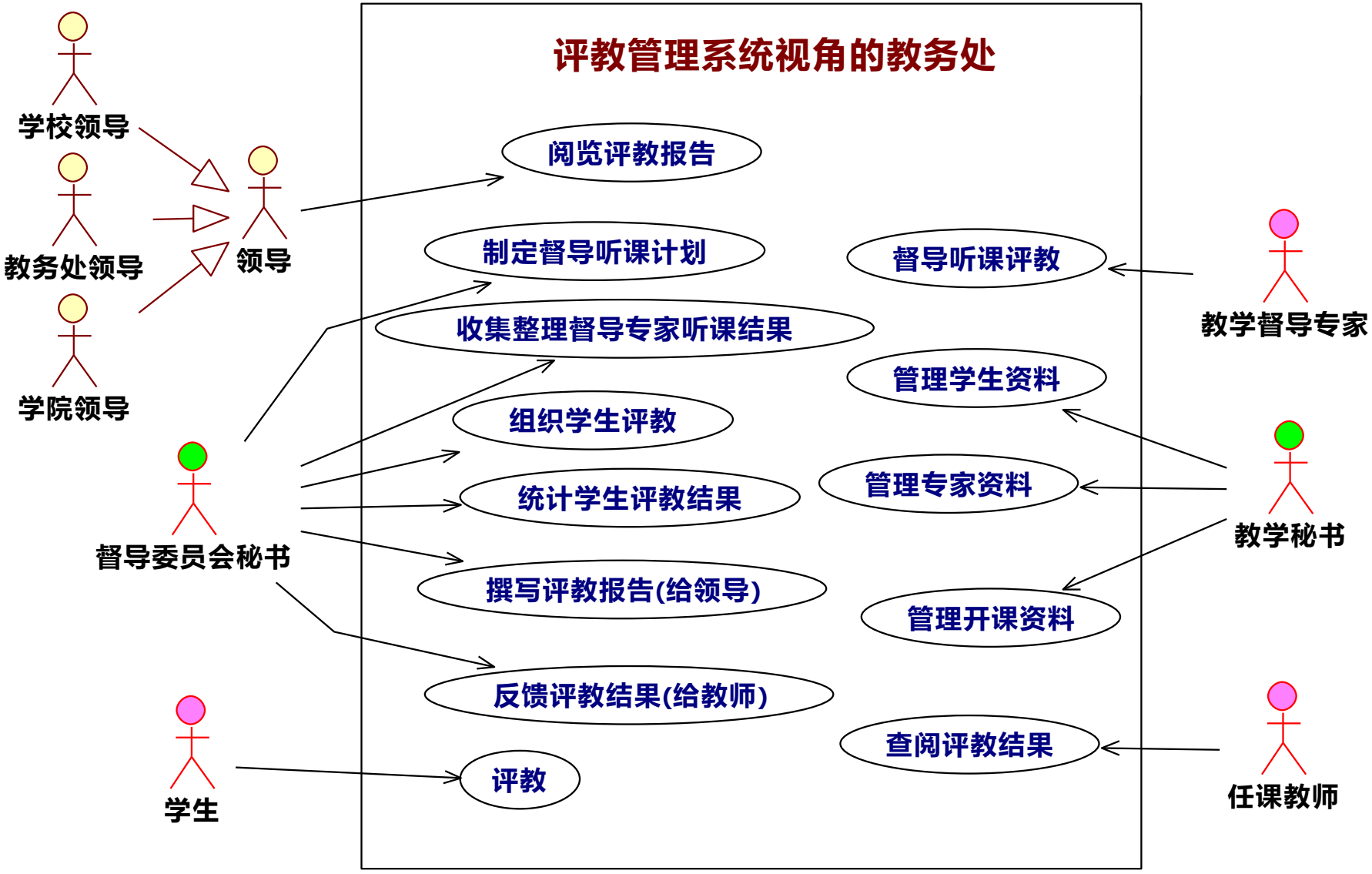
## 评教系统：需求分析

### 概念用例分析 -- 概念用例图



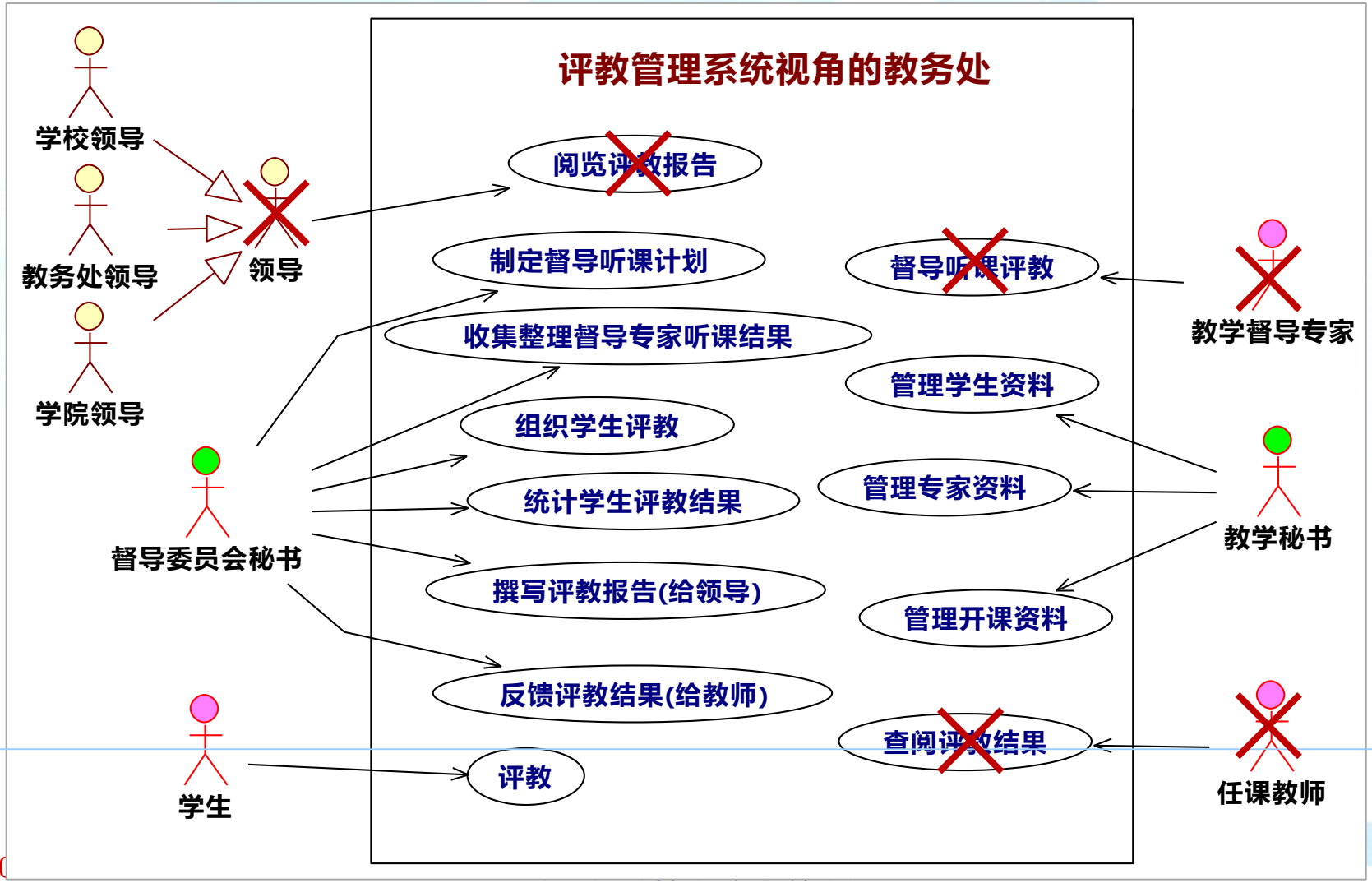
# OOAD案例 -- 评教系统

## 评教系统：需求分析



需求描述：④评教结果将通过书面形式反馈给任课教师、学校、教务处及学院领导，最终达到改进和提高教学质量的目的。

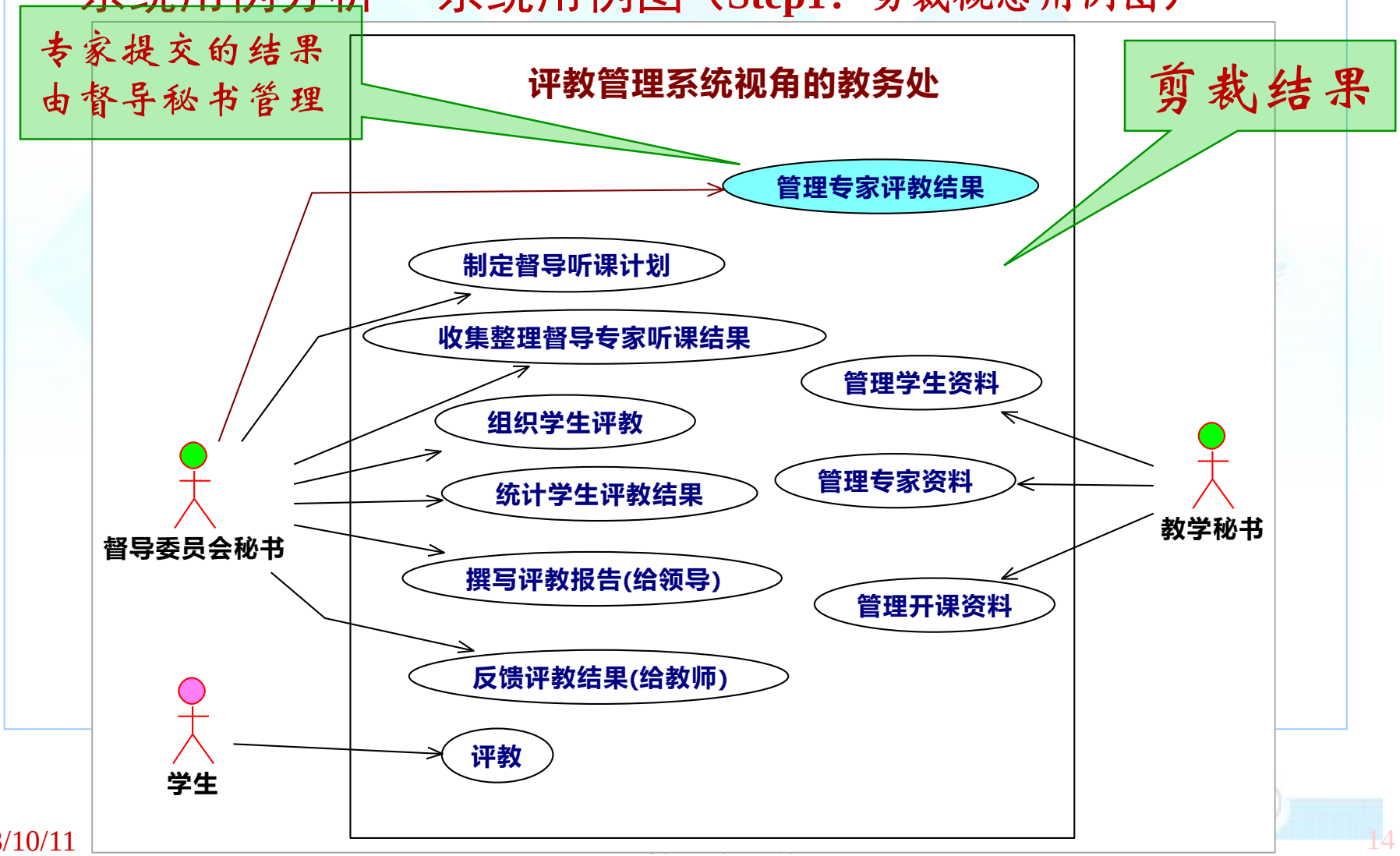
## 系统用例分析 -- 系统用例图（Step1：剪裁概念用例图）



# OOAD案例 -- 评教系统

## 评教系统：系统分析

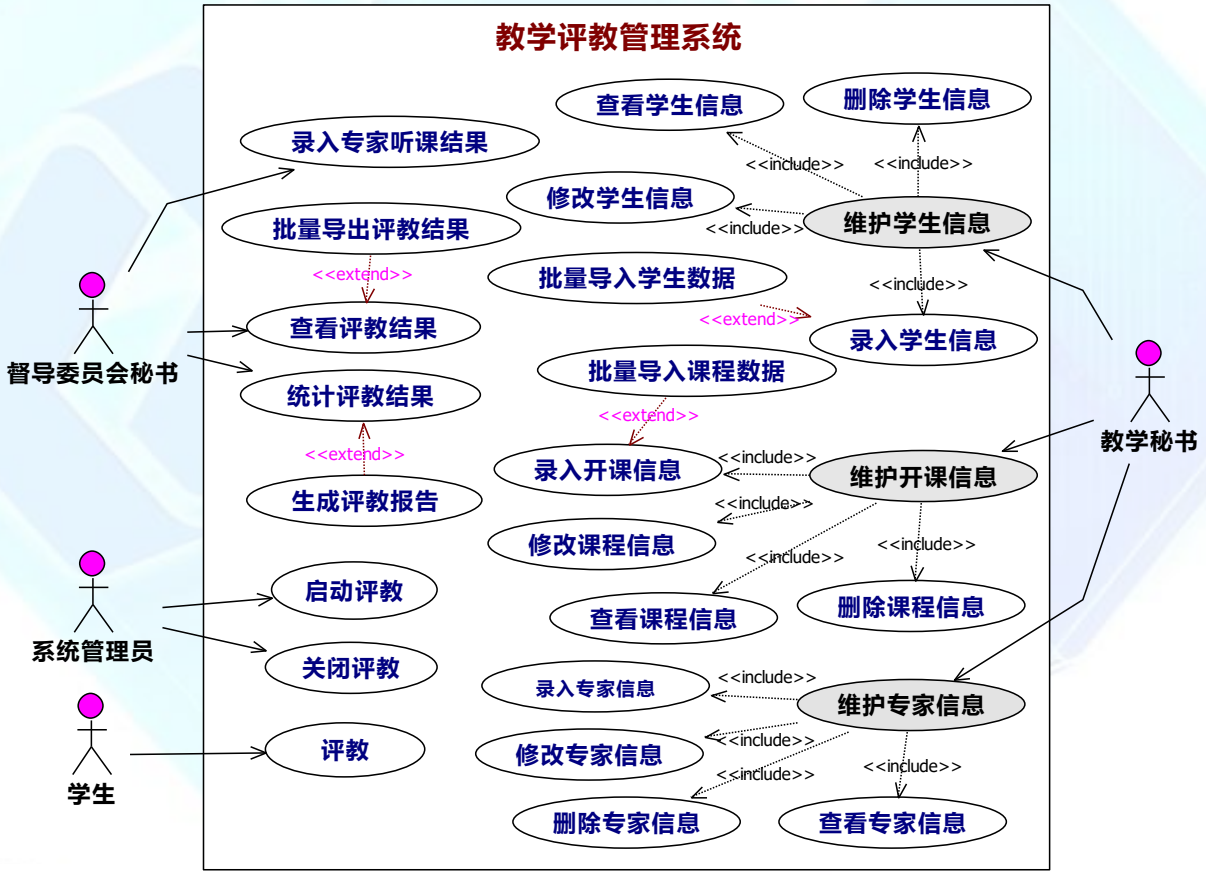
### 系统用例分析 -- 系统用例图（Step1：剪裁概念用例图）

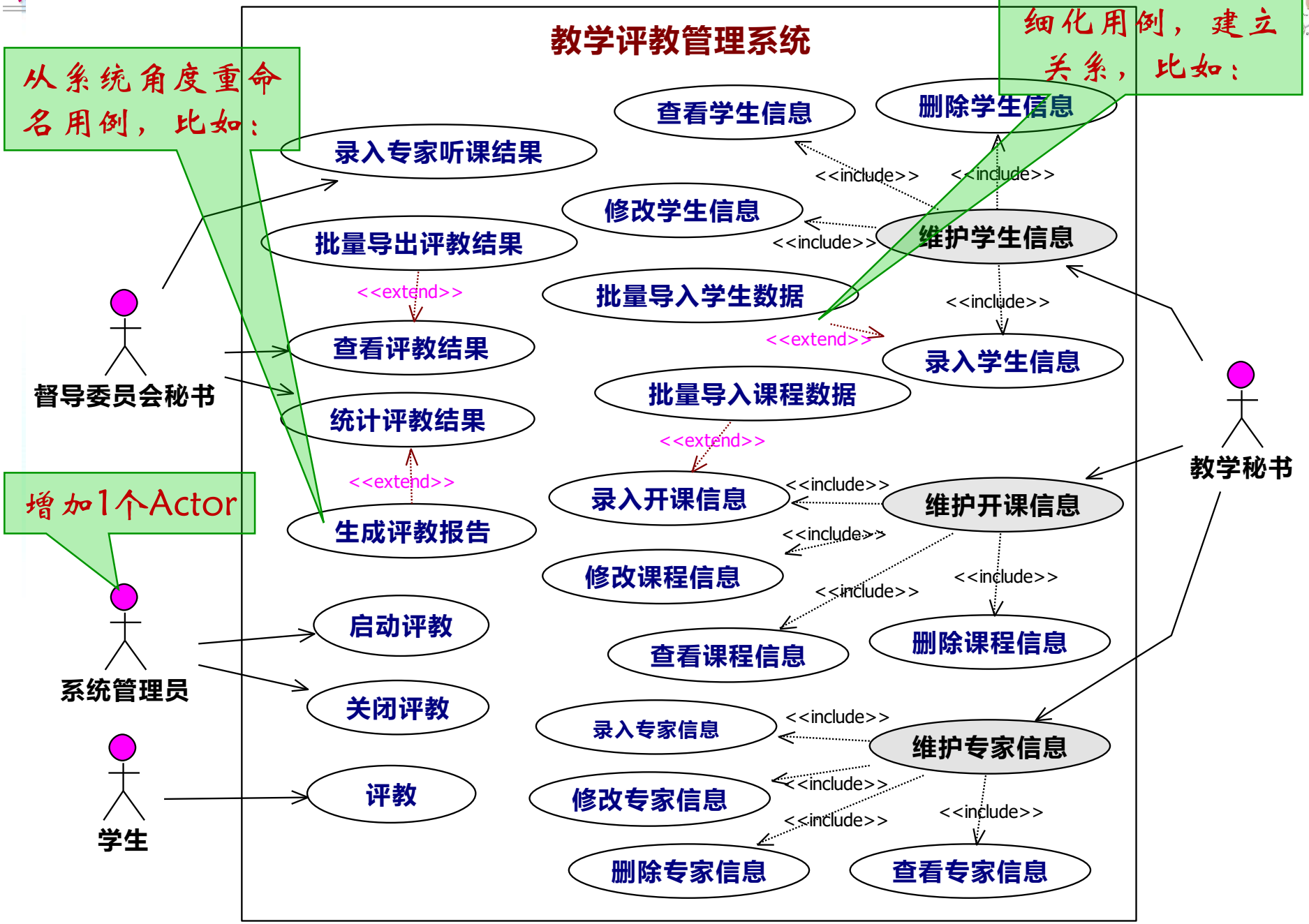


# OOAD案例 -- 评教系统

## 评教系统：系统分析

### 系统用例分析 -- 系统用例图（Step2：从系统角度重构用例图）

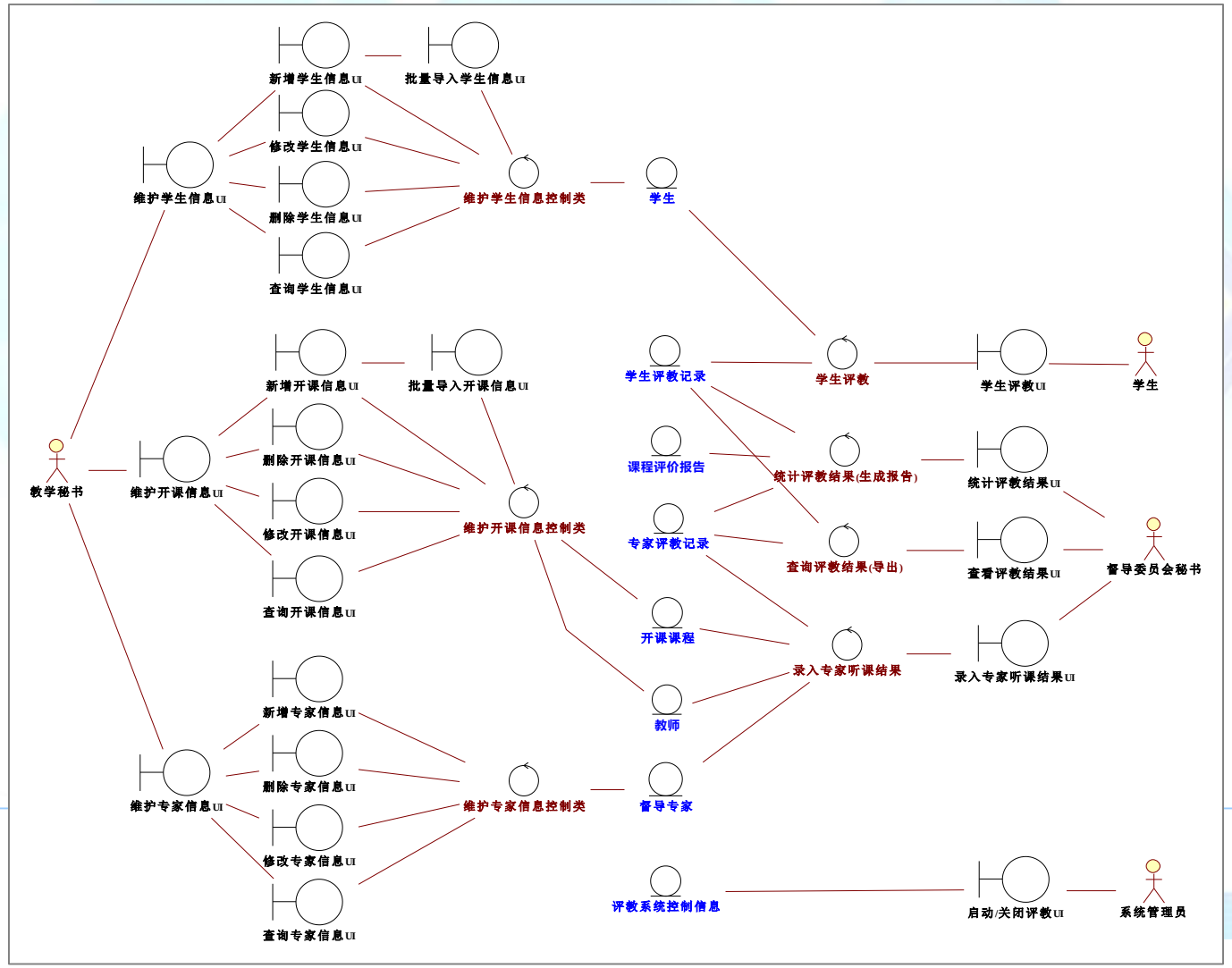



教学评教管理系统
从系统角度重命名用例，比如：
细化用例，建立关系，比如：
增加1个Actor
录入专家听课结果
批量导出评教结果
查看评教结果
统计评教结果
生成评教报告
启动评教
关闭评教
评教
查看学生信息
删除学生信息
修改学生信息
维护学生信息
批量导入学生数据
录入学生信息
批量导入课程数据
录入开课信息
维护开课信息
修改课程信息
查看课程信息
删除课程信息
录入专家信息
维护专家信息
修改专家信息
删除专家信息
查看专家信息
<<include>>
<<extend>>
督导委员会秘书
系统管理员
学生
教学秘书

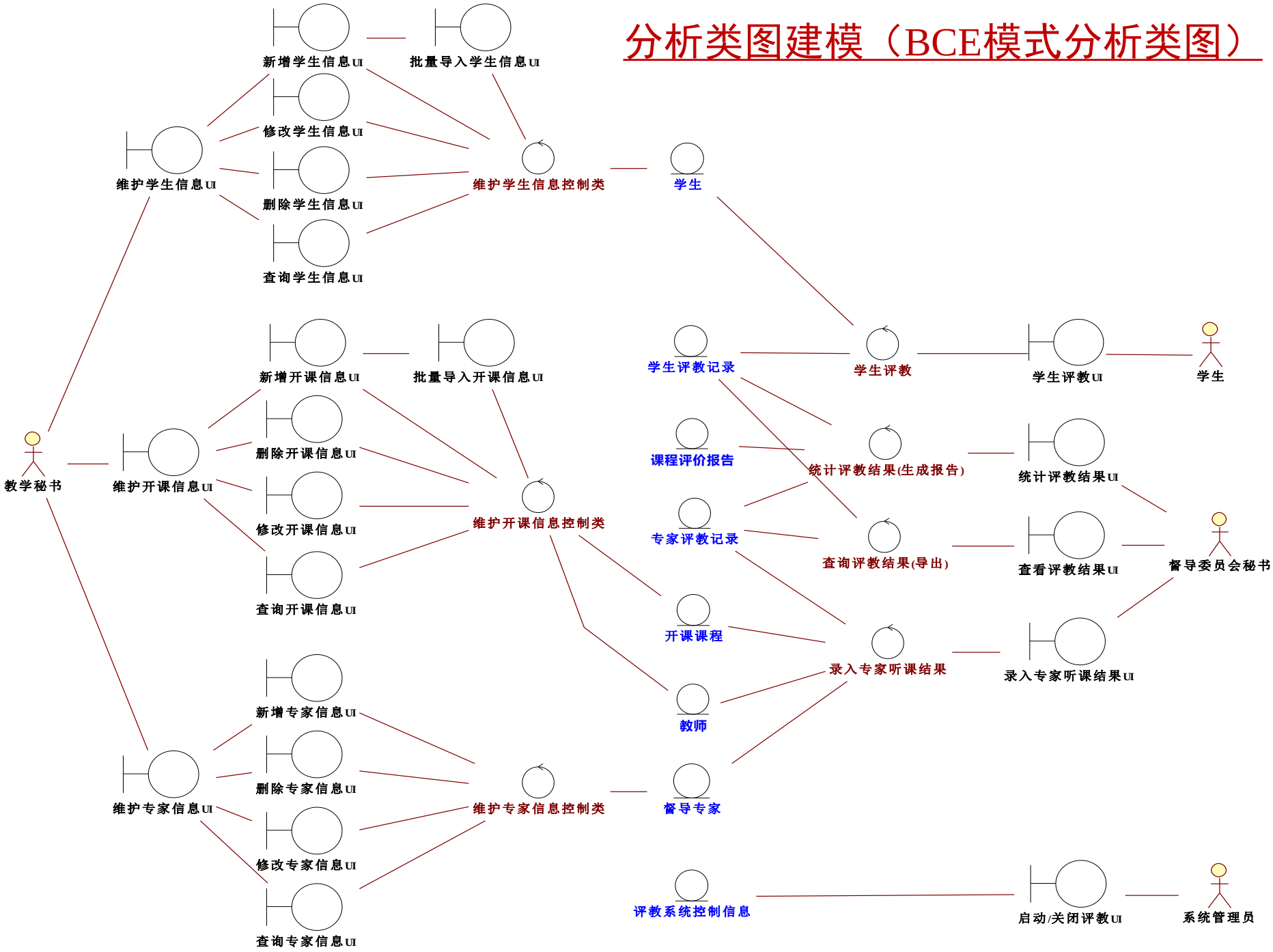
# OOAD案例 -- 评教系统

## 评教系统：系统分析

### 系统分析 -- 分析类建模（BCE模式分析类图）



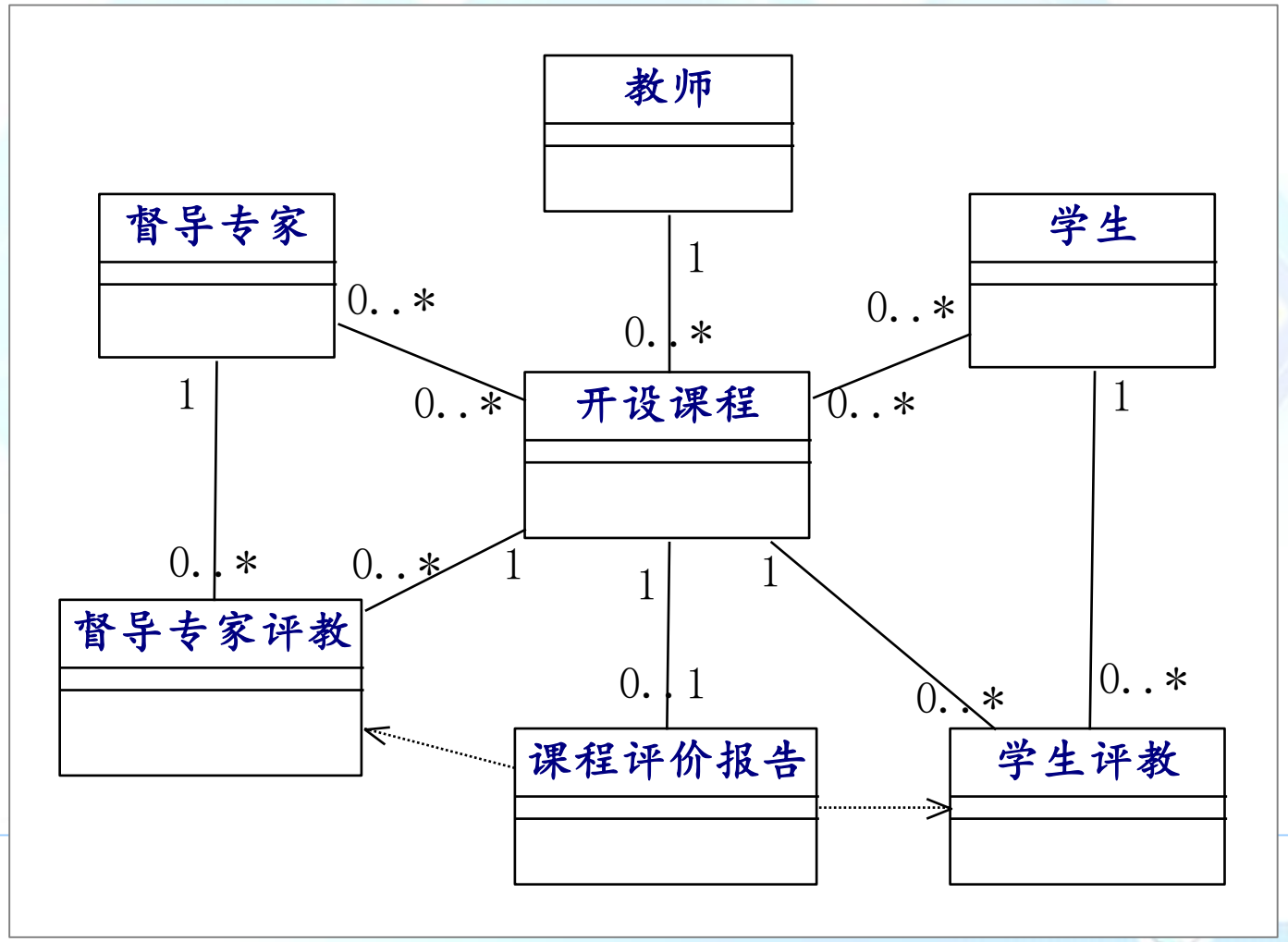
# 分析类图建模（BCE模式分析类图）



# OOAD案例 -- 评教系统

## 评教系统：系统分析

### 系统分析 – 领域类图建模（实体分析类图）



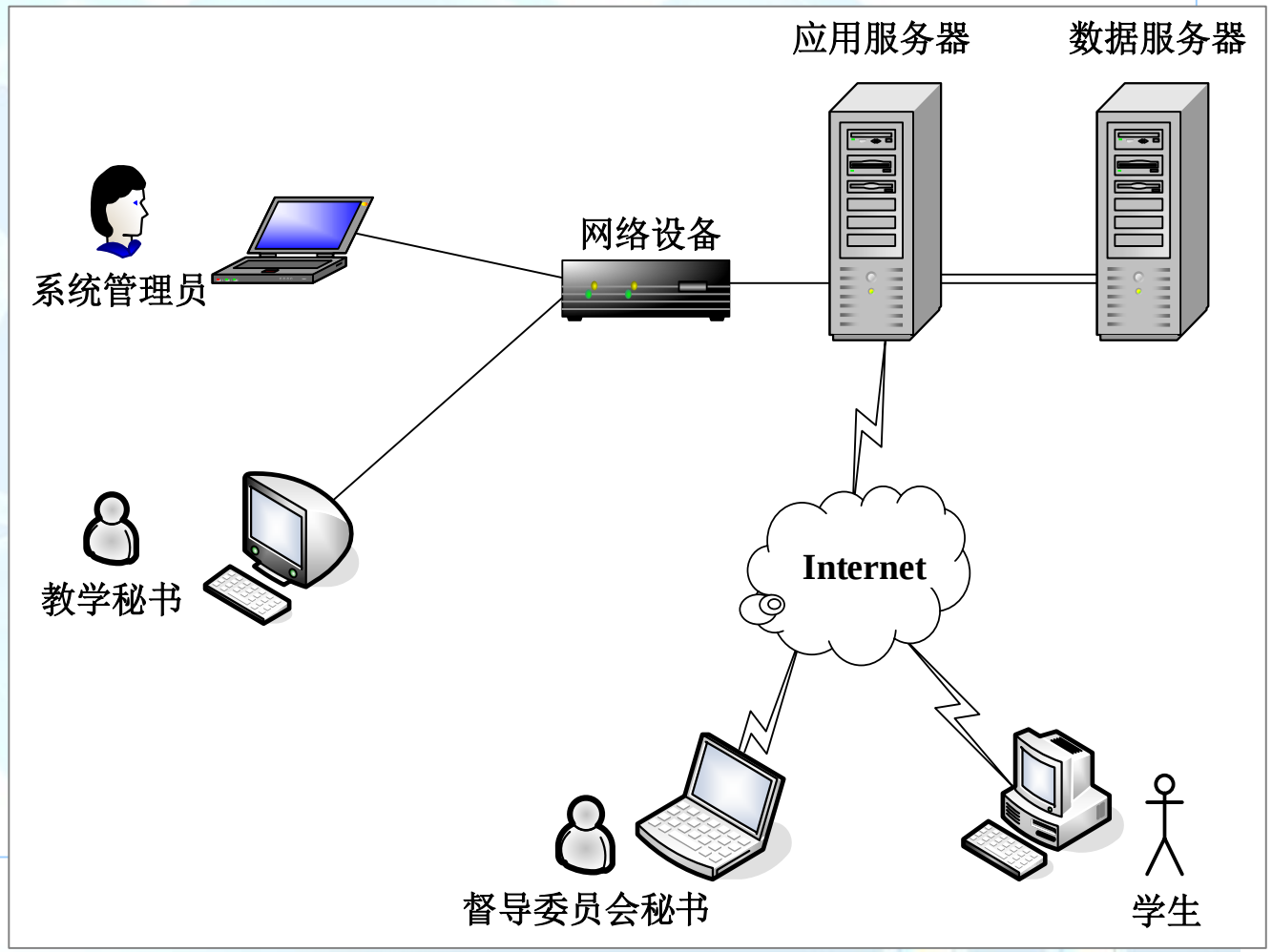
# OOAD案例 -- 评教系统

## 评教系统：系统设计

### 系统拓扑结构设计 -- 网络拓扑图

需求描述：

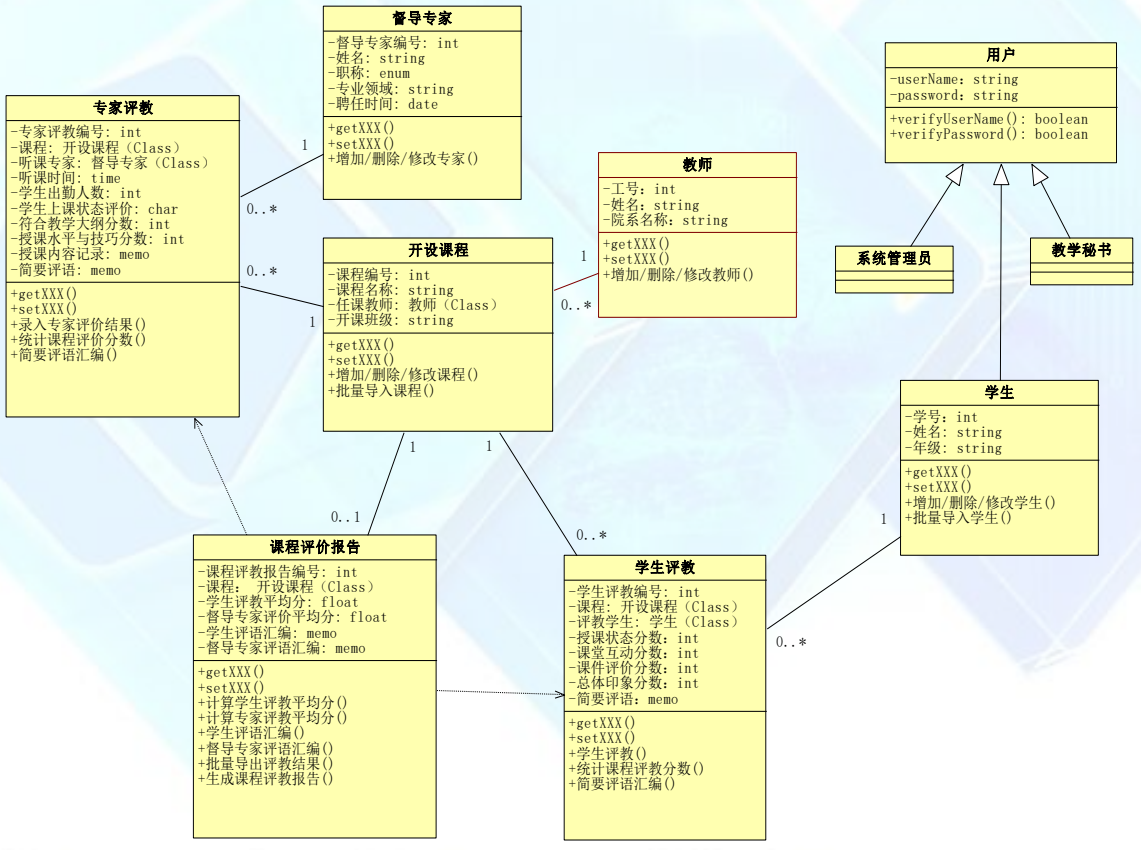
⑥要求该评教系统允许学生、教学督导委员会秘书在任何地点均可以使用，只要能够访问Internet。但系统管理员、教学秘书只能在局域网内使用。

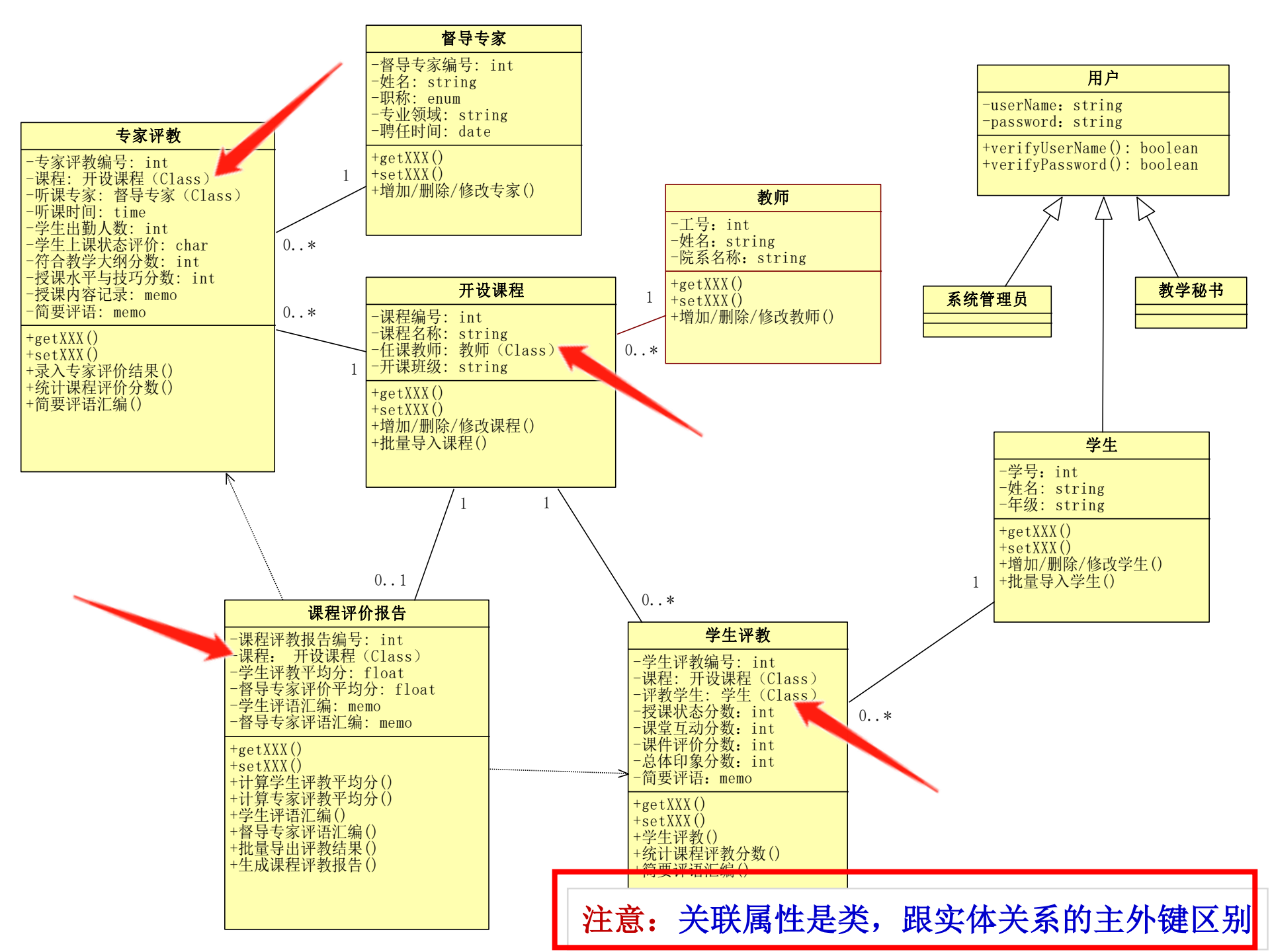


# OOAD案例 -- 评教系统

## 评教系统：系统设计

### 设计类图（此例仅给出实体设计类图）

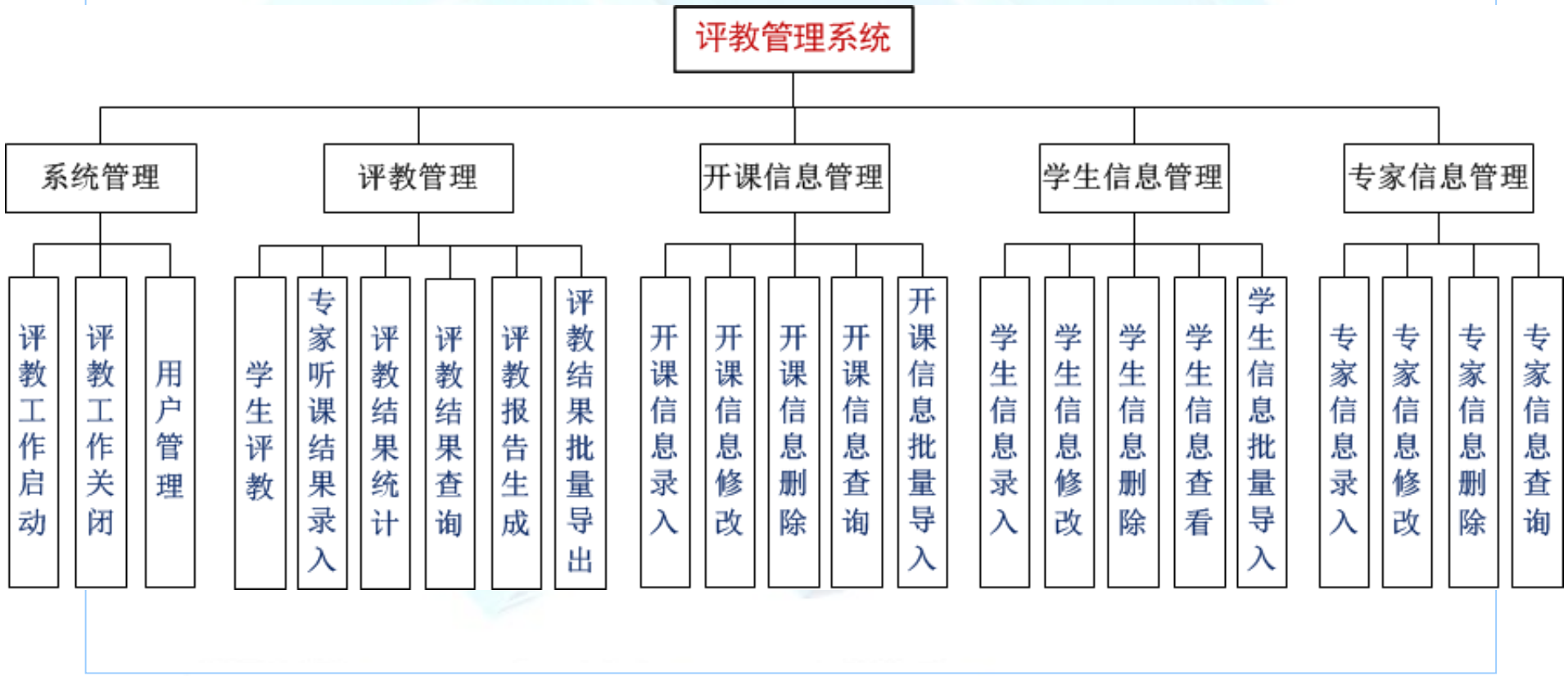



督导专家
-督导专家编号: int
-姓名: string
-职称: enum
-专业领域: string
-聘任时间: date
+getXXX()
+setXXX()
+增加/删除/修改专家()
用户
-userName: string
-password: string
+verifyUserName(): boolean
+verifyPassword(): boolean
专家评教
-专家评教编号: int
-课程: 开设课程 (Class)
-听课专家: 督导专家 (Class)
-听课时间: time
-学生出勤人数: int
-学生上课状态评价: char
-符合教学大纲分数: int
-授课水平与技巧分数: int
-授课内容记录: memo
-简要评语: memo
+getXXX()
+setXXX()
+录入专家评价结果()
+统计课程评价分数()
+简要评语汇编()
教师
-工号: int
-姓名: string
-院系名称: string
+getXXX()
+setXXX()
+增加/删除/修改教师()
系统管理员
教学秘书
开设课程
-课程编号: int
-课程名称: string
-任课教师: 教师 (Class)
-开课班级: string
+getXXX()
+setXXX()
+增加/删除/修改课程()
+批量导入课程()
学生
-学号: int
-姓名: string
-年级: string
+getXXX()
+setXXX()
+增加/删除/修改学生()
+批量导入学生()
课程评价报告
-课程评教报告编号: int
-课程: 开设课程 (Class)
-学生评教平均分: float
-督导专家评价平均分: float
-学生评语汇编: memo
-督导专家评语汇编: memo
+getXXX()
+setXXX()
+计算学生评教平均分()
+计算专家评教平均分()
+学生评语汇编()
+督导专家评语汇编()
+批量导出评教结果()
+生成课程评教报告()
学生评教
-学生评教编号: int
-课程: 开设课程 (Class)
-评教学生: 学生 (Class)
-授课状态分数: int
-课堂互动分数: int
-课件评价分数: int
-总体印象分数: int
-简要评语: memo
+getXXX()
+setXXX()
+学生评教()
+统计课程评教分数()
1
0..*
0..1
注意：关联属性是类，跟实体关系的主外键区别

# OOAD案例 -- 评教系统

## 评教系统：系统设计

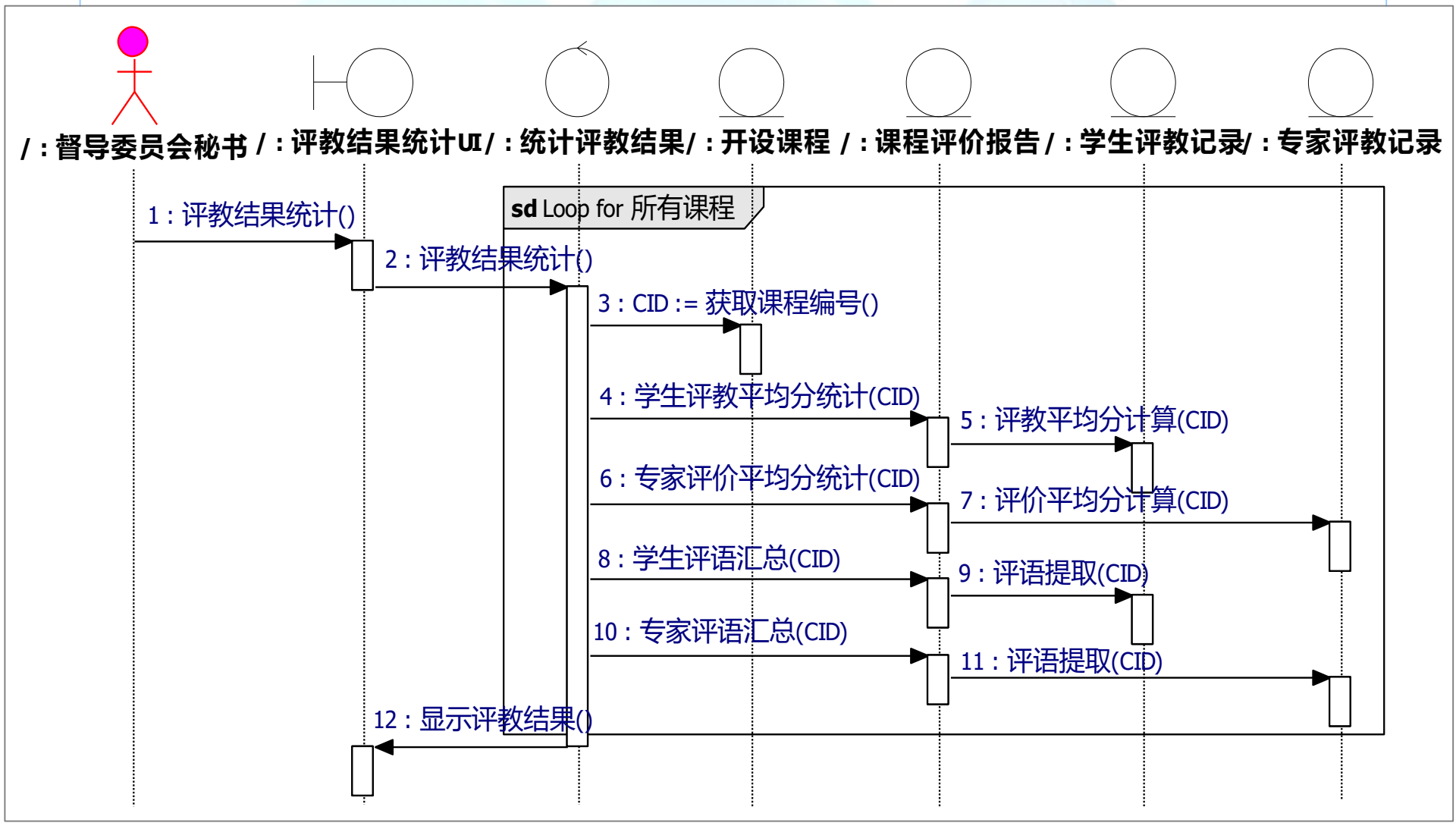
### 系统功能结构设计：评教管理系统功能结构图



# OOAD案例 -- 评教系统

## 评教系统：系统设计

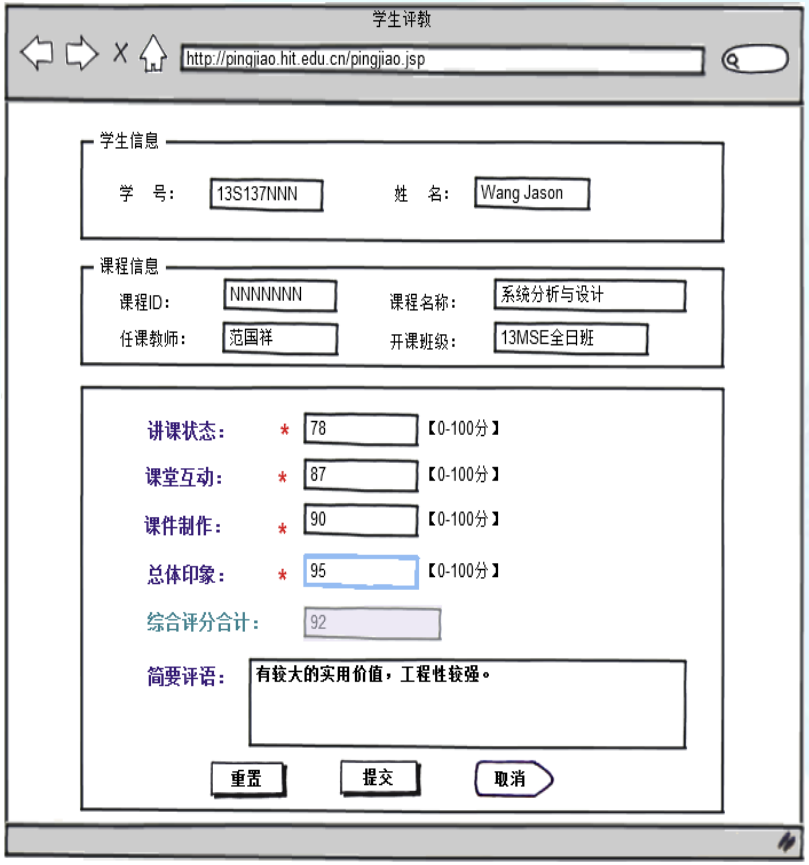
### 系统功能设计 – 具体功能详细设计（评教结果统计功能时序图）



# OOAD案例 -- 评教系统

## 评教系统：系统设计

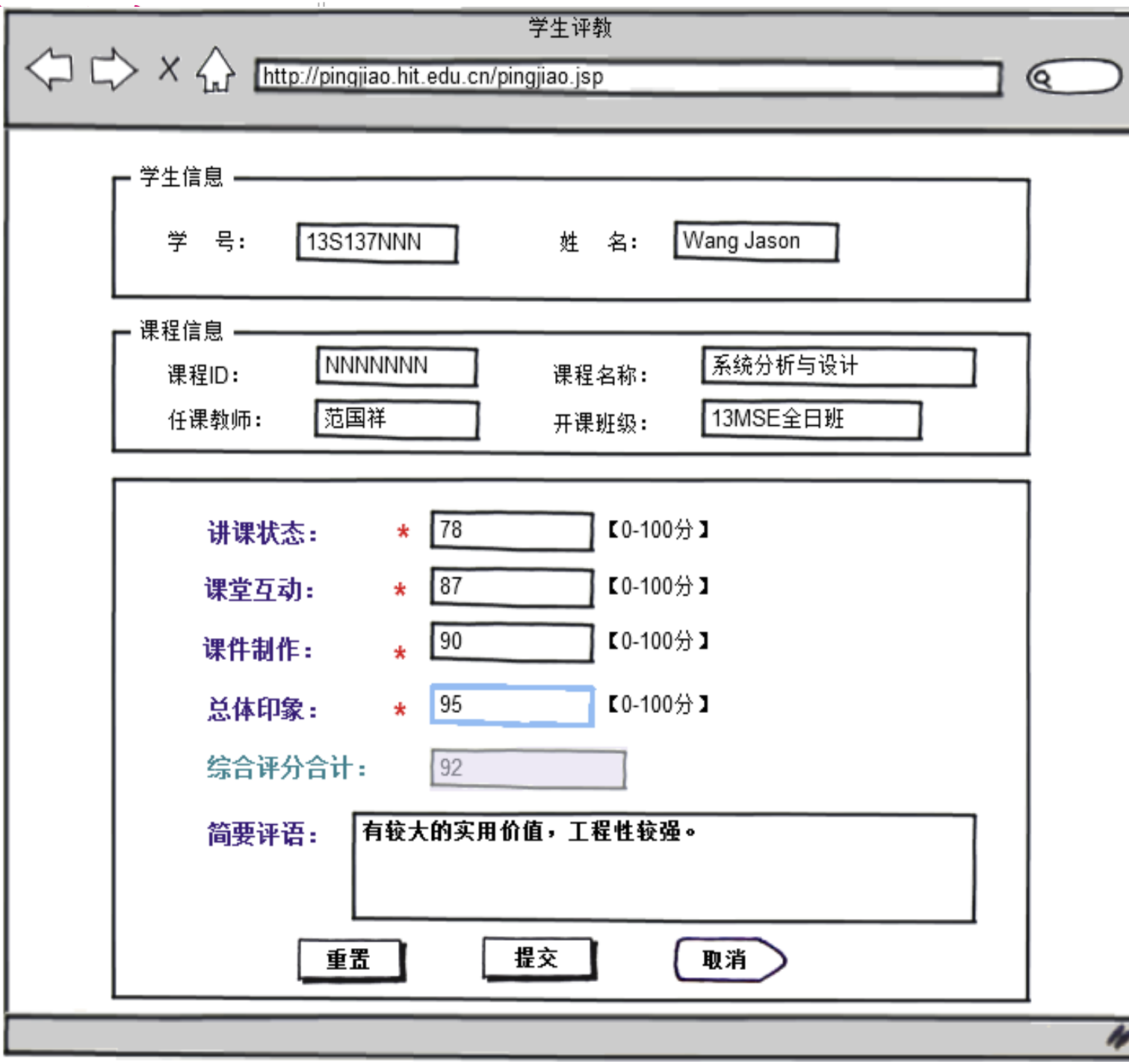
### 用户界面设计（学生评教界面、课程评价报告）




课程授课情况评价报告

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 课程名称 | xxxxxxxxxxxxxxx | | 任课教师 | xxx | 开课班级 | xxxx 级 | |
| 督导专家评价结果 | | | | | | | |
| 听课次数 | xx | 平均出勤人数 | xxx | 学生听课状态评价 | 优秀 xx 次； 良好 xx 次；中等 xx 次；一般 xx 次；较差 xx 次 | | |
| 符合教学大纲平均分数 | xxx. xx | | 授课水平和技巧平均分数 | xxx. xx | | | |
| 专家评语汇编 | 专家 1 | xxxxxxxxxxxxxxxxxxxxxxxxxxxx | | | | | |
| | 专家 2 | xxxxxxxxxxxxxxxxxxxxxxxxxxxx | | | | | |
| | … … | | | | | | |
| | 专家 M | xxxxxxxxxxxxxxxxxxxxxxxxxxxx | | | | | |
| 学生评教结果 | | | | | | | |
| 选课人数 | xx | | 参与评教人数 | xxx | | | |
| 授课状态平均分数 | xxx. xx | 课堂互动平均分数 | xxx. xx | 课件评价平均分数 | xxx. xx | 总体印象平均分数 | xxx. xx |
| 学生评语汇编 | 学生 1 | xxxxxxxxxxxxxxxxxxxxxxxxxxxx | | | | | |
| | 学生 2 | xxxxxxxxxxxxxxxxxxxxxxxxxxxx | | | | | |
| | … … | | | | | | |
| | 学生 N | xxxxxxxxxxxxxxxxxxxxxxxxxxxx | | | | | |
| 备注 | | | | | | | |

# 评教


学生评教
http://pingjiao.hit.edu.cn/pingjiao.jsp
学生信息
学　号：
13S137NNN
姓　名：
Wang Jason
课程信息
课程ID：
NNNNNNN
课程名称：
系统分析与设计
任课教师：
范国祥
开课班级：
13MSE全日班
讲课状态：
*
78
【0-100分】
课堂互动：
*
87
【0-100分】
课件制作：
*
90
【0-100分】
总体印象：
*
95
【0-100分】
综合评分合计：
92
简要评语：
有较大的实用价值，工程性较强。
重置
提交
取消

评教系

用户

## 课程授课情况评价报告

| 课程名称 | XXXXXXXXXXXXXXX | | 任课教师 | XXX | 开课班级 | XXXX 级 |
|---|---|---|---|---|---|---|
| 督导专家评价结果 | | | | | | |
| 听课次数 | XX | 平均出勤人数 | XXX | 学生听课状态评价 | 优秀 XX 次； 良好 XX 次；中等 XX 次；一般 XX 次；较差 XX 次 | |
| 符合教学大纲平均分数 | XXX. XX | | 授课水平和技巧平均分数 | XXX. XX | | |
| 专家评语汇编 | 专家 1 | XXXXXXXXXXXXXXXXXXXXXXXXXXXX | | | | |
| | 专家 2 | XXXXXXXXXXXXXXXXXXXXXXXXXXXX | | | | |
| | … … | | | | | |
| | 专家 M | XXXXXXXXXXXXXXXXXXXXXXXXXXXX | | | | |
| 学生评教结果 | | | | | | |
| 选课人数 | XX | | 参与评教人数 | XXX | | |
| 授课状态平均分数 | XXX. XX | 课堂互动平均分数 | XXX. XX | 课件评价平均分数 | XXX. XX | 总体印象平均分数 | XXX. XX |
| 学生评语汇编 | 学生 1 | XXXXXXXXXXXXXXXXXXXXXXXXXXXX | | | | |
| | 学生 2 | XXXXXXXXXXXXXXXXXXXXXXXXXXXX | | | | |
| | … … | | | | | |
| | 学生 N | XXXXXXXXXXXXXXXXXXXXXXXXXXXX | | | | |
| 备注 | | | | | | |

# 《需求分析与系统设计》
# Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(Mobile)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部

国家示范性软件学院

软件工程教研室

2023. 10

# 面向对象系统设计

## 本章主要内容

1. 面向对象设计概述
2. 系统设计
3. 包的设计
4. 对象设计
5. 面向对象设计总结

# 面向对象系统设计

## 本章主要内容

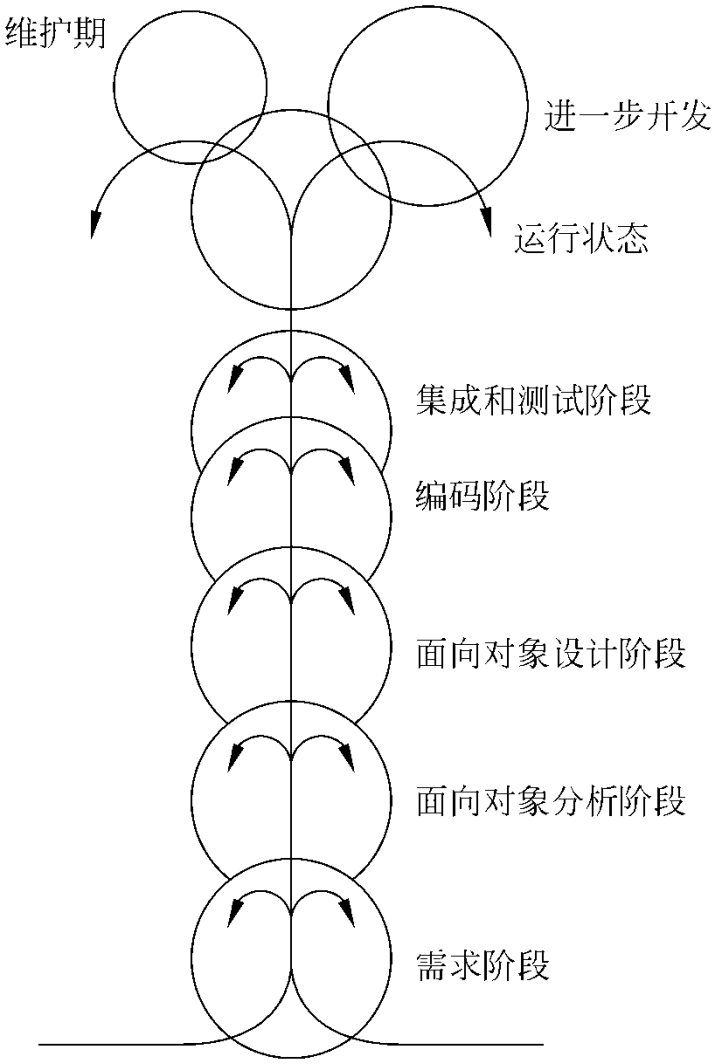
1. 面向对象设计概述
2. 系统设计
3. 包的设计
4. 对象设计
5. 面向对象设计总结

# 面向对象的设计

- 传统的结构化方法：
  分析阶段与设计阶段分得特别清楚，分别使用两套完全不同的建模符号和建模方法

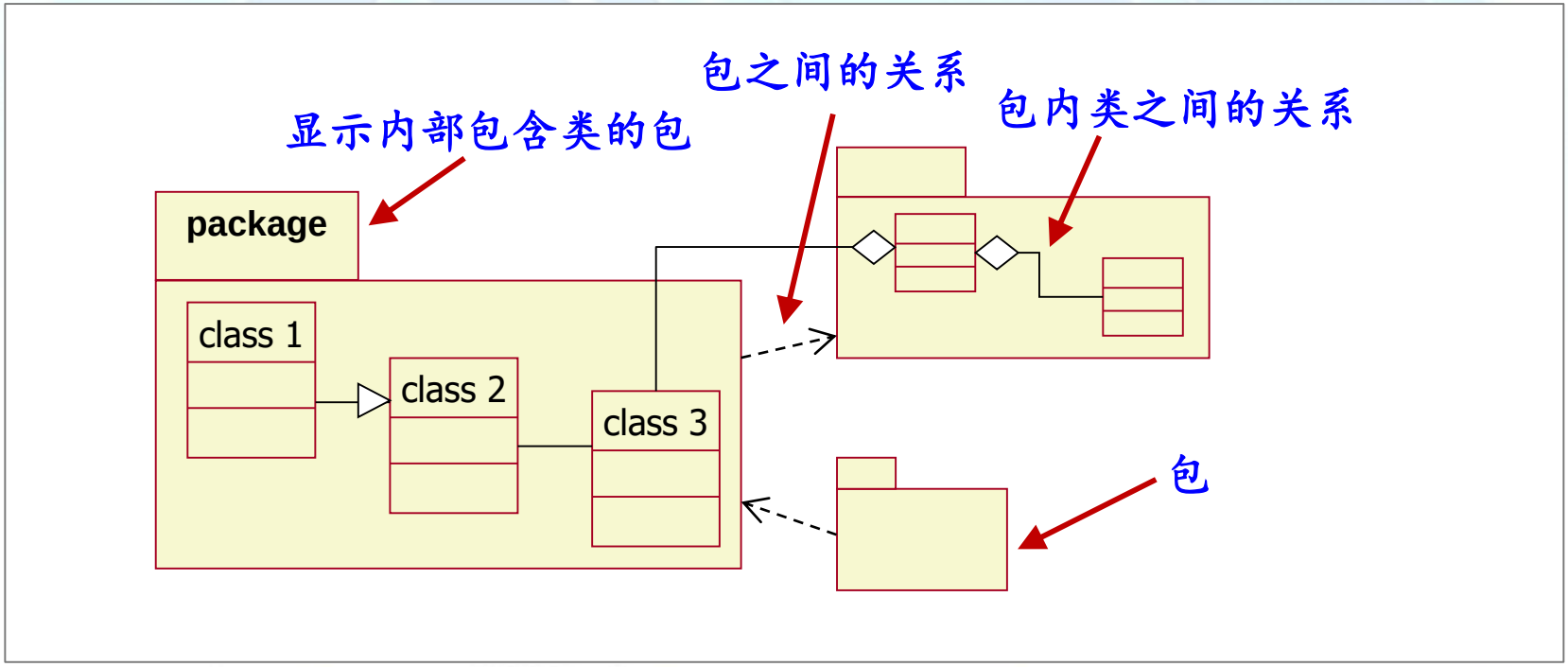
- 面向对象的设计(OOD)：
  OO各阶段均采用统一的“对象”概念，各阶段之间的区分变得不明显，形成“无缝”连接

- 因此，OOD中仍然使用“类、属性、操作”等概念，是在OOA基础上的进一步细化，更加接近底层的技术实现



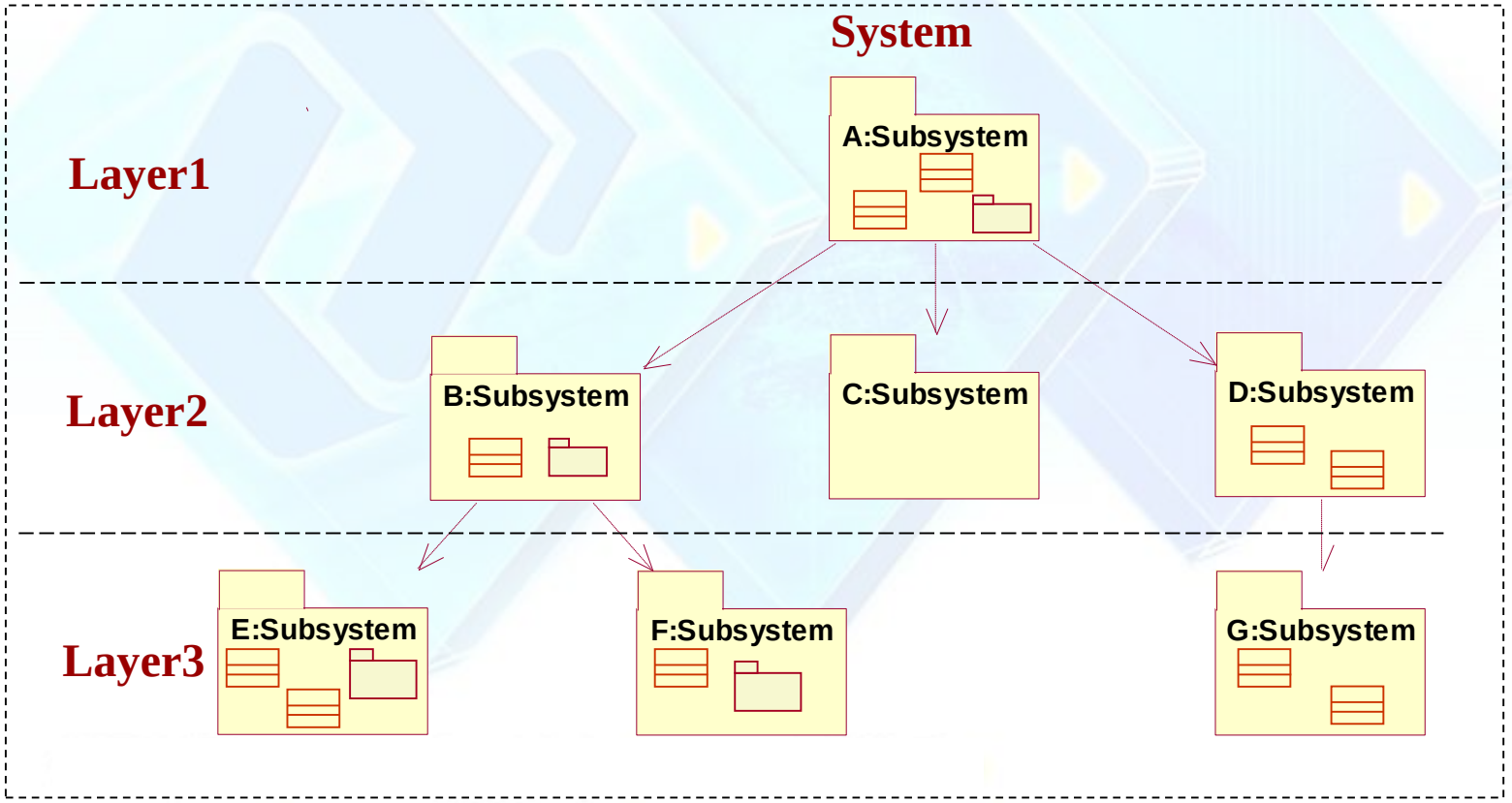
# 面向对象设计中的基本元素

- 基本单元：设计类(design class)
  - 对应于OOA中的分析类(analysis class)
- 为了系统实现与维护过程中的方便性，将多个设计类按照彼此关联的紧密程度聚合到一起，形成大粒度的包(package)



# 面向对象设计中的基本元素

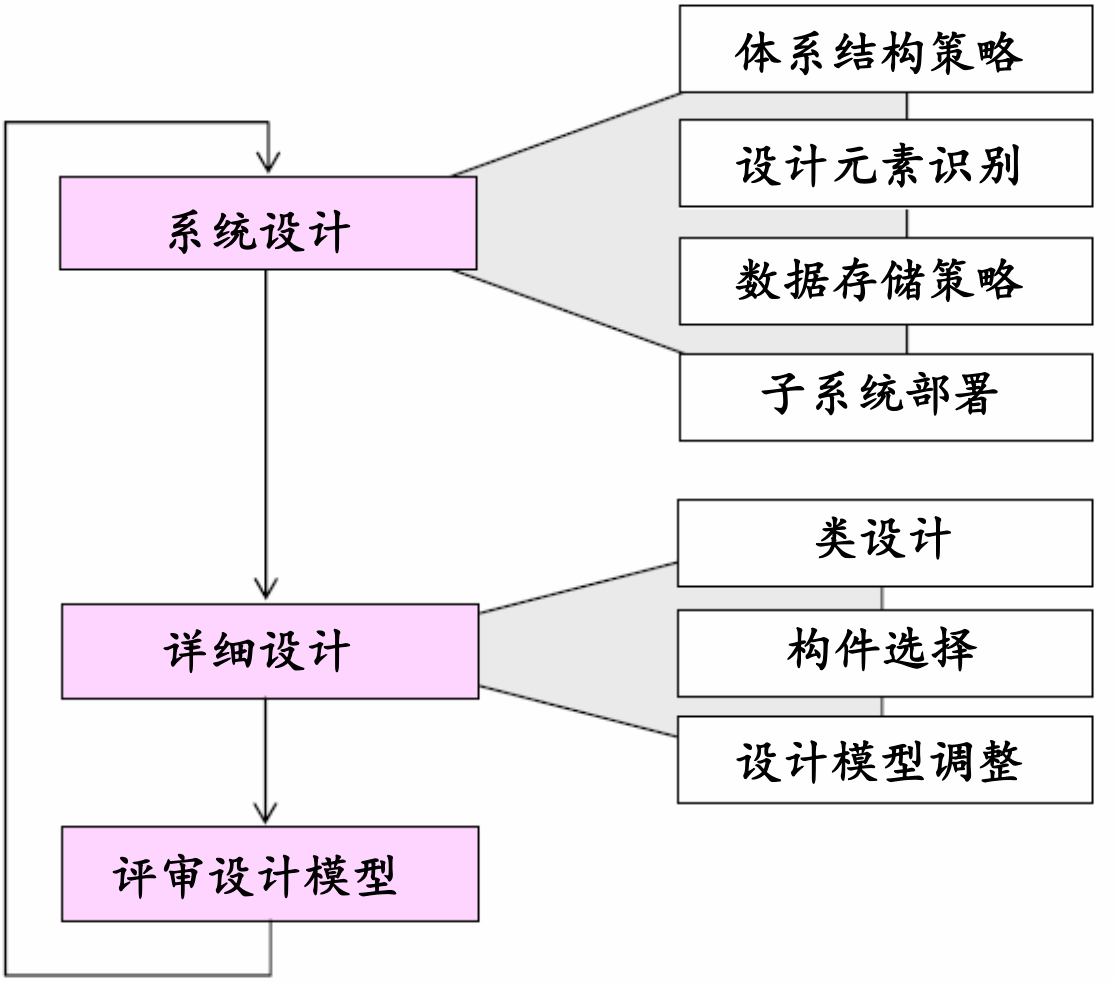
- 一个或多个包聚集在一起，形成子系统(sub-system)
- 多个子系统，构成完整的系统(system)



# 面向对象的设计的两个阶段

- 系统设计(System Design)
  - 相当于概要设计（即设计系统的体系结构）
  - 选择解决问题的基本途径（框架、平台、工具…）
  - 决策整个系统的结构与风格(拓扑结构、软件体系结构…)
- 对象设计(Object Design)
  - 相当于详细设计（即设计对象内部的具体实现）
  - 细化需求分析模型
  - 识别新的对象
  - 在系统所需的应用对象与可复用的商业构件之间建立关联：
    - 识别系统中的应用对象
    - 调整已有的构件
    - 给出每个子系统/类的精确规格说明

# 面向对象设计的过程


体系结构策略
设计元素识别
数据存储策略
子系统部署
系统设计
类设计
详细设计
构件选择
设计模型调整
评审设计模型

# 本章主要内容

1. 面向对象设计概述
2. 系统设计
3. 包的设计
4. 对象设计
5. 面向对象设计总结

# 系统设计概述

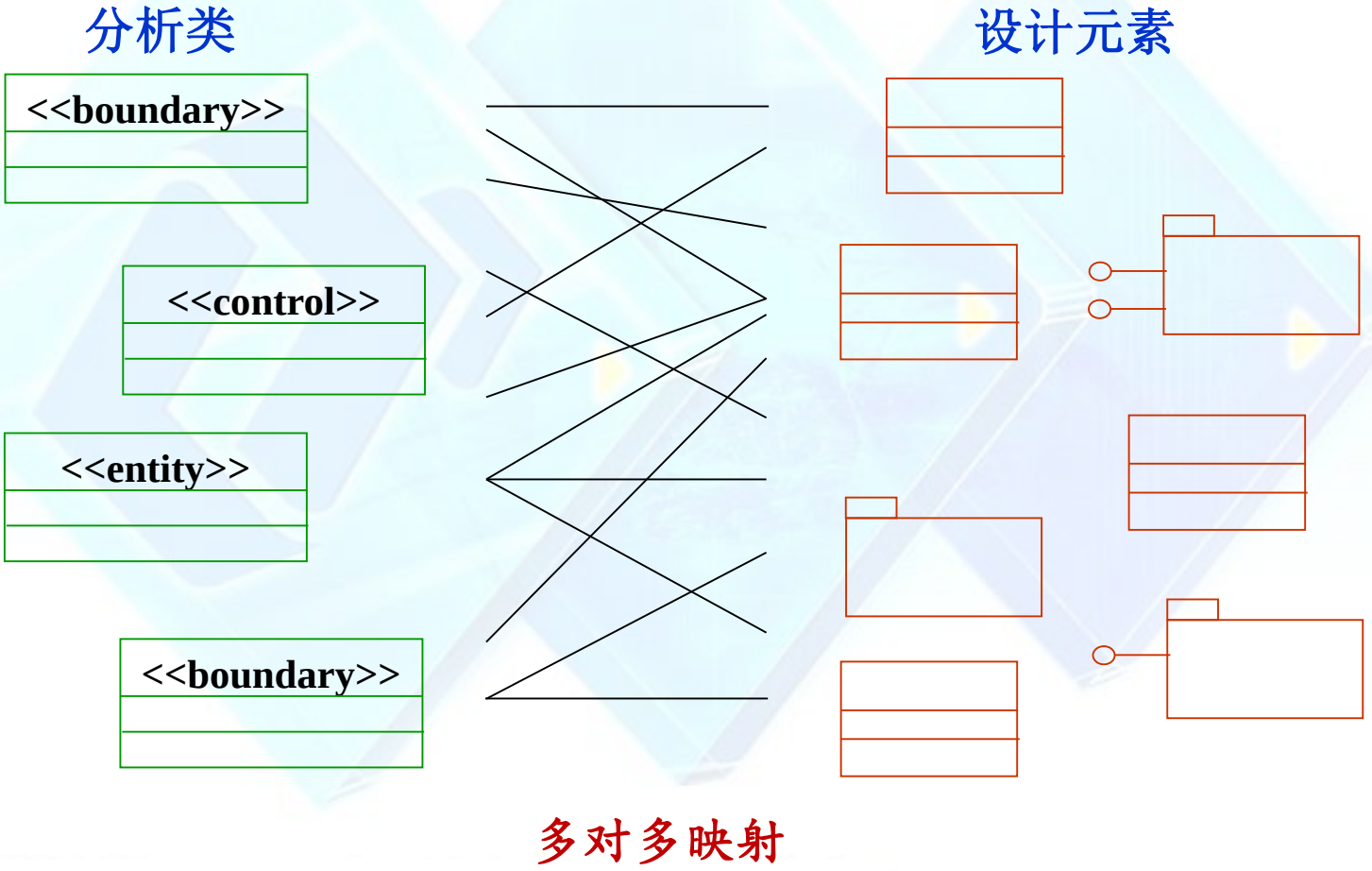
- 设计系统的体系结构 -- 在课程《软件体系结构》重点讲述
  - 选择合适的分层体系结构策略，建立系统的总体结构：分几层？每层的功能分别是什么？
- 识别设计元素 --详见后续章节
  - 识别“设计类”(design class)、“包”(package)、“子系统”(sub-system)
- 部署子系统 -- 在课程《软件体系结构》重点讲述
  - 选择硬件配置和系统平台，将子系统分配到相应的物理节点，绘制部署图(deployment diagram)
- 定义数据的存储策略 --详见后续章节
- 检查系统设计

# 本章主要内容

1. 面向对象设计概述
2. 系统设计
3. 包的设计
4. 对象设计
5. 面向对象设计总结

# 面向对象系统设计

## 识别设计元素



# 确定设计元素的基本原则

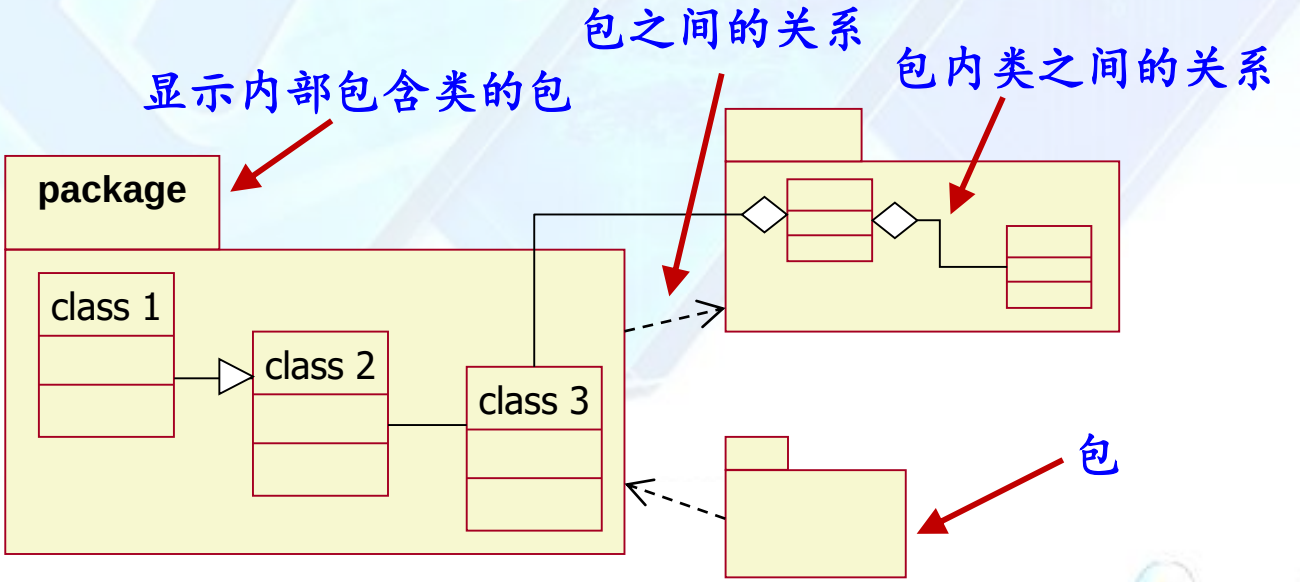
- 如果一个“分析类”比较简单，代表着单一的逻辑抽象，那么可以将其一对一的映射为“设计类”
  - 通常，主动参与者对应的边界类、控制类和一般的实体类都可以直接映射成设计类
- 如果“分析类”的职责比较复杂，很难由单个“设计类”承担，则应该将其分解为多个“设计类”，并映射成“包”或“子系统”
- 将设计类分配到相应的“包”或“子系统”当中
  - 子系统的划分应该符合高内聚低耦合的原则

# 图书管理系统：识别设计元素

| 类型 | 分析类 | 设计元素 |
|---|---|---|
| | ***LoginForm*** | “设计类” ***LoginForm*** |
| | ***BrowseForm*** | “设计类” ***BrowseForm*** |
| | …… | …… |
| | ***MailSystem*** | “子系统接口” ***IMailSystem*** |
| | ***BrowseControl*** | “设计类” ***BrowseControl*** |
| | ***MakeReservationControl*** | “设计类” ***MakeReservationControl*** |
| | …… | …… |
| | ***BorrowerInfo*** | “设计类” ***BorrowerInfo*** |
| | ***Loan*** | “设计类” ***Loan*** |
| | …… | …… |

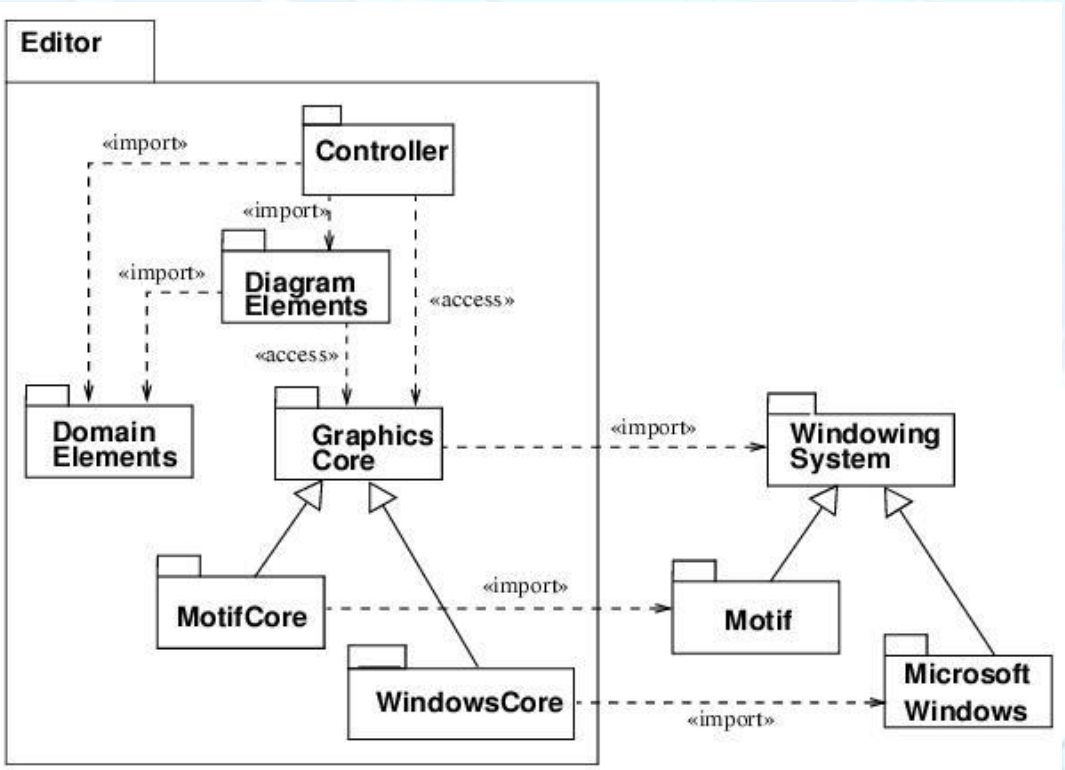
# 绘制包图(package diagram)

- 对一个复杂的软件系统，要使用大量的设计类，这时就必要把这些类分组进行组织
- 把在语义上接近且倾向于一起变化的类组织在一起形成“包”，既可控制模型的复杂度，有助于理解，而且也有助于按组来控制类的可见性
- 结构良好的包是松耦合、高内聚的，而且对其内容的访问具有严密的控制



# 包之间的关系

- 类与类之间存在的“聚合、组合、关联、依赖”关系导致包与包之间存在依赖关系，即包的依赖(dependency)
- 类与类之间的存在的“继承”关系导致包与包之间存在继承关系，即包的泛化(generalization)



# 模型管理视图—包图

- 包图(Package Diagram)是在 UML 中用类似于文件夹的符号表示的模型元素的组合
- UML包图提供了组织元素的方式，包能够组织任何事物：
  - 类、其它包、用例等
  - 系统中的每个元素都只能为一个包所有
  - 一个包可嵌套在另一个包中

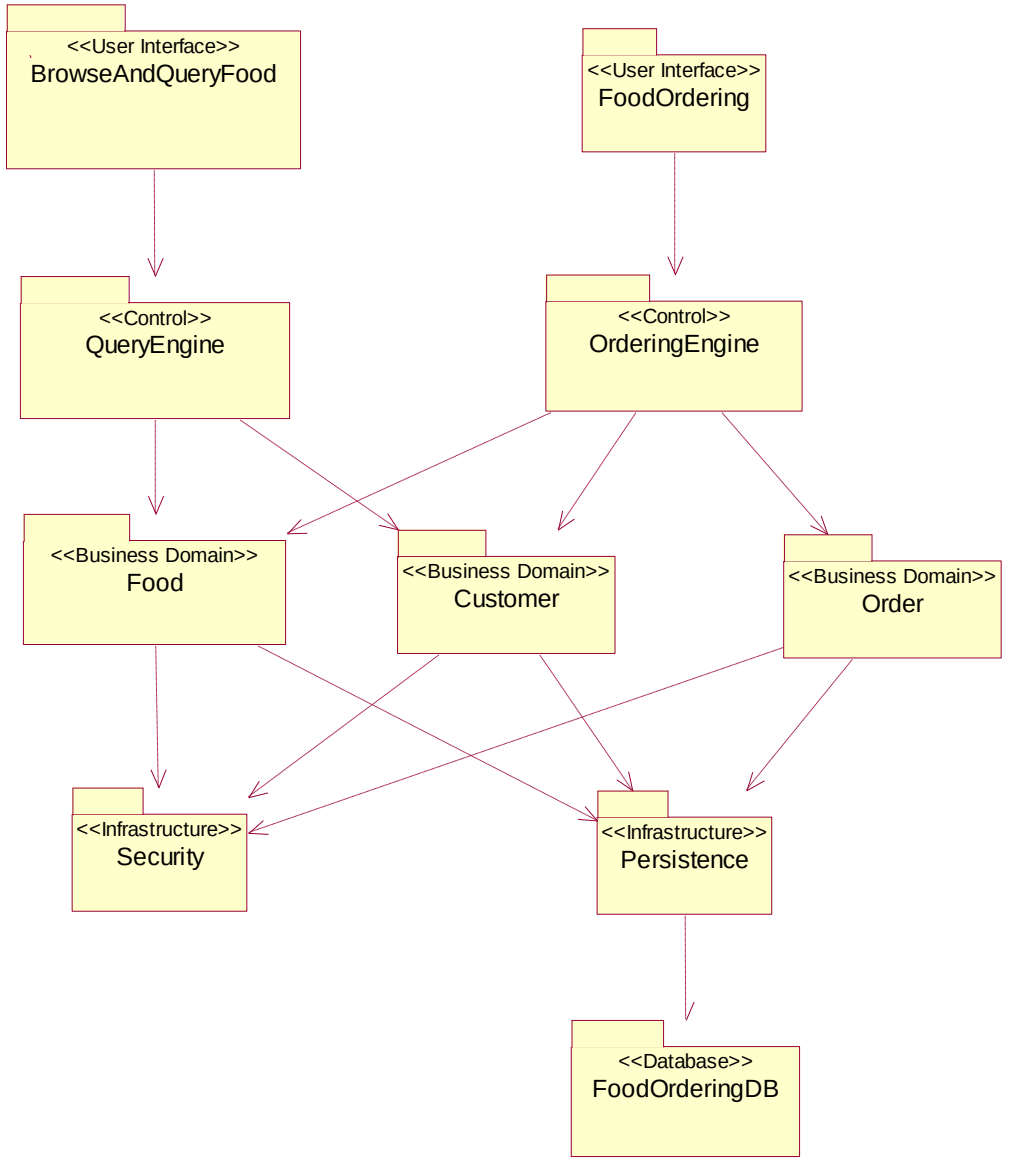
  注：UML中的包为广义概念，不等同于Java包的狭义概念
  - 类、接口、组件、节点、协作、用例、其他包
- 使用包图可以将相关元素归入一个系统
- 一个包中可包含附属包、图表或单个元素

# 绘制包图的方法

- 分析设计类，把概念上或语义上相近的模型元素纳入一个包
- 可以从类的功能相关性来确定纳入包中的类：
  - 如果一个类的行为和/或结构的变更要求另一个相应的变更，则这两个类是功能相关的
  - 如果删除一个类后，另一个类就变成是多余的，则这两个类是功能相关的，这说明该剩余的类只为那个被删除的类所使用，它们之间有依赖关系
  - 如果两个类之间大量的频繁交互或通信，则这两个类是功能相关的
  - 如果两个类之间有一般/特殊关系，则这两个类是功能相关的
  - 如果一个类激发创建另一个类的对象，则这两个类是功能相关的
- 确定包与包之间的依赖关系(<<import>>、<<access>>等)
- 确定包与包之间的泛化关系
- 绘制包图

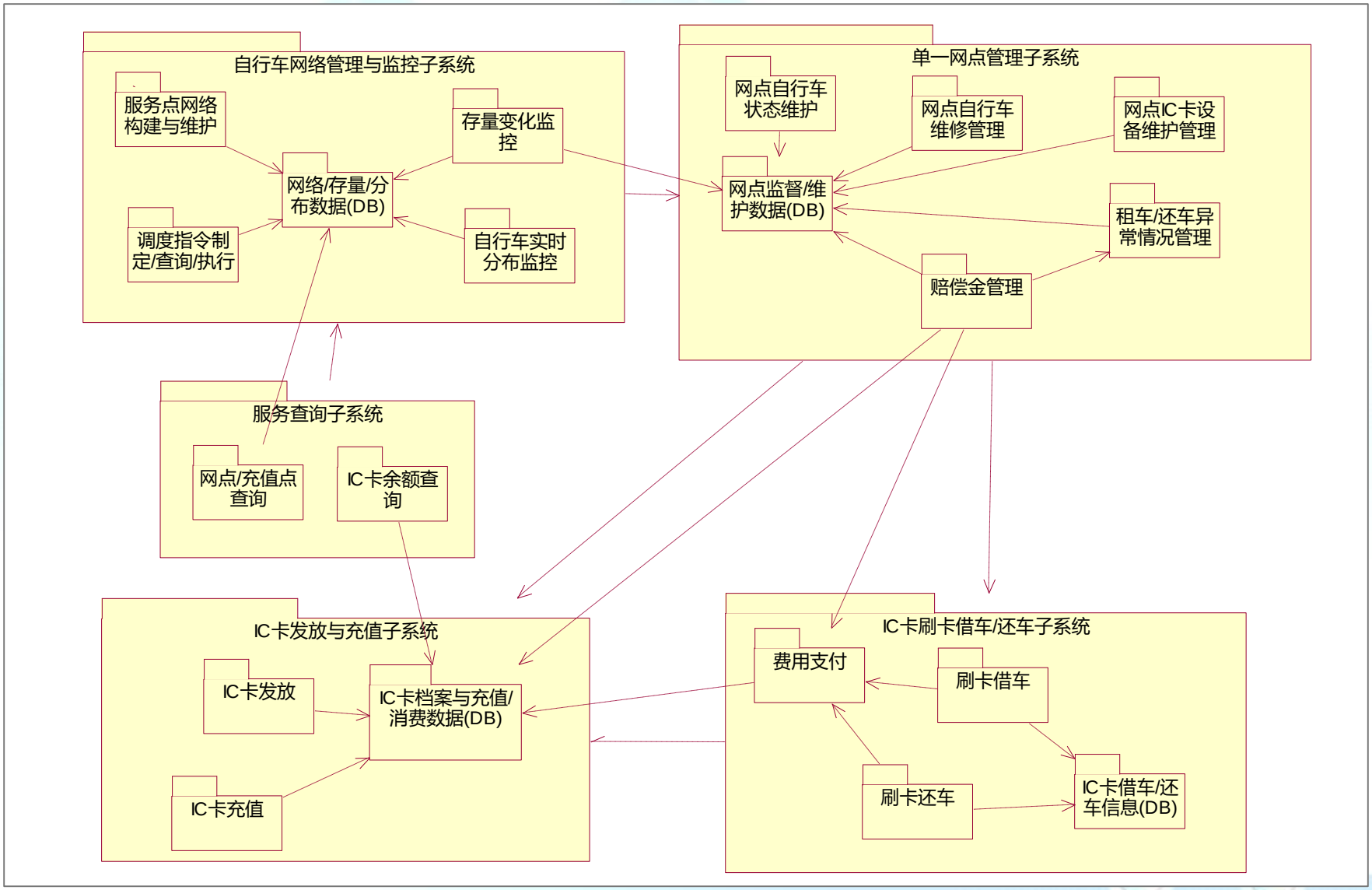
# 面向对象系统设计

## 包图示例1

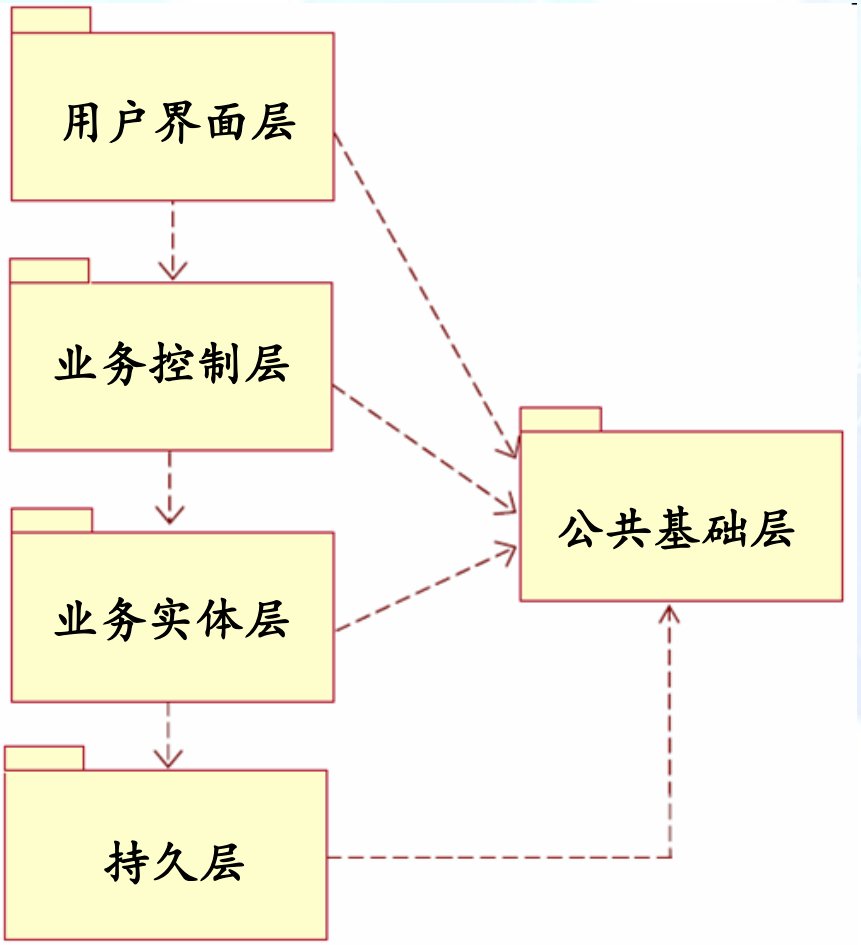


# 面向对象系统设计

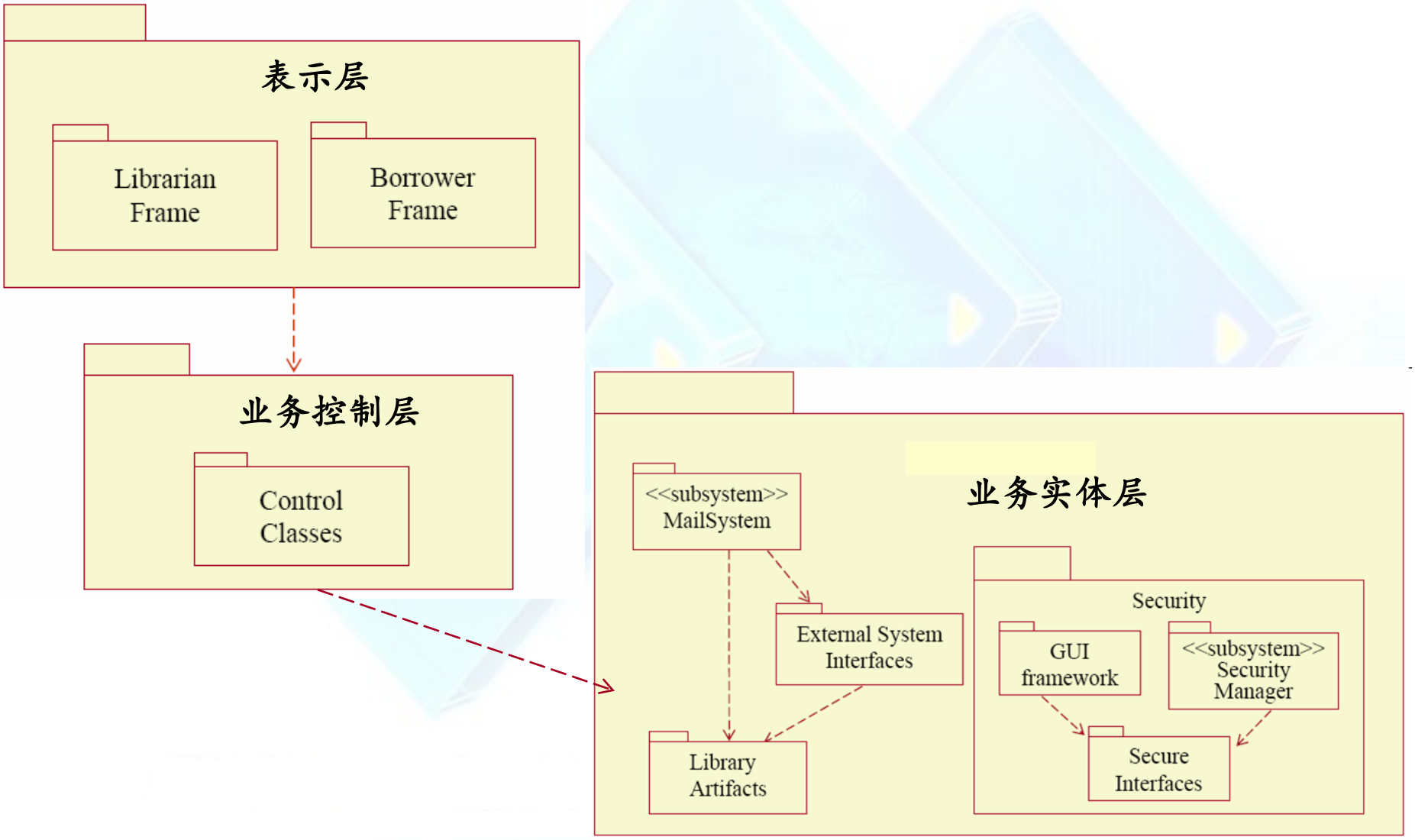
## 包图示例2



# 按实现技术划分包的分层结构



# 图书管理系统：软件体系结构



# JAVA中的“package”

```
java.io.InputStream is = java.lang.System.in;
java.io.InputStreamReader isr= new java.io.InputStreamReader(is);
java.io.BufferedReader br = new java.io.BufferedReader(isr);
```

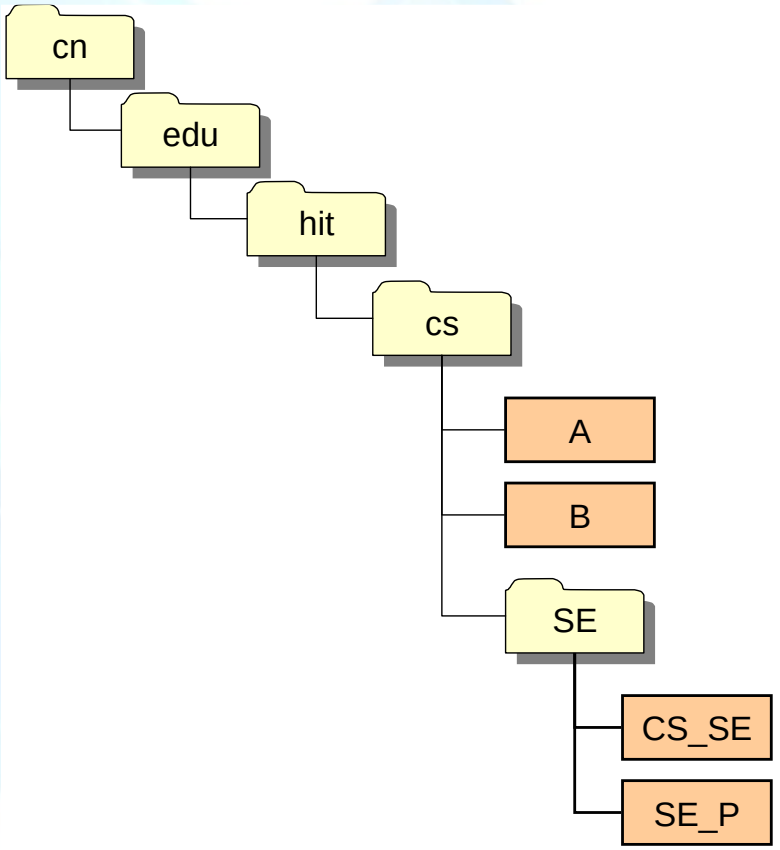
---

```
import java.lang.System;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.BufferedReader;


InputStream = System.in;
InputStreamReader isr = new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
```

# JAVA中的“package”

```
package cn.edu.hit.cs;
public class A{ }
-----------------------------------
package cn.edu.hit.cs;
public class B{ }
-----------------------------------
package cn.edu.hit.cs.se;
public class CS_SE{ }
-----------------------------------
package cn.edu.hit.cs.se;
public class CS_SE_P{ }
```



# 检查系统设计

- 检查“正确性”
  - 每个子系统都能追溯到一个用例或一个非功能需求吗？
  - 每一个用例都能映射到一个子系统吗？
  - 系统设计模型中是否提到了所有的非功能需求？
  - 每一个参与者都有合适的访问权限吗？
  - 系统设计是否与安全性需求一致？
- 检查“一致性”
  - 是否将冲突的设计目标进行了排序？
  - 是否有设计目标违背了非功能需求？
  - 是否存在多个子系统或类重名？

# 检查系统设计

- 检查“完整性”
  - 是否处理边界条件？
  - 是否有用例走查来确定系统设计遗漏的功能？
  - 是否涉及到系统设计的所有方面(如硬件部署、数据存储、访问控制、遗留系统、边界条件)?
  - 是否定义了所有的子系统？
- 检查“可行性”
  - 系统中是否使用了新的技术或组件？是否对这些技术或组件进行了可行性研究？
  - 在子系统分解环境中检查性能和可靠性需求了吗？
  - 考虑并发问题了吗？

# 本章主要内容

1. 面向对象设计概述
2. 系统设计
3. 包的设计
4. 对象设计
5. 面向对象设计总结

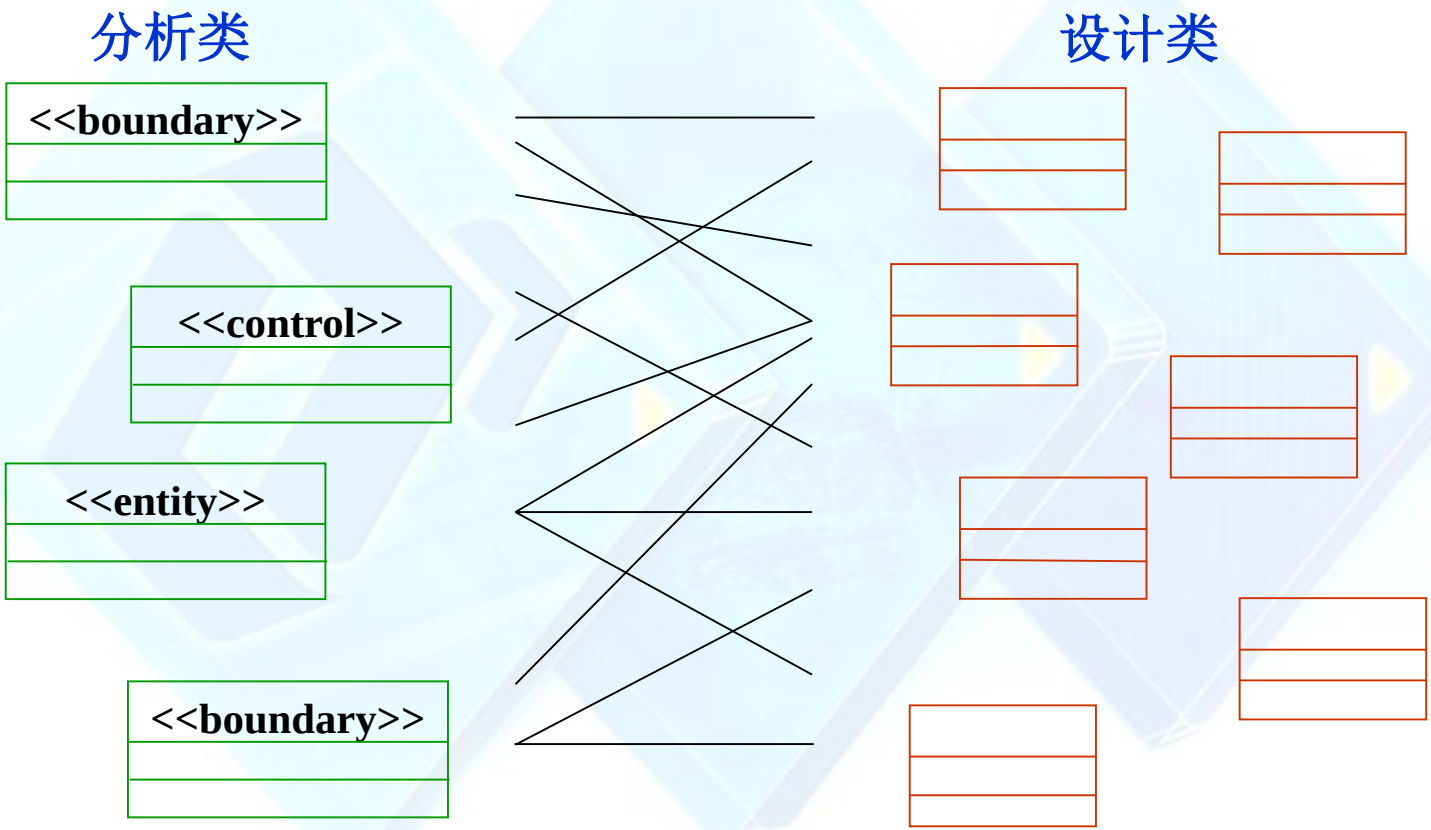
# 对象设计概述

- 对象设计
  - 细化需求分析和系统设计阶段产生的模型
  - 确定新的设计对象
  - 消除问题域与实现域之间的差距
- 对象设计的主要任务
  - 精化类的属性和操作
    - 明确定义操作的参数和基本的实现逻辑
    - 明确定义属性的类型和可见性
  - 明确类之间的关系
  - 整理和优化设计模型

# 对象设计的基本步骤

1. 创建初始的设计类
2. 细化属性
3. 细化操作
4. 定义状态
5. 细化依赖关系
6. 细化关联关系
7. 细化泛化关系

# 1.创建初始的设计类

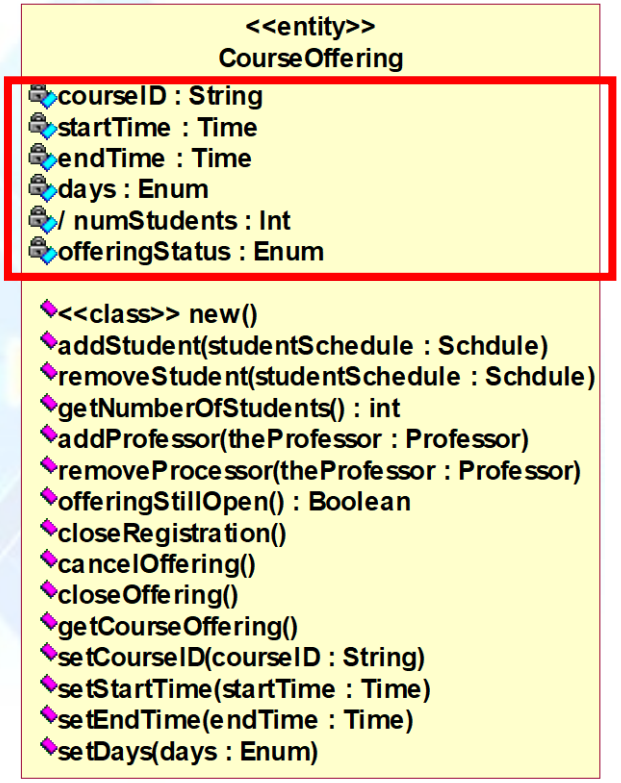


# 2.细化属性

- 细化属性
  - 具体说明属性的名称、类型、缺省值、可见性

    visibility attributeName : Type = Default
  - **Public:** + 对能看到该类的任何元素都可见
  - **Private:** - 只对本类的任何元素可见
  - **Protected:** # 对该类及子孙类的任何元素可见
  - **Package:** ~ 对同包中的任何元素可见
- 属性的来源：
  - 类所代表的现实实体的基本信息
  - 描述状态的信息
  - 描述该类与其他类之间关联的信息
  - 派生属性(derived attribute)：该类属性的值需要通过计算其他属性的值才能得到，属性前面加“/”表示

    例如：类CourseOffering中的“学生数目” / numStudents : int



# 细化属性

- 基本原则
  - 属性命名符合规范(名词组合，首字母小写)
  - 尽可能将所有属性的可见性设置为private
  - 仅通过set方法更新属性
  - 仅通过get方法访问属性
  - 在属性的set方法中，实现简单的有效性验证，而在独立的验证方法中实现复杂的逻辑验证

# 关于状态属性

- 状态由一个或多个属性来表示（实体类中经常含有状态属性）
  - 对“订单”类来说，可以设定一个状态属性“订单状态”，取值为enum{未付款、取消、已付款未发货、已发货、已确认收货未评价、买方已评价、双方已评价、…}
  - 也可以由多个属性来表示状态：是否已付款、是否已发货、是否已确认、买方是否已评价、卖方是否已评价、…
  - 每个状态属性的类型多数用boolean
- 大部分状态属性，可以由该类的其他属性的值进行逻辑判断得到
  - 若订单处于“未付款”状态，则该订单的“订单变迁记录”中一定不会包含有付款信息
  - 若它处于“买家已评价”状态，则它的“买家评价”属性一定不为空
- 从一个状态值到另一个状态值的变迁，一定是由该实体类的某个操作所导致的
  - 订单从不存在到变为未付款，是由new操作导致的状态变化
  - 订单从已发货到已确认的变化，是由确认收货操作所导致的状态变化
  - 检查状态变迁的条件，可以判断你为实体类所设计的操作是否完整

# 关于关联属性

- 两个类之间有association关系，意味着需要永久管理对方的信息，需要在程序中能够从类1的object“导航”(navigate)到类2的object
  - 例如：订单类与买家类产生双向关联，意即一个订单对象中需要能够找到相应的买家对象，反之买家对象需要知道自己拥有哪些订单对象
    - 订单类中有一个关联属性buyer，其数据类型是买家类
    - 买家类中有一个关联属性OrderList，其数据类型是订单类构成的序列
- 关联属性不只是一个ID，而是一个或多个完整的对方类的对象
- 务必与数据库中的“外键”区分开：
  - 例如：订单table中有一个外键（只是一个ID值）：买家ID(字符串类型)，靠它与买家table联系起来
- 不要用关系数据模式的设计思想来构造类的属性
- 关联属性的目标：
  - 在程序运行空间内实现object之间的导航，而无需经过数据库层存取

# 3.细化操作

- 找出满足基本逻辑要求的操作：
  - 从actor出发，分别思考需要类的哪些操作
- 补充必要的辅助操作：
  - 初始化类的实例、销毁类的实例 — Student(…)、~Student( )
  - 验证两个实例是否等同 — equals( )
  - 获取属性值(get操作) — getXXX( )
  - 设定属性值(set操作) — setXXX(…)
  - 将对象转换为字符串 — toString( )
  - 复制对象实例 — clone( )
  - 用于测试类的内部代码的操作 — main( )
  - 支持对象进行状态转换的操作 — ……
- 细化操作时，要充分考虑类的属性与状态是否被充分利用：
  - 对属性进行CRUD
  - 对状态进行各种变更

# 细化操作

- 给出完整的操作描述：
  - 确定操作的名称、参数、返回值、可见性等
  - 应该遵从程序设计语言的命名规则(动词+名词，首字母小写)
- 详细说明操作的内部实现逻辑
  - 复杂的操作过程采用伪代码或者绘制程序流程图/活动图方式
- 在给出内部实现逻辑之后，可能需要：
  - 将各个操作中公共部分提取出来，形成独立的新操作

# 细化操作

- 操作的形式：

  visibility opName ( param : type = default, ... ) : returnType

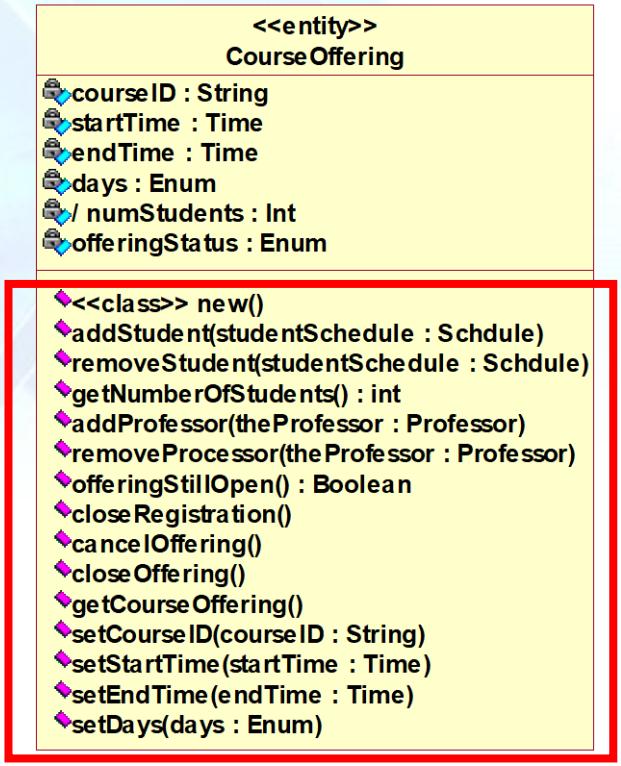
- 一个例子: CourseOffering
  - **Constructor**

    *<<class>>new ( )*
  - **Set attributes**

    + *setCourseID ( courseID:String )*
    + *setStartTime ( startTime:Time )*
    + *setEndTime ( endTime:Time )*
    + *setDays ( days:Enum )*
  - **Others**

    + *addProfessor ( theProfessor:Professor )*
    + *removeProfessor ( theProfessor:Professor )*
    + *offeringStillOpen ( ) : Boolean*
    + *getNumberOfStudents ( ) : int*

    ... ...



# 细化操作

- 一个例子：*BorrowerInfo*类
  - 构造函数

    *<<class>> + new ( )*
  - 属性赋值

    *+ setName( name:String)*

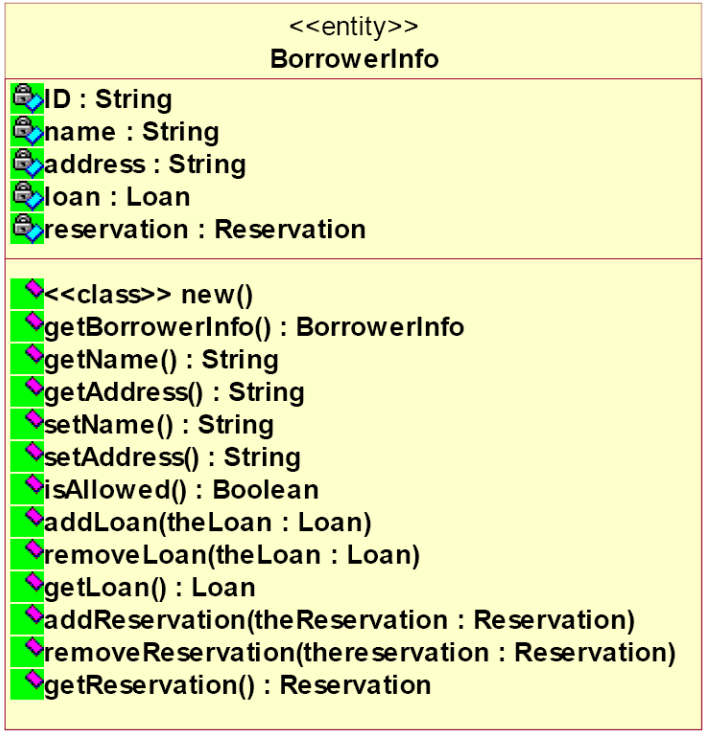
    *+ setAddress( address:String)*
  - 其他

    *+ addLoan( theLoan:Loan)*

    *+ removeLoan( theLoan:Loan)*

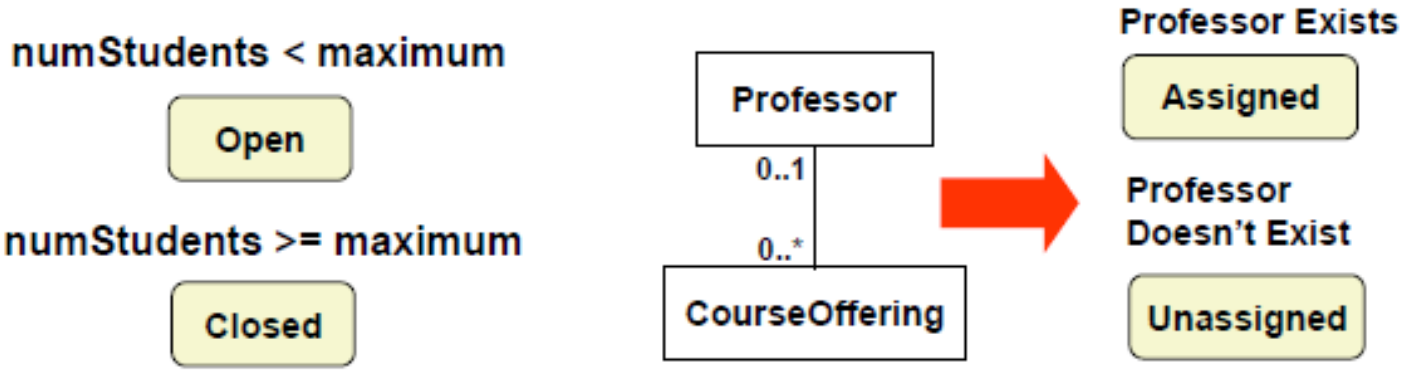
    *+ isAllowed( ) : Boolean*

    *… …*



# 4* 定义状态

- 目的：
  - 设计一个对象的状态是如何影响其行为的
  - 绘制对象状态图
- 需要考虑的要素：
  - 哪些对象有状态？
  - 如何发现对象的所有状态？
  - 如何绘制对象状态图？
- Example: CourseOffering

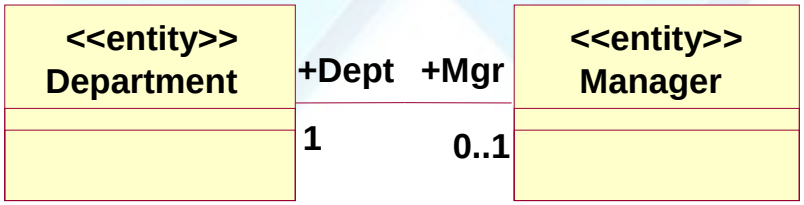


# 5.细化类之间的关系

- 细化关系：关联关系、依赖关系、继承关系、组合和聚合关系
- “继承”关系很清楚
- 在对象设计阶段，需要进一步确定详细的关联关系、依赖关系和组合/聚合关系等

# 定义关联关系(Association/Composition/Aggregation)

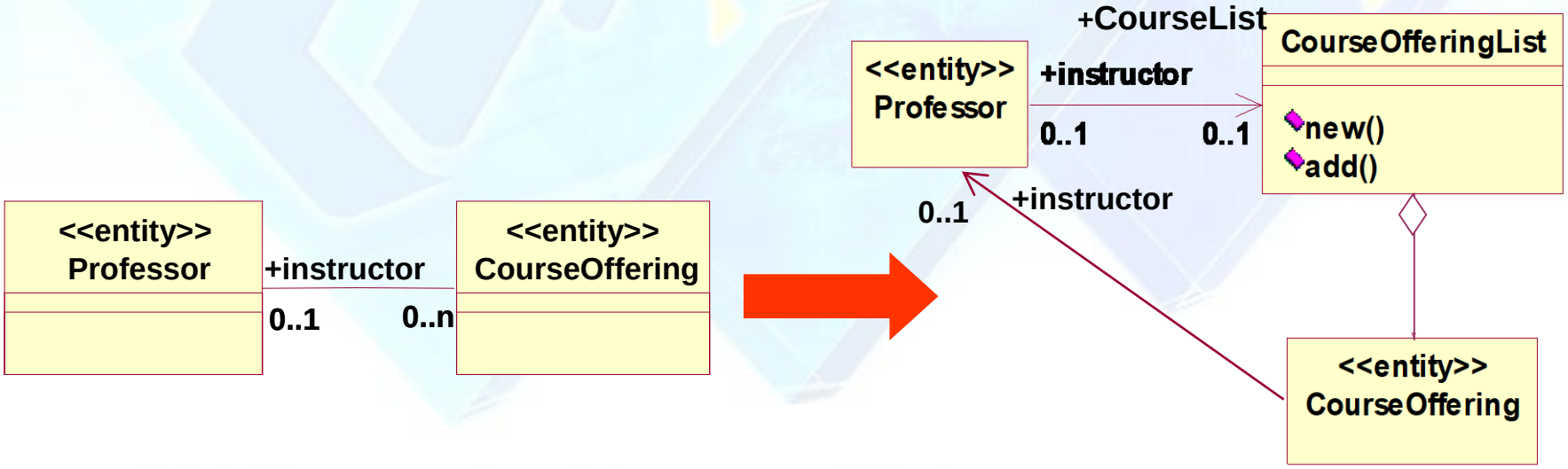
- 根据多重性进行设计(multiplicity-oriented design)
- 情况1：Multiplicity = 1或Multiplicity = 0..1
  - 可以直接用一个单一属性/指针加以实现，无需再作设计
  - 若是双向关联：
    - Department类中有一个属性：+Mgr: Manager
    - Manager类中有一个属性：+Dept: Department
  - 若是单向关联：
    - 只在关联关系发出的类中增加关联属性




设计阶段需要将这种“关联属性”增加到属性列表中，并更新操作列表
分析阶段则不需要在属性列表中加入“关联属性”

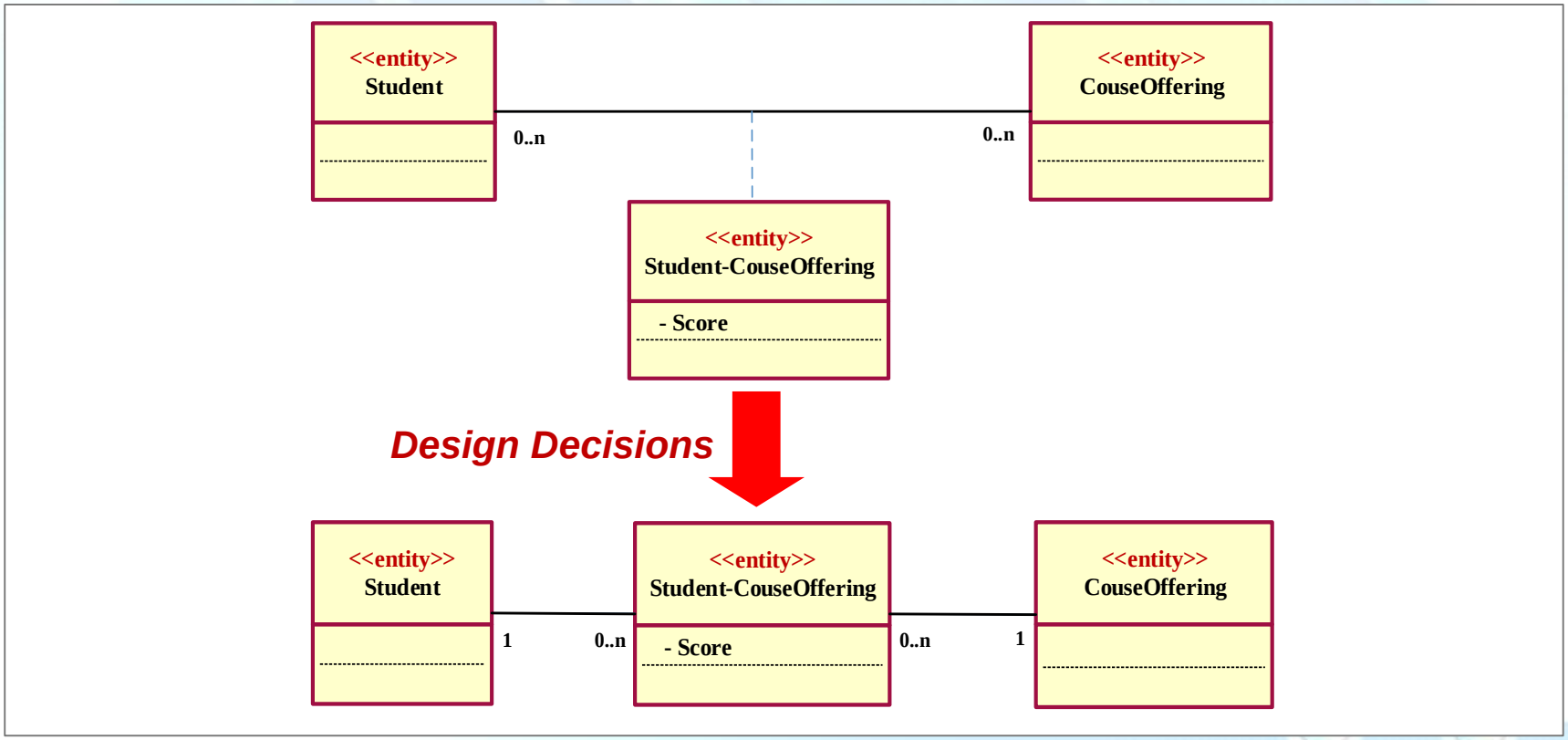
# 定义关联关系(Association/Composition/Aggregation)

- 根据多重性进行设计(multiplicity design)
- 情况2：Multiplicity > 1
  - 无法用单一属性/指针来实现，需要引入新的设计类或能够存储多个对象的复杂数据结构(例如链表、数组等)
  - 将1:n转化为若干个1:1

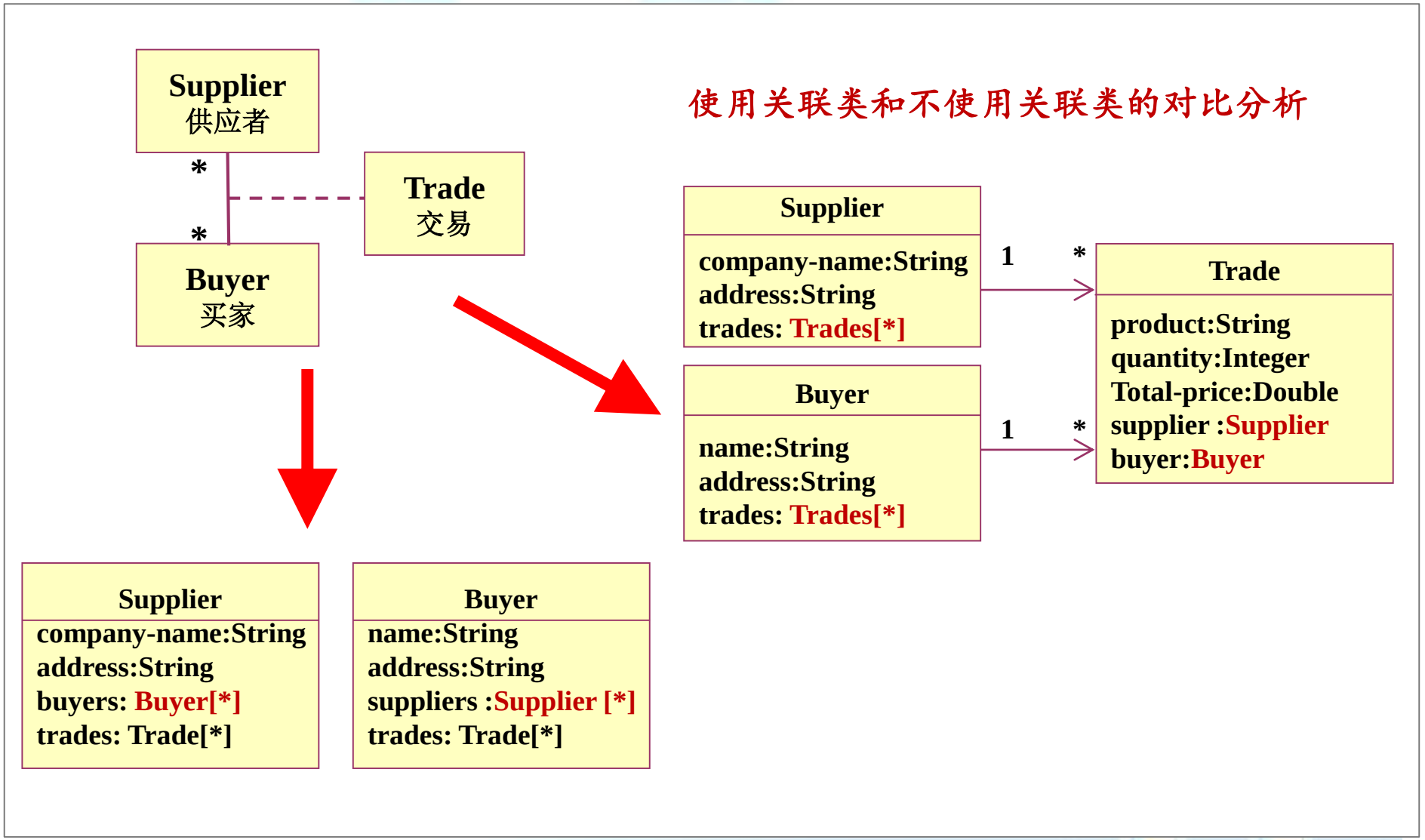


# 定义关联关系(Association/Composition/Aggregation)

- 情况3：有些情况下，关联关系本身也可能具有属性，可以使用关联类将这种关系建模
- 举例：学生Student与开设课程CourseOffering之间的关联关系



# 定义关联关系(Association/Composition/Aggregation)



# 本章主要内容

1. 面向对象设计概述
2. 系统设计
3. 包的设计
4. 对象设计
5. 面向对象设计总结

# 面向对象设计总结

- 系统设计
  - *包图(package diagram) → 逻辑设计
  - 部署图(deployment diagram) → 物理设计
- 对象设计
  - 设计类图(design class diagram) → 更新分析阶段的类图，对各个类给出详细的设计说明
  - *状态图(statechart diagram) → 使用设计类来更新分析阶段的状态图
  - *时序图(sequence diagram) → 使用设计类来更新分析阶段的时序图
  - 关系数据库设计方案(RDBMS design)
  - 用户界面设计方案(UI design)

# 《系统分析与设计》
# System Analysis and Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(微信同号)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部/

国家示范性软件学院

软件工程教研室

2023.03

# 数据库设计

## 本章主要内容

1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. ERD模型及质量评价
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# 数据库设计

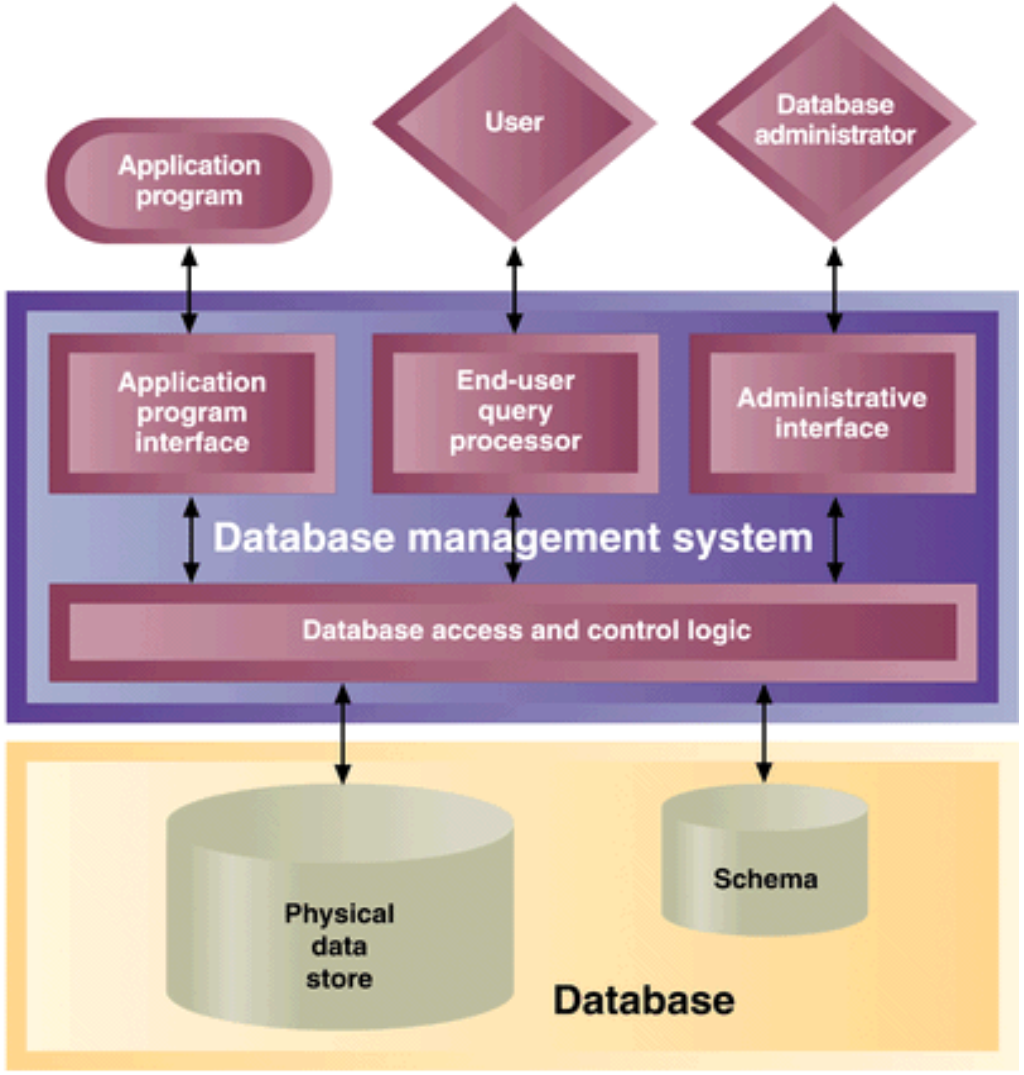
## 本章主要内容

1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. ERD模型及质量评价
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# 数据库设计

## 数据库系统＝数据库(DB)＋数据库管理系统(DBMS)

- **数据库(DB)：被集中控制和管理的存储数据的完整集合**
  - 用ERD模型来描述
  - 以表（Table）对应ERD中的数据实体（Entity），是数据管理的基本单元
- **数据库管理系统(DBMS)：对数据库的访问进行管理和控制的软件**
  - 管理数据的逻辑格式定义、存储和访问
  - 允许多个用户或应用程序并发访问数据库
  - 支持通过SQL语言访问数据库



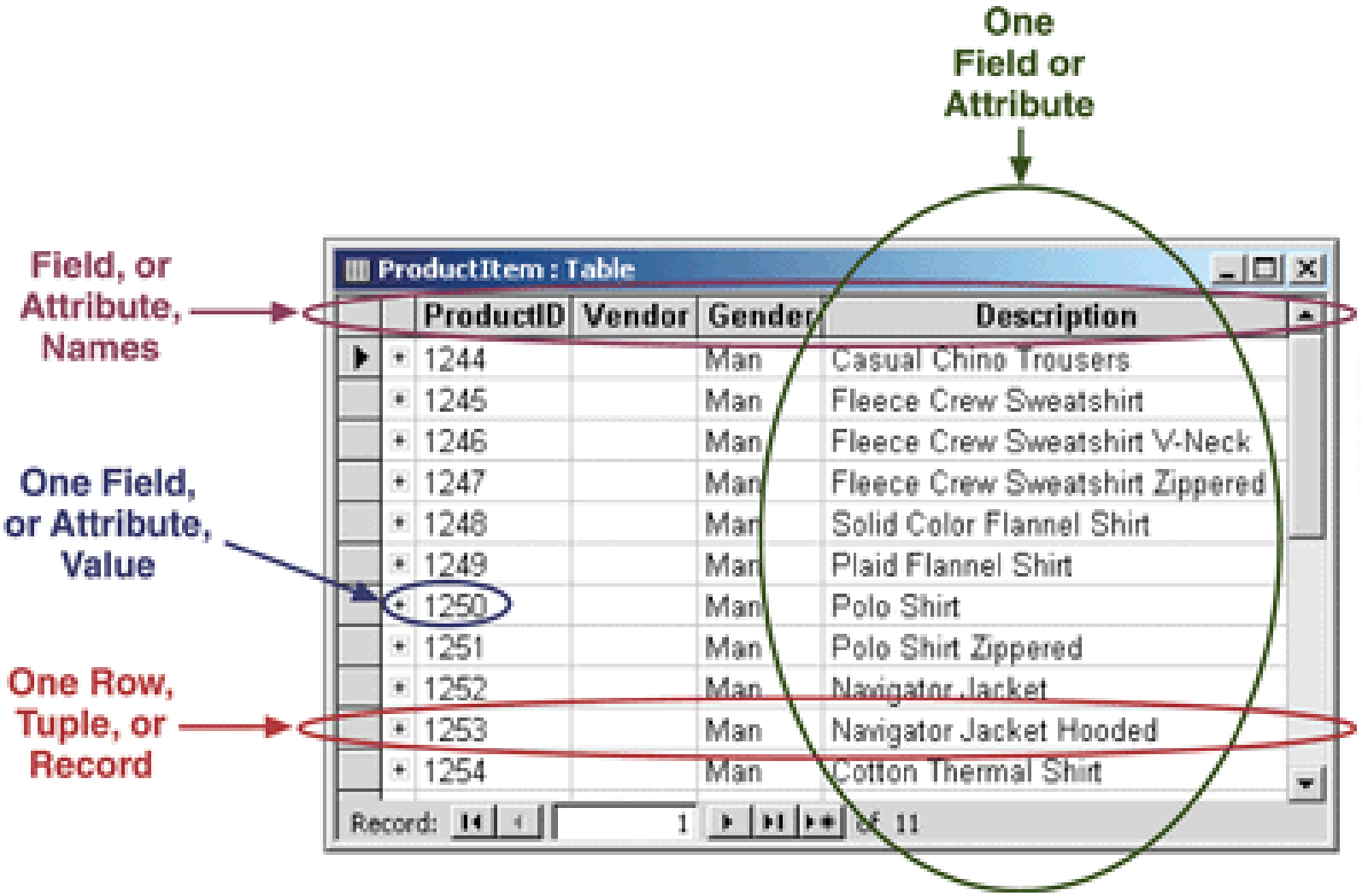

数据库系统体系结构

# 关系数据库

- 关系型数据库管理系统将数据存储成表和关系的结构
- 表（Table）:关系型数据库管理数据的基本单元，类似于Excel中的Sheet，与ERD模型中的实体（Entity）对应
  - 元组: 表中的行，习惯称为“记录”
  - 域:　表中的列，习惯称为“属性”
- 每个表中必须包含能唯一标识“记录”的属性或属性组，称为关键属性（Key Fields），相当于记录的索引关键词
- 数据库的表（Table）之间的关系通过Key Fields来建立
  - 主键（Primary Key）：本表中的关键属性
  - 外键（Foreign Key）：存储在本表中的其他表的关键属性，用来与其他表建立关联关系（相当于指针）

# 数据库设计

## 关系数据库：数据表（Table）的构成



# 数据库设计

## 数据库设计的根本任务

- 获取未来系统需要存储的数据实体（一定粒度的数据单元）
- 分析数据实体的内涵，以及数据实体之间的关系
- 设计物理ERD，即用于数据库系统实现的数据表结构的蓝图

## 数据库设计的基本过程

- 概念ERD设计 – 需求分析阶段：从宏观上识别业务过程和系统层面所存在的数据实体，分析它们之间的关系。概念 ERD数据模型仅需给出存在的数据实体以及基本关系，不关注其他细节。
- 逻辑ERD设计 – 系统分析阶段：逻辑 ERD 是概念 ERD 的详细版本，明确定义每个实体中的属性，识别实体之间的关系基数，并引入关联实体，但不关注物理实现细节。
- 物理ERD设计 – 系统设计阶段：物理 ERD 是数据库的实际设计蓝图，将ERD的数据实体转化为数据的数据表（Table），针对具体的数据库系统，定义Table属性的细节（包括属性类型、长度、可否为空等）以及增加用于表达Table之间关系的新属性。

# 数据库设计

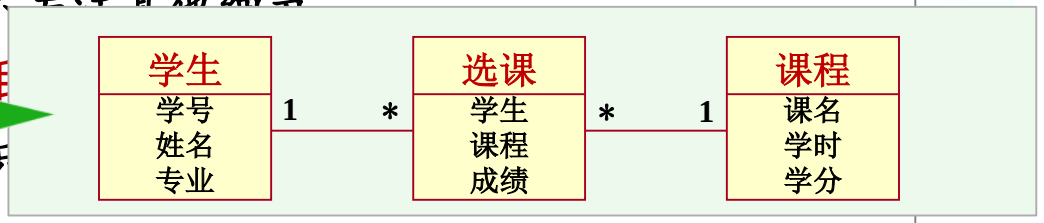
## 数据库设计的根本任务

- 获取未来系统需要存储的数据实体（一定粒度的数据单元）
- 分析数据实体的内涵，以及数据实体之间的关系
- 设计物理ERD，即用于数据库系统实现的数据表结构的蓝图

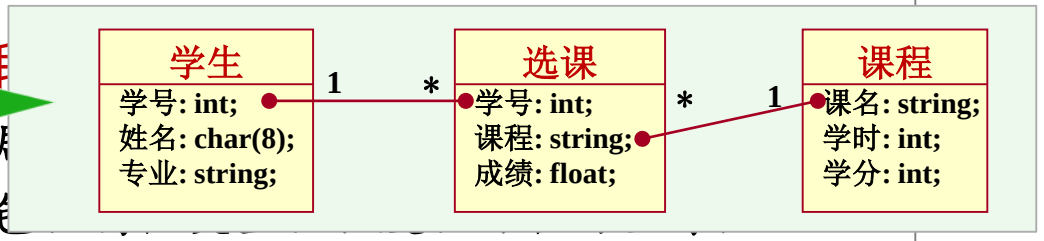
## 数据库设计的基本过程

- 概念ERD设计 – 需求分析阶段
  存在的数据实体，分析它们之间
  在的数据实体以及基本关系，不关注其他细节

- 逻辑ERD设计 – 系统分析阶段
  明确定义每个实体中的属性，识
  ，但不关注物理实现细节。




- 物理ERD设计 – 系统设计阶段
  ，将ERD的数据实体转化为数据
  系统，定义Table属性的细节（包
  增加用于表达Table之间关系的新属性。



# 数据库设计

## 数据库设计的根本任务

- 获取未来系统需要存储的数据实体（一定粒度的数据单元）
- 分析数据实体的内涵，以及数据实体之间的关系
- 设计物理ERD，即用于数据库系统实现的数据表结构的蓝图

## 数据库设计的基本过程

- 概念ERD设计
  存在的数据实体
  在的数据实体以
- 逻辑ERD设计
  明确定义每个实
  ，但不关注物理
- 物理ERD设计
  ，将ERD的数据
  系统，定义Table
  增加用于表达Ta

| ERD形式 | 概念ERD | 逻辑ERD | 物理ERD |
|---|---|---|---|
| 实体（名称） | √ | √ | √ |
| 关系 | √ | √ | √ |
| 关系基数 | 随意 | √ | √ |
| 关联实体 | | √ | √ |
| 属性（名称） | | √ | √ |
| 属性的类型、长度、说明 | | 随意 | √ |
| 主键、外键 | | | √ |

# 数据库设计

## 本章主要内容

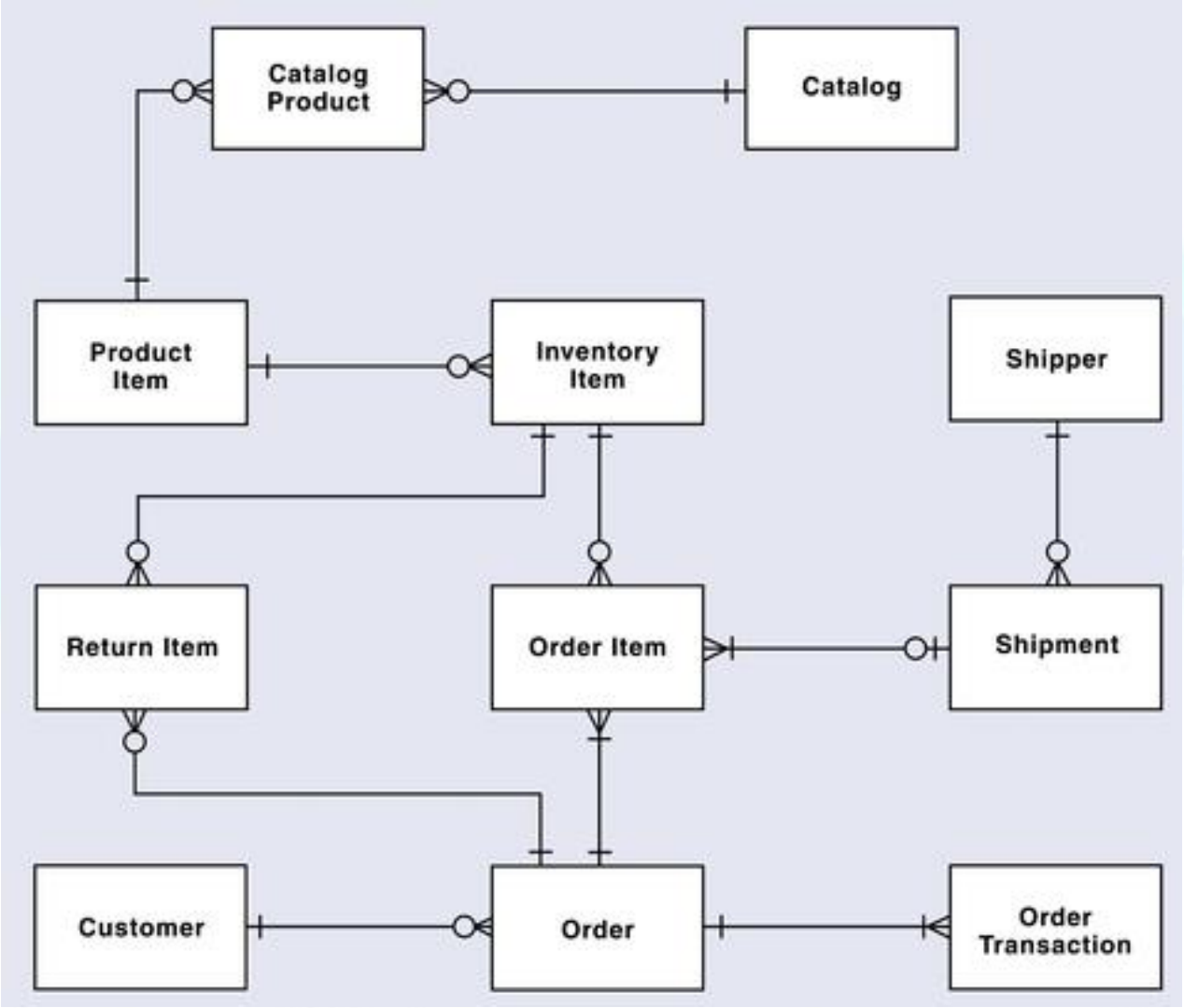
1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. ERD模型及质量评价
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# 数据库逻辑模型设计 – ERD

1. 识别所有“自然”数据实体（Entity）
   - 来自DFD中的全部“数据存储”
   - 来自“分析类图”中的部分“类”
2. 为数据实体命名
3. 给出实体的属性
4. 识别实体之间的关联关系及关联重数
   - 关联关系需命名
   - 关联重数需判别（1:1、1:M、M:N）
5. 建立关联实体（“人造”实体）来消除M:N重关系
6. 实体规范化（一般需满足3NF）
7. 评价ERD质量并做必要的改进

# 数据库设计

## ERD图例子（Crow's feet样式）

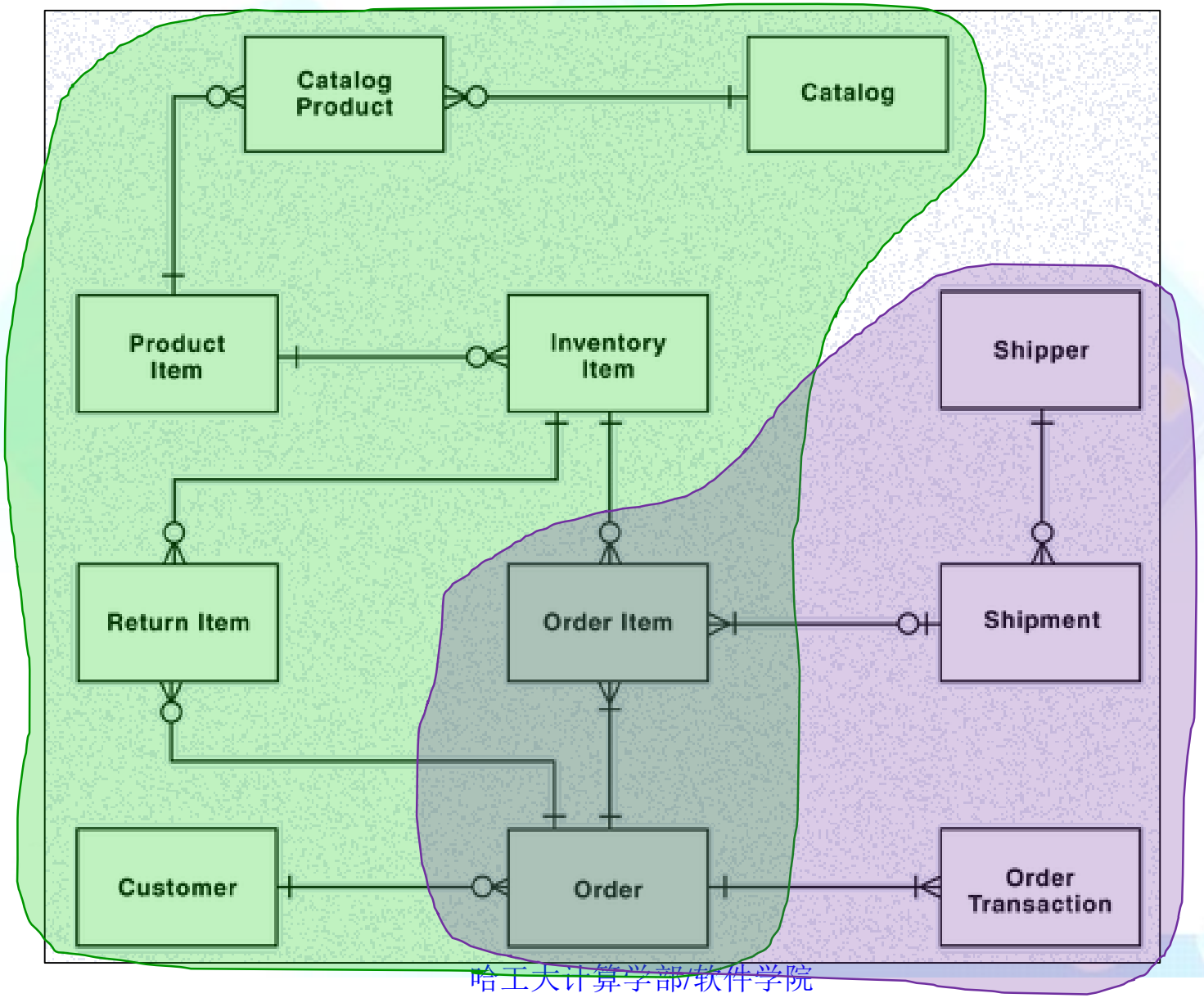


## 复杂ERD分组技巧

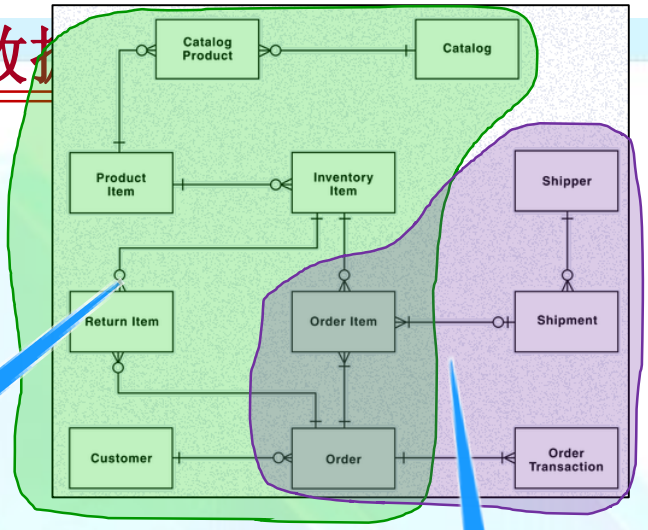
- ERD中实体非常多，以至于画在一张图中过于复杂难以清晰使用，则可以根据系统功能来分组
- 如果组之间存在相互关联的实体出现，则可以将其画多次，分别在不同的局部ERD中出现

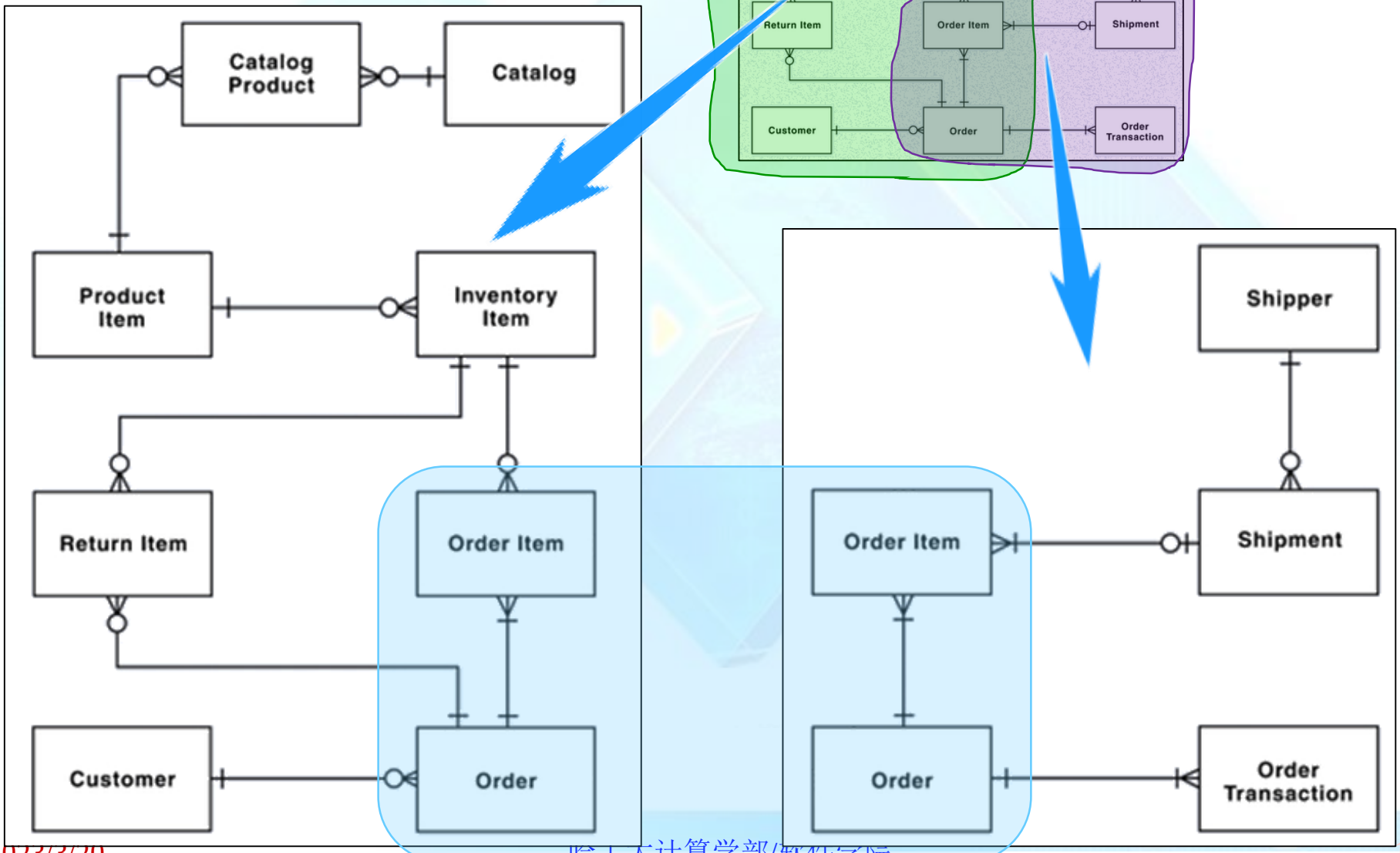
# 数据库设计

## 复杂ERD分组技巧



# 复杂ERD分组技巧


Catalog Product
Catalog
Product Item
Inventory Item
Shipper
Return Item
Order Item
Shipment
Customer
Order
Order Transaction


Catalog Product
Catalog
Product Item
Inventory Item
Return Item
Order Item
Customer
Order
Shipper
Shipment
Order Item
Order
Order Transaction

# 数据库设计

## 本章主要内容

1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. **ERD模型及质量评价**
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# ERD质量的评价

- **ERD图的设计结果没有标准答案，即不唯一**
- **好的ERD标准：**
  - 结构清晰
  - 关联简洁
  - 实体个数适中
  - 属性分配合理
  - 没有低级冗余（重复性冗余）
- **至少符合3NF**

# 数据库的规范化（Normalization）

- 通过最小化数据冗余来确保数据库模型的质量
- 范式（**Normal Form**）的分类
  - **1NF** – 没有重复的属性或属性组
  - **2NF** – 是1NF 且每个非主属性均函数依赖于主属性(主键)
  - **3NF** – 是2NF且非主属性间均不存在函数依赖

**能否用通俗的语言解释1NF、2NF、3NF？**

# 数据库的规范化（Normalization）

- 范式的通俗、易懂的解释：
  - 1NF：同样的东西不能重复拥有！
  - 2NF：领导在，他就在，找到领导就能找到他！
  - 3NF：领导只有一个，不能搞宗派，不能有多级领导！
- 3NF数据库设计可以做到，但是未必是最好的数据库
  - 为了提高运行效率，必须降低范式标准，适当保留冗余数据
  - 具体做法：在逻辑模型设计时遵守3NF，降低范式标准的工作放到物理数据模型设计时考虑，即增加冗余字段

# 数据库设计

## 本章主要内容

1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. ERD模型及质量评价
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# 物理数据库设计及建立

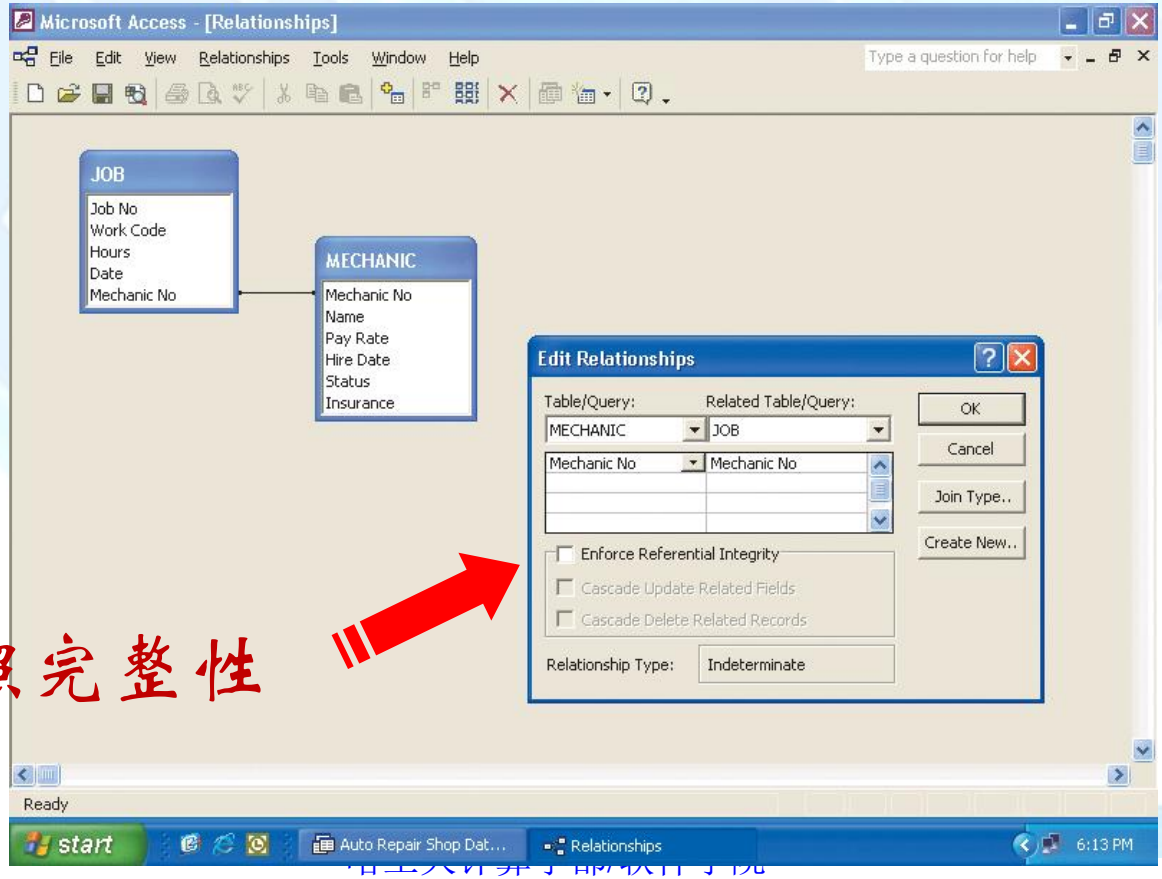
根据ERD建立物理数据库：

1. 为每个ERD实体创建一个二维表（即数据库Table）
2. 根据ERD中的实体属性，在对应的Table中定义对应的字段（Field），并给出适当的数据类型和取值范围
3. 定义每个Table的主键(PK：PrimaryKey)
4. 针对1:M关联关系的子表添加外键（FK：ForeignKey）即对应主表的主键
5. 定义完整性约束

# 数据库设计

## 定义完整性约束（3个层次）

- 域的完整性：设定字段的取值范围
- 参照完整性：用PK、FK、表级触发器来实现
- 用户定义完整性：定义一些业务规则，用存储过程和触发器来实现




执行参照完整性

# 参照完整性的自动执行

- 当建立一个包含外键的记录时，DBMS 确保该外键同时作为主键出现在另一个相关表的记录中
  - 如果关联其他记录，对方必须存在
- 当删除一个记录时，DBMS 确保在其它相关的表中不会出现和该记录主键值相同的外键
  - 如果正在被其他记录关联，则不能删除
- 当主键值改变时，DBMS保证相关表中没有外键与该主键具有相同的值
  - 改变记录主键值时：①确保不冲突；②全局自动更改

# 数据库设计

## 本章主要内容

1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. ERD模型及质量评价
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# 数据库设计

## 提高数据库运行效率的5个办法

(1) 降低范式、增加冗余；少用触发器、多用存储过程

(2) “体外运算”

当计算非常复杂、记录数巨大时，则以文件系统方式用C++等语言计算处理完成之后，最后才入库追加到表中去

（电信计费系统设计的经验）

(3) 水平/垂直分割表

- 当表中记录太多时则水平分割：以该表主键PK的某个值为界线，将该表的记录水平分割为两个表
- 当表中字段太多时则垂直分割：将原来的字段分为二组，分别建立二个1：1的表

(4) 对数据库管理系统DBMS进行系统优化，即优化各种系统参数，如缓冲区个数

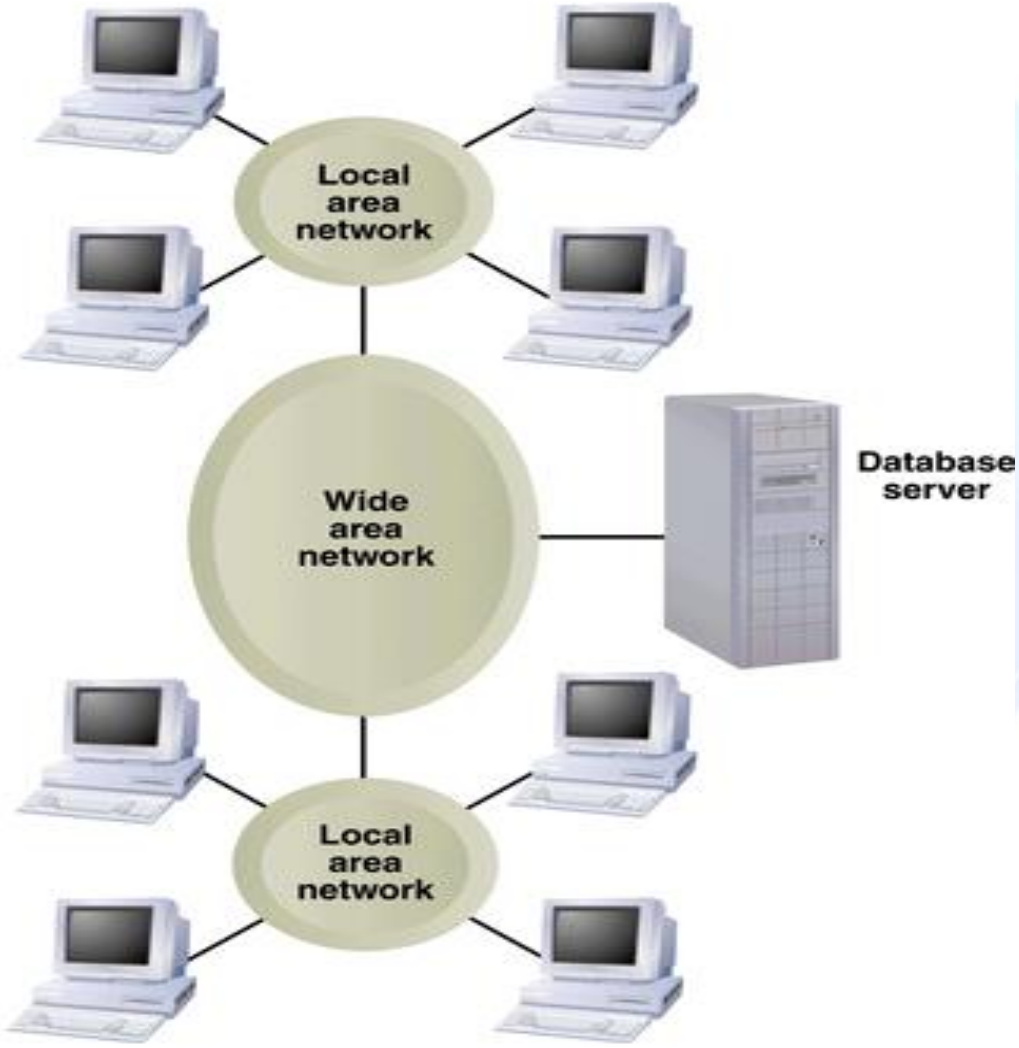
(5) 在使用面向数据的SQL语言进行程序设计时，尽量采取优化算法

# 数据库设计

## 本章主要内容

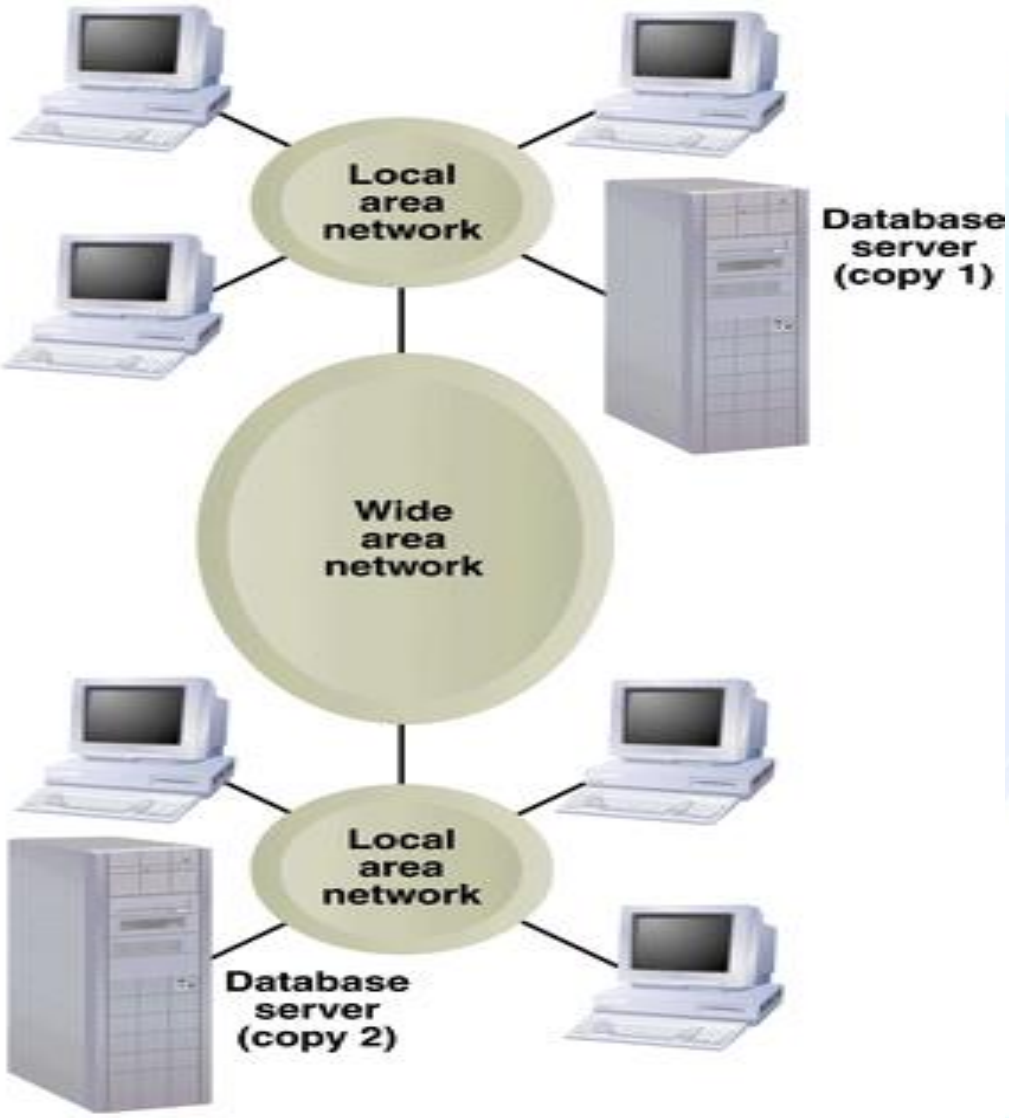
1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. ERD模型及质量评价
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# 单数据库服务器体系结构



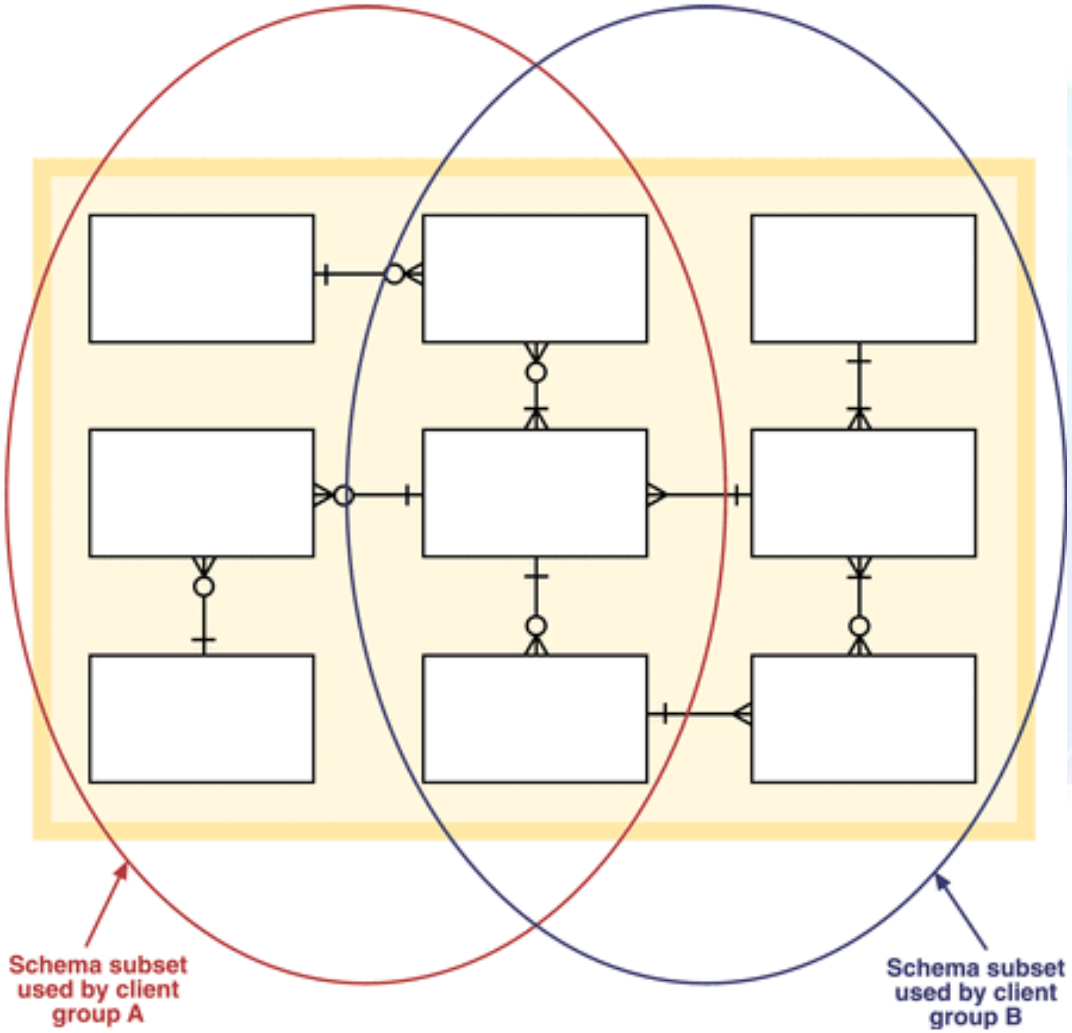
# 数据库设计

## 带备份的数据库服务器体系结构

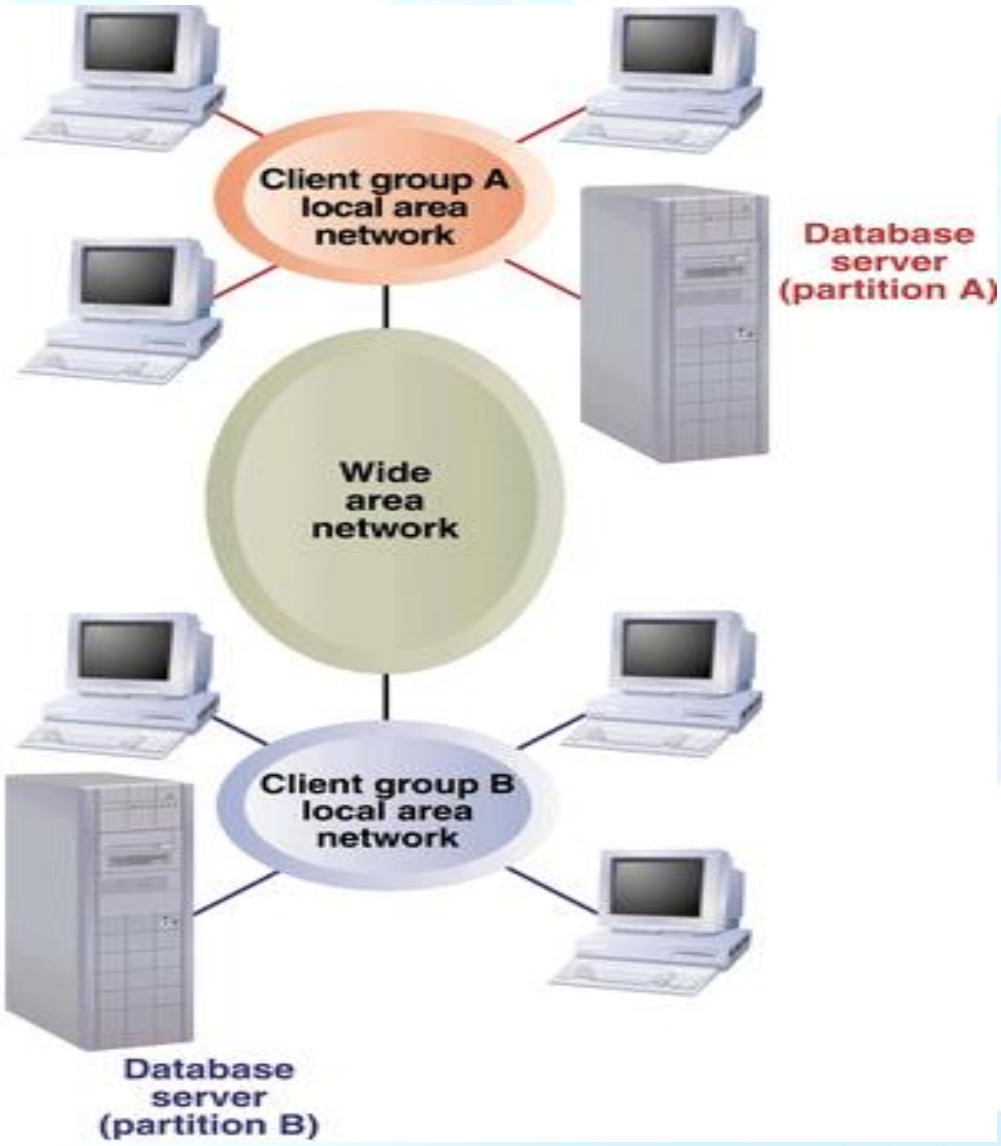


# 数据库设计

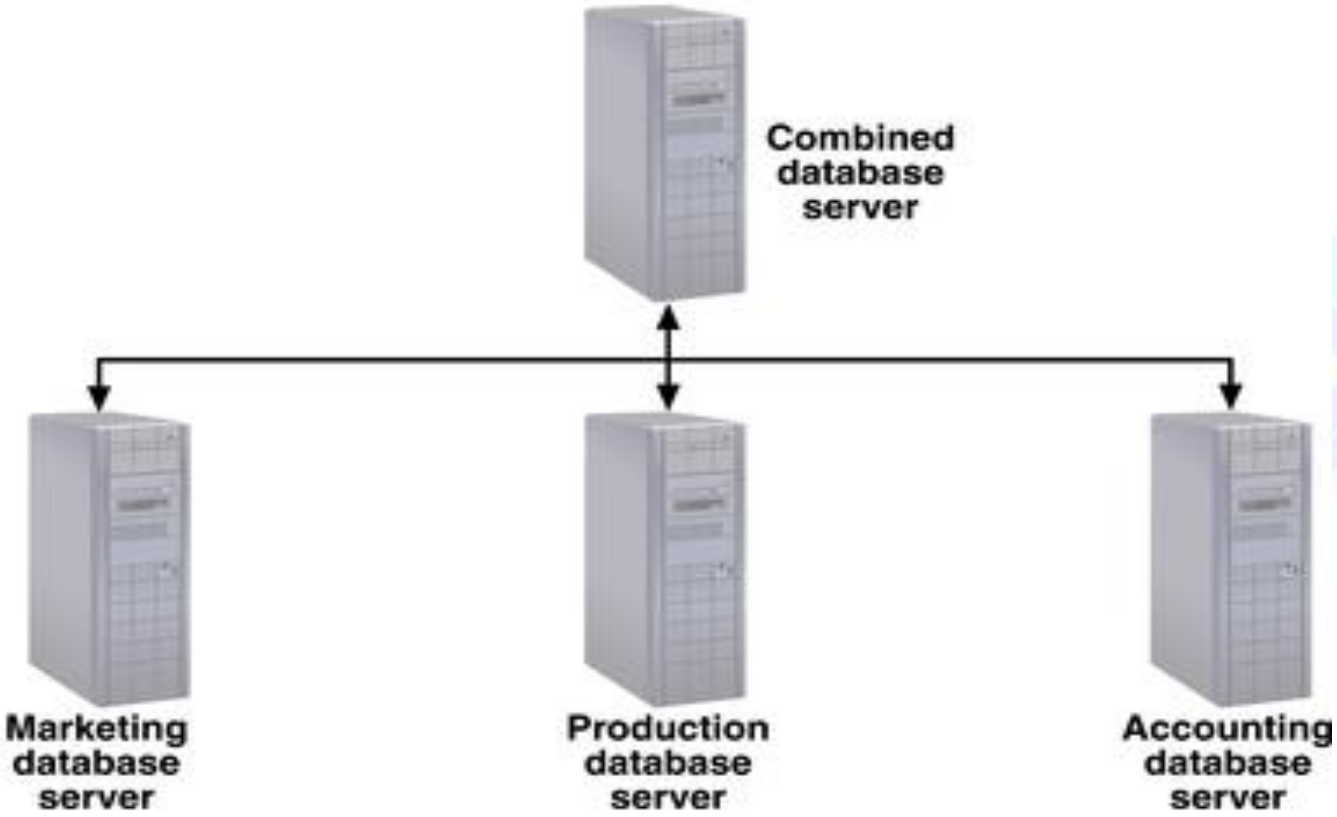
## 将数据库模式划分为多个客户访问子集



# 分区数据库服务器体系结构



# 联合数据库服务体系结构



# 数据库设计

## 本章主要内容

1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. ERD模型及质量评价
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# 数据库设计

## 类和关系数据表的关系

- OOP以Class为基本单位，所有的Object都是运行在内存空间当中
- 若某些Object的信息需要持久化存储，那么就需要用到Database，将Object的属性信息写入关系数据表
  - 假如淘宝没有“保存购物车内容”的功能（意即若不购买，下次进入之后购物车中的内容就被清空），那么“购物车”这个实体的属性就不需要关系数据表
- 在系统执行某些功能的时候，需要首先从Database中将信息取出，使用各实体类的New()操作构造相应的Object，在程序运行空间中使用（充分利用继承/组合/聚合/关联/依赖关系在各Object之间相互导航）
- 在进行OO分析和设计时完全可以不考虑数据永久存储的事情，本节就是要考虑如何使用数据库来解决永久存储的问题！

# 数据存储设计

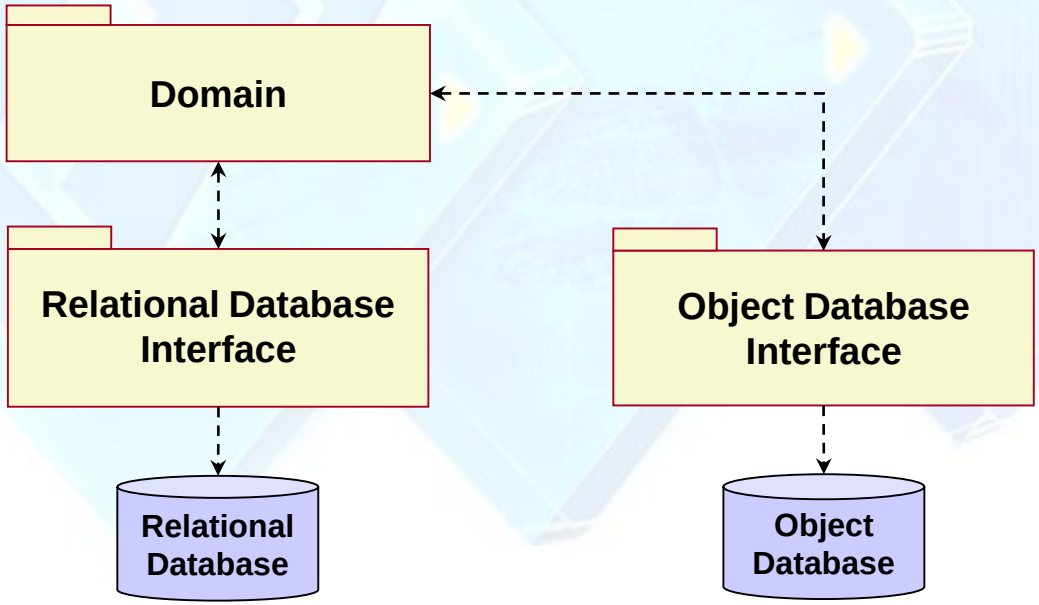
- “对象”只是存储在内存当中，而某些对象则需要永久性的存储起来 -- 持久性数据（**Persistent Data**）
- 数据存储策略
  - 数据文件：
    由操作系统提供的存储形式，应用系统将数据按字节顺序存储，并定义如何以及何时检索数据
  - 关系数据库：
    数据是以表的形式存储在预先定义好的Schema的类型中
  - 面向对象数据库：
    将对象和关系作为数据一起存储
  - 存储策略的选择：
    取决于非功能性的需求

# 数据存储策略的权衡（tradeoff）

- 何时选择文件？
  - 存储大容量数据、临时数据、低信息密度数据
- 何时选择数据库？
  - 并发访问要求高、系统跨平台、多应用程序使用相同数据
- 何时选择关系数据库？
  - 复杂的数据查询
  - 数据集规模大
- 何时选择面向对象数据库？
  - 数据集处于中等规模
  - 频繁使用对象间联系来读取数据

# 数据存储策略

- 如果使用OO数据库，那么数据库系统应提供一个接口供应用系统访问数据
- 如果使用关系数据库，那么需要一个子系统来完成应用系统中的对象和数据库中数据的映射与转换



# OO设计中的数据库设计

- 核心问题：对那些需要永久性存储的数据，如何将UML类图（实体类图）映射为数据库模型（实体关系模型ERD）
- 本质：把每一个类、类之间的关系分别映射到一张表或多张关系型数据库的表中（Table）
- 解决：UML Class Diagram → Relation DataBase (RDB)
- 两个方面：
  - 将类（Class）映射到表（Table）
  - 将关联关系（Association）映射到表（Table）

# 对象关系映射(ORM)

- 对象关系映射（Object Relational Mapping，ORM）：
  - 为了解决面向对象与关系数据库存在的互不匹配的现象
  - 通过使用描述对象和数据库之间映射的元数据，将OO系统中的对象自动持久化到关系数据库中
- 目前流行的ORM产品：
  - Apache OJB
  - Hibernate
  - JPA(Java Persistence API)
  - Oracle TopLink
  - …

# 将对象映射到关系数据库

- 最简单的映射策略 一类一表：表中的字段对应于类的属性，表中的每一行数据记录对应类的实例(即对象)

Object Model

| Person |
| --- |
| name<br>address |
| |

Table Model

| Attribute Name | Datatype | nulls |
| --- | --- | --- |
| Person_ID | int | N |
| Person_name | Char(20) | N |
| address | Char(50) | Y |

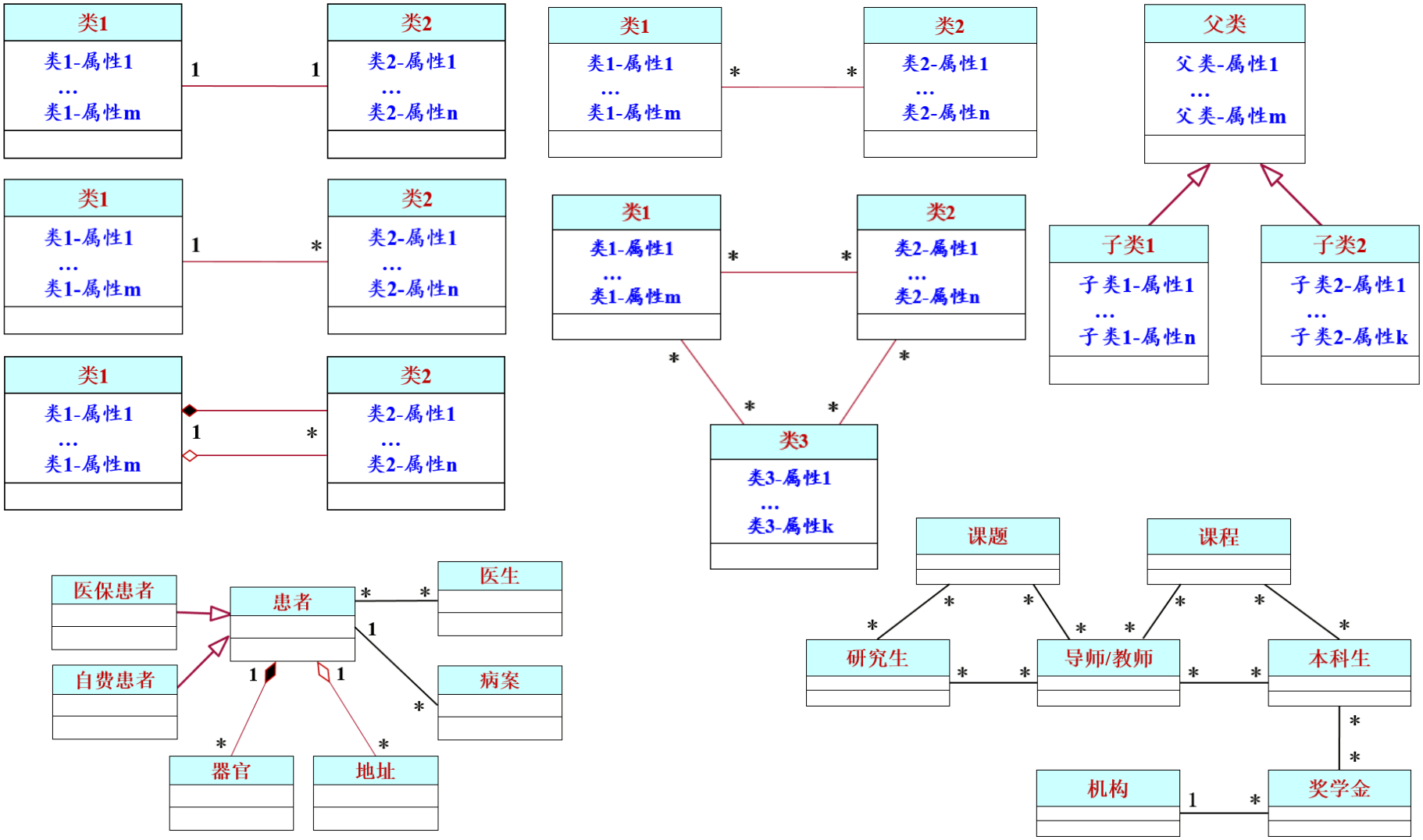
SQL Code

```
CREATE TABLE Person (person_ID int not null,
                     person name char(20) notnull,
                     address char(50),
                     PRIMARY KEY  Person_ID);
```

# 数据库设计

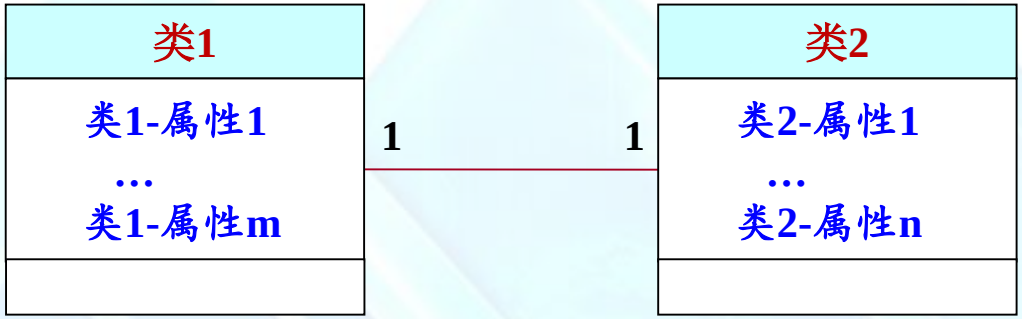
## 将对象映射到关系数据库

- 具有复杂关系的实体类，如何映射到关系数据库中呢？



# 数据库设计

## 实体类图→ERD映射　典型类型一：1:1 关联关系




实例：考生↔报名表

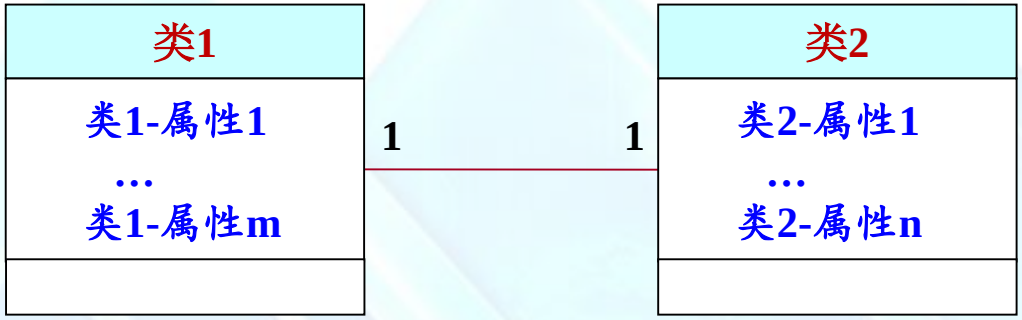
方法1：建立1个表

（1）增加PrimaryKey属性

（2）复制2个类中的所有属性

# 数据库设计

## 实体类图 → ERD映射　典型类型一：1:1 关联关系



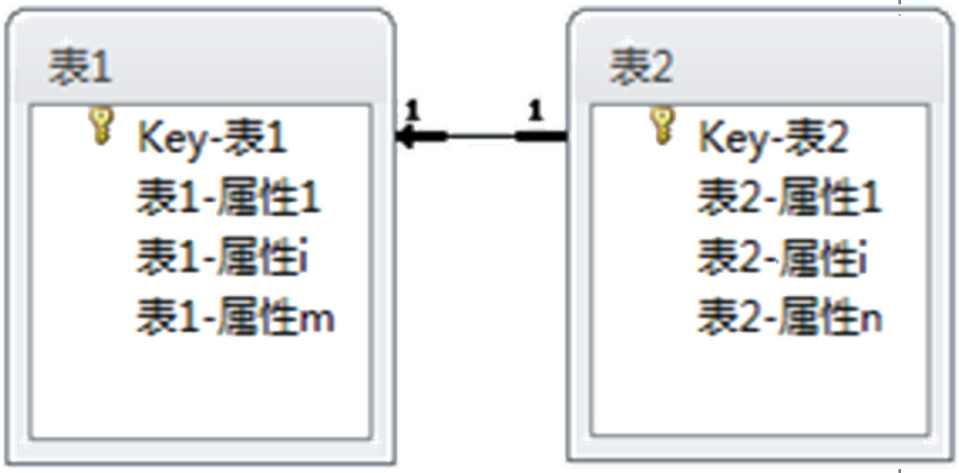

实例：考生 ↔ 报名表

### 方法2：建立1:1关系的2个表

（1）每个表分别增加PrimaryKey

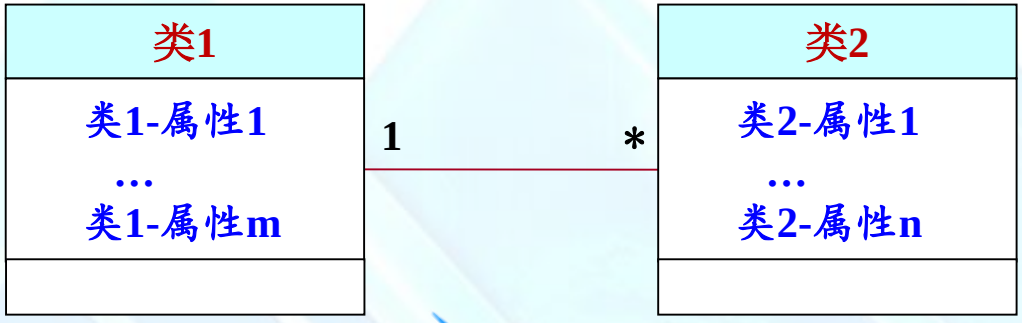
（2）每个表中分别复制对应类中的所有属性

（3）2个表的PrimaryKey建立关联，从而建立参照完整性，任何一个表作为主表均可



# 数据库设计

## 实体类图 → ERD映射　典型类型二：1:* 关联关系




实例：父母 ↔ 子女

**方法：建立1:*关系的2个表**

（1）每个表分别增加PrimaryKey

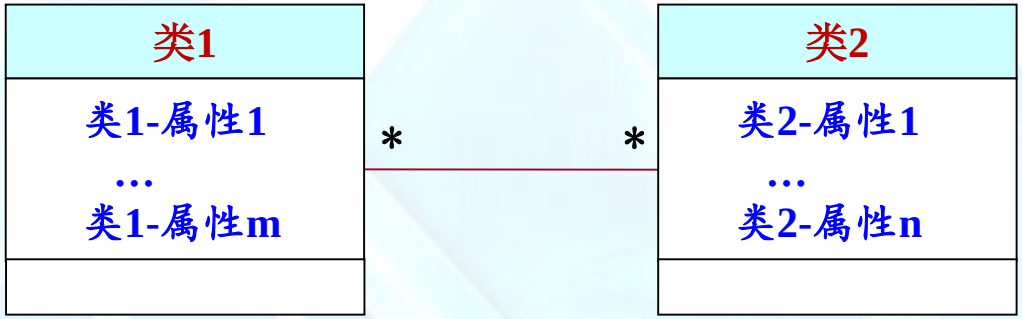
（2）每个表分别复制对应类中的所有属性

（3）表2中增加ForeignKey属性，并与表1的PrimaryKey属性关联，从而建立参照完整性

主表为“基数”为1的那个表

# 数据库设计

## 实体类图 → ERD映射　典型类型三：*:* 关联关系




实例：学生 ↔ 课程

方法：建立3个表，表1、表2分别对应类1、类2，第3个表为关联表，表1、表与关联表之间均为1:*的关系

（1）3个表分别增加PrimaryKey

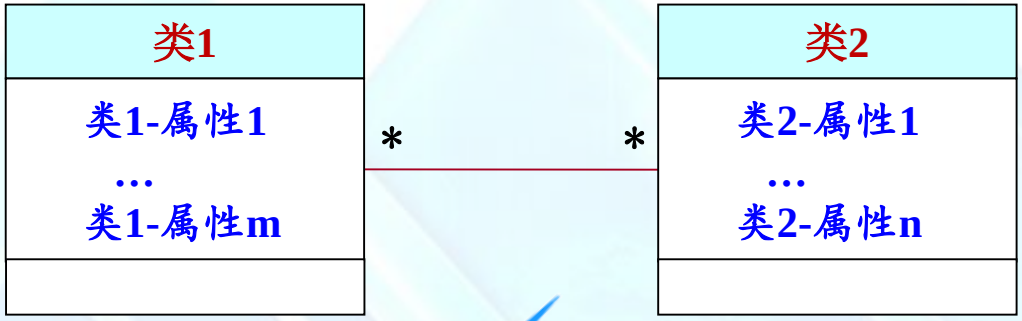
（2）表1、表2的其他属性分别复制类1、类2中的所有属性

（3）关联表中增加2个ForeignKey属性，分别与表1、表2中的PrimaryKey关联，从而建立参照完整性，表1、表2均为主表

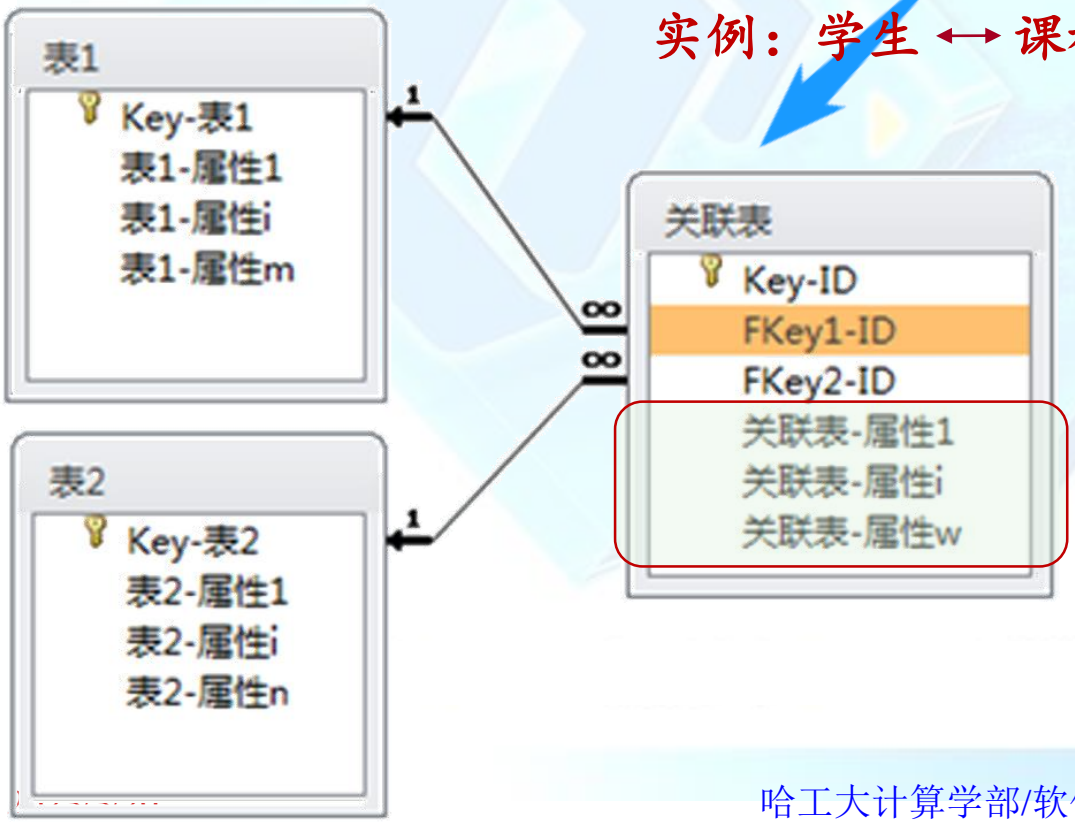
（4）关联表中经常会增加若干其他属性

# 数据库设计

## 实体类图 → ERD映射　典型类型三：*:* 关联关系




实例：学生 ↔ 课程




方法：

（1）3个表分别增加PrimaryKey

（2）表1、表2的其他属性分别复制类1、类2中的所有属性

（3）关联表中增加2个ForeignKey属性，分别与表1、表2中的PrimaryKey关联，从而建立参照完整性，表1、表2均为主表

（4）关联表中经常会增加若干其他属性

# 数据库设计

## 实体类图→ERD映射　典型类型四：循环 *:* 关联关系

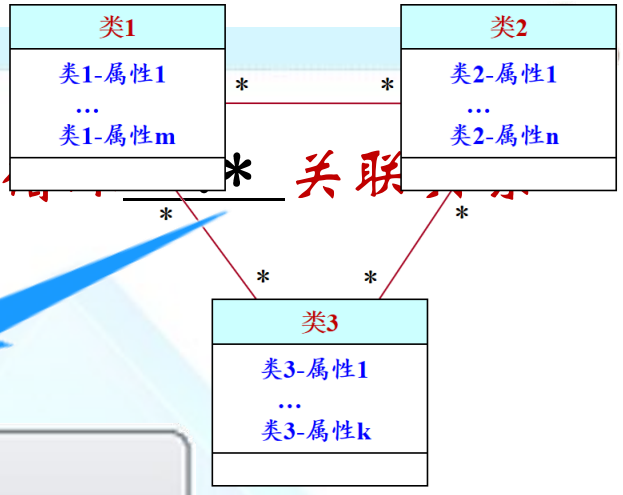
实例：学生↔课程↔教师

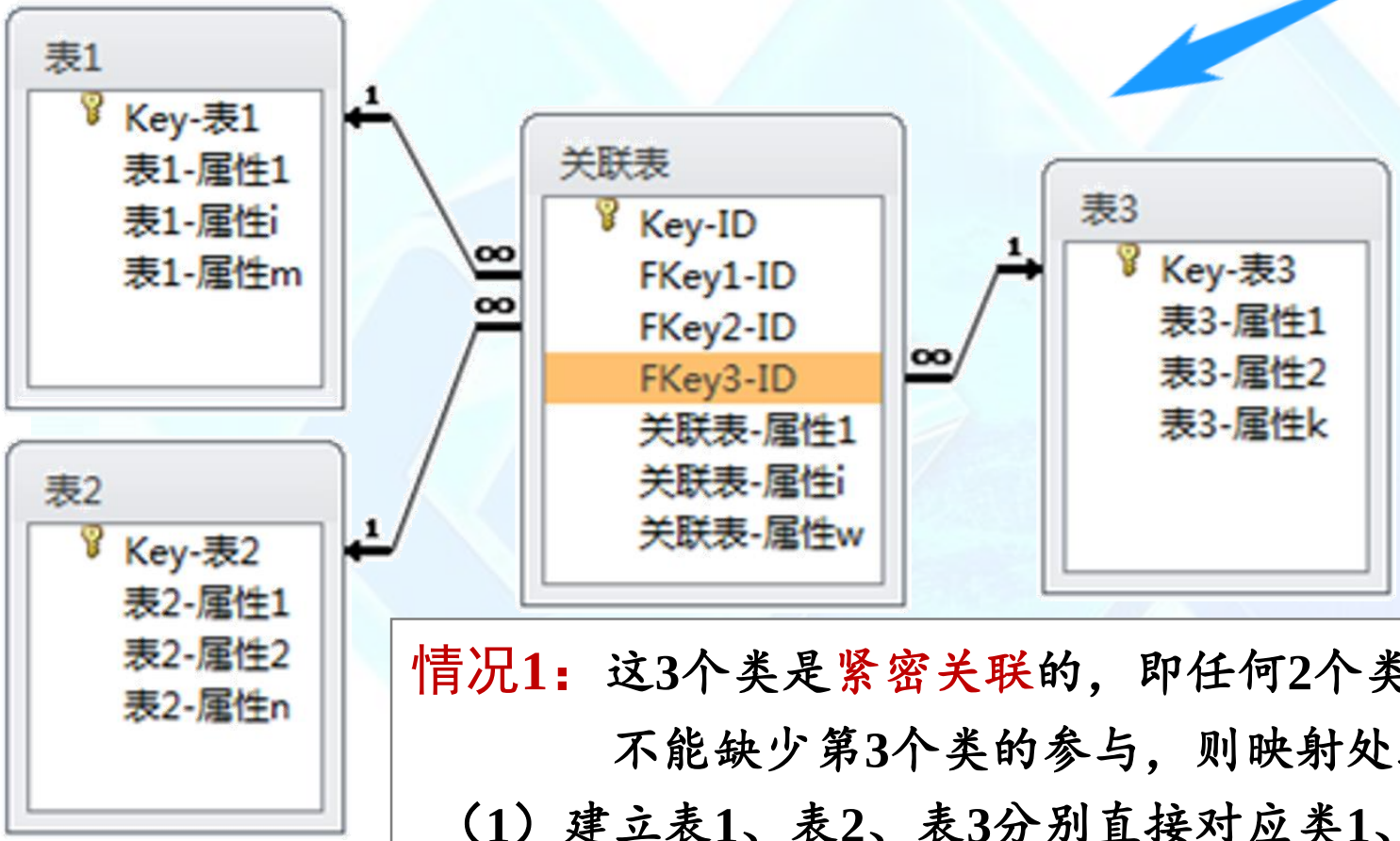
情况1：这3个类是紧密关联的，即任何2个类的关联行为都不能缺少第3个类的参与

情况2：这3个类仅仅是两两相关联

# 数据库设计

## 实体类图 → ERD映射　典型类型四：*:* 关联关系




**情况1：** 这3个类是**紧密关联**的，即任何2个类的关联行为都不能缺少第3个类的参与，则映射处理如下：

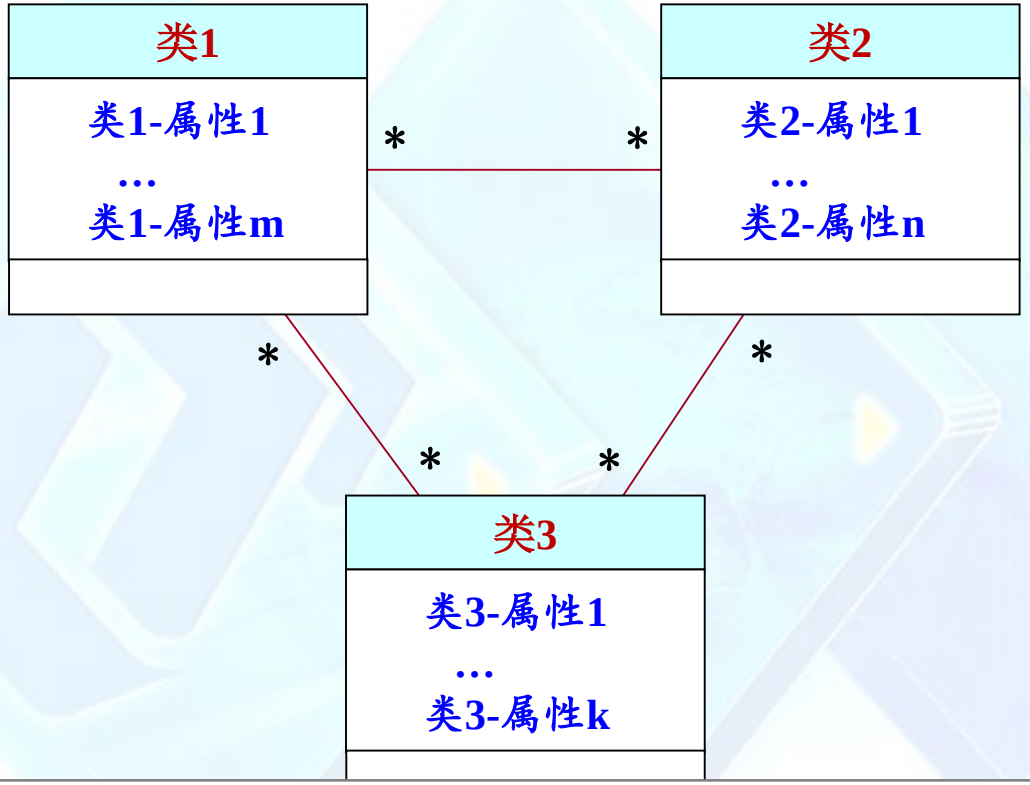
（1）建立表1、表2、表3分别直接对应类1、类2、类3；

（2）建立1个关联表，需要创建3个ForeignKey，分别跟表1～3的PrimaryKey关联，建立参照完整性；

（3）关联表中一般会增加若干其他属性。

# 数据库设计

## 实体类图→ERD映射　典型类型四：循环*:*关联关系

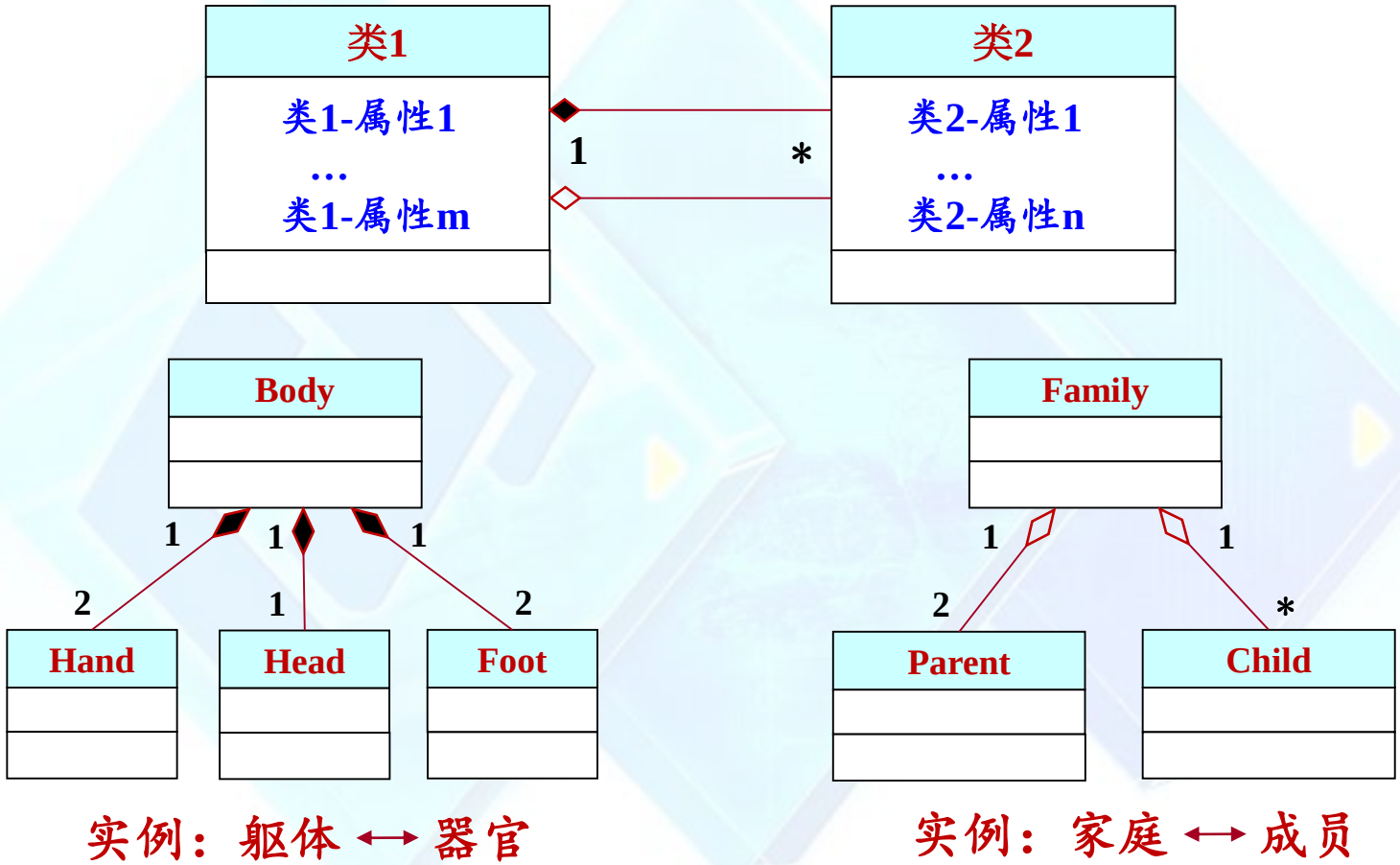



情况2：这3个类仅仅是两两相关联，即该类图包含3个“典型类型三”的情况，即表1-表2、表2-表3、表3-表1，需要创建3个关联表，每对关联的处理方法同“典型类型三”。

例如：学生-课程+成绩、教师-课程+评价、学生-教师+项目。

# 数据库设计

## 实体类图 → ERD映射　典型类型五：组合/聚合关联关系




实例：躯体 ↔ 器官

实例：家庭 ↔ 成员

方法：聚合/组合可以理解为特别紧密的关联关系

在映射到数据库表设计时，参照1:1或1:*关系处理即可

# 数据库设计

## 实体类图→ERD映射　典型类型六：继承关系

实例：雇员←经理/职员
师生←学生/教师

方法1：分别建立父类和子类的3张数据表，形成1:*的类型

方法2：分别建立子类的2张数据表，父类并到子类的表中

方法3：只建立父类的1张数据表，子类并到父类的表中

讨论：（1）从节省空间和访问效率上比较，方法1~3的性能逐次降低

（2）如果属性比较少的情况下，方法2或3还是可以采用的

# 数据库设计

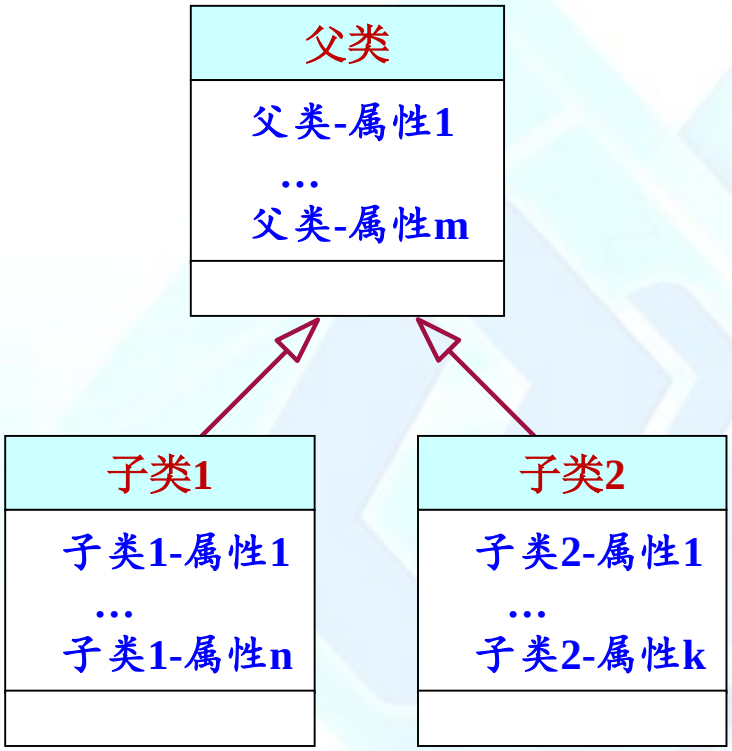
## 实体类图 → ERD映射　典型类型六：继承关系

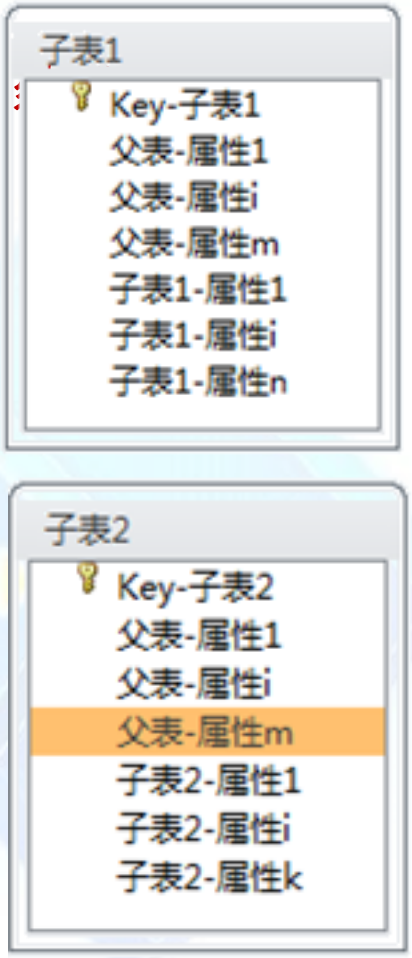
方法1：分别建立父类和子类对应的共3张数据表，形成1:*的映射类型；其中父类表保留父类的全部属性，或者将部分属性（父类与子类是1:1关系的属性）下移到子类对应的表中

讨论：最大程度减少冗余数据的存储

# 数据库设计

## 实体类图 → ERD映射　典型类型六：




方法2：分别建立子类共2张数据表，将父类的属性全部下移到各个子类所对应的数据表中

讨论：（1）会产生一定量冗余数据，浪费存储空间，表的访问效率会降低

（2）当父类中独立的属性较少时，该方案可行

# 数据库设计

## 实体类图→ERD映射　典型类型六：继承关系

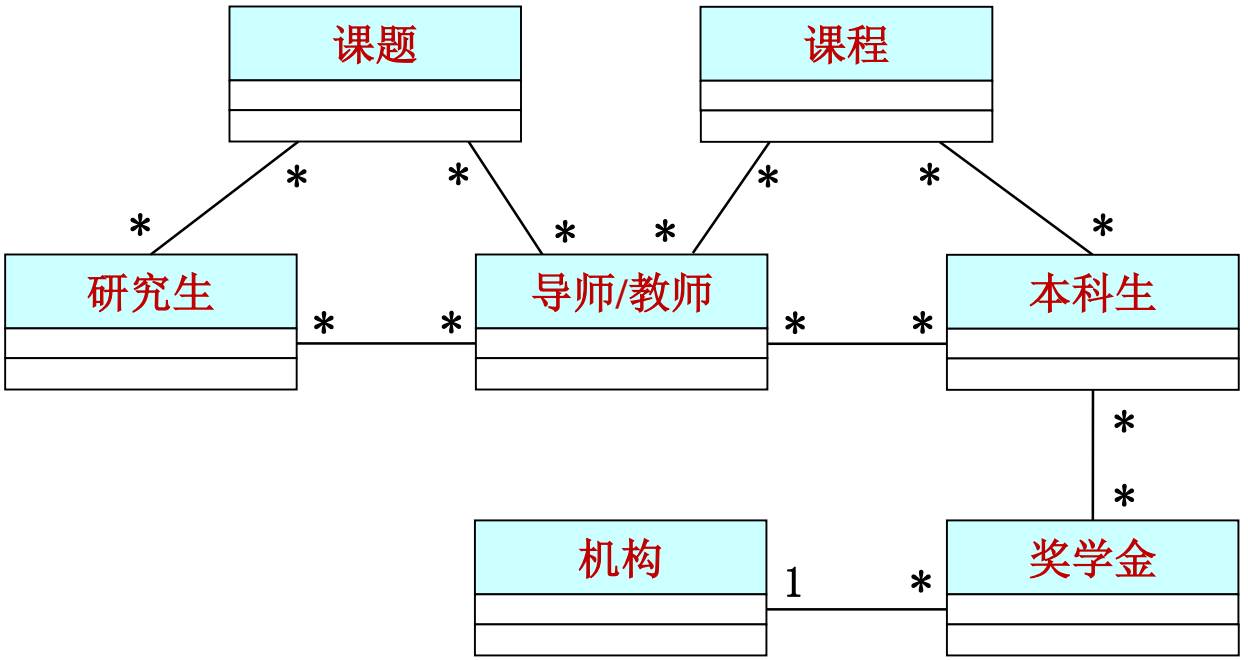
**方法3：** 只建立1张数据表，将子类的属性上移到父类所对应的数据表中，该表包括父类和各子类的全部属性，需增加区别子类的标志性属性

**讨论：**（1）会产生大量的空白数据和冗余数据，比方法2更浪费存储空间，表的访问效率会大大降低

（2）只有子类中属性很少的情况下，该方法才可用

# 数据库设计

## 实体类图 → ERD映射　思考一：复杂 *:* 关联关系



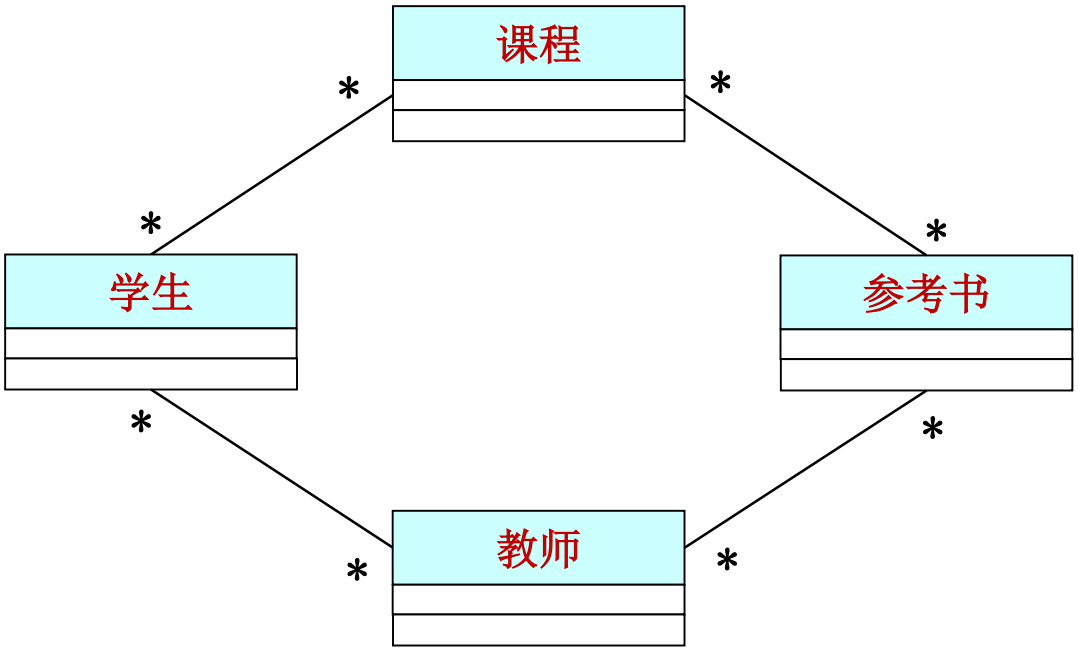

循环1：　导师 -指导- 研究生 -研究- 课题 -申请- 导师

循环2：　教师 -教授- 本科生 -选修- 课程 -讲授- 教师

关联链：本科生 -获得- 奖学金 -设立- 机构

# 数据库设计

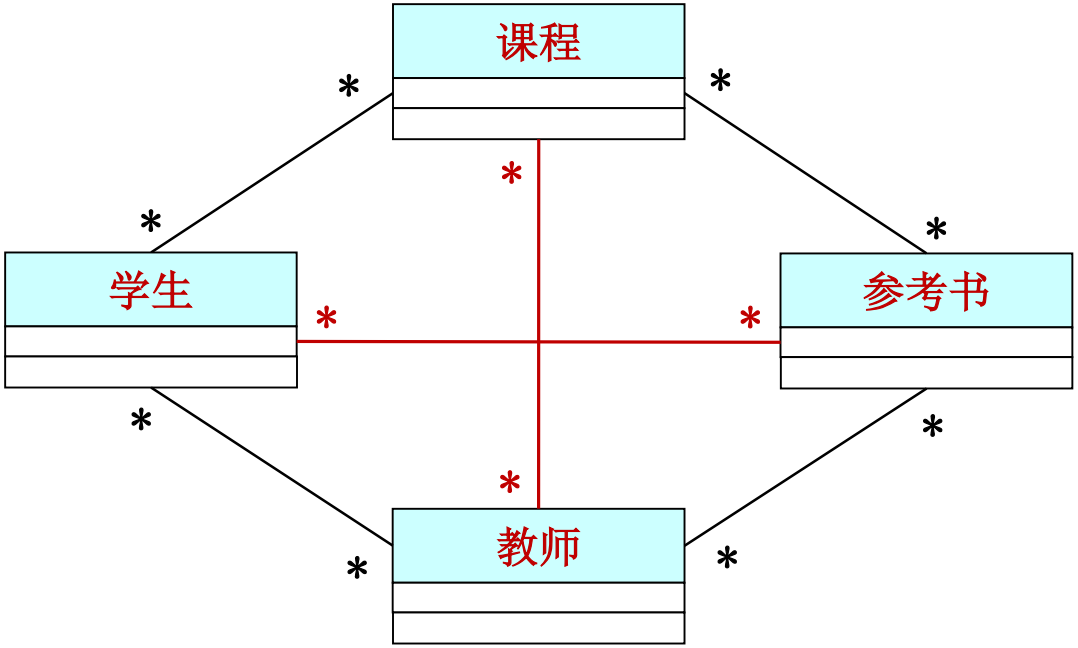
## 实体类图 → ERD映射　思考二：大循环 *:* 关联关系




关系：教师 -指导- 学生 -选修- 课程 -选用- 参考书 -编著- 教师

# 数据库设计

## 实体类图 → ERD映射　思考三：循环交叉 *:* 关联关系




大循环：　教师 -指导- 学生 -选修- 课程 -选用- 参考书 -编著- 教师

小循环1：教师 -指导- 学生 -选修- 课程 -讲授- 教师

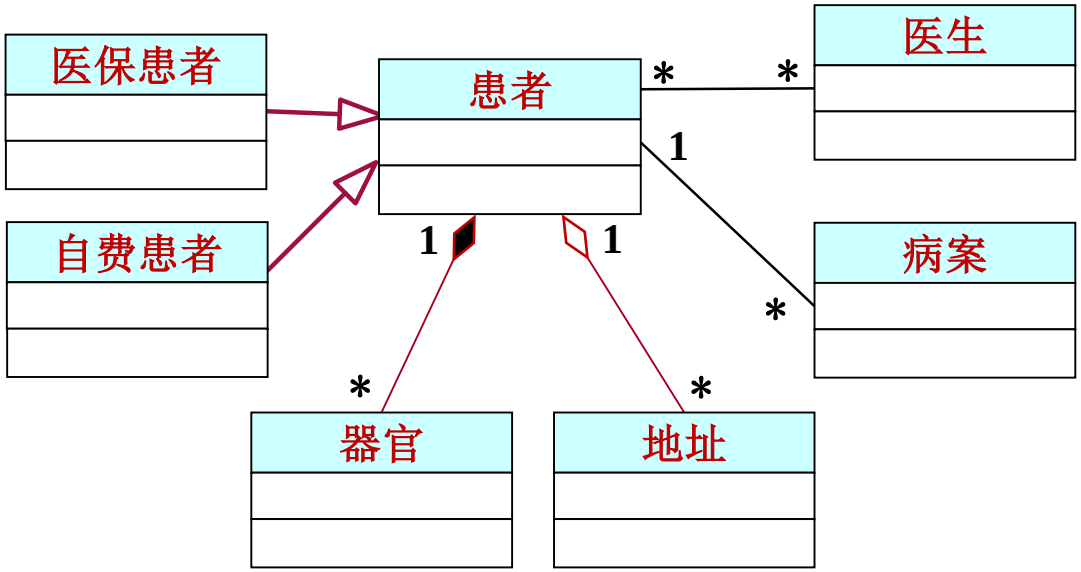
小循环2：教师 -讲授- 课程 -选用- 参考书 -编著- 教师

小循环3：教师 -指导- 学生 -购买- 参考书 -编著- 教师

小循环4：学生 -选修- 课程 -选用- 参考书 -购买- 学生

# 数据库设计

## 实体类图→ERD映射　思考四：多种关系组合




患者：分为医保患者或自费患者，用药范围/结账方式不同

患者：有自然一体的多个器官，有多个隶属关系的地址

患者：可以找多位医生诊病，也可能1次都未去过医院

医生：可以给多位患者诊疗，也可能1次都未做过诊疗

患者：可以有多份或没有病案，每份病案只能对应1位患者

# 数据库设计

## 本章主要内容

1. 数据库系统及关系数据库简介
2. 数据库逻辑模型设计
3. ERD模型及质量评价
4. 物理数据库设计及建立
5. 物理数据库提高效率的技巧
6. 描述分布式数据库的不同的体系结构模型
7. OO实体类图映射到ERD
8. 关系型数据库规范化示例

# 数据库设计

## 数据库规范化例子

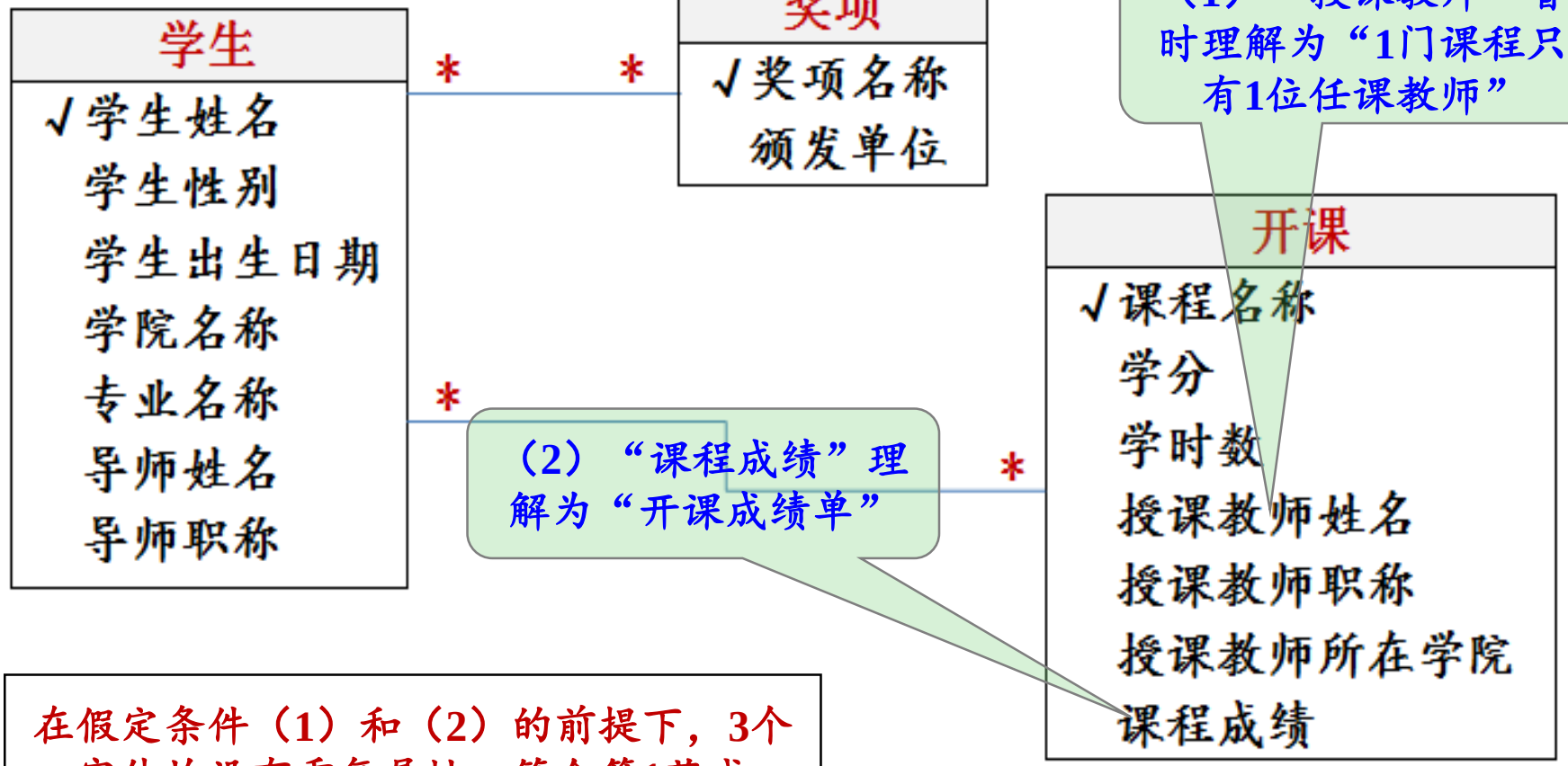
### 第 0 范式

| 学生综合信息 |
| --- |
| √学生姓名 |
| 学生性别 |
| 学生出生日期 |
| 学院名称 |
| 专业名称 |
| 获奖名称(*) |
| 颁发单位(*) |
| 导师姓名 |
| 导师职称 |
| 课程名称(*) |
| 学分(*) |
| 学时数(*) |
| 授课教师姓名(*) |
| 授课教师职称(*) |
| 授课教师所在学院(*) |
| 课程成绩(*) |

# 数据库设计

## 数据库规范化例子

### 第 1 范式



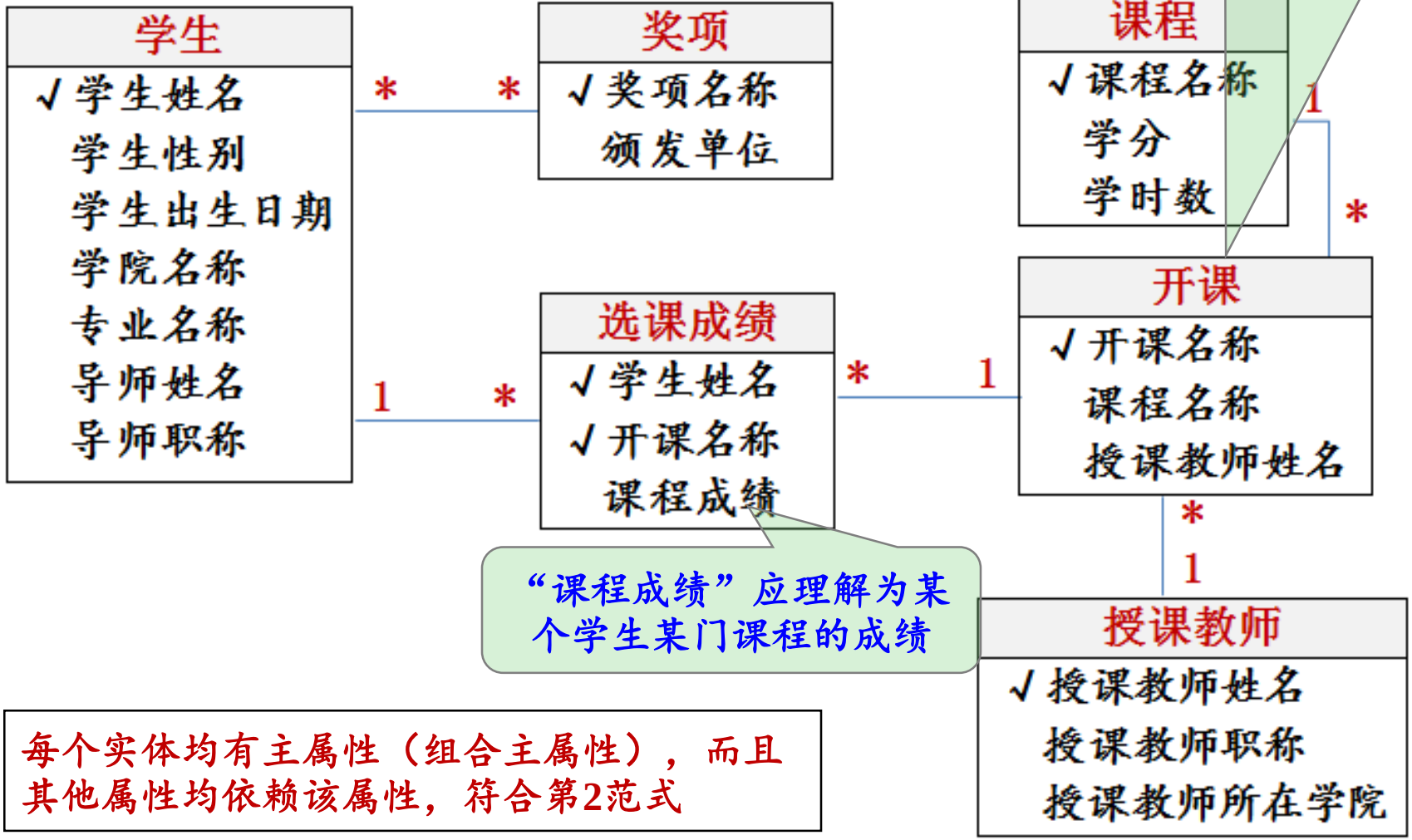

在假定条件（1）和（2）的前提下，3个实体均没有重复属性，符合第1范式

# 数据库设计

## 数据库规范化例子

考虑“课程”与“授课教师”之间应该是“*:*”的关系才符合实际情况，增加一个关联实体才能跟“学生”建立关联关系

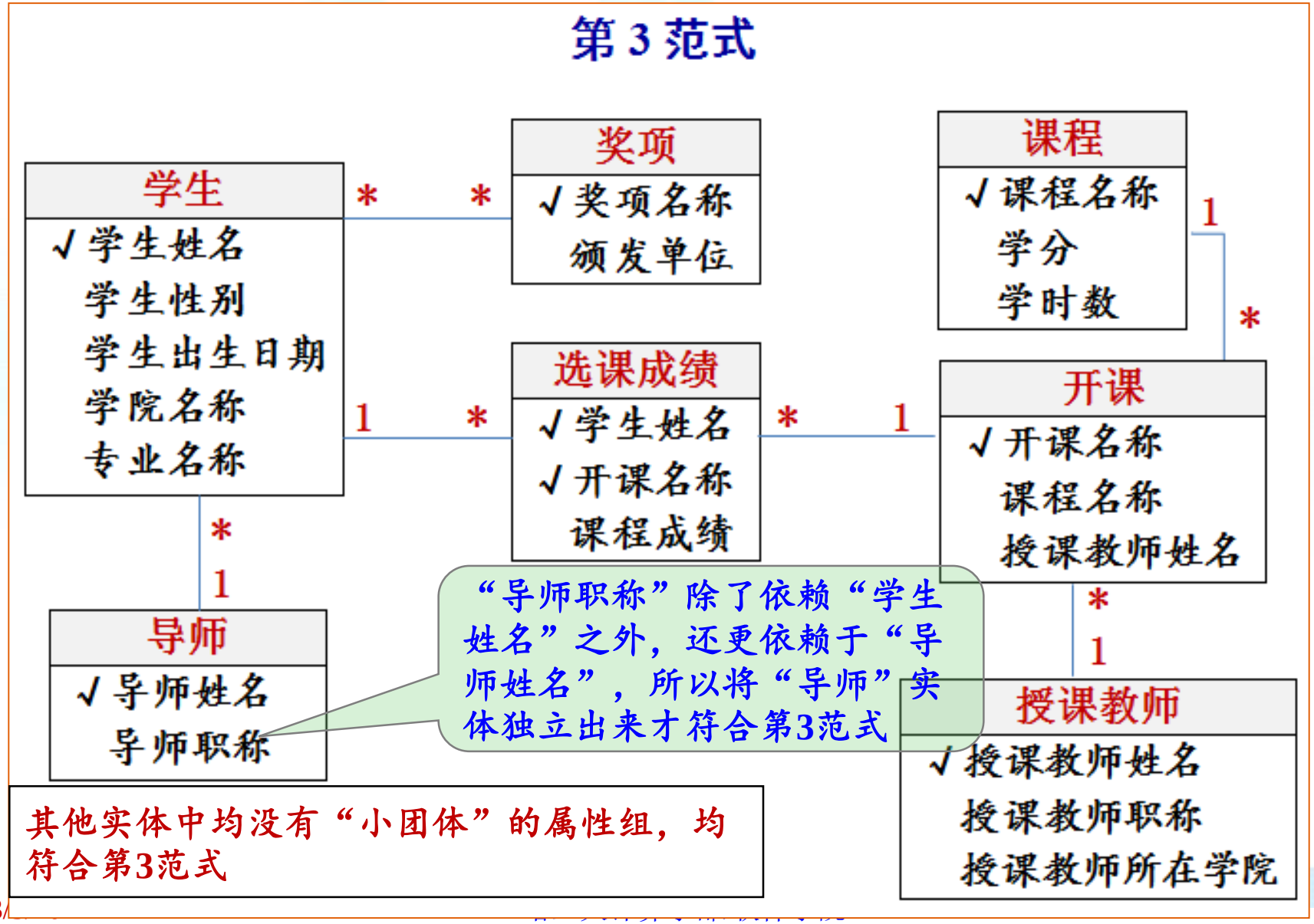
### 第 2 范式




每个实体均有主属性（组合主属性），而且其他属性均依赖该属性，符合第2范式

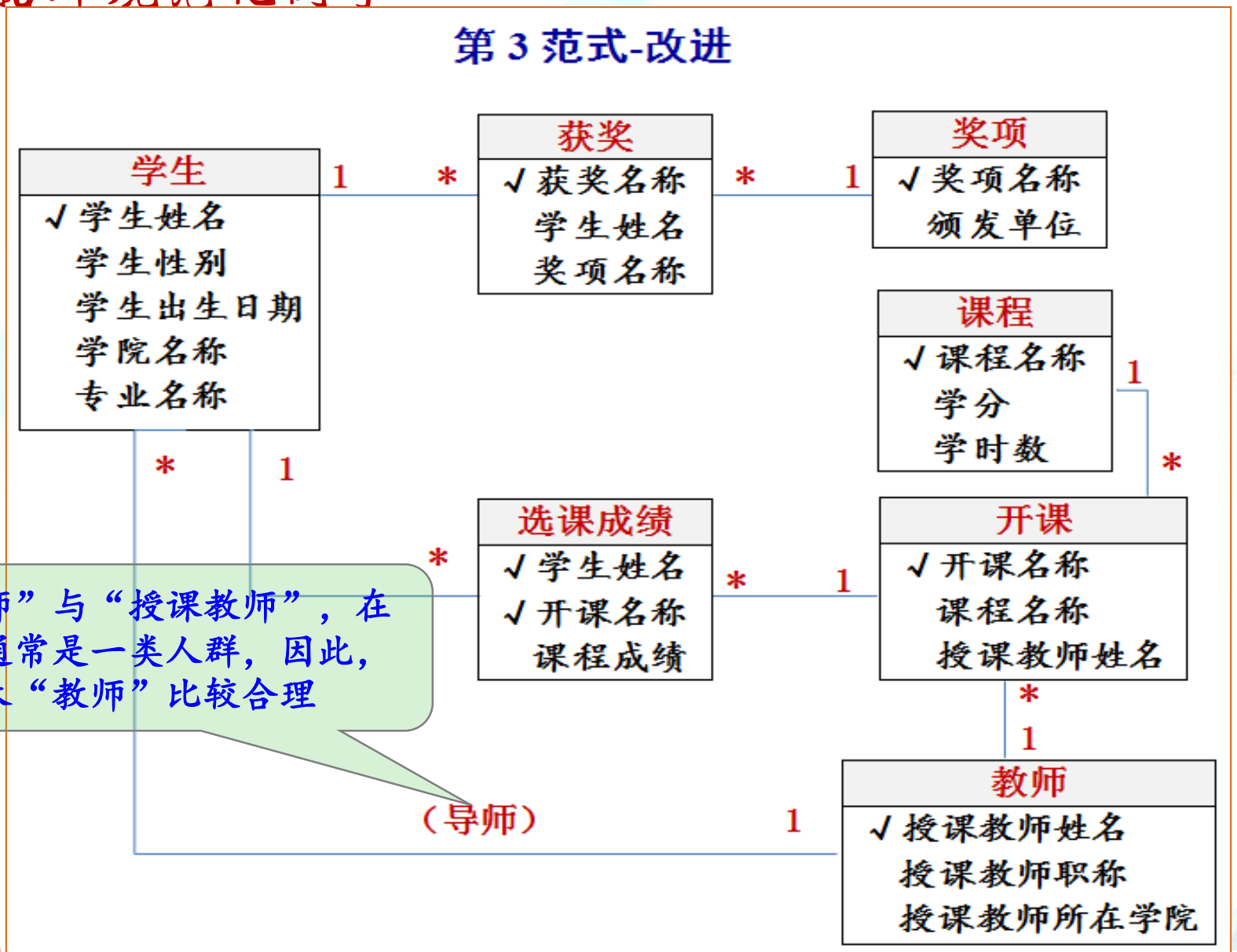
# 数据库设计

## 数据库规范化例子



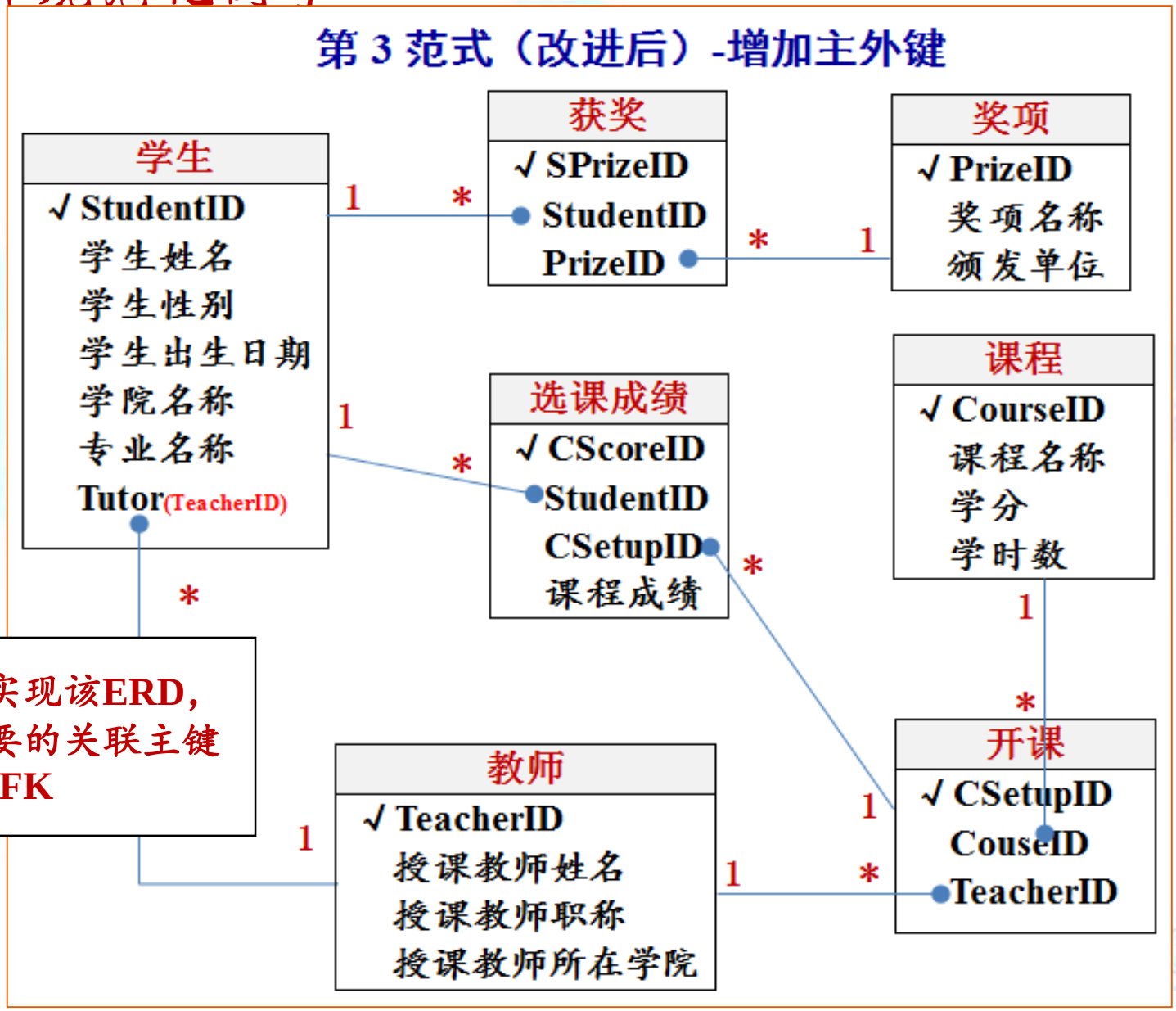
# 数据库设计

## 数据库规范化例子



# 数据库设计

## 数据库规范化例子



# 《系统分析与设计》
# System Analysis and Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(微信同号)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部/

国家示范性软件学院

软件工程教研室

2023.03

# 用户界面设计

## 本章主要内容

1. 用户界面设计的基本概念
2. 用户界面形式及设计原则
3. WIMP用户界面设计
4. *网页界面设计
5. 缺省设计问题
6. 输入验证设计问题
7. 系统响应及信息反馈问题

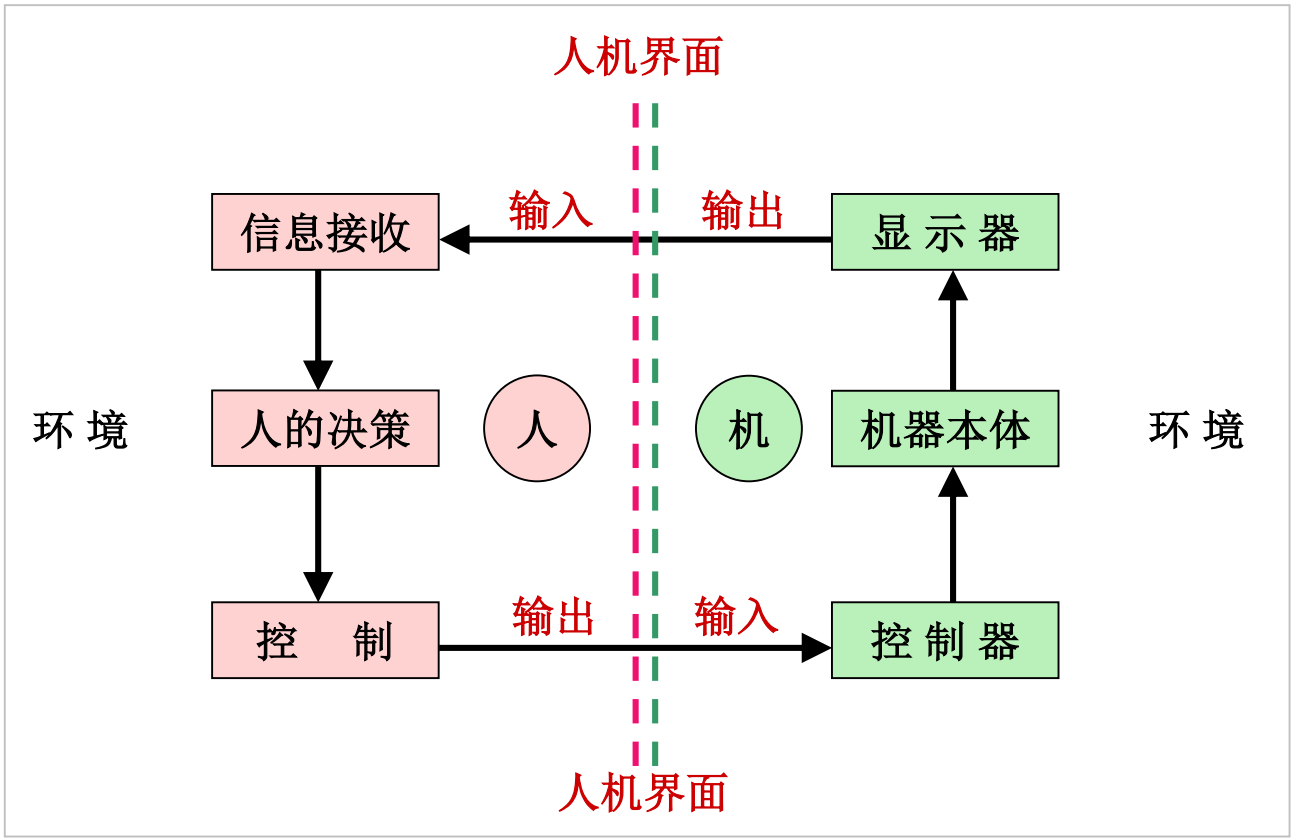
# 用户界面设计

## 本章主要内容

1. 用户界面设计的基本概念
2. 用户界面形式及设计原则
3. WIMP用户界面设计
4. *网页界面设计
5. 缺省设计问题
6. 输入验证设计问题
7. 系统响应及信息反馈问题

# 用户界面设计的基本概念

## 1. 人机系统模型




人机系统模型

# 用户界面设计的基本概念

**2. 广义的人机界面**

在人机系统模型中，人与机器、设备之间存在一个相互作用的“面”，称为人机界面

例如：键盘、显示器、音箱、方向盘、仪表、操纵杆等

说明：

（1）人与机器设备之间的信息交流和控制活动都发生在人机界面上

（2）机器的各种显示都“作用”于人，实现机 → 人信息传递

（3）人通过视觉、听觉等感官接受来自机器的信息，经头脑加工、决策，做出反应，可以再控制机器设备

# 用户界面设计的基本概念

**3. 狭义的人机界面（人机接口、用户界面）**

狭义的人机界面是特指计算机系统中的人机界面（**HCI-Human-Computer Interface**），又称为人机接口、用户界面（**User Interface**）

例如：键盘、鼠标、显示器、软件界面等

说明：

（1）计算机是目前最具智能的机器，应用范围越来越广，几乎无所不包

（2）硬件人机界面—键盘、鼠标、音箱、显示器、游戏杆、打印机等，变化速度相对较慢

（3）软件人机界面可塑性较强，变化无穷，是当今人们研究人机界面最活跃的领域

注：用户界面 — 一般指软件人机界面

# 用户界面设计的基本概念

**4. 用户界面的内容**

软件用户界面一般包含：

（1）界面元素的样式、大小、布局

（2）界面的美观效果（色彩、风格、图像）

（3）操作序列（或功能）的隐喻

（4）交互信息（输入、输出、验证、提示）

**5. 什么是用户界面设计**

运用认知心理学、人机工程学、语言学等基础知识对用户界面完成结构设计、交互设计和视觉设计，从而使软件达到易学、易用、美观、高效的目的

# 用户界面设计的基本概念

**6.** 用户界面设计的基本过程

（1）研究用户，分析软件的需求

（2）用户界面结构设计（采用低保真的原型）

（3）用户界面的交互设计（采用高保真的原型）

（4）用户界面设计方案评审

（5）迭代修改

**7.** 美术设计师、UI设计师、软件设计师的区别

- 美术设计师：关注界面的美观、修饰
- UI 设 计 师：关注界面的可用性和用户体验
- 软件设计师：关注软件界面的实现

# 用户界面设计的基本概念

**8.** 用户界面设计理念

- 以用户为中心、方便用户、易用高效

**9.** 用户界面设计标准

- 可学性：简单易学，用户能很快开始工作
- 效率性：使用效率高，用户能高效率工作
- 记忆性：使用方法容易记忆，减少学习时间
- 容错性：错误率低，避免灾难性操作错误发生
- 满意度：用户能带着愉悦的心情去工作

# 用户界面设计

## 本章主要内容

1. 用户界面设计的基本概念
2. 用户界面形式及设计原则
3. WIMP用户界面设计
4. *网页界面设计
5. 缺省设计问题
6. 输入验证设计问题
7. 系统响应及信息反馈问题

# 用户界面形式及设计原则

（1）用户界面分类

（2）用户界面风格

（3）用户界面形式及设计原则

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

按照计算机发展的历程，软件用户界面可以分为三代：

第一代：CUI－字符用户界面

第二代：GUI－图形用户界面

第三代：MUI－多通道用户界面

**CUI → GUI → MUI**

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

第一代：CUI－字符用户界面

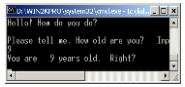
CUI－Character User Interface

描述：以字符为基础的简单交互界面

如常见的命令行，字符菜单等

人机交互设备：以键盘为主

分类：1.问答式界面



2.命令式界面

3.菜单式界面

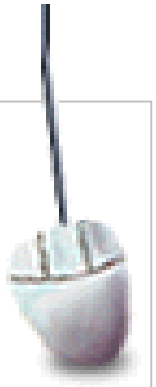
优点：容易设计，效率高

缺点：界面死板单调，记忆量大

不宜非专业人员使用

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类



第二代：GUI—图形用户界面

GUI— Graphics User Interface

描述：以图形为基础的直接操纵用户界面

最新称为WIMP界面

WIMP—Windows, Icons, Menus, Pointers

—Windows, Icons, Mouse, Pull-down menu

人机交互设备：以鼠标为主，键盘为辅

优点：直观、美观、易用

缺点：实现复杂，资源耗费大

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

第三代：MUI－多通道用户界面

MUI－Multimodal User Interface

描述：以多媒体为基础的智能的多通道用户界面

人机交互设备：麦克风、手写板、摄像头、音箱等

优点：智能化，人性化，使用更方便

缺点：资源耗费大，成本高，不易普及

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

从应用软件功能角度分类：

- 操作系统用户界面
- 文字处理软件用户界面
- 工业控制系统用户界面
- 管理信息系统用户界面
- 网页用户界面
- 移动工具用户界面
- 游戏软件用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

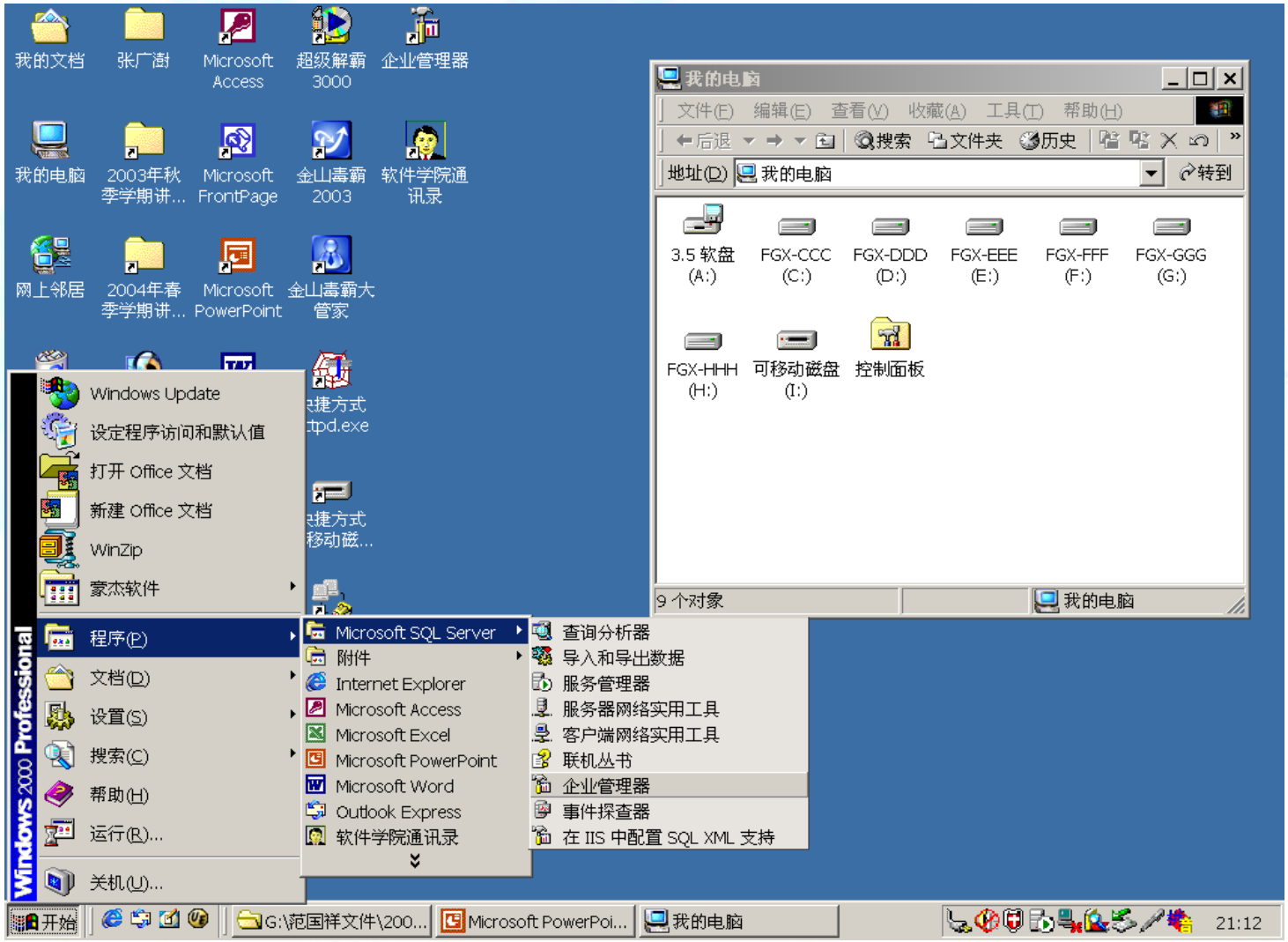
从应用软件功能角度分类：

- **操作系统用户界面**
- 文字处理软件用户界面
- 工业控制系统用户界面
- 管理信息系统用户界面
- 网页用户界面
- 移动工具用户界面
- 游戏软件用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

操作系统用户界面



# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

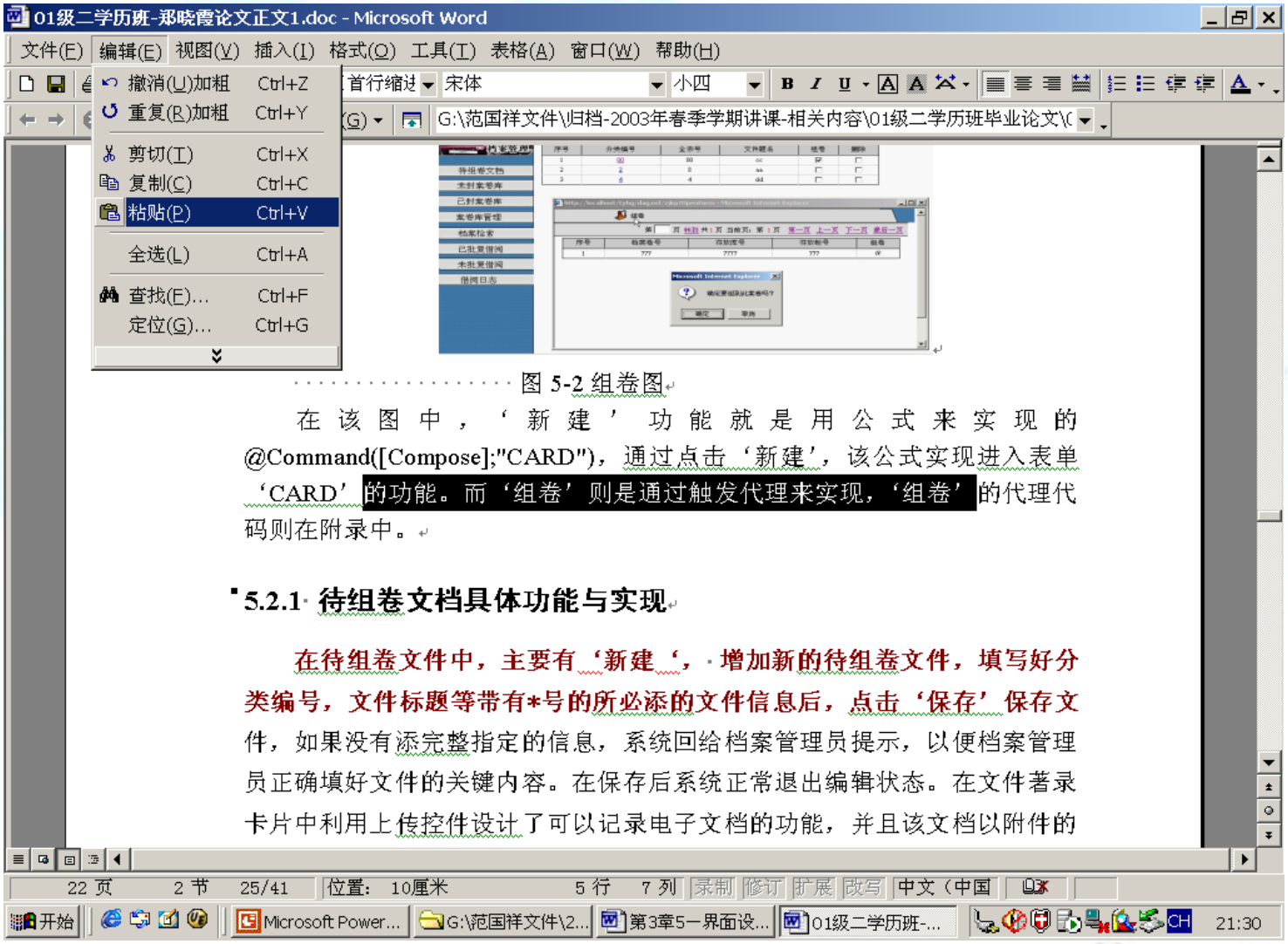
从应用软件功能角度分类：

- 操作系统用户界面
- **文字处理软件用户界面**
- 工业控制系统用户界面
- 管理信息系统用户界面
- 网页用户界面
- 移动工具用户界面
- 游戏软件用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

文字处理软件用户界面



# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

从应用软件功能角度分类：

- 操作系统用户界面
- 文字处理软件用户界面
- **工业控制系统用户界面**
- 管理信息系统用户界面
- 网页用户界面
- 移动工具用户界面
- 游戏软件用户界面

# 用户界面设计

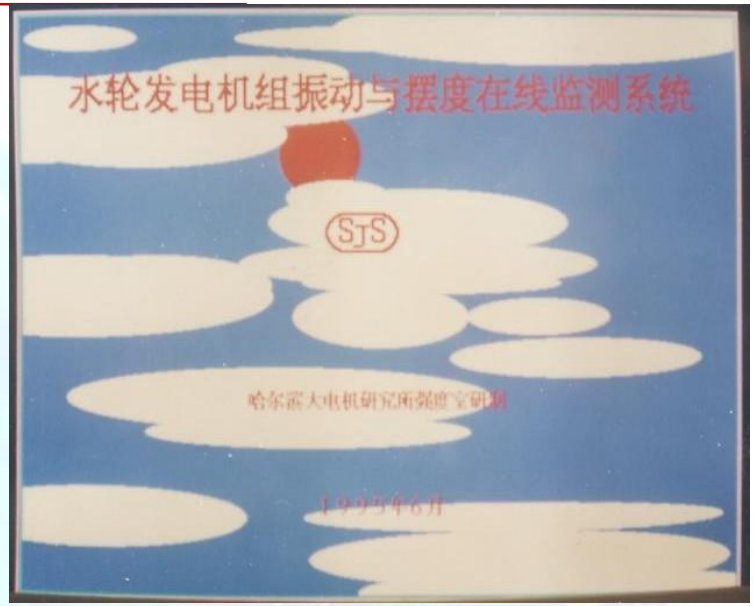
用户界面形式及设计原则，用户界面分类

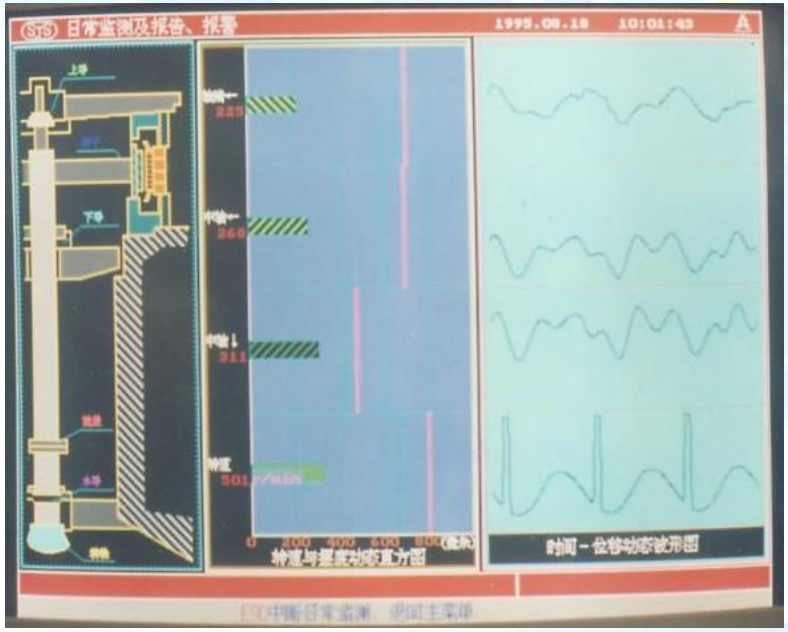
工业控制系统用户界面

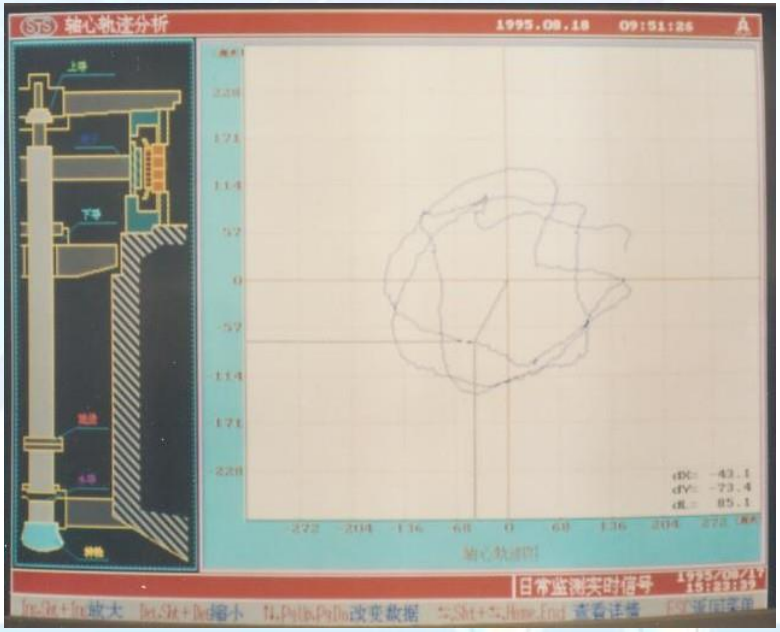
# 用户界面设计

用户界面：

工业控制系统用户界面



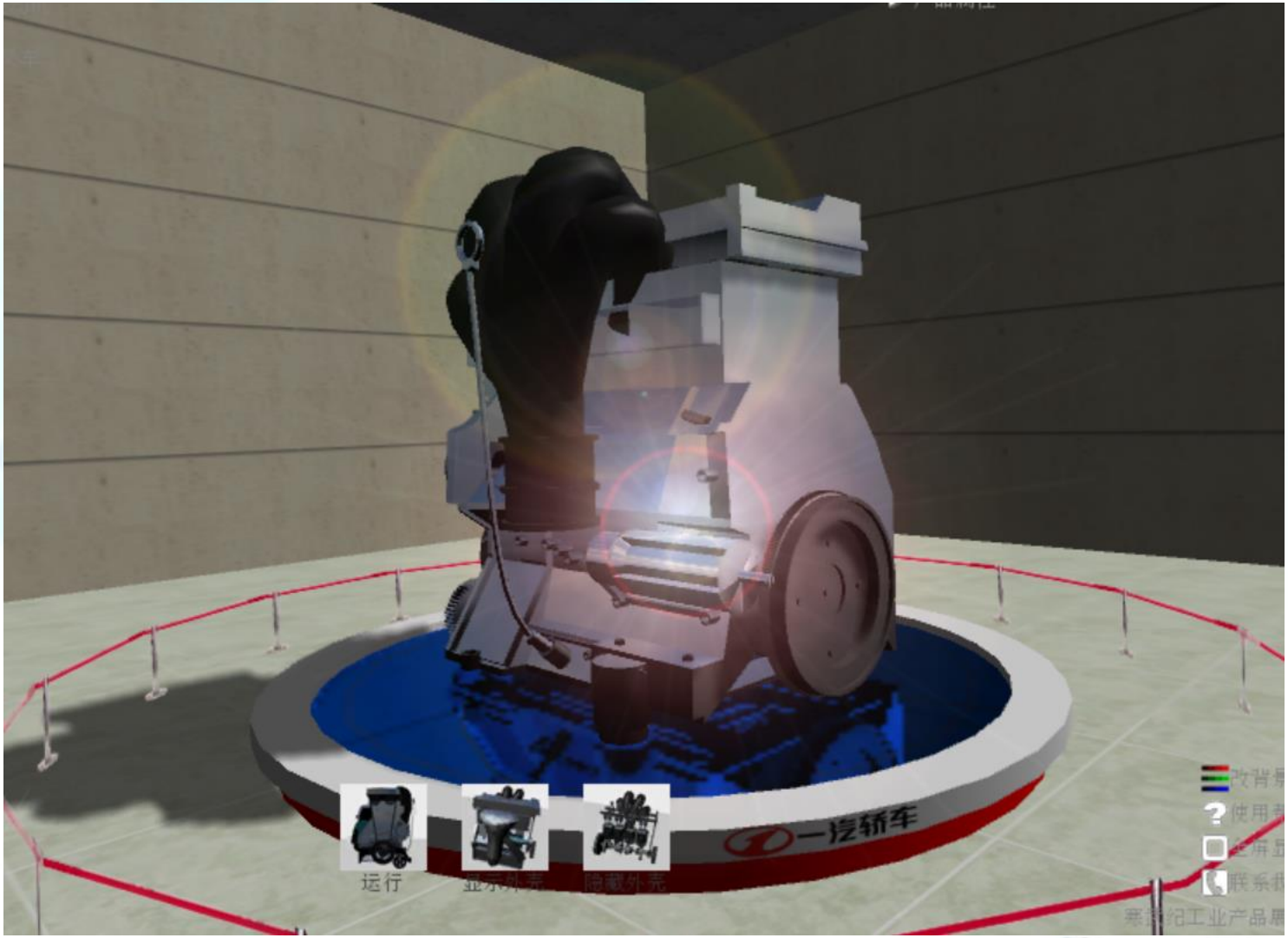



# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

工业控制系统用户界面



# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

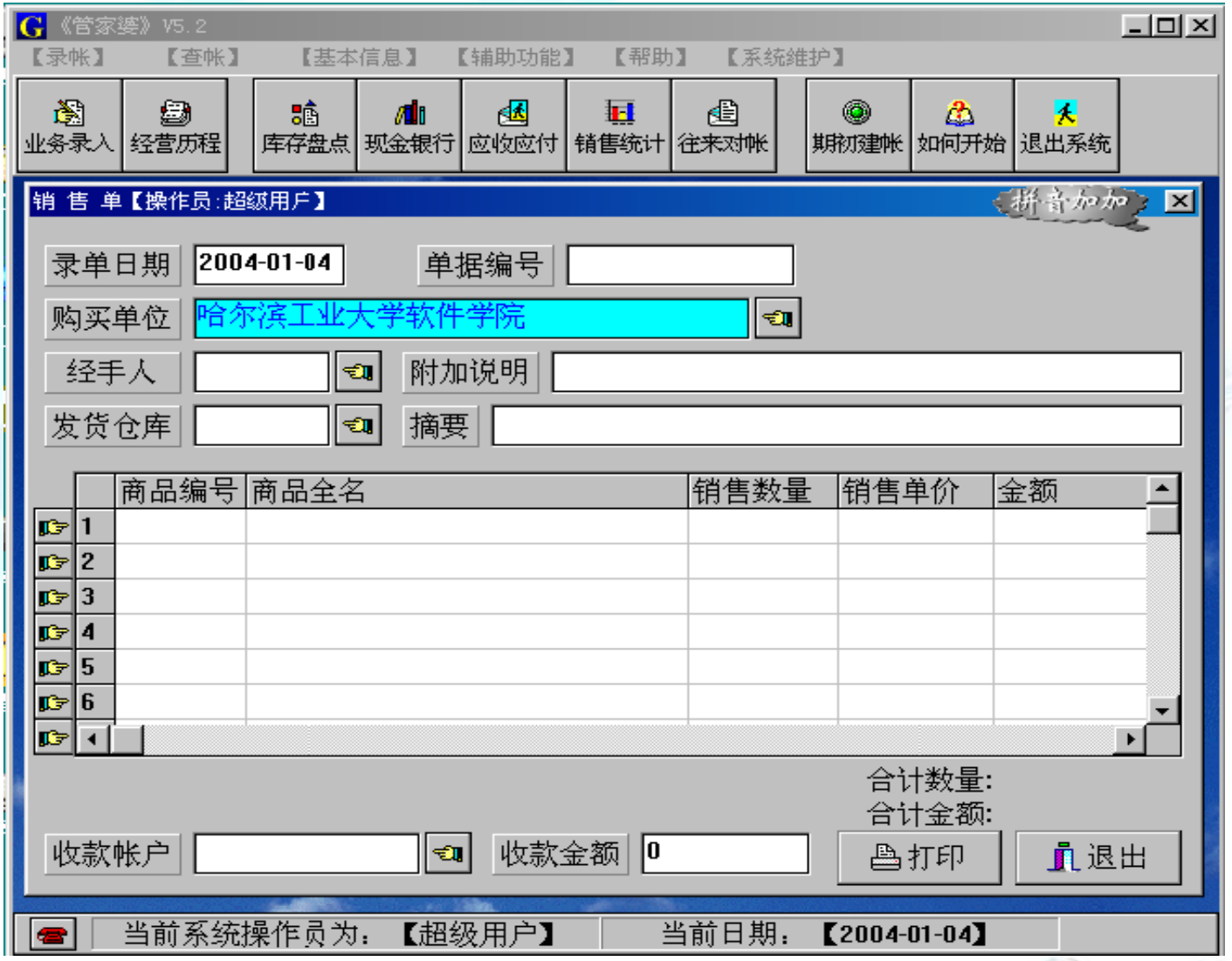
从应用软件功能角度分类：

- 操作系统用户界面
- 文字处理软件用户界面
- 工业控制系统用户界面
- **管理信息系统用户界面**
- 网页用户界面
- 移动工具用户界面
- 游戏软件用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

管理信息系统用户界面



# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

从应用软件功能角度分类：

- 操作系统用户界面
- 文字处理软件用户界面
- 工业控制系统用户界面
- 管理信息系统用户界面
- **网页用户界面**
- 移动工具用户界面
- 游戏软件用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

网页用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

从应用软件功能角度分类：

- 操作系统用户界面
- 文字处理软件用户界面
- 工业控制系统用户界面
- 管理信息系统用户界面
- 网页用户界面
- **移动工具用户界面**
- 游戏软件用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

移动工具用户界面

从应用软件功能角度分类：

- 操作系统用户界面
- 文字处理软件用户界面
- 工业控制系统用户界面
- 管理信息系统用户界面
- 网页用户界面
- **移动工具用户界面**
- 游戏软件用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

从应用软件功能角度分类：

- 操作系统用户界面
- 文字处理软件用户界面
- 工业控制系统用户界面
- 管理信息系统用户界面
- 网页用户界面
- 移动工具用户界面
- **游戏软件用户界面**

# 用户界面设计

## 用户界面形式及设计原则：用户界面分类

游戏软件用户界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

从视觉感官上，图形化界面风格

一致性风格

个性化风格

传统式风格

古典式风格

现代派风格

高科技风格

卡通式风格

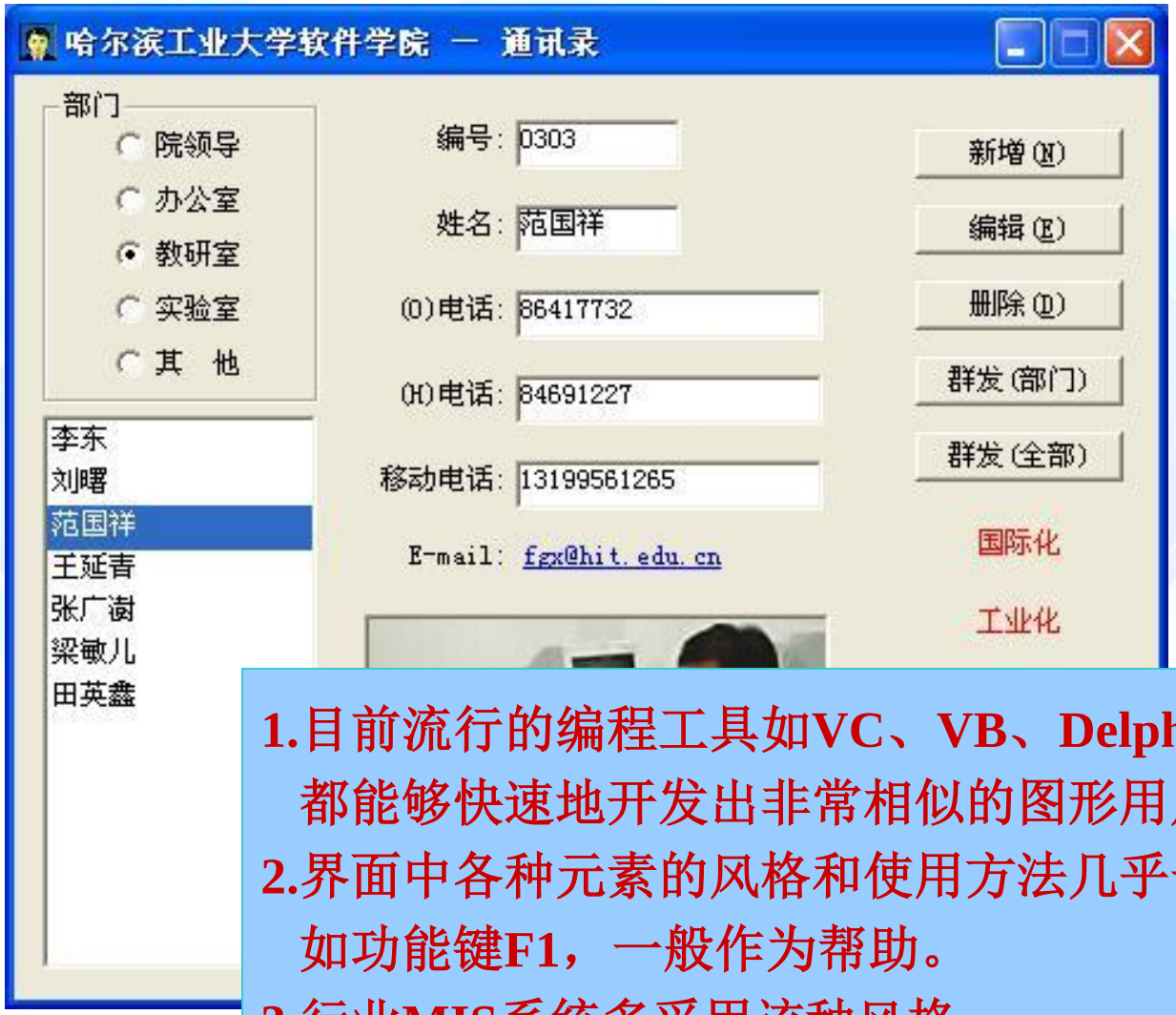
插画式风格

……

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

一致性风格（形式一致）




1.目前流行的编程工具如VC、VB、Delphi、PB等都能够快速地开发出非常相似的图形用户界面。

2.界面中各种元素的风格和使用方法几乎一致。如功能键F1，一般作为帮助。

3.行业MIS系统多采用该种风格

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

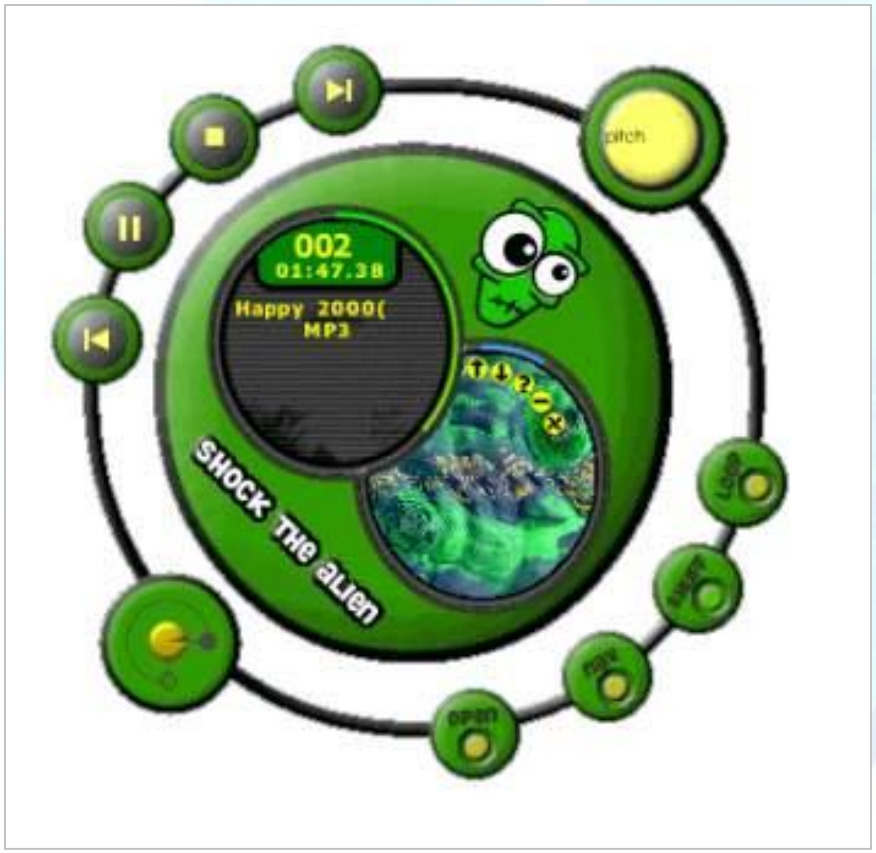
一致性风格（内容一致）

由于早期“钱龙股票分析”软件形成垄断地位致使后起的软件必须在形式和内容的大部分模仿它，与其保持一致，才能得到用户认可

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

个性化风格



音乐播放器软件（2款）

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

### 传统式风格（形式传统）

**VS.**
**个性化风格界面**

**超级解霸－音频&视频播放器软件**

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

传统式风格（内容传统，不做过多夸张）

咖啡宣传网页界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

古典式风格（形式与主题相一致）

光盘播放初始界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

古典式风格

网络游戏—江湖—界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

现代派风格（富于动感，有活力）

**Macintosh OS桌面**

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

现代派风格（清秀、典雅、明快，有现代感）

日本制作的媒体播放软件界面

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

高科技风格（以3D动画、虚拟现实等方式展示界面）

Intel公司服务网站页面

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

卡通式风格

**Disney中国公司网站页面**

# 用户界面设计

## 用户界面形式及设计原则：用户界面风格

插画式风格（油画、蜡笔画、彩铅笔画、水彩画等构成的界面）

适合儿童学习、上网的集成环境

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

**用户界面形式（5种）：**

1. 命令语言界面
2. 菜单界面
3. 数据输入界面
4. 直接操纵和WIMP界面
5. 提示、帮助和出错界面

**用户界面设计总原则（12条）：**

1. 一致性原则
2. 提供信息提示与反馈 [反例]
3. 合理利用空间、保持界面简洁
4. 合理利用颜色与显示效果
5. 实现内容与形式的统一
6. 恰当使用图形和形象比喻
7. 用户出错宽容性
8. 尽量提供快捷方式
9. 尽量允许操作可逆性（Undo与Redo）
10. 尽量减少用户的记忆性要求
11. 合理的系统响应和低的系统成本
12. 良好的联机帮助（Help功能）

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

1. 命令语言界面设计
2. 菜单界面设计
3. 数据输入界面设计
4. 直接操纵和WIMP界面设计
5. 提示、帮助和出错界面设计

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

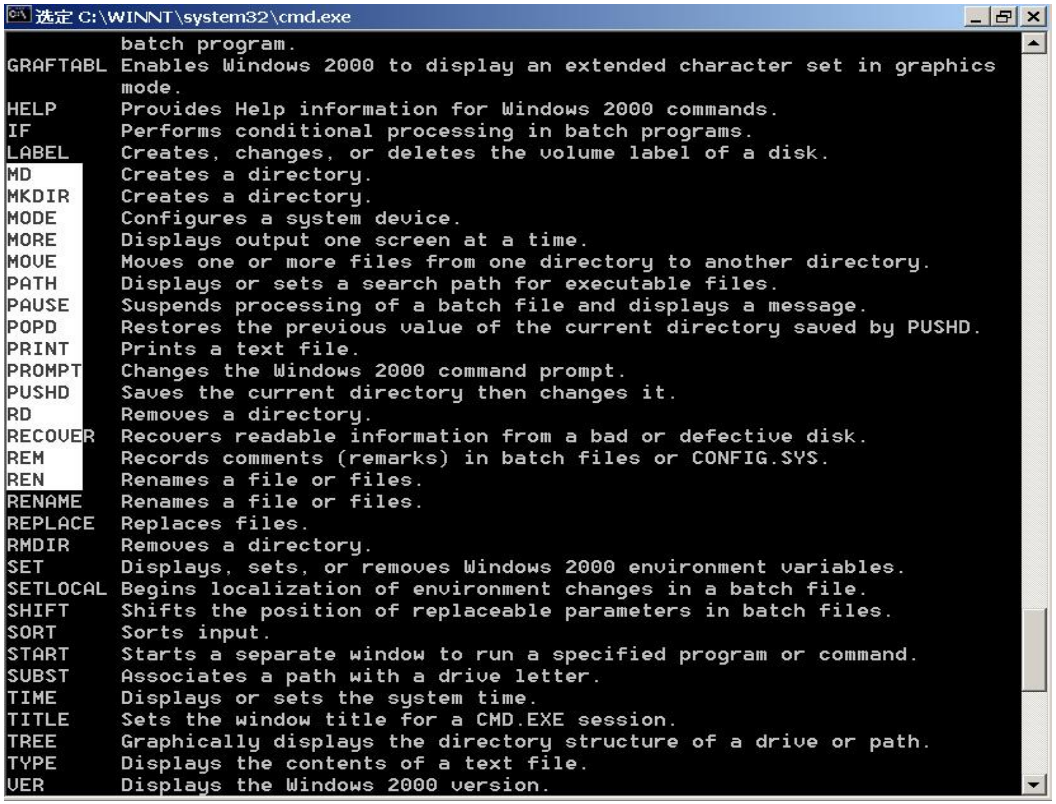
1. 命令语言界面设计
2. 菜单界面设计
3. 数据输入界面设计
4. 直接操纵和WIMP界面设计
5. 提示、帮助和出错界面设计

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 命令语言界面设计

命令语言是交互计算机系统最早使用也是最流行的一种控制系统运行的人机界面形式，并已广泛应用于各类系统应用软件中。



# 用户界面形式及设计原则：形式与原则

## 命令语言界面设计

**1.** 命令语言界面优缺点

优　点：

- 功能强大：一条命令语言语句可以完成需求菜单界面多次转移或多次问答式对话才能完成的系统功能
- 灵活性好
- 效率高：命令越复杂，则越能体现其快速高效的特征
- 占用屏幕空间少

缺　点：

- 难以学习和记忆
- 需要一定的键盘输入技巧
- 出错可能性大

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 命令语言界面设计

2. 命令语言的结构

推理机例1

推理机例2

- 关键词 如 **DIR**
- 关键词 + 参数
  **FORMAT D: /s**
  **XCOPY C:\FAN*.* D:\ZHANG /E/S**
- 基于语法的语言
  **DELETE FILE 、EDIT HELLO.C**
- 基于自然语言
  **Please delete file aaa.doc**

命令的缩写通常有以下做法：

- 简单的截取，即取命令开头几个字母，如：**dir、del**
- 去除元音截取，如：**lst、ls(list)**
- 取第一个和最后一个字母，如：**st(sort)**
- 短语中每个词的首字母，如：**rd、md**
- 着重于发音，例如：**XQT表示execute**

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 命令语言界面设计

**3.** 命令语言界面设计原则

- 命令词要简单、易记

  **DIR，SET，EDIT**

- 命令一致性和普及性

  **EXIT/QUIT、易学人群大**

- 命令词和参数应清晰并符合语法习惯

  **COPY SourceFile DestinationFile**

- 避免不必要的复杂性
- 允许命令串的重现和修改

  **DOS中的F3或↑↓**

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

1. 命令语言界面设计
2. 菜单界面设计
3. 数据输入界面设计
4. 直接操纵和WIMP界面设计
5. 提示、帮助和出错界面设计

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 菜单界面设计

菜单界面也是一种最流行的控制系统运行的人机界面，并已广泛应用于各类系统软件及应用软件中。菜单界面是系统驱动的，它提供各种选择菜单项让用户进行选择，用户不必记忆应用功能命令。

**1. 菜单界面的特点**

- 菜单形式对话是计算机系统驱动的
- 菜单界面适合于结构化的系统，每一菜单项都可以对应于一个子程序功能或下一级菜单
- 菜单界面减轻了用户学习培训、记忆负担，并简化了操作
- 菜单界面要占用一定的屏幕空间和显示时间

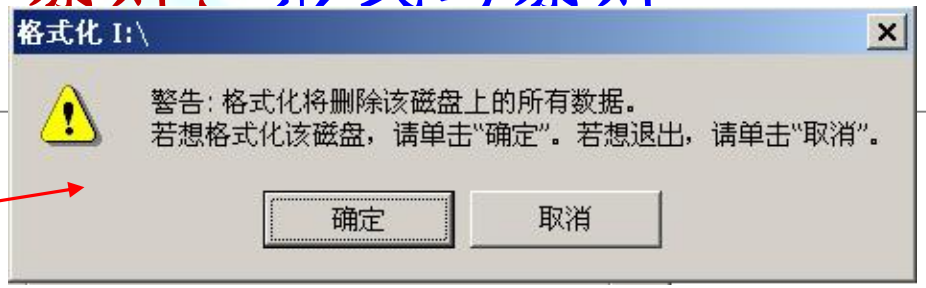
# 用户界面设计

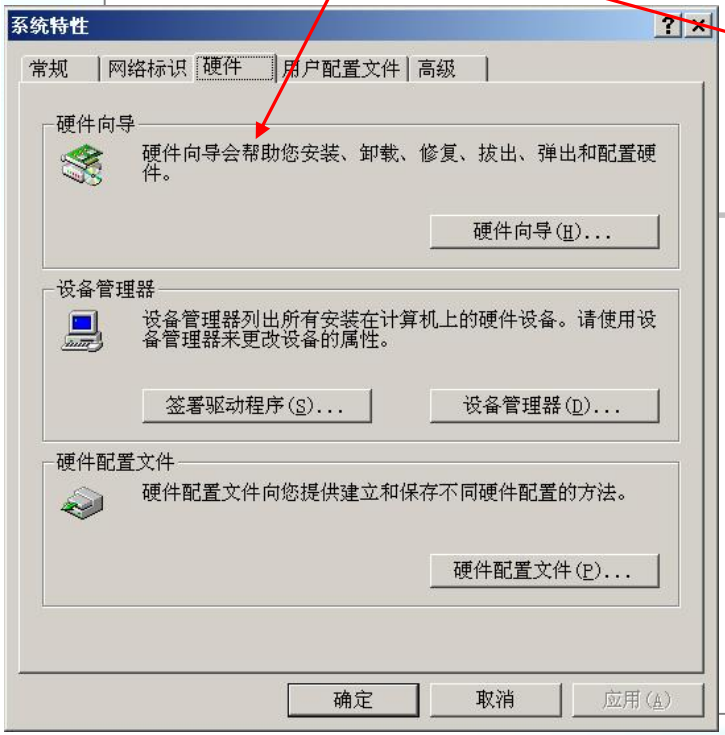
## 用户界面形式及设计原则：形式与原则

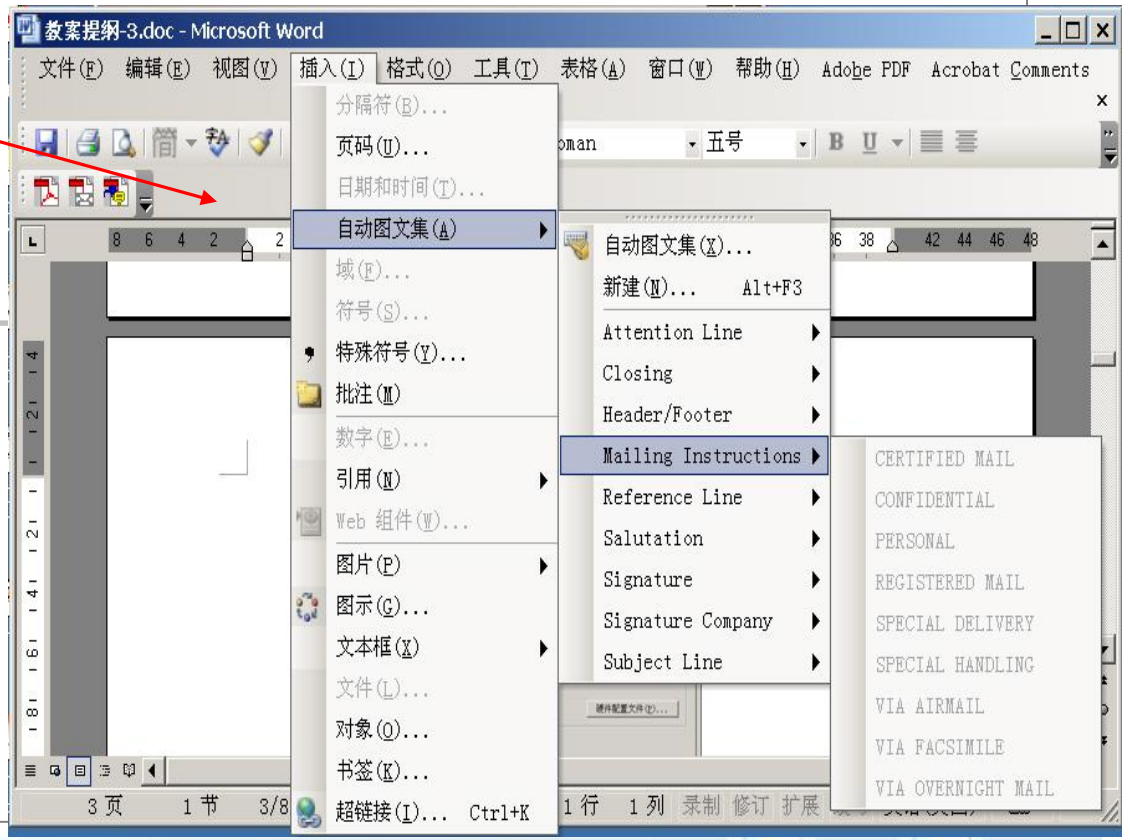
### 菜单界面设计

**2. 菜单形式：**

- 单一菜单
- 线状序列菜单
- 树状结构菜单



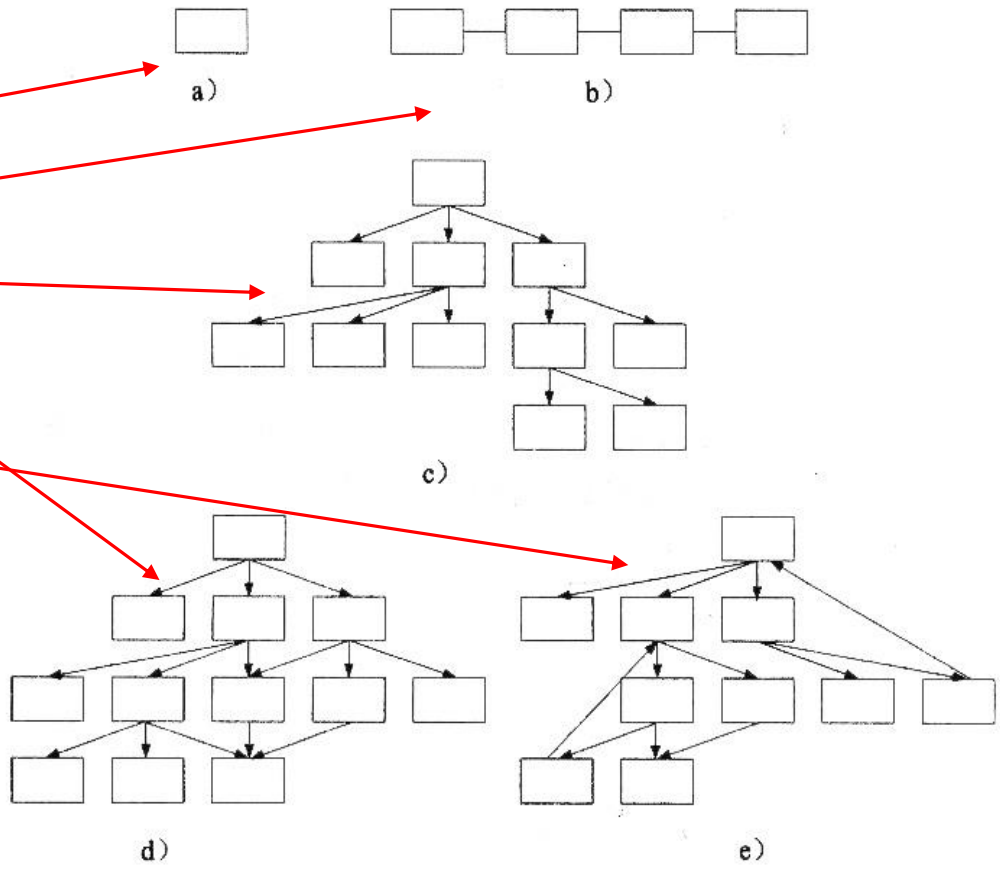
# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 菜单界面设计

2. 菜单形式：

- 单一菜单
- 线状序列菜单
- 树状结构菜单
- 非循环网络菜单
- 循环网络菜单




菜单的多种语义组织

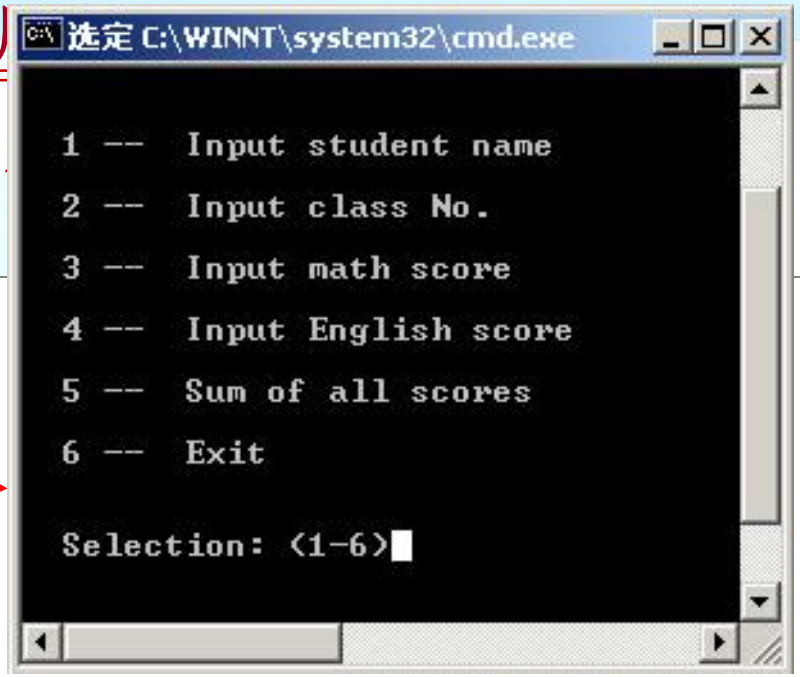
a）单一菜单　b）线状序列　c）树状结构　d）非循环网络　e）循环网络

# 用户界面形式及设计原

## 菜单界面设计

### 3. 菜单界面的式样

- 全屏幕文本菜单
- 条形菜单
  （屏幕顶部和底部）
- 弹出式菜单
- 工具栏
- 下拉式菜单
- 图标菜单
- 滚动菜单
- 浮动菜单



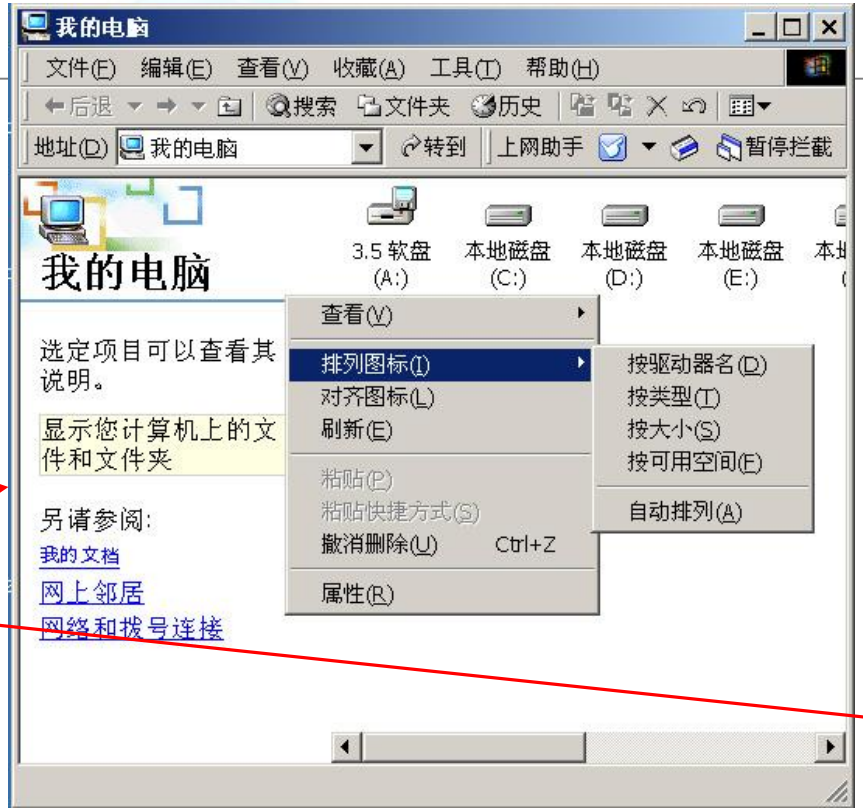
# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 菜单界面设计

**3. 菜单界面的式样**

- 全屏幕文本菜单
- 条形菜单
  （屏幕顶部和底部）
- 弹出式菜单
- 工具栏
- 下拉式菜单
- 图标菜单
- 滚动菜单
- 浮动菜单



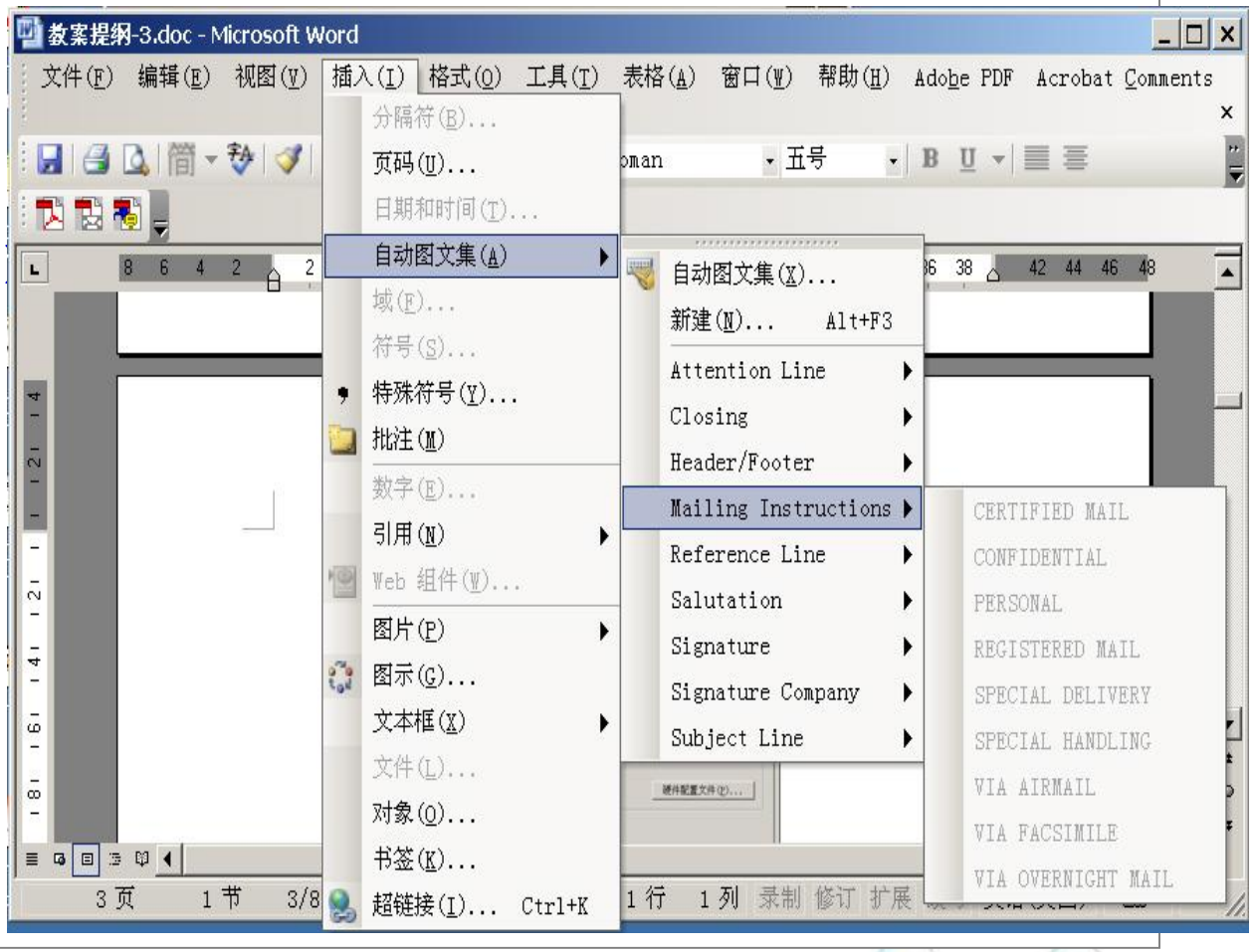
# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 菜单界面设计

**3. 菜单界面的式样**

- 全屏幕文本菜单
- 条形菜单
  （屏幕顶部和底部
- 弹出式菜单
- 工具栏
- 下拉式菜单
- 图标菜单
- 滚动菜单
- 浮动菜单



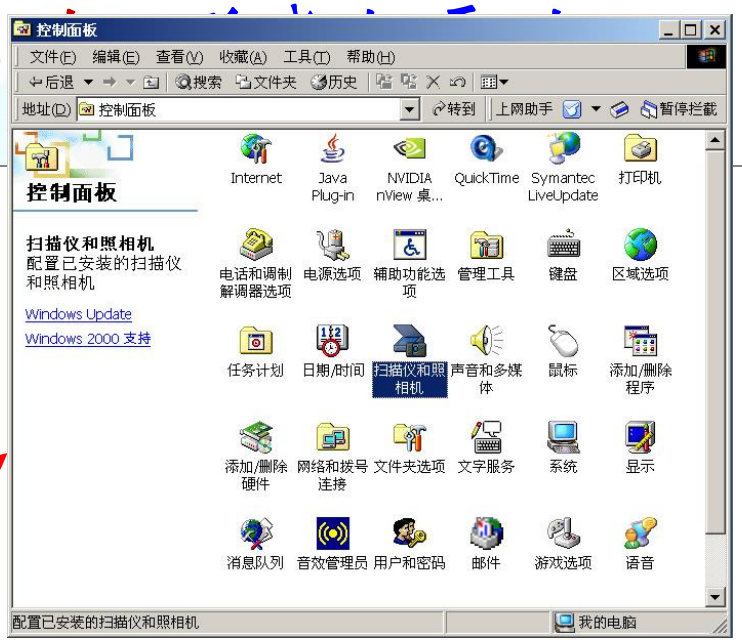
# 用户界面设计

## 用户界面形式及设计原

### 菜单界面设计

**3. 菜单界面的式样**

- 全屏幕文本菜单
- 条形菜单
  （屏幕顶部和底部）
- 弹出式菜单
- 工具栏
- 下拉式菜单
- 图标菜单
- 滚动菜单
- 浮动菜单



# 用户界面设计

## 用户界面形式及设计原则：形式与原

### 菜单界面设计

**4.** 菜单界面的设计原则

- 菜单合理分类：系统功能、逻辑顺序等
- 菜单标题及菜单项命名一致、简明、有意义
- 合理组织菜单界面的结构与层次

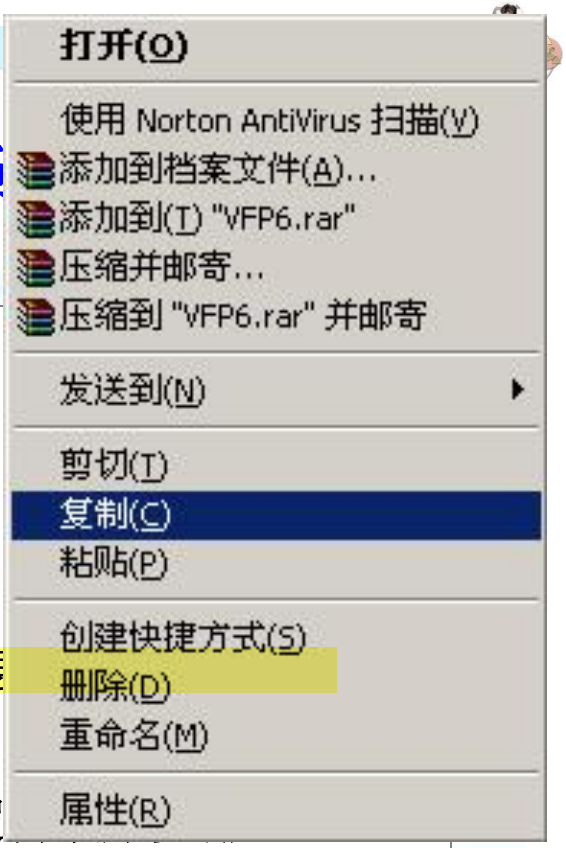
  广而浅优于窄而深的菜单树、最佳项数4-8、最深层
- 菜单项的安排应有利于提高菜单选取速度

  可以依据使用频度、数字顺序、字母顺序、功能逻

**注意：**隔离使用频度高但相邻相似项致使误操作的菜单项，如**Copy与Cut**

- 保持各级菜单显示格式和操作方式的一致性
- 为菜单项提供多于一种的选择途径，以及为菜单选择提供捷径

  键盘、鼠标、快捷方式等
- 对菜单选择和点取设定反馈标记
- 设计良好的联机帮助



# 用户界面设计

## 用户界面形式及设计原则：形式与原则

1. 命令语言界面设计
2. 菜单界面设计
3. 数据输入界面设计
4. 直接操纵和WIMP界面设计
5. 提示、帮助和出错界面设计

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 数据输入界面设计

从输入作用上说，可以分为控制输入和数据输入两类：

控制输入：完成系统运行的控制功能，如执行命令，菜单选择等；

数据输入：则是提供计算机系统运行时所需的数据。

**1. 数据输入方式**

- 问答式对话数据输入
- 菜单选择输入 例如：[1-优秀 2-良好 3-中等]；[1-男，2-女]
- 填表输入

  对要处理大量数据的数据库系统来说，是最合适的数据输入方法
- 直接操纵输入界面

  例如：软键盘，单选框、复选框，各种点击对话框等
- 关键词数据输入：关键词＋参数（类似命令行）

  例如：CAD系统中使用rect、line、circ等输入矩形、直线、圆形等
- 条形码、光学字符识别(OCR)
- 声音数据输入

# 用户界面形式及设计原则：形式与原则

## 数据输入界面设计

### 2. 交互式数据输入界面设计原则

（1）数据输入的一致性

在所有条件下应该使用相同的动作序列，相同的分隔符和缩写符等

（2）使用户输入减至最少

当同样的信息在两个地方都需要时，系统应该复制该信息；

如果有些输入数据项有缺省值，则给出缺省值供用户抉择

（3）为用户提供信息反馈

在需要用户输入时应该向用户发出提示，可以使用闪烁的光标、“？”或“Enter data”等作为提示符

（4）用户输入的灵活性

用户可以集中地一次性输入所有数据，也可以分批输入数据，也可以随时修改错误的输入

（5）提供错误监测和修改机构

尽可能报告错误，也要提供可以修改错误的输入渠道

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 数据输入界面设计

**3. 填表输入数据方式**

- 同菜单界面一样，填表界面是系统驱动的；
- 填表输入界面充分有效地利用了屏幕空间，所输入字段空间同时显示在屏幕上，以让用户同时进行多字段的输入；
- 在填表输入方式中，可以充分利用上下文信息，帮助用户完成输入。

**界面要素：**

界面标题
输入框名称
输入提示信息
输入编辑框
可选择输入控件
（单选框、多选框、组合框、下拉列表框）
控制按钮
（确认、取消、插入、删除、重置等）

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 数据输入界面设计

**4. 填表数据输入界面设计原则**

- 一致性：保证前后用词，语法一致
- 有含义的表格标题
- 使用易于理解的指导性说明文字（一般在编辑框右侧）
- 栏目按逻辑分组排序

  表格结构和用户任务相一致，相关的输入字段按使用频率、重要性、功能关系或使用顺序进行安排和分组
- 光标移动方便

  需要一种简单直观的机制来移动光标，Tab键或箭头键等
- 出错提示

  提示数据的允许范围和输入方法，对不可接受的值都给出出错信息
- 提供帮助

  应该在相应处提供帮助功能，以便不熟悉的用户也可以操作
- 表格显示应美观、清楚，避免过分拥挤

填表界面不当的例子

光标切换输入框

输入错误提示

提供输入帮助

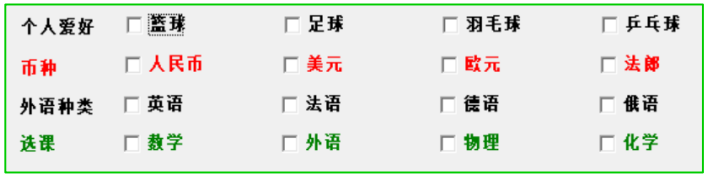
# 用户界面形式及设计原则：形式与原则

## 数据输入界面设计

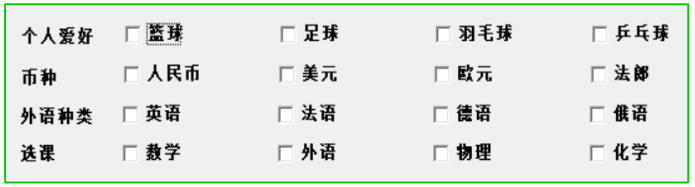
5. 集中数据项布局及显控协调问题

集中数据项布局原则：

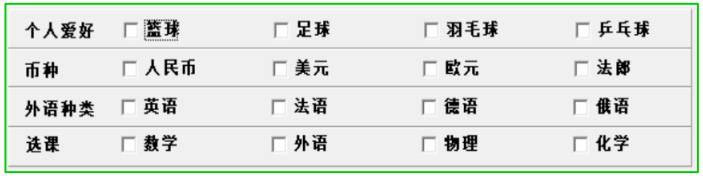
- 接近性原则
- 相似性原则
- 完整性和闭合性原则




（c）相似性布局例子




（a）随意布局例子




（d）闭合性布局例子

（b）接近性布局例子

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 数据输入界面设计

5. 集中数据项布局及显控协调问题

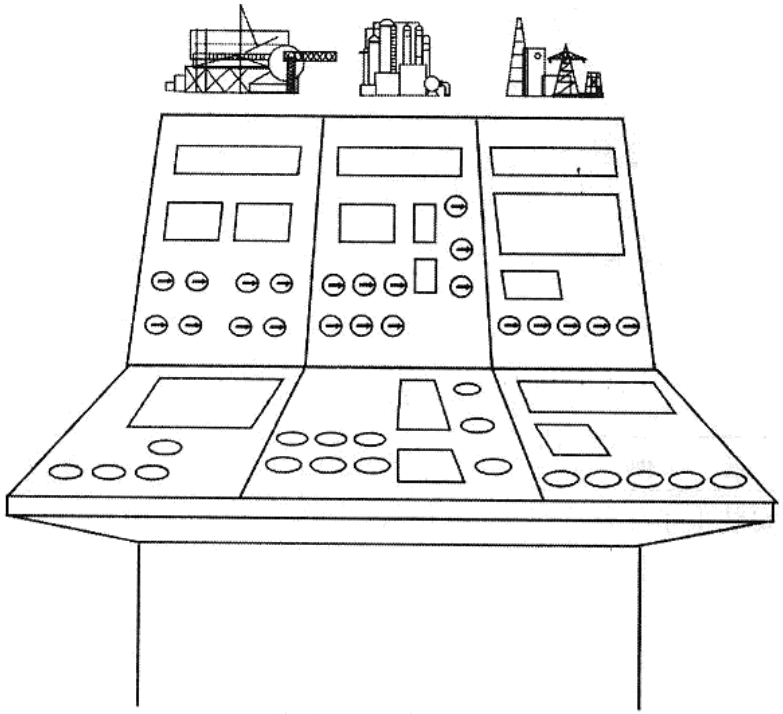
显控协调布局原则：

- 重要性原则
- 操作频率原则
- 功能分组原则
- 操作次序原则

分组凌乱布局例子

分组规范布局例子

显控布局优劣对比例子



控制台的逻辑布局

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

1. 命令语言界面设计
2. 菜单界面设计
3. 数据输入界面设计
4. 直接操纵和WIMP界面设计
5. 提示、帮助和出错界面设计

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 直接操纵和WIMP界面设计

20世纪80年代以来，以直接操纵，WIMP界面和图形用户界面(GUI)，WYSIWYG原理等为特征的技术广泛为许多计算机系统所采用

直接操纵 -- Direct Manipulation

所见即所得 -- WYSIWYG(What you see is what you get.）

WIMP -- Windows,Icons,Menus,Pointers

窗口，图标，菜单，定位器

-- Windows,Icons,Mouse,Pull-down menu

窗口，图标，鼠标器，下拉式菜单

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 直接操纵和WIMP界面设计

**1. 直接操纵界面的特点**

“直接操纵”界面专门用来代表以图形方式工作的窗口、图标、菜单、按钮、滚动条、对话框等界面部件，也包括定位器图标。其主要特点：

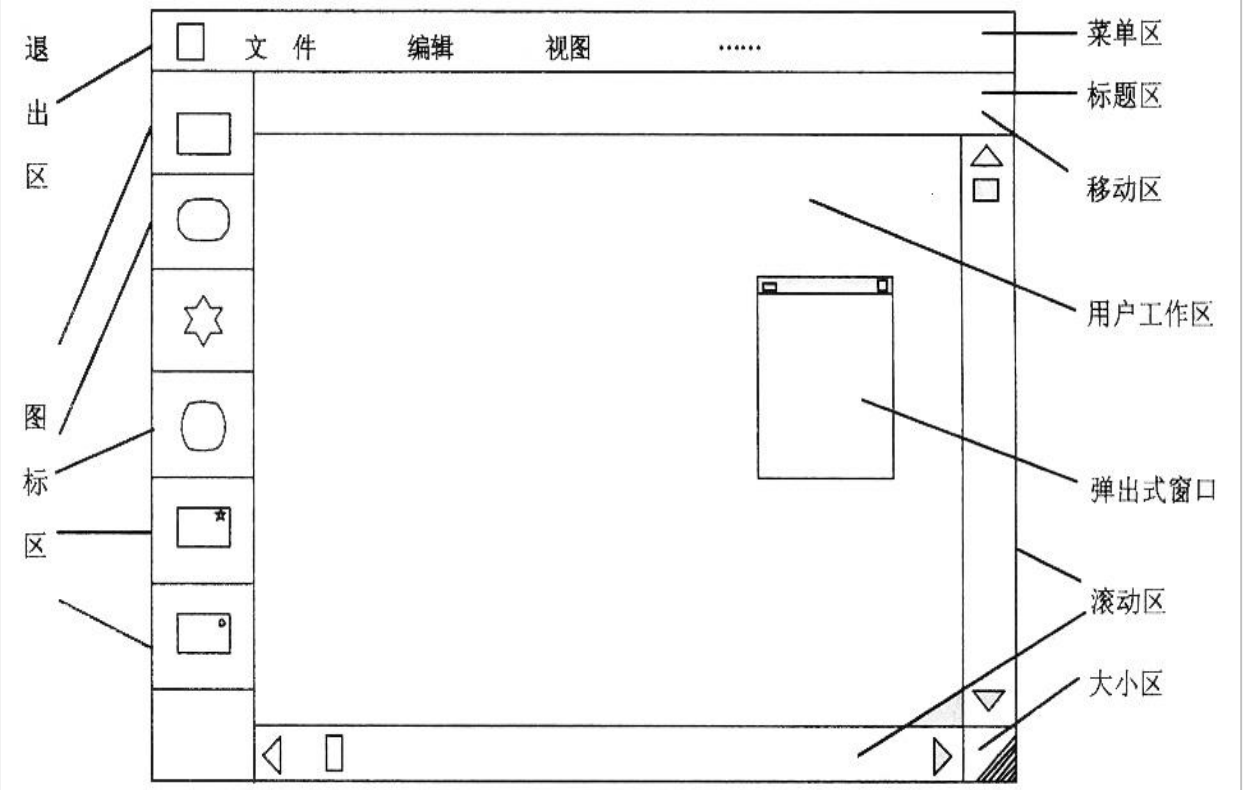
- 以物理动作或标记按钮代替复杂的语法
- 用指点和选择代替键盘输入
- 操作结果立刻可见，具有高度的交互性
- 提供了对用户出错的保护机制和强有力的帮助机制，支持逆向操作
- 新手通常仅需要经过观看系统运行的演示，就能很快地学会系统的基本功能。

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 直接操纵和WIMP界面设计

**2. 窗口的构成**

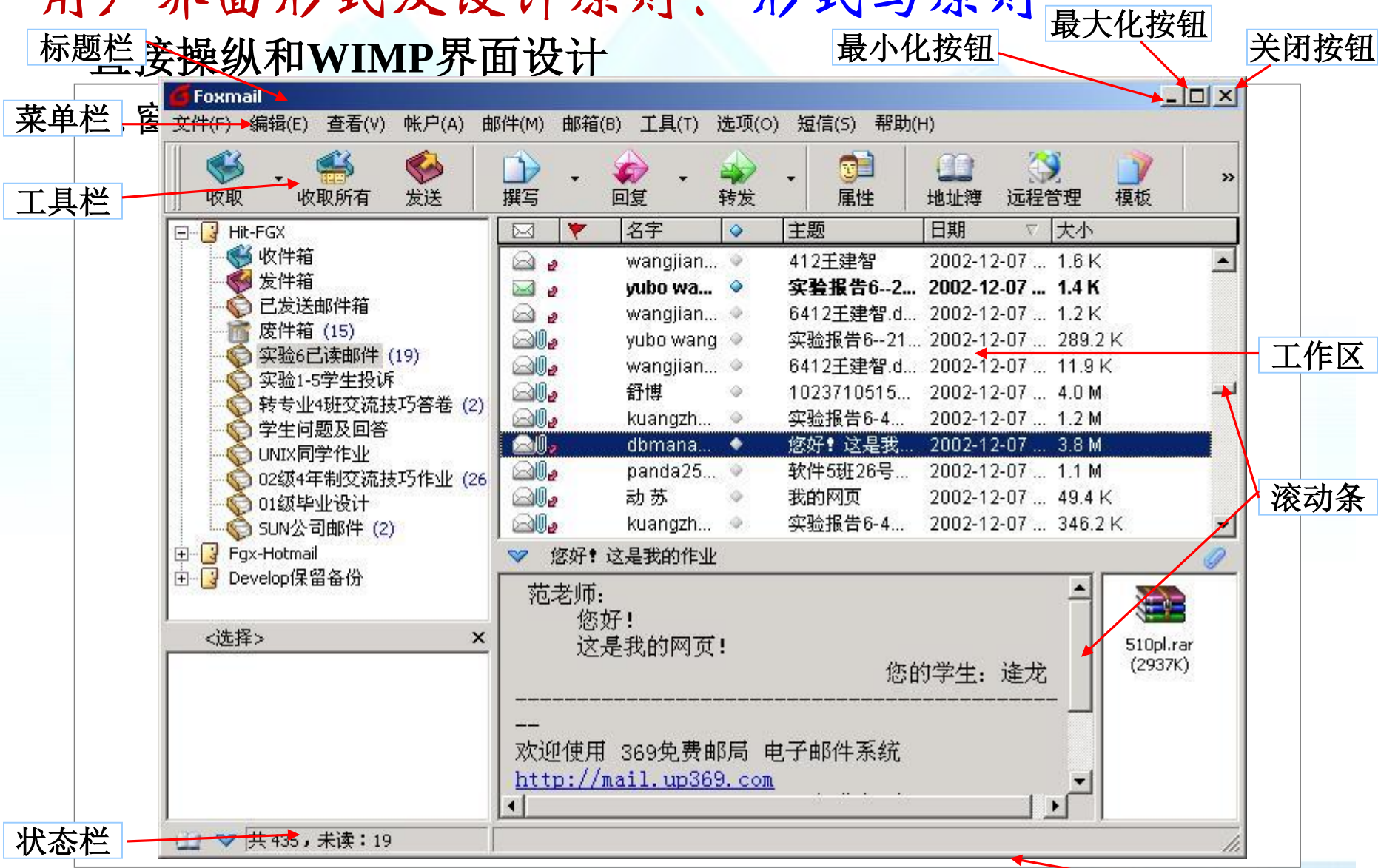



一个实用窗口的配置部件示意图

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

- 直接操纵和WIMP界面设计



# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 直接操纵和WIMP界面设计

**2. 窗口的分类**

按构造方式可以把窗口区分为以下几类：

例 （1）滚动式窗口 最简单的窗口，通过上下翻滚可以看到全部信息

例 （2）开关式窗口 每个时刻仅能显示其中的一个，系统可以通过开关选择当前要显示的窗口

（3）分列式窗口 分列式窗口允许对屏幕按水平进行分区

例 （4）瓦片式窗口 在水平方向或垂直方向规则地把屏幕划分成不重叠的子屏幕，每个子屏幕对应一个窗口，瓦片式窗口不会被其他的窗口遮盖

（5）重叠式窗口 重叠式窗口可以独立地改变窗口大小，位置，并允许叠放在其他窗口之上

例 （6）弹出式窗口 系统运行时临时生成的，一般用来显示提示性信息

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 直接操纵和WIMP界面设计

2. 窗口的分类

按构造方式

例 （1）滚动式

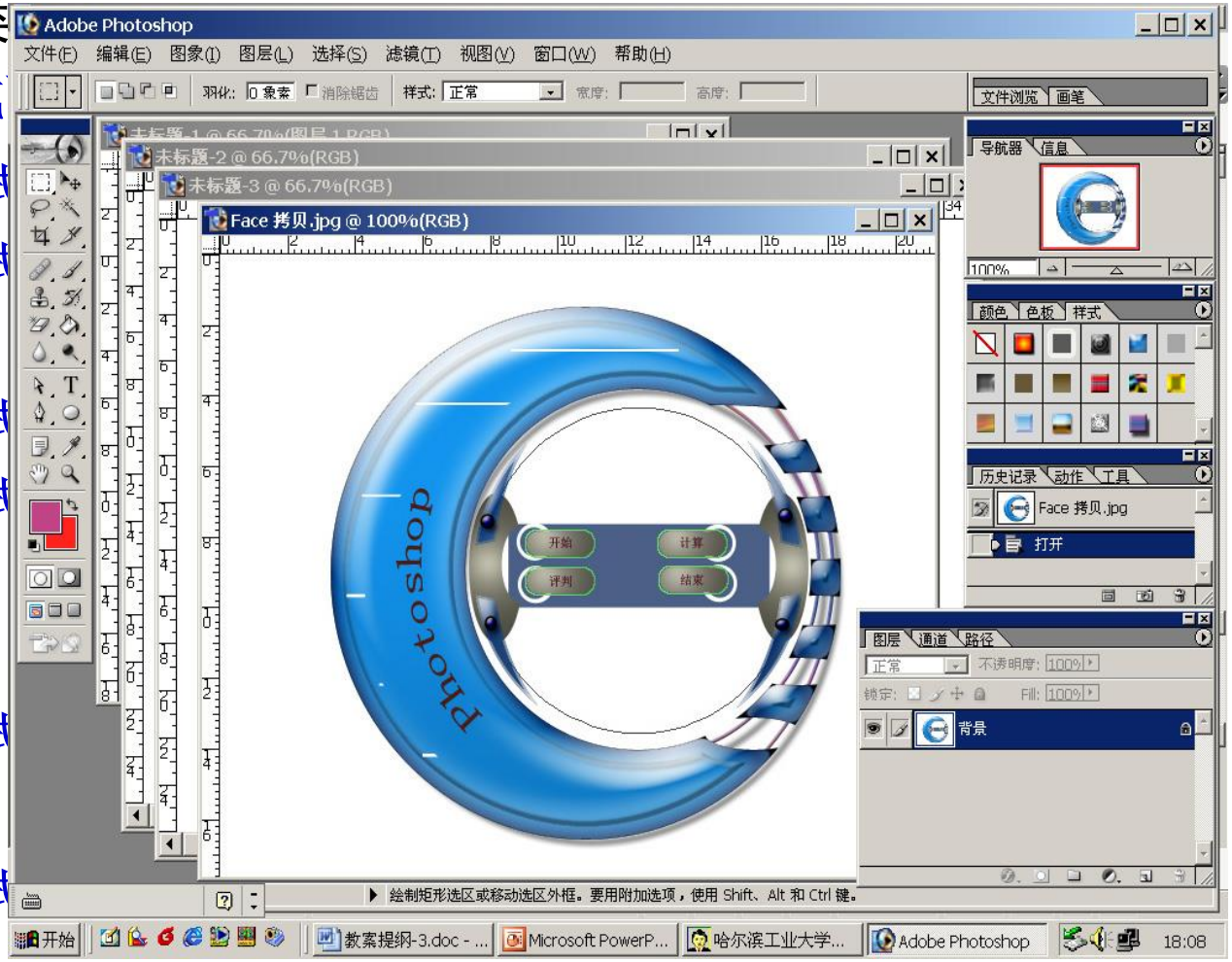
例 （2）开关式

（3）分列式

例 （4）瓦片式

（5）重叠式

例 （6）弹出式



# 用户界面设计

## 用户界面形式及设计原则：形式与原则

1. 命令语言界面设计
2. 菜单界面设计
3. 数据输入界面设计
4. 直接操纵和WIMP界面设计
5. 提示、帮助和出错界面设计

# 用户界面形式及设计原则：形式与原则

## 帮助和出错界面设计

在一个交互系统中，由于用户的原因或系统自身的原因，经常会发生一些错误操作。一个好的交互系统不可能要求用户不犯错误，但应该具有较强的处理各种错误的能力

- 出错处理分析
- 出错处理的设计
- 帮助处理系统的设计

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 帮助和出错界面设计

**1.** 出错处理分析

人-计算机系统错误的原因和类型：

（1）输入错误 -- 用户原因导致的错误

（2）计算错误 -- 程序Bug导致的错误

---

输入错误的根源和类型

（1）感知的误导导致知觉错

由于用户感觉器官不正确的或不充分的知觉暗示会导致知觉错

例如B和8，Z和2, l和1，O和0等发生混淆

（2）人的记忆特性导致知觉错

短时记忆和工作记忆中的信息极易丢失；

人机界面设计的不一致性也会产生认知错误

（3）响应时间过快或过慢

系统的响应速度与用户工作的节奏不适应，促使出错率增加

# 用户界面形式及设计原则：形式与原则

## 帮助和出错界面设计

**2.** 出错处理的设计原则

（1）错误预防原则

- 尽量避免相似的命令名，动作序列等，以免用户产生混淆
- 建立一致性的原则和模式，有利于减少学习和错误
- 提供上下文和状态信息
- 减少用户的记忆负担
- 降低对人从事活动的技能要求

  例如：在计算机操作中，减少Shift、Ctrl等复合键的使用
- 使用大屏幕和清楚可视的反馈
- 减少键盘输入

（2）错误恢复原则

- 提供恢复(undo)功能
- 在程序运行中提供撤销(Cancel)功能
- 对重要的有破坏性的命令提供确认措施，以避免破坏性操作

UnDo&ReDo例

# 用户界面设计

## 用户界面形式及设计原则：形式与原则

### 帮助和出错界面设计

**3. 帮助处理系统的设计原则**

（1）帮助系统的类型

- 联机用户手册
- 联机培训　　＊均为在线形式
- 联机演示

（2）帮助处理设计的基本原则

- 完整性　　– 所有操作都要有帮助说明
- 一致性　　– 帮助信息的说法或术语前后一定要一致
- 上下文相关 – See also， 相关内容
- 可理解性　– 不能产生循环说明
- 可维护性　-- 必要时可留有维护接口
- 方便性　　-- 接到帮助请求时，迅速给出准确信息

  如：Delphi中按F1时会给出光标所在词的相关帮助

# 用户界面设计

## 本章主要内容

1. 用户界面设计的基本概念
2. 用户界面形式及设计原则
3. WIMP用户界面设计
4. *网页界面设计
5. 缺省设计问题
6. 输入验证设计问题
7. 系统响应及信息反馈问题

## WIMP用户界面设计

1. 窗口设计
2. 菜单设计
3. 填表输入界面设计
4. 操作控制元素设计
5. 声音输出界面设计
6. 图标设计
7. 个性化界面设计
8. UnDo/ReDo设计

# WIMP用户界面设计

## 窗口设计

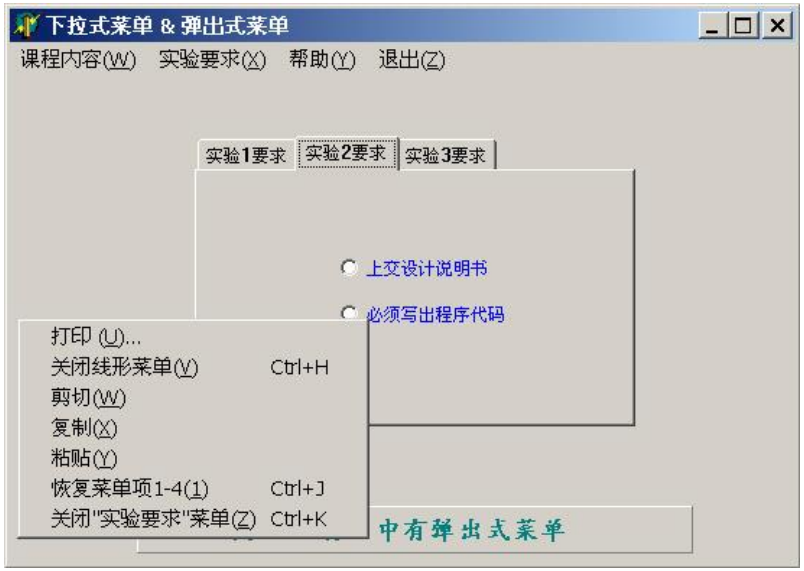
1. 标题、图标
2. 可缩放
3. 可最大化、最小化
4. 可移动
5. 有滚动条
6. 边框风格
7. 光标风格
8. 背景图案

底层编写窗口例

# WIMP用户界面设计

## 菜单设计

1. 下拉式菜单
2. 弹出式菜单
3. 线形菜单
4. 快捷方式
5. 动态菜单项
   - （1）灰显菜单项
   - （2）隐藏菜单项
   - （3）位置可变菜单项
   - （4）内容可变菜单项
6. 菜单项图标

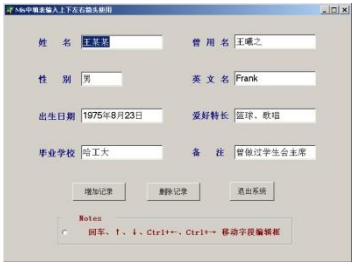


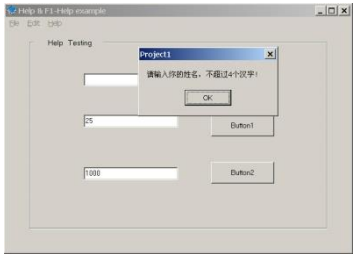
# 用户界面设计

## WIMP用户界面设计

### 填表输入界面设计

1. 提示：
   - 标签、提示信息
2. 输入与编辑：
   - 编辑框、缺省值、信息验证；
   - 字体、字型、颜色；
   - 编辑框确认及跳转
   - 密码编辑; 按钮
3. 选择：
   - 单选框（RadioButton）；
   - 多选框（CheckBox）；
   - 组合框（ComboBox）；
   - 列表框（ListBox）
4. 帮助功能：
   - 全局完整帮助（多见于菜单中）；
   - F1局部帮助
   - Hint（隐式）帮助

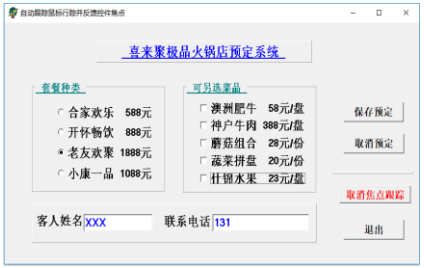


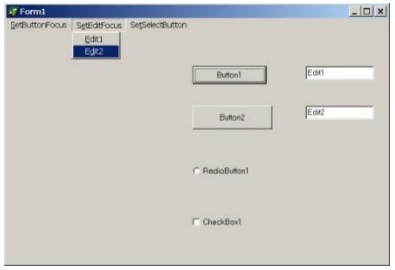


# WIMP用户界面设计

## 操作控制元素设计

1. 按钮：
   - 普通按钮：Button控件
   - 位图按钮：BitBtn控件
2. 控制焦点：
   - 用方法SetFocus
3. 滚动条
4. 进度条



# WIMP用户界面设计

## 声音输出界面设计

同步&异步例子

调用Delphi一个音频播放函数

**PlaySound(pchar(filename),0,snd_async);//异步**

**PlaySound(pchar(filename),0,snd_synch);//同步**

# WIMP用户界面设计

## 图标设计

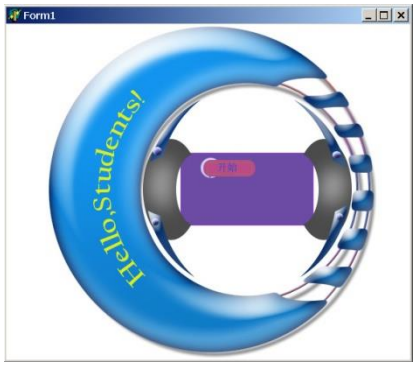
图标典型问题例子

1. 用已经设计好的*.ICO文件
2. 读入已有的图片(.bmp, .jpg等)进行再编辑或修改
3. 抓取屏幕局部图像再编辑或修改
4. 纯像素画设计

## WIMP用户界面设计

### 个性化界面设计

1. 使用2D或3D图形图像设计工具制作背景图片
2. 将图中可活动的局部图像剪切为单个的.bmp格式图片文件
3. 用Image控件装载背景图片，用BitBtn控件装载局部bmp图片



# WIMP用户界面设计

## 个性化界面设计

界面换肤设计与实现

# WIMP用户界面设计

## UnDo/ReDo设计

1. 分析程序运行及操作步骤，分离出可独立的操作对象；
2. 建立一个数据结构，保护每个对象产生前的现场：操作序列和结果数据；
   该数据结构是个环形栈(栈底不固定)；
3. UnDo时做类退栈操作，恢复现场；
4. ReDo时，将由于UnDo退出的栈元素再压入栈(只有当UnDo后无新的操作产生才可做)

# 用户界面设计

## 本章主要内容

1. 用户界面设计的基本概念
2. 用户界面形式及设计原则
3. WIMP用户界面设计
4. *网页界面设计
5. 缺省设计问题
6. 输入验证设计问题
7. 系统响应及信息反馈问题

# 用户界面设计

## 本章主要内容

1. 用户界面设计的基本概念
2. 用户界面形式及设计原则
3. WIMP用户界面设计
4. *网页界面设计
5. 缺省设计问题
6. 输入验证设计问题
7. 系统响应及信息反馈问题

## 缺省设计问题

### 缺省或缺省值设计的目的和原则

缺省或缺省值可能用到的场合：

- 文本编辑框
- 输入选择框：列表框、组合框、单选框、多选框
- 功能按钮：其上的焦点

缺省或缺省值设计的目的：

- 减少用户输入数据量，提高系统使用效率

缺省或缺省值采用原则：

- 缺省值的设定一定要考虑使用的效率，即从统计结果看，每次输入平均输入量（含清除缺省值的输入量）＜无缺省值情况下的平均输入量。

# 用户界面设计

## 缺省设计问题

### 缺省或缺省值设计方法

**方法1：**用户操作设定缺省值（也可以作为强制值输入）

例如：某学院、某专业、某年级、某班的学生成绩录入，可以将学院、专业、年级、班级设定，录入界面不出现这些信息项

**方法2：**经验值作为缺省值

例如：打印参数设置对话中，纸张尺寸为“A4复印纸”/打印色彩为“黑色”/打印份数为“1份”/多份打印时“逐份输出”

**方法3：**统计方法确定出现几率大的值作为缺省值

例如：之前若干个输入的离散数据中，某个数据出现的次数超过一定比例，则可以作为下一个输入的缺省值

例如：学历输入，经统计在“专科/本科/研究生”三选一中“本科”超过50%以上

# 用户界面设计

## 缺省设计问题

### 缺省或缺省值设计方法

**方法4：**安全需要设置缺省值

例如：参与运算的输入编辑框不能为空，若用户不输入则必须填一个值0；如拦截提示中的“取消”按钮

**方法5：**最近使用值作为缺省值

例如：在Foxmail中，附件文件存储目录缺省为当前最近一次使用过的位置；如档案数据输入时，前一个记录的部门可以作为后一个记录的缺省值

**方法6：**输入数据的上下文关联缺省值

例如：上文“职业”填写“军人/护士”，则下文“性别”可缺省填入“男/女”；上文填写姓名时的“姓”可以作为下文中父亲姓名中的“姓”缺省值；“职称”为“教授/副教授”时，导师资格可用“博导/硕导”作为缺省值

# 缺省设计问题

## 缺省或缺省值设计方法

**方法7：**当前状态值可作为缺省值

例如：当前日期/当前时间/当前价格

**方法8：**逻辑强制关系互为缺省值

例如：“出生年份”与“当前年龄”

**方法9：**正常操作序列选项作为缺省值

例如：编辑框按空间布局顺序跳转；
软件安装过程的“下一步”

**方法10：**树状隶属关系约束选项列表

例如：国家、城市、街区；大学、学院、系、专业、博士点

例子

**方法11：**智能焦点

例如：Windows自动移动鼠标Pointer到当前缺省焦点上

# 用户界面设计

## 本章主要内容

1. 用户界面设计的基本概念
2. 用户界面形式及设计原则
3. WIMP用户界面设计
4. *网页界面设计
5. 缺省设计问题
6. 输入验证设计问题
7. 系统响应及信息反馈问题

# 输入验证设计问题

## 输入验证设计的目的和意义

输入验证设计可能用到的场合：

- 文本编辑框
- 功能菜单或按钮

输入验证设计设计的目的：

- 在可能的情况下，保证输入数据的有效性、合理性，减少错误，提高效率、保证安全

# 用户界面设计

## 输入验证设计问题

### 输入验证的内容及方法

**方法1：数据位数验证**

例如：银行卡号码、证件编号、电话号码、车牌号等已知合理位数的情况

**方法2：字符集验证**

例如：整数、小数、正数、日期等已知字符集合的情况；
该情况一般用在单个字符的实时拒绝输入；如在手机中输入电话簿的号码时，只接受‘0’～‘9’的字符

**方法3：范围验证**

例如：月份、某月的日期、取款金额、贷款年限等已知范围 的情况；性别、学历、学位、级别、命令等已知集合的情况

# 用户界面设计

## 输入验证设计问题

### 输入验证的内容及方法

**方法4：合理性验证**

例如：人的年龄、亲生子女的数量等有合理边界范围的情况

**方法5：数据格式验证**

例如：掩码输入，如9999-99-99；99.99

**方法6：安全性验证**

例如：重复输入密码、密码内容复杂性、密码输错次数限制；
防止解密的图形验证码输入验证；
危险功能选定后的拦截性提示验证

**方法7：异常性验证**

例如：文件存在否、数据库记录空否、存储空间满否、除数为0否等会引起处理异常的情况

# 用户界面设计

## 输入验证设计问题

### 操作错误处理的机制及方法

**方法1：选择性确认**

例如：拦截性提示

**方法2：操作中断允许**

例如：在任务执行过程中有“取消”选项

**方法3：回滚**

例如：在任务被中断的情况下回退到操作发生之前的状态

**方法4：Undo/Redo**

例如：随时可以回退到操作发生之前的状态或再取消Undo

# 用户界面设计

## 本章主要内容

1. 用户界面设计的基本概念
2. 用户界面形式及设计原则
3. WIMP用户界面设计
4. *网页界面设计
5. 缺省设计问题
6. 输入验证设计问题
7. 系统响应及信息反馈问题

# 系统响应及信息反馈问题

## 系统响应及信息反馈的目标

系统响应的目标：

- 及时响应
- 适时响应（不能过快/过慢）

信息反馈的目标：

- 反馈信息具体、准确
- 人性化反馈

# 用户界面设计

## 系统响应及信息反馈问题

### 系统响应的内容及方法

**方法1：** 立即确认用户的输入（在0.1秒内）

注意：操作的确认 vs 请求任务的执行，它们有本质的区别

例如：鼠标点击按钮，按钮需要通过一定的变化来响应；
鼠标移动到某控件上时需得到一定的标示来响应；
鼠标或键盘指点某个菜单项需得到反白等方式反馈

**方法2：** 提供忙状态指示（对处理时长在1-10秒数量级）

例如：用最简单的静态等待光标，如常见的沙漏图标；
用等待动画，如动态沙漏、带指针的手表

**方法3：** 任务进度指示器（处理时间在数10秒以上）

例如：电梯移动当前楼层的数字指示；
微波炉、洗衣机的定时器；
数据记录处理个数占应处理个数的百分比

# 系统响应及信息反馈问题

## 系统响应的内容及方法

注意：进度指示最好是精确的，但常用估计并逐步逼近准确值的方法

进度条中“进度”与“任务”完成量之间的匹配关系：

（1）虚拟进度（只起到“忙”反馈的作用）

（2）非精确进度（近似估计）

（3）动态调整的显示进度（进/退）

（4）进度显示的任务粒度（即最小进度单位表示的任务量）：

- 功能模块为基本粒度
- 定量任务为基本粒度

（5）时间方式显示进度（即剩余时间提示进度）

例子

（6）百分比形式显示进度，或时间+百分比形式

（7）工作任务动态显示：

如正处理某文件，正在计算第X个单元 …

（8）趣味进度：如沙漏形式/广告倒计时/颜色由深至浅

# 用户界面设计

## 系统响应及信息反馈问题

### 系统响应的内容及方法

**方法4：** 首先反馈重要信息（即先“答应”后“动作”）

例子：（1）有人敲门，先答应再过去开门；

（2）网页浏览时，可以先下载主要内容，让浏览者边看边等；下载大图片时，可以先部分或轮廓下载；

（3）界面窗口或控件放大操作时，可以只动态显示轮廓，当用户确认后，再做最后处理

比较：Word与PDF滚动翻页处理的不同

**方法5：** 处理任务分解、化整为零

在人机交互过程中，系统经常有等待人“机械”动作的空闲时间，该时间可以将大任务的部分工作量提前完成；从而提高大任务提交时的系统响应时间

例子:（1）数据记录的统计汇总，中间计算过程可以放到记录数据输入时即时完成；

（2）平时积累报销票据，分门别类存放或若干张粘贴一组，并统计张数和合计金额

# 用户界面设计

## 系统响应及信息反馈问题

### 信息反馈的内容及方法

**方法1：** 反馈信息明确

例如：

| 差的表达方式 | 好的表达方式 |
| --- | --- |
| ● 语法错误 | ● 左半括号不匹配 |
| ● 非法输入 | ● 输入Send 、Read、Drop的首字母 |
| ● 无效数据 | ● 日期数值应在1到31之间 |
| ● 不合理的文件名称 | ● 文件名称必须以字母开始 |

**方法2：** 建设性指导原则与积极的基调

例如：提供的信息不是要谴责用户已经做错的事，而是要尽可能地指导用户改正错误

差：字符串严重溢出，任务已终止

好：字符串空间已满，请修改程序使用较短的字符串或者使用扩展的字符串空间

差：标识符未定义

好：在使用标识符之前请先声明

差：非法语句，警告

好：在函数子程序中不能使用返回语句

# 系统响应及信息反馈问题

## 信息反馈的内容及方法

**方法3：**以用户为中心的措辞

例如：

**差：Enter Data含有命令口吻的表达方式在提示信息中应该避免**

**好：READY FOR COMMAND” 或仅为 ” READY” 这样的用户控制语句。**

---

**差：非法电话号码，呼叫失败，错误号为583-2R6.9。认真查阅用户手册。[语气粗鲁的信息]**

**好：对不起，暂时无法为您接通。请核对号码或者参考帮助手册后再拨。[他们承担了过失，并且提供了建设性建议]**

## 系统响应及信息反馈问题

### 信息反馈的内容及方法

**方法4：**用非拟人化的设计

调查显示：拟人化的交互信息不能让人感到亲切，反而让人感到虚伪，并不能改善交互效果

例如：

第一人称形式：嗨，我就是计算机，我将向你提出几个问题。

第二人称形式：你将回答几个问题，你应该……

中性称谓形式：这是一个多选练习。